

Agile

Metodyki zwinne w planowaniu projektów



Mike Cohn

Tytuł oryginału: Agile Estimating and Planning

Tłumaczenie: Radosław Meryk

ISBN: 978-83-283-4487-7

Authorized translation from the English language edition, entitled:
AGILE ESTIMATING AND PLANNING; ISBN 0131479415; by Mike Cohn;
published by Pearson Education, Inc., publishing as Prentice Hall PTR.
Copyright © 2006 by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc. Polish language edition published by HELION S.A. Copyright © 2018.

Wszelkie prawa zastrzeżone. Nieautoryzowane rozpowszechnianie całości lub fragmentu niniejszej publikacji w jakiejkolwiek postaci jest zabronione. Wykonywanie kopii metodą kserograficzną, fotograficzną, a także kopiowanie książki na nośniku filmowym, magnetycznym lub innym powoduje naruszenie praw autorskich niniejszej publikacji.

Wszystkie znaki występujące w tekście są zastrzeżonymi znakami firmowymi bądź towarowymi ich właścicieli.

Autor oraz Wydawnictwo HELION dołożyli wszelkich starań, by zawarte w tej książce informacje były kompletne i rzetelne. Nie biorą jednak żadnej odpowiedzialności ani za ich wykorzystanie, ani za związane z tym ewentualne naruszenie praw patentowych lub autorskich. Autor oraz Wydawnictwo HELION nie ponoszą również żadnej odpowiedzialności za ewentualne szkody wynikłe z wykorzystania informacji zawartych w książce.

Wydawnictwo HELION
ul. Kościuszki 1c, 44-100 GLIWICE
tel. 32 231 22 19, 32 230 98 63
e-mail: helion@helion.pl
WWW: <http://helion.pl> (księgarnia internetowa, katalog książek)

Drogi Czytelniku!

Jeżeli chcesz ocenić tę książkę, zajrzyj pod adres

http://helion.pl/user/opinie/agilem_ebook

Możesz tam wpisać swoje uwagi, spostrzeżenia, recenzję.

- [Poleć książkę na Facebook.com](#)
- [Kup w wersji papierowej](#)
- [Oceń książkę](#)

- [Księgarnia internetowa](#)
- [Lubię to! » Nasza społeczność](#)

Opinie o książce *Agile. Metodyki zwinne w planowaniu projektów*

„Tradycyjne, deterministyczne podejścia do planowania i szacowania projektów po prostu się nie sprawdzają w zestawieniu ze śliskimi zboczami współczesnych, dynamicznych i napędzanych przez zmiany projektów. Przełomowa książka Mike’a Cohna prezentuje czytelnikom nie tylko filozofię, ale także praktyczne wskazówki i sprawdzony zestaw narzędzi, które są niezbędne do osiągnięcia sukcesu w planowaniu, ocenie i opracowywaniu harmonogramów projektów o wysokim współczynniku niepewności. Jednocześnie autor nigdy nie traci z oczu konieczności dostarczania na każdym kroku wartości biznesowej dla klienta”.

— Doug DeCarlo, autor książki *eXtreme Project Management: Using Leadership, Principles and Tools to Deliver Value in the Face of Volatility* (Jossey-Bass, 2004)

„Wiemy, jak tworzyć przewidywalne plany oraz jak nimi zarządzać. Ale tworzenie planów, które jedynie szacują przyszłość, a następnie potrafią reagować na zmiany, jest wyzwaniem dla całej naszej wiedzy i dla naszych umiejętności. W książce *Agile. Metodyki zwinne w planowaniu projektów* Mike Cohn ponownie wypełnia lukę w praktykach Agile — tym razem pokazując skuteczne podejście do zwinnego szacowania i planowania projektów. Mike zagłębia się w szczegóły tematu, przewidując wiele powstających pytań i możliwych detali. Osoby studiujące procesy Agile z pewnością przekonają się, że ta książka naprawdę mówi o zwinności, tworząc pomost między wieloma praktykami metodyki Scrum i programowania ekstremalnego”.

— Ken Schwaber, ewangelista Scrum, współtwórca organizacji Agile Alliance i jeden z sygnatariuszy Manifestu Agile

„Mike Cohn w książce *Agile. Metodyki zwinne w planowaniu projektów* po raz pierwszy scalił niemal wszystko, czego społeczność Agile nauczyła się o przedmiocie. Książka jest przejrzystą, dobrze zorganizowaną, przyjemną i wartościową lekturą. Opisuje wszystkie niezbędne szczegóły, a jednocześnie nie stwarza wielkich obciążeń dla czytelnika. Możemy zagłębić się w szczegóły na tyle, ile potrzeba, bez konieczności poznawania detali, zanim okażą się konieczne. Książka istotnie łączy w sobie wszystko, czego dowiedziano się o szacowaniu i planowaniu w projektach Agile w ciągu ostatnich dziesięcioleci. Z pewnością będzie dobrze służyć jej czytelnikom”.

— Ron Jeffries, www.XProgramming.com, autor książek *Extreme Programming Installed* (Addison-Wesley, 2001) oraz *Extreme Programming Adventures in C#* (Microsoft Press, 2004)

„Książka *Agile. Metodyki zwinne w planowaniu projektów* przedstawia obraz planowania, w którym zachowano właściwą równowagę między teorią a praktyką. Książka jest poparta konkretnym doświadczeniem, co wpływa na jej wiarygodność. Szczególnie cenię sobie sformułowanie „planowanie jest dążeniem do wartości”. Wskazuje ono na nowe, bardziej pozytywne nastawienie do planowania, odchodzące od uznawania go za »zło konieczne« — poglądu, który czasami mi się udziela”.

— Kent Beck, autor książki *Extreme Programming Explained*, wydanie drugie (Addison-Wesley, 2005)

„Planowanie »z góry« nadal jest najważniejszą częścią procesu wytwarzania oprogramowania. Zwinne wytwarzanie oprogramowania wymaga zwinnych metod planowania. Ta książka w zwięzłej, praktycznej i zrozumiałej formie pokazuje, jak skorzystać z planowania Agile”.

— Adam Rogers, Ultimate Software

„Jesteśmy gorącymi orędownikami metod Agile opisanych w tej książce. Z naszych doświadczeń wynika, że ich konsekwentne stosowanie przyniosło bardzo pozytywne efekty. Bardzo polecam tę książkę wszystkim, którzy dążą do tego, aby proces wytwarzania oprogramowania stał się bardziej praktyczny i skuteczny”.

— Mark M. Gutrich, prezes i dyrektor generalny firmy Fast 401k, Inc.

„Mike stworzył doskonałą kontynuację książki *User Stories Applied*. Zaprezentował zespołom Agile praktyczne metody i techniki pozwalające zwiększyć ich zwinność. W tej książce Mike opisuje sprawdzone i dobrze przetestowane metody skutecznej pracy z wymaganymi przez Agile wielopoziomowym planowaniem i szacowaniem. Niniejsza książka jest pierwszą próbą szczegółowego przedstawienia szacowania i planowania Agile w sposób konkurencyjny do metod i technik opisanych w moich artykułach z 1980 roku na temat planowania i szacowania metodą CPM”.

— Ryan Martens, prezes i założyciel firmy Rally Software Development Corporation

„Mike Cohn w czytelny i jasny sposób pokazuje, jak skutecznie wytwarzać oprogramowanie o wysokiej wartości biznesowej. Dzięki zwinnym metodom szacowania i planowania koncentrujemy wysiłki tam, gdzie się to naprawdę liczy, i kontynuujemy te działania również wówczas, gdy zmienia się okoliczności”.

— Rick Mugridge, Rimu Research Ltd., główny autor książki
Fit for Developing Software (Prentice Hall, 2005).

„Nareszcie! Rewolucyjna książka, której domagali się moi klienci! Książka *Agile. Metodyki zwinne w planowaniu projektów* demistyfikuje proces definiowania, sterowania i dostarczania doskonałego oprogramowania, które ma znaczenie dla biznesu. Czytelność opisu, zrozumienie zagadnienia i doświadczenie Mike’a widoczne są na każdej stronie tej książki. Czytelnik zapoznaje się z podejściem, które jest istotne i bezpośrednio przydatne dla wszystkich uczestników projektu Agile”.

— Kert D. Peterson, prezes firmy Enterprise Agile Group, LLC

„To nie jest jeszcze jedna ogólna książka poświęcona zwinnemu wytwarzaniu oprogramowania. Książka *Agile. Metodyki zwinne w planowaniu projektów* wypełnia lukę pozostawioną przez większość innych książek. Opisuje ważne, praktyczne techniki niezbędne do odniesienia sukcesu podczas realizacji zwinnych projektów wytwarzania oprogramowania”.

— Steve Tockey, główny konsultant w firmie Construx Software

„Szacowanie, planowanie i śledzenie to trójca. Jeśli nie robisz jednej z tych czynności, nie potrzebujesz pozostałych dwóch. Ta książka dostarcza bardzo praktycznej wiedzy potrzebnej do szacowania, planowania, ustalania priorytetów i śledzenia projektów. Powinna być lekturą obowiązkową dla kierowników projektów i członków ich zespołów, nawet jeśli wahają się nazywać siebie zwinnymi”.

— Niels Malotaux, Project Coach

„Skuteczne planowanie jest ważną, ale często niewłaściwie rozumianą częścią każdego udanego projektu Agile. W książce *Agile. Metodyki zwinne w planowaniu projektów* Mike Cohn zaprezentował wyczerpujący przewodnik opisujący szeroką gamę różnych zwinnych praktyk szacowania i planowania. Dzięki jasnemu i praktycznemu stylowi Mike nie tylko wyjaśnia, jak skutecznie przystąpić do planowania projektu Agile, ale także dostarcza wielu wskazówek i porad zmierzających do doskonalenia procesów planowania każdego zespołu Agile. Ta książka jest lekturą obowiązkową dla menedżerów, trenerów i członków zespołów Agile”.

— Paul Hodgetts, trener technik Agile i dyrektor generalny firmy Agile Logic

„Styl pisania Mike’a oddaje istotę zwinności — tylko tyle informacji, aby jasno naświetlić temat czytelnikowi. Niniejsza książka to doskonały przewodnik dla wszystkich osób stosujących metody Agile — zarówno doświadczonych, jak i początkujących”.

— Robert Holler, prezes i dyrektor generalny firmy VersionOne, LLC

„To tak, jakby Mike wziął całą wiedzę o planowaniu i szacowaniu doskonałego programisty Agile (którym jest) i wyłożył wszystko, co wie, w łatwo zrozumiałej formie. Co ważniejsze, w książce zaprezentował doskonale połączenie pojęć z praktycznymi przykładami oraz zakończył opisem studium przypadku tak, aby czytelnik mógł dopasować poznane informacje do swojej sytuacji. Jeśli nie jesteś jeszcze ekspertem w dziedzinie technik planowania i szacowania Agile, to ta książka jest dla Ciebie”.

— Alan Shalloway, dyrektor generalny i starszy konsultant w firmie Net Objectives oraz współautor książki *Design Patterns Explained*, wydanie drugie (Addison-Wesley, 2005)

„Chociaż przed przystąpieniem do wypróbowania zwinnych metod planowania Mike’a Cohna miałem wiele doświadczeń w dziedzinie metodologii XP, to skuteczność praktycznych i sprawdzonych technik zaprezentowanych w tej książce mnie uderzyła! Lektura tej książki potwierdza, że to ludzie, a nie narzędzia czy procesy, tworzą wysokiej jakości oprogramowanie oraz że zespoły zyskują najwięcej dzięki poznawaniu projektu i swojego produktu w trakcie codziennej pracy. Przykłady zaprezentowane w książce są konkretne, łatwe do zrozumienia i po prostu emanują „zdrowym rozsądkiem”. Lektura niniejszej książki pomoże zespołom (niezależnie od tego, czy praktykują metody Agile, czy nie) dostarczać więcej wartości, a jednocześnie czerpać przyjemność ze swojej pracy! Nieważne, czy jesteś menedżerem, programistą, testerem, czy szefem firmy, członkiem zespołu Agile, czy po prostu osobą szukającą sposobu na wykorzenienie chaosu i zapobiegnięcie kierowania się w ślepe zaułki — ta książka będzie dla Ciebie wartościowym przewodnikiem”.

— Lisa Crispin, współautorka książki *Testing Extreme Programming* (Addison-Wesley, 2003)

„Mike Cohn wykonał świetną robotę, pokazując, jak dzięki zastosowaniu podejścia Agile można rozwiązać problemy ryzyka i niepewności w celu dostarczenia bardziej sensownych ocen i planów w projektach rozwoju oprogramowania”.

— Todd Little, starszy menedżer rozwoju oprogramowania w firmie Landmark Graphics

„Mike Cohn wyjaśnia swoje podejście do planowania Agile i pokazuje, jak można zastosować koncepcję »krytycznego łańcucha« do skutecznego planowania zarówno harmonogramów, jak i funkcjonalności. Podobnie jak *User Stories Applied*, ta książka jest łatwa w czytaniu i bazuje na realnych doświadczeniach”.

— Bill Wake, autor książki *Refactoring Workbook* (Addison-Wesley, 2003)

„W tej książce Mike prezentuje praktyczne przykłady, które pomagają zrozumieć, dlaczego stosowanie podejścia Agile do planowania projektów rozwoju oprogramowania sprawdza się w praktyce. Ta książka ma ogromny zakres — począwszy od podstaw planowania wydań, a skończywszy na tak zaawansowanych tematach, jak aspekty finansowe ustalania priorytetów. Niniejsza książka może stać się nieocenioną pomocą dla menedżerów projektów Agile, ponieważ zawiera wiele praktycznych porad — na przykład jak określić długość iteracji oraz początkowe tempo oraz w jaki sposób informować o postępach projektu”.

— Rachel Davies, niezależna konsultantka

„Na rynku istniało zapotrzebowanie na solidną, praktyczną książkę dla menedżerów projektów dotyczącą długoterminowej wizji projektu Agile. Książka *Agile. Metodyki zwinne w planowaniu projektów* jest odpowiedzią na tę potrzebę. To nie teoria — niniejsza książka opisuje praktyki sprawdzone w rzeczywistych projektach Agile. W ramach testów z powodzeniem sprawdziliśmy metody Mike’a podczas projektowania gier wideo (jest to jedno z najbardziej nieprzewidywalnych środowisk projektowych, jakie można sobie wyobrazić)”.

— Clinton Keith, dyrektor techniczny firmy High Moon Studios

„Gdy po raz pierwszy usłyszałem wykład Mike’a Cohna, byłem pod wrażeniem rzadkiej kombinacji zalet: bogatego doświadczenia i zrozumienia nowoczesnych, iteracyjnych metod Agile — co pozwoliło znaleźć i zweryfikować łatwe, wysokowydajne rozwiązania wykraczające poza status quo tradycyjnych (zazwyczaj nieskutecznych) metod — z pasją i jasnością przekazu naturalnego trenera. Wspomniane cechy są wyraźnie widoczne w tym doskonałym, praktycznym przewodniku. Sądzę, że czytelnik nie będzie rozczarowany po przestudiowaniu prezentowanych porad i zastosowaniu ich w praktyce”.

— Craig Larman, główny naukowiec w firmie Valtech oraz autor książek *Applying UML and Patterns*, wydanie trzecie (Prentice Hall, 2005) oraz *Agile and Iterative Development* (Addison-Wesley, 2004)

„*Agile. Metodyki zwinne w planowaniu projektów* to kluczowy przewodnik opisujący, w jaki sposób skutecznie dostarczyć wartość dla klientów usług IT. Ta książka jest wypełniona czytelnymi przykładami, które mają kluczowe znaczenie dla wszystkich uczestników projektu — począwszy od szeregowych członków zespołu projektowego, a skończywszy na kierownictwie”.

— Lou Thomas, dyrektor ds. rozwoju aplikacji, Farm Credit Services of America

„Niniejsza praca ma olbrzymie znaczenie. Formułuje i szczegółowo opisuje zorganizowane techniki, które pozwalają zrozumieć prostą, ale kluczową zasadę — planowanie to iteracyjne dążenie do uzyskania wartości, a nie powierzchowne tworzenie harmonogramów. Mike opisuje procesy, które łącznie tworzą tę wartość, a jednocześnie przyczyniają się do zmniejszenia ryzyka i niepewności, wspierają procesy podejmowania decyzji, zyskiwania zaufania i przekazywania informacji. Metody Agile spowodowały przesunięcie akcentu z planów na planowanie, ale wiele szczegółów dotyczących planowania procesów było niejasnych. Mike bardzo prosto i przystępnie pokazuje, jak to można robić”.

— Sanjiv Augustine, dyrektor działu praktyk, konsultant w dziedzinie technologii Lean-Agile w firmie CC Pace i autor książki *Managing Agile Projects* (Prentice Hall, 2005)

„Metody opisane w książce *Agile. Metodyki zwinne w planowaniu projektów* niesamowicie pomogły nam w planowaniu i zarządzaniu naszymi projektami. Książka zapewnia czytelnikom wszystko, czego kiedykolwiek chcieli się dowiedzieć o zarządzaniu projektami Agile”.

— Roman Pichler, menedżer działu inżynieryjnego, Siemens Communications

„Mike Cohn zaprezentował bardzo praktyczne i logiczne podejście pozwalające na osiągnięcie sukcesu w projektach, w świecie, w którym panuje niepewność i zachodzą zmiany. Dzięki swojej marce, czytelności wypowiedzi i bezpośredniości Mike przebija się przez menedżerski bełkot i prezentuje praktyczne techniki możliwe do natychmiastowego wdrożenia. Wykorzystaj rady Mike’a i zastosuj je podczas planowania w kolejnym dużym projekcie”.

— Pete Deemer, wiceprezes ds. rozwoju produktów w firmie Yahoo!

„Ta książka pokazuje podstawowe koncepcje leżące u podstaw planowania i szacowania Agile w przemyśle i przystępny sposób. Jest to lektura obowiązkowa prezentująca jasne, jednoznaczne i praktyczne porady dla wszystkich tych, którzy chcą skutecznie zarządzać nowoczesnymi projektami rozwoju oprogramowania”.

— Scott W. Ambler, prezes firmy Ambyssoft Inc.

Dla Laury, bez cienia wątpliwości.

Spis treści

O autorze	17
Słowo wstępne Roberta Martina	19
Słowo wstępne Jima Highsmitha	21
Słowo wstępne Gabrielle Bennefield	25
Podziękowania	27
Wprowadzenie	29

CZĘŚĆ I PROBLEM I CEL

Rozdział 1.	Cele planowania	33
	Po co to robić?	34
	Co sprawia, że plan jest dobry?	37
	Co sprawia, że planowanie jest zwinne?	38
	Podsumowanie	39
	Pytania do dyskusji	39
Rozdział 2.	Dlaczego planowanie zawodzi?	41
	Planowanie dotyczy aktywności zamiast funkcjonalności	41
	Wielozadaniowość prowadzi do dalszych opóźnień	44
	Funkcjonalności nie są opracowywane według priorytetów	46
	Ignorujemy niepewność	46
	Oceny stają się zobowiązaniami	47
	Podsumowanie	47
	Pytania do dyskusji	48

Rozdział 3.	Zwinne podejście	49
	Zwinne podejście do projektów	50
	Zwinne podejście do planowania	54
	Podsumowanie	58
	Pytania do dyskusji	59

CZĘŚĆ II OCENA ROZMIARÓW

Rozdział 4.	Szacowanie rozmiaru za pomocą punktów	63
	Punktacja historyjek jest względna	63
	Tempo	65
	Podsumowanie	68
	Pytania do dyskusji	68
Rozdział 5.	Szacowanie w dniach idealnych	69
	Czas idealny a wytwarzanie oprogramowania	70
	Dni idealne jako miara rozmiaru	71
	Jedna ocena, a nie wiele	72
	Podsumowanie	72
	Pytania do dyskusji	73
Rozdział 6.	Metody oceny	75
	Oceny są wspólne	77
	Skala oceny	77
	Wyznaczanie oceny	79
	Poker planistyczny	81
	Dlaczego poker planistyczny się sprawdza?	83
	Podsumowanie	84
	Pytania do dyskusji	84
Rozdział 7.	Weryfikacja ocen	85
	Przedstawiamy witrynę SwimStats	85
	Kiedy nie należy weryfikować ocen?	86
	Kiedy weryfikować oceny?	87
	Weryfikacja oceny częściowo zrealizowanych historyjek	89
	Cel weryfikowania ocen	90
	Podsumowanie	90
	Pytania do dyskusji	91
Rozdział 8.	Wybór między punktami historyjek a dniami idealnymi	93
	Powody przemawiające za metodą punktów	93
	Powody przemawiające na rzecz dni idealnych	96

Zalecenia	97
Podsumowanie	98
Pytania do dyskusji	98

CZĘŚĆ III PLANOWANIE POD KĄTEM WARTOŚCI

Rozdział 9. Priorytety tematów	101
Czynniki mające znaczenie przy określaniu priorytetów	102
Połączenie czterech czynników	107
Przykłady	107
Podsumowanie	109
Pytania do dyskusji	109
Rozdział 10. Priorytety finansowe	111
Źródła zwrotu	112
Przykład: WebPayroll	115
Miary finansowe	120
Porównywanie zwrotów	124
Podsumowanie	126
Pytania do dyskusji	126
Rozdział 11. Priorytety atrakcyjności	127
Model Kano satysfakcji klientów	127
Inne podejście — wagi względne	132
Podsumowanie	134
Pytania do dyskusji	134
Rozdział 12. Podział historyjek użytkownika	135
Kiedy należy podzielić historyjkę użytkownika?	135
Podział według granic danych	136
Podział według granic operacyjnych	137
Usuwanie problemów funkcjonalności przekrojowych	138
Nie staraj się spełniać wymagań wydajności	139
Podział historyjek o mieszanych priorytetach	140
Nie należy dzielić historyjki na zadania	140
Unikaj pokusy zmian powiązanych	141
Łączenie historyjek	141
Podsumowanie	141
Pytania do dyskusji	142

CZĘŚĆ IV TWORZENIE HARMONOGRAMÓW

Rozdział 13. Podstawy planowania wydań	145
Plan wydania	146
Aktualizacja planu wydania	150
Przykład	150
Podsumowanie	152
Pytania do dyskusji	153
Rozdział 14. Planowanie iteracji	155
Podczas planowania iteracji nie są przydzielane zadania	157
Różnice między planowaniem iteracji a planowaniem wydania	158
Planowanie iteracji sterowane tempem	159
Planowanie iteracji sterowane zobowiązaniami	167
Moja rekomendacja	170
Związek ocen zadań z punktacją historyjek	171
Podsumowanie	173
Pytania do dyskusji	173
Rozdział 15. Wybór długości iteracji	175
Czynniki przy wyborze długości iteracji	175
Podejmowanie decyzji	179
Studia dwóch przypadków	180
Podsumowanie	182
Pytania do dyskusji	182
Rozdział 16. Szacowanie tempa	183
Wykorzystanie danych historycznych	183
Przeprowadzenie iteracji	184
Prognozowanie	186
Jakiego sposobu należy użyć?	190
Podsumowanie	190
Pytania do dyskusji	191
Rozdział 17. Plany z buforami w celu uwzględnienia niepewności	193
Bufory funkcjonalności	194
Bufory harmonogramu	195
Łączenie buforów	202
Bufor harmonogramu to nie wypełnienie	203
Ostrzeżenia	203
Podsumowanie	204
Pytania do dyskusji	204

Rozdział 18. Planowanie projektu z udziałem wielu zespołów	205
Ustalenie wspólnej podstawy do ocen	206
Wcześniejsze wprowadzanie szczegółów do historyjek użytkownika	206
Planowanie wyprzedzające	207
Wykorzystywanie w planie buforów zasilających	209
Ale to tyle pracy... ..	211
Podsumowanie	211
Pytania do dyskusji	212

CZĘŚĆ V ŚLEDZENIE I KOMUNIKACJA

Rozdział 19. Monitorowanie planu wydania	215
Śledzenie wydania	216
Wykresy wypalania wydania	218
Wykres parkingowy	223
Podsumowanie	223
Pytania do dyskusji	224
Rozdział 20. Monitorowanie planu iteracji	225
Tablica zadań	225
Wykresy wypalania iteracji	228
Śledzenie poświęconego wysiłku	228
Indywidualne tempo	229
Podsumowanie	229
Pytania do dyskusji	230
Rozdział 21. Ogłaszanie planów	231
Ogłaszanie planu	232
Informowanie o postępach	234
Podsumowanie na końcu iteracji	235
Podsumowanie	239
Pytania do dyskusji	240

CZĘŚĆ VI DLACZEGO PLANOWANIE AGILE JEST SKUTECZNE?

Rozdział 22. Dlaczego planowanie Agile się sprawdza?	243
Zmiany planów zdarzają się często	243
Oceny rozmiaru i czasu trwania są oddzielne	244
Plany sporządza się na różnych poziomach	244
Podstawą planów są funkcjonalności, a nie zadania	245
Dzięki niewielkim historyjkom prace postępują	245

W każdej iteracji eliminujemy pracę w toku	246
Śledzenie odbywa się na poziomie zespołu	246
Niepewność jest potwierdzona i zaplanowana	247
Wytyczne dla procesu szacowania i planowania Agile	247
Podsumowanie	249
Pytania do dyskusji	249

CZĘŚĆ VII STUDIUM PRZYPADKU

Rozdział 23. Studium przypadku: Bomb Shelter Studios	253
Dzień 1. — poniedziałek rano	254
Szacowanie historyjek użytkownika	261
Przygotowanie do badania produktu	270
Planowanie iteracji i wydania. Runda 1.	273
Dwa tygodnie później	288
Planowanie drugiej iteracji	289
Kolejne dwa tygodnie później	291
Przegląd planu wydania	291
Zaprezentowanie Filipowi skorygowanego planu	294
Osiemnaście tygodni później	297
Bibliografia	299
Skorowidz	303

O autorze

Mike Cohn jest założycielem Mountain Goat Software — firmy świadczącej usługi konsultingowe i szkoleniowe w dziedzinie prowadzenia projektów i zarządzania nimi. Mike specjalizuje się w pomaganiu firmom we wdrażaniu i doskonaleniu stosowanych procesów i technik Agile w celu zbudowania organizacji wytwarzających oprogramowanie o bardzo wysokiej wydajności. Oprócz tej książki jest autorem książki *User Stories Applied for Agile Software Development*, a także książek poświęconych programowaniu w językach Java i C++. Mike ma ponad dwudziestoletnie doświadczenie. Wcześniej był dyrektorem ds. technicznych w firmach różnych rozmiarów — począwszy od start-upów do firm z listy „Fortune 40”. Jest również autorem artykułów publikowanych w magazynach „Better Software”, „IEEE Computer”, „Cutter IT Journal”, „Software Test and Quality Engineering”, „Agile Times” oraz „C/C++ Users Journal”. Mike często wygłasza referaty na konferencjach branżowych, jest członkiem założycielem organizacji Agile Alliance i zasiada w jej radzie nadzorczej. Jest certyfikowanym trenerem ScrumMaster oraz członkiem stowarzyszeń IEEE Computer Society i ACM.

Aby uzyskać więcej informacji, odwiedź stronę www.mountaingoatsoftware.com lub napisz e-mail na adres mike@mountaingoatsoftware.com.

Słowo wstępne Roberta Martina

Wszędzie w świecie Agile słyszę te same pytania:

- Jak prowadzić planowanie dla dużych zespołów?
- Jaki rozmiar iteracji należy zastosować?
- Jak należy raportować postępy dla kierownictwa?
- W jaki sposób nadawać priorytet historyjkom użytkownika?
- Jak uzyskać „duży obraz” projektu?

Autor niniejszej książki umiejętnie udzielił odpowiedzi na te oraz na wiele innych pytań. Jeśli jesteś menedżerem projektu, liderem, programistą lub dyrektorem, to ta książka dostarczy Ci narzędzi niezbędnych do oszacowania, zaplanowania projektów Agile praktycznie dowolnych rozmiarów i zarządzania nimi.

Znam Mike’a Cohna od pięciu lat. Spotkałem go wkrótce po podpisaniu Manifestu Agile. Mike dołączył do organizacji Agile Alliance z wyjątkowym entuzjazmem i energią. Każdy projekt, za który brał odpowiedzialność, kończył z sukcesem. Był widoczny i przydatny. Bardzo szybko stał się niezbędny dla młodej organizacji.

Teraz ten sam poziom kompetencji, staranności i energii zaprezentował w swojej książce. Ta książka jest tego świadectwem. Świadectwem wielkiego formatu.

To dlatego, że niniejsza książka daje niezwykle praktyczne wskazówki. To nie jest książka o abstrakcjach. Czytelnik podczas jej lektury nie będzie spędzać czasu, bujając w obłokach, patrząc na problemy z wysokości 10 000 metrów. Zamiast tego w książce zamieszczono konkretne praktyki, techniki, narzędzia, wykresy, wzory i — co najbardziej wartościowe — przekonujące wskazówki. Jest to praktyczny przewodnik „jak to zrobić” opisujący zagadnienia szacowania i planowania.

W kilku miejscach w książce znajdziemy anegdoty związane z doświadczeniami Mike’a w korzystaniu z opisywanych metod i narzędzi. Mike mówi nam, kiedy one się sprawdziły, a kiedy nie. Informuje, co może się nie powieść, a co ma szansę zakończyć się sukcesem. Nie daje żadnych obietnic, ofert, srebrnych kul ani gwarancji. Jednocześnie nie pozostawia wątpliwości, że oferuje olbrzymi fragment własnego, zdobytego ciężką pracą doświadczenia.

Było wiele książek, w których poruszano zagadnienia szacowania i planowania w projektach Agile. W przypadku kilku spośród nich był to główny temat. Ale żadnej z tych pozycji nie można porównać pod względem głębi i przydatności z tą książką, która opisuje temat tak wyczerpująco i skutecznie, że śmiało można ją uznać za pracę kompletną.

OK, wiem, że trochę słodzę, ale jestem podekscytowany. Cieszę się, że na wiele ważnych pytań, które długo pozostawały bez odpowiedzi, w końcu znaleziono wystarczająco kompetentną odpowiedź. Cieszę się, że teraz mam narzędzie, które mogę polecić swoim klientom, gdy będą zadawali trudne pytania. Cieszę się też, że ta książka jest dostępna i że czytelnik może ją przeczytać.

Cieszę się i mam zaszczyt mieć tę książkę w swojej serii. Myślę, że to bardzo udana pozycja.

Robert C. Martin
Redaktor serii

Słowo wstępne Jima Highsmitha

Czytając książkę lub rękopis po raz pierwszy, zawsze zadaję sobie to samo pytanie: „Co autor dodaje od siebie do stanu wiedzy na opisywany temat?”. W przypadku Mike’a odpowiedź na to pytanie jest dwójakiego rodzaju: jego książka dodaje nam wiedzy na temat tego, „jak” przeprowadzać ocenę i planowanie oraz „dlaczego” określone praktyki są ważne.

Planowanie Agile może być mylące. Z jednej strony to dość łatwe — wystarczy stworzyć kilka kart z historyjkami użytkowników, nadać im priorytety, przydzielić do iteracji wydań, a następnie wprowadzać nowe detale w celu uzyskania planu kolejnej iteracji. Podstawy planowania można przedstawić zespołowi w kilka godzin i faktycznie zespół może stworzyć zadowalający plan (w przypadku małych projektów) w kilka kolejnych godzin. Książka Mike’a okaże się bardzo pomocna w wykonaniu zadania polegającego na przekształceniu zadowalających planów w bardzo dobre plany. Pisząc to, ostrożnie dobieram słowa — nie powiedziałem „doskonałych planów”, ponieważ, jak przekonuje Mike w niniejszej książce, różnica między dobrym (wystarczająco dobrym) planem a doskonałym planem, prawdopodobnie nie jest warta podejmowania dodatkowego wysiłku.

Moje pierwsze opinie o książce Mike’a mają związek z samą koncepcją zwinnego planowania. Często mnie rozbawia, a czasem smuci, brak zrozumienia planowania w projektach Agile. Słyszymy krytykę w rodzaju „zwinne zespoły projektowe nie planują” albo „zwinne zespoły nie podejmują zobowiązań w odniesieniu do dat i funkcjonalności”. Tego rodzaju pomyłkę popełnili Even Barry Boehm i Richard Turner w książce *Balancing Agility and Discipline: A Guide for the Perplexed* (Addison-Wesley, 2004), gdy mówili o metodach sterowanych planem w zestawieniu z metodami Agile. Boehm i Turner w istocie prawidłowo przedstawili ideę, ale użyli niewłaściwych terminów. Pod pojęciem projektów sterowanych planami rozumieli projekty, w których większą wagę przywiązuje się do przewidywań niż do adaptacji, natomiast w projektach Agile jest odwrotnie.

Problem z określeniami „projekt sterowany planami” i „zwinny” polega na tym, że przekazują one całkowicie błędny komunikat — zespoły Agile nie planują. Nie może być określenia, które by bardziej odbiegało od prawdy. Książka Mike’a wysła prawidłowy komunikat — planowanie jest integralną częścią każdego projektu Agile. Książka zawiera ogromne bogactwo pomysłów dotyczących planowania, wyjaśnia dlaczego jest tak ważne i w jaki sposób skutecznie planować.

Po pierwsze, zespoły Agile wykonują wiele zadań planowania, ale zadania te są równomiernie rozłożone w ciągu całego projektu. Po drugie, zespoły Agile zdecydowanie przeciwstawiają się krytycznemu czynnikowi, który jest ignorowany przez wiele zespołów niestosujących metod Agile

— niepewności. Czy planowanie jest ważne? — absolutnie. Czy korekty planu w miarę zdobywania wiedzy i zmniejszania niepewności są ważne? — zdecydowanie tak. W wielu organizacjach spotykałem się z sytuacjami, kiedy podejmowanie dziwacznych zobowiązań we wczesnej fazie projektu i niedotrzymywanie ich było dopuszczalne, natomiast próby realnej oceny sytuacji (i uwzględniania niepewności) były określane jako „niezrozumienie programu” lub „gra niezespołowa”. W tych firmach niepowodzenie w dostarczaniu wartości wydaje się dopuszczalne, natomiast niepodjęcie zobowiązań (nawet w kwestii zupełnie nierealnych celów) jest nie do przyjęcia. Podejście Agile, zgodnie z tym, co tak umiejętnie opisuje Mike, koncentruje się na rzeczywistym dostarczaniu wartości zamiast tworzenia nierealnych planów i podejmowania takich właśnie zobowiązań. Członkowie zespołów Agile w rzeczywistości mówią: stworzymy plan bazujący na tym, co wiemy dzisiaj; będziemy dostosowywać ten plan w celu zaspokojenia najbardziej kluczowych celów; będziemy korygować projekt i plany w miarę uzyskiwania postępów i zdobywania nowych informacji. Spodziewamy się zrozumienia tego, co jest oczekiwane, oraz uświadomienia sobie, że elastyczność w dostosowywaniu się do zmieniających się warunków biznesowych i całkowita zgodność z planami przyjętymi z góry to cele sprzeczne ze sobą. Książka *Agile. Metodyki zwinne w planowaniu projektów* opisuje wszystkie człony powyższego zdania.

Wracając do kluczowego pytania o zarządzanie niepewnością — Mike robi świetną robotę, przyglądając się temu, w jaki sposób w procesie rozwoju Agile jednocześnie dążymy do zredukowania niepewności względem celu (co naprawdę chcemy zbudować) oraz sposobu (jak mamy zamiar to budować). Wiele tradycyjnych planistów nie rozumie kluczowego pojęcia — nie można „zaplanować” niepewności. Plany bazują na tym, co wiemy w danej chwili. Niepewność to inny sposób wyrażenia tego, czego nie wiemy — o celach lub narzędziach. W przypadku większości sytuacji, w których panuje niepewność (brak wiedzy), jedynym sposobem na zmniejszenie niepewności i zdobycie wiedzy jest wykonanie działań — zrobienie, zbudowanie lub zasymulowanie czegoś — i uzyskanie informacji zwrotnych. W wielu metodykach zarządzania projektami myślą przewodnią jest zdanie „planuj, planuj, planuj, a potem rób”, natomiast podejście Agile można raczej sformułować następująco: „planuj-rób-koryguj, planuj-rób-koryguj”. Im większa niepewność projektu, tym bardziej kluczowe jest stosowanie podejścia Agile dla odniesienia sukcesu.

Chciałbym zwrócić uwagę na odpowiedzi na pytania „jak” i „dlaczego” udzielone w książce Mike’a w rozdziałach 4. i 5., w których szczegółowo przedstawiono sposoby szacowania punktacji historyjek i dni idealnych oraz wyjaśniono zalety i wady każdego z tych podejść. Próbowałem obu podejść z klientami, jednak opis Mike’a pozwolił mi skrystalizować swoje opinie na temat korzyści płynących z oceny punktowej historyjek. Zdałem sobie sprawę, że punkty historyjek są częścią ewolucji — ewolucji zmierzającej do prostoty. Organizacje zajmujące się wytwarzaniem oprogramowania od dawna poszukiwały odpowiedzi na pytanie „Jak duże jest oprogramowanie”. Budowniczy domu może sensownie ocenić skalę budowy na podstawie powierzchni. Chociaż kosztorysy różnych wykonawców mogą różnić się od siebie, to rozmiar jest stały (choć na szacunki mają wpływ prace wykończeniowe, specyfikacja materiałów oraz wiele innych czynników) i utrzymuje się na niezmiennym poziomie. Twórcy oprogramowania od dawna szukali takiej miary.

W branży wytwarzania oprogramowania najpierw wykorzystywaliśmy liczbę wierszy kodu w produkcie (ta miara do dziś ma zastosowanie). Dla większości zadań codziennego planowania szacowanie liczby wierszy kodu z wielu powodów okazało się mieć ograniczone zastosowanie — między innymi ze względu na ilość początkowej pracy wymaganej do przeprowadzenia takiej oceny. Następnie „na scenie” pojawiła się punktacja funkcji (i kilka podobnych pomysłów). Punktacja funkcji wyeliminowała szereg problemów właściwych dla liczenia wierszy kodu. Stosowanie tej metody nadal

jednak wymagało dużego wysiłku „z góry” w celu wykonania obliczeń (trzeba było oszacować wejścia, wyjścia, pliki i tak dalej). Jednak tym, co powoduje, że punktacja funkcji nie może być szeroko zastosowana, jest złożoność tej metody. Z moich obserwacji wynika, że w miarę zwiększania się złożoności obliczeń — aby zorientować się, jaki jest stopień złożoności obliczeń, wystarczy przejrzeć witrynę internetową grupy **IFPUG** (ang. *International Function Point User Group*) — wykorzystanie metody się zmniejsza.

Niemniej jednak potrzeba szacowania „rozmiaru” projektu oprogramowania się nie zmniejszała. Problem z obydwoma historycznymi miarami jest dwojaki — są one trudne do obliczania i bazują na kaskadowej metodzie wytwarzania oprogramowania. Nadal potrzebujemy miary rozmiaru. Potrzebna jest taka miara, która byłaby prosta do obliczania oraz mogłaby być stosowana bez wchodzenia w szczegóły faz wymagań i projektowania.

Dwie ważne różnice między punktacją historyjek a liczbą wierszy kodu i punktacją funkcji polegają na tym, że punktację historyjek da się obliczyć o wiele prościej oraz można to zrobić znacznie wcześniej. Dlaczego obliczenia są prostsze? Ponieważ bazują na rozmiarze względnym, a nie bezwzględnym. Dlaczego mogą być wykonane wcześniej? Ponieważ bazują na rozmiarze względnym, a nie bezwzględnym. Jak zauważa Mike, szacowanie bazujące na punktacji historyjek polega na omawianiu historyjek (zdobywaniu wspólnej wiedzy) i szacowaniu ich względnego rozmiaru. Ocenę względnego rozmiaru, w przeciwieństwie do rozmiaru bezwzględnego, można wykonać bardzo szybko. Ponadto po kilku iteracjach szacowania rozmiaru i dostarczania produktu dokładność szacunków znacznie się poprawia. Opis Mike’a wyjaśnia, „jak” i „dlaczego” metoda szacowania bazująca na punktacji historyjek w zestawieniu z dniami idealnymi daje głęboki wgląd w ten kluczowy temat.

Jeszcze inny przykład staranności Mike’a można zaobserwować w rozdziałach od 9. do 11., poświęconych nadawaniu priorytetów historyjkom. Mike nie zadowolona się udzieleniem czytelnikom rady, że historyjki o najwyższej ocenie punktowej należy realizować w pierwszej kolejności. Zamiast tego zagłębia się w kluczowe aspekty wartości: korzyści finansowe, koszty, innowacje, wiedzę i zagrożenia. Dokładnie definiuje każdy z tych aspektów wartości (uwzględniając pojęcia wartości bieżącej netto — NPV, wewnętrznej stopy zwrotu — IRR oraz innych instrumentów finansowych), a następnie przedstawia kilka schematów (o różnym stopniu uproszczenia) w celu określenia wagi decyzji w odniesieniu do tych aspektów.

Często się zdarza, że osoby początkujące w zwinnych technikach wytwarzania oprogramowania uważają, że skoro zgodnie z określoną metodą realizowały projekt dwanaście, dziesięć lub osiem razy, to są specjalistami w dziedzinie metody Agile, Extreme, Crystal Clear lub dowolnej innej. W rzeczywistości jednak jesteś ekspertem Agile, Extreme lub jakiegokolwiek innej metody, jeśli wiesz o niej wystarczająco dużo, aby umieć ją zaadaptować do własnej, konkretnej sytuacji. Podstawowe znaczenie dla wytwarzania oprogramowania zgodnie z metodami Agile ma ciągłe uczenie się i adaptacja. W tej książce Mike zrobił jedną rzecz bardzo dobrze — zaprezentował pomysły i doświadczenia, które pomagają wnieść stosowane praktyki zwinnego szacowania i planowania na nowy poziom trudności. Mike dokładnie odpowiada nam na pytanie „jak” — na przykład w materiale dotyczącym szacowania za pomocą punktacji historyjek i dni idealnych. Szczegółowo odpowiada na pytanie „dlaczego” — na przykład podając plusy i minusy zarówno punktacji historyjek, jak i dni idealnych. Choć zazwyczaj przekazuje nam swoją osobistą rekomendację (preferuje punktację historyjek), to dostarcza nam także wystarczająco dużo informacji do tego, abyśmy samodzielnie mogli się przekonać, którą metodę należy zastosować w konkretnej sytuacji.

Podsumowując — niniejsza książka stanowi znaczący wkład Mike’a w obecny stan wiedzy — Mike pomaga nam myśleć o praktykach szacowania i planowania na nowym poziomie wiedzy i doświadczenia (odpowiedzi „jak”) oraz pomaga nam podjąć decyzje o zastosowaniu tej nowej wiedzy

w celu przystosowania poznanych praktyk do nowych, wyjątkowych lub po prostu konkretnych sytuacji (odpowiedź na pytanie „dlaczego”). Spośród pół tuzina książek, które zwykle polecam klientom, Mike napisał dwie. *Agile. Metodyki zwinne w planowaniu projektów* znajduje się na mojej liście lektur obowiązkowych dla wszystkich osób chcących poznać stan rzeczy w tym aspekcie zwinnych metod zarządzania projektami.

Jim Highsmith
Dyrektor działu praktyk Agile w firmie Cutter Consortium
Flagstaff, Arizona
Sierpień 2005 roku

Słowo wstępne Gabrielle Bennefield

„Metody Agile wytwarzania oprogramowania nie sprawdzą się w firmie Yahoo!. Wyjątkiem mogą być małe, taktyczne projekty. To dlatego, że zespoły niczego nie planują, a członkowie zespołów nie potrafią oszacować własnej pracy. Trzeba im powiedzieć, co mają zrobić”.

To jest prawdziwy cytat, który słyszałam wielokrotnie, kiedy zaczynałam wdrażać metody Agile w firmie Yahoo!. Ludzie, którzy nie rozumieją koncepcji technik Agile wytwarzania oprogramowania, uważają, że polegają one po prostu na wyeliminowaniu całej dokumentacji i planowania oraz na daniu zespołom zielonego światła na kodowanie. Rzeczywistość nie może być dalej od prawdy.

„Zespoły niczego nie planują”. Ci, którzy tak mówią, zapominają o tym, że zespół Agile co drugi tydzień poświęca pół dnia na stworzenie listy zadań, które powinny być zrealizowane na koniec dwutygodniowego okresu w celu dostarczenia jakiejś wartościowej dla użytkownika funkcjonalności. Fakt, że planowanie w zespołach Agile jest rozproszone na cały okres trwania projektu, a nie jest wykonywane z góry, jest postrzegany jako brak planowania. To nie jest tak. Zespoły Agile w firmie Yahoo! tworzą produkty, z których menedżerowie są zadowoleni znacznie bardziej, niż kiedykolwiek byli z produktów tworzonych przez tradycyjne zespoły.

„Członkowie zespołu nie mogą ocenić swojej pracy i trzeba im powiedzieć, co mają robić”. To klasyczny błąd w interpretacji. Liczenie na to, że menedżerowie produktu lub projektu będą zdolni przewidzieć to, co potrafią przewidzieć osoby będące ekspertami w swojej pracy, jest w istocie biznesowym samobójstwem. Często takie podejście jest de facto sposobem na udzielenie biznesowi zgody na spełnienie nierealistycznych celów. W takich przypadkach wymaga się od członków zespołu, by pracowali na okrągło przez całą dobę oraz by chodzili na skróty. A my się zastanawiamy, dlaczego w naszej branży ludzie są wypaleni i mają tak niskie morale.

Szacowanie i planowanie należą do zagadnień, na których temat otrzymuję najwięcej pytań — zwłaszcza od nowych zespołów. Stosowanie prostego podejścia do planowania — nie tylko w jednej iteracji, ale w całym projekcie ma nieocenione znaczenie. Menedżerowie produktu powinni dążyć do osiągnięcia założonych przychodów oraz tworzenia przewidywalnych planów publikacji. Zespoły nadal mogą być elastyczne i w miarę potrzeby zmieniać kurs. Ważne jest jednak, aby mieć „mapę drogową”, którą można podążać. Nie wystarczy iść szybko, jeśli idziesz w złym kierunku. Nauczenie się sposobu oceniania i planowania to jeden z najważniejszych składników sukcesu podczas wdrażania metody Agile w organizacji.

Wykłady Mike'a dotyczące szacowania i planowania należą do najbardziej popularnych wykładów na temat zwinnych metod wytwarzania oprogramowania, jakie prowadzimy w firmie Yahoo!. Dzięki nim zespoły zyskują umiejętności i narzędzia potrzebne do wykonywania takiej ilości planowania, jaka jest potrzebna do zoptymalizowania wyników. Czy stosowanie się do zaleceń Mike'a rzeczywiście się sprawdza? Tak. Sukces metod Agile w firmie Yahoo! jest niewiarygodny. Miałam do czynienia z zespołami, które wychodziły z wykładów Mike'a i natychmiast przekształcały jego rady w działania. Dzięki temu dostarczamy produkty na rynek w szybszym tempie, a członkowie zespołów naprawdę polubili podejście Agile. Dlaczego zwinne metody oceny i planowania są skuteczniejsze od tradycyjnych? Ponieważ koncentrują się na dostarczaniu wartości i wzbudzeniu zaufania między zespołami biznesowymi a projektowymi. Dzięki zachowaniu dużej przejrzystości oraz informowaniu o wszelkich zmianach, w miarę gdy się pojawiają, biznes zachowuje zdolność do szybkich adaptacji i podejmowania najlepszych decyzji.

W mojej ostatniej firmie miałam okazję obserwować przejście ze stanu permanentnego chaosu, kiedy mieliśmy bardzo ambitny plan, ale nie byliśmy w stanie dostarczać produktów, do stanu przewidywalności, gdy zyskaliśmy zdolność podejmowania się projektów, które byliśmy w stanie zrealizować. Menedżerom biznesowym nie zawsze podobały się odpowiedzi (w końcu oni zawsze chcą, żeby wszystko było na jutro), ale przynajmniej wierzyli w nasze odpowiedzi i nie byli rozczarowani z powodu poczucia, że ciągle ich okłamywano.

Ta książka trzyma się realiów. Nie odpowie Ci, jak zyskać 100-procentową dokładność ocen. To byłoby marnotrawstwo sił i cel niemożliwy do osiągnięcia. Książka Mike'a nie próbuje dostarczyć Ci pięknych szablonów do wypełnienia. Zamiast tego skłania Cię do myślenia i poznawania sposobów podejścia do rozwiązywania problemów. Nie ma dwóch takich samych projektów, produktów lub organizacji, dlatego znacznie ważniejsze jest nauczenie się myślenia i zasad. W tej książce Mike dzieli się swoim rozległym praktycznym doświadczeniem i osobowością. Książka jest prawdziwa i szczerza. Z całą pewnością tę książkę należy umieścić na początku listy lektur obowiązkowych.

Gabrielle Benfield
Dyrektor ds. wytwarzania produktów metodami Agile
Yahoo!

Podziękowania

Mam większy dług wdzięczności niż kiedykolwiek wobec korektorów merytorycznych tej książki. Tom Poppendieck, Steve Tockey, Paul Hodgetts, Mike Searfos i Preston Smith — wszyscy oni przekazali mi pomocne uwagi i sugestie. Dzięki Waszym wysiłkom ta książka niezwykle się poprawiła. W szczególności chciałbym podziękować Steve’owi i Tomowi za pomoc wykraczającą poza zwykły zakres obowiązków. Steve wskazał mi wiele pomysłów i koncepcji, które przeoczyłem, oraz kilka materiałów, których wcześniej nie znalazłem. Najważniejsze jest to, że wskazał mi regułę, którą powtarzam jak mantrę podczas prowadzenia wykładów dotyczących szacowania i planowania: oceniam rozmiar i na tej podstawie wnioskuję, jaki będzie czas trwania. Być może Tom włożył więcej wysiłku w tę książkę niż ja. Tom nieustrudzenie podkreślał mi znaczenie pisania tej książki dla całego zespołu, a nie tylko dla menedżera projektu. To dzięki rozmowom z Tomem zdałem sobie sprawę, że książka dotycząca planowania nie powinna sprowadzać się wyłącznie do udzielenia odpowiedzi na pytanie „Kiedy skończymy?”. W wielkim planie dostarczania wartości dla naszej organizacji to pytanie jest proste.

Dziękuję Johnowi Goodsenowi z firmy RADSoft. John i ja początkowo planowaliśmy napisać tę książkę razem. Nasze plany nie zazębiły się ze sobą, dlatego to nie było możliwe, ale chcę podziękować Johnowi za rozpoczęcie dyskusji o książce.

Jedną z wielkich zalet internetu jest możliwość dzielenia się książką z innymi podczas jej pisania. Prace nad tą książką trwały w mojej witrynie WWW przez dwadzieścia miesięcy, a niniejsza książka znacznie się poprawiła dzięki komentarzom i sugestiom przesłanym przez czytelników tych pierwszych wersji. W szczególności chciałbym podziękować Bryanowi Ambrogiano, Kenowi Auerowi, Simonowi Bakerowi, Rayowi Boehmowi, Leslie Borrell, Clarke’owi Chingowi, Lisie Crispin, Rachel Davis, Mike’owi Dwyerowi, Hakanowi Erdogmusowi, Johnowi Favaro, Chrisowi Gardnerowi, Johnowi Gilmanowi, Svenowi Gortsowi, Paulowi Grewowi, Sridharowi Jayaramanowi, Angelo Kastroulisowi, Lisie Katzenmeier, Lasse Koskeli, Mitchowi Laceyowi, Patrickowi Loganowi, Kentowi McDonałdowi, Erikowi Petersenowi, Kertowi Petersonowi, Mike’owi Polenowi, J. B. Rainsbergerowi, Billowi Ramosowi, Mattowi Readowi, George’owi Reilly’emu, Chrisowi Rimmerowi, Owenowi Rogersowi, Kevinowi Rutherfordowi, Dickowi Scheelowi, Jamesowi Schielowi, Kenowi Scottowi, Karłowi Scotlandowi, Alanowi Shallowayowi, Jagadishowi Shrinivasavadhaniowi, Michele Sliger, Karen Smiley, Hubertowi Smitsowi, Victorowi Szalvayowi, Charliemu Trainorowi, Rajowi Waghrayowi, Rüdigerowi Wolfowi, Scottowi Worleyowi i Jasonowi Yipowi.

Chciałbym również podziękować wszystkim, którzy wzięli udział w jednym z moich wykładów dotyczących zwinnego szacowania i planowania w ciągu ostatnich dwóch lat, zarówno w swoich firmach, jak i podczas konferencji. Dziękuję również każdemu z moich klientów, ale w szczególności tym, dla których prowadziłem wykłady dotyczące szacowania i planowania oraz którzy korzystają z koncepcji zaprezentowanych w tej książce, w tym pracownikom firm Farm Credit Systems of America, Fast401k, High Moon Studios, Nielsen Media Research, Sun Microsystems, Ultimate Software, VisionPace, Webroot, Yahoo! i innych.

Jak zawsze, wspaniale było pracować z zespołem wydawnictwa Prentice Hall. Paul Petralia i Michelle Housley razem ze mną rozpoczęli projekt i pozostali w nim do samego końca. Tyrrell Albaugh pomógł mi odpowiedzieć na niektóre trudne pytania dotyczące narzędzia FrameMaker. Poprosiłem o przydzielenie nieustępliwego redaktora, tak aby powstała książka jak najlepsza. Przydzielono mi Kathy Simpson, która była dokładnie tą osobą, o którą prosiłem. Na koniec Lara Wysong wykonała świetną robotę podczas przekształcania rękopisu w książkę, którą teraz trzymasz w swoich rękach. Ona również była nieustrudzona w udzielaniu odpowiedzi na moje setki pytań i e-maili.

Dziękuję Bobowi Martinowi za umieszczenie tej książki wśród innych wspaniałych książek w swojej serii. Wujek Bob jest jednym z moich ulubionych autorów poczynawszy od dawnych czasów, gdy pracował jako redaktor w magazynie „C++ Report”. Bob zrobił tak wiele dla rozpowszechnienia idei Agile w społeczności programistów, że posiadanie swojej książki w jego serii to wielki honor. Chciałbym również podziękować Jimowi Highsmithowi z firmy Cutter Consortium oraz Gabrielle Benefield z firmy Yahoo! za napisanie słowa wstępnego. Praca z każdym z nich zawsze sprawia mi wielką przyjemność.

Wielkie podziękowania należą się mojej rodzinie za czas, jaki od niej dostałem na pracę nad tą książką. Większą część tej pracy zamierzałem wykonać w podróży, aby zminimalizować czas, jaki spędzałem z dala od rodziny. Tak się nie stało. Laura, moja żona i partnerka we wszystkim, co robię, poświęciła się dla mnie, naszej firmy i tej książki. Wielokrotnie czytała wszystkie rozdziały, a potem robiła to ponownie. Bez jej pomocy nie byłbym w stanie zrobić nawet jednej dziesiątej tego, co robimy razem. Moje piękne córki, Savannah i Delaney, tak przyzwyczyły się do widoku mnie pracującego w moim domowym biurze, że byłoby dla nich dziwne, gdybym tam nie był. Dziękuję im za przytulanie się do mnie, całusy oraz po prostu za to, że są.

Wprowadzenie

Ta książka mogłaby nosić tytuł *Szacowanie i planowanie w projektach Agile*. Zamiast tego nadałem jej tytuł *Agile. Metodyki zwinne w planowaniu projektów*. Różnica może wydawać się subtelna, ale w rzeczywistości taka wcale nie jest. Tytuł daje do zrozumienia, że procesy szacowania i planowania same w sobie muszą być zwinne. Bez zwinnego szacowania i planowania nie możemy mówić o zwinnych projektach.

Ta książka jest przede wszystkim o planowaniu, które postrzegam jako udzielenie odpowiedzi na pytanie „Co powinniśmy zbudować i do kiedy?”. Jednak aby odpowiadać na pytania dotyczące planowania, trzeba także rozważyć kwestie szacowania („Jak duży jest projekt?”) oraz harmonogramowania („Kiedy to będzie zrobione?” oraz „Ile mogę wtedy mieć?”).

Treści w tej książce zorganizowano w siedmiu częściach i dwudziestu trzech rozdziałach. Każdy rozdział kończy się krótkim podsumowaniem kluczowych zagadnień oraz zestawem pytań do dyskusji. Ze względu na to, że szacowanie i planowanie powinny być przedsięwzięciami całych zespołów, mam nadzieję, że jednym ze sposobów, w jaki czytelnicy będą czytać tę książkę, będą wspólne spotkania członków zespołu, którzy zbierają się — na przykład co tydzień — i dyskutują o tym, co przeczytali, oraz odpowiadają na pytania zamieszczone na końcu poszczególnych rozdziałów. Ponieważ zwinne techniki wytwarzania oprogramowania są popularne na całym świecie, starałem się unikać pisania książki, która nadmiernie koncentruje się na realiach Stanów Zjednoczonych. W tym celu używam uniwersalnego symbolu waluty. Kwoty pieniężne są zapisywane jako 500 ₪ zamiast na przykład 500 \$ lub 500 €.

W części I opisano, dlaczego planowanie jest ważne, wskazano problemy, z jakimi mamy często do czynienia, oraz wymieniono cele zwinnego podejścia do wytwarzania oprogramowania. Książka zaczyna się od rozdziału 1., w którym opisałem cele planowania, co czyni plan dobrym oraz co sprawia, że planowanie staje się zwinne. Najważniejsze powody, dla których tradycyjne podejścia do szacowania i planowania dają niesatysfakcjonujące wyniki, wymieniono w rozdziale 2. Wreszcie rozdział 3. rozpoczyna się od krótkiego przypomnienia, czym jest zwinność. Następnie opisano w nim wysokopoziomowe podejście do zwinnego szacowania i planowania, które jest stosowane w dalszej części tej książki.

W części II wprowadzono podstawową zasadę szacowania — mówiącą, że oceny rozmiaru i czasu trwania powinny być wykonywane osobno. W rozdziałach 4. i 5. wprowadzono punktację historyjek oraz dni idealne, dwie jednostki właściwe do szacowania rozmiaru tworzonych własności. W rozdziale 6. opisano techniki szacowania w punktach historyjek lub dniach idealnych. Zawarto w nim również

opis planistycznego pokera. W rozdziale 7. opisano, kiedy i jak ponawiać szacowanie, a w rozdziale 8. zamieszczono wskazówki dotyczące wyboru między punktacją historyjek a idealnymi dniami.

W części III „Planowanie dla wartości” zamieszczono porady na temat tego, w jaki sposób zespół projektowy może zyskać pewność, że buduje najlepszy produkt. W rozdziale 9. opisano zbiór czynników, które należy wziąć pod uwagę przy ustalaniu priorytetów własności funkcjonalnych. W rozdziale 10. zaprezentowano podejście do modelowania finansowego zwrotu z własności lub zestawu własności oraz opisano, w jaki sposób można porównywać zwroty finansowe tak, aby zespół projektowy w pierwszej kolejności pracował nad funkcjonalnościami przynoszącymi największą wartość. Rozdział 11. zawiera wskazówki na temat tego, w jaki sposób oszacować, a następnie ustalić priorytet stopnia oczekiwania funkcjonalności przez użytkowników produktu. Tę część podsumowuje rozdział 12., w którym zamieszczono porady na temat tego, jak dzielić duże funkcjonalności na mniejsze — takie, które zapewniają wygodniejsze zarządzanie.

W części IV przenosimy naszą uwagę na pytania dotyczące tworzenia harmonogramów dla projektu. Rozdział 13. rozpoczyna się od omówienia etapów tworzenia harmonogramów w stosunkowo prostym, jednozespołowym projekcie. Dalej, w rozdziale 14., opisano, w jaki sposób zaplanować iterację. W rozdziałach 15. i 16. pokazano, jak wybrać odpowiednią długość iteracji projektu oraz jak ocenić początkowe tempo postępów zespołu. W rozdziale 17. szczegółowo opisano, jak stworzyć harmonogramy dla projektu z wysokim stopniem niepewności lub z większym prawdopodobieństwem popełnienia błędów w harmonogramie. Tę część kończy rozdział 18., w którym opisano dodatkowe kroki niezbędne do szacowania i planowania projektu, nad którym pracuje wiele zespołów.

Po ustaleniu planu należy podać go do wiadomości pozostałej części organizacji. Po tym fakcie można monitorować postępy zespołu względem przyjętego planu. Dotyczą tego tematy z trzech rozdziałów części V. Rozdział 19. obejmuje konkretnie monitorowanie planu wydań, natomiast rozdział 20. opisuje monitorowanie planu iteracji. W ostatnim rozdziale w tej części, rozdziale 21., przedstawiono ogłaszanie informacji na temat planu i postępów w jego realizacji.

Rozdział 22. to jedyny rozdział części VI. W tym rozdziale opisano powody, dla których szacowanie i planowanie się sprawdzają. Jest to przeciwwaga dla rozdziału 2., w którym opisano, dlaczego tradycyjne podejścia tak często zawodzą.

Część VII, która jest ostatnia, również zawiera tylko jeden rozdział. Rozdział 23. to zaawansowane studium przypadku, w którym potwierdzono główne tezy tej książki w pewnej, fikcyjnej sytuacji.

CZĘŚĆ I

Problem i cel

Aby zaprezentować zwinne podejście do szacowania i planowania, należy najpierw zrozumieć cel planowania. Temu tematowi poświęcony jest pierwszy rozdział tej części. W rozdziale 2. przedstawiono kilka najczęstszych powodów, dla których projekty z tradycyjnym planowaniem zwykle kończą się niepowodzeniem. Albo produkty nie są dostarczane na czas, albo klienci nie są nimi zachwyceni. Ostatni rozdział w tej części prezentuje wysokopoziomowy widok zwinnego podejścia, które opisano w dalszej części książki.

ROZDZIAŁ 1.

Cele planowania

Planowanie jest wszystkim. Plany są niczym.

— feldmarszałek Helmut Graf von Moltke

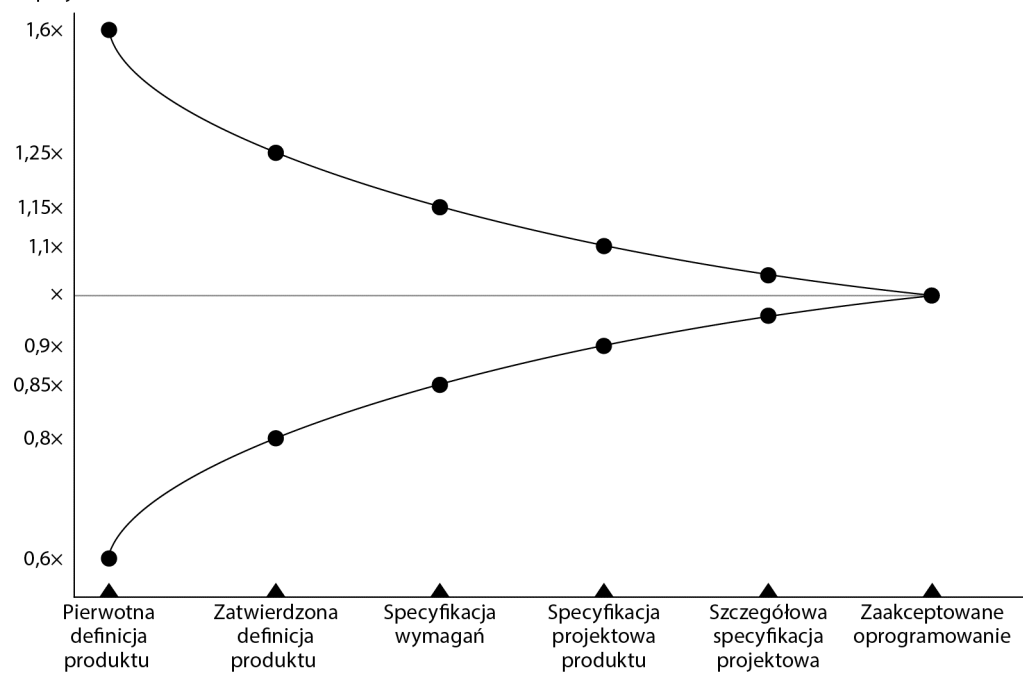
Szacowanie i planowanie mają kluczowe znaczenie dla sukcesu każdego projektu wytwarzania oprogramowania dowolnych rozmiarów lub o dowolnym przeznaczeniu. Plany wpływają na nasze decyzje dotyczące inwestycji: możemy rozpocząć określony projekt, jeśli ocenimy, że zajmie sześć miesięcy i będzie kosztował 1 milion $\square^1$, ale nie zdecydujemy się na zainicjowanie tego projektu, jeśli jego realizacja miałaby pochłonąć dwa lata i 4 miliony $\square$. Plany pomagają nam ustalić, kto powinien być dostępny do pracy nad projektem w ciągu danego okresu. Dzięki planom wiemy, czy projekt zmierza do dostarczenia użytkownikom funkcjonalności, których potrzebują i oczekują. Bez planów wiele problemów czyha na nasze projekty.

Planowanie jest jednak trudne, a plany są często błędne. Zespoły zwykle reagują na to, przyjmując jedną z dwóch skrajności: albo wcale nie planują, albo wkładają w plany tyle wysiłku, że staje się pewne, iż ich plany są słuszne. Zespół, który nie planuje, nie może odpowiedzieć na najbardziej podstawowe pytania — na przykład: „Kiedy skończysz?” albo „Czy możemy zaplanować wydanie produktu na czerwiec?”. Zespół, który planuje zbyt wiele, oszukuje sam siebie, sądząc, że każdy plan może być odpowiedni. Plany mogą być bardziej lub mniej szczegółowe, ale to nie musi oznaczać, że będą one bardziej dokładne lub użyteczne.

To, że szacowanie i planowanie jest trudne, nie jest żadną nowością. Wiemy to od dawna. W 1981 roku Barry Boehm stworzył pierwszą wersję czegoś, co Steve McConnell (1998) nazwał później „stożkiem niepewności”. Na rysunku 1.1 pokazano podstawowe zakresy niepewności na różnych etapach sekwencyjnego (kaskadowego) procesu wytwarzania oprogramowania. Stożek niepewności pokazuje, że w fazie oceny wykonalności projektu ocena harmonogramu zwykle waha się od 60% do 160%. Oznacza to, że projekt, który miał zająć 20 tygodni, może potrwać od 12 do 32 tygodni. Po napisaniu wymagań oszacowanie może nadal wahać się $\pm 15\%$ w dowolnym kierunku. Zatem ocena 20 tygodni może oznaczać, że prace potrwać od 17 do 23 tygodni.

¹ Pamiętajmy, że $\square$ to uniwersalny, generyczny symbol waluty.

Harmonogram projektu



RYSUNEK 1.1. Stożek niepewności zwęża się w miarę postępów rozwoju projektu

Project Management Institute (PMI) prezentuje podobny obraz progresywnej dokładności oceny. PMI nie postrzega jednak stożka niepewności jako symetrycznego. Zamiast tego interpretuje go jako asymetryczny. PMI sugeruje stworzenie *pierwotnego oszacowania*, które waha się od +75% do –25%. Następna estymata do stworzenia to *ocena budżetowa* w zakresie od +25% do –10%. Dopiero po niej następuje *ostateczna ocena*, z zakresu od +10% do –5%.

Po co to robić?

Jeśli oceny i planowanie są trudne i jeśli uzyskanie dokładnej oceny jest możliwe dopiero w późnej fazie projektu, to po co to w ogóle robić? Rzecz jasna są oczywiste powody — na przykład firmy, w których pracujemy, często wymagają przedstawienia ocen. Plany i harmonogramy mogą być potrzebne z wielu uzasadnionych powodów — na przykład z powodu planowania kampanii marketingowych, przygotowywania harmonogramów działań związanych z wydaniem produktów czy szkolenia wewnętrznych użytkowników. Wszystko to są ważne potrzeby, a trudności w ocenie projektu nie usprawiedliwiają rezygnacji z dostarczenia planu lub harmonogramu, które firma może wykorzystać do tych celów. Jednak poza tymi powierzchownymi potrzebami istnieje o wiele bardziej fundamentalny powód podjęcia się ciężkiej pracy związanej z szacowaniem i planowaniem.

Szacowanie i planowanie nie polegają wyłącznie na ustaleniu odpowiedniego terminu lub harmonogramu. Planowanie — zwłaszcza ciągle, iteracyjne podejście do planowania — to dążenie

do wartości. Planowanie to próba znalezienia optymalnej odpowiedzi na ogólne pytanie związane z rozwojem produktu: Co powinniśmy budować? Aby odpowiedzieć na to pytanie, zespół bierze pod uwagę możliwości, zasoby i harmonogram. Na wspomniane pytanie nie można odpowiedzieć natychmiast. Odpowiedzi należy udzielać w sposób iteracyjny i stopniowo. Na początku projektu możemy podjąć decyzję, że produkt musi zawierać określony zestaw funkcjonalności i że będzie wydany 31 sierpnia. Ale w czerwcu możemy zdecydować, że lepsza będzie nieco późniejsza data i nieco więcej funkcjonalności. Możemy także zdecydować, że najlepiej będzie, jeśli produkt zostanie wydany trochę wcześniej, ale z nieco mniejszą liczbą funkcji.

W dobrym procesie planowania jest to realizowane poprzez:

- redukowanie ryzyka,
- zmniejszanie niepewności,
- wspomaganie podejmowania lepszych decyzji,
- wzbudzanie zaufania,
- przekazywanie informacji.

Redukowanie ryzyka

Planowanie zwiększa prawdopodobieństwo sukcesu projektu dzięki zapewnieniu zrozumienia związanych z nim zagrożeń. Niektóre projekty są tak ryzykowne, że kiedy dowiemy się o zagrożeniach, możemy wstrzymać decyzję o rozpoczęciu projektu. Inne projekty mogą zawierać funkcjonalności, których zagrożenia można usunąć, jeśli działania związane z ich wyeliminowaniem zostaną podjęte odpowiednio wcześniej.

Dyskusje prowadzone podczas szacowania projektu inicjują pytania, które odkrywają potencjalne ciemne zaułki projektu. Załóżmy, że poproszono nas o oszacowanie czasu potrzebnego do zintegrowania nowego projektu z istniejącym systemem mainframe, o którym niewiele wiemy. W wyniku tego oszacowania może się okazać, że potencjalne zagrożenie stanowią funkcje integracji. Zespół projektowy może wtedy podjąć decyzję o wyeliminowaniu ryzyka poprzez zdobycie wiedzy o starym systemie. Zespół może również uwzględnić ryzyko i albo zwiększyć oszacowanie dla pracy, albo wyrazić ją w postaci przedziału — aby uwzględnić większą niepewność i ryzyko.

Zmniejszanie niepewności

W trakcie realizacji projektu zespół tworzy nowe możliwości produktu. Członkowie zespołu generują również nową wiedzę — o produkcie, używanych technologiach oraz o sobie jako zespole. Jest bardzo ważne, aby ta nowa wiedza została potwierdzona i uwzględniona w iteracyjnym procesie planowania, który jest zaprojektowany tak, aby pomóc zespołowi udoskonalić swoją wizję produktu. Najbardziej kluczowym zagrożeniem, z którym boryka się większość projektów, jest ryzyko opracowania niewłaściwego produktu. Jednak w większości projektów to ryzyko jest całkowicie ignorowane. Zwinne podejście do planowania może znacznie zmniejszyć (a w idealnej sytuacji wyeliminować) to ryzyko.

W często cytowanych badaniach CHAOS (Standish 2001) udany projekt określono jako produkt wykonany na czas, w ramach budżetu i zawierający wszystkie funkcjonalności, jakie wyspecyfikowano na początku. To niebezpieczna definicja, ponieważ nie bierze pod uwagę tego, że funkcjonalność, która dobrze wyglądała, gdy projekt się rozpoczynał, może nie być warta podejmowanych kosztów

wytwarzania, kiedy zespół rozpocznie prace nad projektem. Gdybym miał zdefiniować nieudany projekt, to jednym z moich kryteriów z pewnością byłby „projekt, w którym nikt nie wymyślił żadnego lepszego pomysłu niż to, co znajdowało się na wstępnej liście wymagań”. Zachęcam do prowadzenia projektów, w których decyzje dotyczące inwestycji, harmonogramu i funkcjonalności są poddawane okresowym weryfikacjom. Projekt, w którym wszystkie funkcjonalności są dostarczone zgodnie z pierwotnym planem, wcale nie musi być sukcesem. Użytkownicy produktu i klient prawdopodobnie nie będą zadowoleni, jeżeli pomysły na wspaniałe nowe funkcjonalności zostały odrzucone na rzecz przeciętnych tylko dlatego, że przeciętne były w pierwotnym planie.

Wspomaganie podejmowania lepszych decyzji

Oceny i plany pomagają podejmować decyzje. W jaki sposób firma może ustalić, czy określony projekt jest wart realizacji w przypadku, gdy nie wykonano oszacowania wartości i kosztów projektu? Oprócz decyzji o tym, czy warto rozpocząć projekt, czy nie, oceny pomagają nam uzyskać pewność, czy pracujemy nad możliwie najbardziej wartościowymi projektami. Załóżmy, że firma rozważa realizację dwóch projektów — zyski z jednego są szacowane na 1 milion zł, natomiast z drugiego na 2 miliony zł.

Po pierwsze, firma potrzebuje oszacowania harmonogramu i kosztów, aby ustalić, czy warto prowadzić te projekty. Czy te projekty zajmą tyle czasu, że nie zdążą trafić do użytkowników we właściwym oknie rynkowym? Czy będą kosztować więcej, niż wynoszą spodziewane zyski? Po drugie, firma potrzebuje oszacowania i planu, aby mogła zdecydować o tym, który projekt ma zostać zrealizowany. W zależności od tego, jak wysokie będą koszty, firma może być w stanie zrealizować jeden projekt, oba projekty lub żaden.

Firmy potrzebują oszacowań nie tylko po to, aby podejmować decyzje, czy rozpoczynać projekt, czy go nie rozpoczynać. Czasami profil obsady osobowej projektu jest ważniejszy od harmonogramu jego realizacji. Na przykład może się nie opłacać rozpoczynać projektu, jeśli będzie się to wiązało z czasem głównego architekta firmy, który jest już w stu procentach zajęty pracą nad innym projektem. Jeśli jednak można opracować plan, który pokazuje, jak zrealizować nowy projekt bez udziału tego architekta, być może warto rozpocząć ten projekt.

Wiele decyzji podjętych podczas planowania to kompromisy. Na przykład w każdym projekcie trzeba przyjąć kompromis między czasem poświęconym na prace rozwojowe a kosztami. Często najtańszym sposobem opracowania systemu mogłoby być zatrudnienie dobrego programisty i wyznaczenie mu dziesięciu lub dwudziestu lat na napisanie systemu i w konsekwencji pozwolenie mu na wiele lat pracy w celu mistrzowskiego poznania dziedziny, zostania ekspertem w administracji baz danych i tak dalej. Oczywiście rzadko można pozwolić sobie na czekanie dwadzieścia lat na system i dlatego do pracy nad tego rodzaju projektami angażujemy zespoły. Prace, które zespołowi złożonemu z trzydziestu programistów mogłyby zająć rok (trzydzieści osobolat), pojedynczemu programiście mogłyby zająć dwadzieścia lat. Koszty rozwoju systemu wzrastają, ale wartość wynikająca z posiadania gotowej aplikacji dziewiętnaście lat wcześniej uzasadnia ten wzrost kosztów.

Podobne kompromisy jesteśmy zmuszeni podejmować stale — między funkcjonalnością a nakładami pracy, między kosztami a czasem. Czy określona funkcjonalność jest warta opóźnienia wydania? Czy trzeba zatrudnić kolejnego programistę tak, aby określoną funkcjonalność można było dołączyć do nadchodzącego wydania? Czy należy opublikować wydanie w czerwcu, czy też poczekać do sierpnia i mieć więcej funkcjonalności? Czy należy kupić to narzędzie programistyczne? Do podejmowania takich decyzji potrzebne są oszacowania dotyczące zarówno kosztów, jak i korzyści.

Wzbudzanie zaufania

Częste i niezawodne dostarczanie obiecanych funkcjonalności buduje zaufanie między twórcami produktu a jego klientami. Wiarygodne oceny stwarzają możliwości dla niezawodnych dostaw. Klient potrzebuje ocen w celu podjęcia ważnych decyzji co do priorytetów i wymaganych kompromisów. Oszacowania pomagają również podjąć klientowi decyzję względem tego, jaką część funkcjonalności należy rozwinąć. Zamiast poświęć dwadzieścia dni na to, by uzyskać wszystkie funkcjonalności, może się okazać, że inwestycja z dziesięciu dni wysiłków przyniesie 80% korzyści. Klienci nie chcą podejmować tego rodzaju kompromisowych decyzji we wczesnej fazie projektu, jeśli oceny programistów nie wzbudzają zaufania.

Programiści zyskują na wiarygodnych szacunkach, ponieważ pozwalają im one pracować w stałym tempie. To prowadzi do poprawy jakości kodu i mniejszej liczby błędów, a to z kolei — do bardziej wiarygodnych ocen, ponieważ potrzeba mniej czasu na wykonywanie pracy trudnej do oszacowania, na przykład naprawiania błędów.

Przekazywanie informacji

Plan odzwierciedla oczekiwania i opisuje jedną z opcji, jaka jest możliwa w trakcie realizacji projektu. Plan nie gwarantuje uzyskania dokładnego zbioru funkcjonalności na ściśle określony czas i z uwzględnieniem określonych kosztów. Jednak plan komunikuje i ustanawia zbiór podstawowych oczekiwań. Bardzo często plan sprowadza się do jednej daty, a zapomina się o wszystkich założeniach i oczekiwaniach, które doprowadziły do jej wyznaczenia.

Załóżmy, że zadano mi pytanie o to, kiedy projekt zostanie ukończony. Odpowiadam, że potrzeba siedmiu miesięcy, ale nie daję żadnych wyjaśnień co do tego, jak określiłem ten czas. Trzeba sceptycznie odnosić się do takich szacunków. Bez dodatkowych informacji nie ma możliwości ustalenia, czy dobrze przeanalizowałem pytanie oraz czy moja ocena jest realna.

Załóżmy, że zamiast tego dostarczyłem plan, w którym oceniłem zakończenie projektu w czasie od siedmiu do dziewięciu miesięcy, pokazałem, jakie prace zostaną zrealizowane w pierwszych kilku miesiącach, udokumentowałem podstawowe założenia i zaprezentowałem podejście do sposobu wspólnego mierzenia postępów. W tym przypadku możesz spojrzeć na mój plan i wyciągnąć wnioski, czy można mu ufać.

Co sprawia, że plan jest dobry?

Dobry plan to taki, który interesariusze uznają za wystarczająco wiarygodny do tego, aby można go było wykorzystać jako podstawę do podejmowania decyzji. Na wczesnym etapie projektu może to oznaczać, że zgodnie z planem produkt może być wprowadzony na rynek w trzecim kwartale, a nie drugim, oraz że w przybliżeniu będzie zawierał opisany zestaw funkcji. W dalszej fazie projektu, aby plan był nadal przydatny do podjęcia, powinien być bardziej dokładny.

Załóżmy, że szacujemy i planujemy nową wersję flagowego produktu firmy. Ustaliliśmy, że wersja ta będzie gotowa do opublikowania za sześć miesięcy. Stworzyliśmy plan określający zestaw funkcji, które z pewnością będą w nowej wersji, oraz inny zestaw funkcji, które mogą być uwzględnione w nowej wersji lub nie — w zależności od postępów.

Inne osoby w firmie mogą korzystać z tego planu do podejmowania decyzji. Mogą przygotowywać materiały marketingowe, planować kampanię reklamową, przydzielać zasoby niezbędne do przeprowadzenia aktualizacji u kluczowych klientów i tak dalej. Taki plan jest przydatny — pod warunkiem że potrafi skutecznie przewidzieć, co naprawdę dzieje się w projekcie. Jeżeli prace rozwojowe zajmują dwanaście miesięcy zamiast planowanych sześciu, to oznacza, że to nie był dobry plan. Jeśli jednak realizacja projektu zajęła siedem miesięcy zamiast sześciu, to plan prawdopodobnie nadal był przydatny. Tak, plan był niedokładny i tak — być może prowadził do pewnych spóźnionych decyzji. Ale dostarczenie w siedem miesięcy projektu oszacowanego na około sześć miesięcy to zazwyczaj nie jest koniec świata i oczywiście mieści się w dopuszczalnych granicach indeksu PMI oceny budżetowej. Plan, choć niedokładny, byłby przydatny nawet z większym prawdopodobieństwem, gdybyśmy uwzględnili potrzebę jego regularnej aktualizacji podczas trwania projektu. W takim przypadku opóźnienie dostawy o miesiąc nie powinno być dla nikogo nagłym zaskoczeniem.

Co sprawia, że planowanie jest zwinne?

Ta książka dotyczy zwinnego planowania, a nie zwinnych planów. Plany są dokumentami lub cyframi. To migawki tego, jak zgodnie z naszymi oczekiwaniami projekt może się rozwijać w nieokreślonej przyszłości. Planowanie to rodzaj aktywności. Zwinne planowanie przesuwa akcent z planów na planowanie.

Zwinne planowanie równoważy wysiłki i inwestycje w planowanie, uwzględniając fakt, że plan będzie korygowany w trakcie trwania projektu. Zwinny plan to taki, który nie tylko jesteśmy skłonni zmieniać, ale także taki, który zmieniamy chętnie. Nie chcemy zmieniać planu dla samych zmian, ale chcemy go zmieniać, ponieważ zmiana oznacza, że czegoś się nauczyliśmy albo że uniknęliśmy pomyłki. Mogliśmy się dowiedzieć, że użytkownicy chcą od określonej funkcjonalności czegoś więcej lub chcą czegoś mniej albo że wygoda użytkownika jest ważniejsza, niż sądziliśmy, albo też że programowanie w nowym języku zajmuje więcej czasu, niż się spodziewaliśmy. Można oszacować finansowe konsekwencje każdej z tych zmian i jeśli są obiecujące, zmodyfikować plan i harmonogram.

Odkrywanie tych elementów wpływa na nasze plany. To oznacza, że potrzebujemy planów, które można łatwo zmieniać. Dlatego planowanie staje się ważniejsze niż plan. Wiedza i zrozumienie, jakie zyskujemy z planowania, trwają jeszcze długo po zniszczeniu jednego planu i wprowadzeniu nowego, zmodyfikowanego. Dlatego zwinny plan to taki, który łatwo zmienić.

To, że zmieniamy plan, nie oznacza, że zmieniamy daty. Możemy to zrobić lub nie. Ale jeśli dowiemy się, że się myliliśmy względem niektórych aspektów docelowego produktu i powinniśmy coś z tym zrobić, to plan będzie musiał ulec zmianie. Jest wiele sposobów wprowadzania zmian w planach bez zmieniania dat. Możemy usunąć funkcjonalność, zredukować jej zakres, wprowadzić nowe osoby do projektu i tak dalej.

Ponieważ przyznajemy, że nie możemy w pełni zdefiniować projektu na samym jego początku, to istotne jest, abyśmy na samym początku nie realizowali wszystkich przedsięwzięć związanych z planowaniem. Zwinne planowanie jest w większym lub mniejszym stopniu rozproszone w całym okresie trwania projektu. Scenę dla projektu ustawia planowanie wydań. Po nim następuje kilka etapów planowania iteracji, po czym cały proces powtarza się co najmniej kilka razy w projekcie.

Dlatego przy próbie zdefiniowania zwinnego planowania dochodzimy do wniosku, że ma ono następujące cechy:

- w większym stopniu koncentruje się na planowaniu niż na planie;
- zachęca do zmian;
- w jego wyniku powstają plany, które można łatwo zmienić;
- jest rozproszone na cały projekt.

Podsumowanie

Procesy oceny i planowania są ważne, ale ich realizacja jest trudna i stwarza okazje do popełnienia błędów. Nie możemy uwolnić się od tych działań tylko dlatego, że są trudne. Oszacowania podawane na początku projektu są znacznie mniej dokładne niż te, które są podawane później. Ta progresywna poprawa ocen jest widoczna na *stożku niepewności*.

Cel planowania polega na poszukiwaniu optymalnej odpowiedzi na ogólne pytanie dotyczące rozwoju produktu — tzn. „co budować”. Odpowiedź na to pytanie powinna uwzględniać funkcjonalności, zasoby i harmonogram. Powinna bazować na procesie planowania, którego celem jest eliminowanie zagrożeń, zmniejszanie niepewności, wspomaganie procesu podejmowania wiarygodnych decyzji, zdobywanie zaufania oraz przekazywanie informacji.

Dobry plan to taki, który jest wystarczająco wiarygodny do tego, by można go było użyć jako podstawy do podejmowania decyzji dotyczących produktu i projektu. Zwinne planowanie w większym stopniu koncentruje się na procesie planowania niż na tworzeniu planu, zachęca do zmian, a jego efektem są plany, które można łatwo zmienić. Proces zwinnego planowania jest rozproszony w całym okresie trwania projektu.

Pytania do dyskusji

1. Ten rozdział zaczął się od stwierdzenia, że zarówno nadmierne planowanie, jak i brak planów są równie niebezpieczne. Co rozumiesz przez odpowiednią ilość planowania w Twoim bieżącym projekcie?
2. Jakie są możliwe powody planowania?
3. Pomyśl o jednym lub dwóch najbardziej udanych projektach, w których uczestniczyłeś. Jaką rolę odegrało w nich planowanie?

ROZDZIAŁ 2.

Dlaczego planowanie zawodzi?

Żaden plan nie przetrwał kontaktu z wrogiem.

— feldmarszałek Helmut Graf von Moltke

W poprzednim rozdziale postawiłem tezę, że celem planowania jest iteracyjne uzyskanie optymalnej odpowiedzi na główne pytanie stawiane podczas rozwijania nowego produktu, a więc co powinno być rozwijane. To oznacza, jakie funkcjonalności powinny się znaleźć w produkcie, kiedy powinny się w nim znaleźć oraz jakie i ile zasobów potrzeba? Dowiedzieliśmy się, że planowanie wspomaga ten proces dzięki zmniejszaniu ryzyka, redukowaniu niepewności co do tego, czym powinien być produkt, wspomaganii podejmowania bardziej świadomych decyzji, wzbudzaniu zaufania oraz przekazywaniu informacji.

Niestety tradycyjne metody, za pomocą których prowadzimy planowanie w projektach, często nas zawodzą. Przy próbie udzielania odpowiedzi na pytania dotyczące ilości, harmonogramów i zasobów dla nowego produktu tradycyjne procesy planowania nie zawsze prowadzą do satysfakcjonujących odpowiedzi i zadowolających produktów. Na potwierdzenie weźmy pod uwagę, że:

- w prawie dwóch trzecich projektów szacowane koszty są znacznie przekroczone (Lederer i Prasad 1992);
- 64% procent funkcjonalności zawartych w produktach jest używanych rzadko lub nigdy (Johnson, 2002);
- w przeciętnym projekcie wyznaczony harmonogram jest przekraczany o 100% (Standish 2001).

W tym rozdziale przyjrzymy się pięciu przyczynom niepowodzenia planowania.

Planowanie dotyczy aktywności zamiast funkcjonalności

Kluczowy problem tradycyjnych sposobów planowania polega na tym, że koncentrują się one na wykonywaniu działań, a nie na dostarczaniu funkcjonalności. Działania realizowane w ramach tradycyjnie zarządzanego projektu identyfikuje wykres Gantta oraz hierarchiczna struktura prac. Właśnie w taki sposób mierzymy postępy zespołu. Pierwszy problem z planowaniem bazującym na aktywnościach polega na tym, że realizacja tych aktywności nie zapewnia klientom żadnej wartości. Jednostkami wartości dla klienta są funkcjonalności. Planowanie powinno być zatem realizowane na poziomie funkcjonalności, a nie aktywności.

Drugi problem pojawia się po utworzeniu tradycyjnego harmonogramu, podczas jego przeglądania. Gdy przeglądamy harmonogram złożony z aktywności, zwracamy uwagę na aktywności, o których zapomnieliśmy, a nie na brakujące funkcjonalności.

Kolejne problemy pojawiają się ze względu na to, że harmonogramy bazujące na aktywnościach często prowadzą do projektów, w których są przekraczane ustalone terminy. Kiedy pojawia się problem z dotrzymaniem terminów, niektóre zespoły starają się oszczędzać czas przez niewłaściwe obniżanie jakości. Inne zespoły ustanawiają strategie zarządzania zmianami mające na celu ograniczenie zmian w produkcie — nawet tych, które są bardzo wartościowe. Oto niektóre powody, dla których planowanie bazujące na aktywnościach prowadzi do przekraczania terminów:

- aktywności nie są kończone wcześniej;
- opóźnienia są przekazywane w dół harmonogramu;
- aktywności nie są niezależne.

Każdy z tych problemów opisano w poniższych punktach.

Aktywności nie są kończone wcześniej

Kilka lat temu uczestniczyłem w dwóch głównych projektach, którym musiałem poświęcić czas. Byłem programistą kilku interesujących nowych funkcjonalności dla produktu. Musiałem również przygotować niezbędną dokumentację w ramach audytu zgodności z normą ISO 9001. Programowanie było przyjemne.

Pisanie dokumentacji na potrzeby audytu zgodności nie było natomiast przyjemne. Nic dziwnego, że tak bardzo rozszerzyłem zakres prac programistycznych, że wypełniły prawie cały mój czas, a na przygotowania do audytu pozostało go bardzo mało.

Nie jestem jedynym, który tak robi. W rzeczywistości takie zachowanie jest tak powszechne, że nadano mu nazwę — jest prawo Parkinsona (1993), które brzmi:

Praca rozszerza się tak, aby wypełnić czas dostępny na jej ukończenie.

Parkinson mówi, że jeśli na realizację zadania zostanie nam przydzielony określony czas, to będziemy się starali wykonać to zadanie jak najpóźniej. Jeśli na ścianie wisi wykres Gantta, zgodnie z którym aktywność ma trwać pięć dni, to programista przypisany do tego zadania zadba o to, aby ta aktywność trwała pełnych pięć dni.

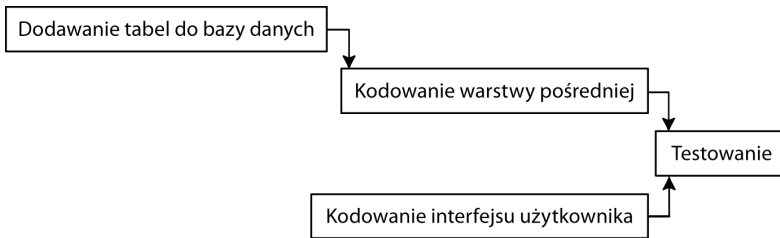
Czasami, jeśli uważa, że istnieje szansa na zakończenie zadania szybciej, dodaje kilka „wodotrysków” (jest to praktyka znana jako „pozlącanie”). Czasami również dzieli czas między przypisaną aktywność a studiowanie nowej technologii, która jego zdaniem może okazać się przydatna. Bardzo rzadko zdarza się, że programista kończy wcześniej. W wielu firmach w przypadku, gdy ktoś skończy zadanie wcześniej, jego szef ma pretensje o niedokładne oszacowania. Z kolei inni szefowie oczekują, że kolejne aktywności także zostaną zakończone wcześniej. Po co ryzykować któryś z tych scenariuszy, jeśli wystarczy trochę poszperać w internecie, gdzie można znaleźć sposób na to, by zadanie zostało zrealizowane zgodnie z harmonogramem?

Jeśli wykres Gantta pokazuje, że aktywność ma trwać pięć dni, to jest to niejawnie zezwolenie na to, że programista może poświęcić na nią właśnie do pięciu dni. Ludzka natura jest taka, że gdy pracę ukończymy przed wyznaczonym terminem, to pozostały czas wypełniamy czymś, co w naszej opinii, choć niekoniecznie w opinii innych, jest wartościowe.

Opóźnienia są przekazywane w dół harmonogramu

Ze względu na to, że tradycyjne plany bazują na aktywnościach, w dużej mierze są one ukierunkowane na współzależności między aktywnościami. Rozważmy wykres Gantta przedstawiony na rysunku 2.1, na którym pokazano cztery aktywności i ich zależności. Wcześniejszy start testowania wymaga szczęśliwego zbiegu następujących zdarzeń:

- kodowanie warstwy pośredniej zakończy się wcześniej, co zależy od tego, kiedy zostanie zakończone dodawanie tabel do bazy danych;
- kodowanie interfejsu użytkownika zakończy się wcześniej;
- tester będzie dostępny wcześniej.



RYСУNEK 2.1. Testowanie rozpocznie się później, jeśli cokolwiek pójdzie gorzej niż planowano; rozpocznie się wcześniej tylko wówczas, jeżeli wszystko pójdzie lepiej niż planowano

Kluczowy w tym przypadku jest fakt, że nawet w tej prostej sytuacji są trzy rzeczy, które muszą się wydarzyć, aby testowanie mogło rozpocząć się wcześniej. Pomimo tego dowolne z podanych poniżej zdarzeń może spowodować, że testowanie rozpocznie się później:

- kodowanie interfejsu użytkownika skończy się później;
- kodowanie warstwy pośredniej potrwa dłużej niż planowano i zakończy się później;
- kodowanie warstwy pośredniej zajmie tyle czasu, ile planowano, ale zacznie się później, ponieważ dodawanie tabel do bazy danych zakończyło się później;
- tester nie jest dostępny.

Innymi słowy, wcześniejszy start wymaga kombinacji pozytywnego przebiegu kilku zdarzeń, natomiast późniejszy start może być spowodowany niepowodzeniem zaledwie jednego zdarzenia.

Problem komplikuje się, ponieważ już uprzednio ustaliliśmy, że aktywności rzadko kończą się wcześniej. Oznacza to, że aktywności rozpoczynają się później i te opóźnienia są przekazywane w dół harmonogramu. Ze względu na to, że wcześniejsze kończenie aktywności jest rzadkie, jeszcze rzadsze jest to, aby takie zdarzenia jak testowanie pokazane na rysunku 2.1 rozpoczęły się wcześniej.

Aktywności nie są niezależne

Aktywności są uważane za niezależne, jeśli czas trwania jednej aktywności nie ma wpływu na czas trwania innej aktywności. Podczas budowy domu ilość czasu potrzebna do wykopania piwnicy nie zależy od ilości czasu potrzebnej do pomalowania ścian. Jeśli aktywności są niezależne, to późniejsze zakończenie jednej aktywności może być zrekompensowane wcześniejszym zakończeniem drugiej.

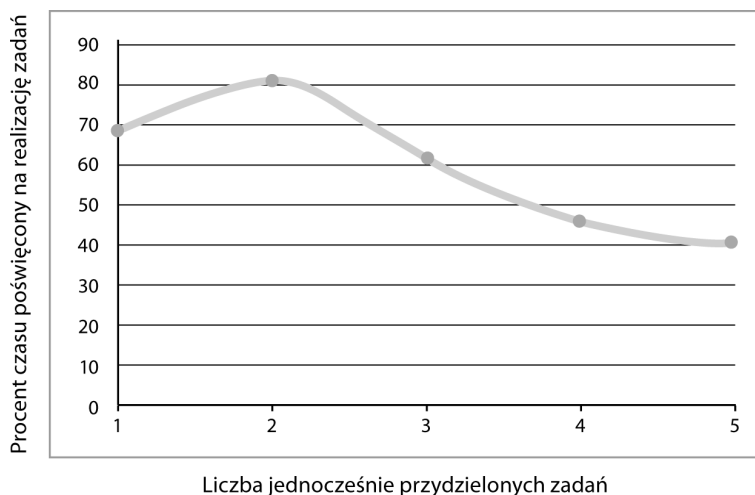
Innym przykładem niezależnych aktywności może być kilkakrotny rzut monetą. Moneta, która upadła orłem do góry w pierwszym rzucie, w kolejnym rzucie nie upadnie orłem do góry ani z mniejszym, ani z większym prawdopodobieństwem.

Czy aktywności podejmowane w ramach procesu wytwarzania oprogramowania są niezależne? Czy wahania w czasie realizacji mają tendencje do równoważenia się? Niestety nie. Wiele aktywności realizowanych w procesie wytwarzania oprogramowania nie jest od siebie niezależnych. Na przykład jeśli piszę kliencką część aplikacji i implementacja pierwszego ekranu zajmuje 50% czasu więcej niż zaplanowano, to istnieje duże prawdopodobieństwo, że implementacja każdego z pozostałych ekranów również zajmie więcej niż zaplanowano. Jeśli aktywności podejmowane w procesie wytwarzania oprogramowania nie będą niezależne, to wahania w czasie realizacji nie będą się równoważyć.

Wiele aktywności w typowym planie projektu nie jest niezależnych, ale ciągle się o tym zapomina. Gdy ktoś się spóźnia podczas realizacji pierwszych kilku podobnych elementów, to zazwyczaj słyszymy od niego: „Tak, spóźniłem się tym razem, ale następnym razem wszystko zrobię na czas”. Wynika to z przekonania, że wiedza zdobyta podczas realizacji pierwszej aktywności pozwoli na realizację podobnych aktywności szybciej, niż to przewiduje plan. Prawdziwa wiedza, jaką powinniśmy uzyskać w takiej sytuacji, jest taka, że jeśli aktywność zajmuje więcej czasu niż planowano, to wszystkie podobne aktywności również mogą zająć więcej czasu niż planowano.

Wielozadaniowość prowadzi do dalszych opóźnień

Drugim powodem, dla którego tradycyjne podejścia do planowania często zawodzą, jest wielozadaniowość definiowana jako jednoczesna praca nad kilkoma zadaniami. Wielozadaniowość powoduje olbrzymie obniżenie wydajności. Efekty wielozadaniowości badali Clark i Wheelwright (1993). Okazało się, że ilość czasu, jaką pracownik poświęca na wykonanie wartościowej pracy, gwałtownie spada, gdy pracuje się nad więcej niż dwoma zadaniami. Zostało to zilustrowane na rysunku 2.2, który bazuje na wynikach tych badań.

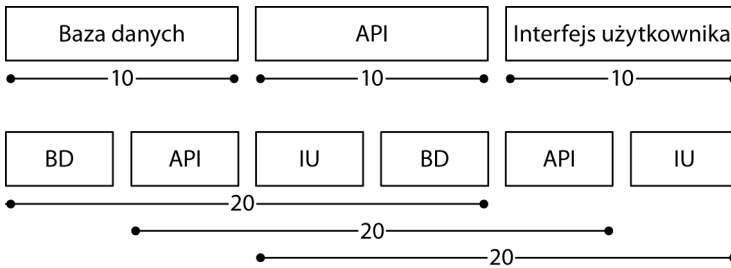


RYСУNEK 2.2. Wpływ wielozadaniowości na wydajność

Z logicznego punktu widzenia wielozadaniowość pomaga przy pracy nad dwoma zadaniami. Gdy zostaniesz zablokowany na jednym, możesz przełączyć się na inne. Logiczny jest również widoczny na rysunku 2.2 gwałtowny spadek ilości czasu poświęconego na zadania w przypadku większej liczby zadań niż dwa. Rzadko blokujemy się na więcej niż jednym zadaniu jednocześnie. Jeśli pracujemy nad trzema lub większą liczbą równoległych zadań, to czas poświęcony na przełączanie się między nimi staje się o wiele bardziej wymiernym kosztem i obciążeniem.

Wielozadaniowość często staje się problemem, gdy niektóre aktywności w projekcie kończą się później. W tym momencie kluczowe znaczenie mają zależności między aktywnościami.

Programista oczekujący na pracę innego programisty prosi o dostarczenie chociaż części pracy tak, by mógł kontynuować realizację swojego zadania. Załóżmy, że zamierzam poświęcić dziesięć dni na wprowadzenie kilku zmian w bazie danych, następnych dziesięć na implementację interfejsu API dostępu do bazy danych i kolejnych dziesięć na opracowanie interfejsu użytkownika. Pokazano to w górnej części rysunku 2.3. Twoja praca jest wstrzymana do czasu, aż otrzymasz ode mnie API. Proszisz mnie o tyle API, ile trzeba, byś mógł zacząć. Na podobnej zasadzie tester prosi mnie o implementację wystarczającej części interfejsu użytkownika, aby mógł rozpocząć automatyzację testów. Zgadzam się, a mój harmonogram przybiera postać podobną do pokazanej w dolnej części rysunku 2.3.



RYСУNEK 2.3. Wielozadaniowość przedłuża datę zakończenia pracy i powoduje, że prace w ramach procesu trwają dłużej

To często daje iluzję szybkości działania, ale — jak pokazano na rysunku 2.3 — prace nad moją bazą danych i API kończą się później niż pierwotnie zakładano. To prawie na pewno będzie mieć wpływ na dalsze aktywności podejmowane w ramach planu. Ponadto w tym przykładzie każda z pożądanych jednostek pracy jest przetwarzana przez dwadzieścia dni zamiast dziesięciu — tak jak było w przypadku, gdy prace były wykonywane szeregowo.

Na domiar złego, zgodnie z tym, co założono na rysunku 2.3, przyczyną spowolnienia nie jest częstsze przełączanie się między tymi aktywnościami. Badania Clarka i Wheelwrighta wskazują, że następuje zmniejszenie wydajności.

Wielozadaniowość staje się problemem w projektach z tradycyjnym planowaniem z dwóch głównych powodów. Po pierwsze, prace zazwyczaj są przypisywane na długo przed rozpoczęciem ich realizacji, dlatego nie jest możliwe wydajne przydzielanie pracy z góry. Przydzielanie pracy pojedynczym osobom zamiast grupom tylko pogarsza problem. Po drugie, taki stan rzeczy zachęca do koncentrowania się na osiągnięciu wysokiego poziomu wykorzystania wszystkich osób w projekcie zamiast utrzymywania pewnej rezerwy w celu obsługi naturalnej zmienności w typowych zadaniach projektowych. Obciążanie wszystkich pracowników do 100% możliwości przynosi taki sam efekt jak w sytuacji, gdy natężenie ruchu na autostradzie sięga 100% jej możliwości: nikt nie może osiągnąć żadnego postępu.

Funkcjonalności nie są opracowywane według priorytetów

Trzecim powodem, dla którego tradycyjne planowanie nie skutkuje wytworzeniem produktów o wysokiej wartości, jest to, że pracom opisywanym przez plan nie są nadawane priorytety zgodne z ich wartością dla użytkowników i klientów. Wiele tradycyjnych planów tworzy się z założeniem, że wszystkie zidentyfikowane działania zostaną zakończone. Oznacza to, że pracom zazwyczaj są nadawane priorytety i jest przydzielana kolejność realizacji według wygody programistów.

Zgodnie z tradycyjnym sposobem myślenia — w przypadku gdy wszystkie prace zostaną zakończone, klientom projektu jest wszystko jedno, w jakiej kolejności były wykonywane. Prowadzi to do sytuacji, w której zespół programistów pracuje nad funkcjonalnościami w kolejności, która dla klientów wydaje się dość przypadkowa. Następnie, wraz ze zbliżającym się końcem projektu, zespół usilnie dąży do spełnienia założeń harmonogramu poprzez usuwanie funkcjonalności. Ponieważ nie podjęto żadnych prób pracy nad funkcjonalnościami w porządku ich priorytetów, to niektóre odrzucone funkcjonalności mają dla klientów większą wartość niż te, które zostały dostarczone.

Ignorujemy niepewność

Czwartą wadą tradycyjnego podejścia do planowania jest negacja niepewności. Ignorujemy niepewność wiedzy o produkcie, zakładając, że wstępna analiza wymagań doprowadzi do pełnej i doskonałej specyfikacji produktu. Przyjmujemy, że w okresie objętym planem użytkownicy nie mogą zmienić zdania, zweryfikować swoich opinii lub wskazać nowych potrzeb.

W podobny sposób ignorujemy niepewność co do tego, w jaki sposób będziemy budować produkt, i udajemy, że możemy wyznaczyć dokładne oceny („dwa tygodnie”) do nieprecyzyjnie oszacowanych prac. Nie możemy liczyć na to, że zidentyfikujemy wszystkie rodzaje aktywności, które będą niezbędne w trakcie realizacji projektu. Pomimo to często nie przyznajemy się do tego w tworzonych planach.

Pomimo całej niepewności harmonogramy często są wyrażane w postaci jednej, bezwarunkowej daty: na przykład „wydanie nastąpi 30 czerwca”. Największa niepewność występuje w najwcześniejszych fazach projektu. Podawane oszacowania powinny odzwierciedlać naszą niepewność. Jednym ze sposobów, aby to zrobić, jest wyrażenie daty zakończenia projektu w postaci zakresu: na przykład „dostarczymy produkt między czerwcem a sierpniem”. W miarę postępów w projekcie oraz eliminowania niepewności i ryzyka można zweryfikować oceny tak, by stały się bardziej dokładne. Taki był sens stożka niepewności zaprezentowanego w rozdziale 1. „Cele planowania”.

Najlepszym sposobem na uwzględnienie niepewności są iteracje. W celu zmniejszenia niepewności dotyczącej tego, co powinien zawierać produkt, najlepiej realizować prace w krótkich iteracjach i co dwa tygodnie prezentować (lub najlepiej dostarczać) użytkownikom działające oprogramowanie. Niepewność sposobu rozwijania produktu również zmniejsza się dzięki stosowaniu iteracji. Na przykład do planu można dodawać brakujące zadania czy korygować złe oceny. Tak oto akcent jest przenoszony z planu na planowanie.

Oceny stają się zobowiązaniami

W każdą ocenę jest wbudowane prawdopodobieństwo, że w ciągu szacowanego czasu prace zostaną zakończone. Załóżmy, że zespołowi zlecono zadanie opracowania nowego, wysokiej jakości procesora tekstu. Prawdopodobieństwo ukończenia tego produktu do końca tygodnia wynosi 0%. Z kolei prawdopodobieństwo ukończenia go w ciągu dziesięciu lat wynosi 100%. Jeśli poproszę o ocenę i uzyskam odpowiedź, że produkt będzie gotowy na koniec tygodnia, to prawdopodobieństwo realności tej oceny wynosi 0%. Jeśli oszacowanie brzmi dziesięć lat, to prawdopodobieństwo jej realności wynosi 100%. Każda ocena między końcem tygodnia a okresem dziesięciu lat ma prawdopodobieństwo realności między 0% a 100% (Armour 2002).

Problemy z tradycyjnym podejściem do planowania mogą wystąpić wtedy, gdy zespół projektowy i jego interesariusze na równi traktują oceny z zobowiązaniami. Jak pokazuje Phillip Armour (2002), ocena jest realna z określonym prawdopodobieństwem, a zobowiązania nie mogą być podejmowane dla prawdopodobieństw. Zobowiązania są podejmowane względem dat. Zazwyczaj dla daty wyznaczonej dla zespołu jako zobowiązująca można przypisać prawdopodobieństwo niższe niż 100%. Przed podjęciem takiego zobowiązania zespół musi ocenić szereg czynników biznesowych i zagrożeń. Ważne jest, aby miał taką możliwość oraz żeby żadna z ocen nie była traktowana jako jawne zobowiązanie.

Podsumowanie

Biorąc pod uwagę wymienioną listę problemów z tradycyjnymi podejściami do planowania, nie należy się dziwić, że tak wiele projektów kończy się niepowodzeniem. Planowanie bazujące na aktywnościach odwraca naszą uwagę od funkcjonalności, które są prawdziwymi jednostkami wartości dla klientów. Następnie, z powodu różnych problemów, powstaje prawdopodobieństwo późniejszego dostarczenia produktu w zestawieniu z planem bazującym na aktywnościach. Mając dobre intencje, uczestnicy projektu postrzegają wielozadaniowość jako możliwy środek zaradczy, ale ostatecznie, z powodu ukrytych kosztów wielozadaniowości, projekt opóźnia się jeszcze bardziej. Kiedy zbliża się termin wynikający z harmonogramu, funkcjonalności są z niego usuwane. Ze względu na to, że funkcjonalności są opracowywane w kolejności ustalonej przez programistów według kryteriów wydajności, nie zawsze są usuwane te funkcjonalności, które mają najniższą wartość dla użytkowników.

Ignorowanie niepewności co do tego, jakich funkcjonalności użytkownicy będą ostatecznie potrzebować, może w końcu doprowadzić do zakończenia projektu zgodnie z harmonogramem, ale bez istotnych własności, które zostały zidentyfikowane już po utworzeniu planu. Na podobnej zasadzie ignorowanie niepewności dotyczącej sposobu rozwijania produktu ostatecznie prowadzi do pomijania aktywności w planie projektu.

To z kolei zwiększa prawdopodobieństwo powstania opóźnień w projekcie, usuwania funkcjonalności oraz przyjęcia nieodpowiednich kompromisów w zakresie jakości.

W wielu firmach oceny są mylone z zobowiązaniami. Natychmiast po wyznaczeniu oceny zespół jest zmuszany do podjęcia zobowiązania względem wskazanego terminu.

Pytania do dyskusji

1. Jakie problemy powodują plany, które bazują na aktywnościach, a nie na dostarczanych funkcjonalnościach?
2. Czy we współczesnych środowiskach oszacowanie jest tym samym co zobowiązanie? Jakie problemy powoduje traktowanie ocen tak jak zobowiązań? Co można zrobić, aby zmienić to błędne przekonanie?
3. W jaki sposób wielozadaniowość wpływa na Twój obecny projekt? Jak można zmniejszyć ten wpływ?

ROZDZIAŁ 3.

Zwinne podejście

*Dobry plan bezwzględnie zrealizowany dziś jest lepszy
niż idealny plan zrealizowany w przyszłym tygodniu.*

— generał George S. Patton

Choć ruch Agile rozpoczął się wiele lat temu, to oficjalnym jego początkiem było stworzenie Manifestu Agile w lutym 2001 roku (Beck i in.). Ten manifest został napisany i podpisany przez siedemnastu zwolenników „lekkich metod”, jak ich wówczas nazywano. Ich dokument zarówno dał nazwę stosowanej metodzie wytwarzania oprogramowania, jak i dostarczył listę wartościowych twierdzeń. Autorzy Manifestu Agile napisali:

- ludzie i interakcje między nimi są ważniejsze niż procesy i narzędzia;
- działające oprogramowanie jest bardziej wartościowe niż obszerna dokumentacja;
- współpraca z klientami jest ważniejsza od negocjowania kontraktów;
- reagowanie na zmiany jest ważniejsze niż trzymanie się planu.

Zespoły Agile cenią bardziej osoby i interakcje od procesów i narzędzi, ponieważ wiedzą, że dobrze funkcjonujący zespół złożony z doskonałych jednostek dysponujących przeciętnymi narzędziami zawsze będzie działał wydajniej niż zespół złożony z przeciętnych osób posługujących się doskonałymi narzędziami i procesami. Świetne oprogramowanie jest tworzone przez świetnych programistów. W naszej branży podejmowaliśmy zbyt wiele wysiłków, które przynosiły zbyt mało sukcesów, aby zdefiniować proces rozwoju, w którym ludzie są tak traktowani jak wymienne tryby w maszynie. W procesach Agile uznajemy wyjątkowe zalety (i wady) indywidualnych osób i staramy się je wykorzystać. Nie traktujemy wszystkiego tak, jakby były to jednolite elementy.

Zespoły Agile bardziej cenią działające oprogramowanie od kompleksowej dokumentacji, ponieważ dzięki temu na końcu każdej iteracji otrzymują coraz lepszą wersję produktu. To pozwala wcześniej i często zbierać opinie dotyczące produktu i procesu. W miarę rozwoju opracowywanego oprogramowania w każdej iteracji można je prezentować potencjalnym lub rzeczywistym użytkownikom. Opinie tych użytkowników trafiają z powrotem do procesu projektowania. W ten sposób można uzyskać pewność, że zespół zawsze pracuje nad najbardziej wartościowymi funkcjonalnościami oraz że te funkcjonalności spełniają oczekiwania użytkowników.

Współpracę z klientem cenimy bardziej niż negocjacje kontraktów, ponieważ zespoły Agile dążą do tego, aby wszyscy uczestnicy projektu starali się osiągnąć taki sam zbiór celów. Negocjacje kontraktu czasami powodują, że zespół programistów i klient od początku projektu stoją po przeciwnych stronach barykady. Uwielbiam grać w większość gier. Gdy moja starsza córka miała cztery lata, kupiłem jej grę kooperacyjną, ponieważ wydawało mi się, że jej się spodoba, oraz ponieważ nie miałem pojęcia, jak zabawne mogą być tego rodzaju kooperacyjne gry. W grze, którą jej kupiłem, na książniczkę zostało rzucone zaklęcie, a gracze muszą pokonać przeszkody (fosę, zamknięte drzwi i tym podobne) znajdujące się między nimi a książniczką. Gracze wykonują ruchy na zmianę tak, jak w większości gier, ale celem jest wspólne usunięcie przeszkód i ocalenie książniczki. Wszyscy gracze wygrywają albo wszyscy przegrywają. Gra w tę grę to wspaniała zabawa. Chcielibyśmy, aby zespoły pracujące nad oprogramowaniem i klienci podchodzili do projektów z taką samą wolą współpracy i poczuciem wspólnych celów. To prawda, że kontrakty często są potrzebne, ale warunki i szczegóły zawarte w kontraktach mogą mieć olbrzymi wpływ na to, czy obie strony będą koncentrowały się na współpracy, czy na rywalizacji.

Zespoły Agile bardziej cenią reagowanie na zmiany od podążania za planem, ponieważ ich ostatecznym celem jest dostarczenie jak największej wartości dla użytkowników i klientów projektu. W żadnych projektach, z wyjątkiem najprostszych, nie ma możliwości, by użytkownicy znali wszystkie szczegóły wszystkich żądanych funkcjonalności. Z całą pewnością użytkownicy będą zgłaszać nowe pomysły i prawie tak samo nieuniknione jest to, że niektóre funkcjonalności, które dziś są pożądane, jutro będą miały niższy priorytet. Dla zespołu Agile plan jest jedną z wizji przyszłości, ale możliwych jest ich wiele. Zespół powinien uwzględniać w planie nową wiedzę i doświadczenia. Możliwe, że zespół pracuje szybciej lub wolniej niż pierwotnie zakładano; być może użytkownikom jeden zbiór funkcjonalności podoba się bardziej niż oczekiwano, a nie podoba się inna funkcjonalność, która początkowo była uznawana za kluczową.

Mając na uwadze cztery stwierdzenia dotyczące wartości z Manifestu Agile, w tym rozdziale przyjrzymy się temu, co to znaczy stosować zwinne podejście do projektu, a także co to znaczy mieć zwinne podejście do oceny i planowania.

Zwinne podejście do projektów

Pamiętając o czterech głównych wartościach Agile, możemy zwrócić uwagę na to, jak zespół Agile wygląda w praktyce. Stosowanie czterech wymienionych wartości łącznie prowadzi do procesów wytwarzania oprogramowania, które są w wysokim stopniu iteracyjne i przyrostowe. W wyniku ich stosowania na koniec każdej iteracji otrzymujemy zakodowane i przetestowane oprogramowanie. W poniższych punktach opisano niektóre ważniejsze cechy pracy zespołów Agile. Zespół Agile:

- działa jako jeden kolektyw;
- pracuje w krótkich iteracjach;
- w każdej iteracji coś dostarcza;
- koncentruje się na priorytetach biznesowych;
- sprawdza i doskonali swoją pracę.

Zespół Agile działa jako jeden kolektyw

Dla powodzenia projektu kluczowe znaczenie ma to, aby wszyscy uczestnicy projektu postrzegali siebie jako jeden zespół, który realizuje wspólny cel. W projekcie Agile nie ma miejsca na mentalność typu „zrobiłem swoje i reszta mnie nie interesuje”. Analitycy nie rzucają projektantom opracowywanych wymagań. Projektanci i architekci nie rzucają projektów przez ścianę programistom, a programiści nie rzucają przez ścianę w połowie przetestowanego kodu testerom. Skuteczny zespół Agile musi prezentować myślenie w rodzaju „uczestniczymy w tym razem”. Chociaż zespół Agile musi pracować razem, jako jedna drużyna, to można w nim wyróżnić kilka konkretnych ról. Warto zidentyfikować i objaśnić te role, które mają znaczenie dla zwinnego szacowania i planowania.

Pierwsza rola to *właściciel produktu* (ang. *product owner*). Do najważniejszych obowiązków właściciela produktu należy dbanie o to, aby wszyscy członkowie zespołu dążyli do wspólnej wizji projektu, ustalanie priorytetów, tak aby prace zawsze dotyczyły funkcjonalności o największej wartości, oraz podejmowanie decyzji prowadzących do wysokiej stopy zwrotu z inwestycji w projekt. W rozwoju komercyjnego oprogramowania właściciel produktu często jest osobą z działu marketingu lub przedstawicielem działu zarządzania produktami w firmie. W przypadku opracowywania oprogramowania do użytku wewnętrznego właściciel produktu może być użytkownikiem, menedżerem użytkowników, analitykiem lub osobą finansującą projekt.

Druga rola to *klient*. Klient jest osobą, która podjęła decyzję o finansowaniu projektu lub zakupie oprogramowania. W projekcie rozwoju oprogramowania do użytku wewnętrznego klient zazwyczaj jest przedstawicielem innego zespołu lub działu. W takich projektach role właściciela produktu i klienta są często połączone. W przypadku produktu rozprowadzanego komercyjnie klientem jest osoba, która kupuje oprogramowanie. W każdym przypadku klient może, lecz nie musi być *użytkownikiem* oprogramowania, co oczywiście jest kolejną, ważną rolą.

Inną rolę wartą podkreślenia jest *deweloper* (ang. *developer*). Określenia *deweloper* używam w odniesieniu do wszystkich osób uczestniczących w procesie rozwoju oprogramowania. Obejmuje to programistów, testerów, analityków, inżynierów bazy danych, specjalistów w dziedzinie użyteczności (ang. *usability*), dokumentalistów, architektów, projektantów i tak dalej. Jeśli korzysta się z tej definicji, jako deweloper w wielu projektach może być traktowany nawet właściciel produktu.

Ostatnia rola to *menedżer projektu* (ang. *project manager*). Jak opisuje Highsmith (2004a), rola menedżera projektu w projektach Agile zmieniła się. Menedżerowie projektów Agile koncentrują się bardziej na roli liderów niż specjalistów w dziedzinie zarządzania. W niektórych projektach Agile osoba pełniąca rolę menedżera projektu pełni również inną rolę — często pracuje jako deweloper, ale od czasu do czasu także jako właściciel produktu.

Zespół Agile pracuje w krótkich iteracjach

W projekcie Agile nie ma wyraźnego podziału na etapy — nie istnieje etap opracowywania wstępnych wymagań, po którym następuje faza analizy, projektu architektury i tak dalej. W zależności od konkretnego zwinnego procesu, który wybierzemy lub ustalimy, na początku projektu możemy przeprowadzić krótką fazę projektowania, modelowania lub inną. Ale kiedy projekt rozpocznie się na poważnie, to wszystkie prace (analiza, projektowanie, kodowanie, testowanie i tym podobne) w każdej iteracji będą odbywać się równolegle.

Czas iteracji jest ściśle określony w *oknach czasowych*. To oznacza, że iteracje kończą się na czas nawet wtedy, gdy decydujemy się na usunięcie funkcjonalności. Okna czasowe iteracji często są bardzo krótkie. Większość zespołów Agile pracuje w iteracjach trwających od dwóch do czterech tygodni, choć niektóre zespoły wydłużają czas ich trwania nawet do trzech miesięcy. Większość zespołów ustala względnie stałą długość iteracji. Niektóre jednak wybierają odpowiedni czas trwania na początku każdej iteracji.

Zespół Agile w każdej iteracji coś dostarcza

Ważniejszy od wybierania przez zespół konkretnej długości iteracji jest fakt, że podczas iteracji następuje przekształcenie jednego lub kilku nieprecyzyjnie określonych twierdzeń w specyfikacji wymagań w zakodowane, przetestowane i potencjalnie gotowe do dostarczenia oprogramowanie. Oczywiście wiele zespołów nie dostarcza wyników każdej iteracji użytkownikom. Celem jest jednak to, że mogłyby to zrobić. To oznacza, że w każdej iteracji zespoły robią postępy poprzez dodawanie jednej bądź kilku małych funkcjonalności, ale każda dodana funkcjonalność jest kodowana, testowana i ma jakość pozwalającą na wdrożenie.

Kluczowe znaczenie ma to, aby na końcu każdej iteracji produkt został doprowadzony do stanu pozwalającego na dostarczenie go użytkownikom. W praktyce nie wynika z tego, że zespół musi zrobić absolutnie wszystko, co jest konieczne do opublikowania oprogramowania, ponieważ często się zdarza, że nie w każdej iteracji następuje publikacja. Na przykład jeden z zespołów, z którym pracowałem, przed opublikowaniem swojego produktu obejmującego zarówno sprzęt, jak i oprogramowanie potrzebował dwóch miesięcy testów średniego czasu między awariami (ang. *mean time between failure* — MTBF). Nie można było skrócić tego dwumiesięcznego okresu ze względu na wymogi kontraktu z klientem. Taka ilość czasu jest często konieczna do sprawdzenia, czy nie nastąpiły awarie sprzętu. Zespół pracował w czterotygodniowych iteracjach i pomimo uruchamiania dwumiesięcznych testów MTBF na koniec każdej iteracji jego produkt był w stanie nadającym się do wydania.

Ponieważ pojedyncza iteracja zwykle nie zapewnia wystarczająco dużo czasu na zrealizowanie tylu nowych funkcjonalności, aby zaspokoić potrzeby lub życzenia użytkownika, wprowadzono szersze pojęcie *wydania* (ang. *release*). Wydanie obejmuje jedną lub więcej (zwykle więcej) iteracji, które bazują na sobie i mają na celu zrealizowanie kompletnego zbioru powiązanych ze sobą funkcjonalności. Chociaż iteracje zazwyczaj trwają od dwóch do czterech tygodni, wydanie zwykle trwa od dwóch do sześciu miesięcy. Na przykład w systemie zarządzania inwestycjami jedno wydanie może obejmować wszystkie funkcjonalności związane z kupowaniem i sprzedażą funduszy powierniczych i funduszy kapitałowych. Realizacja tych funkcjonalności może zająć sześć dwutygodniowych iteracji (około trzech miesięcy). W kolejnym wydaniu mogą zostać dodane funkcjonalności związane z obrotem akcjami i obligacjami, które mogą zająć dodatkowe dwutygodniowe iteracje. Wydania mogą odbywać się z różną częstotliwością. Opracowanie pierwszego wydania może zająć sześć miesięcy. Kolejne wydanie może nastąpić trzy miesiące później i tak dalej.

Zespół Agile koncentruje się na priorytetach biznesowych

Zespół Agile wykazuje zaangażowanie w priorytety biznesowe na dwa sposoby. Przede wszystkim dostarcza funkcjonalności w kolejności określonej przez właściciela produktu, od którego oczekuje się ustalenia priorytetów i połączenia funkcjonalności w wydania w taki sposób, aby zoptymalizować zwrot z inwestycji w projekt dla firmy. Aby osiągnąć ten cel, tworzony jest plan wydań uporządkowany

według priorytetów. Podczas jego tworzenia uwzględniane są możliwości zespołu oraz lista pożądanych nowych funkcjonalności. Aby właściciel produktu mógł uzyskać maksymalną elastyczność w ustalaniu priorytetów, tworzone funkcjonalności muszą być pisane w taki sposób, aby występowało między nimi jak najmniej technicznych zależności. Jeśli wybór jednej funkcjonalności wymaga wcześniejszego zrealizowania trzech innych, to właściciel produktu będzie miał trudności z ustaleniem priorytetów funkcjonalności w taki sposób, aby powstał sensowny plan wydań. Zespół raczej nie osiągnie celu w postaci całkowitego braku zależności, jednak utrzymanie zależności na minimalnym poziomie często jest zupełnie realne.

Poza tym zespoły Agile koncentrują się na realizowaniu i dostarczaniu funkcjonalności wartościowych dla użytkowników, a nie na wykonywaniu odizolowanych zadań (które ostatecznie łączą się w funkcjonalność przynoszącą wartość dla użytkownika). Jednym z najskuteczniejszych sposobów osiągnięcia tego celu jest wykorzystywanie historyjek użytkownika — nieskomplikowanej techniki wyrażania wymagań dla oprogramowania (Cohn 2004). *Historyjka użytkownika* to krótki opis funkcjonalności z punktu widzenia użytkownika lub klienta systemu. Historyjki użytkownika mają swobodną formę. Nie istnieje obowiązkowa składnia. Jednak ogólnie rzecz biorąc, warto zadbać o to, aby historyjki użytkownika przyjęły następującą ogólną formę: „Jako <typ użytkownika> chcę <możliwość>, tak aby uzyskać <wartość dla biznesu>”. W przypadku tego przykładowego szablonu historyjka użytkownika może mieć następujące brzmienie: „Jako nabywca książki chcę mieć możliwość znalezienia książki po numerze ISBN tak, aby móc szybko znaleźć odpowiednią książkę”.

Historyjki użytkownika są nieskomplikowane, ponieważ prace wymagane do ich zebrania i udokumentowania nie są wykonywane wszystkie jednocześnie. Zamiast pisać obszerne specyfikacje wymagań, zespoły Agile odkryły, że lepsze będzie podejście do opracowywania wymagań typu „dokładnie na czas”. Zazwyczaj proces powstawania historyjki użytkownika zaczyna się od ręcznego zapisania historyjki na karteczce z notesu albo — w przypadku dużego zespołu lub zespołu rozproszonego geograficznie — napisania jej na komputerze. Karta historyjki to jednak dopiero początek. Tworzeniu każdej historyjki użytkownika towarzyszy tyle rozmów między deweloperami a właścicielem produktu, ile jest potrzebne. Rozmowy te odbywają się tak często, jak to konieczne. Uczestniczą w nich wszystkie niezbędne osoby. Stosowanie podejścia do wymagań bazującego na historyjkach użytkowników nie wyklucza istnienia pisemnej dokumentacji. Jednak akcent wyraźnie przesuwa się z komunikacji pisemnej na werbalną.

Zespół Agile sprawdza i doskonali swoją pracę

Plan stworzony na początku dowolnego projektu nie jest gwarancją tego, co się wydarzy. Jest to po prostu próba przewidzenia efektów w określonym punkcie w czasie. Plan może zdezaktualizować się z powodu wielu czynników — członkowie projektu przychodzą i odchodzą, technologie sprawdzają się lepiej lub gorzej niż oczekiwano, użytkownicy zmieniają zdanie, konkurencja zmusza nas do reakcji innej, szybszej i tak dalej. Zespoły Agile postrzegają każdą taką zmianę zarówno jako okazję, jak i potrzebę zaktualizowania planu tak, aby lepiej odzwierciedlał rzeczywistość bieżącej sytuacji.

Na początku każdej nowej iteracji zespół Agile wykorzystuje całą nową wiedzę zdobytą w poprzedniej iteracji i odpowiednio się dostosowuje do warunków. Jeśli zespół dowiedział się czegoś, co może mieć wpływ na dokładność lub wartość planu, to wprowadza odpowiednie korekty. Na dokładność planu może mieć wpływ odkrycie, że zespół źle ocenił tempo swojej pracy — tzn. w rzeczywistości pracuje szybciej lub wolniej. Mogło się również okazać, że określony rodzaj pracy jest bardziej czasochłonny niż wcześniej przypuszczano.

Na wartość planu może wpłynąć zdobycie przez właściciela produktu wiedzy na temat funkcjonalności pożądaných przez potencjalnych użytkowników. Może się zdarzyć, że na podstawie obserwacji oprogramowania z wcześniejszej iteracji właściciel produktu dowiedział się, że użytkownicy chcieliby jedną z funkcjonalności rozwinąć bardziej oraz że inna funkcjonalność nie podoba im się tak, jak wcześniej sądzono. W takim przypadku zespół może poprawić wartość planu przez przesunięcie do wydania więcej pożądaných funkcjonalności kosztem funkcjonalności postrzeganych jako mniej cenne.

Nie oznacza to, że zespoły Agile gwałtownie zmieniają ustalone priorytety w wyniku obserwacji bieżącej sytuacji. Priorytety są stosunkowo stabilne między kolejnymi iteracjami. Jednak możliwość zmiany priorytetów między iteracjami istotnie przyczynia się do maksymalizacji zwrotu z inwestycji w projekt.

Zwinne podejście do planowania

Szacowanie i planowanie rozwoju nowego produktu jest żmudnym zadaniem, które dodatkowo utrudnia niewłaściwe zrozumienie istoty projektów. Macomber (2004) twierdzi, że nie powinniśmy postrzegać projektu wyłącznie jako realizacji szeregu etapów. Zamiast tego należy raczej postrzegać projekt jako proces, w wyniku którego szybko i wiarygodnie jest generowany strumień nowych przydatnych możliwości i wiedzy. Nowe możliwości są dostarczane w produkcie. Nowa wiedza jest używana do tego, aby produkt stał się możliwie jak najlepszy.

W projekcie Agile ten strumień nowych możliwości i wiedzy wykorzystujemy jako siłę napędową do wykonywania bieżącej pracy. Nowa wiedza wygenerowana przez projekt może dotyczyć zarówno produktu, jak i projektu. Nowa *wiedza o produkcie* pomaga nam dowiedzieć się więcej o tym, czym powinien być produkt. Nowa *wiedza o projekcie* jest informacją o zespole, stosowanych technologiach czy zagrożeniach.

Często nie uznajemy tej wiedzy i jej nie planujemy. Nieuwzględnianie nowej wiedzy w planie prowadzi do budowania planów z założeniem, że wiemy wszystko, co jest konieczne do stworzenia dokładnego planu. W świecie wytwarzania oprogramowania taka sytuacja zdarza się rzadko, jeśli w ogóle kiedykolwiek się zdarza. Ward Cunningham powiedział: *it's more planning what you want to learn, not what it will be in the end*¹ (Van Schooenderwoert 2004).

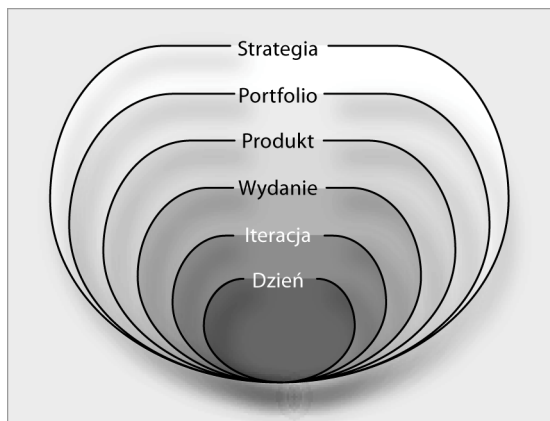
Często porównuję tradycyjne spojrzenie na projekt do biegu na dystansie dziesięciu kilometrów. Wiesz dokładnie, jak jest daleko do mety, a Twoim celem jest dotarcie do niej możliwie jak najszybciej. W projekcie Agile nie wiemy dokładnie, gdzie jest meta, ale często wiemy, że musimy do niej dotrzeć albo jak najbardziej się do niej zbliżyć przed określoną datą. Lepszym porównaniem dla projektu Agile jest wyścig trwający określony czas niż wyścig na ustalonym dystansie dziesięciu kilometrów: przebiegnij tyle, ile zdołasz w ciągu sześćdziesięciu minut. W ten sposób zespół Agile wie, kiedy skończy, ale nie wie, co do tego czasu zdoła dostarczyć. Kiedy uświadomimy sobie zarówno to, że efekt końcowy jest w pewnym sensie nieznany, jak i to, że nie można go z góry poznać, to planowanie staje się procesem ustalania i weryfikowania celów, które prowadzą do celu długoterminowego.

¹ Dosł. Interesuje nas bardziej nauczenie się planowania niż to, czym to [produkt] ostatecznie będzie — *przyp. tłum.*

Wiele poziomów planowania

Podczas ustalania i weryfikowania celów należy pamiętać, że nie jesteśmy w stanie zobaczyć tego, co jest za horyzontem, oraz że dokładność planu gwałtownie spada, im bardziej staramy się planować poza to, co możemy zobaczyć. Wyobraź sobie, że stoisz na małej łodzi, a Twoje oczy są na poziomie trzech metrów nad wodą. W tym przypadku odległość do horyzontu wynosi nieco ponad sześć kilometrów². Jeśli planujesz trzydziestokilometrową podróż, to powinieneś planować ją, patrząc do przodu co najmniej pięć razy — raz na sześć kilometrów. Ponieważ nie możesz zobaczyć tego, co jest za horyzontem, musisz spoglądać na horyzont co jakiś czas i korygować swój plan.

Projekt jest zagrożony, jeśli planowanie mocno wykracza poza horyzont planisty i nie uwzględnia czasu na to, by planista podniósł głowę, spojrzął na nowy horyzont i dokonał korekt. Plan powinien być opracowywany progresywnie. Zespoły Agile osiągają ten cel poprzez planowanie dla trzech różnych horyzontów. Są nimi wydanie, iteracja i bieżący dzień. Relacje między nimi (i innymi) horyzontami planowania zostały zilustrowane na „cebuli planowania” na rysunku 3.1.



RYСУNEK 3.1. Cebula planowania. Zespoły Agile planują co najmniej trzy razy — na poziomie wydań, iteracji i dni

Większość zespołów Agile jest zainteresowana tylko trzema, wewnętrznymi poziomami cebuli planowania. W planowaniu wydania analizowane są historyjki użytkownika lub motywy, które będą opracowywane w nowej wersji produktu lub systemu. Celem planowania wydania jest udzielenie właściwej odpowiedzi na pytania dotyczące zakresu, harmonogramu i zasobów do projektu. Planowanie wydań odbywa się na początku projektu, ale nie jest to pojedyncze przedsięwzięcie. Dobry plan wydań jest aktualizowany w trakcie realizacji projektu (zwykle na początku każdej iteracji). Dzięki temu stale odzwierciedla aktualne oczekiwania co do tego, jakie funkcjonalności zostaną uwzględnione w kolejnym wydaniu.

Na następnym poziomie jest planowanie iteracji, które jest przeprowadzane na początku każdej iteracji. Na podstawie pracy wykonanej w właśnie zakończonej iteracji właściciel produktu identyfikuje

² Aby obliczyć odległość do horyzontu w kilometrach, należy pomnożyć pierwiastek kwadratowy z wysokości, na której znajdują się oczy, przez 3,57.

prace o wysokim priorytecie, które zespół powinien zrealizować w nowej iteracji. Ze względu na to, że patrzymy na bliższy horyzont w porównaniu z planowaniem wydania, komponenty planu iteracji mogą być mniejsze. Podczas planowania iteracji mówimy o zadaniach, które będą wymagane do tego, aby przekształcić żądanie funkcjonalności w działające i przetestowane oprogramowanie.

Na końcu mamy do czynienia z planowaniem dnia. Większość zespołów Agile stosuje jakąś formę codziennych spotkań *stand-up* w celu skoordynowania pracy i zsynchronizowania codziennych wysiłków. Chociaż uwzględnianie takiego planowania w sensie formalnym może wydawać się przesadą, to w czasie trwania tych spotkań zespoły z całą pewnością tworzą, oceniają i weryfikują swoje plany. Podczas codziennych spotkań zespoły ograniczają horyzont planowania do poziomu co najwyżej następnego dnia, kiedy odbywa się kolejne spotkanie. Z tego powodu koncentrują się na planowaniu i koordynacji pojedynczych działań, które prowadzą do realizacji zadań.

Dzięki planowaniu z uwzględnieniem wymienionych trzech horyzontów — wydania, iteracji i dnia — zespoły Agile koncentrują się na tym, co jest widoczne i ważne z punktu widzenia tworzonego planu.

W większości przypadków pojedyncze zespoły Agile nie koncentrują się na planowaniu produktu, portfolio i strategii (ta tematyka nie jest także poruszana w niniejszej książce). Planowanie produktu wymaga od właściciela produktu patrzenia w przyszłość poza bieżące wydanie oraz planowania ewolucji wydawanego produktu lub systemu. Planowanie portfolio zakłada dobór produktów, które będą jak najlepiej realizowały wizję ustaloną podczas planowania strategii organizacji.

Warunki satysfakcji

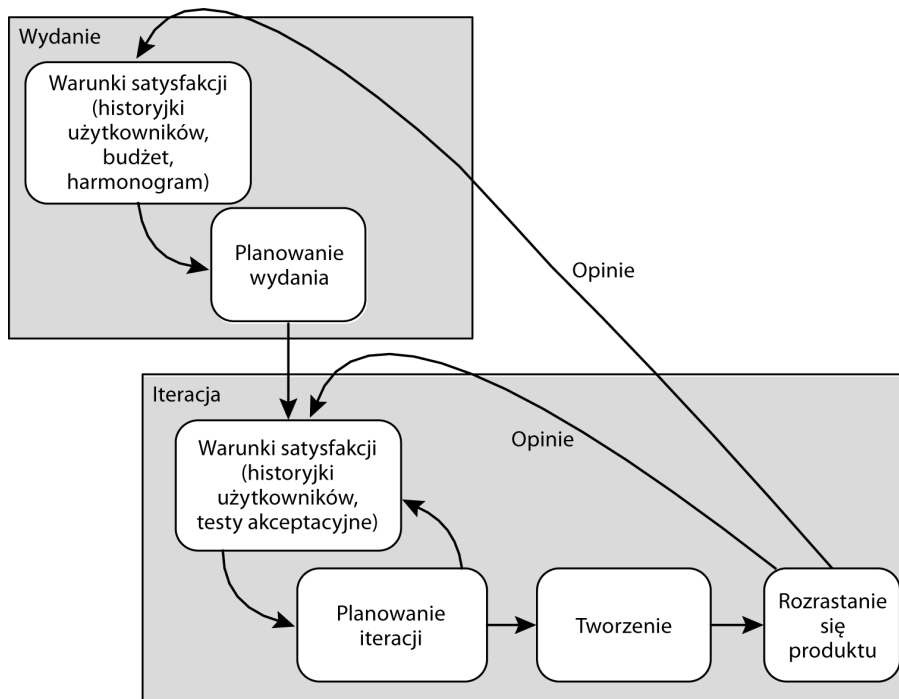
Każdy projekt jest inicjowany począwszy od ustalenia zbioru celów. Naszym bieżącym projektem może być stworzenie najlepszego na świecie edytora tekstu. Utworzenie takiego właśnie edytora tekstu zazwyczaj jest tylko jednym z celów tego projektu. Prawie na pewno istnieją dodatkowe cele związane z harmonogramem, budżetem i jakością. Te cele mogą być rozumiane jako *warunki satysfakcji* klienta lub właściciela produktu. Innymi słowy, warunki satysfakcji określają kryteria wykorzystywane do zmierzenia sukcesu projektu.

Kiedy byłem w szkole średniej i otrzymywałem zadanie domowe polegające na napisaniu wypracowania na temat jakiejś książki, na przykład *Moby Dicka*, zawsze pytałem nauczyciela, jaką objętość ma mieć praca. Odpowiadał coś w stylu „na pięć stron” i wtedy wiedziałem, jakie są jego podstawowe kryteria satysfakcji. Oczywiście istniał szereg dodatkowych, niepisanych kryteriów satysfakcji — na przykład że wypracowanie ma być starannie napisane, że ma to być moja samodzielna praca i napisana zrozumiałym językiem.

Na początku planowania wydania zespół i właściciel produktu wspólnie analizują kryteria satysfakcji właściciela produktu. Są to typowe elementy — zakres, harmonogram, budżet i jakość — choć zespoły Agile zwykle wolą traktować jakość jako niepodlegającą negocjacji. Zespół i właściciel produktu szukają sposobów spełnienia wszystkich kryteriów satysfakcji. Na przykład właściciel produktu może być tak samo zadowolony z wydania za pięć miesięcy, które będzie obejmowało implementację jednego zestawu historyjek użytkownika, jak z wydania o miesiąc później, które będzie obejmowało implementację dodatkowych historyjek użytkownika.

Czasami jednak nie można spełnić wszystkich kryteriów satysfakcji właściciela produktu. Zespół może zbudować najlepszy na świecie edytor tekstu, ale nie uda mu się go stworzyć do końca przyszłego miesiąca. Jeśli nie można znaleźć wykonalnego rozwiązania, to trzeba zmienić kryteria satysfakcji.

Z tego powodu planowanie wydań i analiza kryteriów satysfakcji właściciela produktu to zadania w wysokim stopniu iteracyjne, co pokazano na rysunku 3.2.



RYСУNEK 3.2. Warunki satysfakcji sterują zarówno planowaniem wydania, jak i iteracji

Po ustaleniu planu obejmującego w przybliżeniu kolejne trzy do sześciu miesięcy pracy plan ten jest wykorzystywany w roli danych wejściowych w procesie planowania pierwszej iteracji. Planowanie iteracji, podobnie jak planowanie wydania, rozpoczyna się od analizy kryteriów satysfakcji właściciela produktu. W przypadku iteracji kryteriami satysfakcji właściciela produktu są zazwyczaj funkcjonalności, które według niego powinny być zrealizowane w następnej kolejności, oraz pewne wysokopoziomowe testy poszczególnych funkcjonalności.

Jako przykład rozważmy serwis internetowy biura podróży, w którym zawarto następującą historijkę użytkownika: „Jako użytkownik chcę mieć możliwość anulowania rezerwacji”. Podczas omawiania tej historijki z właścicielem produktu deweloperzy dowiedzieli się, że kryteria satysfakcji dla tej historijki są następujące:

- Użytkownik, który anuluje rezerwację wcześniej niż na dwadzieścia cztery godziny przed planowaną podróżą, otrzymuje pełny zwrot pieniędzy.
- Użytkownik, który anuluje rezerwację później niż 24 godziny przed podróżą, otrzymuje zwrot kosztów pomniejszony o opłatę 25 zł.
- Kod anulowania rezerwacji jest wyświetlany w serwisie oraz wysyłany e-mailem do użytkownika.

Podobnie jak planowanie wydania, także planowanie iteracji jest procesem iteracyjnym. Właściciel produktu wraz z zespołem omawiają różne sposoby najlepszego spełnienia warunków satysfakcji dla iteracji.

Na rysunku 3.2. zaprezentowano pętle sprzężenia zwrotnego, które prowadzą od pola wynikowego powiększonego produktu do pól warunków satysfakcji na początku planowania zarówno wydania, jak i iteracji. Na podstawie doświadczeń z rozwoju produktu w czasie iteracji zespół może uzyskać wiedzę lub umiejętności, które wpływają na planowanie na jednym bądź kilku poziomach. Na podobnej zasadzie zaprezentowanie przyrostu produktu istniejącym lub prawdopodobnym użytkownikom może wygenerować nową wiedzę, która stanie się powodem wprowadzenia zmian do planów. Zespół Agile uwzględni te zmiany w swoich planach, co prowadzi do powstania produktu o wyższej wartości.

Podsumowanie

Zespoły Agile pracują wspólnie jako kolektyw, ale w ramach zespołu istnieją funkcje, które są przypisane do konkretnych osób. Pierwsza to właściciel produktu, który jest odpowiedzialny za wizję produktu oraz za ustalenie priorytetów, zgodnie z którymi zespół będzie pracować. Kolejna to klient — osoba, która płaci za projekt lub kupuje oprogramowanie, gdy tylko stanie się dostępne. Kolejne funkcje w projekcie Agile to użytkownicy, deweloperzy i menedżerowie.

Zespoły Agile pracują w krótkich oknach czasowych, w których pod koniec każdej iteracji dostarczane jest działające oprogramowanie. Funkcjonalności stworzone podczas tych iteracji są wybierane na podstawie priorytetów ustalanych dla biznesu. To gwarantuje, że najpierw są opracowywane funkcjonalności najważniejsze. Powszechnie stosowanym przez zespoły Agile sposobem na wyrażanie wymagań użytkowników są historyjki użytkowników. Zespoły Agile mają świadomość, że plany mogą szybko stać się nieaktualne. Z tego powodu w razie potrzeby dostosowują swoje plany do zmieniającej się sytuacji.

Projekty powinny być postrzegane jako przedsięwzięcia, które szybko i wiarygodnie generują strumień nowych przydatnych funkcjonalności oraz nowej wiedzy, a nie tylko jako wykonywanie ciągu czynności. Projekty generują dwa typy nowej wiedzy: wiedzę o produkcie oraz wiedzę o projekcie. Każda z nich przydaje się do udoskonalania planu produktu po to, aby osiągnąć jak największą wartość dla organizacji.

Zespoły Agile stosują planowanie na trzech poziomach: planowanie wydania, planowanie iteracji oraz planowanie codzienne. W planie wydania uwzględnia się przyszłość równą czasowi trwania wydania — zwykle od trzech do sześciu miesięcy. W planie iteracji ważny jest wyłącznie czas trwania pojedynczej iteracji — zazwyczaj od dwóch do czterech tygodni. Plan dzienny to rezultat podjęcia zobowiązania przez członka zespołu przed innymi członkami zespołu podczas codziennego spotkania *stand-up*.

Zrozumienie kryteriów satysfakcji właściciela produktu ma kluczowe znaczenie zarówno podczas planowania wydania, jak i w czasie planowania iteracji. W czasie planowania wydania cały zespół identyfikuje sposób spełnienia kryteriów satysfakcji dla wydania. Kryteria te obejmują zakres, harmonogram i zasoby. Aby to osiągnąć, może być konieczne, żeby właściciel produktu zrezygnował z jednego lub kilku kryteriów satysfakcji. Podobny proces odbywa się podczas planowania iteracji, gdy kryteriami satysfakcji są nowe funkcjonalności, które zostaną zaimplementowane, oraz wysokopoziomowe przypadki testowe, które pokazują, czy funkcjonalności zostały prawidłowo zaimplementowane.

Pytania do dyskusji

1. W jaki sposób praca w jednolitym kolektywie — jako cały zespół — wpłynęła na Twój bieżący lub poprzedni projekt?
2. Jakie są kryteria satysfakcji w Twoim obecnym projekcie? Czy wszyscy interesariusze i uczestnicy projektu zgadzają się ze wszystkimi tymi kryteriami? Jakie zagrożenia wiążą się z kontynuowaniem projektu, w którym nie ma zgody na wszystkie kryteria satysfakcji?
3. Dlaczego budżet i harmonogram zostały wyszczególnione na rysunku 3.2 jako kryteria satysfakcji do uwzględniania podczas planowania wydania, a nie podczas planowania iteracji?

CZĘŚĆ II

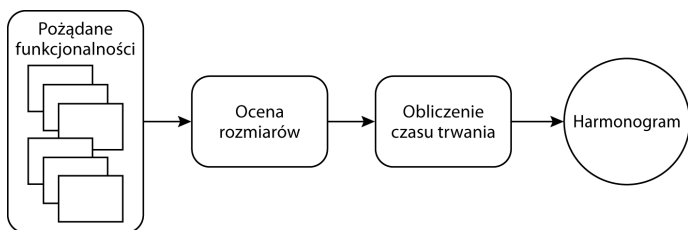
Ocena rozmiarów

Zespoły Agile oddzielają oceny rozmiarów od oceny czasu trwania. Aby zrozumieć różnicę, załóżmy, że otrzymałem zadanie przeniesienia wielkiego stosu śmieci z jednego rogu mojego podwórka do drugiego. Mógłbym spojrzeć na stos śmieci, ocenić, jakie mam narzędzia (łopatę i taczkę), i od razu ocenić czas trwania pracy na pięć godzin. Podczas ustalania tej oceny pominąłem jakiekolwiek szacowanie rozmiarów i przeszedłem od razu do oceny czasu trwania.

Załóżmy, że zamiast postępować w taki sposób, spojrzałem na stos śmieci i oceniłem jego rozmiar. Na podstawie wymiarów oszacowałem, że stos zawiera około 10 metrów sześciennych śmieci. To jest moja ocena rozmiarów projektu. Ale sama ocena rozmiarów w tym przypadku na nic się nie przyda. Chcemy wiedzieć, jak długo potrwa przenoszenie śmieci. Musimy przekształcić ocenę rozmiarów (10 metrów sześciennych) na ocenę czasu trwania.

Zgodnie z etykietą wiem, że moja taczka ma pojemność 0,2 metra sześciennego. Dzieląc 10 metrów sześciennych przez 0,2, oceniłem, że przeniesienie śmieci zajmie mi 50 kursów taczki. Oszacowałem, że każdy kurs potrwa 3 minuty potrzebne na załadowanie taczki, 2 minuty na przewiezienie taczki na drugą stronę podwórka i zrzucenie śmieci oraz 1 minutę na powrót z pustą taczką. Całkowity czas kursu wyniesie 6 minut. Ze względu na to, że muszę wykonać 50 kursów, oszacowałem, że praca potrwa 300 minut, czyli 5 godzin.

W przypadku projektu oprogramowania proces szacowania czasu trwania pokazano na rysunku II.1.



RYСУNEK II.1. Ocena czasu trwania projektu rozpoczyna się od oceny jego rozmiarów

W tej części książki najpierw przyjrzymy się dwóm miarom rozmiaru: punktom (ang. *story points*) oraz czasowi idealnemu (ang. *ideal time*). Następnie omówimy metody oceny, po których zamieścimy porady na temat sytuacji, gdy należy powtórzyć oceny. Ta część kończy się zaprezentowaniem sugestii na temat sposobu wyboru między szacowaniem za pomocą punktacji historyjek a ocenianiem za pomocą czasu idealnego.

ROZDZIAŁ 4.

Szacowanie rozmiaru za pomocą punktów

*Jeśli buty są dobrze dopasowane, to noszę rozmiar sześć,
ale w rozmiarze siedem czuję się tak dobrze, że kupuję ósemkę.*

— Dolly Parton w filmie *Stalowe magnolie*

Założmy, że w okolicy otwarto nową restaurację i postanowiliśmy wypróbować, jak tam gotują. Na pierwsze danie możesz zamówić filiżankę albo talerz zupy. Możesz zamówić przekąskę jako całą porcję lub jako pół porcji. Możesz także zamówić małą lub dużą wodę sodową. Zapewne byłeś w wielu podobnych restauracjach i z łatwością uda Ci się poprosić o odpowiednią ilość jedzenia bez pytania o to, ile uncji mieści się w filiżance, a ile w talerzu zupy, oraz jaka jest dokładnie wielkość przekąsek. Co najwyżej możesz zadać kelnerowi pytanie w rodzaju „jak duża jest sałatka?”. Kelner najprawdopodobniej odpowie, ilustrując dłońmi rozmiar, o który zapytałeś. W takich sytuacjach zamówienie bazuje na rozmiarze względnym, a nie dokładnym. Mówisz: „poproszę dużą porcję” albo „zamawiam małą porcję”. W zamówieniu nie podajesz dokładnych rozmiarów, na przykład „proszę 330 mililitrów wody sodowej, 200 gramów lasanii i 150 gramów chleba”.

W podobny sposób można szacować historyjki użytkowników lub funkcjonalności w projekcie Agile. Kiedy jestem w nieznannej restauracji i zamawiam dużą wodę sodową, to nie wiem dokładnie, ile mililitrów wody otrzymam. Wiem jedynie tyle, że duża woda sodowa to więcej niż mała lub średnia oraz mniej niż porcja XL. Wiem także z doświadczenia, że gdy chciało mi się pić, tak jak teraz, to duża woda sodowa w innych restauracjach była odpowiednim rozmiarem napoju. Na szczęście to cała wiedza, jakiej potrzebuję. W projekcie tworzenia oprogramowania jest nawet łatwiej: wystarczy wiedzieć to, czy określona historyjka użytkownika lub funkcjonalność jest większa czy mniejsza od innych historyjek i funkcjonalności.

Punktacja historyjek jest względna

Punkty to jednostki miary do wyrażania ogólnego rozmiaru historyjki użytkownika, funkcjonalności lub innej jednostki pracy. Kiedy szacujemy rozmiar projektu za pomocą punktów, do każdego elementu przypisujemy wartość punktową. Bezwzględne wartości, które przypisujemy, są bez znaczenia.

Znaczenie mają wartości względne. Historyjka, której przypiszemy dwa punkty, powinna być dwa razy większa od historyjki, której przypiszemy jeden punkt. Powinna także mieć wielkość dwóch trzecich historyjki ocenionej na trzy punkty.

Liczba punktów przypisanych do historyjki reprezentuje jej ogólny rozmiar. Nie ma gotowej formuły na zdefiniowanie rozmiaru historyjki. Ocena punktowa historyjki wynika raczej z połączenia ilości pracy, jaką trzeba włożyć w stworzenie funkcjonalności, złożoności jej utworzenia, typowych zagrożeń i tak dalej.

Są dwa główne sposoby podejścia do oceny punktowej. Pierwsze polega na wybraniu historyjki w naszej opinii najmniejszej, nad którą będziemy pracować, i przydzieleniu jej jednego punktu. Drugie podejście polega na wybraniu historyjki, która naszym zdaniem ma przeciętne rozmiary, i nadaniu jej liczby punktów równej połowie zakresu, jakiego planujemy używać. Osobiście preferuję stosowanie dla większości historyjek oceny w zakresie od jednego do dziesięciu (wyjaśnię to w rozdziale 6. „Metody oceny”). Oznacza to, że wyszukuję historyjki o przeciętnym rozmiarze i przypisuję im pięć punktów. Po dość dowolnym przypisaniu oceny punktowej do pierwszej historyjki wszystkie kolejne historyjki oceniam przez porównywanie ich z pierwszą lub z dowolnymi innymi, które już zostały ocenione.

Najlepszym sposobem, aby zobaczyć, jak działa ta metoda, jest próba skorzystania z niej. Zamiast historyjkom spróbujmy na początek przydzielić punkty różnym rasom psów. Zdefiniujmy punkt przydzielony rasie psa jako reprezentację wysokości psa w kłębie. Spróbujmy przypisać punkty każdej z poniższych ras:

- labrador retriever,
- terrier,
- wielki Duńczyk,
- pudel,
- jamnik,
- owczarek niemiecki,
- bernardyn,
- buldog.

Zanim zaczniesz czytać dalej, poświęć trochę czasu na przemyślenie tego, ile punktów przypisałyś do każdej rasy. Jeśli to zrobisz, opis zamieszczony poniżej powinien być dla Ciebie o wiele bardziej zrozumiały.

Swoje oceny zestawilem w tabeli 4.1. Ustaliłem te wartości, zaczynając od labradora retrievera. Ta rasa, według mnie, ma średni rozmiar, dlatego przypisałem jej pięć punktów. Wielkie Duńczyki wydają się dwa razy większe, więc przydzieliłem im dziesięć punktów. Bernardyny wydają się trochę mniej niż dwa razy wyższe, więc dałem im dziewięć punktów. Jamnik dla mnie to najniższa z możliwych ras, dlatego przydzieliłem jej jeden punkt. Buldogi są niskie, stąd moja decyzja o przyznaniu im trzech punktów. Gdybym jednak miał przydzielać punkty psim rasom na podstawie wagi, to buldog otrzymałby wyższą ocenę.

W projekcie Agile nie należy do rzadkości sytuacja, gdy iterację zaczyna się pomimo nie do końca określonych wymagań. Szczegóły są dostrajane w trakcie jej trwania. Niemniej jednak ocenę trzeba przypisać wszystkim historyjkom — nawet takim, które nie są w pełni zdefiniowane. Widzieliśmy to już

TABELA 4.1. Jedna z możliwych punktacji psich ras

Rasa	Punktacja
Labrador retriever	5
Terrier	3
Wielki Duńczyk	10
Pudel	3
Jamnik	1
Owczarek niemiecki	5
Bernardyn	9
Buldog	3

wcześniej, gdy przypisywaliśmy punkty pudlowi i terrierowi. Bez dodatkowych szczegółów dokonanie oceny punktowej pudla i terriera byłoby trudne. Są pudle toy, miniaturowe i standardowe — każda z tych ras charakteryzuje się inną wysokością. Także terrier to grupa ponad dwudziestu ras. Niektóre terriery (West Highland, Norwich, Norfolk) mają wysokość mniejszą niż 30 centymetrów, natomiast wysokość innych (airedale) sięga 60 centymetrów.

Jeśli mamy ocenić luźno zdefiniowaną historijkę użytkownika (lub rasę psa), możemy przyjąć pewne założenia, coś odgadnąć i dokonać oceny. W tabeli 4.1 przyjąłem określone założenia dla terriera i pudla i przypisałem każdej z tych ras po trzy punkty. Uznałem, że nawet największe z nich są mniejsze niż labradory retrievery oraz że sensowną oceną małych terrierów i pudli byłby jeden lub dwa punkty, więc średnia ocena trzy wydawała się rozsądna.

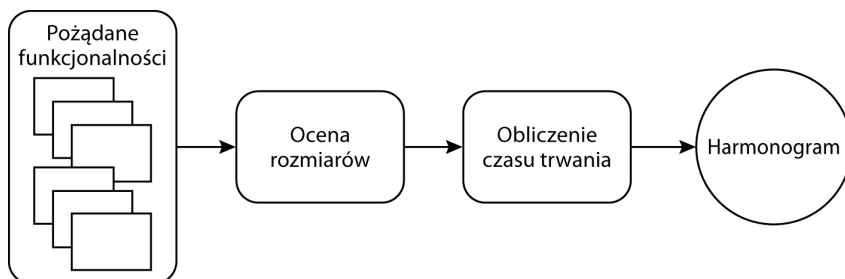
Tempo

Aby zrozumieć sposób szacowania za pomocą bezwymiarowych punktów historyjek, trzeba wprowadzić nowe pojęcie: *tempo* (ang. *velocity*). *Tempo* jest miarą szybkości postępów zespołu w każdej iteracji. Oblicza się je przez zsumowanie liczby punktów przypisanych wszystkim historyjkom użytkownika, które zespół zrealizował podczas iteracji¹. Jeśli zespół zrealizuje trzy historyjki po pięć punktów każda, to pracuje w tempie równym piętnaście. Jeśli zespół zaimplementuje dwie historyjki po pięć punktów, to jego tempo wynosi dziesięć.

Jeśli zespołowi udało się zdobyć dziesięć punktów w poprzedniej iteracji, to istnieją przesłanki do tego, by przypuszczać, że w bieżącej iteracji także uda mu się zdobyć dziesięć punktów. Ze względu na to, że punkty są ocenami względnymi, będzie to prawda niezależnie od tego, czy zespół zrealizuje dwie historyjki ocenione po pięć punktów, czy pięć historyjek ocenionych po dwa punkty.

¹ Na razie taka definicja wystarczy. W następnym rozdziale wprowadzimy pojęcie szacowania z wykorzystaniem czasu idealnego jako alternatywy punktowania historyjek. Zespół, który szacuje pracę, posługując się czasem idealnym, oblicza tempo jako sumę ocen czasów idealnych dla zrealizowanych historyjek użytkownika.

We wstępie do tej części książki zaprezentowano model (rysunek 4.1), za pomocą którego pokazano, jak przekształcić ocenę rozmiarów na ocenę czasu trwania oraz harmonogram. Teraz możemy zobaczyć, jak pasują do tego modelu ocena punktowa historyjek i tempo.



RYSUNEK 4.1. Ocena czasu trwania projektu rozpoczyna się od oceny jego rozmiaru

Jeśli zsumujemy punkty przypisane do historyjek dla wszystkich pożądanych funkcjonalności, uzyskamy ocenę całkowitego rozmiaru projektu. Jeżeli znamy tempo zespołu, to możemy podzielić rozmiar przez szybkość, by uzyskać szacowaną liczbę iteracji. Na podstawie czasu trwania projektu, poprzez dopasowanie czasu trwania do kalendarza, możemy uzyskać harmonogram.

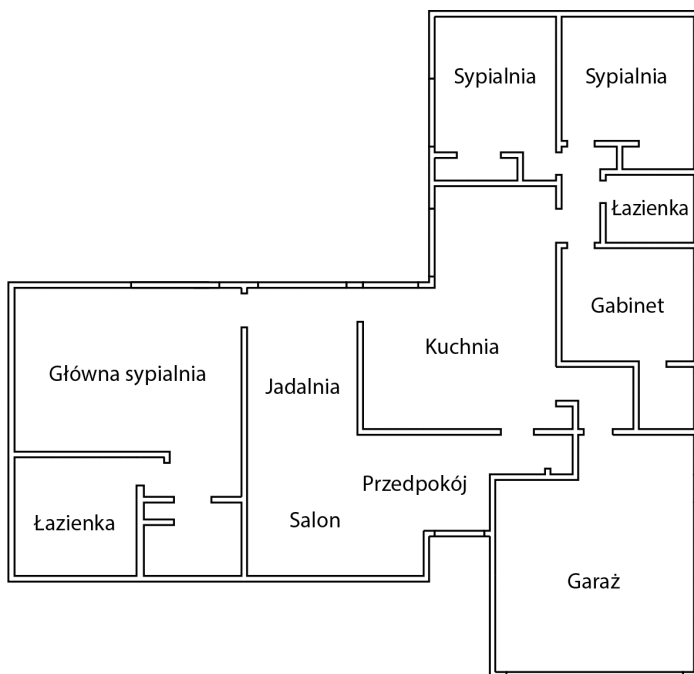
Podstawowa zasada zwinnych ocen i planowania polega na tym, że oceniamy rozmiar i na jego podstawie wyznaczamy czas trwania. Załóżmy, że oceniliśmy wszystkie historyjki użytkownika i otrzymaliśmy sumę tych ocen, która wynosi 100 punktów. To jest szacowany rozmiar systemu. Załóżmy dalej, że tempo zespołu ustalone na podstawie wcześniejszych doświadczeń wynosi dziesięć punktów w ciągu dwutygodniowej iteracji oraz że tempo w tym projekcie pozostanie bez zmian. Na podstawie oceny rozmiaru i znanej wartości tempa możemy obliczyć, że projekt będzie trwał dziesięć iteracji, czyli dwadzieścia tygodni. Możemy odliczyć w kalendarzu dwadzieścia tygodni do przodu i to stanie się naszym harmonogramem.

To bardzo uproszczone wyjaśnienie planowania wydań sprawdza się w tym momencie. Pojęcie to rozszerzymy w części IV „Tworzenie harmonogramów”. Ponadto niniejszy przykład był bardzo uproszczony ze względu na to, że skorzystaliśmy z tempa zespołu z przeszłości. Nie zawsze dysponujemy takimi danymi. Czasami tempo trzeba oszacować. To trudne zadanie, ale istnieją sposoby jego realizacji — omówimy je w rozdziale 16. „Szacowanie tempa”.

Tempo pozwala skorygować błędy szacowania

Na szczęście w miarę postępów zespołu w realizacji historyjek użytkowników w ciągu kilku pierwszych iteracji można ocenić jego tempo. Siła podejścia bazującego na punktacji historyjek polega na tym, że błędy planowania same się korygują dzięki zastosowaniu pojęcia tempa. Załóżmy, że zespół ocenił projekt na 200 punktów. Początkowo członkowie zespołu uważali, że w jednej iteracji zdołają zrealizować zadania oszacowane na dwadzieścia pięć punktów, co oznacza, że projekt zakończy się w ośmiu iteracjach. Gdy jednak projekt się rozpoczął, zaobserwowano, że tempo wynosi zaledwie dwadzieścia. Bez ponownego szacowania pracy prawidłowo oceniono, że realizacja projektu zajmie dziesięć iteracji zamiast ośmiu.

Aby zobaczyć, jak to działa, założmy, że zatrudniono Cię do pomalowania nieznanego domu. Pokazano Ci plan piętra podobny do przedstawionego na rysunku 4.2 i powiedziano, że otrzymasz wszystkie materiały i narzędzia, włącznie z pędzlami, wałkami i farbami. Dla swoich własnych celów chcesz się dowiedzieć, jak długo potrwa ta praca, dlatego postanowiłeś ją oszacować. Przecież masz tylko rzut piętra i jeszcze nie możesz zobaczyć domu, a więc swoje oceny opierasz na tym, co możesz wywnioskować z rzutu piętra. Załóżmy, że wysiłek potrzebny do pomalowania każdej z mniejszych sypialni ocenileś na pięć punktów. Ocena pięć nic szczególnego nie oznacza. Wskazuje jednak, że wysiłek wymagany do pomalowania każdej sypialni będzie w przybliżeniu taki sam. Ponieważ główna sypialnia jest około dwa razy większa od pozostałych, oceniasz jej pomalowanie na dziesięć punktów.



RYSUNEK 4.2. *Ile czasu zajmie pomalowanie tego domu?*

Jeśli jednak przyjrzyj się uważniej rysunkowi 4.2, to zauważysz, że nie podano na nim wymiarów. Czy dwie sypialnie mają wymiary 2,5 m × 3 m, czy 5 m × 6 m? Nie da się tego ustalić na podstawie planu piętra. Czy oceny, których dokonaliśmy, są w tej chwili całkowicie bezużyteczne? Nie. Nasze oceny są nadal przydatne, ponieważ są to oceny względnego wysiłku wnoszonego w pomalowanie każdego pokoju. Jeśli okaże się, że mniejsze sypialnie są dwa razy większe, niż sądziłeś, to główna sypialnia będzie również dwa razy większa, niż pierwotnie myślałeś. Oceny pozostają takie same, ale ponieważ powierzchnia pokoi jest cztery razy większa niż oczekiwałeś, to postępy będą wolniejsze.

Interesującą cechą szacowania za pomocą punktowania historyjek jest to, że przy takim szacowaniu całkowicie oddzielamy szacowanie wysiłku od szacowania czasu trwania. Oczywiście pracochłonność ma związek z harmonogramem, ale dzięki ich rozdzieleniu można je szacować niezależnie od siebie. Właściwie już nie musimy szacować czasu trwania projektu, możemy go bowiem obliczyć lub wyznaczyć. Różnica jest subtelna, ale bardzo ważna.

Podsumowanie

Punkty to względna miara rozmiarów historyjek użytkowników. Historyjka użytkownika oszacowana na dziesięć punktów jest dwa razy większa, bardziej złożona i ryzykowna od historyjki ocenionej na pięć punktów. Podobnie historyjka oceniona na dziesięć punktów jest o połowę mniejsza, mniej złożona lub ryzykowna od historyjki ocenionej na dwadzieścia punktów. Liczą się względne wartości przypisane różnym historyjkom.

Tempo jest miarą szybkości osiągania postępów w każdej iteracji. Po zakończeniu każdej iteracji zespół może przyrzeć się zaimplementowanym historyjkom i obliczyć swoje tempo poprzez zsumowanie ocen punktowych wszystkich zrealizowanych historyjek.

Punkty przypisywane historyjkom są jedynie estymatami rozmiarów pracy do wykonania. Czas trwania projektu nie jest szacowany. Zamiast tego jest obliczany przez podzielenie całkowitej liczby punktów przypisanych historyjkom przez tempo zespołu.

Pytania do dyskusji

1. Jaką ocenę punktową przypisałbyś niektórym funkcjonalnościom w Twoim aktualnym projekcie?
2. Biorąc pod uwagę punktację ras psów zaprezentowaną w tym rozdziale, zastanów się: jaką ocenę przypisałbyś słoniowi, gdyby klient projektu pomylił się, prosząc o ocenę listy psów, gdy w rzeczywistości miał na myśli listę ssaków?
3. Dość łatwo można oszacować, że dwa przedmioty o bardzo podobnym rozmiarze mają taką samą ocenę punktową. W jakim zakresie, Twoim zdaniem, możliwe jest dokonywanie wiarygodnych ocen (od najmniejszego do największego przedmiotu)? Gdy przedmiot jest większy od najmniejszego pięciokrotnie? Dziesięciokrotnie? Stukrotnie? Tysięckrotnie?

ROZDZIAŁ 5.

Szacowanie w dniach idealnych

Aby osiągnąć dużo, potrzebne są dwie rzeczy: plan i niewystarczająco dużo czasu.

— Leonard Bernstein

Ile trwa mecz futbolu amerykańskiego?

Można odpowiedzieć, że mecz składa się z czterech kwart po piętnaście minut, czyli łącznie trwa sześćdziesiąt minut. Można także odpowiedzieć, że jest to gdzieś około trzech godzin. Każda odpowiedź będzie na swój sposób poprawna. Różnica polega na innym liczeniu czasu: w jednym przypadku jest to czas idealny, a w drugim — upływający. *Czas idealny* to czas trwania wydarzenia w przypadku odjęcia wszystkich działań dodatkowych. W tym sensie mecz futbolu amerykańskiego trwa sześćdziesiąt minut. Z drugiej strony *czas upływający* to odcinek czasu, który mija na zegarze (lub być może na kalendarzu). Gra w futbol amerykański wymaga około trzech godzin.

Każdy sposób pomiaru czasu trwania ma swoje przeznaczenie. W meczu futbolu amerykańskiego czas idealny jest używany przez sędziów w celu ustalenia, kiedy kończą się kwarty i kiedy kończy się gra. Oczywiście jest to niezbędna funkcja. Z drugiej strony, jeśli idziesz na mecz, a Twój małżonek pyta, jak długo Cię nie będzie, to odpowiedź, że mecz potrwa sześćdziesiąt minut, na nic Ci się nie przyda. Bliższa prawdy będzie odpowiedź, że nie będzie Cię przez trzy godziny (plus czas niezbędny na podróż).

Prawie zawsze znacznie łatwiej i dokładniej jest przewidzieć czas trwania wydarzenia w czasie idealnym niż w czasie upływającym. Przypuśćmy, że poproszono Cię o oszacowanie czasu trwania konkretnego meczu futbolu amerykańskiego w ten weekend. Jeśli zdecydujesz się podać swoją ocenę w czasie idealnym, możesz udzielić szybkiej, spontanicznej odpowiedzi: sześćdziesiąt minut. Jeśli chcesz podać bardziej dokładne oszacowanie, możesz przejrzeć kilka gier z ubiegłego roku, wykonać jakieś obliczenia i ogłosić, że na podstawie średnich historycznych mecz w najbliższy weekend potrwa 62,3 minuty.

Z drugiej strony założmy, że zdecydowałeś się wyrazić swoją ocenę w czasie upływającym. Niektóre mecze w zeszłym roku zajęły dwie i pół godziny, inne trwały ponad cztery godziny. Różnica po części wynika ze zdarzeń losowych, na przykład urazów, ale innymi składnikami wpływającymi na tę różnicę mogą być styl gry zespołów czy chociażby liczba kar. Mecze, z których są przeprowadzane transmisje telewizyjne, trwają dłużej ze względu na dodatkowe przerwy na reklamy. W zależności od miejsca rozgrywania meczów pogoda może przedłużać albo skracać czas ich trwania. Aby wyznaczyć ocenę, która będzie równie dokładna, jak oszacowanie za pomocą czasu idealnego, należałoby wziąć pod uwagę wszystkie te czynniki.

Oczywiście jeśli spojrzę na program telewizyjny, dowiem się, że mecz zaczyna się o 13.00, a kończy o 16.00. Co oni wiedzą, czego my nie wiemy? Kilka rzeczy. Po pierwsze, planują dodanie lub usunięcie kilku reklam w zależności od tego, jak szybko idzie gra. Po drugie, czas trwania większości gier wynosi w przybliżeniu trzy godziny. Jeśli mecz skończy się wcześniej, to sieć telewizyjna wyemituje dodatkowe reklamy, wywiady lub inne „wypełniacze”. Jeśli mecz potrwa dłużej, to co wtedy? Widzowie zainteresowani meczem chcieliby kontynuować oglądanie gry przez kolejne piętnaście lub trzydzieści minut. Transmisja meczu nie zostanie przerwana tylko dlatego, że w programie uwzględniono, że mecz powinien się zakończyć przed 16.00. Widzowie, których mecz nie interesuje, a którzy przełączyli się na ten kanał, aby oglądać inny program o 16.00, prawdopodobnie również poczekają, jeśli opóźnienie nie będzie trwało zbyt długo. Po wielu dziesięcioleciach transmisji meczów futbolu sieci telewizyjne przyzwyczyły nas, że nie należy oczekiwać, by w programie został określony precyzyjny czas zakończenia meczu.

Ta ostatnia różnica jest bardzo ważna. Po latach oglądania meczów większość fanów futbolu wie, że jeśli koniec futbolowego meczu zaplanowano na 16.00, to jest to tylko szacunkowa ocena. Nikogo nie dziwi, że transmisja meczu trwa do 16.15 (przekroczenie o 8%). Niektórzy są zdenerwowani lub źli, ale nikt nie jest zaskoczony. Dlaczego zatem jesteśmy zaskoczeni, gdy projekt oprogramowania szacowany na dwanaście miesięcy zajmuje trzynaście miesięcy (odpowiednik przekroczenia o 8%)? Który okres wydaje się trudniejszy do oceny?

Czas idealny a wytwarzanie oprogramowania

W projekcie oprogramowania czas idealny różni się od czasu upływającego nie ze względu na przerwy, niepełne przebiegi lub kontuzje, ale z powodu naturalnych zdarzeń, których doświadczamy każdego dnia. Codziennie oprócz wykonywania zaplanowanych działań w projekcie członek zespołu poświęca czas na udzielanie odpowiedzi na e-maile, rozmowy z działami pomocy technicznej dostawców, prowadzenie rozmów kwalifikacyjnych z kandydatami do pracy oraz uczestniczenie w spotkaniach. Oto kilka dodatkowych przyczyn, dla których czas idealny nie jest równy czasowi upływającemu:

- udzielanie wsparcia dla bieżącej wersji,
- zwolnienia lekarskie,
- spotkania,
- demonstracje,
- problemy kadrowe,
- rozmowy telefoniczne,
- projekty specjalne,
- szkolenia,
- obsługa e-maili,
- przeglądanie kodu,
- przeprowadzanie rozmów kwalifikacyjnych z kandydatami,
- przełączanie się między zadaniami,
- poprawianie błędów w aktualnych wersjach,
- kontrole menedżerów.

Przyglądając się powodom, dla których czas idealny nie jest równy czasowi upływającemu, należy ponadto wziąć pod uwagę fakt, że między przerwami menedżerowie średnio mogą pracować tylko pięć minut (Hobbs 1987). Nawet jeśli przeciętnemu deweloperowi przerywa się trzy razy rzadziej, to i tak przerwy następują co piętnaście minut.

Problemy mogą pojawiać się w sytuacji, gdy menedżer zadaje członkowi zespołu nieuniknione pytanie: „Ile czasu to zajmie?”. Na tak postawione pytanie członek zespołu odpowiada: „Pięć dni”. Menedżer odlicza pięć dni w swoim kalendarzu i zaznacza ten dzień wielkim czerwonym krzyżykiem. Jednak w rzeczywistości członek zespołu chciał powiedzieć: „Pięć dni, jeśli to byłoby wszystko, czym bym się zajmował, ale robię wiele innych rzeczy, więc prawdopodobnie dwa tygodnie”.

W projekcie wytwarzania oprogramowania lukę między czasem idealnym a upływającym zwiększa również wielozadaniowość. Trener futbolu amerykańskiego nigdy nie powie swojemu zawodnikowi: „Ponieważ nie jesteś zajęty w każdym meczu, to chciałbym, żebyś w tym samym czasie grał w priorytetowym meczu hokejowym”. Deweloper oprogramowania, któremu zleca się pracę nad wieloma zadaniami równocześnie, traci dużą część wydajności na przełączanie się między dwoma (lub większą liczbą) zadań.

W projekcie oprogramowania możemy zdecydować się na szacowanie historyjek użytkownika lub innej pracy w *dniach idealnych*. W przypadku oceny w dniach idealnych przyjmuje się następujące założenia:

- Praca nad szacowaną historyjką jest jedynym zadaniem, nad którym będziemy pracować.
- Po rozpoczęciu pracy będziemy mieć do dyspozycji wszystko, co jest potrzebne do jej wykonania.
- Nie będzie żadnych przerw.

Dni idealne jako miara rozmiaru

Podczas oceny liczby dni idealnych, jakie zajmą implementacja, testowanie i akceptacja historyjki użytkownika, nie jest konieczne branie pod uwagę wpływu „narzutów” środowiska, w którym zespół pracuje. Jeśli opracowanie konkretnego ekranu zajmuje jeden dzień idealny, to zajmie to tylko jeden dzień idealny niezależnie od tego, czy pracuję w start-upie, gdzie nie ma żadnych dodatkowych zadań, innych wymagań względem mojego czasu, czy też w firmie o rozbudowanej biurokracji. Ilość czasu, która upłynie na zegarze (lub kalendarzu) w tych dwóch środowiskach, oczywiście będzie różna. Może mi się uda osiągnąć czas zbliżony do idealnego w start-upie o znikomych narzutach. W miarę dokładania dodatkowych wymagań do mojego czasu pozostaje mniej czasu na pracę nad artefaktami projektu, a rzeczywista ilość czasu potrzebna do wykonania pracy równej jednemu dniowi idealnemu wzrasta.

Jeżeli zignorujemy czynniki dodatkowych obciążeń w środowisku określonej firmy, wówczas dni idealne będzie można potraktować jako alternatywną ocenę rozmiaru, podobną do punktacji historyjek. W takim przypadku ocenę rozmiaru wyrażoną przez liczbę dni idealnych można przekształcić na szacunkowy czas trwania w taki sam sposób, jak w przypadku punktacji historyjek — korzystając z pojęcia szybkości.

Jedna ocena, a nie wiele

Jeśli zdecydujemy się na oceny historyjek w dniach idealnych, powinniśmy przypisać jedną, zagregowaną ocenę każdej historyjce użytkownika. Niektóre zespoły wykazują tendencję do szacowania liczby idealnych dni dla każdej osoby lub grupy, która będzie pracować nad historyjką. Taki zespół może na przykład ocenić, że określona historyjka użytkownika zajmie dwa idealne dni programisty, jeden idealny dzień inżyniera bazy danych, jeden idealny dzień projektanta interakcji z użytkownikiem oraz dwa idealne dni testera. W takich przypadkach obserwowałem, że oceny dla poszczególnych ról były zapisywane na kartach historyjki markerem innego koloru lub były umieszczane na różnokolorowych samoprzylepnych karteczkach dołączanych do kart historyjek.

W zdecydowanej większości przypadków nie zalecam stosowania takich praktyk. Taki poziom koncentracji na poszczególnych rolach w zespole odwołuje się do myślenia w stylu „uczestniczymy w tym wszyscy”, pożądanego w zespołach Agile. Ponadto znacznie zwiększa ilość pracy potrzebnej do planowania wydania. Jeśli każdej historyjce użytkownika przypiszemy ocenę dla każdej roli, to w planie wydań także należałoby wziąć pod uwagę każdą z ról. To oznacza, że trzeba by również śledzić tempo i pozostałe prace dla każdej roli.

Choć rzadko jest to warte podejmowania dodatkowych wysiłków, to czasami jest konieczne. Niedawno miałem klienta, który pracował nad trzema wersjami produktu — jedną dla komputerów z systemem Mac OS, drugą dla Windowsa i trzecią dla komputerów przenośnych. W takiej sytuacji kluczowe znaczenie miało to, aby każda wersja była wydawana z dokładnie taką samą funkcjonalnością. Ponadto członkowie zespołu pracującego nad tym produktem nie mieli umiejętności pozwalających im na sprawne przełączanie się między środowiskami Mac OS, Windows i PDA. Wówczas zespół może zdecydować o oszacowaniu idealnego czasu dla każdej roli i każdej historyjki użytkownika. Należy jednak zdawać sobie sprawę z dodatkowych obciążeń administracyjnych, które będą wymagane.

Podsumowanie

Czas idealny różni się od czasu rzeczywistego, który upłynął. Czas idealny meczu futbolu amerykańskiego wynosi sześćdziesiąt minut (cztery kwarty po piętnaście minut). Zwykle jednak mijają trzy godziny lub więcej godzin, zanim zakończy się trwający sześćdziesiąt minut mecz. Przyczyną tej różnicy są oczywiście wszelkie zakłócenia, które mogą wystąpić podczas gry.

Ilość czasu, jaką zajmuje implementacja historyjki użytkownika, można łatwiej oszacować w dniach idealnych niż w dniach rzeczywistych. Ocena czasu rzeczywistego wymaga uwzględnienia wszystkich przerw, które mogą wystąpić podczas pracy nad historyjką. Jeśli dokonujemy oceny w dniach idealnych, to uwzględniamy tylko ilość czasu, jaką zajmie praca nad historyjką. Dzięki temu dni idealne można traktować jako ocenę rozmiarów, choć mniej surową od oceny za pomocą punktacji historyjek.

Przy ocenie za pomocą dni idealnych najlepiej przypisać każdej historyjce użytkownika jedną ocenę. Zamiast oszacować, że implementacja historyjki użytkownika zajmie cztery dni programisty, dwa dni testera i trzy dni właściciela produktu, lepiej zsumować te liczby i oszacować całą historyjkę na dziewięć dni idealnych.

Pytania do dyskusji

1. Jeśli dzień idealny to osiem godzin nieprzerwanej, ukierunkowanej na zadanie pracy, to ile godzin rzeczywistych w Twoim środowisku odpowiada jednemu dniowi idealnemu?
2. Co można zrobić, aby to poprawić?

ROZDZIAŁ 6.

Metody oceny

Prognozowanie jest bardzo trudne, zwłaszcza jeśli dotyczy przyszłości.

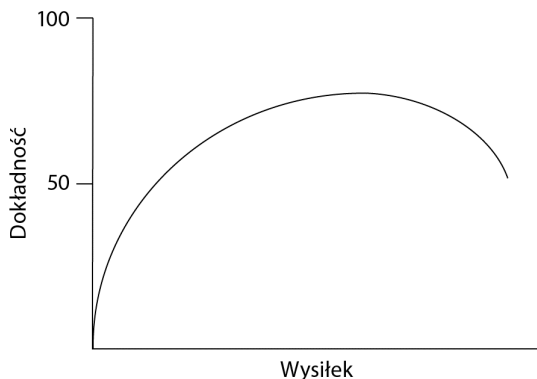
— Niels Bohr, duński fizyk

Im więcej wysiłku wkładamy w realizację jakiegoś przedsięwzięcia, tym lepszy uzyskujemy efekt. Czy to prawda? Być może, ale często można by włożyć zaledwie część tych wysiłków, aby uzyskać *równoważne* rezultaty. Na przykład mój samochód jest brudny i chcę go umyć. Jeśli sam się tym zajmę, to poświęcę na to około godziny. Taki czas wystarczy, aby umyć karoserię, odkurzyć wnętrze i umyć szyby. Po jednogodzinnej inwestycji mam stosunkowo czysty samochód.

Z drugiej strony mógłbym zadzwonić do firmy i zlecić jej umycie mojego samochodu. Firmie zajęłoby to cztery godziny. Zrobiliby wszystko to, co ja, ale znacznie dokładniej. Dodatkowo nawoskowaliby karoserię, nabłyszczyli deskę rozdzielczą i tak dalej. Gdy kiedyś przyglądałem się ich pracy, zauważyłem, że używają małych wacików kosmetycznych, by wyczyścić miejsca, które są zbyt małe, aby dostać się do nich szmatką. To bardzo duży wysiłek w zamian za nieco lepsze rezultaty. W moim przypadku prawo malejących zwrotów uaktywnia się na długo, zanim zdecyduję się skorzystać z wacika kosmetycznego do czyszczenia samochodu.

Jednak prawo malejących zwrotów trzeba również brać pod uwagę przy czasie, jaki poświęcamy na ocenę pracochłonności zadania. Często poświęcając niewiele czasu, uzyskujemy ocenę, która jest niemal tak samo dobra jak w przypadku, gdybyśmy poświęcili na ocenę znacznie więcej czasu. Zależność między dokładnością oceny a włożonym wysiłkiem pokazano na rysunku 6.1. Krzywa na wykresie odpowiada moim doświadczeniom i została potwierdzona podczas dyskusji z innymi osobami. Nie bazuje na empirycznych pomiarach.

Aby lepiej zrozumieć tę zależność, przypuśćmy, że postanowiłeś oszacować, ile ciastek zjadłem w zeszłym roku. Mógłbyś nie zastanawiać się zbyt długo i po prostu podać losową liczbę. Jeśli odwzoruje się to na sytuację z rysunku 6.1, będziesz całkowicie po lewej stronie osi wysiłków, a prawdopodobieństwo, że Twoja ocena jest dokładna, będzie bardzo małe. Mógłbyś przesunąć się w prawą stronę osi wysiłków, poświęcając pół godziny na badanie średnich krajowych dla konsumpcji ciasteczek. To pozwala poprawić dokładność z poziomu czystego zgadywania. Jeśli czujesz potrzebę, aby być bardziej precyzyjnym, możesz przeprowadzić pewne badania — zadzwonić do przyjaciół i rodziny, przesłedzić moje ostatnie zamówienia ciasteczek i tak dalej. Mógłbyś nawet śledzić mnie przez dzień — lub, jeszcze lepiej, miesiąc — a następnie ekstrapolować swoje obserwacje na informację o liczbie ciasteczek, które Twoim zdaniem zjadam w ciągu roku.



RYСУNEK 6.1. *Dodatkowy wysiłek włożony w szacowanie po przekroczeniu pewnej granicy przynosi niewielką wartość*

Należy wziąć pod uwagę wysiłek wkładany w szacowanie zgodnie z celem oceny. Jeśli próbujesz podjąć decyzję o tym, czy wysłać mi pudełko ciastek jako prezent, nie potrzebujesz bardzo dokładnej oceny. Jeśli ocena zostanie wykorzystana do podjęcia decyzji o tym, czy budować oprogramowanie, czy je kupić, to prawdopodobnie wystarczy, jeśli stwierdzisz, że realizacja projektu potrwa od sześciu do dwunastu miesięcy. Uściślanie jej do poziomu, w którym będziesz mógł stwierdzić, że potrwa siedem lub osiem miesięcy, może być zbędne.

Przyjrzyjmy się uważnie rysunkowi 6.1. Można na nim zauważyć kilka rzeczy. Po pierwsze, niezależnie od tego, ile wysiłku zainwestowano, ocena nigdy nie zbliża się do górnej części osi dokładności. Niezależnie od tego, ile wysiłku włożymy w oszacowanie, nadal będzie to oszacowanie. Żaden dodatkowy wysiłek nie może spowodować, że ocena stanie się perfekcyjna. Poza tym zwróćmy uwagę, jak niewiele wysiłku potrzeba, aby dokładność gwałtownie wzrosła od poziomu linii bazowej. Zgodnie z rysunkiem 6.1 około 10% wysiłków pozwala uzyskać 50% potencjalnej dokładności. Na koniec zauważmy, że ostatecznie dokładność oceny zaczyna spadać. Istnieje możliwość, że w rezultacie włożenia zbyt wielkiego wysiłku w ocenę, otrzymamy mniej dokładny wynik.

Przystępując do planowania projektu, warto pomyśleć o tym, w którym miejscu krzywej z rysunku 6.1 chcemy się znaleźć. W wielu projektach dąży się do uzyskania jak najwyższej pozycji na osi dokładności. Zespoły są zmuszane do wkładania coraz większych wysiłków w oceny nawet wtedy, gdy korzyści z większej dokładności gwałtownie się zmniejszają. Często jest to wynikiem przyjęcia uproszczonego punktu widzenia, że można zablokować budżety, harmonogramy i zakres oraz że sukces w projekcie jest równoważny dostarczeniu na czas i zgodnie z budżetem precyzyjnie zaplanowanego z góry zbioru funkcjonalności. Ten typ myślenia prowadzi do dążenia do tworzenia obszernych, zatwierdzanych dokumentów specyfikacji wymagań, wielu podejmowanych z góry prac analitycznych oraz opracowywania szczegółowych planów projektu uwzględniających wszystkie zadania, które przychodzą na myśl członkom zespołu. Pomimo całego tego wysiłku oceny nadal nie są doskonałe.

Z kolei zespoły Agile decydują się być bliżej lewej części wykresu z rysunku 6.1. Przyznają, że z ocen nie można wyeliminować niepewności, ale uwzględniają ideę, że niewielkie wysiłki są nagradzane dużym zyskiem. Pomimo tego, że zespoły Agile są znacznie niżej na skali dokładność-wysiłek, to potrafią tworzyć bardziej wiarygodne plany, ponieważ często dostarczają za pomocą niewielkich przysłów w pełni działający, przetestowany i zintegrowany kod.

Oceny są wspólne

Oceny nie są tworzone przez jedną osobę w zespole. Zespoły Agile nie powierzają wykonania ocen pojedynczemu ekspertowi. Pomijając znane dowody na to, że oceny przygotowane przez tych, którzy będą wykonywać pracę, są lepsze niż oceny przygotowane przez kogoś innego (Lederer i Prasad 1992), trzeba przyznać, że najlepsze oceny są uzyskiwane w wyniku wspólnej pracy zespołu, w którego skład wchodzi osoby, które będą wykonywać pracę. Są dwie przyczyny tego stanu rzeczy.

Po pierwsze, w projekcie Agile zazwyczaj nie wiadomo, kto będzie wykonywał konkretne zadanie. Oczywiście są podstawy, żeby przypuszczać, że zadaniem polegającym na implementacji skomplikowanej procedury składowanej zajmie się ten członek zespołu, który uchodzi za guru w dziedzinie baz danych. Nie ma jednak gwarancji, że to będzie właśnie on. Gdy przyjdzie czas na wykonanie tego zadania, może być zajęty, a wtedy zadaniem zajmie się ktoś inny. Ponieważ każdy może pracować nad wszystkim, ważne jest, aby wszyscy mieli udział w opracowywaniu ocen.

Po drugie, choć możemy się spodziewać, że zadaniem zajmie się guru w dziedzinie baz danych, to inni mogą mieć coś do powiedzenia o jego ocenie. Załóżmy, że guru od baz danych w zespole, Krystyna, ocenił określoną historijkę użytkownika na trzy dni idealne. Ktoś inny w projekcie może nie wiedzieć wystarczająco dużo, aby zaprogramować funkcjonalność, ale może wiedzieć wystarczająco dużo, aby powiedzieć: „Krysiu, nie szalej, kiedy ostatni raz pracowaliśmy nad podobną funkcjonalnością, zajęło Ci to znacznie więcej czasu. Myślę, że zapomniałaś, jakie to było trudne ostatnim razem”. W tym momencie Krystyna może przedstawić dobre wyjaśnienie, dlaczego tym razem jest inaczej. Z moich doświadczeń wynika jednak, że w tego typu sytuacjach członkowie zespołu częściej są skłonni przyznać, że rzeczywiście nie docenili pracochłonności niezbędnej dla szacowanej funkcjonalności.

Skala oceny

Badania wykazały, że najdokładniejsze oceny dotyczą rzeczy w obrębie jednego rzędu wielkości (Miranda 2001, Saaty 1996). W swoim mieście jesteśmy w stanie dość dobrze oszacować względne odległości do takich miejsc jak najbliższy sklep spożywczy, najbliższa restauracja czy najbliższa biblioteka. Na przykład biblioteka może się znajdować dwa razy dalej niż restauracja. Oceny mogą być znacznie mniej dokładne, jeśli ktoś poprosi nas również o ocenę względnej odległości od Księżyca lub stolicy sąsiedniego kraju. Ponieważ najlepiej idą nam oceny w ramach jednego rzędu wielkości, chcielibyśmy, aby większość naszych ocen była dokonywana w ramach tego zakresu.

Oto dwie skale oceny, z którymi odnosiłem sukcesy:

- 1, 2, 3, 5 i 8;
- 1, 2, 4 i 8.

Każdą z tych sekwencji można uzasadnić określoną logiką. Pierwsza jest ciągiem Fibonacciego¹. Ciąg Fibonacciego bardzo dobrze sprawdził się w roli sekwencji szacowania, ponieważ luki w sekwencji stają się odpowiednio większe w miarę wzrostu liczb. Wynosząca jeden punkt luka między 1 i 2 oraz między 2 i 3 wydaje się właściwa, podobnie jak luki od 3 do 5 i od 5 do 8. Te nieliniowe sekwencje dobrze się sprawdzają, ponieważ odzwierciedlają większą niepewność powiązaną z ocenami większych jednostek pracy. Można użyć każdej z tych sekwencji, choć osobiście nieco faworyzuję pierwszą.

¹ Liczba w ciągu Fibonacciego jest obliczana przez zsumowanie dwóch poprzednich liczb.

Każdą z tych liczb można porównać do pojemnika, który zostanie napełniony elementami o odpowiednim rozmiarze. Zamiast porównywać pracę do wody nalewanej do pojemników, lepiej porównać ją do piasku. Jeśli oceniamy z wykorzystaniem sekwencji 1, 2, 3, 5 i 8 i mamy historijkę naszym zdaniem tylko trochę większą od innych historyjek, które oceniono na pięć punktów, to nie ma niczego złego w tym, aby umieścić ją w pojemniku oznaczonym przez pięć punktów. Natomiast historyjka oceniana na siedem z całą pewnością nie zmieści się w pojemniku oznaczonym miarą pięciu punktów.

W niektórych sytuacjach mogłoby być korzystne uwzględnienie zera jako prawidłowej liczby wewnątrz zakresu ocen. Choć jest mało prawdopodobne, aby zespół napotkał wiele historyjek użytkownika lub funkcjonalności, które naprawdę nie wymagają żadnej pracy, to dołączenie 0 do sekwencji ocen często jest przydatne. Są ku temu dwa powody. Po pierwsze, jeśli chcemy zachować wszystkie funkcjonalności w zakresie 10×, to przypisywanie niezerowych wartości niewielkim funkcjonalnościom ogranicza rozmiar tych największych. Po drugie, jeśli rozmiar pracy naprawdę jest bliższy wartości 0 niż 1, to zespół może nie chcieć, aby implementacja funkcjonalności miała wpływ na obliczenia szybkości pracy zespołu. Jeśli zespół zdobędzie jeden punkt w bieżącej iteracji za jakąś trywialną funkcjonalność, to w następnej iteracji jego szybkość spadnie o jeden albo ten jeden punkt będzie musiał zostać zdobyty w wyniku wykonywania pracy, która nie zawsze jest równie trywialna.

Jeśli zespół zdecyduje się na uwzględnienie 0 w swojej skali ocen, to wszystkie osoby uczestniczące w projekcie (zwłaszcza właściciel produktu) muszą mieć tego świadomość. Nigdy nie miałem najmniejszych problemów z wyjaśnieniem tej oceny właścicielom produktu, którzy zdawali sobie sprawę, że historijkę ocenioną na zero punktów można porównać do darmowego obiadu. Mieli jednak również świadomość tego, że istnieje limit darmowych obiadów, jakie można otrzymać w pojedynczej iteracji. Alternatywą do zastosowania oceny 0 jest pogrupowanie bardzo małych historyjek i ocena ich jako pojedynczej jednostki.

Niektóre zespoły wolą pracować z większymi liczbami, na przykład 10, 20, 30, 50 i 100. Jeśli jednak zdecydujesz się na stosowanie większych wartości — na przykład od 10 do 100, to pomimo wszystko radzę wstępnie zidentyfikować liczby, które będziesz wykorzystywać w ramach tego zakresu. Nie należy na przykład pozwalać na to, aby jedna historyjka została oceniona na 66 punktów lub dni idealnych, a inna na 67. Jest to fałszywy poziom dokładności, ponieważ trudno jest zauważyć różnicę w rozmiarze wynoszącą 1,5%. Dopuszczalna jest różnica jednego punktu między takimi wartościami jak 1, 2 i 3. W procentach te różnice są znacznie większe niż między 66 i 67.

Historie użytkownika, epiki i tematy

Chociaż, ogólnie rzecz biorąc, oceniamy historijki użytkowników, których rozmiary mieszczą się w granicach jednego rzędu wielkości, to nie zawsze tak jest. Gdybyśmy oceniali wszystko w ramach jednego rzędu wielkości, oznaczałoby to konieczność pisania dość szczegółowych historyjek. W przypadku funkcjonalności, co do których nie jesteśmy pewni, czy są pożądane (przed poniesieniem zbyt dużych nakładów na realizację funkcjonalności wymagana jest wstępna kalkulacja kosztów), lub funkcjonalności, które nie będą implementowane w najbliższej przyszłości, pożądane jest wcześniejsze napisanie jednej, znacznie większej historyjki użytkownika. Większa historyjka użytkownika czasami jest nazywana *epiką*.

Ponadto zbiór powiązanych ze sobą historyjek można ze sobą połączyć (zwykle, jeśli posługujemy się kartami historyjek, spina się je spinaczem) i traktować jako pojedynczy podmiot zarówno podczas szacowania, jak i planowania wydania. Taki zbiór historyjek użytkownika określa się terminem *temat* (ang. *theme*). Epika, tylko ze względu na jej rozmiar, często sama w sobie jest tematem.

Dzięki agregacji historyjek w tematy i napisaniu niektórych historyjek w formie epik zespół może zmniejszyć wysiłek wkładany w szacowanie. Ważne jest jednak, aby zespół miał świadomość, że oceny tematów i epik są znacznie mniej pewne niż oceny bardziej konkretnych, mniejszych historyjek użytkowników.

Historyjki użytkowników, nad którymi zespół będzie pracował w najbliższej przyszłości (w ciągu najbliższych kilku iteracji) powinny być na tyle małe, aby można je było zrealizować w ciągu jednej iteracji. Te elementy powinny być oceniane w ramach jednego rzędu wielkości. Do tego celu stosuję sekwencję 1, 2, 3, 5 i 8.

Historyjki użytkowników lub inne podmioty oceny, które mogą być realizowane w dalszej perspektywie niż kilka iteracji, można pozostawić w formie epik lub motywów. Zalecam, aby te podmioty były oceniane w jednostkach poniżej zakresu od 1 do 8. Aby umożliwić ocenę tych większych jednostek, dodaję oceny 13, 20, 40 i 100 do mojej preferowanej sekwencji 1, 2, 3, 5 i 8.

Wyznaczanie oceny

Oto trzy najczęściej stosowane techniki oceny:

- opinia eksperta,
- analogia,
- dezagregacja.

Każda z tych metod może być stosowana samodzielnie, ale w celu uzyskania jak najlepszych wyników metody powinny być łączone.

Opinia eksperta

Jeśli chcesz się dowiedzieć, ile czasu zajmie jakieś przedsięwzięcie, zapytaj eksperta. Jest to przynajmniej jedno z możliwych podejść. W przypadku stosowania podejścia do oceny bazującego na opiniach zadajemy ekspertowi pytanie o to, jak długo potrwa realizacja określonego przedsięwzięcia oraz jaki jest jego rozmiar. Ekspert bazuje na intuicji lub subiektywnych odczuciach i daje ocenę.

Takie podejście jest mniej przydatne w projektach Agile niż w projektach tradycyjnych. W projektach Agile oceny przypisuje się historyjkom użytkownika lub innym funkcjonalnościom przynoszącym wartość użytkownikom. Implementacja tej funkcjonalności najprawdopodobniej wymaga różnych umiejętności i działań wykonywanych zwykle przez więcej niż jedną osobę. To sprawia, że trudno jest znaleźć odpowiednich ekspertów, którzy byliby zdolni ocenić wysiłek we wszystkich dyscyplinach. W tradycyjnym projekcie, w którym oceny są powiązane z zadaniami, to nie jest tak znaczący problem, ponieważ każde zadanie zwykle jest wykonywane przez jedną osobę.

Interesującą zaletą oceny na podstawie opinii eksperta jest to, że zwykle taka ocena nie zajmuje zbyt dużo czasu. Zazwyczaj deweloper czyta historyjkę użytkownika, może zadać jedno lub dwa wyjaśniające pytania, następnie dostarcza oceny, bazując na swojej intuicji. Istnieją nawet dowody na to, że ten typ oceny jest dokładniejszy w porównaniu z innymi, bardziej analitycznymi podejściami (Johnson i in. 2000).

Analogia

Alternatywą opinii eksperta jest ocena poprzez analogię. Taką ocenę wykonujemy w przypadku, gdy mówimy: „Ta historyjka jest trochę większa niż tamta”. Przy ocenie przez analogię oceniający porównuje historyjkę poddawaną ocenie z jedną lub kilkoma innymi historyjkami. Jeśli historyjka ma dwa razy większy rozmiar, przypisuje się jej dwa razy większą ocenę. Istnieją dowody na to, że łatwiej jest nam oceniać wielkości względne niż bezwzględne (Lederer i Prasad 1998; Vicinanza i in. 1991).

Przy dokonywaniu oceny tą metodą nie porównujemy wszystkich historyjek z jednym podstawowym lub uniwersalnym punktem odniesienia. Zamiast tego oceniamy każdą nową historyjkę w zestawieniu ze zbiorem historyjek, które zostały ocenione wcześniej. Jest to tzw. triangulacja. Aby dokonać triangulacji, należy porównać ocenianą historyjkę użytkownika z kilkoma innymi historyjkami. Aby zdecydować, czy historyjka powinna zostać oceniona na pięć punktów, należy sprawdzić, czy wydaje się ona nieco większa niż historyjka, którą oceniono na trzy punkty, i trochę mniejsza niż historyjka oceniona na osiem punktów.

Dezagregacja

Dezagregacja (ang. *disaggregation*) odnosi się do podziału historyjki na mniejsze, łatwiejsze do oceny fragmenty. Jeśli większość historyjek użytkownika, które mają być uwzględnione w projekcie, mieści się w zakresie od dwóch do pięciu dni pracy, to bardzo trudno będzie ocenić jedną historyjkę, która może zająć 100 dni. Powszechnie wiadomo, że większe jednostki nie tylko są trudniejsze do oceny, ale w tym przypadku będzie bardzo mało podobnych historyjek, z którymi tak dużą historyjkę będzie można porównać. Zadanie pytania w rodzaju „Czy ta historyjka użytkownika jest pięćdziesiąt razy trudniejsza od tamtej?” to zupełnie co innego niż zadanie pytania „Czy ta historyjka jest około półtora razy większa od tamtej?”.

Rozwiązaniem tego problemu jest oczywiście rozbicie dużej historyjki na mniejsze i dokonanie oceny. Trzeba jednak zachować ostrożność, aby przy stosowaniu tego podejścia nie posunąć się za daleko. Najprostszy sposób zilustrowania tego problemu jest posłużenie się przykładem spoza dziedziny wytwarzania oprogramowania. Spróbujmy skorzystać z dezagregacji do oceny moich wyników gry w golfa w ten weekend. Załóżmy, że grałem na polu golfowym, na którym jest osiemnaście otworów z oceną *par* równą cztery (czytelnikom nieznanym punktacji w golfie wyjaśniam, że *par* oznacza liczbę uderzeń, jaka jest potrzebna dobremu graczowi do umieszczenia piłki w końcowym dołku).

Aby dokonać oceny przez dezagregację, musimy oszacować mój wynik dla każdego dołka. Trafienie do pierwszego dołka jest dość łatwe, dlatego założmy, że otrzymałem ocenę trzy. Ale potem, przy następnym otworze, zwykle trafiał w jezioro, dlatego ten ruch ocenimy na siedem. Jest jeszcze dołek z pułapkami piaskowymi po drodze (ang. *sandtraps*); ocenimy go na pięć. I tak dalej. Jeśli jednak będę odtwarzał w myślach całe pole golfowe, to z dużym prawdopodobieństwem zapomnę o którymś dołku. Oczywiście wówczas bardzo łatwo to sprawdzić, ponieważ wiem, że powinno być osiemnaście indywidualnych ocen. Ale w przypadku stosowania dezagregacji do oceny historyjek użytkownika nie ma takiej „kontroli bezpieczeństwa”.

Jeśli stosujemy zbyt głęboką dezagregację, to nie tylko zwiększa się prawdopodobieństwo, że zapomnimy o jakimś zadaniu, ale również sumowanie ocen dużej liczby niewielkich zadań może prowadzić do problemów. Na przykład każdemu z 18 dołków przypisałem ocenę w zakresie od 3 do 8. Pomnożenie tej wartości przez 18 daje mi ocenę dla całego pola od 54 do 144. Nie ma szans na to, aby poszło mi aż tak dobrze albo aż tak źle. Gdyby poproszono mnie o oszacowanie mojego wyniku

za całe pole, prawdopodobnie podałbym wartość w zakresie od 80 do 120, co stanowi znacznie mniejszy zakres i jest znacznie bardziej przydatną oceną.

Szczegółowe zalecenia dotyczące podziału historyjek użytkownika zaprezentowano w rozdziale 12. „Podział historyjek użytkownika”.

Poker planistyczny

Moim zdaniem najlepszym sposobem oceny historyjek użytkowników przez zespoły Agile jest gra w planistycznego pokera (Grenning 2002). Planistyczny poker łączy cechy metod opinii eksperta, analogii i dezagregacji w przyjemne podejście do oceny, którego stosowanie prowadzi do uzyskania szybkich, ale wiarygodnych ocen.

W grze w planistycznego pokera uczestniczą wszyscy deweloperzy w zespole. Pamiętajmy, że termin *deweloperzy* obejmuje wszystkich programistów, testerów, inżynierów bazy danych, analityków, specjalistów w dziedzinie użyteczności (ang. *usability*), dokumentalistów, architektów, projektantów i tak dalej. W projekcie Agile liczba tych osób zwykle nie przekracza dziesięciu. Jeśli tak się dzieje, to zazwyczaj najlepiej podzielić taki zespół na dwa. Każdy zespół może dokonywać ocen niezależnie, dzięki czemu rozmiary zadań mogą być mniejsze. Właściciel produktu bierze udział w grze w planistycznego pokera, ale nie uczestniczy w ocenianiu.

Na początku gry w pokera każdy oceniający otrzymuje talię kart. Na każdej karcie jest napisana jedna z dopuszczalnych ocen. Każdy oceniający może na przykład otrzymać talię kart z ocenami 0, 1, 2, 3, 5, 8, 13, 20, 40 i 100. Karty powinny być przygotowane przed spotkaniem, na którym ma się odbyć rozgrywka, a liczby powinny być wystarczająco duże do tego, aby były widoczne przy stole. Karty można zachować do następnej sesji gry w planistycznego pokera.

Dla każdej ocenianej historyjki użytkownika lub motywu moderator czyta opis. Moderatorem zazwyczaj jest właściciel produktu lub analityk. Moderatorem może jednak być każdy, ponieważ z tą rolą nie są związane żadne specjalne uprawnienia. Właściciel produktu odpowiada na wszelkie pytania zadawane przez oceniających. Wszyscy jednak powinni dążyć do uwzględniania krzywej zależności między wysiłkiem a dokładnością (rysunek 6.1). Celem gry w planistycznego pokera nie jest wyznaczenie oceny, która oprze się wszystkim przyszłym kontrolom. Zamiast tego jej celem jest pozostanie w lewej części krzywej wysiłku — tam, gdzie wartościowe oceny mogą być uzyskane stosunkowo tanim kosztem.

Po udzieleniu odpowiedzi na wszystkie pytania każdy oceniający wybiera kartę, która wyraża jego ocenę. Karty nie są pokazywane tak długo, aż wszyscy oceniający dokonają wyboru. W tym momencie wszystkie karty są jednocześnie odwracane tak, aby wszyscy oceniający widzieli oceny dokonane przez swoich kolegów.

W tym momencie jest bardzo prawdopodobne, że poszczególne oceny będą się znacznie różnić między sobą. W gruncie rzeczy to dobra wiadomość. Jeśli oceny są różne, to osoby, które dały najwyższą i najniższą ocenę, wyjaśniają swoje oceny. Ważne, aby prośba o te wyjaśnienia nie została odebrana jako atak na tych oceniających. Jest to raczej próba ustalenia, jaki był ich sposób myślenia.

Na przykład osoba, która dała najwyższą ocenę, może powiedzieć: „Do przetestowania tej historyjki powinniśmy utworzyć atrapę obiektu bazy danych. To może zająć cały dzień. Poza tym nie jestem pewien, czy sprawdzi się nasz standardowy algorytm kompresji. Być może będzie trzeba napisać nowy — taki, który jest wydajniejszy pod względem zużycia pamięci”. Z kolei osoba, która dała najniższą ocenę, może na to odpowiedzieć: „Sądziłem, że będziemy przechowywać te informacje

w pliku XML — byłoby to dla nas łatwiejsze niż przechowywanie w bazie danych. Poza tym nie myślałem o większej ilości danych — może to być problem”.

Grupa może omawiać historię i dokonane oceny przez kilka kolejnych minut. Moderator może robić dowolne notatki, które jego zdaniem mogą się przydać w czasie, gdy ta historia będzie implementowana i testowana. Po zakończeniu dyskusji każdy oceniający ponownie dokonuje oceny przez wybór karty. Do czasu, aż wszyscy dokonają oceny, karty nie są odkrywane. Gdy wszyscy wybiorą odpowiednią kartę, są one jednocześnie odkrywane.

W wielu przypadkach w drugiej rundzie oceny będą już zbliżone. Jeśli jednak nadal występują rozbieżności, to proces jest powtarzany. Celem gry jest uzyskanie zgodności ocen i wybranie jednej wartości dla historii. Gra w planistycznego pokera rzadko zajmuje więcej niż trzy rundy, ale proces należy kontynuować tak długo, aż oceny będą jak najbliższe. Nie ma potrzeby, aby wszyscy uczestniczący w grze odwrócili kartę z dokładnie taką samą oceną. Jeśli moderuję spotkanie planistyczne i w drugiej rundzie czterech oceniających podaje oceny 5, 5, 5 i 3, to pytam osobę, która dała tę niską ocenę, czy zgadza się na to, aby została przyjęta ocena 5. Trzeba pamiętać, że celem nie jest maksymalna dokładność, ale rozsądek.

Odpowiednia ilość dyskusji

Podczas dokonywania ocen konieczne i właściwe jest prowadzenie wstępnej dyskusji na temat projektu. Jednak poświęcenie zbyt dużo czasu na dyskusje projektowe sprawia, że znajdujemy się zbyt wysoko na przedstawionej na rysunku 6.1 krzywej zależności między wysiłkiem a dokładnością. Oto skuteczny sposób na to, by dyskusje były prowadzone, a jednocześnie by nie trwały zbyt długo.

Kup dwuminutową klepsydrę i postaw na środku stołu, przy którym siedzą uczestnicy gry w planistycznego pokera. Każdy uczestnik gry w dowolnej chwili może obrócić klepsydrę. Gdy piasek się skończy (po upływie dwóch minut), rozpoczyna się kolejna runda gry. Jeśli nie zostanie osiągnięte porozumienie, dyskusję można kontynuować. Ale ktoś może natychmiast obrócić klepsydrę, co spowoduje, że dwuminutowy okres zacznie być odliczany od początku. Rzadko jest potrzeba obracania klepsydry więcej niż dwa razy. Z czasem to pomaga zespołom nauczyć się szybszego dokonywania ocen.

Krótsze sesje

W pokera planistycznego można grać z częścią zespołu — bez udziału wszystkich osób. To nie jest idealna opcja, ale takie rozwiązanie może być rozsądne zwłaszcza wtedy, gdy jest dużo historii do oceny — tak jak się zdarza na początku nowego projektu.

Najlepszym sposobem, aby to zrobić, jest podzielenie dużego zespołu na dwa lub trzy mniejsze złożone z co najmniej trzech oceniających. Ważne, aby oceny dokonywane przez każdą grupę były spójne. Jeśli jedna grupa ocenia historię użytkownika na trzy punkty lub dni idealne, to byłoby dobrze, gdyby druga grupa przydzieliła tej samej historii taką samą ocenę. Aby to osiągnąć, rozpocznij planowanie od wspólnej sesji gry w planistycznego pokera trwającej przez około godzinę. Zleć im ocenę od dziesięciu do dwudziestu historii. Następnie zaopatrz wszystkie grupy w kopie tych historii wraz z ocenami. Będą one używane jako podstawa ocen historii, które zostaną powierzone do oceny poszczególnym grupom.

Kiedy grać w planistycznego pokera

Zespoły powinny grać w planistycznego pokera w dwóch różnych momentach w projekcie. Po pierwsze, zanim projekt się oficjalnie rozpocznie albo podczas pierwszych iteracji zazwyczaj trzeba dokonać oceny dużej liczby historyjek. Dokonanie oceny początkowego zbioru historyjek użytkownika może zająć zespołowi dwa lub trzy spotkania trwające od jednej do trzech godzin każde. Oczywiście wszystko zależy od liczby historyjek do oceny, rozmiaru zespołu i zdolności właściciela produktu do zwięzłego objaśniania wymagań.

Po drugie, zespoły muszą włożyć trochę wysiłku w ocenę nowych historyjek zidentyfikowanych w czasie trwania iteracji. Jednym ze sposobów, aby to zrobić, jest zaplanowanie bardzo krótkiego spotkania pod koniec każdej iteracji. Zazwyczaj to wystarczy do oceny wszelkich prac, które pojawiły się podczas iteracji, oraz pozwala uwzględnić te prace w czasie ustalania priorytetów w następnej iteracji.

Jako alternatywę Kent Beck sugeruje powieszenie na ścianie koperty i umieszczenie w niej wszystkich nowych historyjek. Kiedy któryś członek zespołu ma kilka wolnych minut, wyciąga z koperty jedną lub dwie historyjki i poddaje je ocenie. Zespoły ustalają obowiązującą regułę — zwykle mówi ona, że wszystkie historyjki muszą być ocenione przed końcem dnia lub do końca iteracji. Pomysł zawieszenia na ścianie koperty z historyjkami użytkownika, które nie zostały ocenione, bardzo mi się podoba. Wolałbym jednak, aby ktoś w sytuacji gdy będzie miał kilka wolnych minut, które mógłby poświęcić ocenie, znalazł co najmniej jedną osobę, by mogli dokonać oceny wspólnie.

Dlaczego poker planistyczny się sprawdza?

Po opisaniu zasad przeprowadzania rozgrywki planistycznego pokera warto poświęcić kilka chwil na zastanowienie się nad powodami, dla których ta metoda tak dobrze się sprawdza.

Po pierwsze, gra w planistycznego pokera pozwala uwzględnić podczas oceny opinie wielu ekspertów. Ze względu na to, że ci eksperci tworzą wielofunkcyjny zespół specjalistów z różnych dziedzin w projekcie oprogramowania, są bardziej przystosowani do wykonywania zadań związanych z oceną niż ktokolwiek inny. Po dokonaniu szczegółowego przeglądu literatury dotyczącej szacowania projektów oprogramowania Jørgensen (2004) doszedł do wniosku, że „oceną zadań powinny zajmować się osoby najbardziej kompetentne w ich rozwiązywaniu”.

Po drugie, podczas gry w planistycznego pokera odbywa się żywy dialog, a oceniający na prośbę swoich współpracowników muszą uzasadniać swoje oceny. Okazało się, że prowadzi to do poprawy dokładności ocen, zwłaszcza w przypadku historyjek o dużym stopniu niepewności (Hagafors i Brehmer 1983). Udowodniono również, że konieczność uzasadniania prowadzi do ocen, które lepiej uwzględniają brakujące informacje (Brenner i in. 1996). Jest to ważne w projekcie Agile, ponieważ historyjki użytkownika poddawane ocenie często są celowo niejasne.

Po trzecie, jak wykazały badania, przyjęcie średniej z oceny pojedynczych członków zespołu prowadzi do lepszych wyników (Hoest i Wohlin 1998). Podobnie lepsze oceny uzyskuje się w wyniku prowadzenia grupowych dyskusji na temat ocen (Jørgensen i Moløkken 2002). Grupowa dyskusja jest podstawą gry w planistycznego pokera, a w wyniku dyskusji następuje uśrednienie wielu indywidualnych ocen.

Wreszcie, gra w planistycznego pokera się sprawdza, ponieważ jest przyjemna.

Podsumowanie

Poświęcenie większej ilości czasu i wysiłku na uzyskanie oceny nie zawsze skutkuje jej zwiększoną dokładnością. Nakład pracy potrzebnej do dokonania oceny powinien wynikać z celu tej oceny. Chociaż najlepszych ocen pracy dokonują osoby, które będą wykonywały tę pracę, to w zespole Agile nie wiadomo z góry, kto będzie wykonawcą określonego zadania. W konsekwencji ocena powinna być wspólnym przedsięwzięciem całego zespołu.

Oceny powinny być wartościami z predefiniowanej skali. Funkcjonalności, które będą realizowane w niedalekiej przyszłości i dla których są potrzebne dość wiarygodne oceny, powinny być na tyle małe, aby mogły zostać ocenione w nieliniowej skali od 1 do 10 — na przykład 1, 2, 3, 5 i 8 lub 1, 2, 4 i 8. Większe funkcjonalności, które najprawdopodobniej nie będą implementowane w kilku kolejnych iteracjach, mogą pozostać większe i być oceniane w takich jednostkach jak 13, 20, 40 i 100. Niektóre zespoły w swojej skali ocen uwzględniają ocenę 0.

W celu wyznaczenia oceny wykorzystujemy opinie ekspertów, analogie i dezagregację. Przyjemnym i skutecznym sposobem połączenia tych technik jest gra w planistycznego pokera. W grze tej każdy oceniający otrzymuje talię kart. Na każdej z kart jest wydrukowana prawidłowa ocena. Zespół omawia funkcjonalność, a każdy oceniający wybiera kartę, która reprezentuje jego ocenę. Karty wszystkich oceniających są odkrywane jednocześnie. Oceny są omawiane, a następnie proces powtarza się aż do uzyskania zgodnej oceny.

Pytania do dyskusji

1. Jak dokładne są dzisiaj Twoje oceny? Z których technik głównie korzystasz: opinii ekspertów, analogii czy dezagregacji?
2. Jaką skalę ocen stosujesz? Dlaczego?
3. Kto powinien uczestniczyć w grze w planistycznego pokera w Twoim projekcie?

ROZDZIAŁ 7.

Weryfikacja ocen

Nie ma sensu być dokładnym w przypadku, gdy nawet nie wiesz, o czym mówisz.

— John von Neumann

Jedno z najczęstszych pytań dotyczących oceniania z wykorzystaniem punktacji historyjek lub dni idealnych brzmi: „Kiedy dokonujemy weryfikacji ocen?”. Aby udzielić odpowiedzi na to pytanie, warto pamiętać, że punktacja historyjek i dni idealne to oceny ogólnego rozmiaru i złożoności implementowanej funkcjonalności. Zwłaszcza punkty historyjek nie są ocenami ilości czasu, jaki zajmuje implementacja funkcjonalności pomimo tego, że często wpadamy w pułapkę myślenia o nich w taki sposób. Ilość czasu potrzebna do zaimplementowania funkcjonalności jest funkcją jej rozmiaru (szacowanej za pomocą dni idealnych lub punktacji historyjek) oraz tempa postępów zespołu (wynikającego z szybkości jego pracy).

Jeśli weźmiemy pod uwagę, że punktacja historyjek i czas idealny to oszacowania czasu, łatwiej zauważyć, że konieczność weryfikacji ocen zachodzi tylko wtedy, kiedy uznamy, że względny rozmiar historyjki uległ zmianie. Jeśli korzystamy z punktacji historyjek lub czasu idealnego, to nie weryfikujemy oceny tylko dlatego, że implementacja historyjki trwała dłużej, niż sądziliśmy. Najlepszym sposobem, aby to zaobserwować, jest analiza kilku przykładów.

Przedstawiamy witrynę SwimStats

W pozostałej części tego rozdziału oraz w kilku kolejnych rozdziałach będziemy pracować nad witryną SwimStats — hipotetycznym serwisem dla zawodników i trenerów pływania. Witryna SwimStats będzie sprzedawana jako usługa koleżeńskim, szkolnym i akademickim zespołom pływackim. Trenerzy będą używali jej do zarządzania swoimi grupami pływaków, organizowania treningów oraz przygotowań do zawodów. Zawodnicy będą korzystali z serwisu w celu śledzenia wyników zawodów, rejestrowania osobistych rekordów oraz obserwacji swoich postępów. Sędziowie zawodów pływackich będą wprowadzali do systemu wyniki zawodów. Przykładowy ekran witryny SwimStats przedstawiono na rysunku 7.1.

Boulder Valley Summer Swim League				Search	League	Dual Meets	Community	News
Savannah Cohn		Lafayette Seals			Gender: F			
Leagues (Email us.)								
League	Team(s)	Season start date	Events Swam	First	Second	Third		
2005 BVSSL	Lafayette Seals	05/15/2005	18	12	4	2		
Click on Team Name, Swimmer's Name or Meet Date to view more information. Meets are sorted with most recent meet on top.								
Indicates Winner								
07/9/2005 Lafayette Seals at Meadow Hills								
Event Number	Age Group	Event	Time	Place	Points			
14	Girls 9-10	100 Free	2:09.48	1	6			
34	Girls 9-10	50 Back	1:06.41	1	6			
64	Girls 9-10	50 Breast	1:13.97	1	6			
07/16/2003 Lafayette Seals host Huntington Beach Blue Dolphins								
Event Number	Age Group	Event	Time	Place	Points			
14	Girls 9-10	100 Free	2:03.48	2	4			
34	Girls 9-10	50 Back	1:04.41	1	6			
64	Girls 9-10	50 Breast	1:13.43	1	6			

RYSUNEK 7.1. Przykładowy ekran witryny SwimStats

Kiedy nie należy weryfikować ocen?

Biorąc za przykład witrynę SwimStats, zastanówmy się najpierw krótko nad przypadkiem, gdy nie zachodzi potrzeba weryfikacji oceny. Załóżmy, że mamy historyjki zaprezentowane w tabeli 7.1. Po zakończeniu pierwszej iteracji pierwsze dwie historyjki zostały zaimplementowane. Zespół nie jest zadowolony z tego stanu rzeczy, ponieważ sądzono, że uda się zrealizować w każdej iteracji historyjki warte dwa razy więcej punktów (dwanaście zamiast sześciu). Zespół doszedł do wniosku, że każda z historyjek jest dwa razy bardziej złożona niż pierwotnie sądzono, i dlatego implementacja zajęła dwa razy więcej czasu. Zespół zdecydował, aby podwoić liczbę punktów przypisanych początkowo każdej historyjce. Oznacza to, że szybkość pracy zespołu wynosiła dwanaście (dwie historyjki po sześć punktów).

TABELA 7.1. Wybrane historyjki i ich oceny dla witryny internetowej SwimStats

ID historyjki	Historyjka	Ocena
1	Jako trener mogę wpisać nazwiska i inne dane osobowe wszystkich zawodników swojej drużyny.	3
2	Jako trener mogę definiować sesje treningowe.	3
3	Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.	3
4	Jako pływak mogę aktualizować swoje dane osobowe.	3

Jednak przed rozpoczęciem projektu zespół uznał, że wszystkie cztery historyjki wymienione w tabeli 7.1 mają równoważny rozmiar i złożoność, dlatego każdej z nich przypisano trzy punkty. Ponieważ zespół nadal uważa, że te historyjki są równoważne, to oceny dla historyjek 3. i 4. teraz także powinny zostać podwojone. Zespół przydzielił sobie więcej punktów za zaimplementowane historyjki, co podwoiło jego szybkość. Jednak ze względu na to, że jednocześnie została podwojona pozostała ilość pracy w projekcie, sytuacja zespołu jest taka sama, jakby wszystkie historyjki nadal były ocenione na trzy punkty, a szybkość zespołu — na sześć.

Szybkość zespołu to główny korektor

W tym przypadku szybkość zespołu spełnia rolę głównego korektora. Z uwagi na to, że ocena każdej funkcjonalności jest względna wobec innych, nie ma znaczenia, czy oceny są poprawne, trochę niepoprawne, czy też bardzo niepoprawne. Ważne, aby wszystkie były spójne. Nie można po prostu rzucić kostką i przypisać liczby jako oceny funkcjonalności. Jeśli jednak zachowamy spójność dokonywanych ocen, to zmierzenie szybkości w ciągu kilku iteracji pozwoli na stworzenie wiarygodnego harmonogramu.

Przyjrzyjmy się innemu przykładowi. Załóżmy, że projekt składa się z pięćdziesięciu historyjek użytkownika, z których każda została oceniona na jeden punkt. Dla uproszczenia założmy, że jestem jedyną osobą pracującą nad tym projektem i oczekuję, że w ciągu jednego dnia roboczego będę w stanie zaimplementować historyjkę ocenioną na jeden punkt. Zatem w przypadku dwutygodniowych iteracji oczekuję, że uda mi się zaimplementować dziesięć historyjek, co oznacza, że moja szybkość będzie wynosić dziesięć. Ponadto sądzę, że projekt zakończę w pięciu iteracjach (dziesięć tygodni). Jednak po pierwszej iteracji okazało się, że nie zaimplementowałem dziesięciu historyjek, a tylko pięć. Jeśli pozwolę, aby szybkość skorygowała moje błędy w ocenie, to uświadomię sobie, że realizacja projektu potrwa dziesięć iteracji, ponieważ moja szybkość wynosi połowę tego, co zakładałem.

Co się jednak stanie, jeśli zweryfikuję oceny? Załóżmy, że zweryfikowałem ocenę pięciu zaimplementowanych historyjek i przypisałem każdej z nich ocenę dwa. Teraz moja szybkość wynosi dziesięć (pięć zaimplementowanych historyjek, ocena każdej zweryfikowana na dwa) i pozostają do zaimplementowania historyjki oszacowane na czterdzieści pięć punktów. Biorąc pod uwagę szybkość wynoszącą dziesięć i czterdzieści pięć pozostałych punktów, oczekuję zakończenia projektu w ciągu 4,5 iteracji. Problem takiego podejścia polega na mieszaniu ocen zweryfikowanych z oryginalnymi. Dzięki wnioskowi z wcześniejszej pracy zweryfikowałem ocenę zrealizowanych historyjek na dwa. Niestety, patrząc w przyszłość na pozostałe czterdzieści pięć historyjek, nie mogę przewidzieć, które z historyjek pierwotnie ocenionych na jeden punkt w rzeczywistości zasługują na dwa punkty.

Kiedy weryfikować oceny?

W tym podrozdziale będziemy kontynuować prace nad serwisem SwimStats. Tym razem z historyjkami użytkowników i ocenami zestawionymi w tabeli 7.2.

Każda z pierwszych trzech historyjek użytkownika ma związek z wyświetlaniem wykresu dla użytkownika. Załóżmy, że zespół zaplanował uwzględnienie w pierwszej iteracji historyjek oznaczonych w tabeli 7.2 numerami 1, 2 i 6. Zaplanowana szybkość wynosi trzynaście. Jednak na koniec iteracji

TABELA 7.2. Wstępne oceny niektórych historyjek z witryny SwimStats

ID historyjki	Historyjka	Ocena
1	Jako pływak mogę obejrzeć wykres liniowy swoich czasów uzyskiwanych w wybranych zawodach.	3
2	Jako trener mogę obejrzeć wykres liniowy postępów wyników w zawodach w ciągu sezonu wszystkich moich pływaków.	5
3	Jako pływak mogę obejrzeć wykres kołowy pokazujący, ile pierwszych, ile drugich, ile trzecich, a ile niższych miejsc zająłem na zawodach.	3
4	Jako trener mogę obejrzeć tekstowy raport pokazujący najlepszy czas poszczególnych pływaków w każdym zawodach.	3
5	Jako trener mogę przesłać wyniki zawodów z pliku wyeksportowanego z systemu pomiaru czasu wykorzystywanego na zawodach.	3
6	Jako trener mogę zarekomendować systemowi, kto powinien uczestniczyć w każdym zawodach z uwzględnieniem ograniczeń dotyczących liczby zawodów, w jakich pływak może uczestniczyć.	5

okazało się, że zostały zrealizowane tylko historyjki 1. i 6. Członkowie zespołu tłumaczą, że zrobili mniej niż oczekiwano, ponieważ historyjka numer 1 była o wiele trudniejsza niż się spodziewano i powinna mieć ocenę „nie niższą niż sześć”. Założmy, że niedoszacowanie trudności nie dotyczy tylko jednej, szczególnie złożonej historyjki, ale ogólnej trudności wyświetlania wykresów. W takiej sytuacji, jeśli historyjka numer 1 była dwa razy większa niż oczekiwano, to tego samego można się spodziewać od historyjek numer 2 i 3.

Zobaczymy, jak to się rozgrywa w trzech scenariuszach.

Scenariusz 1. Bez ponownej oceny

W tym przypadku pozostawimy wszystkie oceny bez zmian. Zespół osiągnął w ostatniej iteracji szybkość ośmiu punktów. To sprawia, że oczekujemy, iż w kolejnych iteracjach będzie osiągał średnio po osiem punktów. Zespół jednak wie, że nie da się zrealizować historyjek 2. i 3. w jednej iteracji, pomimo że ich sumaryczna ocena wynosi tylko osiem punktów. Ponieważ realizacja każdej z tych historyjek wymaga tworzenia wykresów i zespół oczekuje, że każde zadanie związane z wykresami jest dwa razy bardziej złożone w porównaniu z bieżącą oceną (podobnie jak w przypadku historyjki 1.), doszedł do wniosku, że nie uda się zrealizować historyjek 2. i 3. w jednej iteracji. To osiem punktów, a to zbyt wiele.

Scenariusz 2. Weryfikacja oceny zrealizowanej historyjki

W tym scenariuszu spróbujemy się przekonać, czy problem rozwiąże skorygowanie pojedynczej oceny pierwszej historyjki. Po zakończeniu iteracji zespół miał wrażenie, że historyjka 1. była dwa razy trudniejsza niż się spodziewano. W związku z tym zdecydowano, że ocena zostanie zweryfikowana na sześć zamiast trzech punktów. To oznacza, że szybkość w poprzedniej iteracji wyniosła jedenaście — sześć punktów za historyjkę numer 1 i pięć punktów za historyjkę numer 6.

Ze względu na to, że weryfikacja ocen nie jest przeprowadzana dla żadnej innej historyjki, zespół zaplanował, że kolejna iteracja będzie obejmować historyjki 2., 3. i 4. Łączna ocena tych historyjek wynosi jedenaście punktów — co odpowiada tej samej ilości pracy, jaka została wykonana w poprzedniej iteracji. Jednak wystąpiły takie same problemy, jak w pierwszym przypadku: implementacja historyjek 2. i 3. prawdopodobnie potrwa dwa razy dłużej niż oczekiwano, a zespół nie będzie w stanie pracować w każdej iteracji tak, jak oczekiwano — ze średnią szybkością jedenastu punktów.

Scenariusz 3. Weryfikacja ocen w przypadku zmiany względnego rozmiaru

W tym przypadku zespół poddaje ponownej ocenie wszystkie historyjki dotyczące tworzenia wykresów. Oceny dla historyjek 1., 2. i 3. są dwa razy wyższe w porównaniu z ocenami zestawionymi w tabeli 7.2. Podobnie jak w drugim scenariuszu, szybkość dla pierwszej iteracji wynosi jedenaście — sześć punktów za historyjkę numer 1 i pięć punktów za historyjkę numer 6. Ponieważ szybkość w pierwszej iteracji wynosiła jedenaście, zespół oczekuje, że w kolejnej iteracji będzie podobnie. Jednak podczas planowania kolejnej iteracji zostanie wybrana tylko historyjka numer 2. Ocenę tej historyjki, początkowo oszacowanej na pięć, podwojono do dziesięciu i teraz jest tak duża, że nie ma miejsca na kolejną historyjkę.

Weryfikacja oceny okazała się przydatna tylko w tym trzecim scenariuszu. Oznacza to, że ocenę historyjki należy zweryfikować tylko wtedy, kiedy zmieni się jej względny rozmiar.

Weryfikacja oceny częściowo zrealizowanych historyjek

Na weryfikację oceny można zdecydować się również w przypadku, gdy zespół zrealizuje w iteracji tylko część historyjki. Załóżmy, że zespół pracuje nad historyjką, o następującej treści: „Jako trener mogę zarekomendować systemowi, kto powinien uczestniczyć we wszystkich zawodach”. Ta historyjka początkowo została oceniona na pięć punktów, ale wydaje się złożona.

Drużyny w zawodach pływackich otrzymują punkty w zależności od miejsc zajętych przez poszczególnych pływaków wchodzących w skład drużyny. Jednak planowanie zawodów pływackich nie sprowadza się do wyznaczania do zawodów najszybszego pływaka w każdej konkurencji. Każdy pływak ma ograniczenia związane z liczbą zawodów, w których może uczestniczyć. To oznacza, że możemy nie mieć możliwości wybrania Zuzanny do uczestnictwa w konkurencji 100 metrów stylem grzbietowym, ponieważ potrzebujemy jej bardziej w konkurencji 100 metrów stylem klasycznym. Załóżmy zatem, że zespół dotarł do końca iteracji, a system potrafi zoptymalizować przydział zawodników do poszczególnych konkurencji indywidualnych. Jednak zespół nie zaczął jeszcze myśleć nad tym, jak przypisać pływaków do sztafet. Ile punktów powinien zespół doliczyć do oszacowania szybkości bieżącej iteracji? Ile punktów powinien przypisać pozostałym zadaniom?

Przed wszystkim, ogólnie rzecz biorąc, w sprawie obliczania szybkości zwykle jestem zwolennikiem podejścia „wszystko albo nic”: jeśli historyjka została zrealizowana (zakodowana, przetestowana i zaakceptowana przez właściciela produktu), to zespół zdobywa wszystkie punkty, ale jeśli cokolwiek w ramach tej historyjki nie zostało zrobione, to nie zdobywa nic. W takiej sytuacji dokonanie oceny na końcu każdej iteracji jest łatwe: jeśli wszystko jest zrobione, zespół otrzymuje wszystkie punkty; jeśli czegoś brakuje, nie otrzymuje punktów. Jeśli zespół decyduje się na implementację pozostałej części historyjki w kolejnej iteracji, to taki sposób dobrze się sprawdza. Jego szybkość w pierwszej

iteracji jest nieco niższa niż oczekiwano, ponieważ nie otrzymał punktów za historijkę zaimplementowaną częściowo. Jednak w drugiej iteracji szybkość będzie wyższa niż się spodziewano, ponieważ otrzyma wszystkie punkty pomimo tego, że niektóre prace zostały wykonane jeszcze przed rozpoczęciem iteracji. Ten sposób działa dobrze, dopóki każdy pamięta, że wszyscy powinni być zainteresowani tym, aby uzyskać jak najwyższą średnią szybkość zespołu w czasie, a nie tym, aby szybkość w pojedynczej iteracji mocno wzrosła lub zmalała.

Jednak w niektórych przypadkach nieskończona część historyjki nie może być zrealizowana w następnej iteracji. W takich sytuacjach może być uzasadnione przydzielenie zespołowi częściowej liczby punktów za zrealizowany fragment historyjki. Pozostała część historyjki (która jest podzbiorem historyjki pierwotnej) zostanie oceniona ponownie z uwzględnieniem bieżącego stanu wiedzy zespołu. W tym przypadku historyjka została pierwotnie oceniona na pięć punktów. Jeśli zespół odczuwa, że fragment historyjki, który ukończono (planowanie konkurencji indywidualnych), jest równoważny trzem punktom lub dniom idealnym, to przypisuje sobie taką ocenę. Niedokończony fragment pierwotnej historyjki w tej sytuacji może być przypisany postaci: „Jako trener oczekuję od systemu rekomendacji co do tego, kto powinien uczestniczyć w każdej sztafecie”. Następnie zespół może ocenić mniejszą historijkę względem wszystkich pozostałych. Suma ocen rozdzielonych historyjek nie musi być równa wstępnej ocenie pięć.

Jednak najlepszym rozwiązaniem problemu przydziału punktów do niezakończonych historyjek jest unikanie sytuacji, w których historyjki są realizowane częściowo. W tym celu wyznacza się odpowiednio małe historyjki.

Cel weryfikowania ocen

Nie należy się zbyt marnie przejmować koniecznością weryfikacji ocen. Za każdym razem, gdy zespół uznaje, że jedna lub kilka historyjek zostało nieodpowiednio ocenionych w porównaniu z innymi historyjkami, powinien zweryfikować ocenę jak najmniejszej liczby historyjek tak, aby względne oceny stały się bardziej prawidłowe. Weryfikację ocen należy stosować jako narzędzie edukacyjne wykorzystywane do oceny przyszłych historyjek użytkowników. Jak nauczył mnie Tom Poppendieck, „Jedyną prawdziwą porażką jest niewyciąganie wniosków z niepowodzeń”. Wyciągajmy wnioski z każdej weryfikacji oceny historyjki i przekształcajmy nowe doświadczenia w sukces.

Podsumowanie

Świadomość tego, że punktacja historyjek oraz dni idealne to sposoby oszacowania rozmiaru, pomagają zorientować się, kiedy należy zweryfikować wcześniej dokonane oceny. Weryfikację ocen należy przeprowadzać tylko wtedy, kiedy w naszej opinii względny rozmiar jednej lub kilku historyjek się zmienia. Nie należy weryfikować ocen tylko dlatego, że postępy nie są tak szybkie, jak oczekiwano. Szybkość — wielki korektor — powinna prowadzić do wyeliminowania większości niedokładności w ocenach.

Na końcu iteracji nie polecam przydzielania części punktów za częściowo zrealizowane historyjki użytkowników. Polecam, aby zespół zaliczał do swojej szybkości całą ocenę (jeśli historyjka została w pełni zakończona, a funkcjonalność zaakceptowana przez właściciela produktu) albo żeby nie doliczał żadnych punktów (jeśli nie została zaakceptowana). Zespół może jednak zdecydować się na weryfikację ocen częściowo zrealizowanych historyjek użytkownika. Zazwyczaj oznacza to oszacowanie historyjki

użytkownika reprezentującej prace, które zostały zrealizowane podczas iteracji, i jednej lub kilku historyjek użytkownika, które opisują pozostałe prace. Suma tych ocen nie musi być równa ocenie początkowej.

Pytania do dyskusji

1. W jaki sposób szybkość zespołu przyczynia się do korekty niepoprawnych ocen?
2. Dlaczego weryfikację oceny należy przeprowadzać tylko wtedy, gdy względny rozmiar się zmienia? Podaj kilka przykładów bieżącego lub niedawno prowadzonego projektu, w którym względny rozmiar jednego lub kilku elementów się zmienił.

ROZDZIAŁ 8.

Wybór między punktami historyjek a dniami idealnymi

Jeśli powiesz ludziom, dokąd mają zmierzać, ale nie powiesz im, jak to osiągnąć, będziesz zaskoczony wynikami.

— generał George S. Patton

Zarówno ocenianie metodą punktów, jak i dni idealnych ma swoje zalety. Aby pomóc w podjęciu decyzji, z której techniki skorzystać, w tym rozdziale zaprezentuję kluczowe przesłanki przemawiające na rzecz każdej z nich.

Powody przemawiające za metodą punktów

W tym podrozdziale zaprezentowano kluczowe powody przemawiające za dokonywaniem ocen historyjek w punktach. Oto one:

- Punkty historyjek sprzyjają zachowaniom interdyscyplinarnym.
- Ocenę punktową historyjek się nie degradują.
- Ocenę punktową historyjek są czystą miarą rozmiaru.
- Ocenianie w punktach zwykle jest szybsze.
- Dni idealne poszczególnych członków zespołu nie są równe.

Punkty historyjek sprzyjają zachowaniom interdyscyplinarnym

Jednym z powodów, dla których zespoły Agile osiągnęły sukces, jest to, że takie zespoły są interdyscyplinarne. Oznacza to, że w skład zespołów Agile wchodzi przedstawiciele wszystkich dziedzin, które są niezbędne do stworzenia produktu — w tym programiści, testerzy, menedżerowie produktu, projektanci użyteczności, analitycy, inżynierowie baz danych i tak dalej. Gdy zespół interdyscyplinarny jest tworzony po raz pierwszy, często się zdarza, że niektórym jego członkom

trudno jest się wyrzec swojej pierwotnej branżowej tożsamości. Dla produktu będzie lepiej, jeśli uczestnicy projektu będą identyfikowali siebie przede wszystkim jako członków zespołu, a dopiero później jako specjalistów w określonej dziedzinie — tzn. „Uczestniczę w projekcie Napa, gdzie jestem testerem”, a nie „Jestem testerem przydzielonym do projektu Napa”. Różnica być może jest subtelna, ale zmiana w sposobie myślenia — bardzo istotna.

Ocenianie historyjek w punktach pomaga zespołom nauczyć się pracować w trybie interdyscyplinarnym. Ponieważ ocena punktowa historyjki powinna być pojedynczą liczbą reprezentującą pracę całego zespołu, to stosowanie tej metody oceny inicjuje dyskusje między członkami zespołu dotyczące prac, które należy uwzględnić. Z kolei ocenianie za pomocą dni idealnych często wymaga od grup specjalistów dokonania oceny czasu realizacji „ich części” historyjki. Następnie wszystkie oceny częściowe są ze sobą sumowane, co tworzy ocenę łączną. Na przykład programiści mogą dojść do wniosku, że potrzebują trzech dni idealnych, inżynier bazy danych potrzebuje jednego, a tester dwóch. Następnie historyjce jest przypisywana ocena sześciu dni idealnych.

Ta niewielka różnica w sposobie przeprowadzania pierwszych dyskusji na temat historyjek ma znaczący wpływ na późniejszy sposób implementacji.

Oceny punktowe historyjek się nie degradują

Ocena historyjki wyrażona w punktach ma większą trwałość w porównaniu z oceną w dniach idealnych. Ocena w dniach idealnych może się różnić w zależności między innymi od doświadczenia zespołu, używanych technologii, dziedziny oraz członków zespołu. Aby zrozumieć, dlaczego tak się dzieje, założmy, że programista uczy się nowego języka programowania i zadano mu pytanie o to, ile czasu zajmie mu napisanie niewielkiej aplikacji. Jego odpowiedź może brzmieć: „Pięć dni”. Spróbujmy teraz przeskoczyć o kilka miesięcy do przodu i postawić temu samemu programiście pytanie o czas implementacji aplikacji o dokładnie takim samym rozmiarze i poziomie trudności. Jego odpowiedź może brzmieć: „Jeden dzień”, bo w tym czasie nabył nowe umiejętności w posługiwaniu się językiem. Mamy teraz problem, bo obie aplikacje mają ten sam rozmiar, a pomimo to oceniono je zupełnie inaczej.

Chcielibyśmy myśleć, że pomiar szybkości z biegiem czasu skoryguje lub uwzględni ten problem. Niestety tak się nie dzieje. Zamiast tego prędkość jest stała, pomimo tego, że wykonujemy większą pracę. Założmy, że ten programista jest jedyną osobą w zespole i że pracuje w tygodniowych iteracjach. Gdy po raz pierwszy oceniał pracochłonność aplikacji, oszacował, że jej realizacja zajmie pięć dni idealnych. Przyjmijmy, że w jego środowisku dzień kalendarzowy jest równy dniowi idealnemu. Rozpoczął pracę nad aplikacją pierwszego dnia iteracji, a zakończył piątego. Jego szybkość w tej iteracji wynosi pięć. Następnie, kilka miesięcy później, ze względu na to, że podobną aplikację ocenił na jeden dzień idealny, w ciągu jednej iteracji zaimplementuje pięć takich aplikacji. Jego szybkość tym razem także wyniesie pięć, pomimo tego, że zrealizuje pięć razy tyle pracy, co wcześniej. W przypadku pewnych projektów, szczególnie tych, w których wykorzystuje się nowe technologie lub w których dziedzina jest dla zespołu nowa, może to mieć duże znaczenie.

Zwróćmy uwagę, że oceny zarówno w punktach, jak i dniach idealnych muszą zostać zaktualizowane, jeśli rozmiar wysiłku — na przykład ze względu na opracowanie frameworka — się zmieni. Niemniej jednak oceny w dniach idealnych muszą być zmienione w przypadku, gdy zespół stanie się w czymś lepszy.

Oceny punktowe historyjek są czystą miarą rozmiaru

Zgodnie z tym, co napisano we wstępie do tej części, ważnym pierwszym krokiem w określeniu tego, jak długo coś zajmie, jest oszacowanie, jak duża praca jest do wykonania lub ile jej pozostało do zrobienia. Punkty są wyłącznie miarą rozmiaru. Nie można powiedzieć tego samego o dniach idealnych. Dni idealne mogą być stosowane jako miara rozmiaru, ale z pewnymi wadami. Jak wspomniano w poprzednim punkcie, ocena w dniach idealnych zmienia się wraz z poprawą umiejętności dewelopera. Nie dotyczy to punktów przypisywanych historyjkom — rozmiar jest taki sam i się nie zmienia. Jest to bardzo pożądaný atrybut każdego pomiaru rozmiaru.

Fakt, że punkty są czystą miarą rozmiaru, ma kilka zalet. Po pierwsze, oznacza, że możemy przypisywać historyjkom oceny punktowe wyłącznie przez analogię. Istnieją wiarygodne dowody na to, że lepiej oceniamy przez porównanie „to jest podobne do” niż wtedy, gdy staramy się oszacować rozmiar bezwzględny (Lederer i Prasad 1998; Vicinanza i in. 1991). Kiedy oceniamy w dniach idealnych, nadal możemy oceniać przez analogię. Jeśli jednak oceniamy w dniach idealnych, to zwykle myślimy o kalendarzu oraz o tym, jak długo potrwa realizacja historyjki.

Po drugie, ze względu na to, że punkty są czystą miarą rozmiaru i są całkowicie abstrakcyjne, nie ma pokusy, żeby porównać je z rzeczywistością. Zespoły, które oceniają za pomocą dni idealnych, nieuchronnie porównują swoje dni idealne z dniami rzeczywistymi. Następnie starają się usprawiedliwiać, dlaczego udało im się zrealizować „tylko” osiem idealnych dni pracy w ciągu dziesięciodniowej iteracji.

Ocenianie w punktach zwykle jest szybsze

Zespoły, które oceniają z wykorzystaniem punktów, zwykle robią to o wiele szybciej w porównaniu z zespołami oceniającymi za pomocą dni idealnych. Aby ocenić wiele historyjek, konieczne jest przeprowadzenie licznych dyskusji projektowych na ich temat: „Czy zostanie to zaimplementowane w bazie danych? Czy możemy wielokrotnie wykorzystywać interfejs użytkownika? W jaki sposób to wpłynie na warstwę pośrednią?”. Prędzej czy później zespół staje przed koniecznością udzielenia odpowiedzi na wszystkie te pytania.

Z moich doświadczeń wynika, że zespoły dokonujące ocen w dniach idealnych mają tendencję do podejmowania tego rodzaju dyskusji nieco głębiej niż zespoły oceniające w punktach. Różnica przypuszczalnie wynika stąd, że w przypadku dokonywania ocen w dniach idealnych występuje większa pokusa myślenia o pojedynczych zadaniach niezbędnych do implementacji historyjki niż myślenie w kategoriach rozmiaru historyjki względem innych historyjek.

Dni idealne poszczególnych członków zespołu nie są równe

Załóżmy, że dwóch biegaczy: jeden szybki i jeden wolny, stoją na starcie biegu. Każdy może zobaczyć cały przebieg trasy i obaj oceniają ją na jeden kilometr. Mogą porównać ją z inną trasą, na której obaj biegali, i zgodzić się, że jej długość wynosi około połowy długości tej trasy. Ich dyskusje na temat rozmiaru trasy (w tym przypadku właściwie jest to dystans) mają sens.

Załóżmy jednak, że zamiast dyskutować o długości tras, ci dwaj biegacze dyskutują o czasie, jaki zajmie im przebiegnięcie tej trasy. Szybki biegacz może powiedzieć: „To jest trasa na pięć minut”, na co wolny biegacz odpowie: „Nie, przebiegnięcie tej trasy zajmie przynajmniej osiem minut”. Każdy oczywiście będzie miał rację, ale nie będą w stanie rozwiązać swoich problemów inaczej niż

w wyniku dyskusji o trasie w kategoriach udzielenia odpowiedzi na pytanie, jak dużo czasu jednemu z nich (lub jakiemuś innemu biegaczowi) zajmie jej pokonanie.

Ten sam problem dotyczy oceniania za pomocą dni idealnych. Możesz myśleć, że uda Ci się całkowicie zrealizować określoną historijkę użytkownika w ciągu trzech dni idealnych. Ja uważam, że uda mi się to zrobić w pięć dni. Prawdopodobnie obaj mamy rację. W jaki sposób możemy dojść do porozumienia? Możemy zdecydować się na wybranie Twojej oceny, ponieważ uważamy, że to Ty będziesz wykonywał zadanie. Ale to może być błąd, ponieważ kiedy nadejdzie czas na implementację tej historijki, Ty możesz być zbyt zajęty i to ja będę musiał się tym zająć. A ja się spóźnię, ponieważ historijka została oszacowana na trzy dni, a zajmie mi pięć.

Większość zespołów ignoruje ten problem. Jest to dopuszczalne, jeśli wszyscy deweloperzy mają w przybliżeniu takie same umiejętności albo jeśli programiści zawsze pracują w parach, co pomaga zrównoważyć ekstremalne różnice w wydajności.

Powody przemawiające na rzecz dni idealnych

W tym podrozdziale omówiono kluczowe powody przemawiające za ocenianiem za pomocą dni idealnych. Oto one:

- Ocenę w dniach idealnych łatwiej wyjaśnić na zewnątrz zespołu.
- Ocena w dniach idealnych początkowo jest łatwiejsza.

Ocenę w dniach idealnych łatwiej wyjaśnić na zewnątrz zespołu

Dni idealne są bardzo łatwe do intuicyjnego postrzegania — „To ilość czasu, jaka byłaby mi potrzebna na realizację zadania, gdybym pracował wyłącznie nad nim”. Z powodu tego intuicyjnego pojmowania dni idealnych są one łatwe do zrozumienia, co sprawia, że ocenę w dniach idealnych można bezproblemowo wyjaśnić osobom spoza zespołu projektowego. Wszyscy rozumieją, że nie każda minuta dnia roboczego jest poświęcona na programowanie, testowanie, projektowanie albo dążenie w inny sposób do osiągnięcia postępów w implementacji nowych funkcjonalności.

Zazwyczaj trzeba wyjaśnić osobom z zewnątrz (a na początku projektu również członkom zespołu) koncepcję punktu jako miary rozmiaru. Często można jednak skorzystać z możliwości wyjaśnienia zasad punktacji historijek jako sposobu opisanego ogólnego podejścia do szacowania i oceniania przyjętego w projekcie. To świetna okazja, aby przyzwyczaić interesariuszy z zewnątrz do takich pojęć, jak stózek niepewności czy stopniowe udoskonalanie dokładności planu, oraz tego, że obserwacja szybkości pracy zespołu w ciągu kilku okresów prowadzi do większej wiarygodności tworzonych planów.

Ocena w dniach idealnych początkowo jest łatwiejsza

Oprócz tego, że dni idealne łatwiej jest wyjaśnić innym, to ocena taka jest łatwiejsza do wdrożenia w zespołach na początku projektu. Gdy zespół zdecyduje się na ocenę w punktach, to ocena pierwszych kilku historijek jest trudna lub jest wykonywana niepewnie. Bez podstawowego wskaźnika, na przykład dnia pracy od dziewiątej rano do siedemnastej po południu lub kilku wcześniej ocenionych historijek, zespół, który korzysta z punktów, musi znaleźć swoją własną podstawę w wyniku oceny kilku historijek.

Na szczęście większość zespołów bardzo szybko przechodzi przez ten początkowy etap oceny historyjek w punktach. Zwykle wystarczy trochę praktyki, aby wiele zespołów dokonywało oceny w punktach tak samo naturalnie, jakby robiły to przez wiele lat. Jednak przy ocenianiu pierwszych kilku historyjek mogą czuć się nieswojo.

Zalecenia

Osobiście preferuję ocenianie w punktach. Uważam, że korzyści wynikające z tego, że punkty są czystą miarą rozmiaru, są przekonujące. Ogromną zaletą ocen punktowych jest promocja zachowań interdyscyplinarnych. Stąd bierze się duża różnica wynikająca z przesunięcia akcentu z myślenia kategoriami „moja część zajmie mi trzy dni idealne, a Twoja będzie trwać dwa dni idealne, razem więc zajmie to nam pięć dni idealnych” na myślenie „ogólnie rzecz biorąc, ta historyjka wydaje się mieć podobny rozmiar do tamtej, zatem przypiszemy jej także pięć punktów”. Jeszcze jedną wielką zaletą jest to, że punkt przypisany historyjce jest tym samym dla dwóch różnych członków zespołu, czego nie można powiedzieć o dniu idealnym. Dwaj programiści o różnych umiejętnościach lub doświadczeniu mogą się porozumieć co do rozmiaru funkcjonalności, ale mogą się nie zgodzić co do oceny czasu, jaki im zajmie jej stworzenie.

Lista wad oceniania historyjek za pomocą punktów jest niezwykle krótka. To prawda, że na początku projektu ocena w dniach idealnych może być łatwiejsza. Jednak dyskomfort pracy z mglistymi punktami trwa krótko. Dni idealne z pewnością łatwiej wytłumaczyć osobom z zewnątrz zespołu, ale podejmowanie decyzji o metodzie oceny nie powinno bazować na tym, jak trudno tę metodę wytłumaczyć osobom z zewnątrz. Fakt, że dni idealne są tak łatwo zrozumiałe, stwarza również problemy. W niektórych firmach mogą pojawić się naciski, aby dni rzeczywiste były jak najbliższe idealnym. Nie ma niczego złego w dążeniu do tego, by poświęcać więcej uwagi naszej pracy i koncentrować się na niej. Jednak presja na to, aby każdy realny dzień był bliski idealnemu, będzie również skutkować tym, że zaczniemy szacować historyjki w czasie rzeczywistym, a nazywać ten czas idealnym. Zatem definicja dnia idealnego zostanie przekształcona do postaci „dzień, w którym przez sześć godzin osiągałem postępy w projekcie, a przez dwie godziny robiłem inne rzeczy”.

Czasami prowadzę zespoły, w których historyjki są oceniane za pomocą dni idealnych. Zazwyczaj robię to tylko z zespołami, które nie potrafią zaakceptować, że rozróżnienie ocen rozmiaru od ocen czasu trwania jest korzystne. Niektóre osoby są tak przyzwyczajone do tego, że po zadaniu pytania o oszacowanie pracochłonności natychmiast podają datę zakończenia, że przejście na ocenianie w punktach jest dla nich trudne.

W takich sytuacjach początkowo zespół dokonuje ocen w dniach idealnych, ale kiedy tylko staje się to możliwe, zaczynam zadawać pytania w rodzaju „Jaki jest rozmiar tej historyjki w porównaniu z tą, którą ocenialiśmy pięć minut wcześniej?”. Albo pytam: „Czy ta historyjka nie wydaje się wam nieco mniejsza od tej, którą ocenialiśmy przed chwilą?”. Celem tych pytań jest próba zmiany tematu rozmowy na bardziej abstrakcyjny. Dążę do prowadzenia dyskusji raczej o tym, jaki jest względny rozmiar historyjek, niż o tym, ile czasu potrwa zaprojektowanie ekranu, zakodowanie procedury składowanej lub napisanie kodu HTML. Dzięki temu zespół może rozpocząć dokonywanie ocen w dniach idealnych, a następnie stopniowo odchodzić od podawania ocen w postaci liczby dni.

Podsumowanie

Zespół może wybrać między ocenianiem w punktach a dniach idealnych. Każda metoda jest dobra i ma kilka zalet, które sprawiają, że jest godna polecenia.

Zaletą ocen w punktach jest promowanie interdyscyplinarnych zachowań w zespole. Ponadto, ze względu na to, że punkty są czystszy wskaźnikiem rozmiaru, nie wymagają weryfikacji oceny wówczas, gdy zespół osiągnie postępy w określonej technologii lub dziedzinie. Ocena historyjek w punktach często jest szybsza w porównaniu z oceną w dniach idealnych. Wreszcie, w przeciwieństwie do dni idealnych, punkty historyjek mogą być porównywane między członkami zespołu. Jeśli jeden członek zespołu uważa, że implementacja historyjki zajmie cztery dni idealne, a inny członek zespołu uważa, że zajmie ona jeden dzień idealny, to każdy z nich może mieć rację, ale nie mają oni wspólnej podstawy do dyskusji w celu ustalenia pojedynczej oceny.

Zaletą oceniania w dniach idealnych jest większa łatwość w objaśnieniu metody oceny osobom z zewnątrz zespołu oraz prostsze wdrożenie na początku projektu.

Osobiście preferuję ocenianie w punktach. Zalety oceniania historyjek w punktach są bardziej przekonujące. Jeśli zespół ma trudności z ocenianiem samego rozmiaru, zaczynam od oceniania w dniach idealnych, a następnie przechodzę na punkty. Robię to poprzez zadawanie pytań w rodzaju „Jaki jest rozmiar tej historyjki w porównaniu z innymi historyjkami, które ocenialiśmy?” zamiast „Ile dni idealnych zajmie implementacja tej historyjki?”. Większość zespołu prawie nie zauważa stopniowego przejścia, a kiedy ono nastąpi, zaczyna myśleć w kategoriach punktów zamiast dni idealnych.

Pytania do dyskusji

1. Które podejście wolisz — ocenianie historyjek w punktach czy w dniach idealnych? Dlaczego?
2. W jaki sposób wdrożyłbyś wybraną metodę w swojej firmie? Jakich przeszkód oczekujesz i jak można je pokonać?

CZĘŚĆ III

Planowanie pod kątem wartości

Przed przystąpieniem do planowania projektu trzeba się zastanowić, jakie są potrzeby naszych użytkowników. Samo wygenerowanie listy funkcjonalności, które naszym zdaniem są im potrzebne, a następnie utworzenie harmonogramu ich implementacji nie wystarczy. Uzyskanie najlepszego połączenia funkcjonalności produktu (zakresu), harmonogramu i budżetu wymaga świadomego rozpoznania kosztów i wartości historyjek użytkowników i tematów, które zostaną opublikowane w wydaniu.

Część ta rozpoczyna się od opisu czterech konkretnych czynników, które należy wziąć pod uwagę przy ustalaniu priorytetów historyjek użytkowników i tematów. Następnie zostanie opisanych kilka prostych sposobów modelowania finansowego zwrotu inwestycji w historyjkę, włącznie ze sposobem porównywania uzyskanych wartości. Dalej omówimy dwa różne podejścia do oceny celowości historyjek i tematów. Część ta zakończy się prezentacją porady dotyczącej sposobu podziału historyjek lub funkcjonalności na mniejsze, gotowe do zaimplementowania.

ROZDZIAŁ 9.

Priorytety tematów

*Pierwszym krokiem niezbędnym do uzyskania tego, czego chcesz,
jest zdecydowanie o tym, czego chcesz.*

— Ben Stein

Rzadko jest wystarczająco dużo czasu, żeby zrobić wszystko. Dlatego właśnie ustalamy priorytety. Odpowiedzialność za ustalanie priorytetów jest wspólna dla całego zespołu, ale przedsięwzięciem kieruje właściciel produktu. Niestety, ogólnie rzecz biorąc, trudno jest ocenić wartość niewielkich jednostek funkcjonalnych, takich jak pojedyncza historyjka użytkownika. W celu obejścia tego problemu poszczególne historyjki użytkowników lub funkcjonalności łączy się w *motywy*. Następnie historyjkom i tematom nadawane są priorytety tak, aby można było stworzyć plan wydania. Tematy powinny być wybierane w taki sposób, aby każdy definiował dyskretny zestaw funkcjonalności wartościowych z punktu widzenia użytkowników lub klientów. Na przykład przy opracowywaniu witryny SwimStats, można wyszczególnić następujące tematy:

- Śledzenie wszystkich rekordów życiowych i umożliwienie pływakom ich przeglądania.
- Umożliwienie trenerom optymalnego wyznaczania pływaków do zawodów i prognozowania wyniku drużyny w zawodach.
- Umożliwienie trenerom definiowania zajęć treningowych oraz śledzenia dystansu przepłyniętego na treningach.
- Integracja z popularnymi urządzeniami przenośnymi w celu umożliwienia korzystania z serwisu na basenie.
- Import i eksport danych.
- Umożliwienie sędziom śledzenia wyników zawodów i dokonywania ocen.

Każdy z tych tematów ma istotne znaczenie dla użytkowników oprogramowania. Każdemu z nich można by przypisać wartość finansową. Po przeprowadzeniu pewnych badań można by ustalić, że wsparcie dla urządzeń mobilnych może przynieść wzrost sprzedaży wynoszący około 150 000 € . Możemy porównać to z oczekiwanym, wynoszącym 200 000 € wzrostem sprzedaży w przypadku, gdy kolejną wersję oprogramowania będzie można wykorzystać do sędziowania zawodów pływackich. Następnie można nadać tym tematom priorytety. Ustalanie priorytetów to jednak znacznie więcej niż proste ustalanie finansowego zwrotu z implementacji nowego zestawu funkcjonalności.

Czynniki mające znaczenie przy określaniu priorytetów

Określenie wartości tematu jest skomplikowane, a właściciele produktów w projektach Agile często udzielają niejasnych i w zasadzie bezużytecznych porad w rodzaju „przy ustalaniu priorytetów należy brać pod uwagę wartość dla biznesu”. Może to być doskonała wskazówka do oceny wartości nominalnej, ale zastanówmy się, co to jest wartość dla *biznesu*? W celu dostarczenia bardziej praktycznego zbioru wytycznych do ustalania priorytetów przyjrzymy się w tym rozdziale czterem czynnikom, które należy wziąć pod uwagę przy ustalaniu priorytetów rozwoju nowych funkcjonalności.

1. Finansowa wartość funkcjonalności.
2. Koszt opracowania (i ewentualnie wsparcia) nowych funkcjonalności.
3. Ilość i znaczenie nowej wiedzy powstałej dzięki rozwinięciu nowych funkcjonalności.
4. Rozmiar zagrożeń usuniętych dzięki rozwinięciu funkcjonalności.

Ze względu na to, że większość projektów realizowana jest w celu zaoszczędzenia bądź zarobienia pieniędzy, dyskusje dotyczące ustalania priorytetów często są zdominowane przez pierwsze dwa czynniki. Jednak aby ustalać priorytety optymalnie, kluczowe znaczenie ma właściwie uwzględnianie wpływu nauki i zagrożeń związanych z realizacją projektu.

Wartość

Pierwszym czynnikiem branym pod uwagę przy ustalaniu priorytetów pracy jest finansowa wartość tematu. Ile pieniędzy zarobi lub zaoszczędzi firma dzięki uwzględnieniu w temacie nowych funkcjonalności? Kiedy właściciele produktu udzielają porady „ustalajcie priorytety zgodnie z wartością dla biznesu”, często biorą pod uwagę tylko ten jeden czynnik.

Nierzadko idealnym sposobem na określenie wartości tematu jest oszacowanie jego wpływu finansowego w określonym czasie — zazwyczaj w najbliższych kilku miesiącach, kwartałach lub latach. Można to zrobić, jeśli produkt — na przykład nowy edytor tekstu lub kalkulator z wbudowanym oprogramowaniem — będzie sprzedawany na zasadach komercyjnych. Można to również zrobić w odniesieniu do aplikacji, które będą wykorzystywane wewnątrz firm zajmujących się ich opracowaniem.

W rozdziale 10. „Priorytety finansowe” opisano różne podejścia do szacowania finansowej wartości tematów.

Ocena finansowego zwrotu z tematu może być trudna. Zazwyczaj obejmuje ona oszacowanie liczby nowych transakcji sprzedaży, średniej ceny sprzedaży (włącznie z transakcjami powiązanymi oraz umowami dotyczącymi wsparcia), terminów zwiększenia sprzedaży i tak dalej. Ze względu na złożoność dokonania tych ocen często warto wskazać alternatywną metodę oceny wartości. Ponieważ wartość tematu jest związana z jego atrakcyjnością dla nowych i istniejących użytkowników, możliwe jest stosowanie niefinansowych miar atrakcyjności w celu reprezentowania wartości. Zagadnienie to zostanie omówione w rozdziale 11. „Priorytety atrakcyjności”.

Koszt

Oczywiście koszty implementacji funkcjonalności to znaczący czynnik, który powinien być brany pod uwagę w ustalaniu ogólnych priorytetów funkcjonalności. Wiele funkcjonalności sprawia wrażenie

doskonałych, dopóki nie poznamy ich kosztów. Ważnym, choć często niedostrzeganym aspektem kosztów jest fakt, że z biegiem czasu ulegają one zmianom. Dodanie obsługi wielu wersji językowych dziś może zająć cztery tygodnie pracy, natomiast dodanie tej samej funkcjonalności za sześć miesięcy może zająć sześć tygodni. Stąd wniosek, że należy dodać tę obsługę dziś. Nieprawdaż? Być może. Załóżmy, że poświęciliśmy cztery tygodnie na to, by zrobić to teraz. W ciągu najbliższych sześciu miesięcy możemy poświęcić jeszcze trzy tygodnie na zmiany pierwotnej implementacji na podstawie wiedzy zdobytej w ciągu tych sześciu miesięcy. W tym przypadku okazało się, że lepiej byłoby poczekać. Może się również zdarzyć, że dziś poświęcimy na implementację cztery tygodnie, a później odkryjemy, że właściwsza byłaby prostsza i szybsza implementacja. Najlepszym sposobem na obniżenie kosztów zmiany jest implementacja funkcjonalności jak najpóźniej — co w rezultacie oznacza implementację jej wtedy, kiedy nie ma już więcej czasu na zmiany.

Tematy często wydają się uzasadnione tylko wtedy, gdy są postrzegane z perspektywy czasu, jaki zajmie ich implementacja. Choć to brzmi banalnie, ważne jest, aby pamiętać o tym, że czas kosztuje. Często najlepszym sposobem, aby o tym pamiętać podczas ustalania priorytetów, jest zgrubne przeliczenie punktów historyjek lub dni idealnych na pieniądze. Przypuśćmy, że dodałeś pensję wszystkim osobom zaangażowanym w projekt w ciągu ostatnich dwunastu tygodni i wysłała Ci kwota 150 000 zł. Obejmuje ona właściciela produktu i menedżera projektu, a także wszystkich programistów, testerów, inżynierów baz danych, analityków, projektantów interfejsu użytkownika i tak dalej. W ciągu tych dwunastu tygodni zespół zgromadził 120 punktów. Możemy stwierdzić, że przy całkowitym koszcie wynoszącym 150 000 zł 120 punktów kosztuje po 1250 zł każdy. Załóżmy, że właściciel produktu próbuje ustalić, czy w następnym wydaniu należy zaimplementować funkcjonalności warte trzydzieści punktów. Jednym ze sposobów, aby to zrobić, jest zadanie pytania, czy nowa funkcjonalność jest warta inwestycji 37 500 zł ($30 \times 1250 = 37\,500$).

W rozdziale 10. „Priorytety finansowe” opowiemy znacznie więcej na temat kosztów i ustalania priorytetów na podstawie zwrotu finansowego w porównaniu z kosztami.

Nowa wiedza

W wielu projektach znaczną część ogólnych wysiłków poświęca się pogoni za nową wiedzą. Jest istotne, aby ten wysiłek został potwierdzony i uznany za podstawowy dla projektu. Zdobywanie nowej wiedzy jest ważne, ponieważ na początku projektu nigdy nie wiemy wszystkiego, co będziemy musieli wiedzieć na końcu projektu. Wiedzę, którą zespół rozwija, można sklasyfikować w dwóch obszarach:

- wiedza o produkcie,
- wiedza o projekcie.

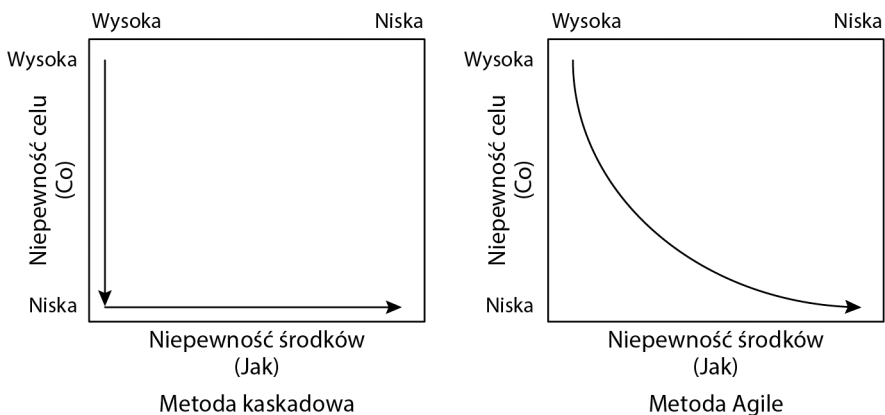
Wiedza o produkcie to informacje o tym, co będziemy tworzyć. Jest to znajomość tych funkcjonalności, które zostaną zawarte w produkcie, oraz tych, które nie zostaną w nim uwzględnione. Im większą wiedzę o produkcie ma zespół, tym właściwsze decyzje dotyczące natury i funkcjonalności produktu potrafi podejmować.

Dla odróżnienia wiedza o projekcie to znajomość tego, jak będzie tworzony produkt. Chodzi o używane technologie, kwalifikacje programistów, sposób funkcjonowania zespołu jako całości i tak dalej.

Konsekwencją zdobywania nowej wiedzy jest zmniejszanie niepewności. Na początku projektu istnieje pewien stopień niepewności co do tego, jakie funkcjonalności powinien zawierać nowy produkt. Istnieje również niepewność tego, w jaki sposób produkt będzie budowany. Laufer (1996) opisuje te

rodzaje niepewności jako *niepewność celu* (ang. *end uncertainty*) i *środków* (ang. *means uncertainty*). Niepewność celu można zmniejszyć poprzez zdobywanie coraz więcej informacji o produkcie. Niepewność środków zmniejsza się w wyniku zdobywania nowej wiedzy o projekcie.

W projekcie prowadzonym metodą kaskadową dąży się do tego, aby całą niepewność w związku z tym, co będzie budowane, wyeliminować przed rozstrzygnięciem tego, jak to będzie budowane. Stąd pochodzi ogólne zalecenie, aby analiza dotyczyła tego, co będzie tworzone, natomiast projekt tego, jak to będzie tworzone. Na rysunku 9.1 pokazano sposoby eliminowania niepewności zarówno w projektach prowadzonych metodą kaskadową, jak i w projektach Agile.



RYСУNEK 9.1. *Podejście tradycyjne i podejście Agile do zmniejszania niepewności — zaadaptowano z Laufer (1996)*

W części rysunku 9.1 dotyczącej projektów kaskadowych strzałka w dół pokazuje próby tradycyjnego zespołu wyeliminowania całej niepewności celu na początku projektu. Oznacza to, że zanim zespół przystąpi do pracy, nie ma żadnej niepewności co do celu, do jakiego dąży. Produkt jest całkowicie zdefiniowany. Strzałka w prawo w części rysunku 9.1 poświęconej projektowi kaskadowemu pokazuje, że niepewność środków (to, jak produkt będzie budowany) zmniejsza się wraz z czasem trwania projektu. Oczywiście całkowite wyeliminowanie z góry całej niepewności celu jest nieosiągalne. Klienci i użytkownicy nie są pewni tego, czego dokładnie chcą, dopóki nie zobaczą przynajmniej fragmentu produktu. Następnie mogą sukcesywnie komunikować swoje potrzeby. Spróbujmy porównać ten obraz z podejściem Agile do redukcji niepewności, które zaprezentowano w prawej części rysunku 9.1. Zespoły Agile przyznają, że na początku projektu nie jest możliwe wyeliminowanie całej niepewności związanej z tym, czym ma być produkt. Należy opracować fragmenty produktu, przedstawić je klientom, następnie zebrać opinie, skorygować je i dostosować plany. Do tego potrzebny jest czas. W tym czasie zespół zdobywa również nową wiedzę na temat sposobu, w jaki będzie pracować nad systemem. Prowadzi to do równoczesnego zmniejszania zarówno niepewności celu, jak i środków. Można to zaobserwować w części rysunku 9.1 dotyczącej podejścia Agile.

W części rysunku 9.1 dotyczącej projektów Agile narysowałem krzywą, aby zaprezentować preferencję dla jak najwcześniejszego zmniejszania niepewności celu. Dlaczego nie narysowałem prostej linii lub takiej, która obrazuje preferowanie wczesnej redukcji niepewności środków? Narysowałem linię w taki, a nie inny sposób, aby pokazać znaczenie jak najwcześniejszego zmniejszania niepewności co do tego, czym powinien być produkt. Niepewność celu nie powinna i nie może być wyeliminowana

na samym początku projektu (tak jak zakłada się w podejściu tradycyjnym). Jednak jednym z największych zagrożeń dla większości projektów jest ryzyko stworzenia nieodpowiedniego produktu. To ryzyko można znacznie zmniejszyć poprzez wczesną implementację tych funkcjonalności, które w jak najlepszy sposób pozwolą zaprezentować działające oprogramowanie rzeczywistym użytkownikom.

Zagrożenia

Z pojęciem nowej wiedzy jest ściśle związany ostatni czynnik w określaniu priorytetów: zagrożenia. Prawie z każdym projektem wiążą się liczne zagrożenia. Dla naszych celów zagrożeniem jest wszystko, co jeszcze się nie wydarzyło, ale może się wydarzyć, a co może uniemożliwić lub ograniczyć sukces projektu. Istnieje wiele różnych rodzajów zagrożeń w projektach, w tym:

- zagrożenie harmonogramu („Być może nie skończymy do października”);
- zagrożenie budżetu („Może nam się nie udać kupić sprzętu za odpowiednią cenę”);
- zagrożenie funkcjonalności („Może nam się nie udać zrealizować tej funkcji”).

Ponadto zagrożenia mogą być sklasyfikowane jako technologiczne i biznesowe.

W projektach występuje typowy dylemat między wyborem funkcjonalności o wysokich zagrożeniach a funkcjonalności o wysokiej wartości. Czy zespół projektowy powinien zacząć pracę od skoncentrowania się na funkcjonalnościach wysokiego ryzyka, które mogą zakłócić cały projekt? Czy też raczej powinien skupić się na funkcjonalnościach, które Tom Gilb (1988) określił jako „soczyste kęsy” — wysokiej wartości usługi, które przyniosą najszybsze zyski?

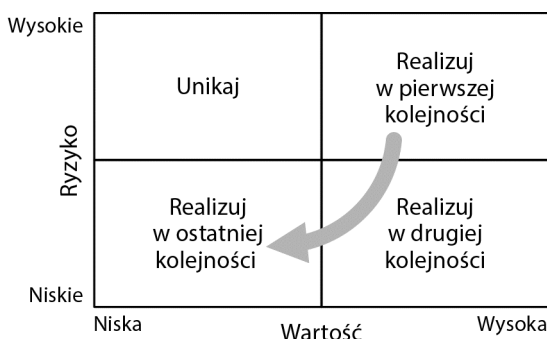
Aby dokonać tego wyboru, zastanówmy się nad tym, jakie są wady każdego podejścia. Zespół, którego główną motywacją są zagrożenia, akceptuje ryzyko, że zrealizowane prace mogą okazać się niepotrzebne lub prezentować niską wartość. Zespół mógł rozwinąć infrastrukturalne wsparcie dla funkcjonalności, które okazały się niepotrzebne, kiedy właściciel produktu sprecyzował swoją wizję projektu na podstawie tego, czego dowiedział się od użytkowników w trakcie trwania projektu. Z drugiej strony zespół, który koncentruje się na wartości w celu wykluczenia ryzyka, mógł rozwinąć znaczną część aplikacji, zanim natrafił na zagrożenie, które może uniemożliwić dostawę produktu.

Oczywistym rozwiązaniem jest wyeliminowanie stanu, w którym o priorytetach całkowicie decydują albo wartość, albo zagrożenia. Aby optymalnie ustalić priorytety pracy, trzeba uwzględnić zarówno zagrożenia, jak i wartość. Rozważmy rysunek 9.2, na którym pokazano zależność między zagrożeniami a wartością funkcjonalności w czterech ćwiartkach. W górnej prawej ćwiartce są funkcjonalności o wysokich zagrożeniach i wysokiej wartości. Te funkcjonalności są bardzo pożądane przez klientów, ale wiążą się z dużymi zagrożeniami podczas procesu projektowego. Być może funkcjonalności zawarte w tej ćwiartce bazują na niesprawdzonych technologiach, integracji z niesprawdzonymi podwykonawcami, nowinkach technicznych (na przykład opracowaniu nowego algorytmu) lub na jednym z wielu podobnych zagrożeń. W prawej dolnej ćwiartce wykresu mieszczą się funkcjonalności, które są równie pożądane, ale mniej ryzykowne. O ile funkcjonalności umieszczone w prawej części rysunku 9.2 są bardzo pożądane, o tyle te, które mieszczą się w lewej części, mają dla klientów mniejsze znaczenie.

Odpowiednie sekwencje rozwijania funkcjonalności pokazano na rysunku 9.3. W pierwszej kolejności powinny być implementowane funkcjonalności o wysokiej wartości i wysokich zagrożeniach. Te funkcjonalności dostarczają największej wartości, a praca nad nimi eliminuje znaczące zagrożenia.



RYSUNEK 9.2. Cztery ćwiartki relacji między ryzykiem a wartością



RYSUNEK 9.3. Połączenie ryzyka z wartością podczas ustalania priorytetów funkcjonalności

W następnej kolejności powinny być implementowane funkcjonalności o wysokiej wartości i niewielkim ryzyku. Te funkcjonalności dostarczają równie wysokiej wartości, co pierwsza grupa, ale wiążą się z mniejszym ryzykiem. Dlatego można je umieścić później w harmonogramie. Z tego powodu należy stosować się do wskazówki, aby najpierw pracować nad funkcjonalnościami o wysokiej wartości, ale używać ryzyka jako czynnika do eliminowania wątpliwości.

W dalszej kolejności są realizowane funkcjonalności o niskiej wartości i niewielkim ryzyku. Są one umieszczone na trzecim miejscu w kolejce, ponieważ ich odrzucenie będzie miało mniejszy wpływ na całkowitą wartość produktu oraz dlatego, że wiążą się z niskim ryzykiem.

Na koniec należy unikać funkcjonalności, które dostarczają niewielkiej wartości, ale wiążą się z dużym ryzykiem. Należy odkładać pracę nad funkcjonalnościami o niskiej wartości, zwłaszcza tymi, które jednocześnie wiążą się z dużym ryzykiem. Należy dążyć do jak najpóźniejszej realizacji funkcjonalności, które przynoszą niską wartość i wiążą się z dużym ryzykiem. Nie ma powodu, aby podejmować duże ryzyko w celu zaimplementowania funkcjonalności o ograniczonej wartości. Należy pamiętać, że profil zagrożeń i wartości funkcjonalności zmienia się z upływem czasu. Funkcjonalność o niskiej wartości i niskim ryzyku, którą dziś umieszczamy na rysunku 9.3 w ćwiartce *Unikaj*, za sześć miesięcy, gdy prace nad innymi funkcjonalnościami zostaną zakończone, może się znaleźć w ćwiartce *Realizuj w pierwszej kolejności*.

Połączenie czterech czynników

W celu połączenia czterech czynników mających wpływ na ustalenie priorytetów należy pomyśleć przede wszystkim o wartości funkcjonalności względem obecnych kosztów jej zaimplementowania. W ten sposób można ustalić początkową kolejność priorytetów dla tematów. Tematy o wysokiej wartości współczynnika wartość-koszt są tymi, które należy zrealizować w pierwszej kolejności.

Następnie należy wziąć pod uwagę inne czynniki ustalania priorytetów i na ich podstawie przesuwając tematy w górę lub w dół. Założmy, że z uwagi na wartość i koszty określonego tematowi przydzielono średni priorytet. Z tego powodu zespół zacznie pracę nad tym tematem mniej więcej w połowie prac nad bieżącym wydaniem. Jednak technologie niezbędne do zrealizowania tej funkcjonalności wiążą się z bardzo dużym ryzykiem. To sprawi, że temat zostanie przesunięty w górę listy priorytetów oraz w harmonogramie.

Proces tworzenia tego wstępnego rankingu oraz następującego później przesuwania priorytetów w górę lub w dół nie musi być formalnym przedsięwzięciem. Może (i często tak się dzieje) odbywać się wyłącznie w głowie właściciela produktu. Następnie właściciel produktu zwykle prezentuje swoje priorytety zespołowi, który może go nakłonić do pewnych modyfikacji priorytetów na podstawie własnej oceny tematów.

Przykłady

Aby zyskać pewność, że wymienione cztery czynniki ustalania priorytetów są sensowne i przydatne, przyjrzyjmy się, w jaki sposób można je zastosować do dwóch typowych zadań ustalania priorytetów: projektu infrastruktury i interfejsu użytkownika. W poniższych punktach przeanalizuję temat i pokażę sposób zastosowania wymienionych wcześniej czynników ustalania priorytetów.

Infrastruktura

Jednym z popularnych wyzwań związanych z ustalaniem priorytetów jest opracowanie infrastruktury lub elementów architektury aplikacji. Jako przykład rozważmy używany w aplikacji framework zabezpieczeń. Biorąc pod uwagę wyłącznie wartość, jaką ten framework dostarcza klientom, jest mało prawdopodobne, aby otrzymał priorytet we wczesnych iteracjach projektu. Ostatecznie, pomimo tego, że względy bezpieczeństwa są niezwykle ważne dla wielu aplikacji, z reguły klienci nie kupują większości aplikacji głównie dlatego, że są one bezpieczne. Zanim bezpieczeństwo aplikacji stanie się istotne, musi ona coś zrobić.

Kolejnym czynnikiem branym pod uwagę przy ustalaniu priorytetów są koszty. Dodanie frameworka zabezpieczeń do witryny internetowej dziś prawdopodobnie będzie kosztować mniej niż dodanie tego samego frameworka później. Podobnie jest w przypadku wielu elementów infrastruktury, co staje się podstawą dla wielu argumentów przemawiających za rozwojem funkcjonalności we wczesnej fazie projektu. Jeśli jednak funkcjonalność zostanie zaimplementowana wcześniej, to istnieje prawdopodobieństwo, że zmieni się, zanim projekt się skończy. Koszty tych zmian trzeba wziąć pod uwagę we wszystkich decyzjach dotyczących implementacji funkcjonalności teraz lub później. Ponadto wprowadzenie frameworka zabezpieczeń we wczesnej fazie projektu może wprowadzić złożoność, która będzie się wiązała z ukrytymi kosztami wszystkich prac wykonywanych w przyszłości. Te koszty również należy wziąć pod uwagę.

Kolejna reguła ustalania priorytetów mówi, że należy przyspieszyć rozwijanie funkcjonalności, które generują nową wiedzę na temat produktu lub projektu. W przypadku większości produktów istnieje niewielkie prawdopodobieństwo, że wdrożenie frameworka zabezpieczeń spowoduje wygenerowanie istotnej nowej wiedzy o produkcie. Jednak zaimplementowanie frameworka zabezpieczeń może wygenerować nową wiedzę o projekcie. Na przykład kilka lat temu pracowałem nad projektem, w którym była potrzeba uwierzytelniania użytkowników za pomocą serwera LDAP. Żaden z deweloperów nie robił wcześniej niczego podobnego, dlatego istniała spora niepewność co do tego, ile wysiłku trzeba będzie włożyć w implementację tej funkcjonalności. W celu wyeliminowania owej niepewności historyjki użytkownika dotyczące uwierzytelniania LDAP przeniesiono z końca harmonogramu w okolice jego środka.

Ostatnim czynnikiem branym pod uwagę przy ustalaniu priorytetów są zagrożenia. Czy istnieje zagrożenie dla sukcesu projektu, które może być zredukowane lub wyeliminowane dzięki implementacji zabezpieczeń wcześniej zamiast później? Być może nie w tym przykładzie. Jednak niepowodzenie implementacji frameworka, kluczowego komponentu lub innej infrastruktury często stwarza znaczące zagrożenie dla projektu. Może to być wystarczające do tego, aby przenieść implementację tego komponentu wcześniej, niż by to wynikało wyłącznie z uwagi na wartość.

Projekt interfejsu użytkownika

W projektach Agile powszechnie stosuje się zalecenia, aby projekt interfejsu użytkownika był w całości zaimplementowany w iteracji, w ramach której jest implementowana funkcjonalność związana z tym interfejsem. Czasami jednak takie podejście przeczy poglądom, zgodnie z którymi użyteczność systemu poprawia się wówczas, gdy projektanci mogą z góry myśleć o ogólnym interfejsie użytkownika. Czego możemy się nauczyć, stosując w praktyce czynniki ustalania priorytetów?

Po pierwsze, czy tworzenie interfejsu użytkownika generuje istotną, przydatną nową wiedzę? Jeśli tak, to powinniśmy przenieść część pracy bliżej początku harmonogramu. Prawdą jest, że w wielu przypadkach tworzenie niektórych głównych komponentów interfejsu użytkownika lub modeli nawigacyjnych generuje istotną, przydatną nową wiedzę o produkcie. Rozwój części interfejsu użytkownika na początku projektu pozwala zaprezentować system w początkowej formie rzeczywistym lub prawdopodobnym użytkownikom. Opinie tych użytkowników są źródłem nowej wiedzy o produkcie. Można z niej skorzystać po to, aby zyskać pewność, że zespół pracuje nad możliwie najbardziej wartościowym produktem.

Po drugie, czy implementacja interfejsu użytkownika przyczynia się do zmniejszenia ryzyka? Prawdopodobnie nie wyeliminuje zagrożeń technicznych (chyba że jest to dla zespołu pierwsza próba implementacji tego typu interfejsu użytkownika). Jednak wczesny rozwój funkcjonalności, które prezentują interfejs użytkownika, często prowadzi do zredukowania poważnego zagrożenia, z którym boryka się większość projektów: ryzyka stworzenia niewłaściwego produktu. Nadawanie wysokiego priorytetu cechom funkcjonalnym, które prezentują istotne funkcjonalności komunikujące się z użytkownikiem, pozwala na wcześniejsze uzyskanie opinii od użytkowników. To najlepszy sposób, aby uniknąć ryzyka stworzenia nieprawidłowego produktu.

Dodatkowym punktem przemawiającym na rzecz wczesnej implementacji interfejsu użytkownika są znacznie niższe koszty w przypadku, gdy implementacja interfejsu jest realizowana we wczesnej fazie projektu. W większości sytuacji nie ma takiej zależności.

Zatem ze względu na dodatkową wiedzę oraz redukcję zagrożeń wydaje się sensowne przeniesienie bliżej początku harmonogramu tych tematów, które umożliwiają użytkownikom dostarczenie większości opinii na temat użyteczności i funkcjonalności systemu. To nie oznacza, że należy pracować nad interfejsem użytkownika w izolacji lub oddzielnie od funkcjonalności, które istnieją pod interfejsem użytkownika. Oznacza to natomiast, że byłoby wskazane przeniesienie na wcześniejsze terminy w harmonogramie realizacji funkcjonalności obejmujących istotne komponenty interfejsu użytkownika, które pozwolą uzyskać jak najbardziej przydatne opinie od klientów i użytkowników.

Podsumowanie

Ponieważ rzadko jest wystarczająco dużo czasu, aby zaimplementować wszystko, istnieje potrzeba ustalenia priorytetów decydujących o tym, nad którymi funkcjonalnościami należy pracować w pierwszej kolejności. Podczas ustalania priorytetów należy wziąć pod uwagę cztery główne czynniki, którymi są:

1. Finansowa wartość implementacji tych funkcjonalności.
2. Koszty utworzenia (i ewentualnie wsparcia) nowych funkcjonalności.
3. Ilość i znaczenie nowej nauki i wiedzy powstałych dzięki utworzeniu nowych funkcjonalności.
4. Rozmiar zagrożeń usuniętych dzięki rozwinięciu funkcjonalności.
5. Czynniki te łączymy, myśląc najpierw o wartości i kosztach tematu. Dzięki temu tematy są uporządkowane zgodnie z pierwotną kolejnością. Następnie można przenosić tematy w przód i wstecz harmonogramu na podstawie innych czynników.

Pytania do dyskusji

1. Funkcjonalność w projekcie, do którego Cię przydzielono, ma dość niski priorytet. Teraz wygląda na to, że zostanie zaimplementowana w bieżącym wydaniu, ale może się okazać, że zostanie pominięta, jeśli będzie brakowało czasu. Funkcjonalność musi zostać zaimplementowana w języku, którego nie zna nikt w zespole. Co zrobisz?
2. Jakie rodzaje wiedzy na temat produktu i projektu zdobył Twój obecny zespół od początku projektu? Czy istnieje dodatkowa wiedza, którą trzeba szybko zdobyć oraz którą należy uwzględnić w priorytetach projektu?

ROZDZIAŁ 10.

Priorytety finansowe

*Jest taka ogólna zasada, że jeśli nikt nie liczy korzyści,
to można założyć, że w ogóle ich nie ma.*

— Tom DeMarco i Timothy Lister

Niektóre projekty są inicjowane w celu uzyskania przychodów, inne aby zmniejszyć koszty, a niektóre dążą do osiągnięcia obu tych celów jednocześnie. Jeśli można oszacować kwotę pieniędzy, które zostaną zarobione lub zaoszczędzone dzięki realizacji każdego tematu, to można skorzystać z tego oszacowania w celu ustalenia priorytetów. Prognozowanie wartości finansowej tematu należy do obowiązków właściciela produktu, ale jest to odpowiedzialność współdzielona z innymi członkami zespołu — programistami, testerami, analitykami, menedżerami projektu i tak dalej. W większości przypadków właściciel produktu powinien również wyciągać wnioski na podstawie wiedzy biznesowej i prognozować na podstawie danych z działów sprzedaży i marketingu.

Osobiście, gdy mam ocenić wartość finansową tematu, to zwołuję spotkanie, w którym uczestniczy tyle osób spośród wspomnianego wyżej grona, ile trzeba. Celem takiego spotkania dotyczącego oceny tematów jest wypełnienie dla każdego tematu formularza podobnego do tego, który pokazano w tabeli 10.1. W zależności od liczby tematów i uczestników spotkania może to potrwać więcej niż jedno spotkanie.

Tabela 10.1 zawiera wiersz dla każdego kwartału w ciągu najbliższych dwóch lat. Wybór horyzontu czasowego zależy od zespołu. W niektórych przypadkach zespoły analizują miesięczny dochód w ciągu jednego lub dwóch lat. W innych sytuacjach odpowiednie są prognozy kwartalne. Z moich doświadczeń wynika, że dla większości projektów dobrze się sprawdza analiza dwóch lat. Taka perspektywa stanowi odpowiednią równowagę między odgadywaniem odległej przyszłości a wybieganiem w przód dostatecznie daleko. Tak długi okres jest potrzebny ze względu na to, że projekty wytwarzania oprogramowania charakteryzują się dużą dozą niepewności. Moje obserwacje potwierdzają badania innych (Bills 2004a).

W tabeli 10.1 zawarto również kolumny dla różnych typów zwrotu, jaki możemy uzyskać z realizacji tematu. Jeśli projekt przynosi różne rodzaje zwrotów, to należy zastosować różne nagłówki kolumn. Na podobnej zasadzie należy użyć różnych nagłówków kolumn, jeśli do tematu będą potrzebne bardziej szczegółowe nazwy kolumn (na przykład *Wzrost przychodów od klientów z USA* lub *Wzrost dochodów od europejskich konsumentów*). Nie trzeba oceniać wszystkich tematów z wykorzystaniem takiego samego zestawu kolumn.

TABELA 10.1. Pusty arkusz analizy zwrotu z tematów

Kwartał	Przychody nowe	Przychody przyrostowe	Przychody zatrzymane	Sprawność operacyjna
1				
2				
3				
4				
5				
6				
7				
8				

Uczestnicy spotkania wypełniają arkusz poprzez ocenę wartości w komórce, na którą ich zdaniem będzie miało wpływ zrealizowanie tematu. Przykładowy, wypełniony arkusz zwrotu z tematów zaprezentowano w tabeli 10.2. W tym przypadku dostępność tematu będzie przyciągać nowych klientów (*Przychody nowe*), a także doprowadzi do wzrostu przychodów od dotychczasowych klientów (*Przychody przyrostowe*). Nowy temat nie wywrze żadnego wpływu na przychody zatrzymane ani na sprawność operacyjną.

Skąd się wzięły te liczby? Byłoby najlepiej, gdyby pochodziły z badania rynku wykorzystanego w przedsięwzięciu biznesowym, które spowodowało zainicjowanie projektu. Minimalnym wymaganiem jest to, aby osoba, która zażądała realizacji tematu, potrafiła określić powody jego realizacji.

Nie można porównywać projektów i podejmować decyzji dotyczących priorytetów wyłącznie przez dodanie wszystkich wierszy z tabeli dla każdego tematu tak jak w tabeli 10.2. Strumień przychodów, który zapewnia 100 000 □ w pierwszym kwartale, 200 000 □ w następnym i 500 000 □ w trzecim, jest znacznie mniej wartościowy niż strumień zapewniający te same zwroty, ale w odwrotnej kolejności. Aby porównać kilka tematów, należy użyć jednego lub kilku standardowych wskaźników finansowych. W tym rozdziale przyjrzymy się następującym wskaźnikom:

- wartość bieżąca netto (ang. *net present value* — **NPV**),
- wewnętrzna stopa zwrotu (ang. *internal rate of return* — **IRR**),
- okres zwrotu (ang. *payback period* — **PP**),
- zdyskontowany okres zwrotu (ang. *discounted payback period* — **DPP**).

Zanim jednak przyjrzymy się tym wskaźnikom finansowym, powinniśmy się zastanowić, w jaki sposób projekty generują dochody bądź oszczędności.

Źródła zwrotu

Zwrot z projektu może pochodzić z różnych źródeł. Dla ułatwienia możemy sklasyfikować je jako przychody nowe, przychody przyrostowe, przychody zatrzymane i sprawność operacyjną. Chociaż powszechna jest sytuacja, w której jedna kategoria dominuje zwrot dla konkretnego projektu, w większości projektów przychody pochodzą z więcej niż jednej kategorii.

TABELA 10.2. Przykładowy arkusz zwrotu z tematów

Kwartał	Przychody nowe	Przychody przyrostowe	Przychody zatrzymane	Sprawność operacyjna
1	25 000 zł	20 000 zł	0	0
2	35 000 zł	30 000 zł	0	0
3	50 000 zł	40 000 zł	0	0
4	70 000 zł	60 000 zł	0	0
Razem	180 000 zł	150 000 zł	0	0

Przychody nowe

Z pewnością jednym z najbardziej powszechnych składników zwrotu z inwestycji w projekt jest okazja osiągnięcia *przychodów nowych*. Bardzo niewiele firm jest zadowolonych ze swojego udziału w rynku. Większość firm dąży do pozyskania nowych klientów. Nawet jeśli produkt-oprogramowanie nie jest sprzedawany bezpośrednio, to dodanie nowych funkcjonalności może dać nowe przychody. Na przykład pracowałem w firmie, która tworzyła oprogramowanie dla potrzeb obsługi klientów szpitala. W pewnym momencie nasz dyrektor uświadomił sobie, że przy pewnych ulepszeniach nasze oprogramowanie może być użyte w celu zapewnienia tych samych usług firmom ubezpieczeniowym. Wprowadziliśmy zmiany i dzięki nowym funkcjonalnościom dostarczonym przez oprogramowanie udało się stworzyć zupełnie nowe źródło przychodów w firmie.

Przychody przyrostowe

Często przydaje się rozróżnienie przychodu od nowych klientów od dodatkowych, przyrostowych przychodów od dotychczasowych klientów. *Przychody przyrostowe* mogą zostać osiągnięte ze względu na to, że nowy system lub produkt:

- Zachęca istniejących klientów do zakupu dodatkowych licencji.
- Zawiera opcjonalne, dodatkowe moduły, które mogą być sprzedawane oddzielnie.
- Zawiera funkcjonalności, które pozwalają na ustalanie wyższych cen.
- Zachęca do korzystania z usług doradczych (na przykład w celu integracji z oddzielną, zewnętrzną aplikacją).

Przychody zatrzymane

Oddzielne od przychodów nowych i przyrostowych są przychody zatrzymane. *Przychody zatrzymane* dotyczą przychodów, które firma straci, jeśli projekt lub temat nie będzie rozwijany. Załóżmy, że udało się nam pomyślnie sprzedać indywidualnym kręgarzom produkt umożliwiający planowanie wizyt pacjentów. Interes niektórych Twoich klientów dobrze się rozwija, dlatego postanowili zatrudnić w swojej firmie jednego lub kilku kręgarzy. Jeśli oprogramowanie nie zostanie ulepszone w taki sposób, aby umożliwiała planowanie wizyt pacjentów u wielu kręgarzy, to stracimy klientów reprezentujących

te rozwijające się praktyki. Projekt dodania nowych funkcjonalności pozwala firmie zatrzymać te przychody. Co ciekawe, może również powstać okazja do osiągnięcia przyrostowych przychodów, ponieważ możemy żądać więcej za wersję oprogramowania, które obsługuje kilku kręgarzy, nie tylko jednego.

Sprawność operacyjna

Żadna firma nigdy nie jest tak sprawna, jak by mogła być. Zawsze istnieją pewne zadania, które mogłyby zostać uproszczone lub wyeliminowane. Jeśli tworzysz oprogramowanie, które ma być używane przez klientów wewnętrznych, to prawdopodobnie jesteś w pełni świadomy znaczenia sprawności operacyjnej. Jednak nawet jeśli pracujesz nad oprogramowaniem komercyjnym, które jest sprzedawane osobom spoza firmy, to niektóre zadania w ramach projektu nadal mogą się przyczyniać do poprawy skuteczności operacyjnej. W takim przypadku najczęściej będzie się to odnosić do poprawy Twojej własnej niesprawności. Na przykład w wielu projektach, w których uczestniczyłem, podejmowano decyzję o opracowaniu własnego narzędzia do mapowania obiektowo-relacyjnego w celu uproszczenia mapowania obiektów języka oprogramowania na tabele relacyjnej bazy danych. Podobnie, praktycznie w każdym projekcie, w którym uczestniczyłem, opracowywaliśmy jakąś formę narzędzia wspomagającego wytwarzanie oprogramowania.

Często dążenie do zwiększenia sprawności operacyjnej wynika z przewidywanego wzrostu. Nieskuteczność, która dziś nie stanowi problemu, szybko przeradza się w problem, gdy firma staje się znacznie większa. Dla przykładu założmy, że Twoja firma opracowała witrynę internetową do sprzedaży ramek do zdjęć. Sprzedajesz ogromne ilości ramek standardowych, ale sprzedajesz również ramki o nietypowych rozmiarach. Biznes idzie dobrze, a firma przewiduje silny wzrost w ciągu najbliższych dwóch lat. Mówiąc dokładniej, oczekuje w tym okresie dziesięciokrotnego wzrostu sprzedaży. Tak jak bywa w każdym biznesie, pewien procent sprzedanych towarów jest odsyłany z powrotem do firmy jako towar zwrócony. Posiadanie wysoco zautomatyzowanego, skutecznego rozwiązania do obsługi zwrotów nigdy nie było priorytetem firmy. Obecnie obsługa zwrotów łącznie z aktualizacją stanów magazynowych i zwrotem zapłaty na rachunek karty kredytowej nabywców zajmuje jednej osobie około dwóch godzin dziennie. Dwie godziny jednej osoby nie są dziś zbyt wielkim problemem. Ale kiedy sprzedaż wzrośnie o 1000%, wtedy liczba zwracanych towarów prawdopodobnie także wzrośnie o 1000%. W takim przypadku obsługa zwrotów zajmie dwadzieścia godzin roboczych każdego dnia. W tym momencie na pewno warto się zastanowić, czy taka niesprawność operacyjna zasługuje na zajęcie się tematem.

Do obszarów wartych zajęcia się problemami sprawności operacyjnej należą:

- wszystkie działania, które zajmują dużo czasu lub które zajmowałyby dużo czasu, gdyby firma się rozwinęła;
- działania zmierzające do lepszej integracji i komunikacji między działami;
- zmniejszenie rotacji pracowników;
- skrócenie czasu szkolenia nowych pracowników;
- wszystkie procesy wrażliwe na czas;
- łączenie wielu procesów;
- wszystko, co zwiększa dokładność i zmniejsza konieczność wprowadzania poprawek.

Przykład: WebPayroll

Spróbujmy zastosować wymienione powyżej techniki w celu oszacowania zwrotu z inwestycji w projekt. Załóżmy, że nasza firma — WebPayroll — oferuje firmom, które są zbyt małe, aby samodzielnie obliczać podatki od wynagrodzeń, drukować czeki i tak dalej, webową aplikację do tworzenia listy płac. Dotychczas nieźle nam się wiedzie, ale usprawniamy oprogramowanie, aby poprawić czas przetwarzania.

Obecnie informujemy klientów, że muszą wprowadzić informacje o zarobkach na naszej stronie internetowej na trzy dni przed planowaną dystrybucją czeków do pracowników. Celem nowego systemu jest zaoferowanie wyników na następny dzień od wprowadzenia danych. Jeśli informacje o zarobkach zostaną wprowadzone w witrynie WebPayroll do 17.00 po południu, to będziemy w stanie wygenerować czeki, wydrukować je i dostarczyć przez noc. Czeki zostaną dostarczone do rąk naszych klientów następnego dnia rano.

Przed przystąpieniem do oceny opłacalności zwrotu z tego projektu musimy ocenić, kiedy nowa wersja oprogramowania będzie dostępna. Załóżmy, że deweloperzy już oszacowali historyjki należące do tego tematu na 150 punktów. Biorąc pod uwagę wcześniejszą szybkość zespołu wynoszącą 20 punktów w ciągu dwutygodniowej iteracji, wychodzi, że realizacja historyjki zajmie $150/20 = 7,5$, czyli w przybliżeniu 8 iteracji. Oznacza to, że wzrost przychodów i poprawa sprawności operacyjnej mogą się rozpocząć po 8 iteracjach (chyba że uda nam się znaleźć sposób na dostarczenie częściowego rozwiązania, co zawsze powinno być naszym celem).

Obliczanie nowych przychodów

Zaoferowanie usługi „na następny dzień” zamiast „za trzy dni” otworzy przed firmą nowe możliwości dochodów. Aby ocenić te możliwości, najpierw oszacujemy liczbę nowych klientów, których zdobędziemy. Nie mamy żadnych twardych danych. Ale nasz główny sprzedawca, Teresa, twierdzi, że około jednej trzeciej klientów, z którymi rozmawia, odrzuca ofertę aplikacji WebPayroll z powodu wymogu wprowadzania danych z trzydniowym wyprzedzeniem. Na podstawie aktualnych prognoz sprzedaży Teresa uważa, że może przyciągnąć w tym roku pięćdziesięciu nowych klientów na kwartał, a następnie, w przyszłym roku, stu nowych klientów na kwartał. Te wartości dodajemy do kolumny *Nowi klienci* z tabeli 10.3. Mimo że funkcjonalność obliczeń przez noc nie będzie dostępna do połowy drugiego kwartału, Teresa uważa, że nadal uda jej się zdobyć w tym kwartale pięćdziesięciu nowych klientów.

Następnie oceniamy wartość przychodu na jednego klienta. Możemy to zrobić, myśląc o prawdopodobnych nowych klientach w porównaniu z obecnymi. Wiemy na przykład, że przeciętny klient aplikacji WebPayroll płaci nam rocznie 400 zł opłat abonamentowych. Uważamy jednak, że oferta dostarczenia wyników przez noc będzie najbardziej atrakcyjna dla małych firm — tych, które płacą nam średnio 200 zł rocznie. Uważamy, że od każdego z tych klientów uda nam się zarobić kolejnych 100 zł rocznie. Całkowita wartość każdego nowego klienta wynosi zatem 300 zł na rok lub 75 zł na kwartał. Ponieważ usługa dostawy na następny dzień będzie dostępna tylko przez dwie trzecie drugiego kwartału, przychody na jednego klienta w tym kwartale są odpowiednio zmniejszone. Obliczone wartości dodajemy do kolumny *Przychód na klienta* w tabeli 10.3. W ten sposób możemy również obliczyć wartości w kolumnie *Przychody nowe*.

TABELA 10.3. Prognozowane nowe przychody z projektu WebPayroll

Kwartał	Nowi klienci	Przychód na klienta	Przychody nowe
1	0	0 zł	0 zł
2	50	50 zł	2500 zł
3	50	75 zł	3750 zł
4	50	75 zł	3750 zł
5	100	75 zł	7500 zł
6	100	75 zł	7500 zł
7	100	75 zł	7500 zł
8	100	75 zł	7500 zł

Obliczanie przychodów przyrostowych

Przychody przyrostowe (ang. *incremental revenue*) to dodatkowe wpływy od istniejących klientów. Na podstawie tego, co wiemy o naszych obecnych klientach, jak często spóźniają się z dostarczaniem danych do generowania listy płac i tak dalej, oceniamy, że zdołamy pozyskać około 100 klientów na kwartał do stanu, w którym 400 naszych obecnych klientów będzie korzystało z usługi obliczeń na następny dzień. Tak jak w przypadku nowych klientów, po udostępnieniu usługi po upływie miesiąca drugiego kwartału usługa będzie generowała około 100 zł rocznie lub 25 zł na kwartał. W celu obliczenia na podstawie tych ocen łącznej wartości przyrostowych przychodów w poszczególnych kwartałach utworzyliśmy tabelę 10.4.

TABELA 10.4. Prognozowane przychody przyrostowe z projektu WebPayroll

Kwartał	Klienci	Przychód na klienta	Przychody przyrostowe
1	0	0 zł	0 zł
2	100	16 zł	1600 zł
3	200	25 zł	5000 zł
4	300	25 zł	7500 zł
5	400	25 zł	10 000 zł
6	400	25 zł	10 000 zł
7	400	25 zł	10 000 zł
8	400	25 zł	10 000 zł

Obliczanie przychodów zatrzymanych

Przychody zatrzymane (ang. *retained revenue*) to kwota, której już nie stracimy ze względu na to, że klienci nie są zadowoleni z naszego produktu, ponieważ nie spełnia już ich oczekiwań albo z innych

względów postanowili zrezygnować z naszego systemu. Obecnie firma nie dysponuje żadnymi dobrymi wskaźnikami umożliwiającymi śledzenie tych wartości. Wiemy, że to zaczyna być problem, który w ciągu najbliższych kilku lat będzie o wiele bardziej znaczący.

Oceniśmy, że dzięki udostępnieniu usługi obliczeń na następny dzień zapobiegniemy utracie dwudziestu klientów na kwartał w pierwszym roku i czterdziestu klientów na kwartał w drugim roku. Nie bez znaczenia jest również to, że ci klienci będą nadal korzystać z aplikacji WebPayroll, pomimo że nowa funkcjonalność nie będzie dostępna wcześniej niż w drugim kwartale. Oznacza to, że korzyści z dostarczenia funkcjonalności obliczeń przez noc zaczniemy odczuwać już w pierwszym kwartale pomimo tego, że funkcjonalność ta stanie się dostępna dopiero w drugim kwartale.

Ponieważ każdy obecny klient jest wart 400 zł, to na jeden kwartał przychody z jednego klienta wynoszą 100 zł. Na podstawie tych danych możemy obliczyć przychody zatrzymane. Obliczenia te zestawiono w tabeli 10.5.

TABELA 10.5. Prognozowane przychody zatrzymane z projektu WebPayroll

Kwartał	Klienci zatrzymani	Przychód na klienta	Przychody zatrzymane
1	20	100 zł	2000 zł
2	20	100 zł	2000 zł
3	20	100 zł	2000 zł
4	20	100 zł	2000 zł
5	40	100 zł	4000 zł
6	40	100 zł	4000 zł
7	40	100 zł	4000 zł
8	40	100 zł	4000 zł

Obliczanie sprawności operacyjnej

Aby projekt obliczeń z dnia na dzień zakończył się sukcesem, trzeba wyeliminować praktycznie wszystkie ręczne operacje, na których system dziś bazuje. Obecnie w systemie WebPayroll jest stanowisko kontrolera listy płac — pełniąc ją osoba sprawdza poprawność danych płacowych, a następnie przesyła je ręcznie, wykonując kilka kroków przepływu pracy. W firmie są zatrudnione dwie osoby na tym stanowisku.

Bez wprowadzenia funkcji obliczeń z dnia na dzień należałoby zatrudnić dwóch dodatkowych kontrolerów w połowie tego roku oraz kolejnych dwóch w połowie przyszłego roku. Ze względu na poprawę sprawności wynikającą z wprowadzenia projektu obliczeń z dnia na dzień spodziewamy się, że uda nam się wyeliminować po jednym z tych stanowisk każdego roku.

Kontrolerzy listy płac zarabiają średnio 20 000 zł rocznie. Jednak każdy z nich zajmuje miejsce w biurze, jest użytkownikiem kilku egzemplarzy sprzętu i oprogramowania oraz otrzymuje bonusy. W sumie te dodatkowe, ukryte koszty stanowią około 50% wynagrodzenia pracownika, czyli realne koszty utrzymania kontrolera listy płac są bliskie 30 000 zł rocznie. Są to tzw. pełne koszty pracy.

Liczba kontrolerów jeszcze niezatrudnionych oraz pełne koszty pracy pomnożone przez liczbę kwartałów pozwalają obliczyć całkowite oszczędności sprawności operacyjnej związanej z projektem, tak jak pokazano w tabeli 10.6.

TABELA 10.6. Prognozowane przychody ze sprawności operacyjnej związanej z projektem WebPayroll

Kwartał	Niepotrzebne wynagrodzenia kontrolerów listy płac	Pełne koszty pracy	Sprawność operacyjna
1	0	0	0
2	0	0	0
3	1	7500 zł	7500 zł
4	1	7500 zł	7500 zł
5	1	7500 zł	7500 zł
6	1	7500 zł	7500 zł
7	2	7500 zł	15 000 zł
8	2	7500 zł	15 000 zł

Szacowanie kosztów wytwarzania oprogramowania

Aby uzupełnić profil inwestycyjny usługi z dnia na dzień w projekcie WebPayroll, należy ocenić oczekiwany koszt implementacji tematu. W tym celu przyjrzymy się wynagrodzeniom wszystkich osób uczestniczących w projekcie, tak jak pokazano w tabeli 10.7.

TABELA 10.7. Zespół projektu WebPayroll

Rola	Roczne wynagrodzenie	Pełne koszty pracy	Pełne koszty każdej iteracji	Czas w projekcie	Skorygowany koszt na iterację
Właściciel produktu	50 000 zł	75 000 zł	2900 zł	100%	2900 zł
Programista	50 000 zł	75 000 zł	2900 zł	100%	2900 zł
Programista	30 000 zł	45 000 zł	1700 zł	50%	850 zł
Analityk	40 000 zł	60 000 zł	2300 zł	100%	2300 zł
Tester	30 000 zł	45 000 zł	1700 zł	100%	1700 zł
Tester	50 000 zł	75 000 zł	2900 zł	100%	2900 zł
				Razem	13 550 zł

W obliczeniach pełnych kosztów pracy w tabeli 10.7 uwzględniono kwoty o 50% wyższe niż te, które wynikają z samych wynagrodzeń. Ze względu na to, że iteracje trwają dwa tygodnie, koszt jednej iteracji wynosi 1/26 pełnych kosztów pracy. Kolumna *Czas w projekcie* pokazuje, jaką część czasu

każdy członek zespołu poświęca na pracę nad projektem. Wszyscy, z wyjątkiem jednego programisty, pracują w pełnym wymiarze godzin. W kolumnie *Skorygowany koszt na iterację* zamieszczono koszty związane z utrzymaniem każdego członka zespołu na podstawie pełnych kosztów pracy oraz ilości czasu poświęconego na pracę w projekcie. W sumie koszty utrzymania zespołu zgodnie z zestawieniem w tabeli 10.7 wynoszą 13 550 zł na iterację. Tę kwotę zaokrąglimy do 13 500 zł.

Często warto wiedzieć, ile wynosi cena jednego punktu (lub dnia idealnego). Aby obliczyć tę wartość, należy podzielić wartość skorygowanych kosztów na iterację przez średnią lub oczekiwaną szybkość zespołu. Ze względu na to, że zespół projektu WebPayroll pracuje ze średnią szybkością 20 punktów na iterację, to jego koszt punktu historyjki wynosi $13\,500/20 = 675$. Ta informacja jest przydatna z tego względu, że jeśli ktoś zada pytanie o to, jakie będą koszty implementacji historyjki oszacowanej na 100 punktów, to będziemy mogli odpowiedzieć: koszty wyniosą 67 500 (100×675) zł.

Koszty utrzymania zespołu projektu WebPayroll można podsumować tak jak pokazano w tabeli 10.8.

TABELA 10.8. Podsumowanie kosztów utrzymania zespołu WebPayroll

Miara kosztów	Koszt
Koszt punktu historyjki	675 zł
Koszty tygodniowe	6750 zł
Koszty iteracji	13 500 zł

Łączna ocena finansowa

Na podstawie analizy kosztów, nowych przychodów, przychodów przyrostowych oraz sprawności operacyjnej możemy zestawić kalkulację zysków i strat tak jak pokazano w tabeli 10.9.

TABELA 10.9. Prognozowany zwrot z inwestycji w projekt WebPayroll

Kwartał	Koszty projektu	Przychody nowe	Przychody przyrostowe	Przychody zatrzymane	Sprawność operacyjna	Przepływy pieniężne netto
1	-87 750 zł	0 zł	0 zł	2000 zł	0 zł	-85 750 zł
2	-20 250 zł	2500 zł	1600 zł	2000 zł	0 zł	-14 150 zł
3		3750 zł	5000 zł	2000 zł	7500 zł	18 250 zł
4		3750 zł	7500 zł	2000 zł	7500 zł	20 750 zł
5		7 500 zł	10 000 zł	4000 zł	7500 zł	29 000 zł
6		7500 zł	10 000 zł	4000 zł	7500 zł	29 000 zł
7		7500 zł	10 000 zł	4000 zł	15 000 zł	36 500 zł
8		7500 zł	10 000 zł	4000 zł	15 000 zł	36 500 zł

Oczekuje się, że implementacja funkcjonalności obliczeń z dnia na dzień zakończy się w ciągu ośmiu iteracji, czyli po szesnastu tygodniach. Pierwszy kwartał będzie zawierał trzynaście spośród tych tygodni, a koszty projektu wyniosą 87 750 (13×6750) zł. Kolejne trzy tygodnie projektu będą w drugim kwartale, a koszty wyniosą 20 250 zł.

Miary finansowe

Po ustaleniu sposobu szacowania strumienia przepływów pieniężnych, które będą generowane przez każdy temat, skupimy uwagę na różnych sposobach analizy i oceny tych przepływów pieniężnych. W tym podrozdziale przyjrzymy się wartości bieżącej netto, wewnętrznej stopie zwrotu, okresowi zwrotu i zdyskontowanemu okresowi zwrotu. Każdy z tych wskaźników można wykorzystać do porównania zwrotu z tematu. Najpierw jednak wyjaśnimy pojęcie *wartość pieniądza w czasie* (ang. *time value of money*).

Wartość pieniądza w czasie

W początkowych odcinkach komiksu Popeye Wimpy zwykł mawiać: „Z przyjemnością zapłacę we wtorek za dzisiejszego hamburgera”. Tylko frajer zgodziłby się na ofertę Wimpy’ego, ponieważ pieniądze dziś są więcej warte niż będą w przyszły wtorek.

Aby określić dzisiejszą wartość kwoty pieniężnej w przyszłości, myślimy o niej w kategoriach sumy, jaką musielibyśmy zdeponować dziś w banku, aby wzrosła do przyszłej kwoty. Aby kupić w następny wtorek hamburgera wartego 5 zł, musiałbym dziś złożyć w banku kwotę 4,99 zł. Kwota, jaką muszę zainwestować dziś po to, aby uzyskać znaną kwotę w przyszłości, nazywa się wartością bieżącą. Oto prosty przykład: jeśli mogę zarobić 10% na lokacie pieniężnej i chcę zyskać 1,00 zł za rok, to muszę dziś zainwestować 0,91 zł. Innymi słowy, przy stopie procentowej wynoszącej 10%, kwota 0,91 zł to wartość bieżąca kwoty 1,00 zł za rok. Żeby móc zarobić 20% na swojej lokacie, musiałbym dziś zainwestować tylko 0,83 zł.

Proces przekształcania przyszłych kwot na wartości bieżące nazywa się *dyskontowaniem*. Kluczowe znaczenie dla określania wartości bieżącej przyszłej kwoty ma oczywiście stopa procentowa stosowana do dyskontowania przyszłych kwot. Stopa procentowa stosowana przez firmy do dyskontowania przyszłych kwot pieniężnych to tzw. *koszt alternatywny* (ang. *opportunity cost*), który określa odsetek zysku, który jest przekazywany w celu wykonania inwestycji. Wszyscy — zarówno pojedyncze osoby, jak i organizacje — mamy różne możliwości inwestowania pieniędzy. Mogę umieścić swoje pieniądze na koncie oszczędnościowym w banku albo zainwestować w akcje. Mogę zainwestować w nieruchomości albo włożyć pieniądze pod materac. Organizacje mogą inwestować swoje pieniądze w taki sam sposób albo mogą je inwestować w różne projekty. Jeśli na projektach prowadzonych w przeszłości organizacja zwykle zarabiała 20%, to nowe projekty powinny być oceniane w odniesieniu do tych samych 20%. Koszt alternatywny organizacji wynosi 20%, ponieważ inwestycja w nowy projekt oznacza, że organizacja zrezygnowała z okazji zainwestowania pieniędzy w jakiś inny projekt, który przyniósłby 20%.

Wartość bieżąca netto

Pierwszy wzór, który zastosujemy do oszacowania tematu, to *wartość bieżąca netto* (ang. *net present value* — NPV). Aby wyznaczyć NPV, należy zsumować wartości bieżące wszystkich elementów strumienia przyszłych wartości. Oto wzór tych obliczeń:

$$NPV(i) = \sum_{t=0}^n F_t (1+i)^{-t}$$

gdzie i oznacza stopę procentową, natomiast F_t to przepływ pieniężny netto w czasie t .

Aby zobaczyć, jak to działa, nadal będziemy analizować przykład aplikacji WebPayroll. Zgodnie z tym, co określono wcześniej, oczekuje się, że projekt obliczeń z dnia na dzień będzie kosztować 108 000 zł i wygeneruje dochody i oszczędności podsumowane w tabeli 10.9, powtórzone następnie w kolumnie *Przepływy pieniężne netto* w tabeli 10.10. Kolumna *Czynnik wartości bieżącej* w tej tabeli to część obliczeń NPV, które reprezentują kwotę dyskontowania przyszłych przepływów pieniężnych. Ostatnia kolumna, *Wartość bieżąca*, to iloczyn wartości z kolumn *Przepływy pieniężne netto* oraz *Czynnik wartości bieżącej*. Oznacza to na przykład, że bieżąca wartość 18 250 zł na końcu trzeciego kwartału roku będzie wynosiła 16 701 zł. Suma wartości w kolumnie *Wartość bieżąca* daje łączną wartość wskaźnika NPV, który w tym przypadku wynosi 46 431 zł.

TABELA 10.10. Wyznaczanie wartości NPV dla aplikacji WebPayroll

Koniec kwartału	Przepływy pieniężne netto	Czynnik wartości bieżącej (12%/rok)	Wartość bieżąca
1	–85 750 zł	0,971	–83 252 zł
2	–14 150 zł	0,943	–13 338 zł
3	18 250 zł	0,915	16 701 zł
4	20 750 zł	0,888	18 436 zł
5	29 000 zł	0,863	25 016 zł
6	29 000 zł	0,837	24 287 zł
7	36 500 zł	0,813	29 677 zł
8	36 500 zł	0,789	28 813 zł
		NPV (12%) =	46 431 zł

Zaletą stosowania bieżącej wartości netto do porównywania i ustalania priorytetów tematów jest łatwość obliczeń i czytelność. Główną wadą współczynnika NPV jest to, że porównywanie wartości dwóch różnych strumieni przepływów pieniężnych może być mylące. Załóżmy, że chcemy dokonać wyboru między dwoma projektami. Pierwszy projekt wymaga ogromnych inwestycji początkowych, ale jego wartość bieżąca netto wynosi 100 000 zł. Drugi projekt wymaga niewielkich inwestycji początkowych, a również ma wartość bieżącą 100 000 zł. Oczywiście wolimy zainwestować w temat, który ma taką samą wartość NPV, ale wiąże mniej środków pieniężnych. W istocie chcielibyśmy mieć możliwość wyrażenia zwrotu z inwestycji w procentach tak, aby móc bezpośrednio porównywać tematy.

Wewnętrzna stopa zwrotu

Sposób wyrażenia zwrotu z projektu w procentach zapewnia *wewnętrzna stopa zwrotu* (ang. *Internal Rate of Return — IRR*), nazywana niekiedy *zwrotem z inwestycji* (ang. *Return on Investment — ROI*). O ile NPV jest miarą tego, jakiej kwoty pieniędzy możemy oczekiwać z projektu (wyrażonej jako wartość bieżąca), o tyle IRR jest miarą tego, jak szybko środki zainwestowane w projekt spowodują wzrost wartości. Wartość IRR pozwala na szybsze porównywanie projektów niż za pomocą NPV. Zilustrowano to w tabeli 10.11. Który projekt wybrałbyś?

TABELA 10.11. Porównanie NPV i IRR dwóch projektów

Projekt	Inwestycje	NPV	IRR
Projekt A	200 000 zł	98 682 zł	27%
Projekt B	100 000 zł	79 154 zł	43%

Większość ludzi woli zarobić 43% zainwestowanych pieniędzy (projekt B) pomimo tego, że wartość NPV jest wyższa dla projektu A, ponieważ projekt A wymaga również większych inwestycji początkowych. Przepływy pieniężne dla tych dwóch projektów przedstawiono w tabeli 10.12.

TABELA 10.12. Przepływy pieniężne dla projektów z tabeli 10.11

Rok	Projekt A	Projekt B
0	–200 000	–100 000
1	50 000	50 000
2	75 000	75 000
3	100 000	50 000
4	170 000	50 000

W wielu firmach określa się tzw. *minimalną atrakcyjną stopę zwrotu* (ang. *Minimum Attractive Rate of Return — MARR*). Finansowane są tylko te projekty lub tematy, których wartość IRR przekracza próg MARR. Ustawianie podobnego progu dla współczynnika NPV nie ma sensu, ponieważ wartości NPV są silnie uzależnione od skali projektu. Gdyby stosowano próg NPV, to niewielkie (choć bardzo wartościowe) projekty nigdy nie zostałyby zatwierdzone.

Współczynnik IRR jest zdefiniowany jako stopa procentowa, przy której wartość NPV lub strumień przepływów pieniężnych jest równy 0. Innymi słowy, jest to wartość i^* taka, że:

$$0 = PV(i^*) = \sum_{t=0}^n F_t (1+i)^{-t}$$

Wzór do obliczania współczynnika IRR jest trudny, a jego omawianie wykracza poza zakres tej książki. Na szczęście w większości głównych aplikacji arkuszy kalkulacyjnych są dostępne funkcje do łatwego obliczania IRR. Osoby, które chcą obliczać IRR ręcznie, zachęcam do zapoznania się z opisem Steve’a Tockeya (2004). Jednak pomimo tego, że możemy użyć arkusza kalkulacyjnego do obliczania IRR, istnieje kilka warunków wstępnych korzystania z tego współczynnika, o których warto pamiętać:

- Pierwszy lub kilka pierwszych elementów w strumieniu przepływów pieniężnych powinno reprezentować koszty (należy zwrócić uwagę, że musi być przynajmniej jedna taka pozycja).
- Kiedy strumień przepływów pieniężnych zmienia wartość na dodatnią, to wartość ta musi pozostać dodatnia.
- Suma komponentów dodatnich jest większa niż suma elementów ujemnych, co oznacza, że — ogólnie rzecz biorąc — uzyskujemy zysk.

Ponieważ przepływy pieniężne dla tematu obliczeń z dnia na dzień w aplikacji WebPayroll spełniają te warunki, możemy obliczyć IRR dla tematu. Aby to zrobić w Excelu, wpisz w komórce następującą formułę:

=IRR({0;-85750;-14150;18250;20750;29000;29000;36500;36500})

Liczyby w nawiasach klamrowych oznaczają strumienie przepływów pieniężnych dla każdego z ośmiu kwartałów. Początkowa wartość 0 oznacza, że w pierwszym dniu projektu nie trzeba było ponosić żadnych kosztów wstępnych (tak by było na przykład, gdyby firma WebPayroll musiała zakupić dodatkowe serwery, aby przystąpić do realizacji projektu). Dla tematu obliczeń z dnia na dzień w projekcie WebPayroll współczynnik IRR wynosi 12%. To oznacza, że oczekiwane przepływy pieniężne odpowiadają przychodom wynoszącym 12% rocznego zwrotu z inwestycji w firmie.

Pierwszą zaletą korzystania ze współczynnika IRR jest brak wymagania ustalenia (lub w najgorszym przypadku próby odgadnięcia) stopy dyskontowej organizacji tak, jak jest to konieczne przy obliczaniu NPV.

Drugą zaletą IRR jest możliwość bezpośredniego wykorzystania do porównywania projektów. Projekt ze współczynnikiem IRR wynoszącym 45% ma wyższy zwrot z inwestycji niż projekt z IRR równym 25%. Zazwyczaj nie możemy jednak stosować do podejmowania decyzji samego współczynnika IRR. Załóżmy, że projekt, którego stopa zwrotu wynosi 45%, jest bardzo mały, zatem firma osiąga 45% zwrotu z niewielkiej inwestycji, ale projekt wiąże kluczowego dewelopera. Załóżmy dalej, że projekt, który ma stopę zwrotu 25%, wymaga znacznych inwestycji oraz tego samego kluczowego dewelopera. W takiej sytuacji, aby zarobić więcej pieniędzy, możemy podjąć decyzję o wyborze drugiego projektu — tego, którego współczynnik IRR jest niższy.

Oto inny przykład. Załóżmy, że projekt ma współczynnik IRR równy 45%, ale wymaga dwóch lat inwestowania, zanim zacznie przynosić duże dochody. Projekt ze współczynnikiem IRR wynoszącym 25% zaczyna zarabiać pieniądze po roku. Jeśli organizacja nie może sobie pozwolić na dwa lata inwestowania, może preferować projekt z niższym współczynnikiem IRR.

Pierwszą wadą IRR są trudności wykonania obliczeń ręcznie, co może skutkować niższym zaufaniem pewnych osób do tak obliczonego współczynnika. Druga wada jest taka, że współczynnika IRR nie można obliczyć we wszystkich sytuacjach. Jak powiedzieliśmy wcześniej, do obliczenia sensownej wartości IRR muszą być spełnione trzy warunki.

Okres zwrotu

Dzięki współczynnikowi NPV możemy postrzegać strumień przepływów pieniężnych jako pojedynczą bieżącą kwotę. Alternatywnie za pośrednictwem współczynnika IRR możemy interpretować strumień przepływów pieniężnych jako stopę procentową. Dodatkowym sposobem analizy strumienia przepływów pieniężnych może być obserwacja czasu potrzebnego do zwrócenia się wstępnej inwestycji. Ten wskaźnik to tzw. *okres zwrotu* (ang. *payback period*). Sposób jego wyznaczania zaprezentowano w tabeli 10.13, gdzie zestawiono obliczenia opłacalności projektu obliczeń z dnia na dzień w aplikacji WebPayroll.

TABELA 10.13. Określanie okresu zwrotu dla projektu obliczeń z dnia na dzień w aplikacji WebPayroll

Kwartał	Wpływy pieniężne netto na koniec kwartału	Kwota narastająco
1	-85 750 zł	-85 750 zł
2	-14 150 zł	-99 900 zł
3	18 250 zł	-81 650 zł
4	20 750 zł	-60 900 zł
5	29 000 zł	-31 900 zł
6	29 000 zł	-2900 zł
7	36 500 zł	33 600 zł
8	36 500 zł	70 100 zł

W ciągu pierwszego kwartału firma WebPayroll zainwestowała w projekt 85 750 zł. W drugim kwartale wykonała kolejną inwestycję w kwocie 14 150 zł. W trzecim kwartale inwestycje zaczęły się zwracać, firma zarobiła 18 250 zł. Dopiero w siódmym kwartale wartość inwestycji w projekt będzie dodatnia, dlatego okres zwrotu projektu wynosi siedem kwartałów.

Są dwie główne zalety korzystania z okresu zwrotu do porównywania i ustalania priorytetów tematów. Po pierwsze, obliczenia i interpretacja są proste. Po drugie, okres zwrotu mierzy ilość i czas trwania ryzyka finansowego, które podejmuje firma. Im dłuższy okres zwrotu, tym bardziej ryzykowny projekt, ponieważ w czasie trwania projektu wszystko może się zmienić.

Główna wada okresu zwrotu jest taka, że nie uwzględnia wartości pieniądza w czasie. Wartość pieniędzy zarobionych za trzy lata jest ceniona tak samo jak tych, które zostały wypłacone dziś. Drugą wadą okresu zwrotu jest to, że nie jest to miara rentowności projektu lub tematu. Wartość okresu zwrotu powie nam, że firma odzyska zainwestowane pieniądze w ciągu siedmiu kwartałów, ale nie powie, ile pieniędzy firma zarobi.

Zdiskontowany okres zwrotu

Pierwsza wada obliczania okresu zwrotu jest łatwa do usunięcia. Aby to zrobić, wystarczy zastosować odpowiedni współczynnik dyskontowania dla każdego elementu przepływu środków pieniężnych. Sposób tych obliczeń dla projektu obliczeń z dnia na dzień w aplikacji WebPayroll pokazano w tabeli 10.14.

Jak widać w tabeli 10.14, obliczana narastająco kwota zdyskontowanego przepływu pieniężnego staje się dodatnia w siódmym kwartale (tak samo jak w obliczeniach zwykłego okresu zwrotu).

Porównywanie zwrotów

W miarę oceniania poszczególnych tematów zdobywamy wiedzę możliwą do wykorzystania w celu porównywania tematów oraz ustalania priorytetów, na potrzeby których została przeprowadzona analiza. Wyniki oceny wartości wielu tematów mogą być przedstawione tak jak pokazano w tabeli 10.15. Takie tabele pozwalają firmom na szybkie zweryfikowanie dostępnych opcji i wybranie do pracy tych tematów, które przynoszą największą wartość.

TABELA 10.14. Wyznaczanie zdyskontowanego okresu zwrotu dla tematu obliczeń z dnia na dzień w aplikacji WebPayroll

Koniec kwartału	Przepływy pieniężne	Czynnik wartości bieżącej (12%/rok)	Zdyskontowany przepływ pieniężny	Kwota narastająco
1	-85 750 zł	0,971	-83 252 zł	-83 252 zł
2	-14 150 zł	0,943	-13 338 zł	-96 590 zł
3	18 250 zł	0,915	16 701 zł	-79 889 zł
4	20 750 zł	0,888	18 436 zł	-61 453 zł
5	29 000 zł	0,863	25 016 zł	-36 437 zł
6	29 000 zł	0,837	24 287 zł	-12 150 zł
7	36 500 zł	0,813	29 677 zł	17 527 zł
8	36 500 zł	0,789	28 813 zł	46 340 zł

TABELA 10.15. Różne oceny wartości tematów w projekcie

Temat	Punkty historyjek	Koszt	NPV	ROI	Zdyskontowany okres zwrotu
Obliczenia z dnia na dzień	150	101 250 zł	46 341 zł	45%	7 kwartałów
Niestandardowe raporty	90	60 750 zł	34 533 zł	15%	6 kwartałów
Integracja z partnerem	60	40 500 zł	30 013 zł	49%	3 kwartały

W tym przypadku temat obliczeń z dnia na dzień charakteryzuje się najwyższą bieżącą wartością netto, ale zwrot z inwestycji następuje po najdłuższym czasie. Temat integracji z partnerem ma najwyższy wskaźnik zwrotu z inwestycji i najkrótszy zdyskontowany okres zwrotu, ale ma najniższą wartość współczynnika NPV. Niestandardowe raporty mają najniższy współczynnik zwrotu z inwestycji. Jednak temat ten można połączyć z integracją z partnerem i zrealizować oba tematy tym samym kosztem co temat obliczeń z dnia na dzień. Sposób podejmowania decyzji nie jest oczywisty. Właściciel produktu wraz z zespołem muszą wziąć pod uwagę różne czynniki specyficzne dla sytuacji, na przykład tolerancję firmy na ryzyko, potrzebę krótkich okresów zwrotu z inwestycji, dostępność zasobów czy inne opcje inwestowania pieniędzy.

Dodatkowe informacje o ekonomii projektów

Chociaż w tym rozdziale zaprezentowano ogólny przegląd sposobu wykorzystania i obliczania czterech wskaźników finansowych, na ich temat można powiedzieć znacznie więcej. Obszerniejsze informacje o ekonomii projektów oprogramowania można znaleźć w doskonałej książce Steve'a Tockeya *Return on Software: Maximizing the Return on Your Software Investment* (2004). Zamieszczone tutaj informacje o czterech wskaźnikach finansowych włącznie z wzorami ich obliczania zostały zaczerpnięte z książki Tockeya.

Podsumowanie

Analiza finansowa tematów pomaga ustalić ich priorytety, ponieważ dla większości organizacji najważniejszą informacją jest liczba zarobionych lub zaoszczędzonych pieniędzy. To zwykle wystarczy do prognozowania przychodów i sprawności operacyjnej na następne dwa lata. Jeśli jednak to konieczne, możemy planować dalej w przyszłość.

Dobrym sposobem modelowania zwrotu z tematu jest wzięcie pod uwagę przychodów, jakie wygeneruje realizacja tematu od nowych klientów, przychodów z zakupu większej liczby kopii lub usług dodatkowych przez klientów bieżących, przychodów od klientów, którzy w przeciwnym razie mogliby kupić konkurencyjny produkt, oraz przychodów wynikających ze sprawności operacyjnej, jaką zapewni realizacja tematu.

Pieniądze zarobione lub wydane dzisiaj są więcej warte niż pieniądze zarobione lub wydane w przyszłości. Aby porównać bieżącą kwotę z kwotą w przyszłości, przyszła kwota jest dyskontowana do wartości bieżącej. Wartość bieżąca to kwota, którą można by zdeponować w banku lub zainwestować w inne stosunkowo bezpieczne mechanizmy finansowe, tak aby za ustalony czas urosła do wartości przyszłej.

Czterema dobrymi sposobami oceny strumienia przepływów pieniężnych są: bieżąca wartość netto, wewnętrzna stopa zwrotu (zwrot z inwestycji), okres zwrotu oraz zdyskontowany okres zwrotu. Dzięki obliczeniu tych wskaźników dla każdego tematu właściciel produktu i zespół mogą podejmować świadome decyzje dotyczące priorytetów poszczególnych tematów.

Pytania do dyskusji

1. Gdyby w Twojej organizacji wybierano spośród tematów zestawionych w tabeli 10.15, to jaka decyzja byłaby najbardziej właściwa? Dlaczego?
2. W jaki sposób zamodelowałbyś zwrot z tematu w Twoim bieżącym projekcie? Czy proponowane kategorie przychodów nowych, przychodów przyrostowych, przychodów zatrzymanych oraz sprawności operacyjnej są właściwe? Jakich kategorii użyłbyś zamiast nich?

ROZDZIAŁ 11.

Priorytety atrakcyjności

Jeśli masz wybór spośród dwóch rzeczy i nie możesz się zdecydować, wybierz obie.

— Gregory Corso

Kiedy piszę te słowa, siedzę w pokoju hotelowym. W pokoju jest łóżko, łazienka i biurko. Jest dość czysto. Każda z tych własności to podstawowy atrybut, jakiego oczekuję od wszystkich pokoi hotelowych. Gdyby któregośkolwiek z nich zabrakło, nie zatrzymałbym się w tym hotelu. Znajduje się w nim także łóżko, które jest bardziej komfortowe niż w większości hoteli, bezpłatny bezprzewodowy dostęp do internetu; pokój jest obszerniejszy niż większość pokoi hotelowych i zapewnia dostęp do centrum fitnessu. Bardzo doceniam te dodatkowe własności, ale nie są one niezbędne do tego, żeby skłonić mnie do zatrzymania się w tym hotelu. Jednak im więcej tego rodzaju cech oferuje ten hotel, tym bardziej jestem skłonny zapłacić w nim za pokój. Na koniec, ten hotel bardzo mi się spodobał z dwóch powodów: każda bieżnia w centrum fitness ma wbudowany telewizor i codziennie znajduję w swoim pokoju hotelowym bezpłatną butelkę wody.

Cechy tego pokoju hotelowego mógłbym pogrupować w następujące kategorie:

- obowiązkowe: łóżko, łazienka, biurko, czystość;
- im więcej, tym lepiej: komfort łóżka, wielkość pokoju, rodzaj i ilość sprzętu w centrum fitness;
- fascynujące: telewizor wbudowany w bieżnię, bezpłatna butelka wody codziennie w pokoju.

Model Kano satysfakcji klientów

Podział cech produktów na wymienione wcześniej trzy kategorie może dostarczyć wielu informacji dotyczących sposobu ustalenia priorytetów prac nad nową wersją produktu. Proces zapoczątkował Noriaki Kano, którego podejście pozwala na podzielenie cech produktu na trzy kategorie:

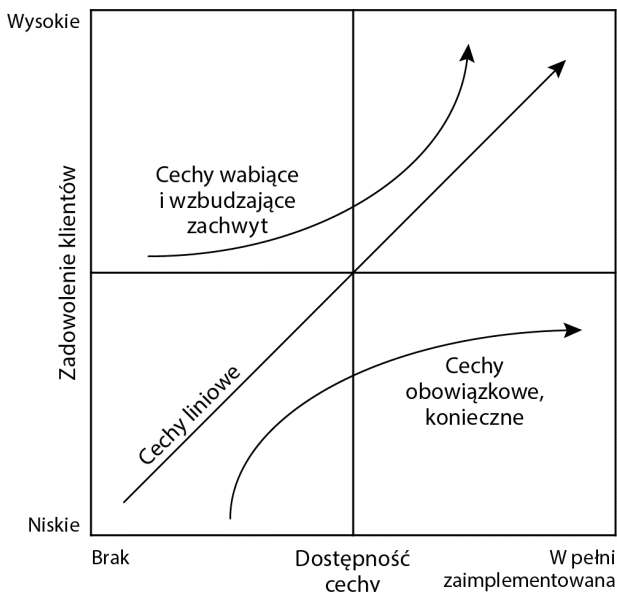
- cechy progowe lub konieczne,
- cechy liniowe,
- cechy wabiące i wzbudzające zachwyt.

Cechy progowe to takie, które muszą być dostępne w produkcie, aby mógł on odnieść sukces. Często określa się je jako *cechy konieczne*. W przypadku mojego pokoju hotelowego są to łóżko, łazienka, biurko i czystość. Zwiększenie wydajności lub ilości cech progowych ma niewielki wpływ na zadowolenie klientów. Na przykład tak długo, jak hotelowa łazienka spełnia moje podstawowe potrzeby, nie obchodzi mnie, z czego są wykonane białe.

Cechy liniowe to takie, dla których obowiązuje reguła „im więcej, tym lepiej”. Im większy pokój hotelowy, tym lepiej. Są to tak zwane cechy liniowe, ponieważ zadowolenie klientów jest skorelowane liniowo z wymiarem określonej cechy. Im lepiej się sprawdza każda z tych cech (lub im jest ich więcej), tym bardziej zadowoleni są klienci. Z tego względu cena produktu jest często związana z atrybutami liniowymi. Jeśli w hotelu są hantle i dwie dobre bieżnie, to będę bardziej zadowolony niż wtedy, gdyby był tam jedynie stary, zardzewiały orbitrek. Będę jeszcze bardziej zadowolony, jeśli w hotelu będą trenażery eliptyczne i rowery treningowe. A co ważne dla tego hotelu, istnieje większe prawdopodobieństwo, że wrócę do niego i zapłacę za pokój więcej.

Wreszcie cechy wabiące i wzbudzające zachwyt to takie, które zapewniają największą satysfakcję — często mają udział w podwyższeniu ceny produktu. Jednak brak cechy wabiącej lub sprawiającej zachwyt nie zmniejszy satysfakcji klientów poniżej poziomu neutralnego. Wbudowany telewizor w hotelowej bieżni wzbudził mój zachwyt, ale nie byłbym niezadowolony, gdyby hotel nie oferował tej cechy, ponieważ nic nie wiedziałem o tym, że ją oferuje. W rzeczywistości cechy wabiące i wzbudzające zachwyt często nazywa się potrzebami nieznanymi, ponieważ klienci lub użytkownicy nie wiedzą, że potrzebują tych cech, dopóki ich nie zobaczą.

Związek między wymienionymi trzema różnymi typami cech przedstawiono na rysunku 11.1. Strzałka w kierunku prawej dolnej części tego wykresu pokazuje, że po zrealizowaniu cechy koniecznej do pewnego stopnia dodanie jej w większej ilości nie potęguje zadowolenia klienta. Ponadto niezależnie od tego, ile cechy obowiązkowej dodamy, zadowolenie klienta nigdy nie wzrośnie powyżej połowy skali.



RYСУNEK 11.1. Model Kano satysfakcji klientów

Z drugiej strony linia w górnym lewym rogu pokazuje, że zadowolenie klientów gwałtownie wzrasta w odpowiedzi chociażby na częściową implementację cechy wabiącej lub wzbudzającej zachwyty. Wreszcie linia przechodząca przez środek wykresu pokazuje bezpośredni związek między dołączeniem cech liniowych a zadowoleniem klientów.

Ponieważ cechy obowiązkowe muszą występować w produkcie, aby mógł on odgrywać jakąkolwiek rolę w swoim segmencie rynku, to należy zadbać o nadanie wysokiego priorytetu pracom nad implementacją wszystkich cech progowych. Cechy obowiązkowe produktu nie muszą być implementowane w pierwszych iteracjach wydania. Ponieważ jednak użytkownicy uważają te funkcjonalności za konieczne, to muszą one być dostępne, zanim produkt trafi na rynek. Należy jednak pamiętać, że wystarczająca może być częściowa implementacja cech koniecznych, ponieważ wzrost satysfakcji klientów szybko spada po osiągnięciu bazowego poziomu implementacji funkcji progowych. Na przykład tę książkę piszę za pomocą programu Adobe FrameMaker, który oferuje funkcję skrótu dla funkcjonalności jednego poziomu operacji cofania. Dla mnie osobiście implementacja funkcji cofania o jeden poziom sprawiła, że program znalazł się dość wysoko na osi mojej satysfakcji jako klienta, choć wolalibyśmy, żeby było możliwe cofanie o więcej poziomów.

Dodatkowy nacisk powinien zostać położony na zrealizowanie jak największej liczby cech liniowych. Ponieważ każda z tych cech prowadzi bezpośrednio do większej satysfakcji klientów, to im więcej takich cech można uwzględnić w produkcie, tym lepiej (oczywiście z wyjątkiem takich sytuacji, gdy produkt jest już „zaśmiecony” zbyt dużą liczbą takich cech). W końcu, jeśli pozwala na to czas, należy nadać priorytet co najmniej kilku cechom wabiącym tak, aby zostały uwzględnione w planie wydania.

Należy pamiętać, że z czasem poszczególne cechy funkcjonalne mają tendencję do migracji w dół diagramu Kano. Jakiś czas temu bezprzewodowy dostęp do internetu w pokoju hotelowym był wabikiem. Obecnie jest to cecha liniowa, która jest na dobrej drodze do tego, by stała się funkcją progową.

Ocena tematów w modelu Kano

Najprostszym sposobem wykorzystania modelu Kano w projekcie Agile jest przeanalizowanie wszystkich tematów i inteligentne odgadnięcie typu każdego z nich. Jednak znacznie lepszym sposobem jest rozmowa z klientami i użytkownikami w celu określenia typu każdego tematu. To zaskakująco łatwe, ponieważ można to zrobić za pomocą pisemnej ankiety. Aby precyzyjnie określić priorytety wymagań, wystarczy przeprowadzić badanie na grupie zaledwie od dwudziestu do trzydziestu użytkowników (Griffin i Hauser 1993).

Kano proponuje ustalenie kategorii cechy funkcjonalnej poprzez zadanie dwóch pytań: jedno pytanie jest o odczucia użytkownika w przypadku, gdyby cecha funkcjonalna została dołączona do produktu, a drugie dotyczy nastawienia do sytuacji, w której cecha funkcjonalna byłaby niedostępna. Pierwsze z tych pytań ma formę funkcjonalną, bo odnosi się do przypadku, gdy funkcja jest dostępna. Drugie pytanie ma formę dysfunkcjonalną, bo dotyczy sytuacji, gdy funkcja nie jest dostępna. Użytkownicy udzielają odpowiedzi na każde z pytań, używając takiej samej pięciostopniowej skali:

1. Podoba mi się.
2. Spodziewałem się, że to będzie właśnie tak wyglądać.
3. Jestem neutralny.
4. Mogę z tym żyć.
5. Nie podoba mi się.

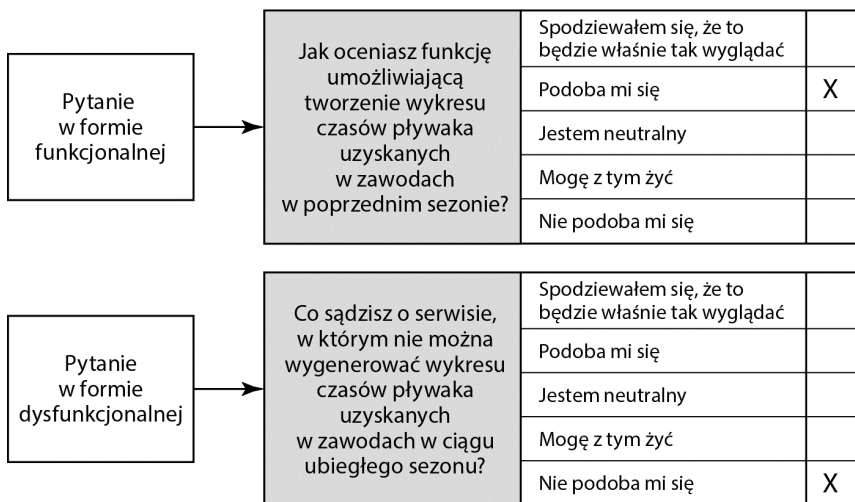
Dla przykładu wróćmy do witryny SwimStats. Załóżmy, że analizujemy trzy nowe funkcje:

- możliwość wyświetlenia wykresu czasów pływaka osiągniętych w zawodach w minionym sezonie,
- możliwość publikowania przez zawodników profili biograficznych,
- możliwość wgrania zdjęć przez każdego zarejestrowanego użytkownika serwisu.

Aby określić typ każdej z tych funkcji, przeprowadzamy wśród użytkowników ankietę, zadając im następujące pytania:

- Jak oceniasz funkcję umożliwiającą tworzenie wykresu czasów pływaka uzyskanych w zawodach w poprzednim sezonie?
- Co sądzisz o serwisie, w którym nie można wygenerować wykresu czasów pływaka uzyskanych w zawodach w ciągu ubiegłego sezonu?
- Co sądzisz o możliwości publikowania profili biograficznych zawodników?
- Jak oceniasz serwis, w którym publikowanie profili biograficznych jest niemożliwe?
- Co sądzisz o funkcji pozwalającej na przysyłanie zdjęć?
- Jak oceniałbyś serwis, w którym przysyłanie zdjęć byłoby niedostępne?

Pierwszą parę tych pytań wraz z hipotetycznymi odpowiedziami udzielonymi przez jednego z użytkowników przedstawiono na rysunku 11.2.



RYSUNEK 11.2. Pytania w formie funkcjonalnej i dysfunkcjonalnej

Przyglądając się rysunkowi 11.2, łatwo zauważyć, że użytkownik może udzielić niespójnych odpowiedzi. Na przykład może odpowiedzieć, że podoba mu się możliwość tworzenia wykresów czasów zawodników w zawodach oraz że podoba mu się to, iż takiej możliwości nie ma. Jest dwadzieścia pięć możliwych kombinacji odpowiedzi na pytania w formie funkcjonalnej i dysfunkcjonalnej. Potrzebny jest sposób spojrzenia na obie odpowiedzi użytkownika jednocześnie i oszacowanie jego opinii.

Podział odpowiedzi na kategorie

Metodę podziału odpowiedzi na kategorie pokazano na rysunku 11.3. Dzięki porównaniu odpowiedzi na pytanie w formie funkcjonalnej z odpowiedzią na pytanie w formie dysfunkcjonalnej potencjalnego użytkownika można zredukować do jednego znaczenia. Tak więc, jeśli użytkownik mówi, że oczekuje możliwości tworzenia wykresów czasów pływaków oraz że nie podobałoby mu się, gdyby takiej możliwości nie było (takie odpowiedzi zostały udzielone w formularzu zaprezentowanym na rysunku 11.2), to porównując odpowiedzi na te pytania, można dojść do wniosku, że ta funkcjonalność jest dla użytkownika konieczna. W jego świadomości tworzenie wykresów czasów pływaków w zawodach jest funkcjonalnością niezbędną.

		Pytanie dysfunkcjonalne				
		Podoba mi się	Oczekuję	Obojętne	Mogę żyć	Nie podoba mi się
Pytanie funkcjonalne	Podoba mi się	W	Wb	Wb	Wb	L
	Oczekuję	O	B	B	B	K
	Obojętne	O	B	B	B	K
	Mogę żyć	O	B	B	B	K
	Nie podoba mi się	O	O	O	O	W

K – Konieczna

O – Odwrotna

L – Liniowa

W – Wątpliwa

Wb – Wabik

B – Bez znaczenia

RYSUNEK 11.3. Podział cech na kategorie na podstawie odpowiedzi na parę pytań

Jeśli powtórzymy ten proces dla grupy od dwudziestu do trzydziestu użytkowników, będzie można podsumować ich odpowiedzi i oszacować ich rozkład tak, jak pokazano w tabeli 11.1. Na podstawie analizy tej tabeli możemy odpowiedzieć, że możliwość tworzenia wykresów czasów na zawodach jest cechą jednowymiarową (liniową), natomiast możliwość wgrywania zdjęć jest funkcją konieczną.

Dla niektórych funkcjonalności (na przykład publikowania profilu biograficznego w tabeli 11.1) uzyskujemy wysokie wartości wyników dla dwóch odpowiedzi. To pokazuje, że różne typy klientów i użytkowników mają różne oczekiwania. W takich sytuacjach należy rozważyć analizę odpowiedzi na podstawie jakiegoś czynnika, który odróżnia podgrupy Twoich klientów lub użytkowników. Można na przykład oddzielić odpowiedzi od użytkowników z małych firm od użytkowników z dużych firm. Można również rozdzielić odpowiedzi na podstawie stanowisk zajmowanych przez użytkowników lub segmentu branży.

TABELA 11.1. Rozkład wyników ankiety wśród użytkowników

Temat	Wb	L	K	B	O	W	Kategoria
Wykres czasów na zawodach	18,4	43,8	22,8	12,8	1,7	0,5	Liniowa
Możliwość przysyłania zdjęć	8,3	30,9	54,3	4,2	1,4	0,9	Konieczna
Publikowanie profilu biograficznego	39,1	14,8	36,6	8,2	0,2	1,1	Wabik (konieczna)

Inne podejście — wagi względne

Karl Wiegiers (1999) zaleca podejście, które jest podobne do modelu Kano w tym sensie, że bierze pod uwagę zarówno dodatni wpływ dostępności funkcjonalności, jak i ujemny wpływ jej braku. Zamiast na korzystaniu z ankiet to podejście bazuje na ocenie ekspertów. Członkowie zespołu wspólnie, ale pod przewodnictwem właściciela produktu oceniają poszczególne funkcjonalności dla kolejnego wydania. Każda funkcjonalność jest oceniana tak z punktu widzenia korzyści, jakie przyniesie dla użytkowników, jeśli zostanie zaimplementowana, jak i pod kątem poniesionych strat w przypadku, gdy jej nie zaimplementujemy. Podobnie jak przy ocenach historyjek za pomocą punktów i dni idealnych oceny korzyści i strat są względne. Do oceny stosujemy skalę od 1 do 9. Przykład zastosowania takich oszacowań dla funkcjonalności serwisu SwimStats zestawiono w tabeli 11.2.

TABELA 11.2. Podejście do ustalania priorytetów bazujące na względnych wagach

Funkcjonalność	Względna korzyść	Względna strata	Łączna wartość	Wartość %	Ocena	Koszt %	Priorytet
Wykres czasów na zawodach	8	6	14	42	32	53	0,79
Możliwość przysyłania zdjęć	9	2	11	33	21	34	0,97
Publikowanie profilu biograficznego	3	5	8	25	8	13	1,92
Razem	20	13	33	100	61	100	

Korzyść dołączenia funkcji tworzenia wykresów czasów pływaków w zawodach oceniono na 8. Ta funkcjonalność jest nieco mniej wartościowa niż przysyłanie zdjęć, ale znacznie bardziej wartościowa niż możliwość publikowania przez zawodników profili biograficznych. Jeśli chodzi o względne straty w przypadku braku funkcjonalności można zauważyć, że straty wynikające z braku możliwości przysyłania zdjęć przez zawodników są bardzo niewielkie (w kategoriach modelu Kano funkcjonalność przysyłania zdjęć jest prawdopodobnie wabikiem — użytkownikom podoba się ona, ale nie odrzuciliby

produktu, gdyby jej nie było). Większa strata byłaby wówczas, gdyby nie można było tworzyć wykresów czasów na zawodach lub publikować profilu biograficznego.

Względne korzyści i straty dla każdej funkcjonalności sumujemy i wprowadzamy w kolumnie *Łączna wartość*. Można również ustalić wagę dla korzyści i strat. Przykładowo, aby względna korzyść była traktowana jako dwa razy ważniejsza od względnej straty, należy przed dodaniem wartości w kolumnie *Względna strata* do wartości w kolumnie *Względna korzyść* pomnożyć tę drugą wartość przez 2.

Suma wartości w kolumnie *Łączna wartość* (w tym przypadku 33) oznacza całkowity koszt dostarczenia wszystkich funkcjonalności. Aby obliczyć względny wkład każdej funkcjonalności w koszty produktu, należy podzielić łączną wartość tej funkcjonalności przez sumę łącznych wartości dla wszystkich funkcjonalności. Na przykład funkcjonalność tworzenia wykresów czasów z zawodów ma wartość 14. Łączna wartość wszystkich funkcjonalności wynosi 33. Podzielenie 14 przez 33 daje wynik 0,42, czyli 42%. To oznacza, że funkcjonalność tworzenia wykresów czasów reprezentuje 42% całkowitej wartości wszystkich funkcjonalności.

W następnej kolumnie, *Ocena*, zapisujemy szacowaną wartość tematu w punktach lub dniach idealnych. Identycznie jak w przypadku wartości w kolumnie *Łączna wartość*, wartości w kolumnie *Ocena* są sumowane (tutaj ich suma wynosi 61), a procent każdej oceny jest obliczany w kolumnie *Koszt %*. W tym przykładzie tworzenie wykresów czasów w zawodach oceniono na 32 punkty lub dni idealne. Łącznie trzy funkcjonalności mają ocenę 61 punktów lub dni idealnych. Wynika stąd, że tworzenie wykresów czasów reprezentuje 53% łącznych kosztów tych trzech historyjek ($32/61 = 0,53$).

Wartość w ostatniej kolumnie, *Priorytet*, wyliczamy poprzez podzielenie wartości w kolumnie *Wartość %* przez wartość w kolumnie *Koszt %*. Wyższe wartości reprezentują wyższe priorytety, ponieważ tworzą większą wartość względem wysiłku, który w nie zainwestowano. Można to zauważyć w tabeli 11.2, przyglądając się funkcjonalności publikowania profilu biograficznych. Ta funkcjonalność generuje nieco ponad połowę wartości funkcjonalności tworzenia wykresów czasów (*Łączna wartość* 8 w porównaniu z 14), ale przy jednej czwartej kosztów (ocena 8 w porównaniu z 32). Ze względu na wysoką wartość współczynnika wartości do kosztów funkcjonalność generowania profilu biograficznych ma zgodnie z tą analizą najwyższy priorytet.

Znaczenie brania pod uwagę względnej straty

Możesz odczuwać pokusę pominięcia kolumny *Względna strata*. Nie radzę tego robić. Tak jak w przypadku modelu Kano, warto przyjrzeć się funkcjonalnościom z perspektywy wpływu na użytkowników zarówno dostępności funkcjonalności, jak i jej braku.

Jeden z moich klientów pracujący w branży finansowej zaprezentował dobry przykład tego, jakie znaczenie ma branie pod uwagę względnej straty. Właścicielowi produktu w jego zespole udało się dość łatwo ustalić priorytety funkcjonalności. Jednak jedna z funkcjonalności, której musiał nadać priorytet, nie przynosiła firmie żadnej innej wartości poza zachowaniem zgodności z pewnymi nowymi przepisami. Żartował, że jedyną stratą w przypadku braku jej implementacji będzie to, że dyrektor generalny będzie musiał spędzić trochę czasu w więzieniu. Wobec tego właściciel produktu (który nie był dyrektorem generalnym) uważał, że można przyjąć takie zagrożenie, jeśli w zamian uda się zaimplementować w produkcie inne funkcjonalności.

To czytelny przykład funkcjonalności o względnej korzyści 1, ale względnej stracie 9.

Podsumowanie

Spróbujmy zrobić krok wstecz i przypomnieć sobie, dlaczego to wszystko robimy. W poprzednim rozdziale dowiedzieliśmy się, że podczas ustalania priorytetów trzeba wziąć pod uwagę cztery czynniki.

1. Ilość i znaczenie nowej nauki i wiedzy powstałej dzięki implementacji funkcjonalności.
2. Rozmiar usuniętych zagrożeń dzięki implementacji funkcjonalności.
3. Finansową wartość implementacji tych funkcjonalności.
4. Koszty implementacji (i ewentualnie wsparcia) nowych funkcjonalności.

W rozdziale 9. „Priorytety tematów” opisano znaczenie wzięcia pod uwagę czynnika nauki i eliminacji ryzyka w ogólnym zadaniu określania priorytetów funkcjonalności. W tym rozdziale przedstawiono dwa podejścia do ustalania priorytetów atrakcyjności funkcjonalności: analizę Kano i względne wagi.

Analiza Kano polega na posegregowaniu funkcjonalności na konieczne, liniowe i wabiące. Aby to zrobić, na temat każdej funkcjonalności są zadawane potencjalnym użytkownikom dwa pytania: jakie jest ich nastawienie do produktu, w którym taka funkcjonalność byłaby dostępna, a jakie wtedy, gdyby nie była dostępna.

Względne wagi to podejście do oceny korzyści z implementacji funkcjonalności, strat wynikających z rezygnacji z niej oraz kosztów i zaprezentowanie pojedynczej wartości określającej priorytet funkcjonalności.

Pytania do dyskusji

1. Jakie są względne korzyści z analizy Kano i określania względnych wag w Twojej obecnej firmie?
2. Jakie „wabiki” zostały dołączone do Twojego bieżącego projektu?
3. Czy w Twoim bieżącym projekcie uwzględniono właściwą mieszankę wabików, funkcjonalności liniowych oraz koniecznych?

ROZDZIAŁ 12.

Podział historyjek użytkownika

*Dzisiaj nie tworzy się oprogramowania moduł po module,
ale funkcjonalność po funkcjonalności.*

— Mary Poppendieck

Kiedy historyjki użytkownika wspinają się w górę planu wydania, co oznacza, że wkrótce zostaną zaimplementowane, często występuje potrzeba ich podzielenia. Trzeba wziąć pod uwagę, że implementacja pewnych historyjek użytkownika może zająć więcej czasu, niż trwa jedna iteracja, dlatego nie ma innego wyjścia niż rozbicie historyjki na dwie lub większą liczbę historyjek. Nauczenie się sposobów podziału historyjek nie jest szczególnie trudne, ale wymaga praktyki i wiedzy. Im więcej różnych historyjek użytkowników dzieliłeś w przeszłości, tym łatwiejsze staje się to zadanie. Mając to na uwadze, w tym rozdziale zaoferowano wskazówki dotyczące podziału historyjek na podstawie kilku przykładów. Z tych przykładów wynika szereg wytycznych, które można wykorzystać w celu dzielenia innych historyjek.

Porady zamieszczone w tym rozdziale można wykorzystać zawsze wtedy, gdy trzeba podzielić jakąś historyjkę. Jednak zaprezentowane wytyczne dotyczą głównie tych historyjek użytkowników lub funkcjonalności, których podzielenie wydaje się trudne.

Kiedy należy podzielić historyjkę użytkownika?

Istnieją sytuacje, gdy określoną historyjkę użytkownika trzeba podzielić na kilka mniejszych części. Po pierwsze, trzeba podzielić historyjkę, gdy jest zbyt duża, aby mogła zostać zrealizowana w ramach jednej iteracji. Czasami historyjka użytkownika nie mieści się w iteracji, ponieważ czas jej realizacji przekracza pełną iterację. Z oczywistych względów w tej sytuacji trzeba podzielić historyjkę na mniejsze. Z drugiej strony historyjka może być na tyle mała, że mieści się w iteracji, ale nie pasuje do zaplanowanej iteracji, ponieważ nie pozostaje na jej realizację wystarczająco dużo czasu. Zespół może odczuwać, że znajdzie czas na zaimplementowanie części historyjki w iteracji, ale nie zdąży zaimplementować całej historyjki.

Po drugie, jeśli konieczne jest dostarczenie dokładniejszej oceny, to może być przydatne rozdzielenie dużej historyjki użytkownika (epiki). Na przykład jeden z moich klientów rozważał implementację nowych funkcjonalności, które zapewnią lepszy dostęp do ich systemu przedstawicielom obsługi

klienta pracującym w innych firmach. Pierwsze pytanie, na jakie był zmuszony odpowiedzieć właściciel produktu, brzmiało: „Czy warto realizować te funkcjonalności?”. Zamiast napisać wiele pojedynczych historyjek użytkowników, napisał jedną obszerną historyjkę i opisał w niej wizję funkcjonalności, którą zaprezentował zespołowi. Została oceniona na siedemdziesiąt punktów. To była wystarczająco dobra ocena, aby podjąć decyzję o dodaniu tych funkcjonalności. Właściciel produktu wiedział, że w tej ocenie jest sporo niepewności, ale nawet gdyby ocena różniła się o 100%, to implementacja tych funkcjonalności nadal byłaby wartościowa. Gdyby siedemdziesiąt punktów oznaczało osiągnięcie granicy, to musiałoby to wpłynąć na decyzję o tym, czy uwzględniać w wydaniu funkcjonalności wsparcie dla pracowników obsługi klienta, czy nie. Wtedy właściciel produktu mógłby podjąć decyzję o podzieleniu dużej historyjki na mniejsze i zlecić zespołowi ocenę wielu mniejszych historyjek.

Podział według granic danych

Jednym z najlepszych sposobów rozbicia dużej historyjki użytkownika jest zastosowanie kryterium danych. Na przykład w jednym z projektów, w których uczestniczyłem, zespół pracował nad produktem do zbierania danych finansowych. Zaczął od jednej epiki: „Jako użytkownik mogę wprowadzić do arkusza dane dotyczące bilansu”. Arkusz bilansu w tym przypadku może zawierać wiele pól. Na najwyższym poziomie obejmuje aktywa i zobowiązania. Do aktywów należą: gotówka, papiery wartościowe, nieruchomości, samochody, kredyty i tak dalej. System pozwalał użytkownikowi na interakcje z tym arkuszem bilansu na różnych poziomach szczegółowości. Użytkownik mógł wprowadzić jedną wartość reprezentującą wszystkie jego aktywa. Mógł także wprowadzić trochę bardziej szczegółowe informacje (na przykład łączną wartość wszystkich kredytów) lub informacje znacznie bardziej szczegółowe (detale dotyczące wszystkich kredytów). Biorąc pod uwagę liczbę pól na ekranie oraz interakcje między nimi, należy stwierdzić, że implementacja wszystkich tych operacji znacznie przekraczała możliwości zespołu podczas jednej, dwutygodniowej iteracji.

Zespół podzielił tę historyjkę, korzystając z kryterium danych, jakie użytkownik może wprowadzić. Pierwsza historyjka miała następujące brzmienie: „Jako użytkownik mogę wprowadzać sumaryczne dane bilansu”. Ta historyjka była bardzo mała (niemal zbyt mała), ponieważ obejmowała tylko utworzenie dwóch prostych formularzy oraz dwóch pól: jednego dla aktywów i drugiego dla zobowiązań. Oto kolejna historyjka: „Jako użytkownik mogę wprowadzać dane bilansu z podziałem na kategorie”. Ta historyjka obejmowała następny poziom szczegółowości (na przykład gotówka, akcje, nieruchomości czy kredyty). Gdyby ta historyjka miała być implementowana, trzeba by utworzyć ekran z kilkudziesięcioma polami do wprowadzania danych. Kolejna historyjka obejmowała walidację danych: „Jako użytkownik chcę, aby wprowadzane wartości były weryfikowane pod względem poprawności tak, aby nie było żadnych pomyłek”. Członkowie zespołu zaczęli dyskutować, co to oznacza, i zgodzili się, że trzeba pozwolić na wprowadzanie liczb dodatnich i ujemnych, można wprowadzać ułamki dziesiętne, ale kwoty będą automatycznie zaokrąglane do całego dolara i tak dalej.

Następna historyjka użytkownika to: „Jako użytkownik mogę wprowadzać szczegółowe informacje o kredycie”. Ta historyjka pozwalała użytkownikowi wprowadzić do 100 kredytów (tę liczbę poddano pod dyskusję, uzgodniono i wskazano na karcie historyjki jako warunek satysfakcji). Historyjka była większa, niż się wydawało, ponieważ uwzględniała wiele problemów związanych z interfejsem użytkownika — na przykład w jaki sposób mają być wyświetlane na ekranie nowe wiersze danych pożyczki. Historyjka była o wiele większa od tych, które zostały wydzielone z historyjki w pierwotnym kształcie. Niemniej jednak nawet ta historyjka była o wiele mniejsza od oryginalnej, ponieważ wymagała pomocniczych

szczegółowych danych tylko na temat kredytów. Historyjkę dotyczącą kredytu wykorzystano jako wzorzec dla wielu innych historyjek użytkownika wydzielonych z oryginału — na przykład „Jako użytkownik mogę wprowadzić szczegółowe informacje na temat swoich nieruchomości” oraz „Jako użytkownik mogę podać szczegółowe informacje na temat swoich środków pieniężnych, w tym rachunków bieżących i oszczędnościowych”.

W wyniku podziału początkowej historyjki użytkownika w opisany powyżej sposób zespół stworzył na podstawie wyjściowej historyjki kilkanaście nowych. Rozmiar każdej z tych nowych historyjek był na tyle mały, że z powodzeniem można je było zrealizować w ciągu dwutygodniowej iteracji. To prowadzi do naszej pierwszej wytycznej:



Wskazówka. Obszerne historyjki użytkownika można dzielić według obsługiwanych danych.

Podział historyjek użytkowników zgodnie z granicami danych to bardzo przydatne podejście, które z pewnością powinno się znaleźć w przyborniku działań. Oto inny przykład. Kilka lat temu pracowałem z zespołem tworzącym automatyczny podsystem faksowy. Projekt obejmował kilka rozbudowanych historyjek użytkowników opisujących możliwe sposoby konfiguracji. Historyjki znacznie zmniejszono dzięki rozdzieleniu obsługi na numery amerykańskie i międzynarodowe.

W niektórych przypadkach obszerne historyjki użytkownika można znacznie zmniejszyć poprzez usunięcie z głównej historyjki obsługi błędów lub wyjątków. Załóżmy, że pracujemy nad systemem przetwarzającym spłaty pożyczki i mamy następującą historyjkę użytkownika: „Jako kredytobiorca chcę mieć możliwość spłaty swojej pożyczki”. Podczas omawiania tej historyjki właściciel produktu zauważył, że jeśli kredytobiorca przypadkiem wyśle czek na sumę przekraczającą kwotę pożyczki pozostałą do spłaty, to trzeba wydrukować czek zwrotny i przesłać do kredytobiorcy. Dodał, że dotyczy to wyłącznie kwot przekraczających 2 \$. Tę historyjkę można podzielić na mniejsze przez napisanie następujących historyjek:

- Jako kredytobiorca chcę spłacić kredyt. Uwaga: możliwość nadpłaty.
- Jeśli jako kredytobiorca przypadkiem spłacę za dużą kwotę, to otrzymam zwrot pieniędzy, o ile nadpłacona kwota przekracza 2 \$.

Podział według granic operacyjnych

Ostatnio pracowałem z zespołem, który otrzymał zadanie opracowania bardzo złożonego ekranu wyszukiwania. W górnej części ekranu były dziesiątki pól do wprowadzania kryteriów wyszukiwania, trzeba było napisać działający w warstwie pośredniej konstruktor kwerend, który scalał kwerendy do bazy danych na podstawie wprowadzonych kryteriów. U dołu ekranu była złożona siatka do wyświetlania danych. Wszystkie te prace pierwotnie zostały opisane jako jedna historyjka użytkownika. Poradziłem zespołowi podzielenie tej historyjki na trzy mniejsze, które zostały rozdzielone na trzy iteracje.

W pierwszej iteracji stworzono podstawowy układ interfejsu użytkownika włącznie z około połową pól kryteriów wyszukiwania w górnej części ekranu. Napisano także fragmenty konstruktora kwerend, który przetwarzał dane wprowadzone w tych polach. Dolna część ekranu była przeznaczona na złożoną siatkę do wyświetlania danych. Pracy było zbyt dużo, aby można ją było zrealizować w czasie dwutygodniowej iteracji. W związku z tym pod koniec pierwszej iteracji w tej części ekranu wyświetlał się

prosty komunikat typu „Znaleziono 312 pozycji”. To oczywiście nie było zbyt przydatne dla użytkownika, który chciał wiedzieć, co zawierały te pozycje. Niemniej jednak był to znaczący postęp, który mogli zobaczyć wszyscy interesariusze projektu.

W czasie drugiej iteracji w tym projekcie dodano siatkę do wyświetlania danych, natomiast w trzeciej iteracji wprowadzono w górnej części ekranu pozostałe pola kryteriów wyszukiwania. Iteracjom nadano takie priorytety ze względu na dużą niepewność związaną z czasem opracowania siatki do wyświetlania danych. Usunięcie tej niepewności w drugiej iteracji uznano za lepsze niż pozostawienie problemu do trzeciej iteracji. Ponieważ zespół już osiągnął dobre zrozumienie tego, co jest potrzebne do utworzenia konstruktora kwerend, uznano, że to zadanie wiąże się z niewielkim ryzykiem. Powyższy opis prowadzi nas do kolejnej wskazówki:



Wskazówka. Duże historyjki użytkownika można dzielić zgodnie z operacjami wykonywanymi w ramach historyjki.

Powszechnie stosowanym podejściem do realizacji tego celu jest podział historyjki zgodnie z popularnymi operacjami **CRUD** (ang. *Create, Read, Update, Delete* — tworzenie, odczyt, aktualizacja, usuwanie). Aby zobaczyć, jak to działa, załóżmy, że pracujesz nad systemem SwimStats, a zespół jest gotowy do realizacji następującej historyjki: „Jako trener mogę zarządzać pływakami w swoim zespole”. Członkowie zespołu porozmawiali z trenerami (użytkownikami) i dowiedzieli się, że trenerzy chcą mieć możliwość dodawania nowych pływaków, edycji istniejących danych dla pływaków oraz usuwania pływaków, którzy opuścili zespół. Historyjkę w takiej wstępnej postaci można łatwo podzielić na trzy historyjki:

- Jako trener mogę dodawać nowych zawodników do swojego zespołu.
- Jako trener mogę modyfikować informacje o pływakach w swoim zespole.
- Jako trener mogę usuwać zawodników, których już nie ma w moim zespole.

Te historyjki są bardzo ściśle powiązane z elementami: tworzenie, aktualizacja i usuwanie operacji CRUD. Podział dużej historyjki użytkownika na takie trzy historyjki to bardzo popularny wzorzec, który prowadzi do następnej wskazówki:



Wskazówka. Duże historyjki użytkownika można dzielić na oddzielne operacje CRUD.

Usuwanie problemów funkcjonalności przekrojowych

W typowej aplikacji istnieje wiele funkcjonalności ortogonalnych lub przekrojowych. Na przykład bezpieczeństwo, obsługa błędów i logowanie — wszystkie te funkcjonalności są wykorzystywane we wszystkich innych funkcjonalnościach aplikacji. Historyjkę, która jest zbyt duża, aby mogła się zmieścić w jednej iteracji, często można zmniejszyć przez wyizolowanie z jednej lub kilku takich przekrojowych funkcjonalności.

Na przykład w wielu aplikacjach występują ekrany, które zachowują się różnie w zależności od uprawnień zalogowanego użytkownika. Jeśli taka funkcjonalność jest zbyt duża, aby można ją było zaimplementować w jednej iteracji, zaimplementuj ekrany w jednej iteracji, a w kolejnej dodaj obsługę uprawnień specyficznych użytkowników.

W projekcie, w którym niedawno uczestniczyłem, klient musiał podzielić historyjkę, która wymagała wyszukiwania danych i wyświetlania wyników. Każda operacja wyszukiwania obejmowała całą bazę danych, ale wyświetlane miały być tylko te wyniki, które użytkownik miał prawo oglądać. Zespół postanowił zignorować to ograniczenie bezpieczeństwa i w pierwszej iteracji tej funkcjonalności użytkownicy mogli oglądać cały zbiór wyników.

Oto inny przykład. Załóżmy, że zespół planuje pracę nad historyjką, która brzmi: „Jako użytkownik przed skorzystaniem z systemu muszę zalogować się przy użyciu nazwy użytkownika i hasła”. Zespół omawia historyjkę i ustala listę ograniczeń na hasło: musi składać się z co najmniej ośmiu znaków, musi zawierać co najmniej jedną cyfrę, nie powinno zawierać żadnych znaków, które powtarzają się trzy lub więcej razy, w przypadku przesyłania lub przechowywania powinno być zaszyfrowane i tak dalej.

Implementacja żadnego z tych ograniczeń nie wydaje się szczególnie czasochłonna, ale łącznie historyjka może stać się zbyt duża, aby mogła się zmieścić w jednej iteracji. Problem ten można rozwiązać przez usunięcie mechanizmów bezpieczeństwa z historyjki logowania i stworzenie wersji historyjki bez zabezpieczeń i z zabezpieczeniami. W drugiej historyjce mogłaby się znaleźć lista planowanych zabezpieczeń, takich jak długość hasła czy ograniczenie dla znaków. Prawdopodobnie nie zdecydujemy się na opublikowanie produktu po zaimplementowaniu wyłącznie pierwszej historyjki — bez żadnych zabezpieczeń, ale podzielenie wyjściowej, dużej historyjki w taki sposób może być wartościowe.

To prowadzi nas do kolejnej wskazówki:



Wskazówka. Należy rozważyć możliwość usunięcia funkcjonalności przekrojowych (na przykład zabezpieczeń, logowania, obsługi błędów) i stworzenie dwóch wersji historyjki: jednej z obsługą funkcjonalności przekrojowych i drugą bez niej.

Nie staraj się spełniać wymagań wydajności

W procesie wytwarzania oprogramowania bardzo często zapominamy (lub ignorujemy) poradę Kernighana i Plaugera (1974): „Najpierw napisz kod tak, żeby działał, a potem, staraj się poprawić go, aby działał szybciej”. Kilka lat temu pracowałem nad projektem, którego zadaniem było wyświetlanie wykresów rynkowych cen akcji. Internauci mogą zażądać wykresu według symbolu akcji. Kod powinien wtedy usunąć historię cen takiej akcji (w ciągu wszystkich okresów do dnia bieżącego z ostatnich pięciu lat) i wyświetlić liniowy wykres ceny akcji. Wśród warunków satysfakcji powiązanych z tą funkcjonalnością były: dokładność wykresu, obsługa brakujących danych i wydajność. Aby spełnić wymagania wydajności, trzeba było zbuforować najczęściej żądane wykresy i generować je ponownie co minutę. Ponieważ buforowanie wymagało znacznych wysiłków, wydzieliliśmy je do nowej historyjki użytkownika i zaplanowaliśmy w następnej iteracji. Bardziej ogólnie to samo podejście może zostać zastosowane do każdego wymagania niefunkcjonalnego. To prowadzi nas do następującej zasady:



Wskazówka. Rozważ podział dużej historyjki przez wydzielenie komponentów funkcjonalnych i niefunkcjonalnych do oddzielnych historyjek.

Takie podejście można zastosować na przykład wtedy, kiedy nowa funkcjonalność musi zużyć mniej niż zdefiniowaną ilość pamięci, wyświetlić się z mniejszą liczbą okien albo wykorzystuje kluczowy zasób krócej niż przez zdefiniowaną ilość czasu.

Podział historyjek o mieszanych priorytetach

Czasami jedna historyjka użytkownika gromadzi wiele mniejszych historyjek o różnych priorytetach. Załóżmy, że projekt zawiera typową historyjkę logowania: „Jako użytkownik muszę się zalogować do systemu”. Właściciel produktu wyraża swoje warunki satysfakcji dla tej historyjki. Zawierają one następujące warunki:

- Jeśli użytkownik wprowadzi poprawny login i hasło, uzyska dostęp.
- Jeśli użytkownik wprowadzi nieprawidłowe hasło trzy razy z rzędu, będzie miał zablokowany dostęp, aż zadzwoni do pomocy technicznej.
- Jeśli użytkownikowi odmówiono dostępu, zostanie do niego wysłana wiadomość e-mail z informacją o nieudanej próbie zalogowania się na jego konto.

Ta historyjka jest zbyt duża, aby mogła się zmieścić w jednej iteracji, dlatego musi zostać podzielona. Historyjkę można podzielić przez wyszukanie elementów o niskim prioryecie. W tym przypadku właściciel produktu nie opublikuje programu, jeśli ten nie będzie wspierał podstawowej funkcjonalności logowania. Może jednak zgodzić się na publikację produktu bez limitu liczby prób lub bez wysyłania wiadomości e-mail o próbie logowania. To prowadzi do kolejnej wskazówki dotyczącej dzielenia historyjek:



Wskazówka. Rozdziel dużą historyjkę na mniejsze w przypadku, gdy mniejsze historyjki mają różne priorytety.

Nie należy dzielić historyjki na zadania

Czasami mamy do czynienia z funkcjonalnością, którą trudno podzielić. W takich sytuacjach kuszący jest podział funkcjonalności na zadania. Większość deweloperów oprogramowania jest tak bardzo przyzwyczajona do analizy funkcjonalności i jej dekomponowania na zadania składowe, że często robi to podświadomie. Na przykład nie należy dzielić historyjki na następujące historyjki składowe:

- Zakoduj interfejs użytkownika.
- Napisz warstwę pośrednią.

Najlepszym sposobem, aby uniknąć tej pokusy, jest stosowanie się do zaleceń Hunta i Thomasa (1999), którzy radzą, aby „wysstrzelić pocisk smugowy” przez system. Pocisk smugowy „przechodzi” przez wszystkie warstwy funkcjonalności. Oznacza to, że należy dostarczyć częściowy interfejs użytkownika, częściową warstwę pośrednią i częściową bazę danych. Dostarczenie spójnego podzbioru wszystkich warstw funkcjonalności prawie zawsze jest lepsze niż dostarczenie wszystkiego w jednej warstwie. To prowadzi do kolejnej wskazówki:



Wskazówka. Nie należy dzielić dużej historyjki na zadania. Zamiast tego postaraj się znaleźć sposób, aby „wysstrzelić pocisk smugowy” przez historyjkę.

Unikaj pokusy zmian powiązanych

Kiedy już podzieliś historyjkę i doprowadzisz je do odpowiedniego rozmiaru, nie pogarszaj sprawy przez dodawanie do historyjki nowej pracy. Często tak się dzieje, jeśli ulegniemy pokusie wprowadzenia zmian powiązanych ze sobą. Mówimy sobie: „Jeśli już jestem przy kodowaniu, to może jeszcze zajmę się innymi, ważnymi zmianami”. Jest wysoce prawdopodobne, że naprawienie lub usunięcie starego problemu przy okazji pracy nad oddzielnym problemem w tej samej części kodu byłoby właściwe. Jednak priorytet wykonania tej pracy powinien być rozpatrywany w taki sam sposób jak priorytety innych funkcjonalności. Innymi słowy, trzeba odpowiedzieć sobie na pytanie: co ma większy priorytet — poświęcenie połowy dnia na wprowadzenie poprawki błędu, o którym wiemy od roku, czy też poświęcenie tej samej ilości czasu na coś innego? Odpowiedź oczywiście w całości zależy od kontekstu i prowadzi do ostatniej wskazówki w tym rozdziale:



Wskazówka. Unikaj pogarszania sytuacji poprzez wprowadzanie powiązanych zmian do historyjki o prawidłowym rozmiarze, jeśli powiązane zmiany nie mają równoważnego priorytetu.

Łączenie historyjek

Zaprezentowana wcześniej duża liczba wskazówek dotyczących podziału historyjek może skłaniać do myślenia, że każda historyjka użytkownika, nad którą zamierzamy pracować, powinna być możliwie jak najmniejsza. Tak nie jest. W przypadku zespołów pracujących w dwutygodniowych iteracjach podział funkcjonalności w taki sposób, że każdą można zrealizować w ciągu od dwóch do pięciu dni, jest odpowiedni (historyjki mogą być szacowane w punktach, ale w momencie, gdy zespół musi podzielić historyjkę, w przybliżeniu wie, ile punktów lub dni idealnych odpowiada okresowi od dwóch do pięciu dni). Historyjki powinny być nieco mniejsze w przypadku iteracji jednodniowych, a w przypadku iteracji trwających dłużej mogą, lecz nie muszą być nieco większe. Przepływ historyjek o odpowiednich rozmiarach jest najlepszy w przypadku krótkich iteracji projektu Agile.

Czasami trzeba dzielić duże historyjki, ale może również dojść do sytuacji, w której pożądane jest scalenie wielu niewielkich historyjek. Połączone historyjki są szacowane jako całość zamiast oceniane indywidualnie. Jeśli to możliwe, należy scalać powiązane ze sobą historyjki. Dzięki temu nadawanie im priorytetów będzie łatwiejsze. Bardzo często łączy się wiele raportów o błędach i traktuje jako jeden element.

Podsumowanie

Czasami może być przydatne podzielenie historyjki, która nie mieści się w iteracji, dlatego że jest zbyt duża, aby zmieścić się w tejże iteracji, lub zbyt duża, aby zmieścić się w czasie pozostałym w planowanej iteracji. Czasami przydaje się również podzielenie dużej historyjki po to, by dostarczyć bardziej precyzyjnej oceny w porównaniu z tą, jaką można wykonać w przypadku jednej, dużej historyjki.

Historyjki można dzielić ze względu na obsługiwane dane. Inne możliwe kryterium podziału bazuje na operacjach właściwych dla historyjki. Powszechnym sposobem dzielenia historyjek jest wykorzystanie popularnych operacji CRUD (tworzenie, odczyt, aktualizacja, usuwanie). Historyjkę

można podzielić na mniejszą poprzez oddzielenie wszelkiego rodzaju funkcjonalności przekrojowych, takich jak zabezpieczenia, logowanie czy obsługa błędów. Historyjkę można również zmniejszyć dzięki zignorowaniu celów wydajnościowych podczas iteracji, w której historyjka została zaimplementowana. Spełnienie celu wydajności może być wydzielone do osobnej historyjki i zrealizowane w późniejszej iteracji. Wiele historyjek opisuje dwie potrzeby lub ich większą liczbę. Jeśli te potrzeby mają różne priorytety, należy podzielić historyjki według priorytetów.

Trzeba unikać dzielenia historyjek na zadania programistyczne niezbędne do zaimplementowania funkcjonalności. Jesteśmy tak przyzwyczajeni do dzielenia pracy na niezbędne zadania, że łatwo zaczynamy w ten sposób dzielić historyjki użytkownika. Unikaj pokusy powiększania dużej historyjki poprzez dołączenie do niej powiązanych zmian, które nie są konieczne do zrealizowania historyjki użytkownika.

Na koniec należy pamiętać, że czasami wskazane jest połączenie historyjek użytkowników — zwłaszcza w przypadku poprawek błędów, kiedy pracy jest zbyt mało na osobną historyjkę użytkownika.

Pytania do dyskusji

1. Wskaż historyjki użytkownika w Twoim bieżącym lub niedawnym projekcie, których podzielenie okazało się trudne? W jaki sposób podzieliłbyś je teraz?
2. Jakie problemy, Twoim zdaniem, powoduje podzielenie historyjki na zadania, a następnie traktowanie tych zadań jako osobnych historyjek użytkownika?

CZĘŚĆ IV

Tworzenie harmonogramów

W poprzednich dwóch częściach opisano sposoby szacowania rozmiaru pożądanej nowej funkcjonalności, a następnie ustalania priorytetów w taki sposób, aby zespół koncentrował się na budowaniu jak najlepszego produktu. W tej części książki skupimy się na tworzeniu harmonogramu.

W kolejnych dwóch rozdziałach przyjrzymy się niezbędnym krokom podczas planowania wydania, a następnie planowania iteracji. W kolejnym rozdziale przedstawiono wskazówki dotyczące wyboru odpowiedniej długości iteracji. Jeśli chcemy ocenić, ile czasu zajmie projekt, to oprócz posiadania wiedzy na temat rozmiaru pracy do wykonania musimy oszacować tempo postępów zespołu podczas wykonywania tej pracy. Z tego powodu ocena szybkości jest tematem kolejnego rozdziału.

W dwóch ostatnich rozdziałach tej części omówiono sytuacje o zwiększonej trudności lub niepewności. Przede wszystkim opisano, jak podejść do planowania projektu, gdy opracowanie błędnego harmonogramu ma ogromne konsekwencje albo kiedy wstępny harmonogram musi zostać opracowany na długo wcześniej i na podstawie bardzo ograniczonych informacji. Tę część kończy rozdział opisujący dodatkowe elementy niezbędne w procesie planowania projektu, który obejmuje wiele zespołów.

ROZDZIAŁ 13.

Podstawy planowania wydań

Improwizujesz. Dostosowujesz się do sytuacji. Przewycięszasz przeszkody.

— Clint Eastwood w *Przełęczu złamanych serc*

Planowanie wydania to proces tworzenia bardzo ogólnego planu obejmującego dłuższy okres niż jedną iterację. Typowe wydanie obejmuje okres od trzech do sześciu miesięcy i w zależności od czasu trwania pojedynczej iteracji — od trzech do dwunastu lub więcej iteracji. Plan wydania jest ważny z wielu powodów.

Po pierwsze, pomaga właścicielowi produktu i całemu zespołowi zdecydować o tym, jaka część produktu powinna zostać zrealizowana, oraz ile czasu upłynie, zanim produkt będzie w stanie nadającym się do opublikowania. Im wcześniej produkt może być wydany (i im większe korzyści uzyskujemy z wydania), tym wcześniej firma zaczyna odzyskiwać środki zainwestowane w projekt.

Po drugie, plan wydania odzwierciedla oczekiwania dotyczące tego, co zostanie zrealizowane oraz w jakim terminie. Wiele firm potrzebuje tych informacji, ponieważ są one danymi wejściowymi dla innych działań planistycznych o strategicznym znaczeniu.

Po trzecie, plan wydania służy jako wyznacznik celu, do którego dąży zespół projektowy. Bez pojęcia wydania zespoły nieustannie przechodzą od jednej iteracji do następnej. Plan wydania dostarcza kontekstu, który pozwala iteracjom połączyć się w satysfakcjonującą całość. Jest to jeden z głównych problemów wszystkich procesów iteracyjnych, nie tylko procesów Agile. Turysta, który chce dotrzeć do szczytu góry, być może zmierza do najwyższego szczytu, który widzi. Jednak kiedy dociera do tego szczytu, dowiaduje się, że to jest fałszywy szczyt. Wyższy szczyt został zakryty przez ten, do którego dotarł. Turysta wyrusza w kierunku wyższego szczytu tylko po to, aby dowiedzieć się, że to również jest fałszywy szczyt, a teraz widoczny jest kolejny, jeszcze wyższy szczyt. Ten okrzęny sposób zdobywania szczytów górskich można równie skutecznie zastosować do projektów iteracyjnych. Na szczęście problemy, które powstają po drodze, są usuwane dzięki planowi wydania, który pokazuje zespołowi bieżące oczekiwania co do celu, do którego zespół ma dotrzeć.

Plan wydania

W pewnym sensie planowanie wydania sprowadza się do ustalenia, ile można osiągnąć w podanym terminie. W niektórych przypadkach zaczynamy od daty i zastanawiamy się, co można zrealizować do tego czasu. Innym razem zaczynamy od zbioru historyjek użytkownika i zastanawiamy się, jak długo potrwa ich implementacja. W obu przypadkach, gdy zespół ma wstępną odpowiedź, poddaje ją ocenie w odniesieniu do celów organizacji względem projektu: czy zaprojektowany produkt pozwoli zarobić odpowiednią sumę pieniędzy? czy pozwoli zaoszczędzić wystarczającą kwotę? czy produkt zdobędzie docelowy udział w rynku? Jeśli nie, to być może wcześniej lub później projekt może osiągnąć akceptowany zbiór celów.

Pobieżna ocena nakładów pracy na wydanie oraz wybranie do niego historyjek użytkownika jest bardzo prostym procesem. Pomnożenie planowanej liczby iteracji przez oczekiwaną lub znaną szybkość zespołu pozwala obliczyć całkowitą ilość zrealizowanej pracy. Następnie wystarczy wybrać pewną liczbę odpowiednich historyjek użytkownika i na tym koniec. Załóżmy, że chcemy dostarczyć nowy produkt za sześć miesięcy. Planujemy pracę w dwutygodniowych iteracjach, zatem projekt będzie trwał trzynaście iteracji. Oczekujemy, że szybkość zespołu będzie wynosić dwadzieścia punktów lub dni idealnych na iterację. Zatem całkowity rozmiar projektu wynosi $13 \times 20 = 260$ punktów lub dni idealnych. Właściciel produktu może zatem przedyskutować z zespołem wszystkie historyjki i ustalić dla nich priorytety tak, aby dostarczyć największą możliwą wartość, a jednocześnie by nie przekroczyć liczby 260.

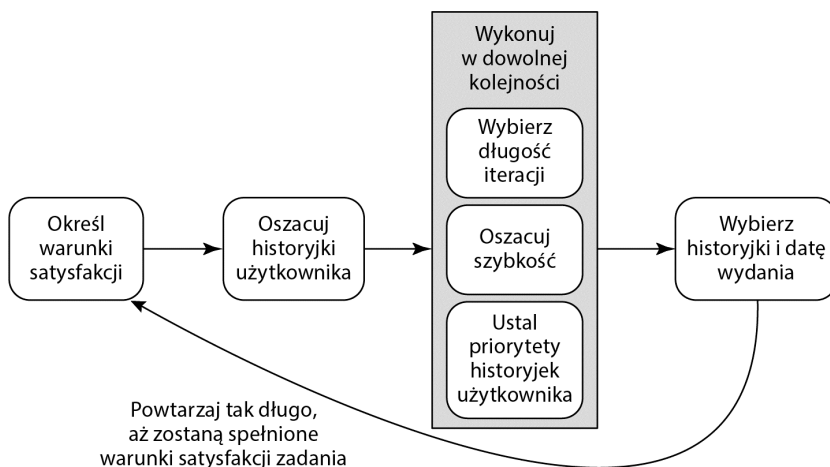
Podczas planowania wydania nie chcemy tworzyć planu wskazującego konkretnych deweloperów, którzy będą pracować nad wybranymi historyjkami użytkownika bądź zadaniami, ani też w jakiej kolejności w ramach jednej iteracji będą realizowane prace. Utworzenie planu na tym poziomie szczegółowości na etapie planowania wydania jest niebezpieczne i mylące. Decyzje o tym, kto pracuje nad jakim zadaniem oraz kolejność działań najlepiej zostawić osobom pracującym nad tymi zadaniami i opóźnić tak bardzo, jak się da. Ponadto należy pamiętać o tym, że pozycjami w planie wydania są historyjki użytkownika, które są opisami dostarczanych funkcjonalności, a nie pojedynczych zadań inżynierskich do zrealizowania. Podczas planowania wydania jest na to zbyt wcześnie, a niektóre historyjki użytkownika mogą nie być wystarczająco jasne do tego, aby można je było podzielić na zadania inżynierskie. Jak przekonamy się w rozdziale 14. „Planowanie iteracji”, zespół ostatecznie podzieli historyjki użytkownika tworzące plan wydania na zadania składowe. Jednak nie zrobi tego wcześniej niż na początku iteracji zawierającej te historyjki.

Oczywiście jeśli Twój projekt, firma i środowisko pracy na to pozwalają, to w planie wydania można umieścić dodatkowe informacje. Na przykład można w nim zamieścić pewne podstawowe założenia leżące u podstaw planu. W szczególności można sformułować założenia dotyczące osób tworzących zespół, czasu trwania iteracji, daty rozpoczęcia pierwszej iteracji oraz daty zakończenia ostatniej. W rozdziale 21. „Ogłaszanie planów” zamieścimy dodatkowe użyteczne informacje, które można uwzględnić w planie wydania.

Ogólne kroki wykonywane podczas planowania wydania zaprezentowano na rysunku 13.1. Każdy z tych kroków zostanie opisany w kolejnych punktach.

Określ warunki satysfakcji

Przed przystąpieniem do planowania wydania trzeba znać kryteria, zgodnie z którymi projekt będzie oceniony jako sukces lub porażka. Dla większości projektów ostatecznym wskaźnikiem jest suma zaoszczędzonych lub zarobionych pieniędzy. W roli wskaźników szans projektu na osiągnięcie



RYSUNEK 13.1. Kroki podczas planowania wydania

wyznaczonych celów finansowych w większości projektów wykorzystuje się triumwirat: harmonogram, zakres i zasoby. Dla większości projektów oznacza to, że warunki satysfakcji właściciela produktu są zdefiniowane za pomocą kombinacji celów dotyczących harmonogramu, zasięgu i zasobów.

Właściciel produktu uwzględnił pożądane cele dla każdego z tych czynników praktycznie na każdym spotkaniu poświęconym planowaniu wydania. Na przykład właściciel produktu może dążyć do opracowania czterech tematów (wartych 200 punktów) w ciągu trzech miesięcy bez dodawania personelu. Chociaż często identyfikowane są cele dla każdego czynnika, to zwykle wiodący jest cel dla jednego z czynników. Projekty zazwyczaj są zarządzane albo przez daty, albo przez funkcjonalności. *Projekt zarządzany przez daty* to taki, który musi być wydany przed ustaloną datą, ale dla którego można negocjować zbiór funkcjonalności. *Projekt zarządzany przez funkcjonalności* to taki, który powinien być wydany jak najwcześniej, ale dla którego za bardziej istotne uważa się zrealizowanie konkretnego zbioru funkcjonalności.

Oszacuj historyjki użytkownika

Ponieważ ocena reprezentuje koszt opracowania historyjki użytkownika, ważne jest dokonanie oceny każdej z nich. Wyobraź sobie, że postanowiłeś wymienić wszystkie elementy garderoby w swojej szafie. Przyjechałeś do centrum handlowego i rozpocząłeś zakupy. Zauważyłeś jednak, że wszystkie metki zostały usunięte i że nie możesz się dowiedzieć, jakie są ceny towarów. Tak mniej więcej czuje się właściciel produktu, jeśli nie dostarczy się mu żadnych ocen.

Nie trzeba oceniać wszystkiego, czego właściciel produktu może kiedykolwiek zażądać. Trzeba jedynie dysponować oceną każdej nowej funkcjonalności, dla której istnieje realna szansa włączenia do najbliższego wydania. Często właściciel produktu ma listę życzeń, która obejmuje dwa, trzy lub więcej przyszłych wydań. Nie ma potrzeby oceniania bardziej odległej pracy.

Wybierz długość iteracji

Większość zespołów Agile pracuje w iteracjach trwających od dwóch do czterech tygodni. Iteracje mogą trwać nieco dłużej, a niektóre zespoły eksperymentowały z krótszymi iteracjami. Podczas planowania wydania należy wybrać odpowiedni czas trwania iteracji. Wskazówki na ten temat znajdują się w rozdziale 15. „Wybór długości iteracji”.

Oszacuj szybkość

Jeśli zespół ma doświadczenie w pracy grupowej, najlepiej wykorzystać ostatnią zaobserwowaną jego szybkość. Oczywiście jeśli technologia lub dziedzina biznesowa diametralnie się zmieniła, to wykorzystanie poprzedniej szybkości zespołu może nie być właściwe. Nadal istnieją techniki, które można zastosować, aby dokonać świadomej oceny szybkości zespołu na podstawie wcześniejszych wyników. Techniki i możliwości szacowania szybkości zespołu omówimy w rozdziale 16. „Szacowanie tempa”.

Ustal priorytety historyjek użytkownika

W większości projektów jest albo za mało czasu, albo zbyt wiele funkcjonalności. W ustalonym czasie często nie da się zrobić wszystkiego. Dlatego właściciel produktu musi ustalić priorytety funkcjonalności, które będą implementowane. Dobry właściciel produktu przyjmuje odpowiedzialność za ustalanie priorytetów, ale słucha rad członków zespołu — zwłaszcza w kwestii ustalania kolejności. Priorytety historyjek użytkowników bazują na wytycznych zaprezentowanych we wcześniejszej części tej książki.

Wybierz historyjki i datę wydania

W tym momencie masz ocenę szybkości pracy zespołu w jednej iteracji oraz założenie na temat planowanej liczby iteracji. Nadszedł czas, aby zobaczyć, czy uda się zaplanować wydanie, które spełnia warunki satysfakcji w projekcie.

Jeśli projekt jest zarządzany przez funkcjonalności, to można podsumować oszacowania dla wszystkich niezbędnych funkcjonalności i podzielić przez oczekiwaną szybkość. W ten sposób można ustalić liczbę iteracji koniecznych do zrealizowania żądanej funkcjonalności.

Jeśli projekt jest zarządzany przez daty, to możemy określić liczbę iteracji, patrząc na kalendarz. W wyniku pomnożenia liczby iteracji przez oczekiwaną szybkość możemy ustalić, ile punktów historyjek lub dni idealnych zmieści się w iteracji. Możemy odliczyć taką liczbę punktów lub dni idealnych na uporządkowanej według priorytetów liście historyjek użytkownika i zobaczyć, jaką część funkcjonalności można dostarczyć w żądanym czasie.

Kolejne pytanie, na które należy znaleźć odpowiedź, dotyczy poziomu szczegółowości planu wydania. Niektóre zespoły w niektórych środowiskach wolą tworzyć plan wydania, który pokazuje, jakie oczekiwane funkcjonalności powinny zostać utworzone w każdej iteracji. Inne wolą ustalić, co ich zdaniem będzie dostarczone podczas całego wydania, a specyfikę każdej iteracji pozostawiają na później. Zespół powinien przedyskutować te podejścia i zdecydować, które wybrać podczas planowania wydania.

Każde z nich ma swoje zalety i wady. Oczywiście przypisanie konkretnych funkcjonalności do poszczególnych iteracji zajmuje więcej czasu. Jednak dodatkowe szczegóły, jakie dzięki temu

uzyskujemy, mogą się przydać podczas koordynowania pracy między wieloma zespołami. Z drugiej strony rezygnacja z przydzielenia pracy do konkretnych iteracji powoduje, że dostarczanych jest mniej szczegółów, ale zajmuje to znacznie mniej czasu. Co więcej, nawet gdybyśmy zdecydowali się na wstępne rozdzielanie prac do konkretnych iteracji, to robilibyśmy to, dysponując mniejszą wiedzą, niż mielibyśmy w przypadku planowania prac na początku każdej iteracji. Bez wątpienia plan będzie się zmieniać w miarę zdobywania nowej wiedzy w czasie trwania projektu. Dlatego decydując się na przydzielenie prac do konkretnych iteracji, trzeba wziąć pod uwagę związane z tym koszty czasowe i nakład pracy. Pomimo kosztów w niektórych projektach stosowanie takiego podejścia może być opłacalne. Dla mnie dobrym kompromisem jest przydzielenie konkretnych zadań dla pierwszych kilku iteracji (na przykład od jednej do trzech) i traktowanie pozostałej części planu wydania jak jednego dużego kontenera. Prawie zawsze opłaca się przypisać konkretne zadania do pierwszej iteracji, zwłaszcza jeśli zaczyna się od razu.

Ponieważ podczas typowej sesji planowania wydania powstaje wiele problemów typu „coś za coś” oraz pytań w rodzaju „co się stanie, gdy”, to potrzebny jest łatwy sposób manipulowania wejściami i wyjściami wydania oraz składających się na nie iteracji. Najlepszym sposobem osiągnięcia tego celu, przy założeniu, że wszystko jest ze sobą powiązane, jest posługiwanie się karteczkami w formacie A6 (lub samoprzylepnymi karteczkami), na których są zapisane historyjki użytkownika lub są opisane funkcjonalności. W przeciwieństwie do oprogramowania karteczki są namacalne i łatwo można się nimi wymieniać.

W celu zaplanowania wydania właściciel produktu wybiera pozycje o najwyższym priorytecie, które zmieszczą się w pierwszej iteracji. Kartki są ułożone w stosach, które są zorganizowane według historyjek wchodzących w skład każdej iteracji. Jeden ze sposobów takiej organizacji karteczek pokazano na rysunku 13.2. Karteczki można ułożyć w ten sposób na stole lub przypiąć na ścianie. Jeśli korzystamy ze ściany, bardzo przydatna jest tablica korkowa, do której można przypinać i odpinać karteczki, zamiast przyklejać je do ściany. Ponieważ każda karteczka historyjki jest opisana oceną, to można spojrzeć na poszczególne kolumny układu z rysunku 13.2 i sprawdzić, czy zaplanowana ilość pracy w każdej iteracji jest właściwa.

1	2	Iteracje 3 – 5		
Jako użytkownik chcę... 3	Jako użytkownik chcę... 6	Jako użytkownik chcę... 5	Jako użytkownik chcę... 4	Jako użytkownik chcę... 3
Jako użytkownik chcę... 5	Jako użytkownik chcę... 5	Jako użytkownik chcę... 5	Jako użytkownik chcę... 5	Jako użytkownik chcę... 5
Jako użytkownik chcę... 3	Jako użytkownik chcę... 2	Jako użytkownik chcę... 1	Jako użytkownik chcę... 4	Jako użytkownik chcę... 4
Jako użytkownik chcę... 2		Jako użytkownik chcę... 2		Jako użytkownik chcę... 1

RYSUNEK 13.2. Organizacja iteracji w kolumnach

Aktualizacja planu wydania

W tym momencie plan wydania jest gotowy. Jednak jest bardzo ważne, aby plan wydania nie został teraz zarchiwizowany lub odłożony na półkę po to, żeby nigdy do niego nie wracać.

Do planu wydania trzeba wracać i należy go regularnie aktualizować. Jeśli szybkość pracy zespołu jest względnie stała, a podczas planowania iteracji nie odkryto żadnych większych niespodzianek, to plan wydania może nie być formalnie aktualizowany nawet przez okres od czterech do sześciu tygodni. Z drugiej strony w wielu projektach korzystne jest ustanowienie reguły, zgodnie z którą plan wydania jest aktualizowany po każdej iteracji.

Przykład

Aby połączyć wszystkie wcześniej zaprezentowane informacje, przyjrzyjmy się krótkiemu przykładowi. Ponownie posłużymy się przykładem witryny SwimStats. W wyniku badania rynku okazało się, że trenerzy i pływacy w naszej grupie docelowej potrzebują funkcjonalności opisanych przez następujące historyjki użytkownika:

- Jako trener mogę definiować sesje treningowe.
- Jako trener mogę definiować niestandardowe pola, które chciałbym śledzić dla każdego pływaka.
- Jako pływak mogę uruchomić raport, który pokazuje moje postępy we wszystkich zawodach, począwszy od podanej daty w przeszłości.
- Jako pływak mogę zaktualizować swoje dane osobowe.
- Jako trener mogę wysłać wiadomości e-mail do wszystkich pływaków w drużynie.
- Jako administrator systemu mogę skonfigurować uprawnienia i grupy użytkowników.
- Jako trener mogę kupić aplikację SwimStats i używać jej dla swojej drużyny.
- Jako pływak mogę porównać swoje czasy z krajowymi rekordami.
- Jako trener mogę wpisać nazwiska i informacje o wszystkich zawodnikach swojej drużyny.
- Jako pływak mogę zobaczyć swoje najlepsze czasy we wszystkich zawodach.
- Jako użytkownik muszę się zalogować i mogę zobaczyć tylko te dane, do których mam prawo.
- Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.

Określanie warunków satysfakcji

Wiosenny sezon pływacki rozpoczyna się za cztery miesiące. Pierwsze wydanie serwisu SwimStats musi być dostępne na miesiąc przed tą datą. Właściciel produktu chciałby zrealizować jak najwięcej funkcjonalności, ale to jest projekt zarządzany przez daty. Jakaś część funkcjonalności musi być gotowa do wydania za trzy miesiące, nawet jeśli są to tylko podstawowe funkcjonalności dla niewielkich drużyn pływackich. Ponieważ firma jest start-upem, to jej budżet również jest ograniczony. Projekt powinien zostać zrealizowany przez jednego programistę, jednego administratora baz danych i jednego testera, którzy już są w zespole.

Szacowanie

Trzech deweloperów, którzy będą pracować nad projektem, spotkało się z właścicielem produktu. Zadali mu pytania dotyczące historyjek użytkownika, a właściciel produktu wyjaśnił cel każdej z nich. Zespół zdecydował, że będzie oceniał historyjki w punktach, i przypisał oceny zestawione w tabeli 13.1.

TABELA 13.1. *Lista historyjek z nadanymi priorytetami*

Historyjka	Ocena
Jako trener mogę kupić aplikację SwimStats i używać jej dla swojej drużyny.	5
Jako użytkownik muszę się zalogować i mogę zobaczyć tylko te dane, do których mam prawo.	5
Jako trener mogę wpisać nazwiska i informacje o wszystkich zawodnikach swojej drużyny.	8
Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.	5
Jako pływak mogę zaktualizować swoje dane osobowe.	5
Jako pływak mogę zobaczyć swoje najlepsze czasy we wszystkich zawodach.	5
Jako pływak mogę uruchomić raport, który pokazuje moje postępy we wszystkich zawodach, počawszy od podanej daty w przeszłości.	5
Jako pływak mogę porównać swoje czasy z krajowymi rekordami.	8
Jako trener mogę definiować sesje treningowe.	8
Jako trener mogę wysyłać wiadomości e-mail do wszystkich pływaków w drużynie.	3
Jako trener mogę definiować niestandardowe pola, które chciałbym śledzić dla każdego pływaka.	8
Jako administrator systemu mogę skonfigurować uprawnienia i grupy użytkowników.	3
Razem	68

Wybór długości iteracji

Ponieważ cały projekt ma trwać tylko trzy miesiące, deweloperzy wspólnie z właścicielem produktu ustalili, że czterotygodniowe iteracje są zbyt długie. W przypadku stosowania czterotygodniowych iteracji byłoby zbyt mało punktów kontroli postępów zespołu oraz zbyt mało okazji do tego, aby właściciel produktu mógł kierować projektem poprzez korektę priorytetów. Na podstawie punktów, które zostaną przedstawione w rozdziale 15. „Wybór długości iteracji”, wszyscy zgodzili się na iteracje dwutygodniowe.

Szacowanie szybkości

W tym projekcie będzie pracowało dwóch programistów i tester. Posługując się technikami, które poznamy w rozdziale 16. „Szacowanie tempa”, przewidują szybkość ośmiu punktów na iterację.

Ustalanie priorytetów historyjek użytkownika

Na podstawie badań rynku właściciel produktu ustalił priorytety historyjek. Listę historyjek według priorytetów wraz z ocenami dla każdej historyjki zestawiono w tabeli 13.1.

Wybór historyjek użytkowników

Ponieważ jest to projekt sterowany datami, to wiemy, ile będzie w nim iteracji. Właściciel produktu chce opublikować wydanie za trzy miesiące, zatem jest czas na sześć dwutygodniowych iteracji, a następnie jeszcze jeden tydzień w razie potrzeby. Przy szacowanej szybkości ośmiu punktów w każdej iteracji oznacza to, że wydanie będzie zawierało $6 \times 8 = 48$ punktów. Właściciel produktu może wybrać z tabeli 13.1 i dołączyć do wydania historyjki o łącznej wartości punktowej wynoszącej czterdzieści osiem punktów.

Historyjki użytkowników w tabeli 13.1 zostały posortowane według właściciela produktu w kolejności malejącej, zgodnie z oceną wartości wybranej historyjki w początkowym wydaniu serwisu SwimStats. Właściciel produktu może wybrać do czterdziestu ośmiu punktów. Początkowo stwierdził, że chciałby wybrać pierwszych osiem historyjek — do historyjki „Jako pływak mogę porównać swoje czasy z krajowymi rekordami”. Łączna ocena punktowa tych historyjek wynosi czterdzieści sześć. Jest to wystarczająco bliska wartość docelowej wartości czterdziestu ośmiu punktów, tak że właściciel produktu postanowił na tym poprzestać.

Jednak deweloperzy zauważyli, że historyjka, dla której właściciel produktu ustalił najniższy priorytet, jest konieczna: „Jako administrator systemu mogę skonfigurować uprawnienia i grupy użytkowników”. Tę historyjkę oszacowano na trzy punkty. Dodanie jej do planu wydania spowodowałoby, że suma punktów historyjek wynosiłaby czterdzieści dziewięć, czyli więcej niż planowane czterdzieści osiem. Czterdzieści osiem punktów to dosyć niedokładne oszacowanie, a po sześciu iteracjach zaplanowano dodatkowy tydzień. W tych okolicznościach niektóre zespoły mogłyby się zgodzić na wydanie o wartości czterdziestu dziewięciu punktów.

Jednak zespół SwimStats zdecydował, że wartość czterdziestu dziewięciu punktów na wydanie jest zbyt wysoka. Jeśli trzeba dodać historyjkę ocenioną na trzy punkty, to z wydania trzeba odjąć co najmniej jeden punkt. Właściciel produktu zdecydował się usunąć historyjkę ocenioną na osiem punktów: „Jako pływak mogę porównać swoje czasy z krajowymi rekordami”. Z tego powodu wartość punktowa wydania spadła do czterdziestu jeden punktów. Właściciel produktu mógłby dodać ocenioną na trzy punkty historyjkę: „Jako trener mogę wysłać wiadomości e-mail do wszystkich pływaków w drużynie”, ale postanowił tego nie robić. Jeśli zespołowi uda się zyskać jeden punkt podczas iteracji, to wolałby ponownie dodać ośmiopunktową historyjkę, z której wcześniej zrezygnował. To sprawia, że ostateczny plan wydania ma postać pokazaną w tabeli 13.2.

Podsumowanie

Plan wydania to wysokopoziomowy plan, który obejmuje okres dłuższy niż jedna iteracja. Dla większości zespołów wydanie odbywa się co trzy do sześciu miesięcy. Nie jest jednak niczym niezwykłym, aby — w zależności od oprogramowania — wydania odbywały się częściej lub rzadziej. Na najprostszym poziomie planowanie wydania jest trywialne: należy pomnożyć oczekiwaną szybkość zespołu przez liczbę zaplanowanych iteracji, a następnie wybrać te historyjki, dla których suma punktów wypełnia wydanie.

TABELA 13.2. *Ostateczny plan wydania aplikacji SwimStats*

Historyjka	Ocena
Jako trener mogę kupić aplikację SwimStats i używać jej dla swojej drużyny.	5
Jako administrator systemu mogę skonfigurować uprawnienia i grupy użytkowników.	3
Jako użytkownik muszę się zalogować i mogę zobaczyć tylko te dane, do których mam prawo.	5
Jako trener mogę wpisać nazwiska i informacje o wszystkich zawodnikach swojej drużyny.	8
Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.	5
Jako pływak mogę zaktualizować swoje dane osobowe.	5
Jako pływak mogę zobaczyć swoje najlepsze czasy we wszystkich zawodach.	5
Jak pływak mogę uruchomić raport, który pokazuje moje postępy we wszystkich zawodach, począwszy od podanej daty w przeszłości.	5
Razem	41

Plan wydania nie musi dokładnie opisywać, jakie prace będą realizowane podczas każdej iteracji. W gruncie rzeczy rzadko potrzebny jest taki poziom szczegółowości. W większości projektów w zupełności wystarczy zidentyfikowanie historyjek, nad którymi będą trwały prace w pierwszych kilku iteracjach, i pozostawienie reszty historyjek do dalszego ustalania priorytetów później — podczas konkretnych iteracji.

Planowanie wydania to proces iteracyjny, który rozpoczyna się od określenia przez właściciela produktu warunków satysfakcji dla projektu. Zazwyczaj obejmują one cele dla harmonogramów, zakresu i zasobów. Jeśli nie można zaplanować projektu spełniającego szeregu początkowych warunków satysfakcji, to proces planowania jest powtarzany. Dzięki temu można się przekonać, czy może zostać spełniony mniej restrykcyjny zbiór warunków — na przykład czy pożądaną funkcjonalność można dostarczyć nieco później lub przy zaangażowaniu większego zespołu.

Po utworzeniu planu wydania nie pozostawia się go tylko po to, żeby wisiał na ścianie. Zwykle plan jest aktualizowany na początku każdej iteracji.

Pytania do dyskusji

1. Czy Twój bieżący projekt jest zarządzany przez daty, czy przez funkcjonalności? Na jakiej podstawie tak twierdzisz?
2. Jak często powinien być aktualizowany plan wydania w Twojej firmie lub w Twoim bieżącym projekcie?

ROZDZIAŁ 14.

Planowanie iteracji

Teoretyzowanie, gdy nie ma się danych, to kardynalny błąd.

— Sherlock Holmes, *Skandal w Czechach*

Plan wydania to doskonały, wysokopoziomowy sposób obserwacji tego, jak zespół zamierza dostarczyć najbardziej wartościowy produkt, jaki jest w stanie dostarczyć. Jednak plan wydania zapewnia jedynie ogólną perspektywę tworzonego produktu. Nie tworzy krótkoterminowego, bardziej szczegółowego obrazu, który zespoły będą mogły wykorzystać do zarządzania pracami podczas iteracji. Za pośrednictwem planu iteracji zespół stara się w bardziej skoncentrowany i szczegółowy sposób przeanalizować wszystkie czynniki, które są niezbędne do całkowitego zaimplementowania tylko tych historyjek użytkownika, które zostały wybrane do nowej iteracji.

Plan iteracji tworzy się na spotkaniu poświęconym planowaniu iteracji. W takim spotkaniu powinni uczestniczyć: właściciel produktu, analitycy, programiści, testerzy, inżynierowie baz danych, projektanci interakcji z użytkownikiem i tak dalej. Na spotkaniu powinny być obecne wszystkie osoby biorące udział w procesie przekształcania surowej koncepcji w działający produkt.

Plan iteracji może mieć formę prostego arkusza kalkulacyjnego lub zbioru kartek z notesu, gdzie na każdej kartce ręcznie zapisano po jednym zadaniu. W każdym przypadku zadania i historyjki powinny być zorganizowane w taki sposób, aby można było ustalić, które zadania dotyczą których historyjek. Dla przykładu spójrzmy na tabelę 14.1, która zawiera plan iteracji w arkuszu kalkulacyjnym. Zadania zostały zapisane po jednym w każdym wierszu i są oznaczone wcięciem w stosunku do historyjki, której dotyczą.

Alternatywę dla arkusza kalkulacyjnego przedstawiono na rysunku 14.1, na którym pokazano zastosowanie do planowania iteracji karteczek z notesu. Karteczki można rozmieścić w taki sposób, jak pokazano na rysunku — na tablicy lub podłodze albo poprzez przyklejenie taśmą lub przypięcie pinezkami na ścianie. W przypadku zespołów pracujących w jednej lokalizacji preferuję planowanie iteracji z wykorzystaniem karteczek. Zespół, jeśli sobie tego życzy, może wprowadzić karteczki do systemu informatycznego natychmiast po zakończeniu spotkania poświęconego planowaniu, jednak największe korzyści osiąga dzięki używaniu karteczek podczas spotkania.

TABELA 14.1. Plan iteracji zaprezentowany w formie prostego arkusza kalkulacyjnego

Wiersz	Historijka użytkownika (zadanie)	Godzin
1.	Jako trener mogę wyznaczać pływaków do udziału w zawodach.	
2.	Określ reguły rządzące tym, kto może brać udział w zawodach pływackich, oraz jakiego rodzaju są to zawody.	6
3.	Określ testy akceptacyjne, które pokazują, jak powinna działać ta funkcjonalność.	8
4.	Zaprojektuj interfejs użytkownika.	16
5.	Zakoduj interfejs użytkownika.	8
6.	Dodaj tabele i procedury składowane do bazy danych.	6
7.	Zautomatyzuj testy.	6
8.	Jako pływak mogę aktualizować swoje dane osobowe.	
9.	Określ testy akceptacyjne.	5
10.	Zmodyfikuj stronę danych osobowych, która jest tylko do odczytu, tak aby pozwalała na edycję danych.	6
11.	...	

Historijka	Zadania	
Jako trener mogę wyznaczać pływaków do udziału w zawodach	Określ reguły rządzące tym, kto może brać udział w zawodach pływackich, oraz jakiego rodzaju są to zawody 6	Określ testy akceptacyjne, które pokazują, jak powinna działać ta funkcjonalność 8
	Zaprojektuj interfejs użytkownika 16	Zakoduj interfejs użytkownika 8
Jako pływak mogę aktualizować swoje dane osobowe	Określ testy akceptacyjne 5	Zmodyfikuj stronę danych osobowych, która jest tylko do odczytu, tak aby pozwalała na edycję danych 6

RYСУNEK 14.1. Planowanie iteracji można zrealizować za pomocą karteczek z notesu na tablicy lub na ścianie

Jedną z najważniejszych zalet korzystania z karteczek podczas planowania iteracji jest możliwość uczestniczenia w procesie wszystkich członków zespołu. Jeśli podczas spotkania poświęconego planowaniu iteracji zadania są wprowadzane do systemu informatycznego, to w czasie tego spotkania ktoś przez cały czas siedzi przy klawiaturze. Posiadanie kontroli nad klawiaturą daje olbrzymie możliwości. Powinny być rejestrowane wszelkie rozmowy. Inaczej nigdy nie uda się stworzyć planu wydania. Co gorsza, osoba, która kontroluje klawiaturę, może zmienić wszystko, co jest wprowadzane do planu wydania.

Te olbrzymie możliwości potwierdzają dwa przykłady. W pierwszym przypadku zespół omówił określoną pozycję i zdecydował, że należy ją oszacować na dwanaście godzin. Kontrolę nad klawiaturą sprawował zespół w składzie: menedżer projektu, lider techniczny. Wprowadzili do systemu ocenę osiem, ponieważ ich zdaniem „w żaden sposób to nie może potrwać tak długo”, pomimo że istniało nikłe prawdopodobieństwo, aby którakolwiek z tych osób została wyznaczona do wykonania tego zadania.

W drugim przypadku zespół, dla którego byłem trenerem, dyskutował nad sposobem implementacji nowej funkcjonalności — czy będzie zaimplementowana w postaci kodu Javy po stronie serwera, czy jako procedura składowana w bazie danych? Wszyscy uczestnicy spotkania, z wyjątkiem lidera zespołu siedzącego przy klawiaturze, zgodzili się, że funkcjonalność zostanie zaimplementowana z wykorzystaniem procedur składowanych. Poproszono go, aby wprowadził do arkusza zadanie „Dodanie procedur składowanych”. Zamiast tego wpisał zadanie „Napisać kod odpowiedzialny za składowanie danych”. Jego przesłanie było jasne: ten problem jeszcze nie został rozwiązany.

Porównajmy te dwie sytuacje ze spotkaniem poświęconym planowaniu iteracji, na którym każdy może w dowolnym momencie wziąć karteczkę i zapisać na niej zadanie. Korzystanie z karteczek to znacznie bardziej demokratyczne i grupowe podejście, które prawdopodobnie doprowadzi do lepszych wyników w czasie iteracji oraz projektu, a nie tylko podczas tego spotkania.

Podczas planowania iteracji nie są przydzielane zadania

Zanim przyjrzymy się działaniom, które są wykonywane w czasie planowania iteracji, ważne, abyśmy wskazali jedną czynność, która nie jest wówczas wykonywana. Podczas planowania iteracji nie przydziela się zadań konkretnym osobom. Na początku iteracji może wydawać się oczywiste, kto będzie pracować nad konkretnymi zadaniami. Jednak na podstawie postępów całego zespołu podczas realizacji całego zbioru zadań może się okazać, że to, co było oczywiste na początku, nie musi być tym, co się wydarzy podczas iteracji. Na przykład podczas planowania iteracji mogliśmy założyć, że zadanie „Dostroić zaawansowaną kwerendę wyszukiwania” zrealizuje nasz administrator bazy danych, ponieważ spośród wszystkich członków zespołu ma największe umiejętności w zakresie języka SQL. Jeśli jednak nie będzie mógł podjąć się wykonania tego zadania, to musi je zrealizować ktoś inny.

Pojedynczy członkowie zespołu nie zgłaszają się do wykonywania zadań, zanim nie zacznie się iteracja i, ogólnie rzecz biorąc, zgłaszają się tylko do jednego lub dwóch powiązanych ze sobą zadań jednocześnie. Nie rozpoczyna się pracy nad nowymi zadaniami, dopóki nie zostaną zakończone prace nad zadaniami wybranymi wcześniej.

Przypisanie konkretnych osób do zadań podczas planowania iteracji nie pozwala zbyt wiele zyskać, a stwarza ryzyko sporych strat. W projektach najlepiej sprawdza się postawa w rodzaju „wszyscy robimy to razem” — gdy członkowie zespołu przejmują odpowiedzialność za zadania innych, wiedząc, że jeśli znajdą się w podobnej sytuacji, to będą mogli liczyć na rewanż. Zgłaszanie się poszczególnych członków zespołu do wykonania określonych zadań na początku iteracji szkodzi wzmocnieniu poczucia jedności w dążeniu do osiągnięcia celu iteracji.

Różnice między planowaniem iteracji a planowaniem wydania

Plan wydania uwzględnia perspektywę jednego wydania produktu, co zwykle odpowiada okresowi od trzech do sześciu miesięcy od chwili rozpoczęcia nowego projektu. Dla odróżnienia plan iteracji uwzględnia jedynie okres trwania pojedynczej iteracji, czyli zazwyczaj od dwóch do czterech tygodni. Historyjki użytkownika wchodzące w skład planu wydania są dekomponowane na zadania w planie iteracji. Historyjki użytkownika wchodzące w skład planu wydania są oceniane w punktach lub dniach idealnych, natomiast zadania w planie iteracji są oceniane w godzinach idealnych.

Dlaczego zadania planu iteracji szacuje się w godzinach, a historyjki użytkownika wchodzące w skład planu wydania są szacowane w punktach lub dniach idealnych? Przede wszystkim dlatego, że można tak to zrobić. Praca w ramach jednej iteracji trwa nie więcej niż kilka tygodni, a zespół powinien dobrze rozumieć wykonywane prace — zwłaszcza po ich omówieniu podczas spotkania poświęconego planowaniu iteracji. Dzięki temu zespół może rzetelnie ocenić zadania w iteracji w godzinach. Każda z historyjek użytkownika, która składa się na wydanie, reprezentuje wiele zadań. Są one bardziej „mgliste” i gorzej rozumiane, dlatego muszą być szacowane w bardziej abstrakcyjnych jednostkach — na przykład punktach lub dniach idealnych.

Podstawowe różnice między planem wydania a planem iteracji zestawiono w tabeli 14.2.

TABELA 14.2. Główne różnice między planem wydania a planem iteracji

	Plan wydania	Plan iteracji
Horyzont planowania	3 – 9 miesięcy	1 – 4 tygodnie
Pozycja w planie	Historyjki użytkownika	Zadania
Szacuje się	W punktach historyjek lub dniach idealnych	W godzinach idealnych

Głównym celem planowania iteracji jest uściślenie założeń dokonanych w mniej szczegółowym planie wydania. Plan wydania jest zazwyczaj celowo nieprecyzyjny, jeśli chodzi o konkretną kolejność, w jakiej będą implementowane określone historyjki użytkownika. Ponadto w czasie planowania iteracji zespół wie więcej niż w czasie ostatniej aktualizacji planu wydania. W momencie rozpoczęcia planowania iteracji zespół może skorzystać z ostatnio zdobytej wiedzy. Tak więc planowanie w projekcie Agile staje się procesem dwufazowym. Pierwsza faza to plan wydania — charakteryzujący się brakiem precyzji i ogólną niepewnością. Druga faza to plan iteracji. W planie iteracji nadal istnieją pewne nieścisłości oraz nadal brakuje pewności. Ponieważ jednak plan iteracji tworzy się równocześnie z rozpoczęciem nowej iteracji, to jest on bardziej szczegółowy od planu wydania.

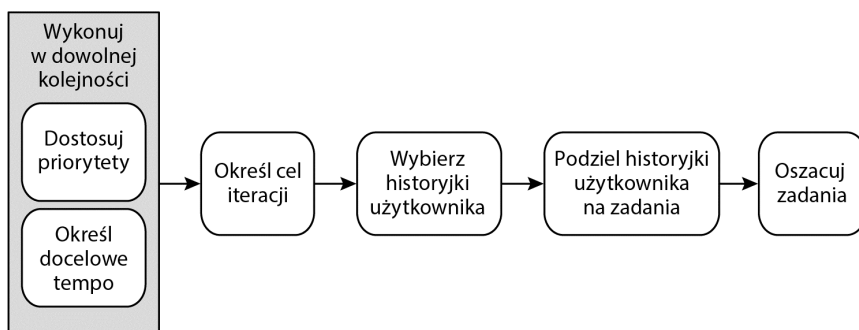
Tworzenie planu iteracji prowadzi zespół do dyskusji na temat zarówno projektu produktu, jak i projektu oprogramowania. Przykładowa dyskusja na temat projektu produktu może dotyczyć takich tematów jak najlepsze połączenie historyjek w celu optymalizacji wartości, interpretacja opinii wynikających z prezentacji działającego oprogramowania użytkownikom oraz stopień, w jakim powinna zostać zaimplementowana żądana funkcjonalność (tzn. czy 20% funkcjonalności i wysiłku przyniesie 80% wartości?). Dyskusje na temat projektu oprogramowania mogą dotyczyć na przykład wyboru odpowiedniej warstwy architektonicznej, w której zostanie zaimplementowana nowa funkcjonalność,

stosowanych technologii, możliwości wykorzystania istniejącego kodu i tak dalej. W wyniku tych dyskusji zespół zyskuje lepsze zrozumienie tego, co powinno zostać zbudowane oraz co zostanie zbudowane. Ponadto zespół utworzy listę zadań potrzebnych do osiągnięcia celu iteracji.

Planowanie iteracji sterowane tempem

Ogólnie rzecz biorąc, istnieją dwa podejścia do planowania iteracji. Określam je terminami *sterowane tempem* (ang. *velocity-driven*) oraz *sterowane zobowiązaniami* (ang. *commitment-driven*). Różne zespoły stosują różne podejścia. Każde z nich może być skuteczne. Ponadto te dwa podejścia można w różnym stopniu ze sobą łączyć. W tym podrozdziale przyjrzymy się planowaniu iteracji zarządzanemu tempem, w następnym skoncentrujemy się na planowaniu iteracji sterowanym zobowiązaniami.

Fazy planowania iteracji zarządzanego tempem zestawiono na rysunku 14.2. Najpierw zespół wspólnie ustala priorytety. Czasami zespół podczas poprzedniej iteracji uczy się czegoś, co wpływa na zmianę ustalonych priorytetów. Następnie określa docelowe tempo nadchodzącej iteracji. W dalszej kolejności zespół wybiera cel iteracji, który stanowi ogólny opis tego, co chciałby zrealizować w trakcie nadchodzącej iteracji. Po wybraniu celu iteracji zespół wybiera historyjki użytkownika o najwyższych priorytetach, które wspierają realizację tego celu. Zespół wybiera tyle historyjek użytkownika, aby suma ich ocen w punktach lub dniach idealnych była równa wartości docelowego tempa zespołu. Na koniec każdą wybraną historyjkę użytkownika dzieli się na zadania, a każde zadanie jest poddawane ocenie. Wymienione kroki opisano bardziej szczegółowo w dalszej części tego rozdziału.



RYSUNEK 14.2. Ciąg działań w planowaniu iteracji zarządzanym tempem

Dostosuj priorytety

Wyobraź sobie, że wszystkie historyjki użytkownika zostały fizycznie ułożone lub posortowane w arkuszu kalkulacyjnym w taki sposób, że historyjka o największej wartości jest na górze, natomiast najmniej wartościowa historyjka znajduje się na dole. Projekt może zostać realizowany w taki sposób, że z tej uporządkowanej według priorytetów listy na początku każdej iteracji są pobierane historyjki o najwyższych priorytetach. Jednak warunki biznesowe i warunki projektu szybko się zmieniają, dlatego zawsze warto poddać priorytety szybkiej ponownej analizie.

Jednym ze źródeł zmian w priorytetach jest *spotkanie przeglądu iteracji* (ang. *iteration review meeting*), które odbywa się po zakończeniu iteracji. Podczas trwania przeglądu iteracji interesariuszom,

szerokiej społeczności projektu oraz wszystkim zainteresowanym stronom prezentowane są nowe funkcjonalności, które zostały dodane podczas iteracji. W trakcie tych przeglądów iteracji często zespół zyskuje cenne opinie. Sam właściciel produktu, ogólnie rzecz biorąc, nie powinien wymyślać nowych pomysłów i zmian podczas przeglądu iteracji, ponieważ w czasie trwania iteracji uczestniczy w pracach zespołu na co dzień. Jednak wiele innych osób (w tym potencjalni klienci i użytkownicy) mogą widzieć wyniki iteracji po raz pierwszy. Często mają one dobre nowe pomysły, które mogą spowodować „wywłaszczenie” pozycji, którym wcześniej nadano wysoki priorytet.

Zgodnie z tym, co napisano w rozdziale 9. „Priorytety tematów”, priorytety historyjek użytkownika i tematów ustala się na podstawie ich finansowej wartości dla produktu, kosztów, ilości i znaczenia wiedzy zdobytej przez zespół oraz rozmiarów wyeliminowanego ryzyka. Byłoby najlepiej, aby zespół poczekał do spotkania przeglądu iteracji, zanim zacznie ustalać priorytety dla nadchodzącej iteracji. Trzeba wziąć pod uwagę, że to, co zespół usłyszy w trakcie spotkania przeglądu iteracji, może mieć wpływ na tenże zespół. Trudno ustalić priorytety prac w następnej iteracji, jeśli nie jesteśmy całkowicie pewni, jakie zadania zostaną zrealizowane w iteracji bieżącej. Jednak w wielu organizacjach sprawdziło się prowadzenie spotkania poświęconego ustalaniu priorytetów na kilka dni przed rozpoczęciem nowej iteracji. W ten sposób łatwiej dopasować harmonogram tak, aby spotkania przeglądu iteracji i planowania iteracji przypadły w tym samym dniu.

Przegląd iteracji zwykle zajmuje od trzydziestu do sześćdziesięciu minut. W przypadku dużego produktu, nad którego realizacją pracuje wiele zespołów, jest całkiem realne, aby właściciel produktu oraz inni interesariusze potrzebni do przeprowadzenia dyskusji na temat priorytetów poświęcili pół dnia na przegląd iteracji. Jeśli dodamy jeszcze cztery godziny na planowanie iteracji, to może nie starczyć czasu na przedyskutowanie priorytetów tego samego dnia.

Zwykle planuję spotkanie dotyczące ustalania priorytetów na dwa dni przed końcem iteracji. Do tego czasu zazwyczaj jest już jasne, czy z bieżącej iteracji pozostaną jakieś niedokończone prace. Dzięki temu właściciel produktu może zdecydować, czy dokończenie tych prac będzie priorytetem na najbliższą iterację. Właściciel produktu przeprowadza spotkanie na temat priorytetów i zaprasza do uczestnictwa w nim wszystkie osoby, które w jego opinii powinny wziąć udział w dyskusji nad priorytetami projektu. Po przeprowadzeniu tego spotkania właściciel produktu, na podstawie danych z przeglądu iteracji, zazwyczaj potrafi szybko i na bieżąco korygować priorytety poszczególnych zadań.

Określ docelowe tempo

Kolejnym krokiem podczas planowania iteracji zarządzanego tempem jest określenie docelowego tempa zespołu. Domyślnie większość zespołów zakłada, że tempo w następnej iteracji będzie równe tempu ostatniej iteracji. Beck i Fowler (2000) nazywają ten sposób „wczorajsza pogoda”, ponieważ najlepszą podstawą do odgadnięcia, jaka będzie dzisiejsza pogoda, jest założenie, że będzie taka jak wczorajsza. Inne zespoły wolą używać kroczącej średniej na przestrzeni na przykład trzech ostatnich iteracji.

Jeśli zespół wcześniej ze sobą nie pracował lub jeśli proces Agile jest dla niego nowością, to będzie zmuszony do skorzystania z prognozy tempa. Techniki prognozowania tempa opisano w rozdziale 16. „Szacowanie tempa”.

Określ cel iteracji

Pamiętając o priorytetach i docelowym tempie, zespół określa cele, jakie chciałby osiągnąć podczas iteracji. Cel zwięźle opisuje to, co zespół chciałby osiągnąć w czasie tego okresu. Dla przykładu zespół produktu SwimStats może wybrać jako cel iteracji „zakończenie implementacji wszystkich funkcjonalności dotyczących danych osobowych”. Oto inne przykłady celów iteracji dla zespołu SwimStats:

- osiągnąć postępy w tworzeniu raportów;
- zakończyć wszystkie raporty z czasów w zawodach;
- zaimplementować mechanizmy zabezpieczeń.

Cel iteracji to zunifikowane oświadczenie opisujące to, co zostanie zrealizowane podczas trwania iteracji. Nie musi być bardzo konkretne. Na przykład dobrym celem iteracji jest „osiągnąć postęp w raportach”. Nie musi być bardziej szczegółowy, jak na przykład „zakończyć 15 raportów” lub „zaimplementować wszystkie raporty z wyników”. Jeśli zdanie „osiągnąć postęp w raportach” jest najlepszym sposobem wyrażenia tego, nad czym będą trwały prace w nadchodzącej iteracji, to jest to dobre wyrażenie celu.

Wybierz historyjki użytkownika

Następnie właściciel produktu i zespół wybierają historyjki, które przyczynią się do osiągnięcia celu iteracji. Jeśli zespół SwimStats wybrał cel iteracji „zakończenie implementacji wszystkich funkcjonalności dotyczących danych osobowych”, to będzie pracować nad wszystkimi historyjkami powiązanymi z danymi osobowymi, które to historyjki jeszcze nie zostały zakończone. Mogą one być następujące:

- Jako pływak mogę aktualizować swoje dane osobowe.
- Jako trener mogę wprowadzać dane osobowe każdego z moich zawodników.
- Jako trener mogę zaimportować plik z wszystkimi danymi osobowymi.
- Jako trener mogę wyeksportować plik z wszystkimi danymi osobowymi.

Podczas wybierania historyjek, nad którymi zespół będzie pracował, właściciel produktu wraz z zespołem biorą pod uwagę priorytety każdej historyjki. Na przykład jeżeli eksport pliku z danymi osobowymi znajduje się w dolnej części listy uporządkowanej według priorytetów wymagań dla produktu, to ta historyjka może nie zostać uwzględniona w iteracji. W tym przypadku lepszym celem iteracji może być: „zaimplementować najważniejsze funkcjonalności dotyczące danych osobowych”.

Podziel historyjki użytkownika na zadania

Po wybraniu odpowiedniego zbioru historyjek użytkownika należy zdekomponować każdą historyjkę na zbiór zadań koniecznych do dostarczenia nowej funkcjonalności. Załóżmy, że historyjka użytkownika o najwyższym priorytecie brzmi: „Jako trener mogę wyznaczyć pływaków do udziału w najbliższych zawodach”. Tę historyjkę użytkownika można przekształcić na listę zadań w następującej postaci:

- Ustal zasady decydujące o tym, kto może być przypisany do jakich zawodów.
- Napisz przypadki testów akceptacyjnych, które pokazują, jak to powinno działać.

- Zaprojektuj interfejs użytkownika.
- Uzyskaj od trenerów opinie dotyczące interfejsu użytkownika.
- Zakoduj interfejs użytkownika.
- Zakoduj warstwę pośrednią.
- Dodaj nowe tabele do bazy danych.
- Zautomatyzuj testy akceptacyjne.

Popularnym pytaniem związanym z planowaniem iteracji jest „co powinno być w niej uwzględnione?”. Należy zidentyfikować wszystkie zadania niezbędne do przejścia od historyjki użytkownika do ukończonego produktu. Jeśli konieczne jest zrealizowanie takich zadań jak analiza, projekt, projekt interakcji z użytkownikiem lub innych zadań, to należy je zidentyfikować i oszacować. Ponieważ celem każdej iteracji jest utworzenie produktu, który potencjalnie nadaje się do publikacji, to należy pamiętać o uwzględnieniu zadań dotyczących testowania i dokumentowania produktu. Ważne jest dołączenie zadań testowych, ponieważ zespół powinien myśleć o sposobie testowania historyjki użytkownika od samego początku iteracji. Takie podejście pomaga zaangażować testerów od początku iteracji, a to wpływa na poprawę zachowań interdyscyplinarnych w zespole.

Uwzględniaj tylko te prace, które dodają wartość do projektu

W planie iteracji należy wskazać tylko te zadania, które dodają bezpośrednią wartość do bieżącego projektu. Oczywiście to obejmuje zadania, które można sklasyfikować jako analizę, projektowanie, kodowanie, testowanie, projekt interfejsu użytkownika i tak dalej. Nie należy uwzględniać godziny na początku dnia, kiedy zajmujemy się odpisywaniem na e-maile. Co prawda niektóre z tych e-maili są związane z projektem, ale zadania w rodzaju „odpowiadanie na e-maile — 1 godzina” nie powinny być uwzględniane w planie iteracji.

Na podobnej zasadzie założmy, że musisz się spotkać się z szefem kadr firmy, aby porozmawiać o nowym procesie rocznego opiniowania. Takich spotkań nie należy uwzględniać w planie iteracji. Chociaż opiniowanie zgodne z nową procedurą będzie dotyczyło członków zespołu projektowego, to spotkanie poświęcone omówieniu tego procesu (oraz wszystkich wynikających z niego prac) nie jest bezpośrednio związane z rozwojem produktu. W związku z tym żadne zadania związane z tym przedsięwzięciem nie powinny się znaleźć w planie iteracji.

Bądź konkretny tak długo, aż pewne działania wejdą w nawyk

Nowe zespoły Agile często nie wiedzą, jak pisać testy jednostkowe, lub nie mają odpowiednich umiejętności do pisania testów automatycznych. Jednak to właśnie tę umiejętność starają się pielęgnować w ciągu pierwszych kilku iteracji. Zachęcam, aby w tym okresie programiści jawnie identyfikowali i szacowali zadania związane z pisaniem testów jednostkowych. Na przykład programista może stwierdzić, że zakodowanie nowej funkcjonalności potrwa osiem godzin, natomiast napisanie dla niej testów jednostkowych zajmie pięć godzin. Później, gdy pisanie testów jednostkowych stanie się nawykiem programistów, wystarczy zapisać jedną karteczkę z zadaniem kodowania nowej funkcjonalności, a następnie nadać jej ocenę uwzględniającą czas na automatyzację za pomocą testów jednostkowych. Kiedy takie działania jak pisanie testów jednostkowych staną się nawykiem, mogą zostać włączone do innych zadań. Jednak zanim to się stanie, warto jawnie formułować zadania, aby zwiększyć świadomość realizowanych zadań w zespole.

Spotkania mają znaczenie (olbrzymie)

Należy wskazać, oszacować i uwzględnić zadania związane ze spotkaniami dotyczącymi projektu. Przy ocenie spotkań należy pamiętać o uwzględnieniu czasu dla wszystkich uczestników, a także czasu potrzebnego na przygotowanie się do spotkania. Załóżmy, że zespół zaplanował spotkanie w celu omówienia opinii użytkowników. Wszystkich siedmiu członków zespołu planuje wziąć udział w jednogodzinnym spotkaniu, a analityk planuje poświęcić dwie godziny na przygotowanie się do spotkania. Ocena dla tego zadania wynosi dziewięć godzin. Zwykle wprowadzam takie zadanie do planu iteracji jako pojedyncze dziewięciogodzinne zadanie, a nie jako oddzielne zadanie dla każdego członka zespołu.

Błędy

Celem zespołu Agile powinno być naprawienie wszystkich błędów w tej iteracji, w której te błędy zostały wykryte. Zespół może osiągnąć ten cel w miarę zdobywania doświadczenia w pracy w krótkich iteracjach, w szczególności, jeśli bazuje na automatycznych testach. Jeśli programista dostarcza oceny zadania polegającego na kodowaniu funkcjonalności, to w tej ocenie powinien uwzględnić czas na skorygowanie wszystkich błędów znalezionych w implementacji. Jeśli tego nie robi, to powinien zidentyfikować i oszacować oddzielne zadanie („Poprawianie błędów”). Osobiście polecam zidentyfikować pojedyncze zadanie, ale nie uznawać go za zakończone do czasu, aż będą przechodzić wszystkie testy.

Defekt znaleziony później (lub niepoprawiony podczas iteracji, w której został wykryty) powinien być traktowany w taki sam sposób jak historyjka użytkownika. Poprawienie defektu powinno być potraktowane priorytetowo w kolejnej iteracji — na równi z innymi historyjkami użytkownika. Poza iteracją cała koncepcja defektu się rozmywa. Naprawianie błędów i dodawanie funkcjonalności to dwa sposoby opisywania tego samego.

Obsługa zależności

Często się zdarza, że implementacja jednej historyjki użytkownika zależy od wcześniejszej implementacji innej historyjki. W większości przypadków te zależności nie są istotnym problemem. Zazwyczaj można zaobserwować coś, co nazywam naturalną kolejnością implementacji historyjek użytkownika — tzn. istnieje sekwencja, która ma sens zarówno dla deweloperów, jak i dla właściciela produktu.

Nie ma problemu, gdy między historyjkami istnieją zależności, które prowadzą do opracowywania ich w naturalnej kolejności. Naturalna kolejność to zazwyczaj taka, którą zespół określił podczas szacowania historyjek. Na przykład zespół SwimStats prawdopodobnie założył, że pływacy mogą być dodawani do systemu, zanim będą mogli być z niego usunięci. Kiedy historyjki są implementowane w innej kolejności niż założono podczas szacowania w trakcie planowania iteracji, zespół często musi uwzględniać dodatkowe zadania, które pozwalają na pracę nad historyjkami w nowej kolejności.

Dla przykładu naturalną kolejnością dla serwisu SwimStats byłoby zrealizowanie funkcjonalności, które pozwalają użytkownikowi na dodawanie nowych pływaków do systemu, a potem funkcjonalności pozwalających użytkownikowi na przeglądanie najlepszych czasów każdego z pływaków w poszczególnych zawodach. Trochę nienaturalne byłoby zaimplementowanie funkcjonalności pozwalających na wyświetlanie najlepszych czasów pływaków przed ekranami pozwalającymi na dodawanie pływaków do systemu. Byłoby to jednak możliwe, gdyby właściciel produktu i zespół chcieli rozwijać funkcjonalności w tej kolejności. Aby to zrobić, byłoby oczywiście konieczne zaprojektowanie tyłu elementów baz danych, ile jest niezbędne do przechowywania pływaków i ich czasów. Trzeba by również wprowadzić

do bazy danych dane co najmniej jednego pływaka oraz jego czasów. Ponieważ nie jest to fragment funkcjonalności, który zespół chce zaimplementować w pierwszej kolejności, to dane pływaka (oraz jego czasy) zostałyby wprowadzone do bazy danych bezpośrednio, a nie za pośrednictwem interfejsu użytkownika i stworzonego oprogramowania.

Aby zespół SwimStats mógł to zrealizować, podczas planowania iteracji musi zidentyfikować kilka zadań, które nie zostałyby zidentyfikowane w przypadku, gdyby te dwie historyjki były implementowane zgodnie z ich naturalnym porządkiem. Na przykład gdyby już istniała możliwość dodawania pływaków do systemu, to zespół nie musiałby dodawać zadania „Zaprojektuj tabele bazy danych dotyczące informacji o poszczególnych pływakach”. Jednak zespół musi uwzględnić to zadanie z uwagi na to, że historyjki użytkownika nie są implementowane w naturalnej kolejności.

Czy to oznacza, że praca nad historyjkami użytkownika niezgodnie z naturalną kolejnością spowoduje, że realizacja projektu zajmie więcej czasu? Są dwie odpowiedzi na to pytanie: „prawdopodobnie nie” oraz „to nie ma znaczenia”.

Po pierwsze, projekt prawdopodobnie nie zajmie więcej czasu. Ostatecznie jedynie przesunięto kilka zadań z jednej historyjki użytkownika do innej. Wspomniane w tym przykładzie projektowanie tabel danych pływaków prędzej czy później musiałoby zostać zrealizowane. Kiedy przyjdzie czas na pracę nad historyjką użytkownika dotyczącą dodawania nowych pływaków do systemu, ta historyjka zostanie zaimplementowana szybciej, bo część pracy już została wykonana.

Powodem do zmartwień może być wpływ tego przesunięcia zadań na oceny wymienionych dwóch historyjek. Mogło dojść do przesunięcia punktu lub idealnego dnia pracy z jednej historyjki do innej. W większości przypadków nie ma to wielkiego znaczenia, a podczas trwania projektu różnice się rozmyją. Zaobserwowałem, że jest to trochę pesymistyczne przesunięcie, ponieważ historyjka oceniona na pięć punktów staje się historyjką sześciopunktową. Ale dzięki temu, że zespół po zakończeniu historyjki przyznaje sobie tylko pięć punktów, nieco zaniża swoje tempo. Ponieważ przesunięcie zadań wywiera niewielki wpływ na projekt, a jedynym problemem są nieco pesymistyczne odczucia, to zazwyczaj nie martwię się tym zbytnio. Jeśli jednak tego rodzaju wpływ na projekt ma znaczenie albo jeśli przesunięcie zadań wywiera większy wpływ na projekt, to należy poddać ponownej ocenie historyjki związane z przesunięciami natychmiast po podjęciu decyzji o pracy nad nimi w kolejności innej niż naturalna.

Po drugie, nawet jeśli praca nad historyjkami użytkownika w tej kolejności powoduje, że projekt zajmuje więcej czasu, to nie ma to znaczenia, ponieważ prawdopodobnie istniał uzasadniony powód, aby pracować nad historyjkami użytkownika w zmienionej kolejności. Zespół może chcieć pracować nad historyjkami użytkownika w określonej kolejności tak, aby móc wcześniej zapobiec technicznym zagrożeniom. Właściciel produktu może także dążyć do wcześniejszego uzyskania opinii użytkowników o historyjce, która bardziej naturalnie zostałaby zrealizowana później. Dzięki implementacji historyjek użytkownika poza naturalnym porządkiem zespół ma możliwość wcześniejszego uzyskania opinii użytkowników i potencjalnie zaoszczędzenia miesiąca lub dwóch na przeróbki, które trzeba by było wprowadzić pod koniec projektu (gdy istnieje mniejsza szansa na to, aby można było łatwo uwzględnić taką zmianę w harmonogramie).

Prace, które trudno rozdzielić

Niektóre funkcjonalności są szczególnie trudne do podzielenia na zadania. Niedawno uczestniczyłem na przykład w spotkaniu planistycznym poświęconym omówieniu niewielkiej zmiany w starszej funkcjonalności. Nikomu spośród uczestników spotkania nie było łatwo przewidzieć wszystkie

możliwe konsekwencje tej zmiany. Mieliliśmy pewność, że będzie ona miała wpływ na niektóre fragmenty kodu, ale nie wiedzieliśmy, czy nie wpłynie na inne. Zmiany we fragmentach kodu, co do których mieliśmy pewność, że potrzebne są zmiany, były niewielkie. Ocenialiśmy je łącznie na cztery godziny. Gdyby trzeba było zmienić także inne fragmenty kodu, to ocena byłaby znacznie wyższa — prawdopodobnie mogłaby sięgnąć dwudziestu godzin. Nie mogliśmy mieć pewności, nie patrząc na kod, a nie chcieliśmy w tym celu wstrzymywać spotkania. Zamiast tego zapisaliśmy następujące dwa zadania:

- Określenie fragmentów, na które wpłynie zmiana — dwie godziny.
- Wprowadzenie zmian — dziesięć godzin.

Pierwsze zadanie to tzw. *spike*. Zadanie typu *spike* dołącza się do opracowywanego planu iteracji wyłącznie po to, aby zyskać wiedzę albo udzielić odpowiedzi na pytanie. W tym przypadku zespół nie potrafił wiarygodnie oszacować pewnych działań, dlatego utworzył dwa zadania: jedno zadanie typu *spike* oraz drugie — wypełniacz z prawdopodobnym czasem trwania. Realizacja zadania typu *spike* pomaga zespołowi dowiedzieć się, w jaki sposób podejść do innego zadania. Zdobycie tej wiedzy może przyczynić się do jego właściwego oszacowania.

Oszacuj zadania

Kolejnym krokiem w planowaniu iteracji zarządzanym tempem jest oszacowanie każdego zadania. Niektóre zespoły wolą oceniać zadania po zidentyfikowaniu wszystkich, inne wolą szacować zadanie po zadaniu osobno zaraz po zidentyfikowaniu każdego z nich. Oceny zadań są wyrażane w dniach idealnych. Jeśli uważam, że zadanie zajmie mi sześć godzin roboczych, to daję mu ocenę sześć godzin. Robię to nawet wtedy, gdy sześć godzin pracy nad zadaniem wypełni mi cały ośmiogodzinny dzień roboczy.

Chociaż zgadzam się z ogólnie przyjętym zaleceniem, że najlepszych ocen dostarczają te osoby, które będą pracowały nad zadaniem (Lederer i Prasad 1992), to uważam, że szacowanie zadań w projekcie Agile powinno być pracą zespołową. Istnieją ku temu cztery powody.

Po pierwsze, ze względu na to, że podczas planowania iteracji zadania nie są przypisywane konkretnym osobom, nie ma możliwości zadania pytania konkretnej osobie, która będzie wykonywać pracę.

Po drugie, chociaż spodziewamy się, że zadanie będzie realizowała konkretna osoba i choć ta osoba może wiedzieć najwięcej na temat tego zadania, nie oznacza to, że inni nie mają nic do dodania. Załóżmy, że w trakcie spotkania planowania iteracji Jan powiedział: „Zaprogramowanie tej funkcjonalności zajmie mi około dwóch godzin — to jest trywialne!”. Jednak pamiętamy, że w zeszłym miesiącu Jan pracował nad podobnym problemem i wyraził podobną opinię, a pomimo tego realizacja zadania zajęła mu około szesnastu godzin. Tym razem, kiedy Jan powie, że implementacja podobnego zadania potrwa tylko dwie godziny, możesz powiedzieć: „Janie, ostatnim razem, gdy pracowałeś nad podobnym zadaniem, też mówiłeś, że to będą dwie godziny, a zajęło ci to szesnaście godzin”. Najprawdopodobniej Jan odpowie na to, podając sensowne wytłumaczenie, dlaczego ten przypadek jest inny, albo zgodzi się, że przy tego typu zadaniach powstają pewne trudności lub potrzebne są dodatkowe prace, o których ciągle zapominamy.

Po trzecie, jeśli członkowie zespołu usłyszą, ile czasu ma zająć realizacja jakiegoś zadania, to często łatwiej im zidentyfikować nieporozumienia dotyczące historyjek użytkowników lub zadań. Słyszac nieoczekiwanie wysokie oszacowanie, właściciel produktu lub analityk może odkryć, że zespół zmierza

w kierunku bardziej szczegółowego rozwiązania niż to konieczne. Ponieważ oszacowania są omawiane wśród członków zespołu, można skorygować oceny, zanim zostaną podjęte niepotrzebne wysiłki.

Wreszcie, jeśli ocenę dostarczy osoba, która będzie wykonywać pracę, to z powodu dumy i ego może jej być trudno później przyznać się do błędu. Gdy oceny dokonują wspólnie członkowie zespołu, niechęć do przyznania się, że ocena była niewłaściwa, znika.

Niektóre projekty są dobre

Podczas tworzenia listy zadań i dokonywania ocen dyskusja w pewnym zakresie jest oczywiście konieczna. Nie możemy stworzyć listy zadań, jeśli nie mamy pojęcia o tym, w jaki sposób zamierzamy wykonać tę pracę. Na szczęście jednak podczas planowania iteracji nie trzeba zbyt daleko wchodzić w szczegóły dotyczące projektu funkcjonalności.

Właściciel produktu, analitycy i projektanci interfejsu użytkownika mogą omawiać projekt produktu, jaką część funkcjonalności trzeba zaimplementować oraz jak to będzie wyglądać z punktu widzenia użytkowników. Deweloperzy mogą omawiać warianty implementacji potrzebnych funkcjonalności. Oba rodzaje dyskusji na temat projektu są potrzebne i właściwe. Nigdy jednak nie byłem na spotkaniu planowania iteracji, na którym trzeba by było narysować diagram klas lub podobny model. Pragnienie, aby to zrobić, jest chyba najlepszym znakiem ostrzegawczym, że podczas planowania iteracji zbyt mocno zajęto się kwestiami dotyczącymi projektu. Takie dyskusje powinny być prowadzone poza spotkaniami planowania iteracji.

Narysowanie projektu to zbyt wiele szczegółów. Na tym etapie wystarczy oszacowanie prac, które będą niezbędne do zrealizowania funkcjonalności. Jeśli rozpoczniesz iterację i odkryjesz, że zidentyfikowane zadania są nieprawidłowe, usuń początkowe zadania i utwórz nowe. Jeśli ocena jest nieprawidłowa, skreśl ją i zapisz nową wartość. Zapisywanie zadań i ocen na karteczkach z notesu to doskonałe podejście, ponieważ każda karteczka niesie w sobie subtelne przypomnienie o nietrwałości.

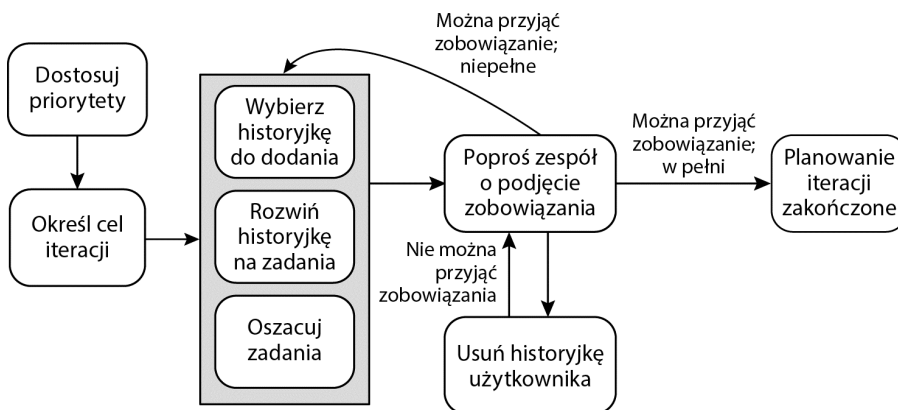
Odpowiedni rozmiar dla zadań

Tworzone zadania powinny mieć taki rozmiar, aby ich zrealizowanie zajęło deweloperowi przeciętnie jeden dzień pracy. Taki rozmiar zapewnia sprawny przepływ pracy przez zwinny proces wytwarzania oprogramowania. Większe zadania zazwyczaj absorbują jednego lub dwóch deweloperów, a pozostali członkowie zespołu muszą w tym czasie czekać. Ponadto jeśli w zespole są prowadzone krótkie codzienne spotkania (Schwaber i Beedle 2002; Rising 2002), to wyznaczanie zadań o właściwym rozmiarze pozwala deweloperowi przez większość dni na zgłaszanie zakończenia co najmniej jednego zadania.

Oczywiście często pojawiają się zadania wymagające kilku dni. Jednak większe zadania, ogólnie rzecz biorąc, powinny być interpretowane jako wypełniacze dla jednego dodatkowego zadania bądź większej liczby dodatkowych zadań, które zostaną dodane natychmiast, gdy zespół je sobie uświadomi. Jeśli podczas spotkania planowania iteracji musisz stworzyć zadanie zajmujące szesnaście godzin, zrób to. Jednak gdy tylko zrozumiesz zadanie lepiej, zaktualizuj je lub zastąp. Może to oznaczać zastąpienie oryginalnej karty zadania kartą z mniejszą lub większą wartością niż pierwotnie oszacowane szesnaście godzin.

Planowanie iteracji sterowane zobowiązaniami

Innym podejściem do planowania iteracji jest metoda planowania sterowanego zobowiązaniami. Planowanie iteracji sterowane zobowiązaniami zawiera wiele takich samych kroków jak planowanie iteracji sterowane tempem. Jednak zamiast tworzenia planu iteracji z wykorzystaniem koncepcji wczorajszej pogody w celu ustalenia liczby punktów lub dni idealnych, które należy zaplanować w bieżącej iteracji, zespół dodaje do iteracji historyjki jedna po drugiej tak długo, aż dojdzie do stanu, w którym nie może zobowiązać się do realizacji kolejnych. Ogólną koncepcję podejścia planowania iteracji sterowanego zobowiązaniami zaprezentowano na rysunku 14.3.



RYСУNEK 14.3. Aktywności podczas planowania iteracji sterowanego zobowiązaniami

Pierwsze kroki — dostosowanie priorytetów i określenie celu iteracji — są takie same jak w przypadku podejścia planowania sterowanego tempem. Następny krok, wybór historyjki, która ma być dodana do iteracji, jest inny. Właściciel produktu wraz z zespołem nadal wybierają historyjkę o najwyższym priorytecie, która wspiera cel iteracji. Jednak w przypadku planowania sterowanego zobowiązaniami historyjki są wybierane i dekomponowane na zadania, a szacowanie zadań dotyczy jednorazowo tylko jednej historyjki. W tym elemencie podejście to różni się od planowania sterowanego tempem, gdzie wybierano zbiór historyjek, dla których suma ocen była równa szacowanemu tempu.

Historyjki są wybierane po jednej, ponieważ po podzieleniu każdej historyjki na zadania zespół podejmuje decyzję, czy może się zobowiązać do dostarczenia tej historyjki podczas trwania iteracji.

Poproś zespół o podjęcie zobowiązania

Larson i LaFasto (1989) w swojej pracy poświęconej czynnikom, które wpływają na sukces zespołu, ustalili, że jednym z kluczowych warunków sukcesu zespołu jest podjęte wspólnie — przez wszystkich członków zespołu — zobowiązanie. W trakcie spotkania poświęconego planowaniu iteracji zadają zespołowi pytanie „Czy możecie się zobowiązać do dostarczenia funkcjonalności, które omawialiśmy?”. Zwróćmy uwagę, że nie zadają pytania „Czy możecie się zobowiązać do realizacji zadań, które zidentyfikowaliśmy?”. To zupełnie inne pytanie i o wiele słabsze zobowiązanie, ponieważ dotyczy wykonania zestawu zadań, a nie dostarczenia nowej funkcjonalności.

Jeśli podczas trwania iteracji zostaną wykryte nowe zadania (a prawie na pewno zostaną wykryte), to zespół, który podjął się dostarczenia funkcjonalności opisanej przez historijkę użytkownika, będzie również dążył do zrealizowania tych nowych zadań. Zespół, który zobowiązał się do wykonania zadań z ustalonej listy, nie będzie dążył do wykonania zadań spoza listy. W każdym przypadku istnieje możliwość, że nowo odkryte zadania zajmą tyle czasu, że nie będą mogły zostać zakończone podczas jednej iteracji. Wówczas zespół będzie zmuszony do omówienia sytuacji z właścicielem produktu i sprawdzenia, czy nadal istnieje sposób osiągnięcia celu iteracji. Może wystąpić potrzeba zmniejszenia zakresu funkcjonalności historijki albo jej całkowitego usunięcia.

Osobiście pytam zespoły, czy mogą podjąć zobowiązanie każdorazowo po podzieleniu historijki użytkownika na zadania i dokonaniu ocen zadań. W przypadku pierwszej historijki użytkownika to pytanie często wydaje się głupie. W zespole może być siedem osób, które planują pracę podczas dwutygodniowej iteracji. Być może dotychczas zidentyfikowały zaledwie trzydzieści cztery godziny pracy, a ja pytam je, czy mogą podjąć takie zobowiązanie. Ich odpowiedź (werbalna albo przez wyrażenie zakłopotania na twarzach) brzmi: „Oczywiście, że możemy to zrobić. Jest nas siedem. Mamy dwa tygodnie, a to są tylko trzydzieści cztery godziny pracy”. Jednak w miarę trwania spotkania oraz dodawania do iteracji nowych historijek użytkownika odpowiedź na pytanie „Czy możecie przyjąć zobowiązanie?” zaczyna wymagać namysłu. W końcu docieramy do punktu, w którym zespół nie może przyjąć dalszych zobowiązań. Jeśli nie może podjąć zobowiązania, to może zdecydować o usunięciu z iteracji jednej z historijek i zastąpieniu jej mniejszą.

Sumowanie ocen

Według moich doświadczeń najlepszym sposobem, aby zespół mógł stwierdzić, czy jest w stanie zobowiązać się do zrealizowania zbioru historijek użytkownika, jest zsumowanie ocen przypisanych do zadań i sprawdzenie, czy suma reprezentuje sensowną ilość pracy. Niektóre zadania mogą wiązać się z dużą dozą niepewności ze względu na to, że prace nie zostały zaprojektowane, a wymagania są niejasne. Jednak zsumowanie ocen daje pewien obraz całkowitego rozmiaru pracy.

Przypuśćmy, że zespół złożony z siedmiu osób pracuje w dwutygodniowych iteracjach. W każdej iteracji ma do dyspozycji 560 godzin ($7 \text{ osób} \times 10 \text{ dni} \times 8 \text{ godzin dziennie}$). Wiemy, że pewna ilość czasu zostanie poświęcona na czynności, które nie są zapisane na kartach zadań — odpowiadanie na e-maile, uczestnictwo w spotkaniach i tak dalej. Niektóre oceny mogą być nieprawidłowe. Są to w końcu *oceny*, a nie gwarancje. Z tych powodów nie możemy oczekiwać, że zespół zobowiąże się do wykonania zadań ocenionych na 560 godzin. W istocie większość zespołów osiąga sukces w sytuacji, gdy zaplanowane przez nie prace (suma ocen na kartach zadań) wynoszą od czterech do sześciu godzin dziennie. W przypadku zespołu złożonego z siedmiu osób praca w iteracjach trwających dwa tygodnie oznacza możliwość zaplanowania od 280 do 420 godzin. To, czy określony zespół zdoła zmieścić się w tym zakresie, zależy od tego, jak dobrze zostały zidentyfikowane zadania dla konkretnej historijki użytkownika, jak dokładnie oszacowano te zadania, jaka jest objętość zewnętrznych zobowiązań podjętych przez członków zespołu oraz jakie jest ogólne obciążenie zespołu zadaniami w firmie. Po zaledwie kilku iteracjach większość zespołów potrafi właściwie oszacować, ile w przybliżeniu godzin powinny zaplanować w jednej iteracji.

Przed podjęciem zobowiązania do wykonania prac w czasie iteracji zespół musi przeanalizować zadania i ustalić, czy rozkład prac do wykonania odpowiada różnym umiejętnościom członków zespołu. Czy jest możliwe, że podczas iteracji programista Javy będzie przeciążony, w czasie kiedy programista HTML nie będzie miał nic do roboty? Czy wybrane historijki użytkownika są łatwe do zaprogramowania, ale czasochłonne, albo trudne do przetestowania, przez co nadmiernie obciążają

testera? Czy każda z wybranych historyjek użytkownika przed rozpoczęciem kodowania wymaga przeprowadzenia analizy i wykonania projektu interakcji z użytkownikiem?

Zespół, który znajdzie się w takiej sytuacji, najpierw musi podjąć próbę poszukania lepszych sposobów na dzielenie pracy. Być może programista HTML w tym przykładzie może pomóc testerowi? Czy pracę może wykonać ktoś inny niż projektant interakcji z użytkownikiem? Jeśli nie, to czy możemy usunąć z tej iteracji jakieś historyjki, które wymagają projektu interakcji z użytkownikiem, a w zamian zająć się takimi historyjkami, które tego nie wymagają? Najważniejsze, aby wszyscy członkowie zespołu czuli się odpowiedzialni za wykonywaną pracę zgodną z własnymi możliwościami, niezależnie od specjalności.

Zobowiązania indywidualne

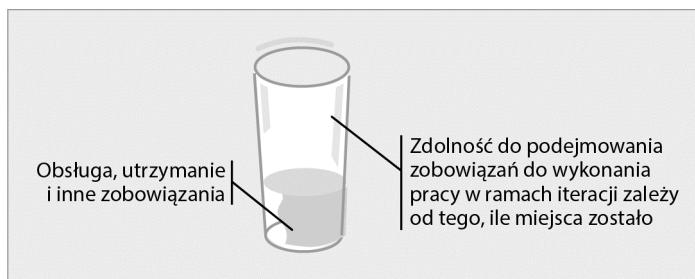
Podczas szacowania zdolności do przyjęcia zobowiązania do zaimplementowania zestawu nowych funkcjonalności niektóre zespoły przydzielają poszczególne zadania konkretnym osobom, a następnie starają się ocenić, czy każda z osób jest w stanie zobowiązać się do wykonania takiej ilości pracy. Takie podejście dobrze się sprawdza. W przeszłości zalecałem jego stosowanie (Cohn 2004). Doszedłem jednak do wniosku, że dzięki rezygnacji z przydzielania zadań podczas planowania iteracji oraz rezygnacji z arytmetyki niezbędnej do podejmowania indywidualnych zobowiązań zespół osiąga korzyść ze względu na ukształtowanie myślenia w rodzaju „jesteśmy w to zaangażowani wszyscy razem”.

Jeśli podczas planowania iteracji okaże się, że przydział zadań do poszczególnych osób jest niezbędny, to przydział ten należy traktować jako tymczasowy oraz przewidzieć możliwość zmian w trakcie trwania iteracji.

Utrzymanie a zobowiązania

Oprócz osiągania postępów w projekcie wiele zespołów jest odpowiedzialnych za udzielanie wsparcia i utrzymywanie innych systemów. Może to dotyczyć poprzedniej wersji produktu albo systemu niezwiązanego z bieżącym projektem. Jeśli zespół podejmuje zobowiązania zrealizowania zbioru historyjek użytkownika podczas iteracji, to powinien pamiętać o obciążeniach związanych z zadaniami dotyczącymi wsparcia i utrzymania. Nie mam na myśli zadań związanych z poprawianiem ogólnych błędów, które zawsze powinny być traktowane priorytetowo. Zadania związane z poprawianiem błędów powinny przejść przez ogólny proces ustalania priorytetów w iteracji. Przez zadania dotyczące utrzymania i wsparcia rozumiem nieprzewidywalne, ale niezbędne elementy życia wielu zespołów — wsparcie dla produkcyjnej witryny internetowej lub bazy danych, odbieranie zgłoszeń dotyczących wsparcia dla kluczowych klientów, zadania pierwszej warstwy pomocy technicznej i tak dalej.

Iterację porównuję do pustej szklanki. W pierwszej kolejności do tej szklanki nalewam niezmiennie zobowiązania zespołu — na przykład zadania związane ze wsparciem i utrzymaniem innych produktów. Miejsce, które pozostanie w szklance, jest dostępne dla zespołu podczas podejmowania zobowiązań do prac realizowanych w czasie iteracji. Tę koncepcję zaprezentowano na rysunku 14.4. Jest oczywiste, że zespół, którego szklanka jest wypełniona w 10% zadaniami wsparcia, będzie miał więcej czasu, by podjąć zobowiązania do prac w ramach iteracji niż zespół, którego szklanka jest od początku wypełniona w 90%.



RYСУNEK 14.4. Inne zobowiązania decydują o tym, ile zadań zespół może podjąć podczas iteracji

W większości sytuacji zespół nie jest w stanie dokładnie przewidzieć obciążeń związanych ze wsparciem w nadchodzącej iteracji. Członkowie zespołu powinni znać długoterminową średnią, ale trzeba pamiętać, że średnio dwadzieścia godzin wsparcia tygodniowo to nie to samo co dwadzieścia godzin każdego tygodnia. Jeśli obciążenia związane ze wsparciem i utrzymaniem przekraczają oczekiwania podczas iteracji, to zespół może nie być w stanie wykonać swoich zobowiązań. Należy temu przeciwdziałać, podejmując próby przekraczania swoich zobowiązań w tych iteracjach, w których obciążenia związane ze wsparciem i utrzymaniem są mniejsze od oczekiwanych. Taka zmienność jest nieunikniona w przypadku zespołów, które podejmują znaczące prace związane ze wsparciem i utrzymaniem.

Moja rekomendacja

Zarówno planowanie iteracji sterowane tempem, jak i zobowiązaniami to podejścia godne uwagi, jednak osobiście preferuję metodę planowania sterowanego zobowiązaniami. Choć tempo odgrywa kluczową rolę w planowaniu zadań, to nie uważam, że powinno odgrywać taką samą rolę w planowaniu iteracji. Są ku temu dwa powody.

Po pierwsze, ze względu na to, że tempo jest miarą niedokładnych ocen (punktów lub dni idealnych), nie jest wystarczająco dokładne do planowania pracy w krótkich iteracjach. Możemy wykorzystywać te zgrubne oceny do oszacowania ogólnego zakresu prac zespołu podczas iteracji. Nie możemy jednak używać ich w taki sam sposób do planowania krótkoterminowej pracy obejmującej pojedynczą iterację.

Po drugie, zespół musiałby zrealizować podczas iteracji od dwudziestu do trzydziestu historyjek użytkownika z powodu błędów w ocenach za pomocą punktów lub dni idealnych. Niewiele zespołów jest w stanie zrealizować w iteracji aż tyle historyjek.

Aby zaobserwować efekt tych problemów, założmy, że w każdej z ostatnich pięciu iteracji zespół pracował z tempem trzydziści. Jest to dość spójna ocena i zachodzi prawdopodobieństwo, że w najbliższej iteracji zespołowi ponownie uda się zdobyć trzydzieści punktów. Wiemy jednak, że nie wszystkie pięciopunktowe historyjki są takie same. Po posortowaniu dużego zbioru pięciopunktowych historyjek mogłoby się okazać, że można zidentyfikować kilka pięciopunktowych historyjek, które wyglądają na nieco łatwiejsze niż reszta historyjek ocenionych na pięć punktów. Być może mylimy się co do niektórych, ale gdybyśmy próbowali tego sposobu pierwszy raz, to prawdopodobnie udałoby się osiągnąć sukces. Moglibyśmy zwiększyć tempo zespołu z trzydziestu do czterdziestu. Z drugiej strony moglibyśmy wybrać tylko takie historyjki ocenione na pięć punktów, które wydają się nieco

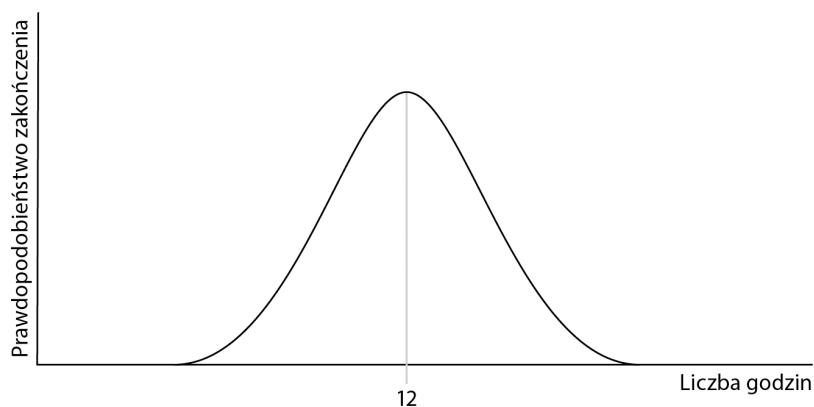
trudniejsze. Nadal uważamy, że powinny być ocenione na pięć punktów, ale są nieco trudniejsze od pozostałych pięciopunktowych historyjek.

W czasie trwania projektu nie mamy zamiaru „przekopywać się” przez kolekcję historyjek użytkownika, próbując znaleźć „łatwiejsze piątki” i „trudniejsze piątki”. Jednak większość zespołów planuje w każdej iteracji implementację od trzech do kilkunastu historyjek. Podczas wybierania tych kilku historyjek użytkownika do iteracji zespołowi z pewnością czasami dopisze szczęście i wybierze same łatwe historyjki, a innym razem będzie miał mniej szczęścia i wybierze same trudniejsze.

Ze względu na to, że w pojedynczej iteracji realizowanych jest zbyt mało historyjek, aby można było obliczać średnią, wolę nie używać tempa do planowania iteracji. Ponieważ jednak te różnice są uśredniane podczas pracy nad wydaniem, to korzystanie z tempa bardzo dobrze się sprawdza w planowaniu iteracji.

Związek ocen zadań z punktacją historyjek

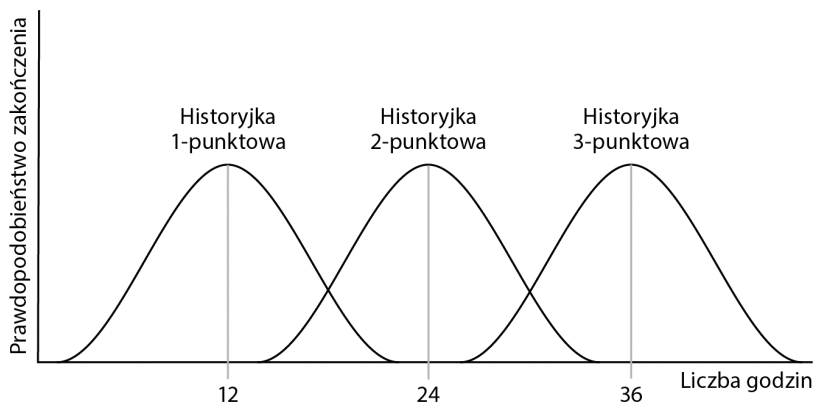
Moi klienci często proszą mnie o wyjaśnienie związku między ocenami zadań stosowanymi podczas planowania iteracji a punktami historyjek lub dniami idealnymi wykorzystywanymi do bardziej długoterminowego planowania wydania. Często zespoły popełniają błąd, sądząc, że istnieje silny związek między punktami historyjek a dokładną liczbą godzin. Na przykład ostatnio pomagałem zespołowi, który śledził rzeczywistą liczbę produktywnych godzin na iterację oraz tempo w każdej iteracji. Na tej podstawie wyliczył, że każdy punkt przypisany historyjce użytkownika odpowiada około dwunastu godzinom pracy. Jego pogląd przekształcił się w błędne przekonanie, że każdy punkt przypisany historyjce zawsze będzie równy dwunastu godzinom pracy. Rzeczywistość lepiej opisuje jednak wykres zaprezentowany na rysunku 14.5.



RYSUNEK 14.5. Rozkład czasu potrzebnego do realizacji historyjki użytkownika ocenionej na jeden punkt

Na tym wykresie widać, że realizacja historyjki ocenionej na jeden punkt średnio zajmuje dwanaście godzin. Widać jednak także to, że realizacja niektórych jednopunktowych historyjek zajmuje mniej, a innych więcej. Zanim zespół nie oszacuje zadań wchodzących w skład historyjki, trudno stwierdzić, w jakim punkcie krzywej takiej jak przedstawiona na rysunku znajduje się określona historyjka użytkownika.

Na rysunku 14.5 pokazano rozkład godzin dla realizacji historyjki ocenionej na jeden punkt, natomiast na rysunku 14.6 przedstawiono hipotetyczny rozkład dla historyjek jedno-, dwu- oraz trzypunktowych. Na tym wykresie każdy punkt nadal odpowiada przeciętnie dwunastu godzinom. Istnieje jednak możliwość, że niektóre historyjki ocenione na jeden punkt zajmą więcej czasu niż niektóre historyjki ocenione na dwa punkty.



RYСУNEK 14.6. Rozkład czasu realizacji jedno-, dwu- i trzypunktowych historyjek użytkownika

Fakt, że implementacja niektórych dwupunktowych historyjek zajmuje mniej czasu niż implementacja niektórych jednopunktowych, jest całkowicie zrozumiały. Nie ma z tym problemu pod warunkiem, że wydanie obejmuje wystarczająco dużo historyjek do tego, aby czas realizacji tych odstających historyjek został uśredniony, oraz pod warunkiem, że wszyscy w projekcie pamiętają, iż niektóre historyjki zajmują więcej czasu niż inne pomimo tego, że ich początkowe, zgrubne oceny były takie same.

Dni tygodnia

Kiedy zacząłem zarządzać projektami Agile, moje zespoły zazwyczaj zaczynały iteracje w poniedziałek, a kończyły w piątek. Było tak niezależnie od tego, czy określony zespół pracował w iteracjach dwu-, trzy- czy też czterotygodniowych. Poniedziałki wydawały się naturalnym dniem na rozpoczęcie iteracji, a piątki były oczywistym dniem na ich zakończenie. Zmieniłem zdanie na ten temat, kiedy zacząłem trenować zespół zajmujący się tworzeniem witryny internetowej, która była intensywnie wykorzystywana w ciągu tygodnia i prawie nie była wykorzystywana podczas weekendu.

Najbardziej logiczną porą do tego, by zespół mógł wdrożyć nowe aktualizacje, był piątek wieczorem. Jeśli coś poszło nie tak, to przez weekend można było naprawić błędy, a konsekwencje były minimalne, ponieważ w tym czasie witryna nie była zbyt intensywnie używana. Aby to uwzględnić, ten zespół zdecydował się na pracę w dwutygodniowych iteracjach, które rozpoczynały się w piątek, a kończyły we czwartek. Takie terminy rozpoczęcia i zakończenia iteracji doskonale się sprawdzały. Piątki były wykorzystywane na przegląd iteracji oraz planowanie następnej iteracji. Trwało to do wczesnego popołudnia w większość piątków. Następnie zespół albo rozpoczynał

pracę nad nową iteracją, albo — od czasu do czasu — wychodził na drinka lub kręgle. Iteracja rozpoczynała się „na serio” w następny poniedziałek. To było fajne, ponieważ nie trzeba było się martwić o poniedziałki wypełnione spotkaniami, tak jak w przypadku gdy poniedziałek był dniem przeglądów i planowania.

Dodatkowo zespół zyskiwał dzięki możliwości wykorzystania — od czasu do czasu — piątkowych poranków na wykonanie prac „na ostatnią minutę”, których nie udało się zakończyć we czwartek. Nie stało się to rutyną i zdarzyło się tylko kilka razy, jednak poświęcenie kilku godzin w piątkowy poranek na dokończenie zadań było lepsze niż pozostawianie ich na weekend (tak jak by było, gdyby iteracja rozpoczynała się w poniedziałek).

Podsumowanie

W przeciwieństwie do planu wydania plan iteracji dotyczy szczegółów pracy podczas jednej iteracji. Zamiast perspektywy trzech do dziewięciu miesięcy, typowej dla planu wydania, plan iteracji nie wychodzi poza pojedynczą iterację. Dość duże historyjki użytkownika planu wydania są dekomponowane na zadania włączane do planu iteracji. Każde zadanie jest oceniane za pomocą liczby idealnych godzin potrzebnych do wykonania zadania.

Istnieją dwa ogólne podejścia do planowania iteracji: zarządzane tempem oraz zobowiązaniami. Te dwa podejścia mają wiele wspólnych kroków i często prowadzą do stworzenia identycznego planu iteracji.

Pytania do dyskusji

1. Jakie podejście do planowania iteracji preferujesz: zarządzane tempem czy zobowiązaniami? Dlaczego?
2. Co sądzisz o rezygnacji z przydzielania zadań konkretnym członkom zespołu podczas planowania iteracji?

ROZDZIAŁ 15.

Wybór długości iteracji

*Wszystko jest mgliste do poziomu, którego nie jesteś nawet świadom,
dopóki nie spróbujesz tego uściślić.*

— Bertrand Russell

W większości metodyk i procesów Agile czas trwania iteracji określa się na okres od dwóch do czterech tygodni. Taki też okres stosuje większość zespołów Agile. Niektóre zespoły stosują dłuższe iteracje, ale czas od dwóch do czterech tygodni jest ogólnie przyjętym standardem dla większości zespołów. Nie istnieje jeden magiczny czas trwania iteracji, który byłby odpowiedni dla wszystkich zespołów i w każdych okolicznościach. Odpowiednia długość iteracji dla zespołu w jednym projekcie nie musi być odpowiednią długością czasu iteracji dla tego samego zespołu w innym projekcie.

Czynniki przy wyborze długości iteracji

Na wybór długości trwania iteracji powinny mieć wpływ następujące czynniki:

- łączny czas wydania, nad którym pracujemy;
- stopień niepewności;
- łatwość uzyskania opinii;
- czas, przez jaki priorytety mogą pozostać bez zmian;
- gotowość do pracy bez zewnętrznych opinii;
- koszty iteracji;
- jak szybko pojawia się presja czasu.

Nie istnieje ustalona względna ważność tych czynników. Znaczenie każdego z nich całkowicie zależy od kontekstu projektu.

Całkowita długość wydania

W krótkotrwałych projektach odpowiednie są krótkie iteracje. Długość iteracji w projekcie określa:

- Jak często oprogramowanie może być prezentowane (potencjalnie w postaci gotowej do wydania) użytkownikom i klientom. Oczywiście oprogramowanie może być prezentowane tym grupom osób w trakcie trwania iteracji, ale zazwyczaj programy mają jakość, która pozwala na ich opublikowanie tylko na końcu iteracji.
- Jak często można mierzyć postępy. Istnieje możliwość uzyskania poczucia tempa zespołu w środku iteracji, ale dopiero na jej końcu możemy w pełni zmierzyć, ile pracy wykonano.
- Jak często właściciel produktu i zespół mogą skorygować swój kurs ze względu na dostrajanie priorytetów i planów między iteracjami.

Jeśli zespół pracuje nad wydaniem, którego publikacja ma mieć miejsce zaledwie za trzy miesiące, to ustalenie czasu trwania jednej iteracji na miesiąc spowoduje, że zespół będzie miał tylko dwie okazje do zbierania opinii na temat produktu, mierzenia postępów i korekty kursu (bowiem są to działania wykonywane po zakończeniu iteracji). W większości przypadków to będzie za mało.

Zgodnie z moją ogólną zasadą w każdym projekcie powinny być co najmniej cztery lub pięć tego rodzaju okazji. Oznacza to, że jeśli całkowity czas trwania projektu wynosi cztery lub więcej miesięcy, to warto się zastanowić nad iteracjami trwającymi miesiąc lub cztery tygodnie. Jeśli jednak całkowity czas pracy nad wydaniem jest krótszy, to dla projektu będzie lepiej, jeśli wybierzemy proporcjonalnie krótsze iteracje.

Stopień niepewności

Niepewność występuje w wielu formach. Często nie ma pewności co do tego, czego dokładnie potrzebują klienci lub użytkownicy, jakie jest tempo zespołu oraz jakie są inne techniczne aspekty projektu. Im większa niepewność dowolnego rodzaju, tym krótszy powinien być czas iteracji. Jeżeli istnieje duża niepewność co do prac, jakie powinny zostać wykonane, albo produktu, który ma być stworzony, wówczas krótsze iteracje pozwalają na częstsze mierzenie postępów za pomocą tempa, a także dają więcej okazji do uzyskania opinii od interesariuszy, klientów i użytkowników.

Łatwość uzyskania opinii

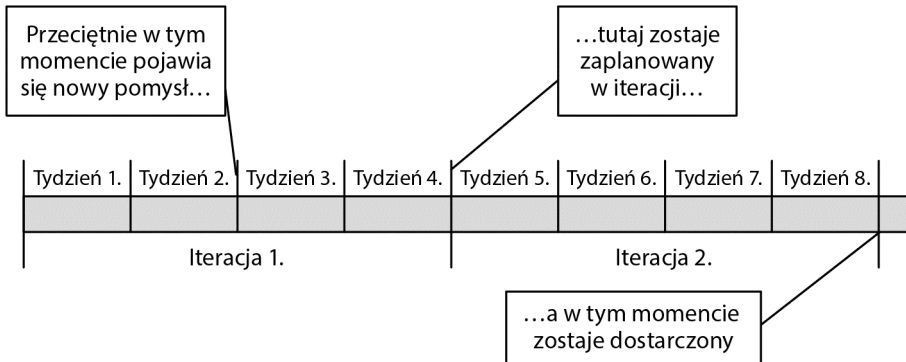
Długość iteracji powinna być dobrana tak, aby zmaksymalizować ilość, częstotliwość i terminowość dostarczania opinii dla całego zespołu. W zależności od środowiska to może oznaczać dłuższe bądź krótsze iteracje. W niektórych firmach można niezwykle łatwo uzyskać nieformalne opinie od wewnętrznych interesariuszy lub użytkowników w czasie trwania iteracji, ale bardzo trudno jest nakłonić te same osoby do udziału w zaplanowanych na koniec iteracji spotkaniach przeglądu iteracji. Inne firmy mają odwrotny problem. Trudno im uzyskać opinie na co dzień, ale interesariusze, użytkownicy i inne osoby chętnie uczestniczą w zaplanowanych formalnych spotkaniach przeglądowych (zwłaszcza jeśli serwowany jest poczęstunek).

Należy dobrać długość iteracji w taki sposób, aby zmaksymalizować wartość opinii, które można uzyskać od osób z wewnątrz oraz z zewnątrz firmy.

Jak długo priorytety mogą pozostać bez zmian?

Kiedy zespół pracujący nad produktem podejmie zobowiązanie do zrealizowania określonego zestawu funkcjonalności w jednej iteracji, trzeba pamiętać, aby nie odwołać go od tego celu. Dlatego jest bardzo ważne, aby właściciel produktu nie zmieniał priorytetów w trakcie trwania iteracji oraz aby pomógł chronić zespół przed osobami, które mogą dążyć do zmian tych priorytetów. Z tego powodu czas, przez jaki priorytety mogą pozostawać bez zmian, powinien być czynnikiem decydującym o wyborze długości iteracji.

Kluczowym czynnikiem jest czas, w jakim dobry, nowy pomysł może zostać przekształcony w działające oprogramowanie. Rozważmy przypadek zespołu, który stosuje czterotygodniowe iteracje. Jeśli założymy, że nowe pomysły mogą z równym prawdopodobieństwem pojawić się w dowolnym momencie trwania iteracji, to można powiedzieć, że nowy pomysł pojawia się przeciętnie w połowie iteracji. Temu nowemu pomysłowi zostanie nadany priorytet w następnej iteracji, która zaczyna się za dwa tygodnie. Miną kolejne cztery tygodnie (pełna iteracja), zanim nowy pomysł zostanie przekształcony w działające oprogramowanie, które potencjalnie jest gotowe do wydania. Tę koncepcję zaprezentowano na rysunku 15.1. Z tego przykładu należy zapamiętać, że czas od pomysłu do działającego oprogramowania wynosi średnio $1\frac{1}{2}$ czasu trwania iteracji stosowanej przez zespół.



RYСУNEK 15.1. Od pomysłu do działającego oprogramowania upływa średnio czas odpowiadający $1\frac{1}{2}$ iteracji

Jak opisano w rozdziale 14. „Planowanie iteracji”, zespół, który równolegle z nowymi pracami rozwojowymi wykonuje zadania związane ze wsparciem i utrzymaniem, musi zarezerwować trochę czasu na realizację tych zadań wsparcia i utrzymania. W istocie na rysunku 15.1 zilustrowano sytuację, w której ktoś podsunął zespołowi pomysł niemieszczący się w rezerwie przewidzianej na zadania wsparcia i utrzymania.

Gotowość do pracy bez opinii z zewnątrz

Nawet przy dobrze zmotywowanym i świetnie komunikującym się zespole może dojść do sytuacji, kiedy wyniki iteracji, po zaprezentowaniu ich szerszemu gronu osób lub użytkownikom z zewnątrz na koniec iteracji, okazują się bezwartościowe. Może się to zdarzyć w przypadku, gdy deweloperzy źle rozumieją właściciela produktu (i nie będą komunikować się z nim zbyt często w czasie trwania iteracji). Może to również nastąpić, jeśli właściciel produktu źle rozumie potrzeby rynku bądź

użytkowników. Strata prawie nigdy nie jest kompletna, pod warunkiem że czegoś się z niej nauczymy. Jednak im rzadziej zespół odbiera opinie z zewnątrz, tym istnieje większe prawdopodobieństwo „zoboczenia z kursu” i tym większe są straty w sytuacji, gdy to się stanie.

Koszty iteracji

Z każdą iteracją są związane koszty. Na przykład w każdej iteracji muszą być uruchomione pełne testy regresji. Jeśli jest to kosztowne (zwykle są to koszty czasowe), to zespół może preferować dłuższe, czterotygodniowe iteracje, aby obniżyć koszty. Oczywiście jednym z celów skutecznego zespołu Agile jest zmniejszenie (lub dążenie do wyeliminowania) kosztów związanych z każdą iteracją. Ale te koszty mogą być znaczne, zwłaszcza podczas pierwszych iteracji zespołu w projekcie, co z pewnością wpływa na decyzję dotyczącą najlepszej długości iteracji.

Jak szybko pojawia się presja czasu?

Niels Malotaux (2004) zauważył, że „tak długo, jak data zakończenia projektu jest odległą przyszłością, nie czujemy żadnej presji i pracujemy w tempie komfortowym. Gdy presja daty zakończenia projektu staje się namacalna, zaczynamy pracować ciężiej”. Nawet w przypadku stosowania iteracji czterotygodniowych data zakończenia projektu nie wydaje się zbyt odległa. Ale to pierwszy tydzień jest najdalej od daty końca iteracji, dlatego wiele zespołów odczuwa znacznie mniej stresu w ciągu tego tygodnia w porównaniu z ostatnim, czwartym tygodniem iteracji.

Rozwiązaniem jest oczywiście wybór długości iteracji, która równoważy presję odczuwaną przez zespół. Nie chodzi o to, aby poddawać zespół większej presji („*Masz to zrobić dziś!*”). Chodzi raczej o uwzględnienie całkowitej ilości stresu, jaką zespół normalnie odczuwa, i jego bardziej równomierne rozłożenie podczas odpowiednio długiej iteracji.

Utrzymanie długości iteracji w celu osiągnięcia stałego rytmu

Niezależnie od wybranego czasu trwania iteracji lepiej jest wybrać określony czas trwania i trzymać się go, niż go często zmieniać. Kiedy zespoły stosują stały, niezmienny czas trwania iteracji, wpadają w naturalny rytm. Gdy zacząłem stosować proces Agile, używając wczesnej wersji metodyki Scrum (Takeuchi i Nonaka 1986; DeGrace i Stahl 1990), moje zespoły wybierały czas trwania każdej iteracji na podstawie oceny nakładów pracy w niej. Po dwutygodniowej iteracji kolejna mogła trwać sześć tygodni, następna cztery tygodnie i tak dalej. W wyniku eksperymentów nad wieloma projektami nauczyłem się, że zespoły uzyskują znacznie lepsze efekty, gdy dostosowują ilość pracy do czasu trwania iteracji (a nie odwrotnie — czas trwania iteracji do ilości pracy).

Regularny rytm iteracji można porównać do bicia serca projektu. Simon Baker, trener metodyki Agile z firmy think-box ltd., opisuje to w następujący sposób: „Tak jak regularne bicie serca utrzymuje ciało przy życiu, tak stały czas trwania iteracji zapewnia stabilizację, która pomaga ustanowić rytm rozwoju (i dostarczania). Według moich doświadczeń rytm jest ważnym czynnikiem, który wspomaga osiągnięcie stałego tempa” (2004).

Podejmowanie decyzji

Jednym z głównych celów przy wyborze długości iteracji jest znalezienie takiego czasu jej trwania, który zachęca wszystkich członków zespołu do pracy podczas iteracji w stałym tempie. Jeśli iteracja trwa zbyt długo, to istnieje naturalna tendencja do tego, aby trochę odpocząć na jej początku, co prowadzi do paniki i większej liczby przepracowanych godzin na jej końcu. Należy dążyć do znalezienia takiego czasu trwania iteracji, który wyrównuje te wahania.

W wyniku eksperymentów z różną długością iteracji moja preferowana długość iteracji wynosi dwa tygodnie. Iteracje tygodniowe (lub krótsze) mogą być bardzo niespokojne i stresujące. Następny termin nie jest nigdy bardziej odległy niż cztery dni. Bardzo krótkie iteracje nie pozostawiają czasu na nadrobienie zaległości w przypadku, gdy któryś z członków zespołu zachoruje lub gdy pójdzie coś nie tak. Jeśli w projekcie nie istnieją jeszcze w pełni automatyczne testy wszystkich części systemu, to niezbyt często polecam rozpoczynanie projektu od tygodniowych iteracji.

Z kolei iteracje trwające cztery tygodnie po doświadczeniach z iteracjami jedno- lub dwutygodniowymi mogą być odczuwane jak wieczność. Przy iteracjach czterotygodniowych zespół często ma czas na poszukiwanie bardziej kreatywnych rozwiązań niż wtedy, gdyby praca odbywała się w krótszych iteracjach. Doświadczony zespół Agile pracujący w badawczej fazie projektu może osiągnąć korzyści ze stosowania iteracji czterotygodniowych. W tak długich iteracjach członkowie zespołu odczuwają bardzo wyraźnie początek, środek i koniec. Nie podoba mi się spowolniona praca zespołu na początku iteracji i szalone tempo na jej końcu.

Uważam, że iteracje dwutygodniowe są idealne. Koszty planowania i testowania są znacznie łatwiejsze do zniesienia, gdy mogą zostać zamortyzowane w ciągu dwóch tygodni. Pierwszy tydzień dwutygodniowej iteracji może być odczuwany inaczej w porównaniu z drugim, ale różnica nie jest tak znaczna jak w przypadku iteracji trwających cztery tygodnie. Ponadto większość organizacji potrafi (przy odpowiednim treningu) nauczyć się dostosowywać priorytety zadań dla iteracji dwutygodniowych, podczas gdy osiągnięcie tego samego w przypadku iteracji trwających cztery tygodnie może być bardzo trudne.

$6 \times 2 + 1$

Ciągła praca w dwutygodniowych iteracjach może powodować napięcia w zespole z powodu stałej presji na dostarczanie produktu oraz ze względu na to, że termin nigdy nie jest dalej niż w przyszłym tygodniu. Moją ulubioną metodą na zmniejszenie tego napięcia jest realizacja makrocyklu sześciu dwutygodniowych iteracji, a następnie jednej iteracji trwającej tydzień. Dla tego cyklu stosuję nazwę „ $6 \times 2 + 1$ ”. Podczas dwutygodniowych iteracji zespół pracuje nad zadaniami, dla których priorytety ustala właściciel produktu.

Z kolei w czasie iteracji trwającej tydzień zespół sam wybiera dla siebie pracę. Nie oznacza to czasu na zabawę lub tygodnia na plaży. Zamiast tego członkowie zespołu wykorzystują ten czas, aby skoncentrować się na tym, co w ich odczuciu jest dla projektu priorytetowe. Programiści mogą wykonywać refaktoryzację, która w ich odczuciu była zbyt ryzykowna w trakcie innych iteracji, albo eksperymentować z nowymi technologiami. Tester może nadrobić zaległości w automatyzacji starszych testów ręcznych. Analityk może pracować nad zaplanowaną w niedalekiej przyszłości obszerną funkcjonalnością, której — jego zdaniem — nie poświęcono wystarczającej uwagi.

Możliwość dysponowania jednym tygodniem pracy, dla którego członkowie zespołu kolektywnie sami ustalają priorytety, może być dla tegoż zespołu bardzo ożywcza. Należy podkreślić, że ta jednotygodniowa iteracja nie ma być okazją do nadrabiania zaległości spowodowanych nieudolną pracą w poprzednich iteracjach. Jest to raczej czas, kiedy zespoły mogą spłacić nieco technicznego długu powstałego podczas pierwszych iteracji — gdy zespół uczył się bycia Agile — albo z czasów sprzed stosowania metod Agile.

Studia dwóch przypadków

Aby zobaczyć, w jaki sposób można zastosować te czynniki w praktyce, rozważmy dwa zespoły: zespół Napa i zespół Goodman. Każdy z tych zespołów jest prawdziwy. Zmieniłem tylko kilka drobnych szczegółów.

Projekt Napa

Siedmioosobowy zespół Napa pracował nad aplikacją desktopową klient-serwer. Aplikacja miała być wykorzystywana przez 600 pracowników firmy. Nie miała być sprzedawana ani używana na zewnątrz firmy. Użytkownicy znajdowali się w trzech miastach, z których jedno było równocześnie siedzibą całego zespołu projektowego. Pomysł na produkt pojawił się jako próba przeprowadzenia aktualizacji technologii istniejącego systemu, którego utrzymanie stało się kosztowne. Jednak w związku ze zmianami w podstawowej działalności firmy zaplanowano w projekcie także sporo nowych funkcjonalności. Oszacowano, że projekt zajmie trzynaście miesięcy, ale szybki rozwój firmy wytwarzał presję na wcześniejsze wydanie, nawet gdyby miało zawierać tylko częściową funkcjonalność.

W projekcie Napa zespół zdecydował się na iteracje czterotygodniowe. Wiedzieliśmy, że projekt potrwa co najmniej pół roku, więc nawet czterotygodniowe iteracje dawały nam wiele możliwości doprowadzenia programu do stanu nadającego się do potencjalnego wydania tak, by można było go dostarczyć do rąk rzeczywistych użytkowników. Projekt zawierał sporo, choć nie nadmiernie dużo wymagań oraz niepewności technologicznej. Wszyscy deweloperzy mieli doświadczenie w stosowanych technologiach (C++ i Oracle). I chociaż nowa aplikacja miała zawierać funkcjonalności znacznie wykraczające poza zakres aplikacji bieżącej, to traktowaliśmy aplikację bieżącą jako podstawowy model potrzebnych funkcjonalności.

Zespół projektowy współpracował z wieloma przyszłymi użytkownikami systemu. Większość z nich była chętna do uczestnictwa w dyskusjach o tym, co stanie się ich nowym systemem. Firma rozwijała się jednak tak szybko, że dostęp do tych użytkowników był częściowo ograniczony. Musieliśmy uważać, aby nie zabrać im zbyt dużo czasu. W tej sytuacji dobrze sprawdzały się iteracje czterotygodniowe. Prezentowanie wersji co dwa tygodnie w tym środowisku byłoby zbyt częste. Dzięki przygotowaniu nowej wersji co cztery tygodnie mogliśmy skłonić większą liczbę użytkowników do eksperymentowania z oprogramowaniem w „piaskownicy”, która została do tego celu stworzona.

Ponieważ była to zupełnie nowa aplikacja, to koszt iteracji był bardzo mały. Dlatego nie musieliśmy go uwzględniać przy podejmowaniu decyzji. Projekt miał kluczowe znaczenie dla podtrzymania sukcesu firmy i miał bardzo wysoką widoczność wśród kierownictwa firmy, począwszy od naczelnego dyrektora. W przeważającej części projektu udało nam się ustalić i utrzymać priorytety dla iteracji czterotygodniowych. Nawet przy iteracji trwającej cztery tygodnie utrzymywała się presja czasu,

ponieważ członkowie zespołu byli świadomi potrzeby jak najszybszego dostarczenia pierwszego wydania aplikacji do rąk użytkowników.

Projekt Goodman

Zespół Goodman pracował nad pierwszą wersją komercyjnej aplikacji skali korporacyjnej. W ciągu pierwszych trzech lat firma planowała sprzedać nie więcej niż 5000 licencji na oprogramowanie. Jednak produkt był drogi. Jego średnia cena wyniosła 50 000 dolarów na jednego użytkownika. Osiemnastu deweloperów podzielono pomiędzy dwa skoordynowane zespoły. Projekt Goodman miał zająć rok, ale wstępne wydanie dla kilku klientów zaplanowano za sześć miesięcy.

Do realizacji projektu Goodman zespół zdecydował się na zastosowanie dwutygodniowych iteracji. Ponieważ naszym celem było pierwsze wydanie za sześć miesięcy, zespół mógł wybrać iteracje czterotygodniowe. Jednak ten projekt niósł ze sobą olbrzymią dżę niepewności. W firmie uważano, że wiadomo kim będą użytkownicy produktu, ale od czasu do czasu pojawiały się na ten temat spory. Nie było jasne, czy produkt powinien być wysokiej klasy drogim systemem tak jak od początku ustalano, czy też docelowa grupa użytkowników miała być bardziej obszerna, a cena niższa. Tę decyzję podjęto, następnie zmieniono, a potem zmieniono jeszcze raz, ale nigdy nie zrobiono tego odpowiednio stanowczo tak, aby zespół miał pewność co do udzielonej odpowiedzi. Pojawiało się również sporo nowych wymagań w rodzaju „będę wiedział, co jest potrzebne, kiedy to zobaczę”.

W tym trudnym projekcie nie było łatwo o opinie. Ponieważ był to produkt komercyjny, nie mieliśmy użytkowników wewnętrznych. Wielu największych obecnych i potencjalnych klientów firmy było z zagranicy, co dodatkowo komplikowało sprawę. W firmie było szereg osób, które mogłyby być potencjalnymi użytkownikami produktu, gdyby pracowały w firmach naszych docelowych klientów, więc wykorzystaliśmy ich jako pośredników naszych prawdziwych użytkowników. Z powodu tej niepewności i zmian w projekcie chcieliśmy otrzymywać opinie jak najczęściej. To przemawiało za krótszymi iteracjami.

W tym środowisku priorytety nie mogły pozostać przez więcej niż kilka dni, co także kierowało nas w stronę krótkich, dwutygodniowych iteracji, na jakie się zdecydowaliśmy. Zespół automatyzacji testów w tym projekcie pracował od samego początku i robił bardzo dobrą robotę. To pomogło utrzymać koszty iteracji na niskim poziomie i pozwoliło na skuteczne stosowanie krótkich iteracji. Wreszcie, ze względu na to, że firma była niedawnym start-upem i nową firmą na giełdzie, było dla nas ważne utrzymanie poczucia pilności projektu. Firma weszła na giełdę dzięki obietnicy szybkiego wydania świetnego oprogramowania. Krótkie, dwutygodniowe iteracje pomogły nam utrzymać koncentrację na jak najszybszym stworzeniu tego oprogramowania.

Unikaj końca kwartału

Chociaż łatwo ulec pokusie wyrównania iteracji z końcem miesiąca, staram się tego unikać za wszelką cenę. Jeśli iteracje będą powiązane z końcem miesiąca, to co trzecia iteracja zbiegnie się z końcem finansowego kwartału. Chociaż nie jest to zbyt duży problem w prywatnych firmach, to w spółkach obecnych na giełdzie występuje olbrzymia presja na osiągnięcie kwartalnych celów przychodów.

Swego czasu pracowałem w projekcie, w którym nowe, ważne wydanie zaplanowano na piątek, 31 marca 2000 roku. To wydanie było kluczowym celem dla większej części firmy na dziewięć miesięcy (co stanowi górny limit czasu opracowywania pojedynczego wydania). Na dwa tygodnie przed wydaniem nasz właściciel produktu wyjechał na ferie wiosenne ze swoimi dziećmi w wieku szkolnym. Będąc w Disneylandzie dzielnie starał się uczestniczyć w niektórych telekonferencjach i udzielać odpowiedzi na szereg ważnych pytań. Mimo wszystko jego nieobecność przypadła na kluczowy moment, a niektóre ważne prace nie zostały zakończone w iteracji, która miała być ostatnią przed ważnym wydaniem.

Gdy właściciel produktu wrócił z wakacji, udało się rozwiązać wszystkie pozostałe niezakończone sprawy w ciągu krótszej niż zwykle, jednodniowej iteracji. To sprawiło, że data wydania przesunęła się z 31 marca na 7 kwietnia. Chociaż różnica jednego tygodnia może nie wydawać się bardzo ważna dla dziewięciomiesięcznego projektu, to fakt, że z powodu opóźnienia wydanie przesunęło się z jednego kwartału na drugi, miało znaczące skutki dla firmy obecnej na giełdzie. Ponieważ planowane pierwsze wydanie kilkuset kopii nie stało się rzeczywistością 31 marca, nie można było uwzględnić dochodów z wstępnej sprzedaży i uaktualnień aż do drugiego kwartału. Równie dobrze jak 7 kwietnia mogliśmy zrealizować wydanie 30 czerwca.

Od tamtej pory nie zaplanowałem wydania na koniec miesiąca. Istnieje zbyt wiele niewiadomych, nieznanych okoliczności i niepewności w rozwoju oprogramowania, abym zdecydował się na podjęcie ryzyka uznania przychodów z powodu ustalenia daty wydania na koniec kwartału.

Podsumowanie

Większość zespołów Agile pracuje w iteracjach trwających od dwóch do czterech tygodni. Nie istnieje uniwersalna „jedynie słuszna” długość iteracji odpowiednia dla wszystkich zespołów. Zamiast tego zespół powinien wziąć pod uwagę swoją wyjątkową sytuację i dobrać dla siebie odpowiednią długość iteracji. Oto czynniki, które wpływają na tę decyzję:

- rozmiar wydania, nad którym pracujemy;
- stopień niepewności;
- łatwość uzyskania opinii zwrotnych;
- czas, przez jaki priorytety mogą pozostać bez zmian;
- gotowość do pracy bez opinii;
- koszty iteracji;
- jak szybko pojawia się presja czasu.

Pytania do dyskusji

1. Jaki jest optymalny czas iteracji dla Twojego bieżącego projektu?
2. Jakich różnic oczekiwałeś w projekcie, gdyby stosowano w nim iteracje jednodniowe? Jakich różnic mógłbyś oczekiwać, gdyby stosowano iteracje dwumiesięczne?

ROZDZIAŁ 16.

Szacowanie tempa

Lepiej podawać dane przybliżone i mieć rację niż precyzyjne i jej nie mieć.

— John Maynard Keynes

Jednym z wyzwań podczas planowania wydania jest oszacowanie tempa pracy zespołu. Dostępne są następujące trzy opcje:

- wykorzystanie danych historycznych,
- przeprowadzenie iteracji,
- wykonanie prognozy.

Są sytuacje, w których każde z tych podejść jest właściwe. Jednak bez względu na to, jakie podejście zastosujesz, to jeśli musisz przeprowadzić ocenę tempa, powinieneś ją wyrazić w postaci zakresu. Załóżmy, że oszacowałeś tempo pewnego zespołu w określonym projekcie jako 20 idealnych dni na iterację. Masz bardzo niewielkie szanse na to, aby dokonać tego oszacowania precyzyjnie. Tempo może wynosić 21 lub 19 albo być może nawet 20,0001. Zatem zamiast mówić, że tempo będzie wynosić 20, podaj tę ocenę jako zakres. Na przykład podaj, że szacowane tempo będzie mieścić się w zakresie między 15 a 24.

W poniższych podrozdziałach opiszę trzy ogólne podejścia — wykorzystanie danych historycznych, przeprowadzenie iteracji i wykonanie prognozy — i dla każdego z tych podejść zaoferuję zalecenia dotyczące wyboru odpowiedniego zakresu.

Wykorzystanie danych historycznych

Dane historyczne są świetne, pod warunkiem że masz do nich dostęp. Problem z danymi historycznymi polega na tym, że największą wartość mają wtedy, gdy zaszły bardzo niewielkie zmiany między starym projektem i starym zespołem a nowym projektem i nowym zespołem. Każda zmiana personalna lub istotna zmiana w technologii zmniejsza przydatność historycznych pomiarów tempa. Przed użyciem danych historycznych zadaj sobie następujące pytania:

- Czy technologia jest taka sama?
- Czy dziedzina dotyczy tego samego?

- Czy jest taki sam zespół?
- Czy właściciel produktu jest ten sam?
- Czy wykorzystywane są te same narzędzia?
- Czy jest takie samo środowisko pracy?
- Czy oceny były dokonane przez te same osoby?

Kiedy zespół przechodzi do nowego wydania produktu, nad którym pracował wcześniej, odpowiedź na każde z tych pytań często brzmi „tak”. W takim przypadku użycie przez zespół danych historycznych jest całkowicie właściwe. Pomimo że tempo w takiej sytuacji jest stosunkowo stabilne, nadal lepiej jest wyrazić je w postaci zakresu. Aby utworzyć zakres, wystarczy dodać kilka punktów do średniej bądź odjąć je od niej. Można też przyjrzeć się najlepszym i najgorszym iteracjom zespołu w ciągu ostatnich dwóch lub trzech miesięcy.

Jeśli jednak odpowiedź na którekolwiek z poprzednich pytań brzmi „nie”, to przed użyciem danych historycznych trzeba pomyśleć dwa razy. Można też używać historycznych temp, ale zastosować dla nich szerszy zakres, aby uwzględnić wewnętrzną niepewność związaną z oszacowaniem. Aby to zrobić, należy najpierw obliczyć średnie tempo zespołu podczas realizacji poprzedniego wydania. Jeśli w czasie ostatnich 10 iteracji udało mu się zrealizować 150 punktów, to jego średnie tempo wynosi 15 punktów.

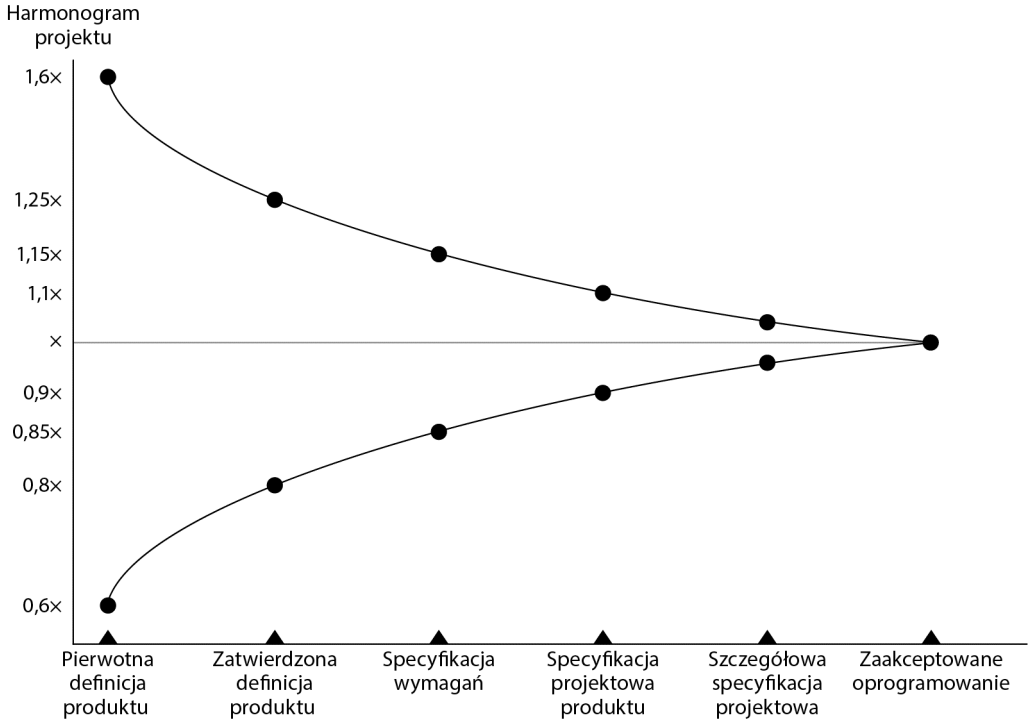
Zanim pokażemy, w jaki sposób przekształcić tę wartość na zakres, spójrzmy na rysunek 16.1. Pokazano na nim stożek niepewności, który wprowadziliśmy w rozdziale 1. „Cel planowania”. Stożek niepewności mówi, że rzeczywisty czas trwania projektu wynosi od 60% do 160% czasu szacowanego. Zatem w celu przekształcenia punktowego, średniego tempa na zakres pomnożę tę wartość przez 60% i przez 160%¹. Jeśli zatem średnie historyczne tempo wynosi 15, to oceniłbym tempo jako zakres między 9 a 24.

Powyższy zakres może się wydawać duży, ale biorąc pod uwagę niepewność w tym momencie, jest on prawdopodobnie właściwy. Budowanie zakresu w takiej formie pomaga zespołowi projektu stosować się do porady sformułowanej w cytacie na początku tego rozdziału — lepiej podawać dane przybliżone i mieć rację niż dokładne i jej nie mieć. Podawanie tempa w formie dużego zakresu wokół oczekiwanej wartości pozwala zespołowi być blisko dokładnej wartości.

Przeprowadzenie iteracji

Idealnym sposobem na prognozowanie tempa jest przeprowadzenie iteracji (albo dwóch bądź trzech), a następnie ocena tempa zespołu na podstawie temp obserwowanych w tym okresie. Ze względu na to, że najlepszym sposobem prognozowania tempa jest jego obserwowanie, zawsze powinno być to nasze podejście domyślne. Wiele tradycyjnych projektów rozpoczyna się od tego, że programiści zaczynają pracować nad „oczywistymi” wymaganiami lub infrastrukturą, analitycy „finalizują” wymagania, a kierownik projektu gromadzi obszerną listę zadań, które stają się planem projektu. Wszystko to wymaga czasu — w projekcie Agile nawet kilku iteracji.

¹ W gruncie rzeczy powinienem podzielić tę wartość przez 0,60 i 1,6. Jednak ze względu na to, że wartości 0,60 i 1,60 są w przybliżeniu odwrotnościami ($0,6 \times 1,6 = 0,96 \sim 1$), to w przybliżeniu te same wartości otrzymasz w wyniku mnożenia.



RYСУNEK 16.1. Stożek niepewności wokół ocen harmonogramu

Niedawno rozmawiałem z dyrektorem ds. rozwoju, który powiedział, że w tradycyjnych projektach w jego firmie terminy nie są ustalane nawet do dwóch miesięcy od rozpoczęcia projektów szacowanych na rok. Tak długo zajmuje im „zablokowanie” wymagań i stworzenie planu. Powiedział mi, że pomimo tego sporego wysiłku ich plany projektowe zawsze były przekraczane o co najmniej 50%, a często więcej. Zgodziliśmy się, że zamiast tego można by wykorzystać ten początkowy czas, by zespół rozpoczął prace nad projektem, obserwował swoje tempo przez dwie lub trzy iteracje, a następnie wykorzystał uzyskane dane do zaplanowania daty wydania.

Z analogicznych powodów, jakie podawał wspomniany dyrektor ds. rozwoju, większość kierowników projektów może wstrzymać się z przekazywaniem ocen przez co najmniej jedną iterację. Warto wykorzystać początkowy czas w projekcie na uruchomienie iteracji i zmierzenie tempa zespołu. Następnie należy stworzyć zakres wokół ustalonego punktu danych, posługując się stożkiem niepewności. Jeśli zatem ustaliłeś po jednej iteracji, że tempo zespołu wynosi 15, przekształć tę wartość na zakres przez pomnożenie jej przez 0,60 i 1,6. W efekcie uzyskasz zakres od 9 do 24.

Jeśli przed podaniem oceny tempa zespół może zrealizować trzy iteracje lub ich większą liczbę, to ma on kilka dodatkowych możliwości określenia zakresu. Pierwszym i najprostszy jest skorzystanie z zakresu obserwowanych wartości. Przypuśćmy, że zespół wykonał trzy iteracje i uzyskał tempa 12, 15 i 16. Może on wyrazić prawdopodobne tempa jako zakres od 12 do 16.

Jako alternatywę zespół może zastosować stożek niepewności. Chociaż nie istnieją solidne, empiryczne podstawy dla podejścia, które zamierzam opisać, to sprawdza się ono w praktyce i ma sens. Oto

wspomniane podejście: oblicz średnie tempo iteracji zrealizowanych przez zespół. Następnie, dla każdej zakończonej iteracji, przemieść się na stożku niepewności o jeden krok w prawo. Zatem dla zespołu, który zrealizował jedną iterację, zastosuj zakres dla kamienia milowego „Pierwotna definicja produktu”. Jeśli zespół zrealizował dwie iteracje, użyj zakresu dla kamienia milowego „Zatwierdzona definicja produktu” (od 80% do 120%) i tak dalej. Dla wygody te liczby zestawiono w tabeli 16.1.

TABELA 16.1. *Mnożniki do oceny tempa na podstawie liczby zrealizowanych iteracji*

Zrealizowane iteracje	Mnożnik dolnego zakresu	Mnożnik górnego zakresu
1	0,6	1,60
2	0,8	1,25
3	0,85	1,15
4 lub więcej	0,90	1,10

Założmy, że zespół zrealizował trzy iteracje i uzyskał w tym czasie średnie tempo dwadzieścia. Dla trzech iteracji właściwy zakres wynosi od 85% do 115%. Oznacza to, że jeśli średnie tempo zespołu po trzech iteracjach wynosi dwadzieścia, to faktyczne tempo na koniec projektu prawdopodobnie będzie w zakresie od siedemnastu do dwudziestu trzech.

Zwykle nie poszerzam tej analizy poza trzy do czterech iteracji. Na przykład nie używam stożka niepewności, aby udawać, że po sześciu iteracjach zespół będzie dokładnie znał swoją szybkość i nie będzie miał żadnych wątpliwości aż do końca projektu.

Niektóre firmy nie chcą rozpoczynać projektu, nie dysponując bardziej konkretnymi przewidywaniami co do tego, ile zajmie czasu. Należy podkreślić, że w takich sytuacjach potrzeba realizacji kilku iteracji nie jest unikaniem szacowania, ale unikaniem podawania ocen bez wystarczających podstaw. Należy zaznaczyć, że celem tych pierwszych iteracji jest ocena „ciemnych kątów” systemu, lepsze zrozumienie stosowanych technologii, precyzyjniejsze zrozumienie wymagań oraz zmierzenie szybkości postępów zespołu.

Prognozowanie

Czasami nie dysponujemy historycznymi danymi, a zrealizowanie kilku iteracji po to, by zaobserwować tempo zespołu jest niewykonalne. Założmy, że ocena dotyczy projektu, który nie rozpocznie się w ciągu najbliższych dwunastu miesięcy. Albo projekt może się rozpocząć w najbliższym czasie, ale dopiero po podpisaniu z klientem kontraktu. Takie sytuacje wyróżniają dwie charakterystyczne cechy. Po pierwsze, dążymy do minimalizowania kosztów projektu, dlatego nie rozpoczynamy realizacji iteracji w projekcie, który może się nie rozpocząć lub który zacznie się w zbyt odległej przyszłości. Po drugie, w każdej ocenie tempa takich projektów trzeba uwzględnić wysoki stopień niepewności.

W takich przypadkach trzeba skorzystać z prognozy tempa. Prognozowanie tempa rzadko jest pierwszą opcją podczas szacowania, ale jest to ważna opcja i warto pamiętać o jej istnieniu. Najlepszy sposób na prognozowanie tempa obejmuje rozwinięcie historyjek użytkownika do składowych zadań, ocenę tych zadań (tak jak się robi podczas planowania iteracji), ocenę, ile pracy można zrealizować w ciągu jednej iteracji, a następnie obliczenie tempa, jakie zostałoby osiągnięte, gdyby prace zostały zakończone w tej iteracji. Wykonanie tej procedury obejmuje następujące kroki:

1. Oszacuj, przez ile godzin każdy członek zespołu będzie mógł pracować codziennie nad projektem.
2. Określ całkowitą liczbę godzin, które zostaną przeznaczone na pracę nad projektem podczas iteracji.
3. W sposób dowolny i w pewnym sensie losowo wybierz kilka historyjek i rozwiń je do ich zadań składowych. Powtarzaj do chwili zidentyfikowania wystarczającej liczby zadań do wypełnienia godzin w iteracji.
4. Dokonaj konwersji tempa ustalonego w poprzednim kroku na zakres.

Zobaczmy, jak to działa na przykładzie.

Oszacowanie liczby godzin

Prawie każdy człowiek ma jakieś obowiązki poza projektem, który jest jego podstawowym zajęciem. Trzeba odpowiadać na e-maile, odbierać telefony czy uczestniczyć w spotkaniach. Ilość czasu, jaką to zajmuje, jest różna dla różnych osób i różnych organizacji. Wszystko sprowadza się jednak do tego, że członkowie zespołu projektowego, ogólnie rzecz biorąc, nie poświęcają 100% swojego czasu na pracę nad projektem.

Na podstawie obserwacji i dyskusji z kolegami mogę wyrazić opinię, że większość osób wyznaczonych do pracy nad projektem w pełnym wymiarze poświęca na tę pracę od czterech do sześciu godzin dziennie. Jest to zgodne z doniesieniami o tym, że członkowie zespołów poświęcają od 55% (Ganssle 2004 r.) do 70% (Boehm 1981) swojego czasu na pracę związane z projektem. Kennedy (2003) podaje dane „z najwyższej półki”. Informuje, że inżynierowie w Toyocie — gdzie stosowane są wysokowydajne procesy Lean — są w stanie poświęcić do 80% swojego czasu na pracę w wyznaczonych projektach.

Te liczby należy wykorzystać jako parametry do oszacowania ilości czasu, jaką członkowie zespołów projektowych będą w stanie poświęcić każdego dnia na pracę związane z projektem. Jeśli pracujesz w mocno zbiurokratyzowanej firmie, to prawdopodobnie znajdziesz się na dolnym końcu skali. Jeśli zaś pracujesz w trzyosobowym start-upie, w garażu, to znajdziesz się na jej górnym końcu. Na potrzeby tego przykładu założmy, że każdy członek zespołu SwimStats oszacował, że będzie w stanie poświęcić na projekt sześć godzin dziennie.

Oszacuj czas dostępny w iteracji

Ten krok jest prosty: należy pomnożyć liczbę godzin każdego dnia przez liczbę osób w zespole oraz liczbę dni w każdej iteracji. Załóżmy, że zespół SwimStats obejmuje jednego analityka, jednego programistę, jednego inżyniera bazy danych i jednego testera. Mamy cztery osoby, z których każda pracuje codziennie sześć godzin, zatem codziennie mamy do dyspozycji dwadzieścia cztery godziny. W czasie dziesięciodniowej iteracji osoby te poświęcają na projekt około 240 godzin.

Podczas prezentacji takiego podejścia w niektórych zespołach można się spotkać z prośbami o uwzględnienie w szacunkach takich czynników jak urlopy, zwolnienia lekarskie oraz inne podobne przerwy. Nie należy się nimi martwić. Nie są one warte dodatkowego wysiłku i jest mało prawdopodobne, aby ich uwzględnienie przyczyniło się do zwiększenia dokładności. Właśnie z powodu tego rodzaju zdarzeń nie planuję, że wszystkie osoby będą pracowały nad projektem przez 100% czasu.

Uzyskanie większej ilości czasu na projekt

Niezależnie od tego, ile godzin członkowie zespołu są w stanie poświęcić na projekt każdego dnia, prawdopodobnie chciałbyś zwiększyć tę liczbę. Najlepszy sposób, aby to zrobić, opisał Francesco Cirillo z firmy XPLabs. Cirillo trenuje zespoły do pracy w ścisłej koncentracji w trzydziestominutowych przedziałach (Cirillo 2005). Każdy trzydziestominutowy przedział składa się z dwóch części: dwudziestu pięciu minut intensywnej pracy, po których następuje pięciominutowa przerwa. Te trzydzieści minut jest określane nazwą „pomodori” (z włoskiego „pomidor”). Nazwa ta wywodzi się od stosowania klepsydry w kształcie pomidorów, które po upływie dwudziestu pięciu minut wydają sygnał dźwiękowy.

Cirillo zaproponował stosowanie tej techniki Piergiuliano Bossiemu, który udokumentował jej skuteczność w pracy z wieloma zespołami (Bossi 2003; Bossi i Cirillo 2001). Zespoły planowały realizację dziesięciu pomodori (pięć godzin) dziennie. Jeśli w swojej firmie zaobserwowałeś mniejszą produktywność, niż życzyłybyś sobie, możesz rozważyć zastosowanie tego podejścia.

Rozwiń historyjki do zadań i zobacz, ile zadań zdołasz zrealizować w iteracji

Następny krok to rozwinięcie historyjek do zadań, oszacowanie zadań i podjęcie próby wypełnienia iteracji szacunkową liczbą godzin (w tym przypadku 240). Nie jest konieczne, aby historyjki były rozwijane zgodnie z porządkiem priorytetów. Staraj się wybrać losowy zbiór historyjek. Nie wybieraj na przykład do rozwijania samych jedno- i dwupunktowych historyjek i żadnej historyjki trzy- bądź pięciopunktowej. Podobnie nie rozwijaj tylko takich historyjek, które obejmują głównie zadania związane z tworzeniem interfejsu użytkownika lub bazą danych. Spróbuj znaleźć reprezentatywny zbiór historyjek.

Wybieraj historyjki i rozwijaj je do zadań tak, aby wybrane zadania nie przekroczyły możliwości poszczególnych osób w zespole. Przykładowo, w przypadku zespołu SwimStats nie należy zakładać, że programista i analityk są jednocześnie zaawansowanymi inżynierami bazy danych. Wybieraj historyjki tak długo, aż członkowie zespołu o jednej specjalności nie będą w stanie wykonać więcej pracy. Dodaj punkty lub dni idealne dla wybranych zadań. Otrzymana wartość to prawdopodobne tempo zespołu.

Załóżmy, że zebraliśmy planowany zespół (lub jego namiastkę, jeśli projekt nie rozpocznie się w ciągu roku) i rozwinęliśmy niektóre historyjki tak jak pokazano w tabeli 16.2. Jeśli uważamy, że czteroosobowy zespół SwimStats może się zobowiązać do zrealizowania tych zadań, ale prawdopodobnie nie zdoła zrobić niczego więcej, to zatrzymujemy się w tym miejscu. Te 221 godzin pracy wydaje się rozsądnie pasować do 240 godzin dostępnego czasu. Punktowa ocena naszego tempa wynosi zatem dwadzieścia pięć.

Wyznacz zakres

Użyj dowolnej wybranej metody, aby przekształcić punktową ocenę tempa w zakres. Jak wspominałem wcześniej, aby wyznaczyć zakres, mnożę uzyskaną ocenę przez 60% i 160%. Dla zespołu SwimStats oznacza to, że ocena dwadzieścia pięć punktów każdej iteracji jest zakresem od piętnastu do czterdziestu.

TABELA 16.2. *Godziny i punkty dla niektórych historyjek zespołu SwimStats*

Historyjka	Ocena w punktach	Liczba godzin na zadanie
Jako trener mogę wpisywać nazwiska i dane osobowe wszystkich pływaków z mojej drużyny.	3	24
Jako trener mogę definiować sesje treningowe.	5	45
Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.	2	18
Jako pływak mogę aktualizować swoje dane osobowe.	1	14
Jako pływak mogę obejrzeć wykres liniowy swoich czasów w wybranych zawodach.	2	14
Jako trener mogę obejrzeć wykres liniowy postępów wyników w zawodach w ciągu sezonu wszystkich swoich pływaków.	3	30
Jako pływak mogę obejrzeć wykres kołowy pokazujący, ile pierwszych, ile drugich, ile trzecich, a ile niższych miejsc zająłem na zawodach.	2	12
Jako trener mogę obejrzeć tekstowy raport, pokazujący najlepszy czas każdego pływaka w każdym zawodach.	2	14
Jako trener mogę przesłać wyniki zawodów z pliku wyeksportowanego z systemu pomiaru czasu wykorzystywanego na zawodach.	5	50
Razem	25	221

Odmiana metody dla niektórych zespołów

Niektóre zespoły — zwłaszcza te, które są złożone z dużej liczby pracowników czasowych — nie powinny planować z uwzględnieniem stałej liczby godzin typowej dla sytuacji, kiedy wszyscy są dostępni. Członkowie tych zespołów mogą poświęcić na projekt znacznie mniejszą część swojego czasu. W takich przypadkach może być przydatne stworzenie zestawienia podobnego do pokazanego w tabeli 16.3.

TABELA 16.3. *Obliczenia dostępności w zespole z pracownikami zatrudnionymi w niepełnym wymiarze godzin*

Osoba	Liczba dostępnych godzin dziennie	Liczba dostępnych godzin na iterację
Sergiusz	4	40
Jerzy	6	60
Karina	2	20
Stanisław	6	60
Razem		180

W przypadku zespołu SwimStats zgodnie z tym, co pokazano w tabeli 16.3, Jerzy i Stanisław pracują w projekcie w pełnym zakresie godzin. SwimStats to jedyny projekt Sergiusza, ale wykonuje on kilka innych zadań menedżerskich i korporacyjnych, które zajmują mu trochę czasu. Karina dzieli swój czas między SwimStats a inny projekt. Ma bardzo mało obowiązków poza tymi dwoma projektami, a zatem mogłaby poświęcić na ich realizację blisko sześć godzin dziennie. Musi jednak przełączać się kilka razy dziennie między dwoma projektami. Ta wielozadaniowość wpływa na jej wydajność, dlatego — jak wynika z tabeli — może poświęcić na projekt SwimStats tylko dwie produktywne godziny dziennie.

Pamiętaj, po co to robimy

Należy pamiętać, że powodem, dla którego prognozujemy tempo w ten sposób, jest albo brak możliwości, albo nieopłacalność wykonania iteracji, a zespół, jak dotąd, nie dysponuje żadnymi danymi historycznymi. Może tak właśnie być, ponieważ zespół jeszcze nie istnieje, a otrzymaliśmy zadanie zaplanowania projektu, który rozpoczyna się za kilka miesięcy.

Jeśli na przykład pracujesz w środowisku, w którym realizujesz strategiczne planowanie i budżet z dużym wyprzedzeniem przed rozpoczęciem realizacji projektu, to prognozowanie tempa w opisany sposób może być najlepszym możliwym podejściem.

Jakiego sposobu należy użyć?

Ustalenie sposobu, jaki należy zastosować, często jest łatwiejsze, niż może się wydawać na pierwszy rzut oka. Często na wybór opcji wpływają okoliczności, które mogą być dla nas zarówno wskazówką, jak i ograniczeniem. Przystępując do szacowania tempa, postępuj zgodnie z poniższymi wskazówkami w malejącej kolejności celowości:

- Jeśli można wykonać jedną lub kilka iteracji przed oszacowaniem tempa, to zawsze to rób. Żadne szacowanie nie może się równać z rzeczywistymi danymi, dlatego obserwacja rzeczywistego tempa zespołu to zawsze nasz najlepszy wybór.
- Skorzystaj z rzeczywistego tempa wybranego zespołu w podobnym projekcie.
- Oszacuj tempo przez wybór zadań, które można zmieścić w iteracji.

Bez względu na zastosowane podejście przełącz się na korzystanie z właściwych, zaobserwowanych wartości tempa możliwie jak najszybciej. Załóżmy, że zdecydowałeś się oceniać tempo przez obserwację zadań, które zmieszczą się w iteracji. Projekt nie ma się rozpocząć wcześniej niż za sześć miesięcy, a firma potrzebuje jedynie zgrubnych szacunków co do tego, ile czasu projekt zajmie. Kiedy projekt się rozpocznie i będziesz mógł zmierzyć rzeczywiste tempo pracy, wtedy podczas dyskusji nad projektem oraz wyznaczania prawdopodobnego zakresu dat zakończenia zacznij używać rzeczywistych danych.

Podsumowanie

Istnieją trzy sposoby szacowania tempa. Po pierwsze, można użyć historycznych średnich, o ile są dostępne. Jednak przed użyciem średnich, historycznych wartości należy wziąć pod uwagę, czy nie było znaczących zmian w zespole, istocie projektu, technologiach i tak dalej. Po drugie, można odrzucić

ocenę tempa do momentu wykonania kilku iteracji. Zazwyczaj jest to najlepsza możliwość. Po trzecie, można przewidzieć tempo poprzez rozbicie kilku historyjek użytkownika na zadania i sprawdzenie, ile zadań zmieści się w iteracji. Proces ten jest bardzo podobny do planowania iteracji.

Bez względu na zastosowane podejście ocena tempa powinna być podana w postaci zakresu, który odzwierciedla niepewność charakterystyczną dla szacowania. Dane dotyczące rozmiaru używanego zakresu można czerpać z analizy stożka niepewności.

Pytania do dyskusji

1. Dla historyjek z tabeli 16.2, które oceniono na taką samą liczbę punktów, liczba godzin przypisanych do poszczególnych zadań nie jest taka sama. Dlaczego? (jeśli potrzebujesz odświeżenia wiadomości, zajrzyj do podrozdziału „Związek ocen zadań z punktacją historyjek” w rozdziale 14. „Planowanie iteracji”).
2. Uzupełnij tabelę podobną do tabeli 16.3 dla Twojego bieżącego projektu. Co mogłoby Cię skłonić do zwiększenia ilości czasu, jaką każdy członek zespołu może poświęcić na pracę nad projektem?

ROZDZIAŁ 17.

Plany z buforami w celu uwzględnienia niepewności

*Być niepewnym jest niewygodnie,
ale gdy ktoś nie ma żadnych wątpliwości, jest śmieszny.*

— chińskie przysłowie

Często słyszę, że planowanie Agile nie sprawdza się w niektórych środowiskach. Zazwyczaj w tym kontekście podaje się środowiska, w których:

- projekt jest zaplanowany z dużym wyprzedzeniem;
- projekt musi być wydany w ustalonym w firmie terminie i uwzględniać sensownie stały zbiór funkcjonalności;
- projekt został zamówiony przez jedną firmę w innej firmie;
- wymagania są rozumiane tylko na bardzo powierzchownym poziomie;
- firma nie chce pozwalać na zbyt wiele elastyczności w harmonogramach, nawet w takich projektach, które nie wymagają ściśle określonych terminów i wyników pracy.

Niezwykle ważna w tego rodzaju środowiskach jest możliwość tworzenia niezawodnych planów. Często nie wystarczy zastosować podejścia omówionego w poprzednich rozdziałach. Wspólne dla tych środowisk jest to, że w każdym z nich albo występuje dodatkowa niepewność, albo są większe konsekwencje popełnienia błędów. Na przykład gdy swego czasu byłem wiceprezesem ds. wytwarzania oprogramowania w firmie z listy „Fortune 40”, nasze projekty były zazwyczaj planowane w momencie, kiedy zaczynaliśmy planować budżet na kolejny rok — na dwanaście do osiemnastu miesięcy przed ich rozpoczęciem. Oczywiście nie blokowaliśmy wymagań, ale chcieliśmy określić charakter produktów, które będziemy budować. Pomimo tego, że wymagania były jedynie mgliście rozumiane, musieliśmy podjąć pewne zobowiązania tak, by można było odpowiednio dobrać członków zespołu. W tych okolicznościach istniała dodatkowa niepewność, ponieważ rzadko wiedziałem, kto z mojego zespołu będzie pracować nad projektami w tak odległej przyszłości. Często członkowie zespołu nie byli nawet zatrudnieni.

Spróbujmy zestawić tę sytuację z projektem, w którym trzeba określić ścisły termin i zobowiązać się do realizacji podstawowego zestawu funkcjonalności. Niedotrzymanie terminu lub dostarczenie znacznie mniej funkcjonalności może zniszczyć reputację firmy w branży i Twoją reputację w firmie. Nawet jeśli dobrze rozumiesz wymagania i znasz osoby, które znajdują się w zespole (inaczej niż w przypadku, który opisałem wcześniej), ryzyko popełnienia błędu jest znaczne.

W takich sytuacjach albo istnieje większa niepewność, albo większe ryzyko popełnienia błędów przy ustalaniu harmonogramów. Z tego powodu przy określaniu harmonogramu warto uwzględnić bufor. Bufor to margines błędu przy dokonywaniu oceny. Uwzględnienie bufora jest sensowne wtedy, gdy istnieje znaczna niepewność albo koszty pomyłki są duże. Bufor pomaga chronić projekt przed negatywnym wpływem niepewności. Dzięki temu buforowanie harmonogramu projektu staje się odpowiednią strategią zarządzania ryzykiem. W tym rozdziale przyjrzymy się dwóm typom buforów: buforom funkcjonalności oraz buforom harmonogramu.

Bufory funkcjonalności

Wczoraj wieczorem poszedłem do sklepu spożywczego z listą trzydziestu siedmiu produktów do kupienia. Miałem tylko trzydzieści minut na zakupy i jazdę do domu, ponieważ chciałem obejrzeć mecz koszykówki, który miał się rozpocząć o tej porze. Tak jak zawsze zacząłem od jednego końca sklepu i szedłem w przeciwnym kierunku. Ale kiedy przechodziłem każdą alejką, byłem skupiony na około dwudziestu towarach, których najbardziej potrzebowaliśmy. Wiedziałem, że gdybym wrócił do domu bez mleka, chleba lub indyka, to byłbym w tarapatkach i w przerwie musiałbym wrócić do sklepu. Ale gdybym zapomniał o mniej ważnych produktach, to moja żona byłaby usatysfakcjonowana, moje córki miałyby co jeść, a ja mógłbym obejrzeć mecz koszykówki.

Buforowanie w projekcie za pomocą funkcjonalności działa dokładnie tak samo. Mówimy naszym klientom: „Dostarczymy Wam wszystkie funkcjonalności z tego zbioru i — w idealnym przypadku — niektóre funkcjonalności z tego zbioru”. Utworzenie bufora funkcjonalności w projekcie Agile jest proste. Najpierw klient wybiera wszystkie absolutnie niezbędne prace. Oceny tej pracy są ze sobą sumowane. To reprezentuje minimum, które może zostać opublikowane. Następnie klient wybiera od 25% do 40% dodatkowych zadań, przy czym wyższa wartość zakresu dotyczy projektów z większą niepewnością lub mniejszą tolerancją na ryzyko ustalenia nieprawidłowego harmonogramu. Oceny dla tej pracy są dodawane do oceny wyjściowej i w ten sposób ustalana jest całkowita ocena dla projektu. Następnie planowanie w projekcie odbywa się tak jak zwykle — dotyczy dostarczenia *kompletnego* zbioru funkcjonalności. Pewna część pracy jest jednak opcjonalna i zostanie uwzględniona tylko wtedy, kiedy pozwoli na to czas. Prace opcjonalne są wykonywane w ostatniej kolejności — tylko wtedy, kiedy zostaną zrealizowane prace obowiązkowe.

Aby zobaczyć, jak to działa, załóżmy, że właściciel produktu zidentyfikował 100 punktów jako obowiązkowe. Każda wybrana historyjka musi się znaleźć w wydaniu produktu, który ma być pozytywnie przyjęty przez rynek. Następnie właściciel produktu wybiera kolejne 30% pracy, identyfikując historyjki użytkownika warte dodatkowych 30 punktów. Są one dodawane do projektu jako prace opcjonalne. Łącznie projekt jest teraz oceniany na 130 punktów. Za pomocą metod opisanych w rozdziale 16. „Szacowanie tempa” zespół ocenił tempo na 10 punktów na trwającą jeden tydzień iterację. W związku z tym zaplanowano, że projekt będzie obejmował trzynastą iterację (130/10). Jeśli wszystko pójdzie dobrze, to obowiązkowe prace zostaną wykonane po pierwszych dziesięciu iteracjach, a pozostałe trzy iteracje zostaną przeznaczone na funkcjonalności dodatkowe.

Proces buforowania funkcjonalności jest zgodny z używaną w procesach Agile metodyką DSDM (*Dynamic Systems Development Method*). W projektach DSDM wymagania są klasyfikowane według czterech kategorii: obowiązkowe (*must have*), pożądane (*should have*), mile widziane (*could have*) i zbędne (*won't have*). W metodyce DSDM ten rodzaj sortowania określa się terminem „reguły MoSCoW” (dosł. reguły moskiewskie). Na realizację obowiązkowych wymagań w projekcie można przeznaczyć nie więcej niż 70% planowanego wysiłku. Dzięki temu w projektach DSDM tworzy się bufor funkcjonalności równoważny 30% czasu trwania projektu.

Bufory harmonogramu

Założmy, że muszę dotrzeć na lotnisko i zdążyć na lot do Włoch (ponieważ to tylko założenia, mógłbym równie dobrze założyć, że chcę polecieć w jakieś inne przyjemne miejsce). Lot samolotem ma ściśle określony termin. Samolot wystartuje niezależnie od tego, czy będę na pokładzie, czy nie. Podczas tworzenia planu, jak dotrzeć z mojego domu do właściwej bramki na lotnisku, muszę uwzględnić to, żeby wyjechać na tyle wcześnie, aby być pewien, że zdążę na lot, ale nie za wcześnie — na przykład trzy dni przed datą lotu.

Myszę o wszystkich fazach swojej podróży: jeździe na lotnisko, parkowaniu samochodu, odprawie bagażu i przejściu przez punkt kontrolny bezpieczeństwa. Myszę o tym, ile czasu zajmie każdy z etapów, jeśli wszystko pójdzie dobrze, i stwierdzam, że wszystko powinno mi zająć siedemdziesiąt minut. Taka ocena sprawia wrażenie wiarygodnej. Wszystko może potrwać nawet kilka minut mniej, ale równie dobrze może zająć znacznie więcej czasu. Jeśli zdarzy się wypadek na autostradzie lub jeśli zabraknie miejsca na parkingu albo będzie długa kolejka do odprawy bagażu lub do kontroli bezpieczeństwa, to całość może potrwać znacznie dłużej. Nie muszę planować, że wszystko pójdzie źle w tej podróży, ale ze względu na to, że to jest ważny wyjazd i nie chcę się spóźnić na lot, powinienem dodać do oszacowanych siedemdziesięciu minut, gdy wszystko pójdzie dobrze, trochę czasu.

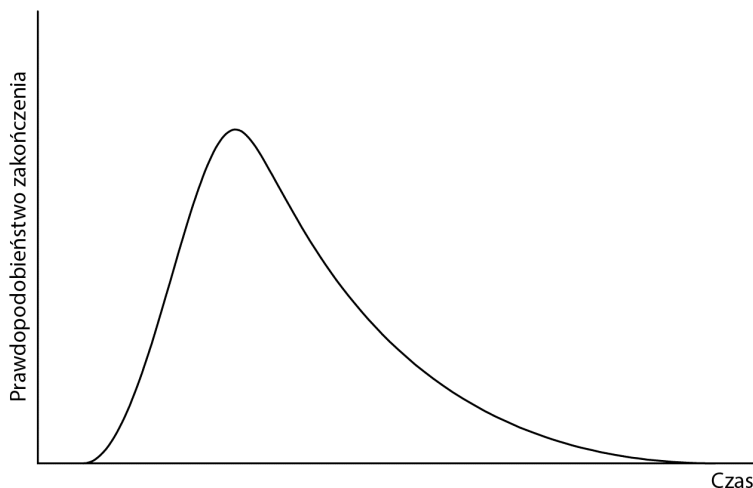
Powiedzmy, że zdecydowałem się wyjechać na lotnisko sto minut przed odlotem. Jeśli wszystko pójdzie dobrze, to zostanie mi trzydzieści minut na spacer po porcie lotniczym, co nie jest złe. Jeśli wszystko pójdzie wyjątkowo dobrze (wszystkie światła będą zielone, uda się zaparkować w pierwszym rzędzie, w kolejce do odprawy i kontroli bezpieczeństwa nie będzie przede mną nikogo), to być może będę miał czterdzieści minut na lotnisku. Jeśli jednak utknę w korku albo w długiej kolejce do odprawy, to dzięki uwzględnieniu dodatkowego czasu będzie go wystarczająco dużo, bym zdążył na samolot, zanim odleci. Dodatkowe trzydzieści minut to mój bufor harmonogramu, który zabezpiecza terminową realizację mojego ogólnego projektu (dotarcia na lotnisko).

W przypadku mojej podróży na lotnisko nie jest właściwe odgadywanie tempa, a następnie podawanie linii lotniczej (klientowi) okresowych aktualizacji planowanej godziny przyjazdu. Moja planowana godzina przyjazdu oraz moje tempo nie mają znaczenia dla linii lotniczych. Godzina odlotu jest ustalona, podobnie jak terminy w wielu projektach wytwarzania oprogramowania. W tych przypadkach bufor harmonogram chroni przed niepewnością, która może wpłynąć na terminową realizację projektu.

Zwróćmy uwagę, że nie obchodzi mnie to, czy pojedyncza aktywność (jazda samochodem, parkowanie, odprawa bagażu lub kontrola bezpieczeństwa) zajmuje zbyt dużo czasu. Interesuje mnie tylko to, czy całkowity łańcuch aktywności potrwa zbyt długo. Aby zdążyć na swój lot, dodaję trzydziestominutowy bufor do ogólnego harmonogramu dotarcia na lotnisko. Powinniśmy dodać podobny bufor harmonogramu dla projektów z większą niepewnością lub poważniejszymi konsekwencjami z powodu niedotrzymania terminu.

Uwzględnianie niepewności w ocenach

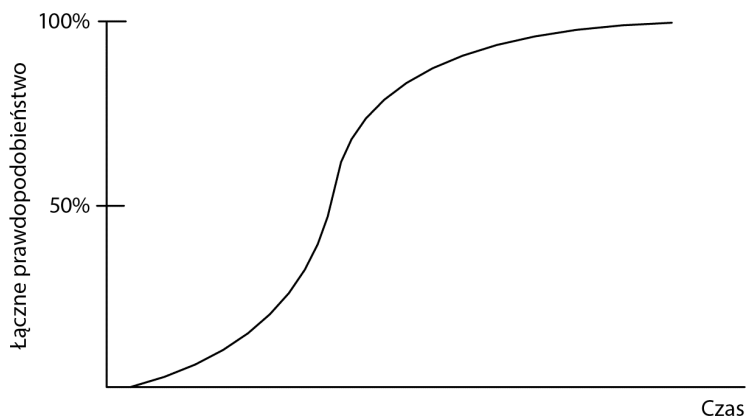
Aby ochronić harmonogram projektu przed niepewnością, potrzebny jest sposób na ilościową ocenę niepewności. Kiedy oszacujemy historyjkę użytkownika i przypiszemy jej pojedynczą wartość, wtedy udajemy, że ta pojedyncza liczba odzwierciedla nasze oczekiwania dotyczące ilości czasu, jaką zajmie praca nad tą funkcjonalnością. Jednak gdy próbujemy ocenić sytuację bardziej realistycznie, wtedy wiemy, że praca może zostać zakończona w terminie określonym przez zakres. Zespół może ocenić określoną historyjkę użytkownika na trzy dni idealne, wiedząc, że trzy dni idealne zazwyczaj odpowiadają czterem lub pięciu fizycznym dniom pracy. Nikogo jednak nie będzie szokowało, jeśli realizacja historyjki zajmie sześć dni. Niektóre rzeczy niekiedy zajmują więcej czasu niż planowano. Jeśli utworzymy wykres możliwych terminów zakończenia zadania, to będzie on wyglądał mniej więcej tak jak pokazano na rysunku 17.1.



RYSUNEK 17.1. Rozkład czasów zakończenia

Krzywa przyjmuje taki ogólny kształt ze względu na to, że zazwyczaj nie można zrobić zbyt wiele, aby przyspieszyć wykonanie zadania. Istnieje jednak nieskończona liczba czynników, które mogą się nie powieść i wpłynąć na opóźnienie wykonania zadania. Na przykład przed samym końcem kodowania określonej nowej funkcjonalności mój komputer ulega awarii i tracę niezapisane zmiany. Piorun uderza w budynek i „smaży” nasze repozytorium kodu źródłowego. Prosimy o dostarczenie taśmy z kopią zapasową na kolejny dzień, ale kurier gubi taśmę. Mnie to jednak nie obchodzi. Zupełnie tak jakby potrafił mnie przysłowiowy autobus w drodze do pracy.

Najbardziej prawdopodobny czas zakończenia na wykresie z rysunku 17.1 to miejsce, gdzie wykres osiąga szczyt. Jednak, ogólnie rzecz biorąc, prawdopodobieństwo zakończenia pracy do tego czasu wynosi mniej niż 50%. Wiemy to stąd, że mniej niż 50% powierzchni pod krzywą znajduje się po lewej stronie wartości szczytowej. Gdyby deweloper miał dostarczyć ocenę odpowiadającą wartości maksymalnej z rysunku 17.1, to zakończenie pracy najprawdopodobniej zajęłoby mu więcej czasu niż podana ocena. Bardziej przydatny sposób wizualizacji zilustrowano na rysunku 17.2, na którym pokazano łączne prawdopodobieństwo zakończenia prac w terminach przedstawionych na osi poziomej lub wcześniej.



RYСУNEK 17.2. Łączny rozkład czasów zakończenia

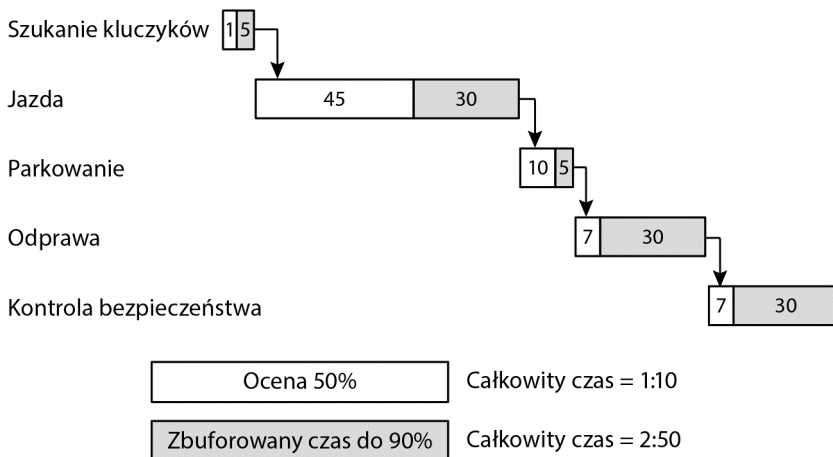
O ile na rysunku 17.1 zaprezentowano prawdopodobieństwo zakończenia prac w ściśle określonym czasie, o tyle na rysunku 17.2 pokazano prawdopodobieństwo zakończenia prac w tym momencie lub wcześniej. Przy ocenie i planowaniu jest to dla nas ważniejsze niż prawdopodobieństwo zakończenia określonego dnia (tak jak pokazano na rysunku 17.1).

Innym sposobem myślenia o wykresie z rysunku 17.2 oraz łącznym prawdopodobieństwie czasów zakończenia jest założenie, że 100 różnych, ale równie wykwalifikowanych i doświadczonych programistów niezależnie pracuje nad nową funkcjonalnością. Do kiedy skończą? Wyniki mogą być podobne do tych, które przedstawiono w tabeli 17.1. W tabeli tej pokazano liczbę programistów, którzy skończyli pracę określonego dnia oraz, co ważniejsze, łączną liczbę programistów, którzy do tego dnia skończyli pracę.

TABELA 17.1. Liczba programistów, którzy ukończyli funkcjonalność określonego dnia

Dzień	Liczba programistów, którzy skończyli pracę tego dnia	Całkowita liczba programistów, którzy skończyli pracę
Dzień 1.	5	5
Dzień 2.	40	45
Dzień 3.	25	70
Dzień 4.	15	85
Dzień 5.	10	95
Dzień 6.	5	100

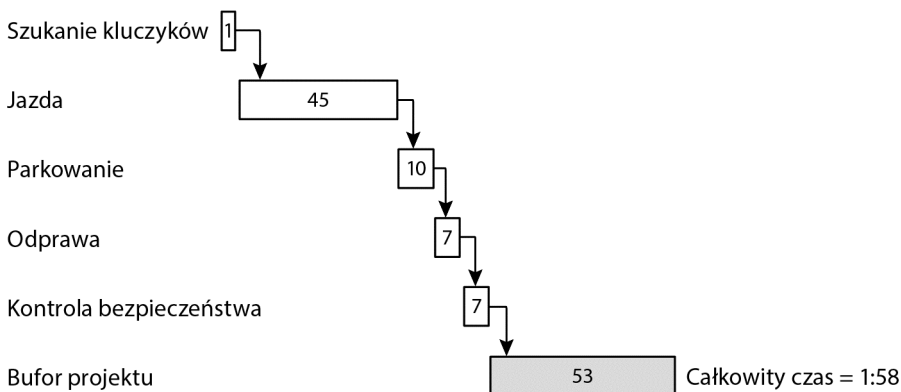
Załóżmy, że chcemy mieć 90-procentową pewność poprawności harmonogramu, do którego się zobowiązaliśmy. Początkowe podejście do wykonania tego zadania mogłoby polegać na oszacowaniu z 90-procentowym prawdopodobieństwem czasu realizacji każdej historyjki użytkownika w projekcie, a następnie wykorzystaniu tych ocen. Jeśli jednak zastosujemy taki sposób, to niemal na pewno projekt będzie trwał za długo. Aby zobaczyć, jak działa bufor harmonogramu, spróbujemy jeszcze raz posłużyć się przykładem mojej podróży na lotnisko. Możliwy harmonogram tej podróży przedstawiono na rysunku 17.3.



RYSUNEK 17.3. Oceny 50% i 90% dotarcia na lotnisko

Pierwsza liczba dla każdego zadania (w jasnym polu) to ocena czasu wykonania zadania z prawdopodobieństwem 50%. Spodziewam się, że w połowie przypadków zadania zajmą więcej czasu, a w drugiej połowie zajmą mniej czasu. Druga liczba (w szarym polu) to dodatkowa ilość czasu, aby osiągnąć moją ocenę z prawdopodobieństwem 90%. Dodatkowy czas między oceną 50% a 90% określa się jako *lokalne zabezpieczenie*. Do oceny często dodaje się lokalne zabezpieczenie w sytuacji, gdy chcemy mieć większą pewność, że uda się sprostać zadaniu. W tym przypadku sądzę, że uda mi się znaleźć kluczyki w ciągu od jednej do sześciu minut. Zdążę dojechać na lotnisko w czasie od czterdziestu pięciu do siedemdziesięciu pięciu minut i tak dalej.

Zsumowanie wartości dla oceny z prawdopodobieństwem 50% daje mi oczekiwany czas trwania wynoszący jedną godzinę i dziesięć minut. Jeśli jednak wyjdę z domu tak blisko czasu wylotu, to najmniejsze opóźnienie spowoduje, że nie zdążę na samolot. Z drugiej strony po zsumowaniu ocen z prawdopodobieństwem 90% uzyskałem wynik 2 godziny i 50 minut. Nie chcę wychodzić z domu prawie trzy godziny przed odlotem, ponieważ jest bardzo mała szansa, że wszystko pójdzie źle. Tym, czego chcę naprawdę, jest plan projektu, który wygląda tak jak pokazano na rysunku 17.4.



RYSUNEK 17.4. Podróż na lotnisko z buforem

Plan z rysunku 17.4 utworzono z wykorzystaniem ocen 50%, po których zsumowaniu dodano bufor dla projektu. Taki typ planu ma znacznie większy sens niż plan zbudowany w całości przez zsumowanie ocen 50% lub 90%. Plan z rysunku 17.4 chroni tylko jeden termin, który ma znaczenie: termin zakończenia całego projektu. Ponieważ nie jest ważne, czy wykonanie któregoś zadania na drodze na lotnisko zakończy się później, to nie ma potrzeby buforowania wykonania każdego zadania z osobna. W ten sposób mogą zbudować harmonogram, w którym usunięto lokalne zabezpieczenia z pojedynczych zadań, a część tego czasu pozostawiono do utworzenia bufora, który chroni cały harmonogram. Zwróćmy uwagę, że łączny czas harmonogramu z buforem zaprezentowanego na rysunku 17.4 wynosi zaledwie 1 godzinę i 58 minut — czyli prawie o godzinę krócej niż po zsumowaniu ocen 90%.

Co ważniejsze, dzięki przeniesieniu lokalnych zabezpieczeń do ogólnego bufora projektu udało się uniknąć wpływu prawa Parkinsona oraz zespołu studenta. Jak pamiętamy z rozdziału 2. „Dlaczego planowanie zawodzi”, prawo Parkinsona mówi, że praca rozszerza się, aby wypełnić czas dostępny na jej zakończenie. Syndrom studenta (Goldratt 1997) odnosi się do rozpoczęcia realizacji zadania w ostatnim możliwym momencie, który nie wyklucza możliwości pomyślnego zakończenia — na przykład rozpoczęcie pisania zaliczenia semestralnego na trzy dni przed terminem oddania. Ze względu na to, że harmonogram z buforem przeciwdziała problemom spowodowanym przez prawo Parkinsona i zespół studenta, szanse zrealizowania krótszego harmonogramu uwzględniającego bufor są większe niż harmonogramu dłuższego.

Aby utworzyć bufor harmonogramu dla projektu wytwarzania oprogramowania, przede wszystkim trzeba skorygować proces szacowania tak, aby dla każdej historyjki użytkownika bądź funkcjonalności były generowane dwie oceny. Tak jak w przypadku jazdy na lotnisko, dla każdej z wymienionych pozycji trzeba wyznaczyć oceny, których prawdopodobieństwo wynoszą, odpowiednio, 50% oraz 90%. Jest to dość łatwe: podczas spotkania poświęconego ustalaniu ocen zaczynamy od szacowania przypadku 50% dla pierwszej historyjki. Następnie dla tej samej historyjki dokonujemy oceny z prawdopodobieństwem 90% i przechodzimy do następnej historyjki lub funkcji.

Określanie rozmiaru bufora projektu

Bufor projektu z rysunku 17.4 miał rozmiar pięćdziesięciu trzech minut. Jak ustaliłem taki rozmiar? Podstawą do jego obliczenia były pokazane na rysunku 17.4 oceny dla prawdopodobieństw 50% i 90%. Olbrzymia zaleta przypisywania dwóch ocen do każdej historyjki użytkownika polega na tym, że te liczby bardzo wyraźnie wskazują na stopień ryzyka harmonogramu powiązanego z każdym z nich. Na przykład jeśli jedną historyjkę oceniono od trzech do pięciu, a inną od trzech do dziesięciu, to wiadomo, że druga historyjka stwarza większe zagrożenie dla harmonogramu projektu. Rozmiar bufora projektu powinien uwzględniać skalę zagrożeń dla harmonogramu, jakie wnosi praca zaplanowana w projekcie. Pomyślmy o ekstremalnym przykładzie: jeśli planujesz harmonogram projektu, to potrzebujesz mniejszego bufora dla zadań szacowanych od trzech do siedmiu niż dla zadań ocenionych od trzech do stu. Jest oczywiste, że jeśli w projekcie występują zadania ocenione na trzy punkty, które mogą się przekształcić w zadania stupunktowe, to bufor projektu powinien być większy niż wtedy, gdyby najgorszym przypadkiem dla tych zadań było dziesięć punktów. Na rozmiar bufora projektu wpływa zatem rozrzut między ocenami 50% a 90%¹.

¹ Ustalanie rozmiaru bufora w taki sposób zasugerowali wcześniej Reinertsen (1997), Newbold (1998) i Leach (2000).

Ponieważ nasze oceny ustalono dla każdej pozycji harmonogramu w punktach 50% i 90%, to różnica między tymi dwiema ocenami wynosi około dwóch odchyłeń standardowych. Odchylenie standardowe dla każdej pozycji wynosi zatem $(w_i - a_i)/2$, gdzie w_i reprezentuje najgorszy przypadek (ocenę 90%), natomiast a_i reprezentuje przypadek średni (ocenę 50%) dla tej samej historyjki. Chcielibyśmy, aby bufor projektu zabezpieczał cały projekt na tym samym poziomie 90% tak, aby każde zadanie było chronione przez własną ocenę 90%. Oznacza to, że bufor projektu powinien mieć rozmiar dwóch odchyłeń standardowych. Można go wyliczyć z następującego wzoru:

$$2\sigma = 2 \times \sqrt{\left((w_1 - a_1)/2\right)^2 + \left((w_2 - a_2)/2\right)^2 + \dots + \left((w_n - a_n)/2\right)^2}$$

gdzie σ oznacza odchylenie standardowe. Ten wzór można uprościć do następującej postaci:

$$2\sigma = \sqrt{(w_1 - a_1)^2 + (w_2 - a_2)^2 + \dots + (w_n - a_n)^2}$$

Przyjrzyjmy się, jak skorzystać z powyższego wzoru w celu ustalenia bufora harmonogramu. Załóżmy, że projekt obejmuje sześć historyjek użytkownika przedstawionych w tabeli 17.2 oraz że dla każdej historyjki wyświetlono oceny 50% i 90%. Oceny te mogą zostać zaprezentowane w punktach lub dniach idealnych. Wartości w ostatniej kolumnie z tabeli 17.2 obliczono przez odjęcie od oceny historyjki w najgorszym przypadku (90%) oceny tej samej historyjki w przypadku średnim (50%) i podniesienie tej wartości do kwadratu. Na przykład dla pierwszej historyjki mamy $(3 - 1)^2 = 4$. Bufor harmonogramu to pierwiastek kwadratowy z sumy tych kwadratów². W tym przypadku bufor harmonogramu ma wartość pierwiastka kwadratowego z 90, czyli 9,4. Wartość tę zaokrąglamy do 9. Całkowity czas trwania projektu jest sumą ocen 50% i bufora projektu, w tym przypadku wynosi zatem $17 + 9 = 26$.

Intuicyjnie bufor obliczony w tabeli 17.2 ma sens. Historyjka użytkownika, która ma największy wpływ na rozmiar bufora harmonogramu (pierwsza historyjka), jest jednocześnie historyjką o największym stopniu niepewności (różnica między ocenami 50% i 90% wynosi osiem). Z kolei historyjka o zerowej niepewności (ostatnia historyjka w tabeli) w ogóle nie ma wpływu na rozmiar bufora.

Wprowadzenie bufora harmonogramu może, lecz nie musi spowodować dodanie do projektu jednej lub kilku iteracji. Najczęściej zostaną one dodane. Załóżmy, że w tym przykładzie zespół prognozuje swoje tempo jako dziewięć punktów na iterację. Gdyby oceniono projekt na siedemnaście punktów (suma ocen 50%), to można by oczekiwać, że zespół zakończy projekt w dwóch iteracjach. Jeśli jednak uwzględnimy bufor projektu, to ocena dla całego projektu wyniesie dwadzieścia sześć punktów, a realizacja projektu, przy tempie równym dziewięć, potrwa trzy iteracje.

Prostsze obliczenia bufora

Opisane powyżej podejście do obliczeń rozmiaru bufora projektu to najlepszy sposób wyznaczania tego rozmiaru. Ale jeśli z jakiegoś powodu nie można ustalić ocen 50% i 90%, to można skorzystać z łatwiejszego sposobu ustalenia rozmiaru bufora projektu. Oceń każdą historyjkę na poziomie 50%, a następnie oszacuj rozmiar bufora jako połowa rozmiaru sumy ocen 50%. Należy pamiętać, że cały zespół musi być świadomy tego, że pewność ocen wynosi 50%. Trzeba wyznaczyć takie oceny, które z takim samym prawdopodobieństwem są za wysokie jak za niskie.

² Dlatego właśnie tę technikę ustalania rozmiaru bufora określa się powszechnie jako „pierwiastek kwadratowy z sumy kwadratów”.

TABELA 17.2. Obliczanie bufora projektu dla projektu obejmującego sześć historyjek

Historyjka użytkownika	Przypadek średni (50%)	Przypadek najgorszy (90%)	(najgorszy – średni) ²
Jako dowolny użytkownik serwisu chcę mieć możliwość oglądania rekordów życiowych każdego pływaka.	5	13	64
Jako dowolny użytkownik serwisu muszę przeprowadzić uwierzytelnianie, zanim uzyskam dostęp do jego poufnych części.	1	3	4
Jak pływak chcę mieć możliwość oglądania zaplanowanych zajęć treningowych.	3	5	4
Jako pływak lub rodzic chcę wiedzieć, gdzie są zlokalizowane baseny ligowe.	5	8	9
Jako dowolny użytkownik serwisu chcę móc oglądać krajowe rekordy według grup wiekowych i zawodów.	2	5	9
Jako dowolny użytkownik serwisu chcę móc oglądać wyniki wybranych zawodów.	1	1	0
Razem	17	35	90

Chociaż to obliczenie jest znacznie prostsze, to ma poważną wadę — nie wpływa na nie faktyczna niepewność dotycząca określonych historyjek użytkownika w projekcie. Załóżmy, że są dwie historyjki użytkownika — każda z nich jest szacowana na cztery punkty. Każda z tych historyjek w takim samym stopniu wpływa na rozmiar bufora projektu (połowa rozmiaru lub po dwa i pół punktu). Byłaby to prawda nawet wtedy, gdyby jedna z historyjek miała ocenę 90% wynoszącą sto, a ocena 90% pozostałych wynosiłaby dziesięć.

Z tych powodów oraz ze względu na to, że w czasie dokonywania oceny można z łatwością poprosić o podanie dwóch ocen, wolę podejście bazujące na pierwiastku kwadratowym z sumy kwadratów.

Wskazówki dotyczące bufora

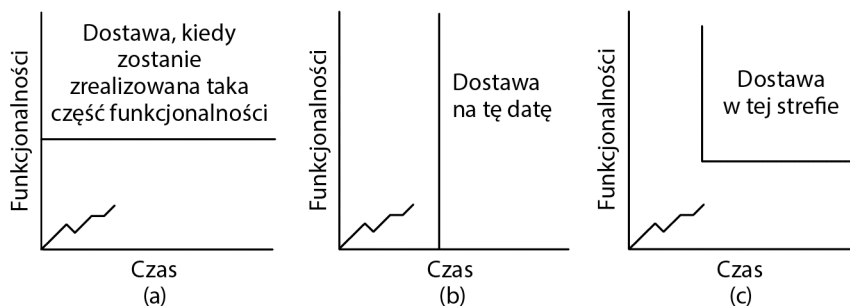
Niezależnie od tego, czy preferujemy podejście polegające na wyznaczaniu pierwiastka kwadratowego z sumy kwadratów, czy też podejście z ocenami 50%, należy wziąć pod uwagę wymienione poniżej wytyczne wynikające z porad wskazanych przez Leacha (2000).

- Obliczanie bufora jako pierwiastka kwadratowego z sumy kwadratów jest najbardziej wiarygodne, jeśli oceniamy co najmniej dziesięć historyjek użytkownika lub funkcjonalności. Jeśli jednak projekt zawiera mniej niż dziesięć pozycji, to prawdopodobnie nie powinniśmy uwzględniać w planowaniu bufora.
- Bufor projektu powinien stanowić nie mniej niż 20% całkowitego czasu trwania projektu. Bufor o mniejszych rozmiarach nie zapewni odpowiedniej ochrony dla całego projektu.

Łączenie buforów

Uwzględnianie wielu buforów może, na pierwszy rzut oka, być uznane za przesadę. Często jednak stosowanie wielu buforów jest zalecane, ponieważ w ten sposób zabezpieczamy projekt przed wieloma typami niepewności. Mój samochód ma zarówno pasy bezpieczeństwa, jak i poduszki powietrzne, ponieważ każdy z tych mechanizmów zabezpiecza mnie przed innego rodzaju zderzeniami. Określony typ niepewności w projekcie zawsze należy buforować z wykorzystaniem właściwych jednostek. Oznacza to, że trzeba buforować niepewność funkcjonalności, wyrażając ją za pomocą funkcjonalności, natomiast niepewność harmonogramu za pomocą czasu. Ponadto, w przypadku korzystania z wielu buforów, rozmiar każdego z nich może być mniejszy.

Projekty, w których uwzględniamy zarówno bufor funkcjonalności, jak i harmonogramu, stają się rzeczywiście zabezpieczone przed niepewnością. Rozważmy trzy projekty pokazane na rysunku 17.5. Na rysunku (a) przedstawiono projekt, który powinien dostarczyć określony zestaw funkcjonalności, ale dopuszcza zmiany w harmonogramie. Rysunek 17.5 (b) pokazuje odwrotny projekt: jego data jest ustalona, ale istnieje pełna elastyczność co do tworzonych funkcjonalności. Przyjrzyjmy się teraz rysunkowi 17.5 (c). Można zauważyć, że dzięki uwzględnieniu zarówno bufora funkcjonalności, jak i harmonogramu zespół może podjąć zobowiązanie zarówno co do daty dostawy, jak i minimalnego zestawu funkcjonalności. Podczas tworzenia planu wydania naszym celem jest takie wykorzystywanie buforów, aby zespół mógł podejmować tego typu zobowiązania.



RYSUNEK 17.5. Trzy projekty z różnym podejściem do buforowania

Poza tym trzeba pamiętać, że w projekcie oprócz funkcjonalności i harmonogramu mogą być wykorzystywane także inne bufony. Projekt może uwzględniać *bufor budżetu*, kiedy na przykład do realizacji projektu zatrudniono trzydziestu deweloperów, podczas gdy budżet pozwala na zatrudnienie do trzydziestu trzech. Jest to standardowa praktyka w średnich i dużych projektach, ale w mniejszych jest stosowana rzadziej z dwóch powodów:

1. Dodatkowa jedna lub dwie osoby, które tworzą personalny bufor w małym projekcie, prawie na pewno mogą wnieść bezpośredni wkład w projekt. Zastosowanie kompletnej obsady etatowej zwiększonej z trzydziestu do trzydziestu trzech deweloperów może przynieść niewielki zysk lub nie przynieść żadnego zysku. Jednak zysk wydajności zaobserwujemy prawie na pewno przy zmianie obsady etatowej z czterech deweloperów na pięciu.
2. W małych ilościach trudno buforować cokolwiek. Gdy w projekcie, w którym bierze udział trzydziestu osób, występuje trzyosobowy bufor do pełnej obsady trzydziestu trzech osób,

to bufor personelu wynosi 10%. Podobny bufor w projekcie trzysobowym oznaczałby 3/10 dewelopera. Z oczywistych względów łatwiej dodawać do projektu całe osoby niż ułamki.

Bufor harmonogramu to nie wypełnienie

Termin *wypełnienie* (ang. *padding*) ma pejoratywne znaczenie opisujące czas losowo dodany do oceny. Wypełniam ocenę wtedy, gdy myślę, że wykonanie zadania zajmie trzy dni, ale tak na wszelki wypadek podaję ocenę pięć. Niektórzy dodają wypełnienia do ocen w sytuacji, gdy oczekują dotkliwych kar, gdy się pomylą. Bufor harmonogramu odgrywa inną rolę: jest niezbędnym marginesem bezpieczeństwa dodanym do sumy ocen, z których usunięto lokalne zabezpieczenia.

Kiedy pozostawiasz pięć długości samochodu między swoim pojazdem a tym, który jedzie przed Tobą, robisz to, ponieważ oczekujesz, że wykorzystasz większość tego bufora, gdy będziesz zmuszony do gwałtownego hamowania. To prawda, że istnieje możliwość jazdy przez wiele godzin z odstępem wynoszącym jedną długość samochodu, ale to jest mało prawdopodobne. Bufor wokół samochodu ma kluczowe znaczenie dla Twojego bezpieczeństwa. Odpowiednie bufor w projekcie mają kluczowe znaczenie dla bezpieczeństwa Twojego projektu.

Kiedy zezwalamy na niewielką elastyczność zarówno co do daty dostawy, jak i funkcjonalności, możemy zastosować bufor dla dwóch wymiarów projektu. Co ważniejsze, buforujemy każde ograniczenie projektu z wykorzystaniem odpowiedniego zasobu: termin buforujemy z wykorzystaniem czasu, natomiast funkcjonalność za pomocą funkcjonalności. Jeśli nie możemy właściwie buforować określonego ograniczenia, jesteśmy zmuszeni zwiększyć rozmiar innych buforów. Jeśli jestem zmuszony do zagwarantowania funkcjonalności, będę wspierać tę gwarancję większym buforem harmonogramu.

Ostrzeżenia

Choć umiejętność dodawania jednego lub kilku buforów do projektu jest ważna i warto z niej korzystać, to trzeba również pamiętać o pewnych ostrzeżeniach dotyczących jej użycia.

- Przy dodawaniu bufora harmonogramu należy stosować opisane w rozdziale podejście z dwoma ocenami albo zadbać o to, aby pojedyncze oceny dotyczyły 50% pewności. Dodanie bufora harmonogramu do już pesymistycznej oceny 90% doprowadzi do zbyt długotrwałego harmonogramu.
- W wielu projektach nie jest potrzebny dokładny termin razem ze ściśle ustalonym zakresem dostarczanych funkcjonalności. Zamiast tego zespół po prostu musi jak najszybciej dostarczyć wysokiej jakości oprogramowanie w pewnym przedziale czasu. Jeśli jesteś w takiej sytuacji, nie sil się na wprowadzanie do projektu dodatkowych buforów.
- Uważaj, w jaki sposób informujesz członków zespołu o buforach. Nie należy ukrywać ich istnienia ani też sposobu ich wykorzystywania. Bufor (zwłaszcza bufor harmonogramu) może jednak sprawiać wrażenie wypełnienia. Oznacza to, że powinieneś informować, jak zostały ustalone oceny i bufor oraz w jaki sposób bufor zapewnia wszystkim utworzenie harmonogramu, którego wszyscy będą pewni.

Podsumowanie

Większość projektów zawiera ogromną ilość niepewności. Ta niepewność często nie znajduje pełnego uzasadnienia w harmonogramach i terminach, które tworzy zespół projektowy. Zdarzają się sytuacje, kiedy niepewność jest tak duża lub znacząca, że należy podjąć specjalne działania podczas oceny trwania projektu. Może tak być w przypadku, gdy projekt jest planowany z dużym wyprzedzeniem, należy bezwzględnie dotrzymać terminu dostarczenia ustalonego zbioru funkcjonalności, wykonanie projektu zostało zlecone firmie zewnętrznej, wymagania są znane dość powierzchownie albo istnieją poważne skutki (finansowe lub inne) wynikające z niedotrzymania terminu.

Dwa najbardziej popularne rodzaje buforów to bufor funkcjonalności i bufor harmonogramu. Bufor funkcjonalności tworzy się w przypadku, gdy dla wymagań dla produktu są ustalone priorytety oraz gdy ustalono, że nie muszą być dostarczone wszystkie funkcjonalności. Na przykład dla procesu Agile DSDM zaleca się, aby 30% wysiłków związanych z projektem było opcjonalne, co tworzy dla projektu bufor funkcjonalności. Gdy zaczyna brakować czasu, nadal można dotrzymać harmonogramu poprzez usunięcie pozycji z bufora funkcjonalności.

Z kolei bufor harmonogramu tworzy się poprzez wzięcie pod uwagę w harmonogramie marginesu czasu uwzględniającego niepewność wynikającą z rozmiaru zespołu. Bufor funkcjonalności można zbudować poprzez oszacowanie dla każdej historyjki użytkownika rozmiaru z prawdopodobieństwem 50% oraz rozmiaru z prawdopodobieństwem 90%. Dzięki obliczeniu pierwiastka kwadratowego z sumy kwadratów dla wszystkich ocen 50% i 90% można oszacować bufor harmonogramu o odpowiednim rozmiarze.

Przed niepewnością funkcjonalności w projekcie należy zabezpieczyć się za pomocą bufora funkcjonalności, natomiast przed niepewnością harmonogramu — za pomocą bufora harmonogramu. Bufor funkcjonalności może być połączony z buforem harmonogramu. Zazwyczaj jest to dobry pomysł, ponieważ dzięki temu rozmiar każdego bufora może być mniejszy.

Pytania do dyskusji

1. Czy w Twojej firmie istnieją warunki pozwalające na osiągnięcie korzyści z dodatkowych wysiłków związanych z wyznaczeniem bufora harmonogramu?
2. Czy w Twoim bieżącym projekcie wyznaczono bufor zabezpieczający przed wszystkimi rodzajami niepewności? Jeśli tak, to w jaki sposób? Jeśli nie, to jakie rodzaje buforów byłyby najbardziej korzystne?
3. Czy w swojej firmie obserwujesz dowody słuszności prawa Parkinsona lub syndromu studenta? Jakie mechanizmy można by zastosować, oprócz sugestii zaprezentowanych w tym rozdziale, aby zminimalizować ich wpływ na projekt?

ROZDZIAŁ 18.

Planowanie projektu z udziałem wielu zespołów

Przeprowadzaj planowanie, ale wyrzuć plany.

— Mary Poppendieck

Zespoły Agile są opisywane jako grupy złożone z nie więcej niż siedmiu do dziesięciu deweloperów. Zespoły o takim rozmiarze mogą zrobić całkiem sporo, zwłaszcza wtedy, gdy stosują proces Agile, który pozwala na poprawę wydajności i zachęca je do działań z tym związanych. Jednakże realizacja niektórych projektów może być łatwiejsza dzięki zaangażowaniu większego zespołu. Zgodnie z podejściem Agile w takiej sytuacji zamiast tworzyć jednolity 100-osobowy zespół, lepiej stworzyć kilka mniejszych zespołów. W projekcie Agile zamiast jednego 100-osobowego zespołu może uczestniczyć kilkanaście mniejszych zespołów.

W tym rozdziale zbierzemy wiadomości, z którymi zapoznaliśmy się w poprzednich rozdziałach, i zastosujemy je do planowania projektu, w którym uczestniczy wiele zespołów. Planowanie dużego projektu, w którym bierze udział kilka zespołów, może wymagać wykonania kilku czynności, którymi są:

1. Ustalenie wspólnej podstawy do ocen.
2. Wcześniejsze wprowadzenie szczegółów do historyjek użytkownika.
3. Stosowanie planowania wyprzedzającego (ang. *lookahead planning*).
4. Uwzględnienie w planach buforów zasilających (ang. *feeding buffers*).

Taki projekt może wymagać niektórych lub wszystkich wymienionych powyżej metod. Wszystko zależy od liczby zespołów składowych uczestniczących w projekcie oraz częstotliwości i intensywności koordynowania swoich działań. Ogólnie rzecz biorąc, zalecam zespołom stosowanie tylko tyłu spośród wymienionych dodatkowych technik, ile jest niezbędne — począwszy od ustalenia wspólnej podstawy planowania, a skończywszy na zastosowaniu buforów zasilających.

Ustalenie wspólnej podstawy do ocen

Chociaż byłoby miło, gdyby każdy zespół składowy miał możliwość wyboru, czy będzie dokonywał ocen w punktach, czy w dniach idealnych, to w większości projektów, w których uczestniczy wiele zespołów, korzystne jest dokonywanie ocen z wykorzystaniem wspólnej jednostki i ustalenie jej podstawowego znaczenia. Wyobraźmy sobie, jak trudno byłoby przewidzieć, o ile przedłuży się realizacja zbioru historyjek użytkownika, gdyby niektóre z nich były oszacowane w dniach idealnych, a inne w punktach. Co gorsza, wyobraźmy sobie, o ile trudniej byłoby wtedy, gdyby jeden zespół oszacował zbiór historyjek na dwadzieścia punktów, a inny oszacowałby tę samą pracę na sto punktów.

Na początku projektu zespoły powinny się ze sobą spotkać i zdecydować, czy oceny będą wykonywane w punktach, czy w dniach idealnych. Następnie powinny stworzyć wspólną bazę dla swoich ocen tak, aby ocena dokonana przez jeden zespół była podobna do oceny dokonanej przez inny. Każda historyjka użytkownika powinna być oceniana tylko przez jeden zespół, ale oceny powinny być sobie równoważne niezależnie od tego, który zespół dokonywał oceny.

Są dwa dobre sposoby utworzenia wspólnej podstawy oceny. Pierwsze podejście sprawdza się tylko wtedy, kiedy w przeszłości zespoły pracowały ze sobą nad innym projektem. W takiej sytuacji zespoły mogą wybrać kilka historyjek użytkownika z poprzedniego projektu i uzgodnić dla nich oceny. Przypuśćmy, że zespoły oceniają w dniach idealnych. W takim przypadku powinny znaleźć dwie lub trzy historyjki, z których każda została oceniona na jeden dzień idealny. Następnie powinny znaleźć kilka historyjek, które oceniają na dwa dni idealne i tak dalej. Mogą zidentyfikować dwadzieścia lub więcej starych historyjek, uzgodnić dla każdej z nich nową ocenę, uwzględniając bieżącą wiedzę na ich temat. Po uzgodnieniu podstawowych historyjek zespoły mogą osobno ocenić historyjki poprzez porównanie ich z historyjkami bazowymi (czyli ocenić je przez analogię).

Drugie podejście jest podobne, ale uwzględnia dokonanie wspólnej oceny zbioru nowych historyjek użytkownika. Zespoły wybierają zbiór historyjek planowanych dla nowego wydania. Historyjki powinny mieć różne rozmiary oraz dotyczyć obszarów systemu, do których może się odnieść większość oceniających. Następnie cały duży zespół albo jeśli cały zespół jest za duży, przedstawiciele każdego zespołu składowego spotykają się i uzgadniają oceny dla tych historyjek. Podobnie jak w przypadku pierwszego podejścia, oceny te są następnie wykorzystywane jako podstawa do porównań z przyszłymi ocenami.

Jedynym przypadkiem, kiedy pojedyncze zespoły powinny rozważyć ocenianie w różnych jednostkach, bez wspólnej podstawy, jest sytuacja, kiedy tworzone produkty są rzeczywiście oddzielne i nie ma możliwości, aby deweloperzy z jednego zespołu mogli przechodzić do innego. Nawet wtedy jednak zalecam ustalenie wspólnej podstawy, ponieważ jej istnienie ułatwia komunikację w projekcie.

Wcześniejsze wprowadzanie szczegółów do historyjek użytkownika

W idealnej sytuacji zespół Agile rozpoczyna iterację z niedokładnie zdefiniowanymi wymaganiami, a następnie do końca iteracji przekształca te niewyraźne wymagania w działające i przetestowane oprogramowanie. Przejście od nieprecyzyjnie sformułowanego wymagania do działającego programu w jednej iteracji jest zazwyczaj łatwiejsze w projekcie, nad którym pracuje jeden zespół, niż wtedy, gdy zespołów jest kilka. W projekcie, w którym pracuje wiele zespołów, często wskazane i konieczne

jest dokładniejsze przemyślenie historyjek użytkownika przed rozpoczęciem iteracji. Dodatkowe szczegóły pozwalają wielu zespołom na skoordynowanie swojej pracy.

Aby osiągnąć ten cel, w większych zespołach często pracują dedykowani analitycy, projektanci interakcji z użytkownikiem oraz inne osoby, które poświęcają część swojego czasu w danej iteracji na przygotowanie pracy w następnej iteracji. Ogólnie rzecz biorąc, nie zalecam analitykom, projektantom interakcji i innym osobom w pełni angażować się w pracę nad przyszłą iteracją. Zamiast tego ich głównym obowiązkiem powinna być praca nad bieżącą iteracją, jednak podczas planowania bieżącej iteracji powinni uwzględniać pewne zadania związane z przygotowaniem się do następnej iteracji.

Najbardziej korzystnym efektem pracy przed iteracją jest, w mojej ocenie, identyfikacja warunków satysfakcji właściciela produktu dla historyjek użytkownika, które prawdopodobnie będą implementowane podczas iteracji. Warunki satysfakcji właściciela produktu dla historyjki użytkownika to wysokopoziomowe testy akceptacyjne, które właściciel produktu poleca wykonać dla oprogramowania implementującego historyjkę, zanim uzna ją za zrealizowaną. Historyjka użytkownika kończy się, kiedy można udowodnić, że spełnia wszystkie warunki satysfakcji ustalone przez właściciela produktu.

Chociaż znajomość warunków satysfakcji dla historyjki użytkownika przed rozpoczęciem iteracji jest bardzo przydatna, to jest mało prawdopodobne (i niepotrzebne), aby zespół zidentyfikował je dla wszystkich historyjek użytkownika przed rozpoczęciem iteracji. Patrząc realnie, dokładny zbiór historyjek użytkownika, które będą implementowane w następnej iteracji, nie jest znany aż do zakończenia spotkania poświęconego planowaniu iteracji, które rozpoczyna tę iterację. Jednak w większości przypadków właściciel produktu wraz z zespołem może rozsądnie odgadnąć te historyjki, które w następnej iteracji najprawdopodobniej będą miały wysoki priorytet. Dla tych historyjek użytkownika warto zidentyfikować warunki satysfakcji przed rozpoczęciem iteracji.

Planowanie wyprzedzające

Dla większości zespołów, w których występują umiarkowanie złożone lub częste zależności, korzystne będzie podczas planowania wydania i iteracji utrzymywanie ruchomego okna podglądu (ang. *lookahead window*). Przypuśćmy, że nad aplikacją SwimStats pracują dwa zespoły. Część funkcjonalności w aplikacji SwimStats obejmuje wyświetlanie informacji statycznych, takich jak godziny treningów, adresy i wskazówki dojazdu do basenów. Aplikacja SwimStats powinna jednak również dostarczać informacji dynamicznych, pochodzących z bazy danych, takich jak wyniki wszystkich zawodów w ciągu ostatnich piętnastu lat czy rekordów życiowych pływaków we wszystkich zawodach.

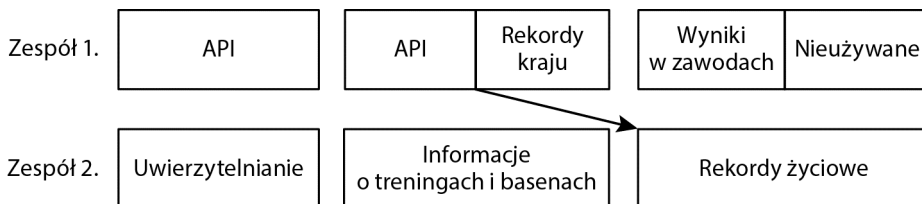
Rekordy kraju oraz grup wiekowych są przechowywane w bazie danych na zdalnym serwerze utrzymywanym przez krajowy związek pływacki. Dostęp do bazy danych nie jest tak prosty, jak chciałby tego zespół, a krajowy związek pływacki w ciągu roku lub dwóch planuje zmienić dostawcę bazy danych. Z tych powodów właściciel produktu i zespoły projektowe uzgodniły, że opracują API (interfejs programowania aplikacji), który umożliwi uzyskanie dostępu do bazy danych. Dzięki interfejsowi API późniejsza zmiana na innego dostawcę bazy danych będzie znacznie prostsza. Wstępne historyjki użytkownika i oceny dla każdej z nich zestawiono w tabeli 18.1.

Szacowane tempo każdego zespołu wynosi dwadzieścia punktów na iterację. Ponieważ praca jest oceniona na sto dziesięć punktów, to zespoły powinny dostarczyć całą funkcjonalność w trzech iteracjach. Jednak trzydzieści punktów dotyczy opracowania API, a kolejnych czterdzieści (ostatnie trzy historyjki z tabeli 18.1) może być wykonane tylko po utworzeniu interfejsu API. W celu zrealizowania

TABELA 18.1. Wstępne historyjki użytkownika i oceny dla aplikacji SwimStats

Historyjka użytkownika	Ocena w punktach
Jako członkowie zespołu SwimStats chcemy mieć możliwość łatwej zmiany dostawcy bazy danych.	30
Jako dowolny użytkownik serwisu muszę przeprowadzić uwierzytelnianie, zanim uzyskam dostęp do jego poufnych części.	20
Jako pływak chcę mieć możliwość oglądania zaplanowanych zajęć treningowych.	10
Jako pływak lub rodzic chcę wiedzieć, gdzie są zlokalizowane baseny ligowe.	10
Jako dowolny użytkownik serwisu chcę móc oglądać krajowe rekordy według grup wiekowych i zawodów.	10
Jako dowolny użytkownik serwisu chcę móc oglądać wyniki wybranych zawodów.	10
Jako dowolny użytkownik serwisu chcę mieć możliwość oglądania rekordów życiowych każdego pływaka.	20

tych siedemdziesięciu punktów w trzech iteracjach oba zespoły muszą skorzystać z API. Te dane doprowadziły do podziału pracy pokazanego na rysunku 18.1. Współzależności między zespołami przedstawiono na nim w postaci strzałki między pracą nad API wykonywaną przez pierwszy zespół a pracą nad rekordami życiowymi wykonywaną przez drugi zespół.

**RYСУNEK 18.1.** Koordynacja pracy dwóch zespołów

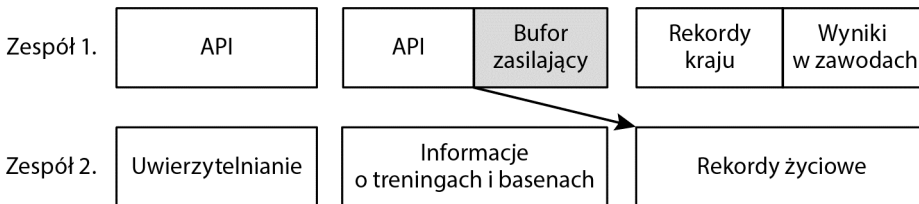
Jak pamiętamy z rozdziału 13. „Podstawy planowania wydań”, zalecałem, aby plan wydania zawierał szczegóły tylko kilku najbliższych iteracji. To dlatego, że często tyle wystarczy do uwzględnienia współzależności między wieloma zespołami. Jeśli trzeba skoordynować pracę kilku zespołów, to należy zaktualizować plan wydania w taki sposób, aby pozwalał na zaprezentowanie i skoordynowanie pracy podczas następnych dwóch lub trzech iteracji. Dokładna liczba iteracji oczywiście zależy od częstotliwości i znaczenia zależności między zespołami. Po zrealizowaniu iteracji szczegółowe informacje na ich temat są usuwane z planu. W takiej sytuacji plan wydania staje się *ruchomym planem wyprzedzającym* (ang. *rolling lookahead plan*), w którym zawsze są wyszczególnione oczekiwania dotyczące kilku kolejnych iteracji. Laufer (1996) opisuje ten mechanizm jako „zagłądanie w przyszłość”.

Na rysunku 18.1 pokazano sytuację, w której przekazywanie pracy między zespołami następuje między iteracjami. Jest to bezpieczniejsze niż przekazywanie pracy w trakcie iteracji. Na początku każdej iteracji każdy z zespołów definiuje pracę, którą może wykonać, a następnie zobowiązuje się do jej wykonania. W sytuacji pokazanej na rysunku 18.1 na początku trzeciej iteracji drugi zespół

zobowiązał się do zrealizowania historyjki dotyczącej rekordów życiowych, ponieważ wiedział, że prace nad API zostały zakończone. Załóżmy, że zamiast tego, gdy drugi zespół planował swoją trzecią iterację, API nie było jeszcze zaimplementowane, ale oczekiwano, że zostanie ukończone w ciągu kilku dni. Nawet gdyby drugi zespół mógł zakończyć pracę nad historyjką dotyczącą rekordów życiowych bez posiadania API już pierwszego dnia, to byłoby to znacznie trudniejsze zobowiązanie, które stwarzałoby większe zagrożenie dla całego harmonogramu. Zespoły często muszą podejmować zobowiązania dotyczące artefaktów, które muszą być dostarczone w środku iteracji. Jednak w miarę możliwości powinny one ograniczać zobowiązania do prac wykonanych przed rozpoczęciem iteracji.

Wykorzystywanie w planie buforów zasilających

Dla większości zespołów w większości przypadków właściwy jest ruchomy plan wyprzedzający. Są jednak sytuacje, w których współzależności między zespołami są tak złożone lub częste, że proste ruchome planowanie wyprzedzające opisane w poprzednim podrozdziale nie wystarcza. W takich sytuacjach należy najpierw spróbować znaleźć sposób zmniejszenia liczby współzależności tak, by zastosowanie ruchomego planu wyprzedzającego było właściwe. Jeśli nie można tego zrobić, zastanów się nad wprowadzeniem podczas iteracji bufora zasilającego, dostarczającego funkcjonalności wymagane przez inne zespoły. Bufor zasilający, podobnie jak bufor harmonogramu opisany w poprzednim rozdziale, zapewnia dostarczenie na czas zestawu nowych funkcjonalności. Jest to dość skomplikowany sposób na to, aby powiedzieć, że jeśli Twój zespół potrzebuje czegoś od mojego zespołu jutro na 8.00 rano, to mój zespół nie powinien planować ukończenia tej pracy o 7.59. Zatem potrzebny jest plan podobny do tego, który pokazano na rysunku 18.2.



RYСУNEK 18.2. Dodawanie bufora zasilającego

W sytuacji pokazanej na rysunku 18.2 bufor zasilający wstawiono między zakończeniem przez pierwszy zespół zadania związanego z przygotowaniem API a rozpoczęciem przez drugi zespół pracy nad historyjką użytkownika dotyczącą rekordów życiowych. Bufor zasilający zabezpiecza datę rozpoczęcia pracy nad historyjką dotyczącą rekordów życiowych przed opóźnieniami prac nad tworzeniem API.

Wszystko, co jest potrzebne do uwzględnienia bufora zasilającego w planie wydania, to tymczasowe planowanie z uwzględnieniem niższego tempa dla zespołu dostarczającego określone własności innemu zespołowi. W przypadku pokazanym na rysunku 18.2 wysiłek pierwszego zespołu jest równomiernie podzielony między zakończenie prac nad API a buforem zasilającym. Dzięki temu możemy mieć pewność, że drugi zespół zdoła rozpocząć pracę nad historyjką użytkownika dotyczącą rekordów życiowych na początku trzeciej iteracji. To nie oznacza, że pierwszy zespół ma mniej pracy w drugiej iteracji. Oczekuje się raczej, że zespół ten zrealizuje dziesięć punktów pracy nad API, zużyje pewną część bufora, a następnie, podczas drugiej iteracji, rozpocznie pracę nad historyjką użytkownika dotyczącą rekordów życiowych.

Co jest buforowane?

W przypadku pokazanym na rysunku 18.2 dodanie bufora zasilającego nie wydłużyło całkowitego czasu trwania projektu, ponieważ trzecia iteracja pierwszego zespołu nie była jeszcze pełna. Jednak w wielu sytuacjach dodanie buforów zasilających powoduje wydłużenie oczekiwanego czasu trwania projektu. Zazwyczaj jednak dzieje się to w wyniku realnych oczekiwań w stosunku do prawdopodobnego harmonogramu, nie jest zaś reprezentacją poglądu w rodzaju „wypełnijmy harmonogram tak, aby nie trzeba było ciężko pracować”. Ponieważ stosowanie buforów zasilających może doprowadzić do wydłużenia harmonogramów, należy je dodawać tylko wtedy, kiedy jest to konieczne.

Aby stwierdzić, czy bufor zasilający jest potrzebny, należy najpierw rozdzielić historię użytkownika między zespoły i iteracje. Następnie trzeba poszukać kluczowych zależności między iteracjami a zespołami. Na koniec należy wprowadzić bufor zasilający tylko między tymi krytycznymi zależnościami. Oznacza to, że bufor zasilający należy dodać tylko wtedy, kiedy zespół nie jest w stanie wykonać zaplanowanej pracy o wysokim priorytecie bez wyników pracy innych zespołów. Nawet jeśli tak jest, to jeśli zespół jest w stanie przełączyć się na inną, wartościową pracę, bufor zasilający jest niepotrzebny. Na podobnej zasadzie nie należy dodawać bufora zasilającego w przypadku, gdy drugi zespół jest w stanie robić postępy po otrzymaniu od pierwszego zespołu częściowo wykonanych artefaktów. Jeśli Twój zespół może rozpocząć pracę nawet wtedy, kiedy mój zespół dostarczył tylko połowę zaplanowanej funkcjonalności, to bufor zasilający jest niepotrzebny.

Ustalanie rozmiaru bufora zasilającego

W celu określenia rozmiaru bufora zasilającego można zastosować wytyczne podane w rozdziale 17. „Plany z buforami w celu uwzględnienia niepewności”. Na szczęście jednak większość współzależności między zespołami bazuje na nie więcej niż kilku historijkach bądź funkcjonalnościach. Z tego powodu zazwyczaj nie dysponujemy odpowiednią liczbą historyjek, aby skutecznie zastosować opisane w tamtym rozdziale podejście bazujące na pierwiastku kwadratowym z sumy kwadratów. W takich przypadkach należy ustawić rozmiar bufora jako procent historyjek tworzących współzależność. W roli domyślnego rozmiaru bufora można użyć 50%, ale wartość tę należy skorygować na podstawie ocen zespołu.

Istnieje możliwość stosowania bufora zasilającego, który jest dłuższy od pełnej iteracji. Rzadko jednak jest zalecane stosowanie tak dużego bufora. Bufor zasilający, który jest dłuższy niż iteracja, zwykle jest wynikiem planowania zmierzającego do przekazania dużej części funkcjonalności innemu zespołowi. Są dwa powody, dla których w takich sytuacjach projekt prawdopodobnie nie wymaga dużego bufora zasilającego. Po pierwsze, przekazanie funkcjonalności między zespoły prawie na pewno można tak podzielić, aby funkcjonalność była dostarczana stopniowo. Dzięki temu drugi zespół może rozpocząć pracę natychmiast po otrzymaniu od pierwszego zespołu początkowego zbioru funkcjonalności. Po drugie, zamiast zużywać bardzo duży bufor zasilający, zespoły mogą znaleźć sposób na podzielenie pracy lub wprowadzenie innych korekt między iteracjami natychmiast po zauważeniu zaległości w drugim zespole. Spełnianie przez zespół otrzymujący roli właściciela produktu lub klienta zespołu dostarczającego zazwyczaj pozwala opracować obu zespołom przyrostową sekwencję dostaw, która sprawdza się dla tych zespołów.

Nigdy nie musiałem stosować bufora zasilającego, który byłby dłuższy niż pełna iteracja, i rzadko stosowałem taki, którego czas trwania przekraczałby więcej niż połowę iteracji. Za każdym razem, gdy

zaistnieje taka potrzeba, zadaję sobie pytanie o słuszność swoich założeń i zastanawiam się nad możliwością podjęcia działań, które skrócą łańcuch produktów przekazywanych między zespołami.

Ale to tyle pracy...

To prawda, lecz w dużym projekcie obejmującym wiele zespołów jest to normalne. Należy pamiętać, że nie trzeba podejmować tego rodzaju działań w projektach, w których uczestniczy tylko jeden zespół. Prawdopodobnie nie trzeba tego robić także wtedy, gdy w projekcie biorą udział tylko trzy lub cztery mniej więcej siedmioosobowe zespoły pod warunkiem, że często się ze sobą komunikują.

W wielu dużych projektach jest jednak potrzeba ogłaszania terminów i podejmowania zobowiązań na wiele miesięcy do przodu. W wielu dużych projektach występują również liczne współzależności między zespołami podobne do tych, które przedstawiono w tym rozdziale. W przypadku gdy pojawi się taki projekt, warto poświęcić kilka dodatkowych godzin na planowanie. Dzięki temu można pewniej i dokładniej oszacować docelową datę ukończenia projektu, a także uzyskać rodzaj zabezpieczenia przed opóźnieniami w harmonogramie, których można łatwo uniknąć.

Podsumowanie

W projektach Agile unika się dużych zespołów. Zamiast tego duże projekty realizują zespoły złożone z mniejszych zespołów. Gdy nad jednym projektem pracuje kilka zespołów, istnieje potrzeba skoordynowania ich pracy. W tym rozdziale opisano cztery techniki, które mogą się przydać w sytuacji, gdy nad jednym projektem pracuje wiele zespołów.

Po pierwsze, zespoły powinny ustalić wspólną podstawę swoich ocen. Wszystkie zespoły powinny szacować pracę za pomocą tej samej jednostki: punktów lub dni idealnych. Powinny również ustalić znaczenie tych jednostek poprzez uzgodnienie ocen dla niewielkiego zbioru historyjek.

Po drugie, jeśli musi ze sobą pracować kilka zespołów, to często przydatne jest wcześniejsze dodawanie szczegółów do ich historyjek użytkownika. Najlepszym sposobem, aby to zrobić, jest zidentyfikowanie dla historyjki warunków satysfakcji właściciela produktu. Są to warunki, których prawdziwości można dowiedzieć w sytuacji, gdy historyjka zostanie w pełni zaimplementowana.

Po trzecie, wiele zespołów może skorzystać na zastosowaniu w procesie planowania wydań ruchomego planu wyprzedzającego. Ruchomy plan wyprzedzający po prostu sięga o kilka iteracji do przodu (z reguły dwie lub trzy) i pozwala zespołom na skoordynowanie pracy dzięki wymianie informacji o zadaniach, jakie będą realizowały poszczególne zespoły w najbliższej przyszłości.

Po czwarte, w bardzo złożonych projektach, z dużą liczbą współzależności między zespołami, może być sensowne włączenie do planu buforów zasilających. Bufor zasilający to pewien odcinek czasu zapobiegający sytuacjom, w których opóźnienia w dostarczaniu pracy przez jeden zespół powodują opóźnienia w rozpoczęciu pracy przez inny zespół.

Opisane techniki, ogólnie rzecz biorąc, są wprowadzane w projekcie w opisanej kolejności. Równie dobrze mogą być jednak wprowadzane w dowolnym porządku.

Pytania do dyskusji

1. W jaki sposób rozwiązałbyś problem ustalenia wspólnej podstawy ocen w projekcie z wieloma zespołami?
2. Jak istotne są współzależności między zespołami w Twoim projekcie? Zastosowanie której z technik zaproponowanych w tym rozdziale mogłoby przynieść największe korzyści dla projektu?

CZĘŚĆ V

Śledzenie i komunikacja

W części IV tej książki opisano sposób ustalania odpowiedniego harmonogramu dla projektu. Planowanie nie kończy się po utworzeniu planu i harmonogramu. Ważne jest śledzenie postępów w realizacji planu, komunikowanie jego postępów, a następnie — na podstawie obserwacji — korygowanie planów.

W dwóch pierwszych rozdziałach tej części najpierw omówimy sposoby monitorowania planu wydania, a następnie opiszemy, jak monitorować plan iteracji. Na zakończenie tej części przedstawimy różne sposoby komunikowania ocen, planów i postępów. Dołączono również przykład podsumowującego raportu wykonywanego na koniec iteracji.

ROZDZIAŁ 19.

Monitorowanie planu wydania

Gwiazdy czasami kłamią, ale liczby nigdy.

— Mary Chapin Carpenter, *I Feel Lucky*

Starożytni żeglarze mieli dwa problemy. Po pierwsze, musieli znać swoją szerokość geograficzną — czyli pozycję na mapie w kierunku północ-południe. Po drugie, musieli znać swoją długość geograficzną, czyli położenie w kierunku wschód-zachód. Określenie szerokości geograficznej poprzez obserwację Gwiazdy Polarnej było stosunkowo łatwe. Robiono to już około 300 roku przed naszą erą. Wyznaczenie długości geograficznej stwarzało większe problemy, ponieważ wymagało użycia stosunkowo dokładnego zegara lub stopera. Niestety do początku XVIII wieku nie było wystarczająco precyzyjnych zegarów (zwłaszcza takich, które zmieściłyby się na pokładzie statku).

Przed wynalezieniem chronometru żeglarze mogli jedynie odgadywać swoją długość geograficzną. W celu jej ustalenia stosowali szereg oszacowań i korekt znanych jako tzw. nawigacja zliczeniowa (ang. *dead reckoning* lub *deduced reckoning*). Nawigacja zliczeniowa polega na oszacowaniu, jak daleko na wschód lub na zachód odpłynął statek, a następnie na skorygowaniu tego oszacowania za pomocą współczynnika wiatru i dryfu. Załóżmy, że prędkościomierz na jachcie mówi nam, że płyniemy osiem mil na godzinę. Jeśli jednak natrafisz na fale, wiatr lub prąd morski, to przy tej prędkości możesz posuwać się zaledwie pięć mil na godzinę. Oszacowania w technice nawigacji zliczeniowej powinny uwzględniać wszystkie te czynniki. W przeciwnym razie ocena długości geograficznej byłaby nieprawidłowa.

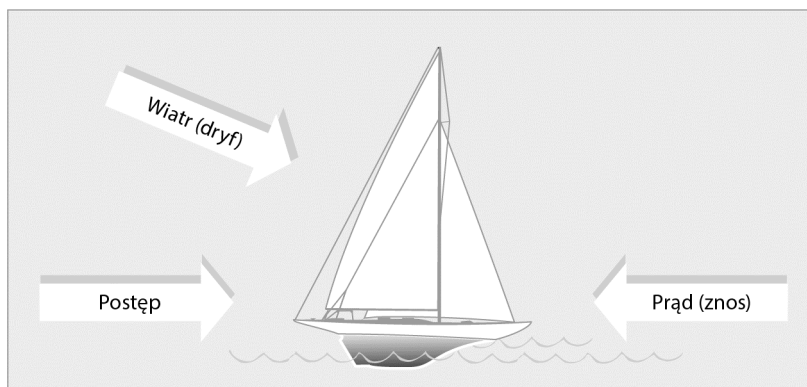
Śledzenie postępów zespołu pracującego nad oprogramowaniem jest bardzo podobne do wykreślenia pozycji statku — zwłaszcza poprzez stosowanie nawigacji zliczeniowej. Chcielibyśmy tak sterować projektem wytwarzania oprogramowania, aby płynął prostą drogą z punktu A do punktu B. To jednak zdarza się rzadko choćby dlatego, że wymagania zmieniają się lub są uściślane, nasze tempo pracy nie zawsze jest takie, jakiego oczekiwaliśmy, a czasami popełniamy błędy w sposobie pomiaru naszej pozycji. W podrozdziałach tego rozdziału opisano metody śledzenia postępów. Ich stosowanie pozwala zminimalizować wpływ tych i podobnych problemów.

Śledzenie wydania

Na początku wydania tworzymy plan, który mówi coś w stylu „W ciągu najbliższych czterech miesięcy, czyli ośmiu dwutygodniowych iteracji, zrealizujemy około 240 punktów [lub dni idealnych] pracy”. W miarę jak dowiadujemy się coraz więcej o prawdziwych potrzebach naszych użytkowników i o rozmiarach projektu, ocena ta może ulec zmianie. Jednak w każdej chwili chcielibyśmy mieć możliwość oceny, gdzie jesteśmy w stosunku do celu polegającego na zrealizowaniu określonego zakresu prac w określonym przedziale czasu.

Na naszą pracę wpływa wiele czynników. Należy je wszystkie uwzględnić. Przede wszystkim (w idealnej sytuacji jest to najbardziej istotne) trzeba uwzględnić postępy zespołu. Jednak drugie w kolejności są wszelkie zmiany zakresu projektu. Właściciel produktu może dodać lub usunąć wymagania. Jeśli właściciel produktu doda czterdzieści punktów, a zespół zrealizuje trzydzieści, to zespół będzie miał więcej pracy do wykonania niż wtedy, gdy rozpoczął poprzednią iterację. Cel się przesunął. Warto wtedy wiedzieć, jak daleko zespołowi jest do nowego celu. Może się również zdarzyć, że deweloperzy nauczyli się podczas iteracji czegoś, co wpływa na oceny punktowe historyjek przypisane do niektórych prac wyznaczonych w późniejszej fazie planu wydania.

Te czynniki (wykonane prace, zmieniające się wymagania i skorygowane oceny) można porównać do oddziaływania wiatru (tzw. dryfu — ang. *leeway*) oraz morza (tzw. znosu — ang. *drift*) na łódź. Rozważmy rysunek 19.1, na którym pokazano siły działające na łódź. Łódź pokazana na tym schemacie pokona mniejszą odległość, niż można by wnioskować ze wskazań jej prędkościomierza. Podobnie, choć kompas łodzi wskazywał wschód przez całą podróż, z powodu wiatru łódź dryfowała na południe. Bez korekty kursu ta łódź nie dotrze do pierwotnie wyznaczonego celu. Spróbujmy zmienić w myślach oznaczenia strzałek z rysunku 19.1, tak aby dryf i znos stały się, odpowiednio, zmianą wymagań (dodanie lub usunięcie funkcjonalności) oraz zmianą ocen. Na rysunku 19.1 pokazano wyzwania związane ze śledzeniem postępów projektu oprogramowania według ustalonego harmonogramu.



RYСУNEK 19.1. Siły działające na łódź

Tempo

Prędkość dla łodzi mierzy się w węzłach. Zespół Agile mierzy postępy za pomocą tempa. Tempo wyraża się liczbą punktów (lub dni idealnych) zrealizowanych podczas iteracji. O zespole, który w ostatniej iteracji zrealizował dwanaście historyjek ocenionych łącznie na trzydzieści punktów, mówimy, że jego tempo wynosi „trzydzieści” lub „trzydzieści punktów na iterację”. Jeśli zakres projektu nie uległ zmianie, to zespół ma o tyle mniej pracy do wykonania.

Ponieważ tempo jest głównym wskaźnikiem postępów zespołu, należy ustalić kilka podstawowych zasad jego obliczania. Najważniejszą regułą jest liczenie punktów względem tempa tylko dla tych historyjek lub funkcjonalności, które zostały ukończone na końcu iteracji. Ukończone nie oznacza czegoś w rodzaju „Kodowanie zakończyło się, ale funkcjonalność nie została przetestowana” albo „Funkcjonalność jest zakodowana, ale trzeba ją zintegrować”. Funkcjonalność ukończona oznacza, że kod jest dobrze napisany, ma właściwą strukturę, został przesłany do repozytorium i jest „czysty” — tzn. zgodny ze standardami kodowania i przechodzi wszystkie testy. Aby dowiedzieć się, jaki postęp osiągnięto, liczymy tylko punkty dotyczące zakończonej pracy. Otwarcie drzwi do częściowego liczenia ukończonych pracy — na przykład przyznawanie częściowej liczby punktów za niedokończone prace — sprawia, że nie można dokładnie oszacować, gdzie jesteśmy.

Istnieją trzy podstawowe problemy z liczeniem niedokończonej pracy. Po pierwsze, zmierzenie niedokończonej lub niekompletnej pracy jest bardzo trudne. Kiedy w opisanych poniżej sytuacjach jesteśmy dalej? Kiedy historyjka użytkownika została zaprogramowana, ale nie uruchomiono dla niej testów, czy kiedy historyjka użytkownika została częściowo zaprogramowana i częściowo przetestowana? Jak daleko jest programista, który zaprojektował rozwiązanie dla historyjki, ale jeszcze nie zaczął jej kodować? Potrafimy dość łatwo ocenić, że coś jeszcze nie zostało rozpoczęte albo kiedy zostało zakończone. Powinniśmy oceniać pracę w jednym z tych dwóch stanów i nic poza tym.

Po drugie, niedokończone historyjki użytkownika niszczą zaufanie między zespołem deweloperów a zespołem klientów projektu. Jeśli podczas iteracji nie można dokończyć historyjki zgodnie z planem, to zespoły programistów i klientów muszą wspólnie rozwiązać problem natychmiast po jego wykryciu. Zazwyczaj oznacza to, że historyjka zostaje usunięta z iteracji albo podzielona, a jej części zostają przeniesione do innych iteracji. Właściciel produktu i zespół klientów mogą podejmować takie decyzje podczas iteracji w czasie rzeczywistym. Mogą także zmieniać priorytety, bazując na nowej wiedzy o kosztach historyjki. Inną możliwością jest podjęcie przez zespół klientów decyzji o aktualizacji kryteriów akceptacji historyjki. W takim przypadku może ona zostać zaakceptowana z zastosowaniem mniej restrykcyjnych kryteriów. Nie musi to oznaczać akceptacji wersji historyjki z błędami albo nieprzetestowanej, ale można zmniejszyć wymagania dotyczące wydajności, obsługi przypadków szczególnych i tak dalej.

Po trzecie i najważniejsze, niedokończone prace prowadzą do nagromadzenia się w procesie wytwarzania oprogramowania tzw. pracy w toku. Im więcej „pracy w toku” zespół pozwoli zgromadzić, tym więcej czasu zajmie przekształcenie koncepcji nowych funkcjonalności w działające oprogramowanie. Z biegiem czasu prowadzi to do zmniejszenia wydajności całego zespołu. Na podobnej zasadzie w przypadku dużej ilości nagromadzonej pracy w toku zespołowi trudniej zebrać opinie dotyczące wykonywanej pracy. To oznacza, że zdobywanie nowej wiedzy także jest opóźnione.

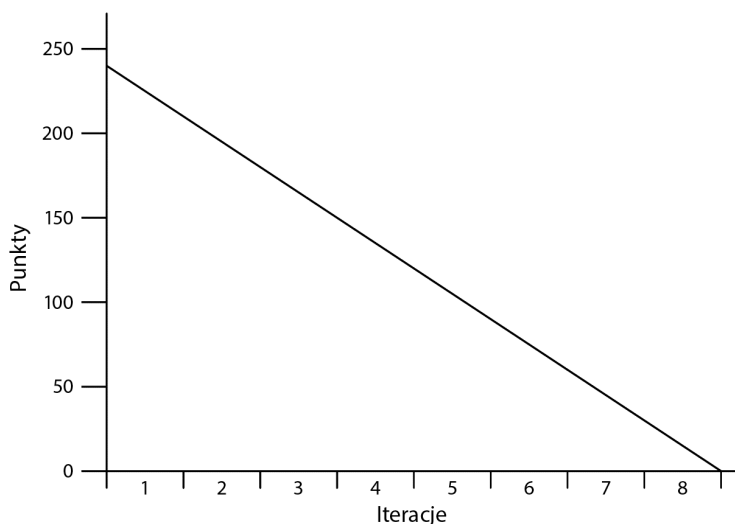
Jeśli na końcu iteracji zespół ma niedokończone historyjki użytkownika, to oznacza, że pracuje nad funkcjonalnościami lub historyjkami, które są zbyt duże. Niewielkie historyjki użytkownika prowadzą do stabilnego przepływu procesu projektowego. Jeżeli historyjki są niedokończone, to trzeba je podzielić na mniejsze. Najlepiej jeśli proces tego podziału zacznie się przed rozpoczęciem

iteracji. Jeśli jednak podczas iteracji historyjka okaże się większa niż się spodziewano, to należy o tym poinformować właściciela produktu. We współpracy z zespołem znajdzie on sposób podzielenia historyjki lub ograniczenia jej zakresu tak, aby jej fragment można było zakończyć w ramach danej iteracji, a pozostałą część przenieść do następnej iteracji.

Zatem jak zespół powinien liczyć częściowo zakończoną historyjkę podczas obliczania tempa? Sposób liczenia takiej historyjki jest mniej ważny od ustalenia, dlaczego nie udało się historyjki ukończyć oraz w jaki sposób można zapobiec podobnej sytuacji w przyszłości. Być może nie udało się zakończyć historyjki dlatego, że została źle oszacowana. Jeśli tak, to zespół powinien się zastanowić, jakie prace zostały źle ocenione lub o jakich pracach zapomniano, i starać się o tym pamiętać podczas oceny podobnej pracy w przyszłości. Mogło się także zdarzyć, że historyjka nie została zakończona ze względu na to, że do bieżącej iteracji wybrano zbyt dużo historyjek. Jeśli to było przyczyną, należy podjąć odpowiednie kroki, aby iteracje były planowane uważniej.

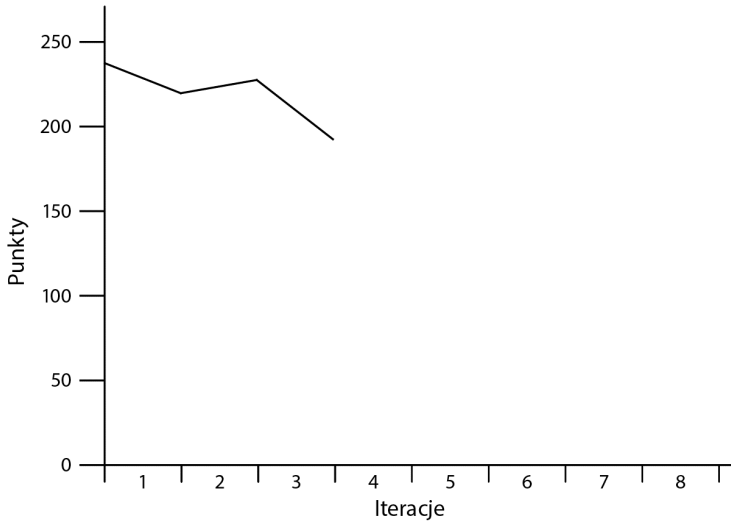
Wykresy wypalania wydania

Na rysunku 19.2 przedstawiono *wykres wypalania wydania* (Schwaber i Beedle 2002). Na osi pionowej zaprezentowano liczbę punktów pozostałych do zrealizowania w projekcie (równie łatwo można by zaprezentować liczbę pozostałych dni idealnych). Na osi pionowej pokazano iteracje. Wykres wypalania wydania pokazuje ilość pracy pozostałej na początku każdej iteracji. Wykres ten jest potężnym wizualnym wskaźnikiem szybkości, z jaką zespół zmierza do celu. Na rysunku 19.2 pokazano hipotetyczne wypalanie wydania projektu składającego się z 240 punktów, które równomiernie rozłożono na osiem iteracji.



RYСУNEK 19.2. Projekt złożony z 240 punktów zrealizowany w ośmiu iteracjach

Oczywiście jest mało prawdopodobne, aby zespół, którego średnie tempo wynosi trzydzieści, miał dokładnie takie samo tempo w każdej iteracji. Bardziej prawdopodobną postać wykresu wypalania wydania projektu złożonego z 240 punktów pokazano na rysunku 19.3.



RYСУNEK 19.3. *Bardziej realistyczny wykres wypalania 240-punktowego projektu po trzeciej iteracji*

Na rysunku 19.3. zaprezentowano postępy zespołu po trzech iteracjach. Postępy tego zespołu nie są spójne. Podczas pierwszej iteracji zrealizowano prace ocenione na około trzydzieści punktów. Ale na końcu drugiej iteracji okazało się, że do zrobienia jest więcej pracy niż wtedy, gdy zaczynano tę iterację. Może się to zdarzyć, jeśli zespół na przykład uświadomił sobie, że opracowanie interfejsu użytkownika będzie znacznie trudniejsze niż początkowo oceniano i zwiększył swoje oceny dla wszystkich pozostałych historyjek związanych z interfejsem użytkownika.

Na wykresie wypalania mogą pojawić się fragmenty rosnące ze względu na dodanie nowych prac do wydania. Porównajmy to do łodzi, która porusza się z prędkością ośmiu mil na godzinę, ale w kierunku przeciwnym do prądu o wartości dwunastu mil na godzinę. W przypadku żeglugi w tej sytuacji łódź oddala się od pierwotnego celu. Jednak w przypadku projektu tworzenia oprogramowania dodatkowa praca może wynikać stąd, że zespół dowiedział się czegoś, co kieruje projekt w stronę bardziej wartościowego wydania.

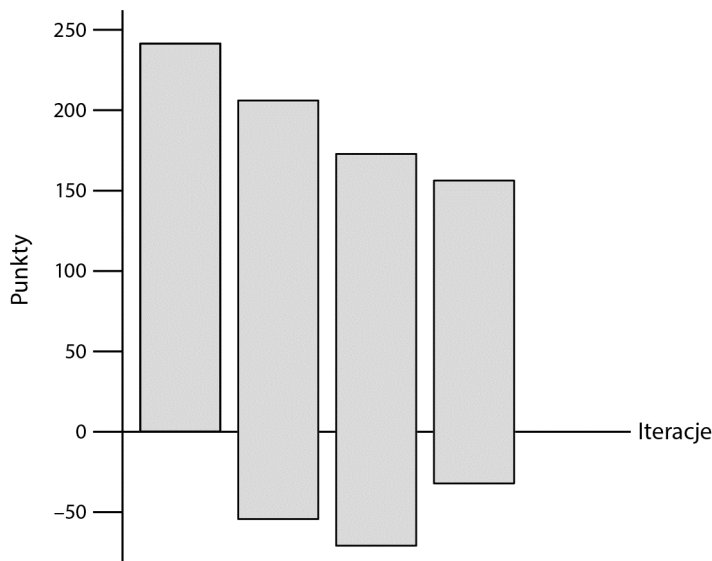
Aby zobaczyć, jak to działa, załóżmy, że w drugiej iteracji zespół ponownie zrealizował zaplanowane trzydzieści punktów, ale właściciel produktu zdefiniował dodatkowe czterdzieści punktów pracy, które trzeba zrealizować w bieżącym wydaniu. W tej sytuacji rezultat netto jest taki, że na końcu iteracji jest więcej pracy, niż było w czasie, gdy iteracja się rozpoczęła. Ze względu na to, że wykres wypalania odzwierciedla postępy zespołu netto, na wykresie widać ten wzrost.

Można by się zastanawiać, jaki jest cel rysowania wykresu w taki sposób. Rysujemy go tak dlatego, że dzięki temu na jednym wykresie czytelnie i prosto prezentujemy dwie najważniejsze liczby pozwalające ustalić, czy realizacja projektu idzie zgodnie z planem: ile pracy pozostało oraz jakie są postępy zespołu netto uwzględniające wszystkie zmiany zakresu projektu. Wyobraź sobie, że jesteś w zespole, którego postępy zaprezentowano na rysunku 19.3. Na końcu trzeciej iteracji zadano Ci pytanie, czy wydanie

zostanie ukończone w ciągu zaplanowanych ośmiu iteracji. Jeśli nie zostanie zakończone, to masz dostarczyć dokładniejszą ocenę planowanej daty ukończenia. Na takie pytanie można odpowiedzieć, patrząc na wykres wypalania z rysunku 19.3. Wystarczy poprowadzić prostą między wartością 240 punktów na osi pionowej a liczbą punktów, które pozostały do zrealizowania w projekcie. Miejsce, w którym ta prosta przecnie oś poziomą, oznacza oczekiwaną datę zakończenia tego projektu. Wystarczy rzut oka na rysunek 19.3, aby stwierdzić, że wydanie nie zostanie ukończone w ciągu planowanych ośmiu iteracji.

Śłupkowy wykres wypalania wydania

Pod jednym względem wykres wypalania z rysunku 19.3 jest świetny. Jest łatwo zrozumiały i można go szybko wyjaśnić każdej osobie w firmie. Wykres wypalania taki jak ten jest bardzo pouczający. Mówi nam, kiedy projekt najprawdopodobniej się skończy w przypadku, jeśli wszystkie czynniki, które na niego wpływają, pozostaną bez zmian. Czasami jednak warto narysować wykres wypalania wydania w taki sposób, aby można było osobno obserwować tempo zespołu oraz zmiany zakresu. W tym celu można narysować wykres wypalania podobny do tego, który pokazano na rysunku 19.4.



RYСУNEK 19.4. Rozdzielenie wpływu tempa i zmian zakresu

Każdy słupek na rysunku 19.4 przedstawia ilość pracy w wydaniu na początku iteracji. W takim typie wykresu wypalania wykorzystywane są słupki zamiast linii, aby lepiej odróżnić obszary powyżej i poniżej osi poziomej w punkcie 0. Dolna część słupka opada poniżej osi poziomej zawsze wtedy, gdy do projektu są dodawane nowe prace. Dolna część podnosi się wtedy, gdy z iteracji są usuwane prace. Jeśli dolna część słupka opadnie poniżej poziomej osi 0, to oznacza, że dodano prace do wydania.

Najlepiej można zobaczyć, jak to działa, na przykładzie. Na wykresie pokazanym na rysunku 19.4 zaplanowano wydanie na 240 punktów pracy. Na początku pierwszej iteracji wykres wypalania składa się z jednego słupka od 0 do 240. Tak jak wcześniej, zespół oczekuje zachowania średniego tempa trzydzieści

oraz planuje, że projekt zostanie zrealizowany po ośmiu iteracjach. Podczas pierwszej iteracji zespół osiągnął oczekiwane tempo, a górna krawędź drugiego słupka sięga punktu 210. Jednak właściciel produktu uświadomił sobie, że w wydaniu trzeba uwzględnić więcej funkcjonalności niż pierwotnie sądzono. Zdefiniowano dodatkowych 50 punktów pracy, które trzeba będzie zrealizować w tym wydaniu. To spowodowało, że słupek dla drugiej iteracji opadł poniżej poziomej linii 0. Drugi słupek ma zakres od -50 do 210. Mówiąc inaczej, do realizacji wydania potrzeba teraz 260 punktów pracy, czyli więcej, niż było na początku projektu. Na koniec drugiej iteracji z rysunku 19.4 można odczytać trzy interesujące fakty.

1. Tempo zespołu jest zgodne z oczekiwaniami. Można to zobaczyć po spadkach górnych poziomów dwóch pierwszych słupków.
2. Do wydania dodano sporo zadań. Można to zobaczyć po spadku dolnej krawędzi drugiego słupka. Prawdopodobnie pracę dodano dlatego, że jej wykonanie doprowadzi do bardziej wartościowego wydania. Należy jednak zwrócić uwagę na tempo zmian zakresu projektu — dotychczas więcej pracy dodano niż zakończono. Nie trzeba się tym martwić. Wszystko zależy od tego, czy trend będzie się utrzymywał oraz jak ważne jest dotrzymanie pierwotnie ustalonego terminu wydania.
3. Całkowita ilość pracy pozostającej w wydaniu jest większa niż na początku projektu. Widać to wyraźnie, ponieważ całkowita wysokość drugiego słupka jest większa od pierwszego.

Jest oczywiste, że rysowanie takiego wykresu wypalania daje większe możliwości ekspresji. Jego wadą jest to, że znaczenie wykresu nie jest natychmiast jasne.

Przyjrzyjmy się drugiej i trzeciej iteracji projektu z rysunku 19.4. W czasie drugiej iteracji zespół ponownie osiągnął docelowe tempo. Właściciel produktu znowu dodał pracę, ale tym razem dodał jej mniej niż podczas poprzedniej iteracji.

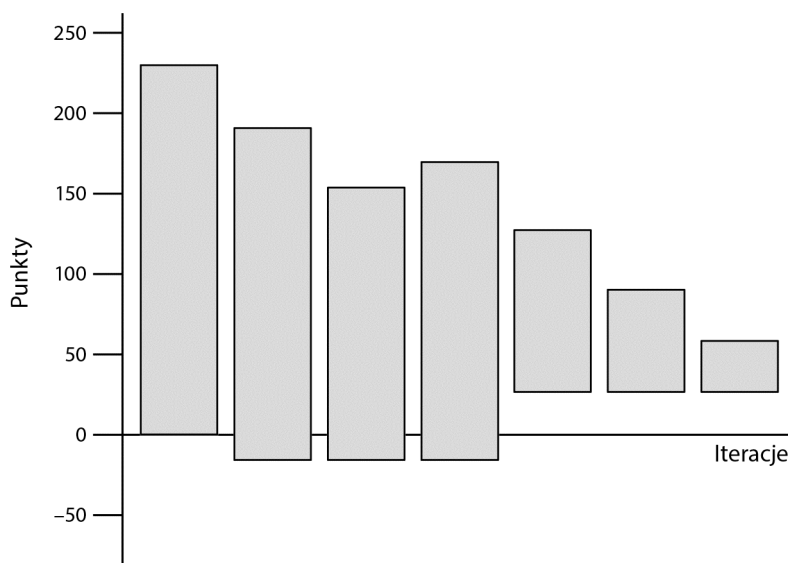
Co jednak się stało podczas trzeciej iteracji? Wówczas tempo spadło do dwudziestu. Może to wynikać z nieprawidłowej oceny niektórych historyjek w tej iteracji, choroby lub urlopu członka zespołu albo weryfikacji oceny części pozostałej pracy. Zespół być może zrealizował planowane trzydzieści punktów pracy, ale być może zwiększył oceny dla niektórych pozostałych historyjek, w związku z czym postęp netto wynosi dwadzieścia, a nie trzydzieści.

Jednak to, co w trzeciej iteracji było najciekawsze, widać w dolnej części czwartego słupka. Podczas tej iteracji właściciel produktu usunął funkcjonalności z produktu. W chwili wydania ten projekt nadal będzie obejmował więcej punktów niż pierwotnie planowano. Można to stwierdzić, patrząc na słupek, który wciąż opada poniżej poziomej osi 0. Jednak w tej chwili projekt zawiera mniej zaplanowanych punktów, niż miał na początku poprzedniej iteracji. Nie jest ważne, czy usunięte będą funkcjonalności, które były uwzględnione w pierwotnym planie wydania, czy te, które właściciel produktu dodał w poprzednich iteracjach. Ustalanie priorytetów pracy nadal należy do właściciela produktu, który może dodawać lub usuwać funkcjonalności w miarę potrzeb. Efekt netto pokazano w dolnej części słupka wypalania.

Są cztery proste zasady, o których należy pamiętać podczas rysowania tego rodzaju wykresu wypalania.

- Każdorazowo, gdy zostaną ukończone prace, obniża się górna część słupka.
- W razie weryfikacji oceny zadania górna część słupka przesuwa się w górę lub w dół.
- W przypadku dodawania nowej pracy dolna część słupka opada.
- W razie usuwania pracy dolna część słupka się podnosi.

Spójrzmy na słupkowy wykres wypalania wydania rzeczywistego projektu, który pokazano na rysunku 19.5. Można na nim zaobserwować, że podczas pierwszych dwóch iteracji zespół osiągnął duże postępy. Właściciel produktu dodał niewielką ilość pracy przed rozpoczęciem drugiej iteracji, co zdarza się dość powszechnie w wielu zespołach. Podczas trzeciej iteracji zespół odkrył, że oceny niektórych jego historyjek użytkownika były zaniżone, dlatego zweryfikowano oceny niektórych spośród pozostałych prac. To doprowadziło do wzrostu górnej części czwartego słupka na rysunku 19.5. Przed rozpoczęciem czwartej iteracji właściciel produktu usunął pracę z planu wydania. W efekcie dolna część słupka podniosła się nawet powyżej poziomej linii 0. Podczas tej iteracji zespół osiągnął znaczne postępy. Od tego momentu plan wydania pozostał bez zmian i do końca trwania projektu obserwowano stałe postępy.



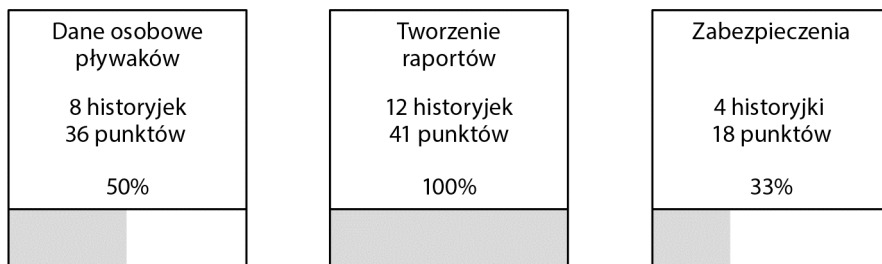
RYСУNEK 19.5. *Usuwanie pracy z wydania*

Ostrzeżenie dotyczące słupkowych wykresów wypalania wydania

Chociaż w mojej ocenie słupkowe wykresy wypalania wydania są bardziej ekspresywne (i dlatego często bardziej przydatne) niż tradycyjne wykresy wypalania, to mam kilka zastrzeżeń dotyczących ich użycia. Po pierwsze, słupkowy wykres wypalania jest trudniej zrozumieć. Z tego powodu w nowych zespołach prawie zawsze początkowo stosuję prostszy, liniowy wykres wypalania. Po drugie, słupkowy wykres wypalania może być wykorzystywany wyłącznie w firmach, które są na tyle dojrzałe, że nie kłócą się, czy coś należy do obszaru powyżej linii, czy poniżej linii. Przy pierwszych dyskusjach tego rodzaju oznajmiam wszystkim zainteresowanym, że nie możemy posługiwać się słupkowym wykresem wypalania i że wracamy do wykresu liniowego.

Wykres parkingowy

Jeff De Luca (2002) zaproponował jeszcze inny skuteczny sposób wizualizacji postępów zespołu w realizacji zaplanowanych funkcjonalności w wydaniu. Na rysunku 19.6 pokazano wersję tego, co De Luca nazwał wykresem parkingowym. *Wykres parkingowy* zawiera dużą prostokątną ramkę odpowiadającą każdemu tematowi (grupie historyjek użytkownika) w wydaniu. Każda ramka jest opisana nazwą, liczbą historyjek składających się na ten temat, liczbą punktów lub dni idealnych dla tych historyjek oraz wartością procentową ukończonych historyjek.



RYSUNEK 19.6. Wykres parkingowy pokazuje, ile pracy ukończono z każdego tematu

W ramce „Dane osobowe pływaków” na rysunku 19.6 można zauważyć, że temat ten składa się z ośmiu historyjek, które zostały łącznie ocenione na trzydzieści sześć punktów. Osiemnaście z tych punktów zrealizowano, ponieważ wiemy, że 50% funkcjonalności ukończono. Nie możemy powiedzieć, ile historyjek użytkownika zrealizowano. Pojedyncze ramki na wykresie parkingowym mogą być nawet pokolorowane. To pokazuje, czy określony temat został zrealizowany, uwzględniony w harmonogramie, potrzebuje uwagi czy jest znacząco poza harmonogramem.

Wykres parkingowy jest doskonałą metodą na upakowanie dużej ilości informacji na małej przestrzeni. W wielu przypadkach, dzięki zastosowaniu wykresu parkingowego, wszystkie tematy wchodzące w skład wydania można zestawić na jednej stronie.

Podsumowanie

Tempo jest miarą ilości pracy wykonanej przez zespół w każdej iteracji. Tempo powinno być obliczane przy użyciu reguły „wszystko albo nic”. Jeśli historyjka została zakończona, zespół podczas szacowania tempa dolicza jej pełną ocenę. Jeśli historyjka została zrealizowana częściowo, to przy ustalaniu tempa nie jest liczona w ogóle.

Wykres wypalania wydania pokazuje liczbę punktów lub dni idealnych pozostałych w projekcie na początku każdej iteracji. Wykres ten dla zespołu nigdy nie jest idealnie gładki. Zmienia się na przykład z powodu niedokładnych ocen, zmian w ocenach oraz zmiany w zakresie. Wykres wypalania może nawet wykazywać tendencję rosnącą podczas iteracji. Oznacza to, że nawet jeśli zespół prawdopodobnie zakończył jakąś pracę, to albo zdano sobie sprawę, że pozostałe prace zostały źle oszacowane, albo zakres projektu się zwiększył. Kluczem do interpretacji wykresu wypalania wydania jest zrozumienie, że ten wykres pokazuje czysty postęp zespołu — tzn. postęp pomniejszony o prace dodane do wydania.

Słupkowy wykres wypalania wydania to przydatna odmiana standardowego wykresu wypalania wydania. Na tego rodzaju wykresie oddziela się postępy zespołu względem zaplanowanej pracy od zmian zakresu wydania. Zmiany zakresu są widoczne jako opadanie słupka poniżej osi poziomej. Ten typ wykresu jest bardziej ekspresywny od standardowego wykresu wypalania, ale należy się nim ostrożnie posługiwać, ponieważ w niektórych firmach może być przyczyną sporów co do tego, czy zmiana wpływa na dolną, czy na górną część słupka na wykresie.

Wykresy parkingowe są przydatne do prezentowania wysokopoziomowej wizji postępów zespołu w implementacji różnych tematów zaplanowanych w projekcie.

Pytania do dyskusji

1. Jeśli nie korzystasz z wykresu wypalania wydania w swoim bieżącym projekcie, to zastanów się, jaki byłby efekt utworzenia takiego wykresu na końcu każdej iteracji?
2. Która z technik monitorowania i generowania raportów opisana w tym rozdziale przyniesie największe korzyści w Twoim bieżącym projekcie?
3. Którzy interesariusze w Twoim projekcie nie otrzymują o nim informacji, które mogłyby się okazać im pomocne?

ROZDZIAŁ 20.

Monitorowanie planu iteracji

Fakty są lepsze niż marzenia.

— Winston Churchill

Oprócz śledzenia postępu w osiągnięciu wysokopoziomowego celu wydania zawsze warto śledzić postępy zespołu w realizacji prac w ramach jednej iteracji. W tym rozdziale przyjrzymy się dwóm narzędziom do śledzenia iteracji: tablicy zadań (ang. *task board*) oraz wykresom wypalania.

Tablica zadań

Tablica zadań spełnia dwojaki cel: dostarcza zespołowi wygodnego mechanizmu organizacji jego pracy oraz sposobu na to, by natychmiast zobaczyć, ile pracy pozostało. Jest bardzo ważne, aby tablica zadań (lub mechanizm jej równoważny) dawała zespołowi większą elastyczność w sposobie organizacji swojej pracy. Członkowie zespołu Agile nie zgłaszają się do zadania (lub im się go nie przydziela), zanim nie są gotowi do rozpoczęcia nad nim pracy. Oznacza to, że z wyjątkiem ostatniego dnia lub dwóch dni iteracji zwykle jest dużo zadań, do których jeszcze nikt się nie zgłosił. Dzięki tablicy zadań te pozycje są bardzo dobrze widoczne. W ten sposób wszyscy członkowie zespołu mogą zobaczyć, nad czym trwają prace i które są dostępne, aby można się było zgłosić do ich wykonania. Przykład tablicy zadań pokazano na rysunku 20.1.

Tablica zadań to często duża biała tablica, albo — jeszcze lepiej — tablica korkowa. Do tablicy zadań przykleja się lub przypina karty historyjek wraz z kartami zadań zapisanymi podczas planowania iteracji. Tablica zadań zawiera jeden wiersz dla każdej historyjki użytkownika lub funkcjonalności, nad którymi będą prowadzone prace podczas iteracji. Pierwszy wiersz z rysunku 20.1 zawiera informacje dotyczące historyjki użytkownika ocenionej na pięć punktów. Karta historyjki znajduje się w pierwszej kolumnie tablicy. Ze względu na to, że karta ta zawiera ocenę punktową przypisaną do historyjki, to każdy, kto spojrzy na tablicę zadań, może szybko ocenić liczbę punktów dla każdej historyjki uwzględnionej w iteracji.

W drugiej kolumnie znajdują się wszystkie karty zadań, które zespół ocenił jako niezbędne do zaimplementowania historyjki lub funkcjonalności. Każda z tych kart zawiera ocenę prac pozostałych do wykonania zadania.

Historyjka	Do zrobienia	Testy gotowe	W trakcie	Do sprawdzenia	Liczba godzin
Jako użytkownik mogę... 5	Kodowanie... 8 Kodowanie... 5 Kodowanie... 6	√	Kodowanie... SC 6 Kodowanie... DC 4	Kodowanie... LC 4	33
Jako użytkownik mogę... 2	Kodowanie... 8 Kodowanie... 5				13
Jako użytkownik mogę... 3	Kodowanie... 3 Kodowanie... 6	√	Kodowanie... MC 4		13

RYСУNEK 20.1. Tablica zadań wykorzystywana podczas iteracji

Trzecia kolumna określa, czy dla historyjki są gotowe testy akceptacyjne. Jestem wielkim zwolennikiem techniki tworzenia oprogramowania sterowanego testami (Beck 2002) zarówno na poziomie kodu, kiedy zachęcam programistów do pisania nieprzechodzących testów jednostkowych przed napisaniem kodu, jak i na poziomie funkcjonalności, kiedy zachęcam zespoły do projektowania wysokopoziomowych testów akceptacyjnych przed rozpoczęciem kodowania.

Jeśli w ramach planowania iteracji dla każdej historyjki użytkownika zidentyfikowano warunki satysfakcji (zgodnie z zaleceniami z rozdziału 14. „Planowanie iteracji”), to jest to łatwe, ponieważ warunki satysfakcji są niczym innym jak wysokopoziomowymi testami akceptacyjnymi historyjki użytkownika. Ten rodzaj specyfikacji przez przykład bardzo przydaje się programistom, ponieważ mogą oni odnosić się do konkretnych przykładów opisujących oczekiwane działanie poszczególnych funkcjonalności i reguł biznesowych.

Pracowałem z zespołami, które po zdefiniowaniu wysokopoziomowych testów historyjki umieszczali w kolumnie *Testy gotowe* duży znak potwierdzenia. Idąc dalej, zachęcam zespoły do tego, by nie przenosiły zbyt wielu kart historyjek do kolumny *W trakcie*, zanim nie zostaną określone testy. Kolumna *Testy zdefiniowane* być może nie jest potrzebna, ale jest to przydatne, widoczne przypomnienie dla zespołu, który przyzwyczaja się do definiowania testów akceptacyjnych przed przystąpieniem do kodowania funkcjonalności.

W kolumnie *W trakcie* znajdują się karty historyjek, do których zgłosili się programiści. Zazwyczaj programista bierze kartę z kolumny *Do zrobienia*, umieszcza na niej inicjały, a następnie przenosi do kolumny *W trakcie*. Dzieje się to w ciągu dnia, w miarę jak programiści kończą pracę i wybierają zadania, którymi będą się zajmować w następnym dniu. Nikt nie powinien się zgłaszać do więcej niż jednego lub dwóch zadań jednocześnie. To pomaga utrzymać spójny przepływ pracy w procesie

i obniża koszty przełączania kontekstu między wieloma zadaniami. Kiedy razem z zespołem tworzymy tablicę zadań, to aby ciągle przypominać o tym, że nie należy zgłaszać się do więcej niż jednego zadania, ustalam szerokość kolumny *W trakcie* na jedną kartę historyjki. Kolumna *Do zrobienia* jest zazwyczaj szersza (niż pokazano na rysunku 20.1), ponieważ zwykle zawiera w wierszu nawet cztery karty.

Kolumna *Do sprawdzenia* to kolejna kolumna, która może, lecz nie musi być potrzebna. Ja uważam, że się przydaje — zwłaszcza podczas pracy z zespołem, który uczy się, jak stosować metody Agile. W idealnej sytuacji podczas planowania iteracji należy przemyśleć wszystkie działania związane z testowaniem i napisać dla nich karty zadań. W takim przypadku po zrealizowaniu zadania zapisanego na karcie („Kodowanie interfejsu użytkownika boojum”) usuwa się ją z tablicy (lub przenosi do ostatniej kolumny — *Wykonano*). W tym momencie ktoś z zespołu może zgłosić się do realizacji zadania zapisanego na karcie testu („Testowanie interfejsu użytkownika boojum”). Czasami jednak występują sytuacje, gdy deweloper uważa, że karta zadania została zrealizowana, ale chciałby, żeby ktoś spojrział na nie i je zweryfikował. W takich przypadkach oraz gdy z zadaniem nie ma powiązanych zadań testowych, kartę zadania umieszcza się w kolumnie *Do sprawdzenia*.

Deweloperzy mogą w dowolnej chwili zmieniać oceny na każdej karcie zadania. Na przykład jeśli zacznę pracować nad określoną kartą i zdam sobie sprawę, że ocena dwie godziny jest zła, to podchodzę do tablicy zadań, przekreślam dwójkę i zastępuję ją — przykładowo — liczbą sześć. Jeśli uważam, że ocena jest jeszcze bardziej odległa, to mogą zerwać tę kartę zadania i zastąpić ją dwiema lub trzema kartami, z których każda jest oznaczona własną oceną. Ostatnia kolumna na tablicy zadań zawiera sumę godzin pozostałych do realizacji funkcjonalności lub historyjki. Zwykle co rano sumuję liczbę godzin w każdym wierszu. Wartości uzyskanych z podsumowania używam do narysowania wykresu wypalania iteracji, który jest drugim narzędziem śledzenia postępów podczas iteracji.

Śledzenie błędów na tablicy zadań

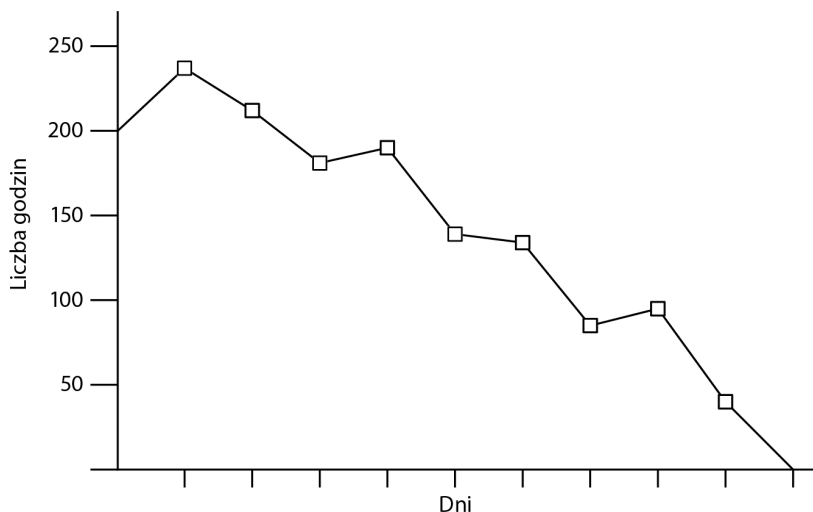
Wiele zespołów po rozpoczęciu przechodzenia na proces wytwarzania Agile boryka się z dużą liczbą starych błędów. Nie tylko zwykle istnieje duża liczba błędów, które mają być „kiedys” naprawione, ale także w szybkim tempie pojawiają się nowe błędy. Częstym wyzwaniem dla zespołów przechodzących na proces Agile jest sposób radzenia sobie z tego rodzaju błędami. Tablica zadań to wygodny mechanizm pozwalający na złagodzenie tego problemu.

Aby zaprezentować przykład, w jaki sposób może pomóc tablica zadań, założmy, że właściciel produktu dołączył do nowej iteracji zadanie „Naprawić dziesięć najpoważniejszych błędów”. Właściciel produktu wybiera dziesięć błędów, a deweloperzy piszą karty zadań (z oceną) dla każdego z nich. Karty są przypinane na tablicy zadań w kolumnie *Do zrobienia*. Podczas iteracji deweloperzy pracują nad dziesięcioma błędami w taki sam sposób, jak pracują nad innymi kartami zadań. Przypuśćmy teraz, że w połowie iteracji użytkownik znajdzie nowy błąd. Jeśli priorytet nowego błędu zostanie uznany za wyższy od jednego lub większej liczby błędów pozostałych w kolumnie *Do zrobienia*, to właściciel produktu może zastąpić równoważną ilość pracy nad usuwaniem innych błędów zadaniem związanym z naprawieniem nowego błędu.

Takie podejście pozwala zespołowi naprawiać stare błędy w dowolnym, wybranym przez właściciela produktu tempie. Zespół może przeznaczyć na poprawki błędów 40 lub 100 godzin. O tym, jaka część iteracji powinna być poświęcona poprawkom błędów zamiast pracy nad nowymi funkcjonalnościami, decyduje właściciel produktu. Następnie właściciel produktu i zespół wspólnie ustalają, jakie błędy można poprawić w tym czasie.

Wykresy wypalania iteracji

Narysowanie wykresu wypalania iteracji to doskonały sposób, by przekonać się, czy projekt idzie we właściwym kierunku. Czasami, zwłaszcza dla odpowiednio długo trwających iteracji, warto utworzyć wykres wypalania iteracji. W przypadku iteracji trwających tydzień to prawdopodobnie nie jest potrzebne. Zanim na wykresie wypalania iteracji będzie widoczny trend, to jednotygodniowa iteracja zdąży się skończyć. Uważam jednak, że wykresy wypalania iteracji są niezwykle przydatne w przypadku iteracji trwających dwa tygodnie lub dłużej. Na wykresie wypalania iteracji zaznacza się pozostałe godziny w poszczególnych dniach — tak jak pokazano na rysunku 20.2.



RYСУNEK 20.2. Wykres wypalania iteracji

Aby utworzyć wykres wypalania iteracji, wystarczy zsumować wszystkie godziny na tablicy zadań dla poszczególnych dni i narysować na wykresie. Jeśli tablica zadań zespołu jest narysowana na białej tablicy, to wykres wypalania iteracji zwykle rysuję ręcznie, po jednej stronie tablicy zadań. Jeśli tablica zadań jest na tablicy korkowej, to przypinam do niej duży arkusz papieru i na nim rysuję wykres wypalania.

Śledzenie poświęconego wysiłku

W poprzednim rozdziale wprowadziłem analogię projektu do żaglówki, aby podkreślić, że postępy żaglówki nie zawsze jest łatwo zmierzyć. Żaglówka, która płynęła wczoraj przez osiem godzin, a następnie zakotwiczyła w jakimś miejscu może, lecz nie musi być osiem godzin bliżej miejsca docelowego. Wiatr i prądy morskie mogły zepchnąć łódź z jej planowanego kursu. Łódź może być bliżej swojego miejsca przeznaczenia lub dalej od niego. W takiej sytuacji najbardziej sensownym działaniem załogi jest ocena położenia łodzi względem miejsca przeznaczenia. Pomiary przebytej odległości lub czasu podróży nie pomogą, jeśli nie będziemy pewni, czy poruszamy się w dobrym kierunku.

W projektach znacznie ważniejsza jest świadomość tego, ile pozostało do zrobienia, niż ile dotąd zrobiono. Co więcej, śledzenie poświęconego wysiłku i porównywanie go z wysiłkiem oszacowanym może prowadzić do zjawiska znanego jako *obawa przed szacowaniem* (ang. *evaluation apprehension*) (Sanders 1984). Jeśli oceniający boją się dostarczać oszacowań, pojawia się znany syndrom „walcz lub uciekaj” i oceniający zaczynają bardziej polegać na instynkcie niż na analitycznym myśleniu (Jørgensen 2004).

Śledzenie poświęconego wysiłku w celu zwiększenia dokładności ocen to balansowanie na bardzo cienkiej linii. Metoda może się sprawdzać (Lederer i Prasad 1998; Weinberg i Schulman 1974). Jednak menedżer projektu lub dowolna osoba, która realizuje śledzenie, musi uważać, aby nie wywierać zbyt dużej presji na oceniających. Może to bowiem doprowadzić do sytuacji, w której oceniający zaczęli podawać gorsze oceny zamiast lepszych.

Ponadto należy pamiętać, że częścią każdej oceny jest zmienność. Niezależnie od tego, ile wysiłku włożymy w poprawę ocen, szacunki nigdy nie będą doskonałe. Świadectwem na prawdziwość tej tezy może być poranny dojazd do pracy. Istnieje integralna doza zmienności w tej podróży niezależnie od tego, w jaki sposób się poruszamy, jaki dystans musimy przebyć oraz gdzie mieszkamy. Żadne umiejętności prowadzenia pojazdów nie są w stanie wyeliminować zmienności podczas dojazdu do pracy.

Indywidualne tempo

Niektóre zespoły opisują indywidualne tempo jako liczbę punktów lub dni idealnych zgromadzonych przez pojedynczego członka zespołu. Nie należy śledzić indywidualnych temp. Śledzenie indywidualnych temp prowadzi do zachowań, które mają ujemny wpływ na powodzenie projektu. Przypuśćmy, że ogłoszono, iż w poszczególnych iteracjach będą mierzone i śledzone indywidualne tempa. W jaki sposób, Twoim zdaniem, zareagują członkowie zespołu? Jeśli jestem zmuszony wybierać między zakończeniem mojej własnej historii użytkownika a udzieleniem pomocy innemu członkowi zespołu, to jaki bodziec zadziała, jeśli w zespole są mierzone indywidualne tempa?

Członkom zespołu należy dostarczać wszystkich możliwych bodźców do tego, by pracowali jako zespół. Jeśli wydajność zespołu wzrośnie dzięki mojej pomocy innemu członkowi zespołu, to powinienem jej udzielić. Liczy się tempo zespołu. Tempo indywidualne nie ma znaczenia. Nie jest to nawet parametr, który warto mierzyć czasowo.

Dodatkowym argumentem przemawiającym przeciwko mierzeniu indywidualnego tempa są trudności w jego obliczaniu. Większość historii użytkownika należy pisać w taki sposób, aby musiała nad nimi pracować więcej niż jedna osoba — na przykład projektant interfejsu użytkownika, programista, inżynier bazy danych i tester. Jeśli większość Twoich historii użytkownika może być zrealizowana przez jedną osobę, to powinieneś przemyśleć, w jaki sposób są pisane Twoje historie użytkownika. Zazwyczaj oznacza to, że powinny być napisane na wyższym poziomie, tak aby uwzględniały pracę wielu osób.

Podsumowanie

Tablica zadań — która często ma postać białej tablicy, tablicy korkowej lub po prostu wyznaczonych miejsc na ścianie — pomaga zespołom zorganizować i zwizualizować ich pracę. Kolumny na tablicy zadań są oznaczone etykietami, a w miarę postępów w pracach członkowie zespołu przemieszczają karty zadań między kolumnami.

Wykres wypalania iteracji jest podobny do wykresu wypalania wydania, ale jest używany do śledzenia prac wyłącznie bieżącej iteracji. Na osi pionowej wykresu prezentowana jest liczba pozostałych godzin, natomiast na osi poziomej — poszczególne dni iteracji.

Zespoły nie powinny śledzić liczby godzin wykorzystanych na realizację poszczególnych zadań, aby doskonalić umiejętności dokonywania ocen. Zagrożenia i wysiłek, które są z tym związane, zwykle przewyższają uzyskane korzyści.

Zespoły nie powinny obliczać lub śledzić tempa indywidualnych członków zespołu.

Pytania do dyskusji

1. Czy w Twojej firmie korzyści z monitorowania rzeczywistych nakładów pracy na realizację zadań i ich porównywanie z szacunkami przewyższają zagrożenia i koszty podejmowania tego rodzaju działań?
2. Co możesz zrobić, aby osiągnąć te same korzyści, jakie uzyskują zespoły posługujące się tablicami zadań, jeśli zespół w Twoim bieżącym projekcie nie pracuje w tej samej lokalizacji?

ROZDZIAŁ 21.

Ogłaszanie planów

Im bardziej wyszukane środki przekazu, tym mniej treści przekazujemy.

— Joseph Priestley

W tym rozdziale omówimy kilka sposobów ogłaszania planów. Jednak ważniejsze od tego, o czym dokładnie informujemy, jest to, jak podchodzimy do pracy polegającej na informowaniu o ocenach i planach. Chcemy, aby cała komunikacja, a zwłaszcza komunikacja dotycząca ocen i planów odbywała się często, była rzetelna i dwukierunkowa.

Częste informowanie o planach jest istotne ze względu na częstotliwość, z jaką aktualizowane są plany w projektach Agile. Na przykład nawet jeśli zespół korzysta z ruchomych planów wyprzedzających (tak jak opisano w rozdziale 18. „Planowanie projektu z udziałem wielu zespołów”), jego plan wydania może pokazywać tylko to, nad czym będą trwały prace w ciągu kilku najbliższych iteracji. Do planu wydania mogą być dołączone historyjki użytkownika, które zostaną zaimplementowane do końca wydania, ale nie zostaną uwzględnione w kolejce poza ramami planu wyprzedzającego i mogą być wyszczególnione tylko jako szerokie tematy.

Nie możemy (i nie chcemy) tworzyć pierwszego dnia trwania projektu planu, który będzie później pozostawiony na okres od trzech do sześciu miesięcy. Plany są aktualizowane w trakcie realizowania projektu i te aktualizacje powinny być ogłaszane. Niezastosowanie się do tego wymagania jest ignorowaniem cennych opinii zwrotnych, które przyczyniają się do poprawy atrakcyjności i sukcesu produktu oraz użyteczności planu. Dla członków zespołu jest ważne, aby w krótkim okienku czasowym trwania iteracji móc zobaczyć codzienne zmiany na wykresie wypalania iteracji tak, by można było wprowadzić korekty w celu zakończenia wszystkich prac podczas iteracji. W dłuższym okresie wydania interesariusze i uczestnicy projektu potrzebują wglądu w postęp prac oraz wersje planu wydania.

Jeśli zespół deweloperów oraz klienci mają sobie wzajemnie ufać, to potrzebna jest między nimi uczciwa komunikacja. Jeśli na przykład deweloperzy uświadomią sobie, że właściciel produktu podał im nieprawdziwe terminy, to przestaną mu ufać. Z kolei gdy deweloperzy będą podawali oceny, które w opinii właściciela produktu są nierealne, zaufanie również zniknie. Niedawno pracowałem z zespołem, którego członkowie poinformowali mnie, że menedżerowie żądają wykonania więcej pracy, niż można rozsądnie oczekiwać, ponieważ „w ten sposób, jeśli otrzymają tylko 80%, to i tak będą zadowoleni”. Chodziło o to, że menedżerowie zawsze otrzymywali 80% tego, czego żądali, dlatego zawsze żądali więcej, niż było trzeba.

Bez zaufania trudno o uczciwą komunikację, dlatego utratę zaufania należy traktować bardzo poważnie. Jeśli deweloper wie, że określone zadanie zajmie mu znacznie więcej czasu, niż się w tej chwili oczekuje, powinien móc bezpiecznie podzielić się tą wiedzą z resztą zespołu, włącznie ze swoim menedżerem. Jeśli tego rodzaju uczciwa komunikacja nie będzie zalecana, to opisywane powyżej problemy przez dłuższy czas pozostaną ukryte.

Ważne jest, aby komunikacja dotycząca oceny i planowania była dwukierunkowa. Chcemy zachęcać do dialogu i dyskusji na temat planu tak, aby zawsze mieć pewność posiadania możliwie najlepszego planu (przy bieżącym poziomie wiedzy) pozwalającego na dostarczanie wartości dla firmy. Chcemy sukcesywnie udoskonalać plany na podstawie opinii i nowej wiedzy. Z tego powodu potrzebujemy dialogu na temat planów, a nie jednostronnej prezentacji.

Na koniec powinniśmy zadbać o to, aby wszyscy adresaci dobrze rozumieli przekazywany komunikat. Nie tylko powinni go usłyszeć, ale także zrozumieć. Jeśli jesteś menedżerem projektu, który przekazuje komunikat o tym, że harmonogram projektu się opóźnia, to zrób to w taki sposób, aby wszyscy dobrze to rozumieli. Jest to jedna z przyczyn, dla których zespoły Agile preferują jako formę komunikacji duże, widoczne wykresy. Jeśli w miejscu pracy mamy nie więcej niż kilka dobrze widocznych wykresów, to istnieje możliwość, że wszystkie osoby w projekcie będą rozumiały znaczenie każdego z nich. Z drugiej strony, jeśli informacja o tym, że projekt się spóźnia, jest „wyraźnie” pokazana na trzydziestej drugiej stronie czterdziestostronicowego tygodniowego raportu o stanie projektu, to jest bardzo prawdopodobne, że nikt się o tym nie dowie.

Pamiętając o tych celach, w pozostałej części rozdziału zaprezentujemy kilka wskazówek i porad na temat sposobów komunikowania o ocenach i planach.

Ogłaszanie planu

Kiedy ktoś zada pytanie o harmonogram projektu, pierwszą myślą jest zsumowanie liczby punktów przypisanych do historyjek, które mają zostać dostarczone, podzielenie tej wartości przez szacowane tempo i udzielenie odpowiedzi w stylu „Produkt będzie gotowy na 14 czerwca, czyli za 7,2 iteracji od teraz”. To źle, ponieważ sprawia wrażenie, że wiedza, na której podstawie budujemy ten plan, jest wystarczająco dokładna, aby można było formułować tego rodzaju oceny. Zawsze gdy jest to możliwe, należy dołączać do informacji o docelowym terminie dane na temat poziomu zaufania do oceny, przedział możliwych dat albo jedno i drugie. Na przykład możesz powiedzieć:

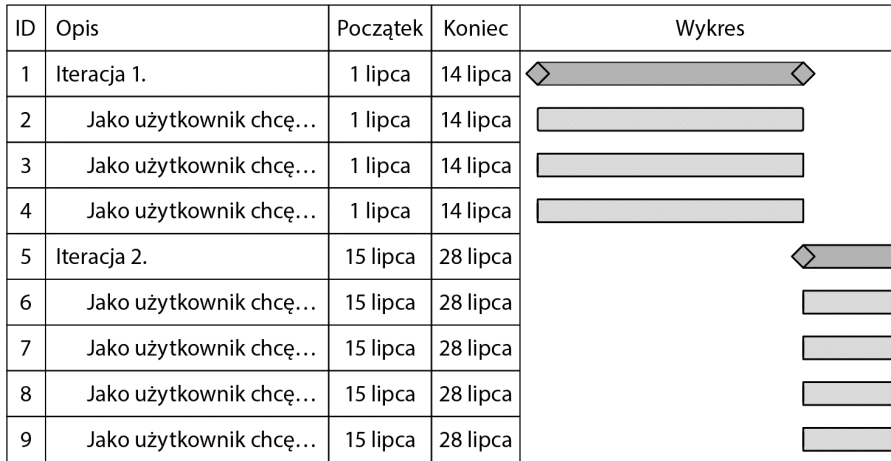
- Jestem na 90% pewien, że zaimplementujemy zaplanowaną funkcjonalność do 31 lipca.
- Biorąc pod uwagę zakładany rozmiar projektu i wydajność zespołu, realizacja projektu potrwa trzy do czterech miesięcy.

Podając przykład tego typu komunikacji Ron Jeffries (2004) z serwisu www.XProgramming.com zaproponował, aby mówić coś takiego:

Obecnie projekt jest oceniany na 200 punktów. Na podstawie wiedzy o naszej wydajności w innych projektach (lub na podstawie losowej oceny) oraz mając do dyspozycji N programistów oraz przyjmując ich pełne zaangażowanie w projekt, można założyć, że realizacja projektu o tych rozmiarach zajmie od czterech do sześciu miesięcy. Niemniej jednak będziemy dostarczać oprogramowanie co dwa tygodnie i będziemy realizować kolejne historyjki użytkownika tak, byś był zadowolony. Dobra wiadomość jest taka, że jeśli nie będziesz zadowolony, możesz

wstrzymać projekt. Oto jeszcze lepsza wiadomość: „Jeśli będziesz zadowolony, zanim wszystkie funkcjonalności zostaną zaimplementowane, także możesz zatrzymać projekt”. Zła wiadomość jest taka, że musisz pracować razem z nami, aby dla wszystkich było jasne, co oznaczają Twoje warunki satysfakcji. Najlepsza wiadomość jest z kolei taka, że za każdym razem, gdy produkt będzie zawierał wystarczająco dużo funkcjonalności, aby stać się użyteczny, możesz poprosić nas o przygotowanie go do wdrożenia, a my to zrobimy. W miarę osiągania postępów wszyscy zobaczymy, jak szybko idziemy do przodu, a oszacowania potrzebnego czasu będą się poprawiały. W każdym razie zobaczysz, co się dzieje, dostrzeżesz konkretne dowody na to, że oprogramowanie przechodzi testy, które podasz, i będziesz wiedzieć wszystko natychmiast po tym, kiedy ja się o tym dowiem.

Niektóre firmy są przyzwyczajone do prezentowania harmonogramów projektu za pomocą znajomego wykresu Gantta, którego przykład pokazano na rysunku 21.1. Wykresy Gantta zyskały złą reputację, a to ze względu na częstość ich wykorzystywania do prognozowania, tworzenia harmonogramów i koordynacji zadań. Pomimo problemów z ich używaniem wykresy Gantta mogą być doskonałym narzędziem prezentowania przydziału funkcjonalności do iteracji.



RYSUNEK 21.1. Prezentacja zawartości iteracji na wykresie Gantta

Jest kilka drobnych, ale kluczowych różnic między tym, co pokazano na rysunku 21.1, a wykresem Gantta w bardziej tradycyjnej postaci. Po pierwsze, na rysunku 21.1 zatrzymano się na poziomie funkcjonalności i nie dokonano dekompozycji każdej historyjki użytkownika na składowe zadania. To, co zaprezentowano, jest zatem *strukturą podziału funkcjonalności* (ang. *feature breakdown structure* — **FPS**), a nie strukturą podziału pracy w projekcie. Ponieważ to ukończone funkcjonalności, a nie zrealizowane zadania składowe, które tworzą funkcjonalność, przynoszą wartość w produkcie, na wykresie Gantta zaprezentowano funkcjonalności.

Po drugie, każdą funkcjonalność zaprezentowano tak, aby zajmowała całą iterację, w której została zaplanowana. Funkcjonalność może zostać zrealizowana w połowie iteracji, ale nie jest dostarczana przed zakończeniem iteracji, dlatego to zostało uwidocznione na wykresie Gantta.

Po trzecie, na rysunku 21.1 nie zaprezentowano przydziału zasobów. Ponieważ za dostarczenie wszystkich funkcjonalności jest odpowiedzialny cały zespół, nie ma sensu umieszczania imienia Marii

przy jednej funkcjonalności, a imienia Władysława obok innej. Oczywiście w przypadku tworzenia wykresu Gantta prezentującego pracę kilku zespołów, może być przydatne wskazanie odpowiedzialności całego zespołu przez przypisanie funkcjonalności (lub całych iteracji) pierwszemu zespołowi, drugiemu zespołowi i tak dalej.

Informowanie o postępach

Naturalnym, podstawowym sposobem informowania o postępach w projekcie są wykresy wypalania omówione w rozdziale 19. „Monitorowanie planu wydania”. Postępy na wykresie wypalania wydania to funkcja ilości pozostałej pracy oraz tempa pracy zespołu. Prostym sposobem na prognozowanie liczby pozostałych iteracji jest podzielenie liczby pozostałych punktów do zaimplementowania przez tempo pierwszego zespołu, a następnie zaokrąglenie uzyskanego wyniku do najbliższej liczby całkowitej. Jeśli zatem do zrealizowania pozostało sto punktów, a tempo zespołu wynosi dziesięć, to moglibyśmy powiedzieć, że pozostało dziesięć iteracji.

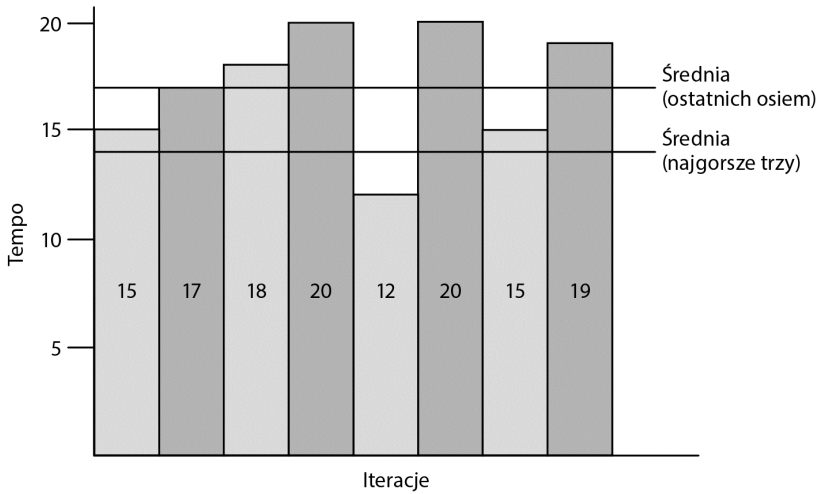
Jednak wskaźniki tempa są niedokładne i można oczekiwać, że tempo będzie się wahać. Jeśli zespół oszacował swoje średnie tempo na dziesięć, to nie jest wykluczone, że w ciągu kolejnych kilku iteracji średnie tempo wyniesie dziewięć lub jedenaście. W rzeczywistości nie jest wykluczone, że w ciągu najbliższych kilku iteracji tempo wyniesie siedem lub trzynaście. W takich przypadkach do końca projektu może pozostać od ośmiu do piętnastu iteracji. Przy prognozowaniu liczby pozostałych iteracji zwykle lepiej korzystać z zakresu możliwych temp.

Jeden z możliwych sposobów wyboru zakresu temp pokazano na rysunku 21.2, na którym przedstawiono tempo zespołu w ciągu poprzednich ośmiu iteracji. Tempo w ostatniej iteracji wynosiło dziewiętnaście. Jednak średnie tempo w ciągu ostatnich ośmiu iteracji to siedemnaście, a średnia z najgorszych trzech iteracji tego zespołu wynosi czternaście. Każda z tych wartości prowadzi do różnych oczekiwań związanych z tym, kiedy zaplanowane obecnie prace zostaną zakończone. Dlatego często lepiej wykorzystywać więcej niż jedną wartość tempa i prezentować wnioski w postaci zakresu prawdopodobnych wyników.

Przy wyborze zakresu wartości temp zwykle biorę pod uwagę dane maksymalnie z ośmiu ostatnich iteracji. Wiele zespołów przez tak długi czas pracuje razem, ale sięganie pamięcią wstecz wcześniej niż osiem iteracji wydaje się dla mnie równie odległe, jak sięganie do starożytności. Jeśli nie masz danych z ośmiu iteracji, korzystaj z tego, co masz. Na podstawie danych z tych poprzednich iteracji staram się wyznaczyć trzy wartości.

1. Tempo ostatniej iteracji.
2. Średnie tempo.
3. Średnie tempo z trzech najwolniejszych iteracji.

Te trzy wartości pozwalają uzyskać dobry obraz tego, co właśnie się wydarzyło, „długoterminowej” średniej oraz najgorszego przypadku, który mógłby się wydarzyć. Osobiście używam tych trzech różnych wartości temp, aby przewidzieć w przybliżeniu ilość pracy, jaką można zrealizować do podanej daty. Na przykład gdybyśmy chcieli opublikować produkt za pięć iteracji, a ostatnie tempo zespołu wynosiło dziewiętnaście, to moglibyśmy przewidzieć, że zespół może uzyskać kolejne dziewięćdziesiąt pięć punktów. Podobne obliczenia można wykonać dla innych miar tempa. W ten sposób możemy uzyskać zakres prawdopodobnej ilości pracy, jaką może wykonać zespół. Zazwyczaj sumuję te wyniki tak, jak pokazano w tabeli 21.1.



RYSUNEK 21.2. Tempa zmierzone w ciągu ostatnich ośmiu iteracji

TABELA 21.1. Oczekiwana liczba punktów na podstawie temp z rysunku 21.2

Opis	Tempo	Liczba iteracji	Razem punktów
Najgorsze trzy	14	5	70
Ostatnie osiem	17	5	85
Ostatnia	19	5	95

Choć staram się obserwować trendy w tempie, to w wielu projektach zostało zrealizowanych zbyt mało iteracji, aby trendy zmian tempa stały się wyraźne albo istotne. Jeśli zauważę tendencję do wzrostu tempa, to cieszę się, ale nie wyciągam wniosków na tej podstawie. Na przykład nigdy nie narysowałem linii trendu z rysunku 21.2 i nie prognozowałem, że w następnej iteracji tempo zespołu wyniesie dwadzieścia. Na podobnej zasadzie, gdyby okazało się, że tempo zaczyna spadać, to raczej chciałbym wyeliminować przyczynę spadku, niż starać się dostosować plan z założeniem, że tempo będzie spadało nadal.

Obliczenie trzech oczekiwanych wartości temp (tak jak pokazano w tabeli 21.1) pozwala właścicielowi produktu i zespołowi na prognozowanie ilości pracy, jaka może zostać zrealizowana do zaplanowanej daty wydania. Na przykład na podstawie tabeli 21.1 właściciel produktu powinien móc spokojnie ocenić, że podczas kolejnych pięciu iteracji uda się zrealizować historyjki na co najmniej siedemdziesiąt punktów. Właściciel produktu nie powinien jednak zakładać, że uda się zrobić więcej niż osiemdziesiąt pięć punktów.

Podsumowanie na końcu iteracji

Co prawda sugerowanie napisania raportu z podsumowaniem na koniec iteracji może się wydawać sprzeczne z metodyką Agile, ale prawie wszyscy menedżerowie, z którymi rozmawiałem, pytali mnie, czy robię takie raporty. Kiedy odpowiadałem twierdząco, prawie zawsze proszono mnie o podzielenie się

szablonem, który stosuję. Musisz sam zdecydować, czy chcesz poświęcić czas na wykonanie takiego raportu. Z moich doświadczeń wynika, że utworzenie takiego krótkiego dokumentu podsumowującego nie zajmuje więcej niż trzydzieści minut na iterację. Jestem zwolennikiem dwutygodniowych iteracji dla większości projektów, zatem sporządzenie raportu zajmuje mi zaledwie piętnaście minut tygodniowo.

W poniższych punktach zaprezentowano przykładowy raport z podsumowaniem iteracji dla projektu SwimStats. Zwykle zamieszczam większość z tych informacji w większości projektów. Można jednak swobodnie usuwać punkty lub dodawać własne w miarę potrzeb.

Kontekst

Daty

Pierwszy dzień iteracji	1 września
Ostatni dzień iteracji	14 września
Liczba dni roboczych	9

Zespół

Podczas iteracji były dostępne następujące osoby. Wyszczególniono także liczbę oczekiwanych dni pracy dla każdego członka zespołu oraz liczbę faktycznych dni pracy.

Imię	Dni planowanych	Dni przepracowanych
Karina	9	9
Władysław	9	7
Stanisław	8	9
Daniel	9	9
Razem	35	34

Dni robocze

Ze względu na to, że święta zmniejszają liczbę dni roboczych w iteracji, liczby zaplanowanych dni roboczych w poszczególnych iteracjach mogą się różnić między sobą. W tym przykładzie Władysław pracował dwa dni mniej niż planowano. Być może miał zwolnienie lekarskie. Z drugiej strony Stanisław pracował o jeden dzień więcej niż planowano. Być może miał zaplanowany dzień urlopu, którego ostatecznie nie wziął.

Wskaźniki

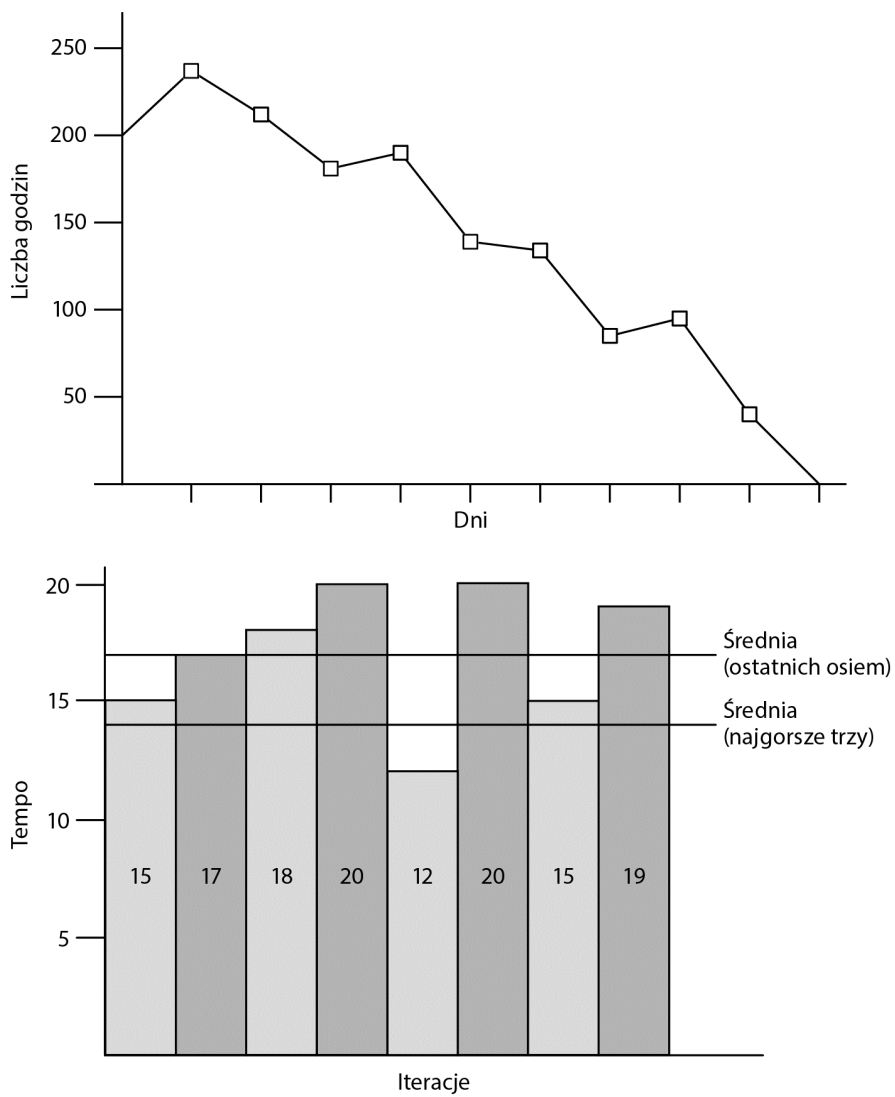
Wyniki nocnych kompilacji

Data	Stan
Poniedziałek, 6 czerwca	Kompilacja się nie powiodła.
Wtorek, 7 czerwca	812 testów jednostkowych przeszło. 1431 testów FitNesse przeszło. 104 testy interfejsu użytkownika przeszły.
Środa, 8 czerwca	826 testów jednostkowych przeszło. 1433 testy FitNesse przeszły. 104 testy interfejsu użytkownika przeszły.
Czwartek, 9 czerwca	1 test jednostkowy nie przeszedł.
Piątek, 10 czerwca	841 testów jednostkowych przeszło. 1442 testy FitNesse przeszły. 105 testów interfejsu użytkownika przeszło.
...	...

Kolorowanie wierszy

Wiersze w powyższej tabeli zazwyczaj się koloruje: kolor zielony oznacza pomyślną kompilację, natomiast czerwony kompilację z jakiegoś powodu zakończoną niepowodzeniem. Zwróćmy uwagę, że w kolumnie *Stan* wyszczególniono liczbę testów, które przeszły tylko wtedy, kiedy wszystkie testy przeszły pomyślnie. Jeśli jakikolwiek test zakończy się niepowodzeniem, to zostanie podana wyłącznie liczba nieprzechodzących testów. Jeśli jeden test się nie powiedzie, to jest nieważne, ile testów przejdzie. Raportowanie o tym, że przechodzi 99 spośród 100 testów prowadzi do myślenia, że ponieważ przechodzi 99% testów, to wykonujemy dobrą robotę. Chciałbym unikać ulegania tego rodzaju pokusom, dlatego albo podaję liczbę testów, które przechodzą (jeśli przeszły wszystkie), albo liczbę testów, które nie przechodzą.

Wykres wypalania iteracji



Treść i oszacowania

Historyjka	Wynik	Punkty planowane	Punkty zdobyte
Jako trener mogę wpisywać nazwiska i dane osobowe wszystkich pływaków z mojej drużyny.	Zakończona	8	8
Jako trener mogę definiować sesje treningowe.	Zakończona; większa niż planowano	8	8
Jako pływak mogę zobaczyć wszystkie swoje czasy w określonych zawodach.	Zakończona	3	3
Jako pływak mogę aktualizować swoje dane osobowe.	Rozpoczęta; niezakończona	1	0
Razem		20	19

Przegląd iteracji

Spotkanie poświęcone przeglądowi iteracji przeprowadzono 15 września o godzinie 9.00.

Działanie	Osoba odpowiedzialna
Marek zauważył, że większość znanych mu trenerów stosuje dzienniki planowania sesji treningowych, w których informacje są prezentowane w odwrotnej kolejności niż kolejność prezentowana przez nas na ekranie. Musimy zebrać opinie o tych ekranach od kilku trenerów.	Karina
Dariusz zasugerował, aby dodać historyjkę dotyczącą tworzenia wykresu wyników pływaka w zawodach.	Karina

Podsumowanie

Chcemy, aby informacje dotyczące ocen i planów były prezentowane często, rzetelnie oraz by były komunikowane w dwie strony. Przydatnym narzędziem do przekazywania informacji na temat planu może być wykres Gantta. Nie należy go jednak rozbijać poniżej poziomu funkcjonalności, które na wykresie Gantta powinny być prezentowane jako realizowane przez cały czas trwania iteracji.

Podstawowym sposobem informowania o postępach projektu są wykresy wypalania. Często towarzyszą im wykresy pokazujące tempo zespołu deweloperów w iteracji. Warto myśleć o tempie jak o przedziale wartości, a nie jak o pojedynczej liczbie. Dobrym sposobem wyznaczania tego przedziału jest użycie tempa z ostatniej iteracji, średniej z poprzednich ośmiu iteracji oraz średniej z najgorszych trzech spośród poprzednich ośmiu iteracji. Te trzy wartości pozwalają zaprezentować dobry obraz tego, co się niedawno wydarzyło, długoterminową średnią oraz najgorszy przypadek tego, co może się wydarzyć.

W niektórych projektach przydaje się opracowywanie raportu podsumowującego iterację — zarówno jako mechanizm rozpowszechniania bieżących informacji, jak i jako historyczny dokument do wykorzystania w przyszłości.

Pytania do dyskusji

1. Załóżmy, że w projekcie pozostało do zrealizowania 150 punktów. W ciągu ostatnich 10 iteracji zespołowi udało się osiągnąć tempa 10, 12, 13, 5, 14, 7, 6, 12, 16 i 14. Zadano Ci pytanie o to, kiedy projekt się zakończy. Co na to odpowiesz?
2. Czy termin w Twoim bieżącym projekcie jest określony w postaci konkretnej daty (18 września), czy jako zakres? Dlaczego?
3. W jakim zakresie iteracji lub dat zakończy się Twój bieżący projekt?

CZĘŚĆ VI

Dlaczego planowanie Agile jest skuteczne?

Jeśli dotąd nie jest jasne, dlaczego szacowanie i planowanie Agile są skuteczne, to w idealnym przypadku wkrótce stanie się jasne. Część VI, która obejmuje jeden rozdział, koncentruje się wyłącznie na odpowiedzi na pytanie, dlaczego szacowanie i planowanie Agile się sprawdzają. W rozdziale tym krótko przypomnę, jaki jest cel planowania, po czym przeanalizuję kilka najważniejszych powodów, dla których szacowanie i planowanie Agile pozwalają skutecznie osiągnąć ten cel.

Rozdział kończy się zbiorem kilkunastu zaleceń dotyczących stosowania technik szacowania i planowania Agile w Twoich projektach.

ROZDZIAŁ 22.

Dlaczego planowanie Agile się sprawdza?

Jeśli chcesz gwarantowanego działania, kup sobie toster.

— Clint Eastwood w filmie *Debiutant*

Jeśli dotarłeś do tego punktu, to jesteś dobrze przygotowany, aby przyjrzeć się, dlaczego szacowanie i planowanie Agile są skuteczne. Przede wszystkim należy pamiętać o celu szacowania i planowania. Planowanie jest próbą znalezienia optymalnej odpowiedzi na ogólne pytania związane z rozwojem produktu: funkcjonalnościami, zasobami i harmonogramem. Zmiana któregokolwiek z tych elementów powoduje zmiany w pozostałych. Przy planowaniu odkrywamy całe spektrum możliwych połączeń tych elementów tak, aby stworzyć możliwie najlepszy produkt. Oceny i plany, które tworzymy, powinny być wystarczające do tego, aby mogły stanowić podstawę do podejmowania decyzji biznesowych. W tym rozdziale przyjrzymy się procesowi szacowania i planowania Agile opisanemu dotychczas w tej książce, aby przekonać się, w jaki sposób stosowanie tych technik przyczynia się do osiągnięcia wymienionych celów.

Zmiany planów zdarzają się często

Częste modyfikacje planów to jeden z mechanizmów należących do technik planowania i szacowania Agile, który wspomaga skuteczne odkrywanie przestrzeni rozwiązań podczas tworzenia nowego produktu. Na początku każdej nowej iteracji tworzony jest jej plan. Po każdej iteracji, a w najgorszym przypadku raz na kilka iteracji aktualizowany jest plan wydania. Uznanie, że utworzenie doskonałego planu jest niemożliwe, znacznie przyczynia się do zminimalizowania uczucia niepokoju, które towarzyszy próbom osiągnięcia takiego celu (Githens 1998). Świadomość, że na początku następnej iteracji plan może zostać zmieniony, przesunąć koncentrację zespołu z prób tworzenia doskonałego planu (zadanie niemożliwe do wykonania) na tworzenie planu, który jest przydatny od razu.

Aby plan mógł być przydatny, musi być dokładny, ale godzimy się na to, że pierwsze plany są niedokładne. Jednym z powodów zmiany planów jest sukcesywne usuwanie tych niedokładności. Na przykład w pierwszej wersji planu możemy zakładać dostarczenie od 300 do 400 punktów w trzecim

kwartale. Może się później okazać, że ten plan był dokładny (w sierpniu dostarczono 305 punktów), ale nie jest szczególnie precyzyjny. Na wczesnych etapach projektu taki plan prawdopodobnie jest wystarczającą podstawą do podejmowania decyzji. Później jednak, aby plan pozostał przydatny, musi zyskać precyzję. Wtedy będziemy mówić, że we wrześniu dostarczymy od 380 do 400 punktów.

Stosując proces szacowania i planowania Agile, przyznajemy, że nasza wiedza zawsze jest niepełna i wymaga korekty planów wraz ze zdobywaniem nowych informacji. W czasie gdy zespół zdobywa nową wiedzę o tworzonej produkcie, do planu wydania mogą być dodawane nowe funkcjonalności. Wraz ze zdobywaniem nowych umiejętności w posługiwaniu się stosowanymi technologiami oraz pracy zespołowej korygowane są oczekiwania dotyczące tempa osiągnięcia postępów. Aby plan nadal był przydatny, trzeba w nim uwzględnić tę nową wiedzę.

Oceny rozmiaru i czasu trwania są oddzielne

Popularnym błędem podczas planowania (popelnianym zarówno przez zespoły stosujące techniki tradycyjne, jak i Agile) jest mylenie ocen rozmiaru z czasem trwania. To oczywiste, że nakłady pracy oraz czas potrzebny do jej zrealizowania są ze sobą związane, ale na czas trwania projektu wpływa wiele dodatkowych czynników. Realizacja projektu o określonym rozmiarze zajmie programistom o różnym poziomie kwalifikacji i doświadczenia różną ilość czasu. Na podobnej zasadzie na czas trwania projektu wpływa rozmiar zespołu pracującego nad projektem. Czteroosobowemu zespołowi realizacja projektu może zająć sześć miesięcy (dwadzieścia cztery osobomiesięce). Zespół ośmioosobowy może skrócić ten czas do czterech miesięcy kalendarzowych, ale to będzie oznaczało trzydzieści dwa osobomiesięce, choć w obu przypadkach projekt ma ten sam rozmiar.

Aby zaobserwować różnice między ocenami rozmiaru i czasu trwania, założmy, że pokazałem Ci książkę i zapytałem, jak dużo czasu zajmie Ci jej przeczytanie. Na podstawie tytułu możesz stwierdzić, że to jest powieść, ale nie pozwolę Ci zajrzeć do środka książki, abyś nie mógł zobaczyć, ile jest w niej stron, jak szerokie są marginesy lub jak mały jest druk. Aby móc odpowiedzieć na moje pytanie, jak długo będziesz czytać tę książkę, w pierwszej kolejności oceniasz liczbę stron. Założmy, że powiedziałeś 600. Następnie oceniasz swoje tempo czytania na jedną stronę na minutę. Odpowiadasz, że przeczytanie książki zajmie Ci 600 minut, czyli 10 godzin. Aby podać tę ocenę czasu trwania (10 godzin), najpierw oceniłeś rozmiar zadania (600 stron).

Szacowanie i planowanie Agile jest skuteczne, ponieważ oceny rozmiaru i czasu trwania są od siebie oddzielone. Zaczynamy od oceny w punktach rozmiaru historyjek użytkownika wchodzących w skład projektu. Ponieważ punkty są abstrakcyjne i umowne, są czystymi ocenami rozmiaru. Później oceniamy szybkość osiągnięcia postępów, którą nazywamy tempem. Następnie łączymy oceny rozmiaru i tempa, by na tej podstawie wyznaczyć ocenę czasu trwania. Na tym czytelnym i całkowitym oddzieleniu ocen rozmiaru od czasu trwania korzysta proces szacowania i planowania.

Plany sporządza się na różnych poziomach

Ponieważ plany Agile obejmują trzy różne poziomy — wydanie, iterację i bieżący dzień — to każdy z tych planów może być wykonany z różnym stopniem dokładności. Stosowanie planów o różnych horyzontach czasowych i na różnych poziomach dokładności przynosi dwie główne korzyści. Przede wszystkim pokazuje rzeczywistość, zgodnie z którą różne plany są tworzone z różnych powodów. Plan

dzienny, do którego zobowiązuje się każdy uczestnik codziennego spotkania zespołu, jest dość dokładny: członkowie zespołu wyrażają zobowiązania do wykonania w całości lub co najmniej rozpoczęcia realizacji konkretnych zadań w ciągu dnia. Plan iteracji jest mniej dokładny — wyszczególnia historyjki użytkownika, które zostaną zrealizowane podczas iteracji, oraz zadania niezbędne do zrealizowania tych historyjek. Ponieważ każda historyjka użytkownika jest zdefiniowana niedokładnie, istnieją również pewne niejasności co do tego, co to znaczy, że w czasie trwania iteracji historyjka zostanie zrealizowana. Wreszcie, plan wydania jest najmniej dokładny. Zawiera jedynie uporządkowaną według priorytetów listę pożądanых historyjek użytkownika oraz jedną lub więcej ocen opisujących, jaka część pożądaney funkcjonalności najprawdopodobniej zostanie dostarczona do pożądaney daty wydania.

Drugą zaletą planowania na różnych poziomach jest to, że dzięki temu zespół może obserwować projekt z różnych punktów widzenia. Plan iteracji jest konieczny do wykonania wysoce skoordynowanych działań interpersonalnego zespołu w czasie krótkiej iteracji. Plan wydania zapewnia szersze spojrzenie na projekt i daje pewność, że zespół nie straci z oczu lasu, patrząc na pojedyncze drzewa iteracji. Zespół, który pracuje od iteracji do iteracji bez świadomości bardziej odległego celu, ryzykuje, że będzie ciągle podążał za celami krótkoterminowymi, tracąc z oczu wysoce lukratywny cel długoterminowy. Cele krótkoterminowe mogą być niespójne z pożądanym, długoterminowym rezultatem.

Podstawą planów są funkcjonalności, a nie zadania

Tradycyjne plany w postaci wykresu Gantta, wykresu PERT lub struktury podziału pracy koncentrują się na zadaniach niezbędnych do zrealizowania w celu stworzenia produktu. Natomiast plan Agile koncentruje się na funkcjonalnościach dostarczanych przez produkt. Jest to zasadnicza różnica, ponieważ zmusza to zespół do myślenia o produkcie na właściwym poziomie — pod kątem funkcjonalności. Jeśli iteracyjnie są dostarczane funkcjonalności, to istnieje mniejsza potrzeba myślenia z góry o konkretnych zadaniach, które trzeba zrealizować. Pojawiają się prace do wykonania podczas iteracji i zespół definiuje wszystkie zadania, które będą potrzebne do ich zrealizowania. Jeszcze ważniejsze jest to, że zespół myśli o funkcjonalnościach, które są rozwijane. Jim Highsmith (2004b) stwierdził, że „planowanie Agile jest »lepsze«, ponieważ wykorzystuje funkcjonalności (historyjki i tym podobne), a nie zadania. Łatwo zaplanować cały projekt, posługując się standardowymi zadaniami bez faktycznego zrozumienia tworzonego produktu. Przy planowaniu według funkcjonalności zespół ma znacznie lepsze rozumienie produktu”. Na poziomie zadań wiele planów projektów wygląda tak samo; każdy plan Agile jest specyficzny dla tworzonego produktu.

Dzięki niewielkim historyjkom prace postępują

Z teorii masowej obsługi (Poppendieck i Poppendieck 2003; Reinertsen 1997) wiemy, jak ważne jest skoncentrowanie się na *czasie cyklu* — ilości czasu potrzebnego do przejścia od początku określonego procesu do jego końca. W projekcie oprogramowania czas cyklu to czas od chwili, kiedy zespół zaczyna pracować nad funkcjonalnością, do momentu, kiedy ta funkcjonalność dostarczy wartość dla użytkowników. Im krótszy czas cyklu, tym lepiej.

Kluczowy wpływ na czas cyklu ma zmienność czasu potrzebnego na realizację nowej funkcjonalności. Jednym z najlepszych sposobów na zminimalizowanie zmienności jest praca z wykorzystaniem stosunkowo niewielkich jednostek pracy o podobnych rozmiarach. W procesach szacowania

i planowania opisanych w tej książce uwzględniono ten wymóg za pomocą porady, aby zespoły szacowały krótkoterminowe zadania, które w przybliżeniu mają taki sam rząd wielkości. Większe historyjki użytkownika mogą występować na dalszych miejscach uporządkowanej według priorytetów listy wymagań projektu. Jednakże w miarę, jak funkcjonalności opisywane przez historyjki użytkownika zbliżają się do szczytu listy (co sprawia, że zostaną zaplanowane w iteracji, która właśnie się rozpoczyna), są dzielone na mniejsze fragmenty.

W każdej iteracji eliminujemy pracę w toku

Duża ilość pracy w toku spowalnia zespół. W projekcie oprogramowania praca w toku odnosi się do wszystkich funkcjonalności, nad którymi zespół rozpoczął pracę, ale których jeszcze nie zakończył. Im więcej pracy w toku, tym więcej czasu zajmie implementacja funkcjonalności, ponieważ prace nad nowymi funkcjonalnościami muszą czekać na zakończenie prac, które już się rozpoczęły. Może się również zdarzyć, że nowe funkcjonalności muszą być wykonane priorytetowo przed zadaniami, które są już realizowane. To jednak tylko komplikuje pracę nad następną funkcjonalnością, której nie można przyspieszyć. Tak więc prace w toku powodują wydłużenie czasu cyklu. Jak pamiętamy z poprzedniego podrozdziału, utrzymywanie krótkiego czasu cyklu jest bardzo ważne.

Jednym z powodów, dla których planowanie Agile jest skuteczne, jest fakt, że na końcu każdej iteracji następuje wyeliminowanie pracy w toku. Ze względu na to, że praca nie przechodzi automatycznie z jednej iteracji do następnej, każda iteracja jest planowana od początku. Oznacza to, że prace nad funkcjonalnością, która nie została w pełni zaimplementowana w jednej iteracji, niekoniecznie będą kontynuowane w następnej iteracji. Często tak będzie, ale nie jest to w żaden sposób gwarantowane. W efekcie następuje wyeliminowanie całej pracy w toku na początku każdej iteracji.

Dzięki wyeliminowaniu pracy w toku na początku każdej iteracji zespołom jest łatwiej wydajnie pracować w krótkich iteracjach. Pozwala to znacznie skrócić pętlę sprzężenia zwrotnego od użytkowników do zespołu projektowego, co prowadzi do szybszego uczenia się, a także skuteczniejszego łagodzenia zagrożeń i większej kontroli.

Śledzenie odbywa się na poziomie zespołu

W tradycyjnym podejściu do oceny i planowania działań postępy są mierzone i wynagradzane na poziomie indywidualnych członków zespołu. To prowadzi do szeregu problemów. Na przykład jeśli z powodu wcześniejszego zakończenia zadania programistę oskarża się o zawyżenie oceny pracowitości zadania, to ten programista nauczy się, aby nie kończyć pracy wcześniej. Zamiast skończyć wcześniej, będzie wstrzymywał się ze zgłoszeniem, że zadanie zostało ukończzone, aż nadejdzie termin.

Stosowanie procesu szacowania i planowania Agile skutecznie zapobiega tego rodzaju problemom dzięki śledzeniu postępów wyłącznie na poziomie zespołu. Jest to jeden z powodów, dla których w rozdziale 14. „Planowanie iteracji” zamieszczono wskazówkę, aby podczas planowania iteracji indywidualni członkowie zespołu wstrzymali się od zgłaszania się do konkretnych zadań. Z podobnych powodów nie przygotowuje się wykresów wypalania dla pojedynczych członków zespołu, a jedynie dla całego zespołu.

Niepewność jest potwierdzona i zaplanowana

W wielu tradycyjnych metodach planowania „z góry” popełniany jest błąd polegający na założeniu, że funkcjonalność można zablokować na początku projektu. Następnie sporządzane są plany, aby nie dopuszczać do zmian lub aby zmiany były realizowane za pośrednictwem uciążliwego procesu zarządzania zmianami. Takie podejście prowadzi do realizowania projektów zawierających funkcjonalności, których użytkownicy nie chcą. Kiedy stworzymy plan we wczesnej fazie projektu i nie będziemy go aktualizować w miarę zdobywania nowej wiedzy, to stracimy możliwość synchronizacji planu z rzeczywistością.

Dzięki stosowaniu podejścia Agile do oceny i planowania zespoły potwierdzają zarówno niepewność celu, jak i środków. Niepewność celu (dotycząca tworzonego produktu) zmniejsza się w miarę prezentowania kolejnych fragmentów produktu potencjalnym użytkownikom i innym interesariuszom na końcu każdej iteracji. Ich opinie i odpowiedzi są wykorzystywane do korekty przyszłych planów. Niepewność środków (dotycząca sposobu budowania produktu) zmniejsza się w miarę zdobywania przez zespół nowej wiedzy o stosowanych technologiach oraz o sobie samych. Dla przykładu wczesne odkrycie, że określony komponent dostarczany przez zewnętrzną firmę nie spełnia wymagań wydajności, może doprowadzić zespół do podjęcia decyzji, aby znaleźć alternatywę. Czas na znalezienie nowego komponentu i przełączenie się na niego powinien być uwzględniony w nowych planach natychmiast po zidentyfikowaniu takiej potrzeby.

Wytyczne dla procesu szacowania i planowania Agile

Poniżej zamieszczono listę kilkunastu wytycznych skutecznego szacowania i planowania Agile uwzględniającą informacje opisane w tej książce.

1. **Zaangażuj cały zespół.** Główną odpowiedzialność za określone działania może ponosić jedna osoba lub grupa. Zgodnie z tą zasadą nadawanie priorytetów wymaganiom to przede wszystkim odpowiedzialność właściciela produktu. Niemniej jednak w realizację projektu musi być zaangażowany cały zespół, który powinien dążyć do stworzenia końcowego produktu o jak najwyższej wartości. Uwzględniono to na przykład we wskazówce, aby oceny były dokonywane przez cały zespół, nawet jeśli jest oczywiste, że tylko jeden lub dwóch konkretnych członków zespołu będzie pracować nad szacowaną historijką lub zadaniem. Im więcej wspólnych obowiązków zespołu, tym większy wspólny sukces całego zespołu.
2. **Planuj na różnych poziomach.** Nie wolno popełniać błędu polegającego na myśleniu, że tworzenie planu wydania zwalnia z obowiązku tworzenia planu iteracji lub odwrotnie. Każdy z planów: wydania, iteracji i dnia opisuje inny horyzont czasowy, jest tworzony na innym poziomie dokładności i każdy z nich służy wyjątkowemu celowi.
3. **Wykonuj oceny rozmiaru i czasu trwania osobno poprzez stosowanie różnych jednostek.** Najlepszym sposobem na utrzymanie czytelnego rozdzielenia ocen rozmiaru od czasu trwania jest korzystanie z różnych jednostek, których nie można pomylić. Doskonałym sposobem, aby to zrobić, jest ocena rozmiaru historyjek w punktach i konwersja ich na czas trwania z wykorzystaniem tempa.
4. **Wyrażaj niepewność za pomocą funkcjonalności albo daty.** Żaden plan nie jest pewny. Pamiętaj o uwzględnieniu niepewności w każdym planie wydania, który tworzysz. Jeśli zakres nowych funkcjonalności jest stały, sformułuj niepewność w postaci zakresu dat („skończymy

w trzecim kwartale” albo „skończymy w ciągu od siedmiu do dziesięciu iteracji”). Jeśli natomiast jest stała data, to wyraż niepewność co do funkcjonalności, która ma być dostarczona („wykonamy to na 31 grudnia, a produkt będzie uwzględniał co najmniej następujące nowe funkcjonalności, ale prawdopodobnie nie więcej niż te nowe funkcjonalności”). Gdy stopień niepewności jest wyższy, używaj większych jednostek (na przykład iteracji, miesięcy, a następnie kwartałów).

5. **Koryguj plany często.** Wykorzystaj początek każdej nowej iteracji, aby oszacować ważność bieżącego planu wydania. Jeśli plan wydania bazuje na przestarzałych informacjach lub założeniach, o których teraz wiesz, że są fałszywe, zaktualizuj go. Wykorzystaj możliwości korygowania planów, by zyskać pewność, że celem projektu zawsze jest dostarczanie firmie jak największej wartości.
6. **Śledź postępy i informuj o nich.** Wielu interesariuszy projektu jest bardzo zainteresowanych jego postępami. Informuj ich poprzez regularne publikowanie prostych, zrozumiałych wskaźników postępów zespołu. Najlepiej nadają się do tego wykresy wypalania oraz inne wskaźniki umożliwiające dokonanie oceny postępów w projekcie w mgnieniu oka.
7. **Pamiętaj o znaczeniu uczenia się.** Ponieważ projekt w równym stopniu zarówno generuje nową wiedzę, jak i wprowadza nowe funkcjonalności do produktu, to należy aktualizować plany tak, aby uwzględniały tę nową wiedzę. W miarę jak dowiadujemy się coraz więcej o potrzebach naszych klientów, do projektu są dodawane nowe funkcjonalności. Kiedy dowiadujemy się więcej o używanych technologiach albo o tym, jak dobrze pracujemy jako zespół, korygujemy oczekiwania dotyczące naszego tempa pracy i pożądanego podejścia.
8. **Planuj funkcjonalności o odpowiednich rozmiarach.** Funkcjonalności, które zostaną dodane w najbliższej przyszłości (w ciągu najbliższych kilku iteracji), należy zdekomponować na stosunkowo niewielkie historyjki użytkownika — zazwyczaj będą to zadania, których realizacja zajmie jeden lub dwa dni, ale nie więcej niż dziesięć dni. Najlepiej oceniamy prace, które — jeśli chodzi o rozmiar — mieszczą się w obrębie jednego rzędu wielkości. Praca z historyjkami użytkownika, które mieszczą się w tym zakresie, zapewnia optymalne połączenie wysiłku z dokładnością. Dostarcza również historyjek, które są wystarczająco małe do tego, aby większość zespołów mogła je zrealizować w ciągu jednej iteracji. Oczywiście praca z małymi historyjkami użytkownika w przypadku długoterminowych projektów może stać się dość uciążliwa. W celu zapewnienia równowagi podczas tworzenia planu wydania, który obejmuje przyszłość dalszą niż dwa do trzech miesięcy, należy napisać kilka większych historyjek (zwanych *epikami*) albo szacować bardziej odległe prace na poziomie tematów. W ten sposób można uniknąć dekompozycji dużych historyjek użytkownika na małe zbyt daleko w przyszłość.
9. **Nadawaj funkcjonalnościom priorytety.** Pracuj nad funkcjonalnościami w takiej kolejności, która pozwala zoptymalizować całkowitą wartość projektu. Przy ustalaniu priorytetów, oprócz oceniania wartości i kosztów funkcjonalności, weź pod uwagę wiedzę, którą zdobywasz, oraz zagrożenia, które eliminujesz w związku z opracowaniem tej funkcjonalności. Wczesne wyeliminowanie znaczącego zagrożenia często uzasadnia szybsze tworzenie funkcjonalności. Na podobnej zasadzie, jeśli wcześniejsze stworzenie określonej funkcjonalności pozwoli zdobyć zespołowi znaczącą wiedzę o produkcie albo zminimalizować nakłady pracy potrzebne do jego stworzenia, to warto rozważyć wcześniejsze stworzenie tej funkcjonalności.
10. **Twórz oceny i plany na podstawie faktów.** W miarę możliwości opieraj swoje oceny i plany na rzeczywistości. Co prawda w niektórych firmach czasem zdarzają się przypadki, kiedy trzeba oszacować takie parametry jak tempo w sytuacji, gdy nie można zbyt mocno opierać się na faktach.

Dla tego rodzaju sytuacji w rozdziale 16. „Szacowanie tempa” zaprezentowano kilka przydatnych technik. Jednak w miarę możliwości oceny i plany powinny bazować na rzeczywistych, zaobserwowanych wartościach. Dotyczy to również oceny tego, jaka część funkcjonalności została zrealizowana. Łatwo ocenić, kiedy funkcjonalność została zrealizowana w 0% (prace nad nią jeszcze się nie zaczęły), oraz stosunkowo łatwo można stwierdzić, kiedy funkcjonalność zrealizowano w 100% (przechodzą wszystkie testy dla wszystkich warunków satysfakcji właściciela produktu). Trudno zmierzyć stan realizacji funkcjonalności w jakimkolwiek punkcie pośrednim — czy zadanie zrealizowano w 50%, czy w 60%? Ze względu na fakt, że tak trudno odpowiedzieć na to pytanie, trzeba posługiwać się wyłącznie tym, co wiemy: 0% i 100%.

11. **Pozostaw trochę luzu.** Zwłaszcza podczas planowania iteracji nie należy planować z uwzględnieniem 100% czasu każdego członka zespołu. Podobnie jak na autostradzie wypełnionej do 100% jej możliwości tworzą się korki, tak zespół projektowy spowolni tempo w sytuacji, gdy czas wszystkich członków zespołu zostanie zaplanowany do granic możliwości.
12. **Koordinuj pracę zespołów za pośrednictwem planowania wyprzedzającego.** W projektach, w których bierze udział wiele zespołów, koordynuj prace z wykorzystaniem ruchomych planów wyprzedzających. Dzięki spoglądaniu w przyszłość i przypisywaniu konkretnych funkcjonalności do konkretnych iteracji w przyszłości można zaplanować i uwzględnić zależności między zespołami.

Podsumowanie

Celem planowania Agile jest iteracyjne odkrywanie optymalnego rozwiązania ogólnych problemów dotyczących rozwoju produktu — jakie funkcjonalności, jakimi zasobami oraz w jakim czasie należy realizować. Podejście Agile do oceny i planowania w poszukiwaniu tego rodzaju rozwiązań sprawdza się dzięki temu, że plany są budowane na różnych poziomach i są często przebudowywane; plany bazują na funkcjonalnościach, a nie na zadaniach; najpierw jest szacowany rozmiar, a na jego podstawie ocenia się czas trwania; opracowuje się niewielkie historyjki użytkownika, bo dzięki nim uzyskujemy lepszy przepływ pracy; eliminujemy pracę w toku na końcu każdej iteracji; postęp jest mierzony na poziomie całego zespołu, a nie indywidualnych osób; potwierdzamy istnienie niepewności i uwzględniamy ją w planach.

Pytania do dyskusji

1. Czy przychodzą Ci na myśl inne powody, dla których szacowanie i planowanie Agile się sprawdza?
2. Która z kilkunastu wskazówek ma zastosowanie do Twojego bieżącego procesu szacowania i planowania? Czy ten proces działałby sprawniej, gdyby stosowano inne spośród wymienionych wskazówek?

CZĘŚĆ VII

Studium przypadku

Wszystko już zostało powiedziane. Jednak w jedynym rozdziale części VII chcę powiedzieć to czytelnikom jeszcze raz — tym razem w zupełnie innej formie.

Poniższy rozdział ma formę fikcyjnego opowiadania, które podsumowuje wiele kluczowych informacji z tej książki. W rozdziale zaprezentowano fikcyjną firmę Bomb Shelter Studios, która próbuje zrealizować swój pierwszy projekt Agile. W projekcie wezmą udział:

- Alek, programista;
- Karol, trener Agile;
- Daniela, analityk;
- Franek, menedżer produktu;
- Laura, dyrektor finansowy;
- Filip, dyrektor generalny;
- Przemek, tester;
- Róża, grafik;
- Staś, programista.

ROZDZIAŁ 23.

Studium przypadku: Bomb Shelter Studios

Podróż była długa, ale konferencja okazała się sukcesem. Podróże z Warszawy zawsze są męczące, lecz tym razem Franek dopłacił do pierwszej klasy, wymieniając trochę punktów uczestnika programu „częsty pasażer” na więcej miejsca i komfortu. Rozmyślając o wydarzeniach z poprzedniego tygodnia, rozsiadł się w swoim fotelu.

Jako menedżer produktu w firmie Bomb Shelter Studios, wiedział, że najnowsza gra *Deep Black & White* dobrze się sprzeda. Bazowała na grze o nazwie *Go*, która była bardzo popularna w Japonii, Chinach i Korei, ale miała niezbyt wielu zwolenników w Europie, Ameryce Północnej i reszcie świata. Programiści z jego zespołu skorzystali z technik sztucznej inteligencji, dzięki którym gra w *Deep Black & White* była możliwa na poziomie określanym jako 5. dan. Było to jeszcze daleko od poziomu 9. dan najlepszych, profesjonalnych graczy, ale znacznie dalej w porównaniu ze wszystkimi konkurentami firmy Bomb Shelter Studios.

Franek był zachwycony perspektywą wydania gry *Deep Black & White* i wprowadzenia jej na rynek azjatycki na podstawie umowy dystrybucyjnej, którą wynegocjował z wydawcą na konferencji. Przychody ze sprzedaży na tych rynkach naprawdę pomogłyby firmie Bomb Shelter. Franek wiedział, że dodatkowe sześć miesięcy, jakie zajęło ukończenie *Deep Black & White*, niemal doprowadziło do upadku małą, prywatną firmę zajmującą się tworzeniem gier, której był współzałożycielem.

Pomimo niepomyślnych początków trzy lata temu firma Bomb Shelter Studios zdobyła uznanie jako wiodący twórca gier strategicznych. Oprócz nowo ukończonej gry *Deep Black & White* firma ta opracowywała gry w szachy, backgammona, reversi, brydża, warcaby, mancalę i tym podobne. Po stworzeniu gry prawa do jej dystrybucji zostały sprzedane wydawcy, którego zadaniem było dbanie o wszystkie zadania związane z produkcją i dystrybucją. Dzięki temu firma Bomb Shelter mogła skupić się na tworzeniu nowych gier.

W czasie gdy Franek był na konferencji, jego analityk wraz z niewielkim zespołem w Santa Barbara obmyślali nową grę: *Havannah*, nad którą prace właśnie się miały rozpocząć. Na podstawie wniosków z pracy nad grą *Deep Black & White* — nie tylko z powodu sześciomiesięcznego opóźnienia, ale także późnego znalezienia zbyt wielu błędów i odkrycia pewnych problemów związanych z użytecznością — Franek wiedział, że trzeba znaleźć inny sposób planowania i prowadzenia projektów. Staś, główny

architekt w firmie, przeanalizował kilka pomysłów zgłaszanych przez zespół. Zaproponował wykorzystanie w następnym projekcie tzw. procesu Agile. Franek nie wiedział dokładnie, co to oznaczało, ale był pewien, że ten projekt musi być prowadzony inaczej. Sześć miesięcy opóźnienia przy kolejnej grze nie wchodziło w rachubę. Wszyscy w zespole byli podekscytowani perspektywą wypróbowania nowego procesu i wszyscy zdawali sobie sprawę, jaka jest stawka.

Dzień 1. — poniedziałek rano

— Dzień dobry wszystkim — powiedział Franek, wchodząc do sali konferencyjnej. Była za minutę dziewiąta i prawie cały zespół już na niego czekał. To był dobry znak. Mimo że wszyscy odczuwali zmęczenie ostatnim „atakami” na grę *Deep Black & White*, wyglądało na to, że byli gotowi na podjęcie wyzwań związanych z *Havannah*.

— Dzień dobry, Franku. Widziałem twój e-mail na temat *Deep Black & White*. Informacja o umowie dystrybucyjnej to świetna wiadomość — powiedział Alek, programista C++, twórca silnika sztucznej inteligencji, dzięki któremu gra *Deep Black & White* okazała się tak dobra.

— Poczuj się, Franku — powiedział Staś, podsuwając mu prawie puste pudełko ze słodyczami.

— Dziękuję — powiedział Franek, wyjmując batonik.

— Franku, poznaj Karola — powiedział Staś. — Karol to doświadczony trener Agile. Zaprośmy go, aby pomógł nam nauczyć się pracować zgodnie z nową „zwinną” metodą wytwarzania oprogramowania.

Franek z Karolem uściślił sobie dłonie i wymienili pozdrowienia.

— Wygląda na to, że wszyscy oprócz Róży są obecni — powiedział Franek. — Zaczynamy. Później wszystko jej opowiemy. W końcu raczej nie będziemy mówić zbyt dużo o grafice na tym spotkaniu.

— Nie możemy rozpocząć, Franku — powiedział Karol. — To bardzo ważne. W spotkaniu musi uczestniczyć *cały zespół*. Duża część korzyści, jakich oczekujemy po zastosowaniu procesu Agile, wymaga uczestnictwa wszystkich członków zespołu. Być może Róża nie ma zbyt dużo do powiedzenia na temat sztucznej inteligencji w silniku zarządzania ruchami w grze. Pomimo to potrzebujemy jej punktu widzenia, jeśli chcemy zaplanować najlepszą możliwą grę.

— W poniedziałki Róża zawsze przychodzi około pięć po dziewiątej. Odprowadza Bartka do przedszkola w poniedziałki, środy i piątki. Zaraz tu będzie — powiedział Przemek, kończąc swoją uwagę niemal dokładnie w chwili, gdy Róża otworzyła drzwi do sali konferencyjnej.

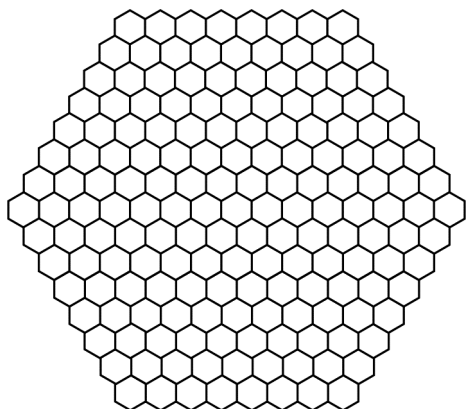
— Przepraszam, korki — powiedziała Róża, szybko siadając na krześle.

— Zatem, Danielo — Franek zwrócił się do analityczki — przeprowadziłaś analizę produktu dotyczącą gry *Havannah*. Już od jakiegoś czasu myślę o tej grze. Przepraszam, że pytam, ale czy możesz mi jeszcze raz powiedzieć, jak się w nią gra?

— Oczywiście, Franku. Po pierwsze, plansza wygląda tak — powiedziała Daniela, wyjmując drewnianą planszę z torby i kładąc ją na stole (plansza wyglądała tak jak na rysunku 23.1). — W grze bierze udział dwóch graczy, którzy na zmianę wykonują ruchy pionami na planszy. Każdy gracz posługuje się pionami w różnym kolorze. Kiedy pion znajdzie się na planszy, nie można nim ruszać.

— Tak samo jak w *Deep Black & White* — powiedział Franek.

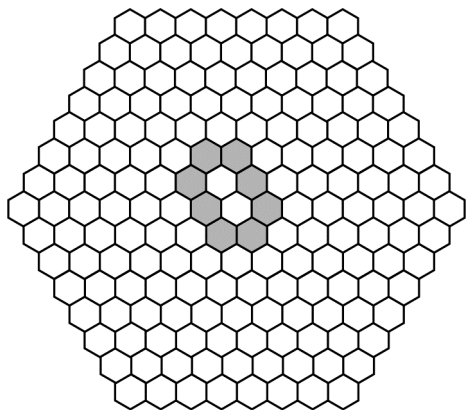
— Zgadza się, ale w odróżnieniu od gry w *Go* pionów nie można zbijać. Celem gry w *Havannah* jest wykonanie pierścienia, mostu albo rozwidlenia. Gracz, który zrobi to jako pierwszy, wygrywa.



RYСУNEK 23.1. *Pusta plansza do gry w Havannah*

— A czym są pierścienie, mosty i rozwidlenia? — zapytał Franek w czasie, gdy Daniela wzięła garść pionków i zaczęła je ustawiać na planszy.

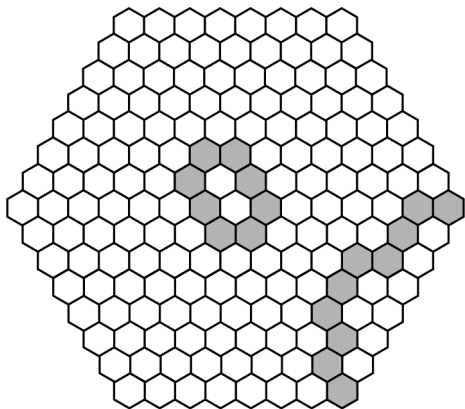
— Najprostszy jest pierścień. Wygląda tak — powiedziała Daniela i narysowała rysunek 23.2.
— Pierścień to zbiór połączonych ze sobą elementów, które okalają jeden lub więcej punktów.



RYСУNEK 23.2. *Pierścień*

— OK. To brzmi dość prosto. Co sprawia, że ta gra jest trudna? — zapytał Alek, który już myślał naprzód, zastanawiając się nad napisaniem silnika korzystającego z mechanizmów sztucznej inteligencji, który wybiera ruchy w grze.

— Pamiętaj, że stworzenie pierścienia to tylko jeden ze sposobów na wygraną. Można również wygrać, tworząc rozwidlenie lub most — kontynuowała Daniela, układając pionki w sposób pokazany na rysunku 23.3. — Most łączy dwa narożniki. Może on przebiegać w linii prostej, wzdłuż krawędzi — od jednego narożnika do narożnika sąsiedniego. Jednak jest bardziej prawdopodobne, że most będzie wyglądać mniej więcej tak — wskazała na most po prawej stronie planszy z rysunku 23.3.



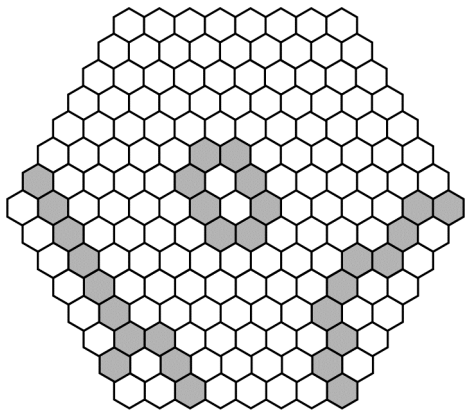
RYСУNEK 23.3. *Pierścień i most*

— Czy przed rozpoczęciem gry gracz musi zadeklarować, jaki kształt będzie starał się stworzyć?
 — zapytał Alek.

— Nie, to jest część zabawy, a jednocześnie część wyzwania. Możesz zacząć od próby stworzenia mostu, a gdy zdasz sobie sprawę, że to się nie uda, próbujesz stworzyć na przykład rozwidlenie.

— Co to jest rozwidlenie?

— To coś takiego, Franku — powiedziała Daniela, ustawiając kilka dodatkowych pionków na planszy tak jak pokazano na rysunku 23.4. — Rozwidlenie łączy trzy krawędzie, a nie narożniki. Narożniki nie są krawędziami, dlatego nie mogą być wykorzystywane w rozwidleniu tak jak w przypadku mostu.



RYСУNEK 23.4. *Zwycięskie figury w grze w Havannah: rozwidlenie, pierścień i most*

— Zaprogramowanie silnika ruchów będzie wyzwaniem. Przy tak wielu możliwościach wygranej oraz tak wielu polach na planszy istnieje mnóstwo opcji.

— Masz rację, Alku — powiedziała Daniela. — Wiele osób twierdzi, że ta gra jest trudniejsza od szachów, ponieważ jest więcej możliwości, a także dlatego, że nie możemy skorzystać z ogromnej książki zakończonych partii tak jak w przypadku szachów. Kiedy w szachach pozostanie na planszy

tylko kilka pionów, można skorzystać z książki najlepszych ruchów prowadzących do wygranej. W tym momencie, aby wybrać ruch, nie trzeba polegać wyłącznie na silniku ruchów. W grze w *Havannah* jest zbyt wiele pionów i zbyt wiele pozycji.

— Chyba nie chcesz programować łatwej gry, prawda, Alku? — rzucił Staś, drażniąc drugiego programistę.

Alek sprawiał wrażenie, jakby tym razem chciał programować łatwą grę.

— Nie martw się. Dobra wiadomość jest taka, że w *Havannah* gra teraz tak mało osób, że nie potrzebujemy tak mocnego silnika, jak w przypadku gry *Deep Black & White* — powiedziała Daniela, zauważając lekki wyraz ulgi na twarzy Alka. — Chcemy w końcu mieć bardzo mocny silnik, ale nie jest on potrzebny w wersji 1.0. Wystarczy przyzwoity silnik, który zwycięży z większością ludzi w większości przypadków.

— W porządku, Danielo. Czy są jakieś inne zasady, o których powinniśmy wiedzieć? — zapytał Franek.

— Nie, to wszystko. Zasady są proste, ale ta gra naprawdę zmusza do myślenia. Dlatego uważamy, że chętnie przyjmą ją klienci, którzy wcześniej kupili nasze inne gry.

— Czy dokument z listą wymagań już został napisany?

— Jeszcze nie, Franku. Na podstawie tego, czego Karol nauczył nas o zwinnym wytwarzaniu oprogramowania, chcemy, aby cały zespół wspólnie opracował wymagania.

— Dokładnie tak — dodał Karol. — Dzisiejsze spotkanie mamy zamiar zacząć od napisania historyjek użytkownika. Są to krótkie opisy funkcjonalności, ale z punktu widzenia użytkownika. Na przykład: *Jako użytkownik chcę zapisać grę, którą właśnie rozgrywam.* Coś w tym stylu.

— To brzmi dość prosto. Co zrobimy z tymi historyjkami, kiedy skończymy je pisać?

— Oszacujemy każdą z nich, określimy priorytety, a następnie ustalimy najlepszy kompromis między funkcjonalnościami a harmonogramem — powiedział Karol.

— Jak zabrać się do pisania historyjek użytkownika? — zapytał Franek.

— Skorzystamy z tych kartek z notesu — powiedział Karol, podając plik kartek wszystkim osobom w pokoju. — Lubię pisać historyjki użytkownika, stosując ten szablon. — Karol wziął marker i napisał na tablicy „Jako <typ użytkownika> chcę <cel>, tak aby <powód>”. — Danielo, Ty najwięcej myślałaś o grze w *Havannah*, więc czy możesz zacząć?

— Oczywiście — powiedziała analityk. — Zaczniemy od historyjki: *Jako gracz mogę cofnąć ruch po to, by wycofać się z niego.*

— Czy to jest nasza pierwsza historyjka? — zapytał Przemek, tester.

— Tak, dlatego zapiszę ją na kartce z notesu, abyśmy o niej nie zapomnieli — powiedziała Daniela i zapisała kartę historyjki 23.1.



Karta historyjki 23.1. Jako gracz mogę cofnąć ruch po to, by wycofać się z niego.

— Czy teraz oszacujemy tę historyjkę? — zapytał Alek.

— Na razie nie, Alku. Będzie łatwiej, jeśli najpierw napiszemy kilka historyjek, a następnie ocenimy je wszystkie naraz — powiedział Karol.

— Teraz moja kolej. Jeśli mamy historyjkę dla cofnięcia ruchu, potrzebujemy historyjki do powtórzenia ruchu. *Jako gracz chciałbym mieć możliwość powtórzenia ruchu, aby móc powtórzyć sekwencję* — powiedział Franek.

— Dobra historyjka, Franku. Zapisz ją na karcie — powiedział Karol menedżerowi produktu.

— Czy nie byłoby lepiej, gdybyśmy zapisali historyjki w arkuszu kalkulacyjnym? — zapytał Franek.

— Być może później. Ale na takim spotkaniu jak to lepiej posługiwać się fizycznymi kartami. Przekonasz się — powiedział Karol. — Każdy z nas może napisać dowolną historyjkę, kiedy tylko o niej pomyśli. Nie trzeba czekać, aż ktoś ją napisze.

— A kiedy dojdziemy do planowania, karty przydadzą się jeszcze bardziej — dodał Staś. — Możemy ułożyć karty w stos według priorytetów albo według tego, czym będziemy się zajmować w każdej iteracji.

Franek zaczął dostrzegać niektóre inne zalety korzystania z kartek. Na kartach można pisać dodatkowe notatki i rysować. Nie da się tego zrobić w arkuszu kalkulacyjnym. Franek nadal nie wiedział zbyt dużo na temat nowego „zwinnego procesu”, ale spodobało się mu to, co zobaczył do tej pory. Z rosnącym entuzjazmem napisał kartę historyjki 23.2.



Karta historyjki 23.2. Jako gracz chciałbym mieć możliwość powtórzenia ruchu, aby móc powtórzyć sekwencję.

— Dobrym sposobem na utworzenie wszystkich historyjek jest... — zaczął Karol.

— Czy naprawdę masz na myśli wszystkie, Karolu? — zapytał Staś nowego trenera zespołu.

— Tak. W pewnym sensie. Chciałbym wykorzystać to spotkanie, aby zapisać tyle kart historyjek, ile nam się uda. Historyjki cofania i powtarzania ruchu są bardzo małe. Nie potrzeba, aby nasze historyjki były tak małe. W zasadzie należy chyba połączyć cofanie z powtarzaniem ruchu.

— Dobrze. Zrobię to — powiedział Franek, zapisując kartę historyjki 23.3. — W jaki sposób możemy pokazać, że ta karta jest związana z dwoma innymi kartami? Czy będziemy je numerować?

— Nie, po prostu zniszczymy pierwsze dwie — powiedział Karol. — Nie potrzebujemy ich. Franek przedarł pierwsze dwie karty na połowę.



Karta historyjki 23.3. Jako gracz chciałbym mieć możliwość cofania i powtarzania ruchów.

— Karolu, Franek nie podał powodu na karcie historyjki. Nie ma w niej frazy „tak aby”. Czy to problem? — zapytał Alek.

— Hej, to dopiero moja druga historyjka! — Franek zawzięcie się bronił.

— W istocie — wyjaśnił Karol — nie zawsze potrzebna jest klauzula „tak aby”. Jeśli dzięki niej historyjka staje się czytelniejsza, to ją stosujemy. Ale nie warto się tym martwić, zwłaszcza w przypadku historyjek takich jak ta, kiedy jest oczywiste, po co użytkownik to robi.

— Karolu, powiedziałeś, że chcemy zapisać dziś wszystkie historyjki, nawet jeśli niektóre z nich będą napisane dość ogólnie. Zakładam, że możemy dodać jeszcze jedną. Czy tak? — zapytała Róża.

— Zdecydowanie. Najlepiej będzie, jeśli później wymyślimy jakieś nowe historyjki. W przeciwnym razie oznaczałoby to, że nie myślimy o grze wystarczająco twórczo. Przeraża mnie myśl o tym, że nie wpadniemy później na kilka pomysłów, które byłyby lepsze niż te, od których dziś zaczęliśmy. Celem napisania „wszystkich” historyjek dzisiaj jest to, abyśmy wszyscy byli świadomi — tak jak wspominałem wcześniej, chociaż na ogólnym poziomie — jakie funkcjonalności są prawdopodobne. To szczególnie pomoże Stasiowi i Alkowi opracować architekturę gry, a nawet znaleźć sposób zaprojektowania silnika ruchów.

— W porządku. To brzmi świetnie, Karolu. Kontynuujmy. Czy wszyscy powinniśmy po prostu zacząć pisać karty?

— Moglibyśmy — powiedział Karol. — Ale byłoby najlepiej, gdybyśmy zastosowali do nich jakąś strukturę. Porozmawiajmy o tym, co użytkownik może chcieć zrobić w różnych momentach w grze, zanim gra się rozpocznie, podczas gry, a nawet po jej zakończeniu.

Kilku członków zespołu rzuciło jakieś komentarze i pokiwało głowami na znak zgody. Karol kontynuował: — Co mógłby chcieć robić użytkownik zaraz po załadowaniu *Havannah*?

— Rozpocząć grę.

— Przywrócić zapisaną grę.

— Wybrać poziom gracza komputerowego.

— Dobrze, zapiszmy to — powiedział Karol. Kilka minut później zespół przygotował historyjki w postaci zaprezentowanej w tabeli 23.1.

TABELA 23.1. Historyjki wymyślone podczas zastanawiania się nad tym, czego użytkownicy chcą, zanim gra się rozpocznie

Tekst historyjki
Jako gracz mogę zacząć nową grę.
Jako gracz mogę przywrócić zapisaną grę.
Jako gracz mogę wybrać poziom gry komputera.
Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze.
Jako gracz chciałbym, aby gra wyglądała estetycznie.
Jako gracz chciałbym mieć możliwość wyboru między drewnianą planszą z drewnianymi pionami a metalową planszą z metalowymi pionami.
Jako nowicjusz w grze chcę mieć możliwość przeglądania interaktywnego samouczka gry.
Jako gracz chciałbym, aby system odtwarzał muzykę w tle.
Jako gracz mogę wybrać podkład muzyczny odtwarzany przez system.

— Karolu — zapytał Franek — jeśli te historyjki są czynnościami, które gracz może chcieć wykonać przed rozpoczęciem gry, to dlaczego dołączamy historyjkę *Jako gracz chciałbym, aby system odtwarzał muzykę w tle*? Czy to nie coś, czego gracz chce podczas gry, a nie przed grą?

— Po prostu prowadzimy burzę mózgów na temat historyjek użytkowników — odpowiedział Karol. — Nie będziemy się zastanawiać nad tym, jakie historyjki dotyczą stanu po rozpoczęciu gry, a jakie zdarzają się w jej trakcie. Po prostu stosuję takie podejście, aby pomóc w myśleniu o tym, czego potrzebujemy w grze.

— W porządku. Co robimy dalej?

— Teraz pomyślimy o tym, co użytkownik może robić w trakcie gry.

Zespół odpowiedział na pytanie Karola, zapisując historyjki przedstawione w tabeli 23.2.

TABELA 23.2. *Historyjki wymyślone podczas zastanawiania się, czego użytkownicy chcą podczas gry*

Tekst historyjki
Jako gracz chciałbym mieć możliwość ustawienia piona na planszy za pomocą klawiatury lub myszy.
Jako gracz chciałbym widzieć wizualny wskaźnik określający, czyja jest kolej.
Jako gracz chciałbym mieć wizualny wskaźnik ostatniego ruchu (może on na przykład migać).
Jako gracz chciałbym mieć możliwość cofania i powtarzania ruchów.
Jako gracz chciałbym czasem poprosić o odpowiedź.
Jako gracz chciałbym mieć możliwość zapisania gry.
Jako gracz chciałbym mieć możliwość wyjścia z gry.
Jako gracz chciałbym mieć możliwość zrestartowania gry, aby zrezygnować i zacząć od początku.
Jako nowicjusz w grze potrzebuję dostępu do systemu pomocy online programu.
Jako gracz chciałbym, aby wszystkie piony zwycięskiej figury migały lub świeciły, tak bym mógł widzieć zwycięską figurę.
Jako nowicjusz w grze chciałbym, żeby system ostrzegał mnie, jeśli wykonam bardzo zły ruch, i otrzymać szansę na jego cofnięcie.
Jako nowicjusz w grze chciałbym, aby — gdy będzie moja kolej — system wyświetlił mi pola, gdzie powinienem umieścić pion, ponieważ jeśli tego nie zrobię, to zrobi to komputer i wygra.
Jako gracz chciałbym, aby na komputerze PC z zegarem 2,0 GHz wykonanie ruchu przez komputer nie zajmowało więcej niż dwie sekundy.

— W porządku. Przypuszczam, że teraz powinniśmy przejść do czynności, które użytkownik mógłby chcieć zrobić po zakończeniu gry — powiedział Franek.

— Nie ma zbyt wielu czynności, które gracz chciałby zrobić po zakończeniu gry. Nie pomyśleliśmy wcześniej o zapisaniu gry, więc może teraz dodamy tę historyjkę — powiedział Przemek.

— Gracz może także chcieć wrócić do swoich ruchów i je skomentować. Gra *Deep Black & White* ma taką możliwość — powiedziała Róża. — Można by wpisać przy ruchu uwagę w rodzaju „myślałem także o ustawieniu piona w sześciokącie A3”.

— OK. Zapiszmy te działania w postaci historyjek — powiedział Karol, co znalazło odzwierciedlenie w tabeli 23.3.

TABELA 23.3. *Historyjki wymyślone podczas zastanawiania się, czego chce użytkownik po zakończeniu gry*

Tekst historyjki
Jako gracz chciałbym, żeby system śledził, ile gier wygrałem, a ile przegrałem (według poziomu gracza komputerowego?).
Jako gracz chciałbym móc wprowadzać komentarze do zapisanych gier.
Jako gracz chciałbym móc poruszać się między ruchami, aby prześledzić grę, w którą grałem.
Jako gracz mogę zapisać grę.

— Co dalej, Karolu? — zapytał Alek.

— W ciągu dwóch tygodni, gdy wy będziecie wprowadzać ostatnie poprawki w grze *Deep Black & White*, Daniela zamierza przeprowadzić więcej badań nad produktem.

— Tak — dodała Daniela. — Chcę zapytać potencjalnych użytkowników o to, które funkcjonalności są według nich najważniejsze.

— Ale zanim skończymy na dzisiaj, powinniśmy stworzyć wysokopoziomową ocenę każdej historyjki użytkownika — powiedział Karol.

— OK, spróbujmy je oszacować. Ale najpierw zróbmy dziesięć minut przerwy — powiedział Alek.

Szacowanie historyjek użytkownika

— Teraz — Karol wznowił spotkanie — oszacujemy każdą historyjkę użytkownika w postaci liczby punktów.

— Czym jest *punkt*? — zapytał Franek.

— Punkt to bezwymiarowa ocena rozmiaru i złożoności historyjki — odpowiedział Karol. — Załóżmy, że zdecydowaliśmy, że historyjka opisująca zapisanie gry ma trzy punkty. Jeśli uznamy, że przywrócenie zapisanej gry będzie wymagało takiego samego wysiłku, to tej historyjce również przypiszemy trzy punkty. Jeśli natomiast dojdziemy do wniosku, że odtwarzanie gry zajmie trochę mniej czasu, przydzielimy tej historyjce dwa punkty.

— Zatem liczby nic nie znaczą? — zapytał Franek.

— Mają znaczenie wyłącznie względne. Dwa to dwa razy więcej niż jeden. Osiem to cztery razy dwa. Coś w tym stylu. To wszystko, co ma znaczenie — powiedział Karol. — Szacowanie będziemy wykonywać, grając w planistycznego pokera. Karol wręczył każdej z osób zestaw kart z napisanymi na nich wartościami 1, 2, 3, 5 i 8. — Będziemy to robić w następujący sposób. Daniela odczyta jedną z kart historyjek. Możemy o niej rozmawiać i zadawać pytania. Następnie każdy z nas wybierze jedną z kart, które przed chwilą wam rozdałem. Te liczby to nasze oceny. Kiedy wszyscy wybiorą kartę, odwrócimy je jednocześnie. Jeśli wszystkie oceny będą zgodne, to taka będzie ocena historyjki. Jeśli nie, to podyskutujemy na temat oceny i historyjki przez kilka minut, a następnie ocenimy jeszcze raz. Będziemy to robić tak długo, aż uzgodnimy ocenę historyjki.

Karol odpowiedział na kilka pytań dotyczących techniki, a następnie poprosił Daniłę o odczytanie pierwszej historyjki: — *Jako gracz mogę zacząć nową grę.*

— Pierwszych kilka historyjek zawsze najtrudniej ocenić — dodał Staś. — Ponieważ oceniamy historyjki względem siebie, to jest trudno, gdy nie mamy niczego do porównania. W zasadzie możemy tylko zdecydować, czy ta pierwsza historyjka jest jedną z mniejszych, czy jedną z większych spośród wszystkich historyjek, które wymyśliliśmy. Jeśli później ocenimy cztery lub pięć historyjek i zdecydujemy, że trzeba zmienić ocenę tej pierwszej, to możemy to zrobić.

— Co mamy na myśli, mówiąc o tej historyjce? — zapytał Alek. — Nie wiadomo dokładnie, co oznacza „rozpoczęcie nowej gry”.

— Napisałyśmy tę historyjkę, kiedy zastanawialiśmy się nad tym, co użytkownik może chcieć zrobić po uruchomieniu naszego programu po raz pierwszy — powiedziała Daniela. — Powiedzieliśmy, że prawdopodobnie chce rozpocząć nową grę lub przywrócić starą grę. To wszystko, co mieliśmy na myśli.

— Czy ta historyjka obejmuje narysowanie pustej planszy i ustawienie wszystkiego tak, aby komputer był gotowy do rozpoczęcia gry? — zapytał Alek.

— Myślę, że tak. Nie sądzę, aby do realizacji tej historyjki plansza musiała być estetyczna. Mamy inną historyjkę dotyczącą estetyki ekranu. Ale ta historyjka wymaga narysowania na ekranie planszy do gry w *Havannah*.

Karol poczekał kilka dodatkowych sekund, aby przekonać się, czy są jeszcze jakieś pytania.

— W porządku. Wszyscy wybierają kartę ze swoją oceną.

— Wybrałem jeden — powiedział Alek, pokazując wszystkim swoją kartę.

— Jeszcze nie, Alku — powiedział Karol. — Wszyscy muszą mieć szanse pomyślenia o ocenie, nie patrząc na oceny innych. W ten sposób zapobiegamy wszelkim sugestiom. — Karol zamilkł na chwilę. — Wygląda na to, że wszyscy wybrali swoje karty. Odwróćcie je.

Na kartach pojawiły się różne oceny — począwszy od jedynek Alka i Róży, a skończywszy na piątce Przemka.

— Dlaczego dałeś piątkę, Przemku? — zapytał Karol.

— Myślę, że ta historyjka będzie jedną z największych, jakie mamy do zrealizowania — odpowiedział.

— Zupełnie się nie zgadzam — powiedział Alek. — Wszystko, czego wymaga ta historyjka, to interfejs użytkownika, w którym gracz może wybrać przycisk *Rozpocznij nową grę*, a następnie zostanie narysowana pusta plansza do gry w *Havannah*. Plansza nie musi być piękna, a użytkownik nie musi ustawiać żadnych pionków. Mamy na to oddzielne historyjki. Ta historyjka jest łatwa.

— OK. Nie pomyślałem o tym w ten sposób — odpowiedział Przemek.

Karol pozwolił, aby dyskusja była kontynuowana, po czym ponownie poprosił wszystkich o wybranie karty reprezentującej ich ocenę. Teraz większość osób oceniła historyjkę na jeden, a Przemek wybrał dwójkę. Karol zaproponował jeszcze jedną, krótką dyskusję, a następnie zapytał Przemka, czy może poprzeć ocenę jeden. Odpowiedział twierdząco i pierwsza historyjka została oceniona na jeden punkt.

— Przejdźmy do następnej historyjki — powiedziała Daniela i odczytała: *Jako gracz mogę przywrócić zapisaną grę*. Po dwóch minutach dyskusji o tym, co oznacza ta historyjka, Karol poprosił wszystkich członków zespołu o wybranie karty. W pierwszej rundzie nie było jednomyślności ocen. Staś twierdził, że ta historyjka jest dwukrotnie większa od pierwszej, ponieważ obejmuje konfigurację pustej planszy *Havannah*, odczytanie zapisanej gry z pliku i umieszczenie na tablicy pionów, które już były zagrane. Argumenty Stasia przekonały zespół. Wszyscy zgodzili się na ocenę dwa dla tej historyjki.

— OK, następna historyjka brzmi: *Jako gracz mogę wybrać poziom gry komputera*. Spróbujmy ją ocenić — powiedział Staś.

— To jest cały silnik wyboru ruchów — powiedział Alek. — To będzie trudne. Czy możemy teraz pominąć tę historyjkę i wrócić do niej po dokonaniu oceny niektórych większych historyjek?

— Jak dla mnie OK — powiedział Staś.

— Chociaż, poczekajcie — powiedziała Róża. — Myślałam, że ta historyjka dotyczy wyboru poziomu gry przez gracza, a nie tego, żeby komputer mógł grać na tym poziomie. To proste — wystarczy do tego jedno pole lub dwa pola na ekranie.

— To miałem na myśli, gdy pisałem tę historyjkę — powiedział Franek.

— W porządku, ale w takim razie potrzebna jest nam historyjka *Jako gracz mogę grać z komputerem na różnych poziomach* — powiedziała Róża.

— Czy możemy podzielić tę historyjkę w taki sposób? — zapytał Alek. — Myślę, że to będzie trudne. A co myślicie o historyjce *Jako gracz mogę grać przeciwko komputerowi, który jest słabym przeciwnikiem*? A potem *Jako gracz mogę grać przeciwko komputerowi, który jest silnym przeciwnikiem*?

— Jasne. Zróbmy to w ten sposób — powiedział Staś. — Zaczniemy od historyjki opisującej gracza, który wybiera poziom gry komputera. — Wszyscy wybrali jedynkę spośród kart planistycznego pokera. — Wszyscy gotowi? Odwróćcie swoje karty. — Wszyscy odwrócili swoje jedynki. — Ta ocena wydaje mi się rozsądna. Mówimy, że ma w przybliżeniu taki sam rozmiar jak historyjka *Jako gracz mogę zacząć nową grę*.

Członkowie zespołu zgodnie przyznali, że te dwie historyjki rzeczywiście mają w przybliżeniu ten sam rozmiar.

— Przejdźmy teraz do historyjki *Jako gracz mogę grać przeciwko komputerowi, który jest słabym przeciwnikiem* — zaproponował Alek palący się do rozmowy o historyjkach dotyczących silnika ruchów, którymi prawdopodobnie to on będzie się zajmował. — Jak silny powinien być nasz słaby silnik ruchów?

— Nie może grać losowo — powiedziała Daniela. — Ale nie musi być bardzo silny. Ta gra jest dość trudna i ustalenie, co to znaczy „dobry ruch”, zajmie większości osób sporo czasu. Jednak nawet na najsłabszym poziomie komputer powinien wiedzieć — na podstawie tego, co robi gracz — czy lepiej spróbować stworzyć pierścień, most, czy rozwidlenie.

— OK. Spróbujmy to oszacować — powiedział Alek.

— Poczekajcie — powiedział Przemek. — Jak będziemy to testować? Wygląda na to, że przetestowanie tej funkcjonalności będzie naprawdę bardzo trudne.

— Dobre pytanie — powiedział Karol. — Czy są jakieś pomysły? Rozejrzał się po pokoju.

— Jednym ze sposobów mogłoby być zidentyfikowanie kilku pozycji, zaobserwowanie tego, co zaoferuje silnik, i poproszenie dobrego gracza-człowieka o ocenę, czy te ruchy były dobre — powiedziała Róża.

— Brzmi dobrze. Czy możemy to jakoś zautomatyzować? — zapytała Daniela. — Musimy zautomatyzować testy tak, żebyśmy się dowiedzieli od razu, że silnik zaczął wykonywać dziwne ruchy. Tak było w grze *Deep Black & White* w kwietniu.

— Bez trudu — odpowiedział Przemek. — Możemy mieć plik, który opisuje położenie na tablicy, a następnie informuje, w jaki sposób powinien zareagować komputer.

— Niekoniecznie będzie tylko jeden dopuszczalny ruch — powiedział Alek. — Ten plik powinien pozwolić nam na wskazanie wielu prawidłowych odpowiedzi.

— Doskonałe odpowiedzi. Nie musimy jednak projektować tego pliku teraz, pod warunkiem że będziemy wiedzieć o istnieniu pewnych opcji. Spróbujemy oszacować tę historyjkę — powiedział Karol.

W pokoju zrobiło się cicho. Wszyscy członkowie zespołu zaczęli się zastanawiać, jakie zadania obejmie ta historyjka. Gdy wszyscy wybrali swoje karty, odwrócili je równocześnie.

— Alku, myślę że powinieneś odwrócić tylko jedną kartę, a nie wszystkie. Czy masz trudności z podjęciem decyzji?

— Nie, raczej nie mam karty, na której byłaby wystarczająco duża ocena. Dlatego obróciłem je wszystkie. Wszystkie te karty — programista wskazał na karty z wartościami 1, 2, 3, 5 i 8 — dają w sumie 19.

— Mam kilka kart o większych wartościach: 13, 20, 40 i 100 — powiedział Karol. Rozdał wszystkim osobom zbiór dodatkowych kart. — Miałem nadzieję, że nie będziemy ich potrzebować, ale widzę, że mogą być potrzebne.

— Dlaczego miałeś nadzieję, że nie będziemy potrzebować tych kart?

— Z historyjek, które oceniliśmy do tej pory, wynika, że żadna z historyjek ocenionych na 100 punktów nie zmieści się w jednej iteracji. Takie historyjki zajęłyby zbyt dużo czasu. To samo dotyczy pewnie pozostałych wartości na kartach, które właśnie wam rozdałem. Nie ma niczego złego w istnieniu kilku dużych historyjek i jest całkowicie właściwe przypisanie im dużych ocen. Ale powinniśmy pamiętać, że każdą z takich historyjek musimy podzielić przed przystąpieniem do pracy tak, aby każda mogła zostać zrealizowana w jednej iteracji. Powinniśmy także pamiętać, że oceny bardzo dużych funkcjonalności prawdopodobnie będą mniej dokładne. Właśnie dlatego zakresy stają się większe.

— Zatem uważasz, Alku, że ta historyjka powinna być oceniona na dziewiętnaście. Naprawdę? — zapytał Staś, przyglądając się uważnie ósemce w swoim ręku.

— Zdecydowanie. Przede wszystkim muszę dowiedzieć się więcej o tej grze. Silnik musi umieć rozpoznać, czy gracz próbuje wykonać pierścień, most, czy rozwidlenie. Musi zdecydować, jaką figurę próbuje stworzyć. Powinien zdecydować, czy ruch ma być ofensywny, czy defensywny. Opracowanie nawet pierwszej wersji słabego silnika zajmie sporo czasu.

— Cóż. Znasz się na tym. Ale dziewiętnaście to wysoka ocena — powiedział Staś.

— Przemku, dlaczego uważasz, że ta historyjka powinna mieć ocenę trzy? — zapytał Karol, koncentrując rozmowę wokół oceniających, którzy przypisali skrajnie różne oceny.

— Testowanie nie wydaje się trudne. Nie sędzę, abym miał dużo do zrobienia. Utworzę te pliki tekstowe dla testów i to wszystko.

— OK. Ale my oceniamy całą historyjkę, a nie tylko własny wkład. Testy mogą być ocenione na trzy, lecz Alek także potrzebuje czasu na wykonanie swoich zadań — wyjaśnił Karol.

— Ups. Jeśli tak, to Alek ma rację — powiedział Przemek. — Ta historyjka wygląda na dużą.

— Zobaczmy, czy ma rację. Weźcie karty i oceńcie tę historyjkę na podstawie tego, co właśnie usłyszeliście — polecił Karol. Zamilkł na kilka sekund, aby mieć pewność, że wszyscy mają czas, żeby się zastanowić. — Obróćcie swoje karty.

Tym razem na wszystkich kartach pojawiły się dwudziestki.

— Wygląda na to, że przekonałeś wszystkich, Alku — powiedział Karol. — Dwadzieścia to chyba jednak za dużo jak na jedną historyjkę. Alku, czy istnieje sposób, aby podzielić tę historyjkę? Być może jeszcze słabszy silnik w pierwszej wersji? A potem go ulepszymy przed wydaniem.

— Nie widzę żadnego dobrego sposobu — powiedział Alek. Pomyślał przez chwilę, po czym kontynuował: — Moglibyśmy stworzyć silnik, który wykonuje każdy ruch ofensywnie, zawsze dąży do stworzenia własnej figury i ignoruje ruchy człowieka. Ale nie sądzę, żeby to był dobry pomysł.

— A gdyby pierwsza wersja silnika potrafiła rozpoznawać wyłącznie pierścienie? — zapytała Róża. — Moglibyśmy pominąć mosty i rozwidlenia — tylko tymczasowo. Alku, czy mógłbyś napisać silnik, który gra zarówno ofensywnie, jak i defensywnie, ale stara się tworzyć tylko pierścienie i tylko przed nimi się broni?

— Oczywiście. To może się udać. Podzielmy więc tę historyjkę na trzy części — po jednej dla każdej figury.

Wszyscy się zgodzili i oceniali trzy historyjki tak jak pokazano w tabeli 23.4.

TABELA 23.4. Trzy mniejsze historyjki po rozdzieleniu epiki słabego silnika

Tekst historyjki	Ocena
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie.	8
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty.	5
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje rozwidlenia.	8

— Gdy te trzy historyjki tworzyły jedną, to oceniliśmy je na dwadzieścia punktów. Teraz ocena wynosi dwadzieścia jeden. Czy powinniśmy odjąć punkt od jednej historyjki, tak aby nadal było w sumie dwadzieścia? — zapytał Franek.

— Nie. Oceny są takie, jakie są. Nie ma potrzeby, aby były szczególnie dokładne — odpowiedział Karol. — Dzielenie historyjek pomaga nam zobaczyć lepiej, jakie zadania wchodzą w skład historyjki. Suma ocen historyjek składowych nie musi być równa ocenie większej historyjki.

— Alku, co powinniśmy zrobić z silnikami o średniej i dużej mocy? Czy dla każdego z nich stworzymy po jednej historyjce, czy także podzielimy je na pierścienie, mosty i rozwidlenia?

— Po jednej historyjce na każdy silnik. Myślę, że powinniśmy zdefiniować silnik o średniej mocy jako taki, który nigdy nie popełnia oczywistego błędu — powiedział Alek. — Nie zawsze musi zagrywać najlepszego ruchu, ale nigdy nie popełnia oczywistej pomyłki. Możemy to przetestować w sposób, który opisaliśmy, i dać do sprawdzenia dobrym graczom-ludziom. Jeśli silnik będzie wykonywał ruchy, które eksperci uznają za rozsądne, to oznacza, że jest dobry. Dla mocnego silnika możemy zaostrzyć to wymaganie tak, by silnik zawsze wykonywał najlepszy możliwy ruch. Na podstawie wydajności możemy ustalić, jak daleko do przodu powinien sięgać silnik ruchów.

— Zatem w jaki sposób chcecie zapisać te historyjki użytkownika? — zapytał Franek.

— Myślę, że *Jako gracz mogę grać przeciwko silnikowi średniej mocy* jest w porządku — odpowiedziała Daniela.

— Tak — zgodził się Karol. — Możemy umieścić na karcie historyjki notatkę „zawsze wykonuje ruchy, które w opinii dobrego gracza-człowieka można uznać za rozsądne”. Będzie to jeden z warunków

satysfakcji dla tej historyjki. To pomoże nam przypomnieć, czym jest „silnik o średniej mocy” bez konieczności tworzenia kolejnej historyjki o rozsądnych ruchach.

Zespół podyskutował o tej historyjce trochę dłużej, grając w planistycznego pokera, a następnie uzgodniono dla niej ocenę ośmiu punktów.

— No i mamy kolejną historyjkę, która brzmi: *Jako gracz mogę grać przeciwko mocnemu silnikowi* — powiedział Franek.

— To co najmniej dwa razy większa historyjka od średniego silnika — powiedział Alek. — Tę pierwszą oceniliśmy na osiem. Nie mam jednak karty z oceną szesnaście. Czy damy trzynaście punktów? To najbliższe szesnastu. Czy też dajemy szesnaście?

— Karty — powiedział Karol — najlepiej porównać do pojemników. Jeśli ta historyjka ma szesnaście punktów, to nie zmieści się w pojemniku o objętości trzynaście. Nie chcemy wymyślać nowych wartości, bo zaczynają sprawiać wrażenie zbyt dokładnych. Nie wiemy, czy ta historyjka powinna być oceniona na trzynaście, czy też na szesnaście lub siedemnaście. Powinniśmy jednak pamiętać, że są to oszacowania, a nie gwarancje.

— OK. Ja myślę, że ta historyjka jest dwa do trzech razy większa od średniego silnika — powiedział Alek. — To oznacza, że powinna mieć ocenę od szesnastu do dwudziestu czterech. Ponieważ dwadzieścia cztery nie zmieści się w pojemniku o objętości dwadzieścia, to czy powinniśmy ocenić ją na czterdzieści? To wydaje się za dużo. Ta historyjka nie jest pięć razy większa od średniego silnika.

— Nie, nie — odpowiedział Karol. — To są pojemniki, ale zawartość można raczej porównać do piasku niż do wody. Możemy je trochę przepelnić.

— Spróbujmy je zatem ocenić — powiedział Alek.

Na podstawie dyskusji, którą właśnie usłyszeli członkowie zespołu, nikt nie wyraził sprzeciwu co do oceny Alka i wszyscy dali dwudziestki.

— Zatem oceniamy tę historyjkę na dwadzieścia — powiedział Karol. — To obecnie wystarczy. Prawdopodobnie potrzebujemy sposobu, aby później podzielić tę historyjkę na co najmniej dwa osobne fragmenty.

— Czy nie trzeba podzielić tej historyjki na dwie mniejsze tak jak w przypadku słabego silnika? — zapytał Alek.

— Nie, możemy to zrobić później, kiedy będzie bliżej do rozpoczęcia pracy nad tą historyjką — odpowiedział Karol. — Będziemy wtedy wiedzieć więcej, a nic nie zyskamy, jeśli podzielimy ją dzisiaj.

— W porządku. Idźmy dalej — zaproponowała Daniela. — Następna historyjka brzmi: *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze*. Chodziło o to, że dwóch graczy siedzi przy jednym komputerze i przekazuje sobie klawiaturę lub mysz i na zmianę wykonuje ruchy. Chcemy tylko, żeby komputer powiedział nam, kto wygrał.

— Ale inne funkcjonalności, które zidentyfikowaliśmy, nadal są dostępne, prawda? — zapytał Przemek. — Dwaj gracze mogą chcieć korzystać z możliwości cofania i powtarzania ruchu albo mogą chcieć poprosić o podpowiedź.

— Oczywiście — odpowiedziała Daniela.

— Myślę, że powinniśmy dodać historyjkę *Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę* — powiedział Alek.

— Czy to nie jest część historyjek dotyczących silników ruchów?

— Tak, ale jeśli wyodrębnimy taką historyjkę, to będziemy mogli zrealizować ją osobno. To pozwoli nam napisać wersję gry dla dwóch osób bardzo szybko, a jednocześnie pozwoli wykorzystać silniki ruchów.

Nie było zastrzeżeń, a nową historyjkę oceniano na dwa punkty.

— Alku, czy to obniża którąkolwiek z ocen historyjek dotyczących słabego silnika? — zapytał Franek.

— Właściwie nie. Nadal oceniłbym każdą z nich w taki sam sposób. Moglibyśmy zmniejszyć ocenę historyjek ośmiopunktowych do siedmiu, jeśli chcecie.

— Nie, nie chcemy tego robić — powiedział Karol. — W ten sposób liczby wyglądałyby na zbyt dokładne. Jeśli chcemy zmniejszyć oceny do pięciu, to możemy to zrobić. W przeciwnym razie zostawmy je jako ósemki.

— Nie, pięć to zbyt niska ocena — powiedział Alek.

— W porządku. Zatem powróćmy do oceny historyjki *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze*.

Po dwóch rundach planistycznego pokera członkowie zespołu uzgodnili, że ta historyjka zasługuje na trójkę.

— Następna historyjka to *Jako gracz chciałbym, aby gra wyglądała estetycznie* — przeczytała Daniela.

— W końcu. To jedna z moich historyjek — powiedziała Róża, grafik.

— Tak, tylko nie podoba mi się, jak ta historyjka jest napisana — powiedział Staś. — Brzmi dość niejasno i wygląda na dużą. Podoba mi się następna historyjka: *Jako gracz chciałbym mieć możliwość wyboru między drewnianą planszą z drewnianymi pionami a metalową planszą z metalowymi pionami*. Co jeszcze rozumiemy pod pojęciem „estetyczna”?

— To ma związek z ogólnym wyglądem i wrażeniami, jakie daje gra — odpowiedział Franek. — Prawdopodobnie nie będzie zbyt wielu menu, ale chcielibyśmy, aby wyglądały dobrze. Chcemy, aby pozycje menu znalazły się w logicznych miejscach. Chcemy atrakcyjnego ekranu powitalnego w czasie, gdy gra się ładuje. Plansza i piony to cały interfejs użytkownika. Prawdopodobnie będzie jakaś grafika w tle za planszą gry. Róża potrzebuje czasu, żeby ją wykonać.

— Te historyjki brzmią dobrze. Czy możemy wydzielić niektóre z nich jako osobne historyjki? — zapytał Staś.

— Moglibyśmy — odpowiedziała Daniela — ale to może być trudne. Myślę, że pozycje menu powinny być opracowane w kontekście funkcjonalności, które wprowadzają potrzebę nowego menu lub pozycji menu. Różo, co myślisz o tym, żeby dodać jedną historyjkę opisującą ekran powitalny i drugą dotyczącą grafiki w tle?

— Dla mnie w porządku. Oczywiście ja i tak bym je robiła oddzielnie.

Zespół zrobił szybkie postępy, oceniając następny zbiór historyjek.

— A co robimy z tą historyjką? — zapytał Przemek, wskazując na kartę *Jako gracz chciałbym czasem poprosić o podpowiedź*.

— Co masz na myśli?

— Jeśli zaimplementujemy silnik ruchów, to ta historyjka będzie naprawdę bardzo prosta. Wystarczy zażądać od silnika ruchów zaproponowania kolejnego ruchu. Tym razem jednak graczowi, a nie

komputerowi. To byłoby trywialne. Ale jeśli nie będziemy mieli silnika ruchu, to ta historyjka będzie musiała być co najmniej tak duża, jak napisanie całego silnika ruchu.

— Mamy więc zależność, zgodnie z którą historyjka dotycząca odpowiedzi może być zrealizowana dopiero po silniku ruchów — powiedział Alek.

— Tak, ale nie sądzę, aby w tym przypadku to był problem — powiedział Staś. — Zaprogramowanie silnika ruchu przed funkcjonalnością odpowiedzi wydaje się naturalnym sposobem programowania tego rodzaju historyjek. Zależność nie będzie nam przeszkadzać. Możemy ją zignorować. Albo jeśli mamy obawy, że zapomnimy o tej zależności, to możemy zanotować właściwą informację na karcie. Tak czy inaczej, możemy oszacować historyjkę z założeniem, że silnik ruchów istnieje.

Wszyscy się zgodzili, a historyjkę oceniono na jeden punkt.

— Co byśmy zrobili, gdybyśmy naprawdę chcieli mieć odpowiedzi przed zaimplementowaniem silnika ruchów?

— Czasami nie można uwolnić się od zależności i trzeba z nią żyć tak samo, jak zanim zaczęliśmy stosować proces Agile — wyjaśnił Karol. — Jednak w większości przypadków można się pozbyć zależności. Gdybyśmy chcieli pozbyć się tej zależności, moglibyśmy napisać kod, który pozwala użytkownikowi poprosić o odpowiedź, wyświetlić ją i wykonać ruch za użytkownika, jeśli ta odpowiedź mu się spodoba. Do uzyskania odpowiedzi w systemie przydałby się obiekt o nazwie `HintGenerator`. Ostatecznie `HintGenerator` wywoływałby silnik ruchu w celu uzyskania dobrej odpowiedzi. Na razie jednak moglibyśmy poprzestać na tym, aby zwracał pierwsze wolne pole lub losowe wolne pole. W ten sposób moglibyśmy rozpocząć pracę nad historyjką generowania odpowiedzi przed rozpoczęciem pracy nad silnikiem ruchu.

— Musielibyśmy pamiętać, aby później wrócić do tej historyjki i zadbać o to, aby `HintGenerator` rzeczywiście wywoływał silnik ruchu.

— Tak — powiedział Karol. — Moglibyśmy stworzyć jeszcze jedną historyjkę, która brzmiałaby jakoś tak: *Jako gracz chciałbym otrzymywać dobre odpowiedzi, a nie dowolne odpowiedzi.*

— Taka zmiana sprowadzałaby się do jednej linijki — powiedział Alek. — Usunęlibyśmy wszystkie działania, które wykonywał `HintGenerator` w celu wyszukania pustego pola, i zamiast tego wywołalibyśmy silnik ruchów. Musimy ocenić tę historyjkę na jeden — tak samo jak historyjkę wyjściową. Nie wydaje się, abyśmy coś na tym zyskali.

— Ach — powiedział Karol — w takich sytuacjach czasami przydaje się posłużenie kartą, na której jest ocena zero, zamiast ocen, które mamy teraz. Ponieważ musimy wrócić do historyjki i podpiąć obiekt `HintGenerator` do prawidłowego kodu, na potrzebujemy karty historyjki, aby o tym nie zapomnieć. Z drugiej strony to naprawdę jest tak prosta zmiana, że nadamy jej ocenę zero.

Karol wręczył każdemu członkowi zespołu kolejną kartę do gry w planistycznego pokera z oceną zero. — Wstrzymywałem się z tym, ponieważ nie chciałem, żebyście posługiwali się tą kartą, zanim nie wyjaśnię, czym jest historyjka oceniona na zero. — Każdy członek zespołu miał teraz karty z ocenami 0, 1, 2, 3, 5, 8, 13, 20, 40 i 100.

— Jednak w tym przypadku mamy zamiar dodać wsparcie dla odpowiedzi po zakończeniu pracy nad silnikiem ruchów. Dobrze rozumiem? — zapytał Franek.

— Tak, po prostu zostawmy tę historyjkę jako jednopunktową — odpowiedział Staś.

Zespół kontynuował ocenianie w ten sposób do chwili, gdy oszacowano wszystkie napisane historyjki. Historyjki wraz z ich ocenami zestawiono w tabeli 23.5.

TABELA 23.5. *Oceny dla wszystkich napisanych dotąd historyjek użytkownika*

Tekst historyjki	Ocena
Jako gracz mogę zacząć nową grę.	1
Jako gracz mogę przywrócić zapisaną grę.	2
Jako gracz mogę wybrać poziom gry komputera.	1
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie.	8
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty.	5
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje rozwidlenia.	8
Jako gracz mogę grać przeciwko silnikowi średniej mocy.	8
Jako gracz mogę grać przeciwko mocnemu silnikowi.	20
Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze.	3
Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę.	2
Jako gracz chciałbym widzieć estetyczny ekran startowy podczas ładowania programu.	5
Jako gracz chciałbym widzieć estetyczną grafikę w tle, która integruje się z planszą gry.	5
Jako gracz chciałbym mieć możliwość wyboru między drewnianą planszą z drewnianymi pionami a metalową planszą z metalowymi pionami.	8
Jako gracz chciałbym czasem poprosić o podpowiedź.	1
Jako nowicjusz w grze chcę mieć możliwość przeglądania interaktywnego samouczka gry.	8
Jako gracz chciałbym, aby system odtwarzał muzykę w tle.	5
Jako gracz mogę wybrać podkład muzyczny odtwarzany przez system.	1
Jako gracz chciałbym mieć możliwość ustawienia pionu na planszy za pomocą klawiatury lub myszy.	3
Jako gracz chciałbym widzieć wizualny wskaźnik określający, czyja jest kolej.	2
Jako gracz chciałbym mieć wizualny wskaźnik ostatniego ruchu (może on na przykład migać).	2
Jako gracz chciałbym mieć możliwość cofania i powtarzania ruchów.	2
Jako gracz chciałbym mieć możliwość zapisania gry.	3
Jako gracz chciałbym mieć możliwość wyjścia z gry	1
Jako gracz chciałbym mieć możliwość zrestartowania gry, aby zrezygnować i zacząć od początku.	1
Jako nowicjusz w grze potrzebuję dostępu do systemu pomocy online programu.	8
Jako gracz chciałbym, aby wszystkie piony zwycięskiej figury migały lub świeciły, tak bym mógł widzieć zwycięską figurę.	3

TABELA 23.5. Oceny dla wszystkich napisanych dotąd historyjek użytkownika — ciąg dalszy

Tekst historyjki	Ocena
Jako nowicjusz w grze chciałbym, żeby system ostrzegał mnie, jeśli wykonam bardzo zły ruch, i otrzymać szansę na jego cofnięcie.	8
Jako nowicjusz w grze chciałbym, aby — gdy będzie moja kolej — system wyświetlił mi pola, gdzie powinienem umieścić pion, ponieważ jeśli tego nie zrobię, to zrobi to komputer i wygra.	3
Jako gracz chciałbym, aby na komputerze PC z zegarem 2,0 GHz wykonanie ruchu przez komputer nie zajmowało więcej niż dwie sekundy.	8
Jako gracz chciałbym, aby system śledził, ile gier wygrałem, a ile przegrałem (według poziomu gracza komputerowego?)	3
Jako gracz chciałbym móc wprowadzać komentarze do zapisanych gier.	3
Jako gracz chciałbym móc poruszać się między ruchami, aby prześledzić grę, w którą grałem.	5

— Zatem jakie mamy plany na najbliższą przyszłość? Potrzebujemy jeszcze jednej, dwutygodniowej iteracji na dokończenie gry *Deep Black & White*, zanim wyślemy ją do wydawcy — powiedział Franek.

— W czasie gdy wszyscy będą nad tym pracować, ja mam zamiar porozmawiać z kilkoma potencjalnymi nabywcami tej gry — powiedziała Daniela. — Chcę zobaczyć, jakie funkcjonalności są najważniejsze.

— Czy to potrwa długo?

— Nie, skończę przed końcem iteracji — odparła Daniela. — Spotkajmy się ponownie za dwa tygodnie, zaraz po wysłaniu *Deep Black & White* do wydawcy i po tym, jak Franek zabierze nas gdzieś wszystkich, aby uczcić zakończenie projektu.

— Brzmi świetnie. Dobrze się przygotuję na to świętowanie. Czy hamburgery będą OK, Danielo? Czy raczej lepsza będzie pizza?

Przygotowanie do badania produktu

W ciszy wczesnego poranka następnego dnia Daniela usiadła przy swoim biurku w nowej sali *open space* zespołu i zaczęła przygotowywać kwestionariusz do sporządzenia ankiety wśród potencjalnych nabywców gry. Franek poprosił o spotkanie z nią w ciągu dnia tak, żeby mu wytłumaczyła, w jaki sposób chce podejść do ustalenia priorytetów funkcjonalności. Jako menedżer produktu to Franek podejmie ostateczne decyzje dotyczące tego, co znajdzie się w produkcie, ale Daniela zdawała sobie sprawę, że przy podejmowaniu tej decyzji będzie on mocno polegać na jej badaniach. Chciała być przygotowana na spotkanie z nim. Do czasu przybycia Franka Daniela miała prawie gotowy kwestionariusz.

— Dzień dobry, Danielo — Franek przywitał analityczkę. — Czy nadal masz czas, żeby spotkać się dziś rano?

— Oczywiście. Przysłałam wcześniej, żeby zakończyć ankietę, którą chcę wysłać. Chcę ci ją pokazać.

— Pokaż mi ją — powiedział Franek, przysuwając krzesło. — Pokaż mi, co tam masz.

— Wydrukowałam to około pół godziny temu — powiedziała Daniela, wręczając arkusz podobny do pokazanego w tabeli 23.6. — Po wydrukowaniu dodałam jeszcze kilka pytań. Ale to wystarczy, abyś miał wyobrażenie o tym, co chcę wysłać.

TABELA 23.6. *Początek kwestionariusza Daniela*

Jak oceniasz następującą funkcjonalność...	Podoba mi się	Spodziewałem się jej	Jestem neutralny	Jest mi obojętna	Nie podoba mi się
Możesz wybrać opcję gry ze słabym przeciwnikiem komputerowym?					
NIE możesz wybrać opcji gry ze słabym przeciwnikiem komputerowym?					
Możesz wybrać opcję gry z przeciwnikiem komputerowym reprezentującym przeciętny poziom?					
NIE możesz wybrać opcji gry z przeciwnikiem komputerowym reprezentującym przeciętny poziom?					
Masz dostęp do interaktywnego samouczka, który pomoże Ci nauczyć się grać?					
NIE masz dostępu do interaktywnego samouczka, który pomoże Ci nauczyć się grać?					

Franek przejrzał arkusz, który wręczyła mu Daniela, i zapytał: — Wygląda na to, że każde pytanie zadajesz dwukrotnie. Dlaczego?

— Pierwsza forma pytania to tzw. forma funkcjonalna, natomiast druga to forma dysfunkcjonalna. Celem tej drugiej jest wskazanie, że funkcjonalność jest niedostępna. Zadawanie takich dwóch pytań daje nam lepsze zrozumienie w porównaniu z zadaniem pytania w prostej formie „Ile z tej funkcjonalności chcesz w produkcie?”. Odpowiedzi mówią nam, jak użytkownik ocenia sytuację, gdy określona funkcjonalność jest dostępna, a jak gdy nie jest dostępna.

— Czy możesz podać mi przykład, dlaczego to może być ważne?

— Jasne. Załóżmy, że zapytamy „Jak oceniasz istnienie funkcjonalności cofania i powtarzania ruchów?”, a użytkownik odpowie, że oczekuje jej obecności. Następnie zadamy mu pytanie „Jak oceniasz brak funkcjonalności pozwalającej na cofanie i powtarzanie ruchów” i otrzymamy odpowiedź, że użytkownikowi się to nie podoba. To nam mówi, że ta funkcjonalność jest dla użytkownika obowiązkowa. Oczekuje możliwości cofania i powtarzania ruchów i nie podoba mu się sytuacja, gdyby tej funkcjonalności miało nie być.

— OK. Na razie cię rozumiem.

— Teraz załóżmy, że temu samemu użytkownikowi zadajemy pytanie „Jak oceniasz istnienie interaktywnego samouczka, który pomaga ci nauczyć się grać?”. Użytkownikowi podoba się ten pomysł. Zadajemy także to samo pytanie w formie negatywnej: „Jak oceniasz brak interaktywnego samouczka?” i otrzymujemy odpowiedź, że ocena użytkownika jest neutralna. Taką funkcjonalność nazywamy atrakcją lub wabikiem. Znaleźliśmy funkcjonalność, której użytkownik nie oczekiwał lub bez której może się obyć. Ale odpowiedział nam, że podoba mu się ta funkcjonalność. Zatem dowiedzieliśmy się, że nie jest ona obowiązkowa, ale ze względu na to, że podoba się użytkownikowi, to być może będzie gotowy dodatkowo za nią zapłacić.

— Ale czy nie moglibyśmy osiągnąć tego samego, prosząc użytkowników o ocenę ważności funkcji od jednego do pięciu, tak jak w przypadku gry *Deep Black & White*? — zapytał Franek.

— Nie do końca — odpowiedziała Daniela. — Zastosowanie pojedynczej skali tego rodzaju mówi nam tylko to, jak bardzo użytkownik ceni tę funkcjonalność. Nie mówi nam niczego na temat tego, jak mocno zareagowałby, gdyby tej funkcjonalności nie było. Nie dałabym bardzo wysokiego priorytetu kołom, gdyby ktoś zapytał mnie o funkcjonalności auta, ale byłabym wściekła, gdybym otrzymała auto bez kół.

— Komu masz zamiar wysłać ten kwestionariusz?

— Mam zamiar wysłać e-mail do około 500 członków pewnych klubów, w których gra się w *Havannah*. Jest ich wiele w Europie i w większych miastach w USA. Oferujemy kupon zniżki w kwocie 10 dolarów na jedną z naszych obecnych gier dla wszystkich osób, które udzielą odpowiedzi. Spodziewam się uzyskać około pięćdziesięciu odpowiedzi. Mam zamiar również przeprowadzić kilka ankiet telefonicznych.

— Czy to nie będzie zbyt dużo do odpowiadania, jeśli do każdej historijki użytkownika zadamy dwa pytania? — zapytał Franek.

— Nie sędzę. Mogę pogrupować niektóre historijki użytkownika, które zdefiniowaliśmy, i utworzyć tematy. Na przykład mamy dwie historijki użytkownika dotyczące muzyki w tle. Można połączyć je w jeden temat muzyki w tle. Połączę historijki na temat grafiki w tle oraz ekranu powitalnego w jeden temat dotyczący estetycznego wyglądu gry. Podobnie nie zamierzam tracić czasu na pytania dotyczące zapisywania gry i jej przywracania. Wiemy, że te funkcjonalności są obowiązkowe.

— To wspaniale. Chcę mieć pewność, że ankieta będzie łatwa i prosta. W jaki sposób wykorzystamy odpowiedzi, które otrzymamy?

— Chcę podzielić nasze funkcjonalności na trzy kategorie. Do pierwszej będą należały te, które musimy mieć. Są to funkcjonalności podstawowe, takie jak zapisywanie i przywracanie gry. Druga kategoria dotyczy funkcjonalności, których im więcej, tym lepiej. Przypuszczam, że w tej kategorii będą się mieścić takie funkcjonalności jak poziomy gry. Im więcej poziomów gry — takich jak mocny, średni i słaby — tym lepiej. Trzecia kategoria to wabiki. Są to funkcjonalności, których większość użytkowników nie oczekuje, ale gdy je zobaczą, to będą mile widziane. Odpowiednia mieszanka funkcjonalności z dwóch ostatnich kategorii wraz z wszystkimi funkcjonalnościami obowiązkowymi składają się na bardzo kuszący produkt.

Zanim Franek zdołał odpowiedzieć, Alek przysunął swoje krzesło obok Franka i Daniela. — Dzień dobry — powiedział. — Nie mogłem się powstrzymać i podsłuchałem część waszej rozmowy. Brzmi bardzo ciekawie. Czy pozostali członkowie zespołu będą mieli dostęp do wyników ankiety?

— Oczywiście, Alku.

— To dobrze. Wiesz co, z chęcią przysłuchałbym się jednej lub dwóm ankietom telefonicznym, aby usłyszeć, co użytkownicy mają do powiedzenia. Czy to możliwe? — zapytał Alek.

— Oczywiście — odpowiedziała Daniela. — Zrozumienie naszych odbiorców jest czymś, na czym skorzysta cały zespół. Im bardziej osoby techniczne w zespole chcą uczestniczyć w procesie komunikowania się z użytkownikami, tym lepiej z mojego punktu widzenia. Ja skorzystam z waszych pomysłów, a wy skorzystacie, słysząc opinie klientów.

— Zgadza się — powiedział Alek. — Dzisiaj nie mogę uczestniczyć w ankietach telefonicznych. Muszę poprawić błąd w silniku punktowania gry *Deep Black & White*. Ale kiedy będziesz dzwonić do klientów każdego innego dnia w tym tygodniu, daj mi znać.

— Będę pamiętała — powiedziała Daniela.

— Wydaje się, że opracowaliście solidny plan — powiedział Franek. — To ekscytujące. Czy będą jakieś wyniki w przyszłym tygodniu?

— Na pewno. Zespół IT jutro umieści formularz z ankietą na naszej stronie internetowej. Wkrótce po opublikowaniu ankiety możemy zacząć zbierać wstępne wyniki.

— Doskonale. Dziękuję, że mi to pokazałaś — powiedział Franek, wstając, po czym wyszedł z pokoju.

Planowanie iteracji i wydania. Runda 1.

W dwa tygodnie po spotkaniu, na którym członkowie zespołu napisali i oszacowali historyjki użytkownika, zespół ponownie zebrał się w sali konferencyjnej przylegającej do ich sali *open space*. O dziewiętej na spotkaniu byli obecni wszyscy członkowie zespołu. Była nawet Róża, która zaczęła odprowadzać Bartka do przedszkola pięć minut wcześniej, tak by być w biurze do czasu, gdy zespół zaczyna swoje codzienne zebrania *stand-up*.

— Mamy na dziś trzy cele — rozpoczął Karol. — Najpierw zaplanujemy naszą pierwszą dwutygodniową iterację. Następnie Daniela opowie nam o wynikach przeprowadzonych badań nad produktem. Dowiemy się, czego chcą potencjalni nabywcy. Na koniec dokonamy wstępnego, przybliżonego oszacowania tego, jak może wyglądać plan wydania i harmonogram. Jakies pytania?

Staś poczekał kilka sekund, aby zobaczyć, czy nie ma pytań. Nie było.

Planowanie pierwszej iteracji

— Zatem co powinniśmy zaimplementować w pierwszej kolejności? — zapytała Róża.

— Chcę zacząć od silnika ruchów — powiedział Alek. — To jest nasze największe zagrożenie.

— Nie mam zastrzeżeń — powiedział Staś, drugi programista w zespole. — Czy jest coś, w czym mógłbym ci pomóc, Alku? Zazwyczaj wszystkie zadania związane ze sztuczną inteligencją należały do ciebie.

— Tak, są rzeczy, które możesz zrobić. Dziękuję — odpowiedział Alek. — To mi bardzo pomoże, ponieważ martwię się tym, jak trudno będzie zaimplementować mocny silnik.

— Zatem zacznijmy od historyjki *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie*. Czy to tę historyjkę chcesz zaimplementować w pierwszej kolejności, Alku? — zapytała Daniela.

— Zgadza się. Przede wszystkim musimy zakodować klasy, które będą śledzić stan planszy, śledzić, czyja kolej i takie rzeczy. Stasiu, z pewnością będziesz mógł mi w tym pomóc.

— W porządku. Ile czasu, twoim zdaniem, to zajmie? Kilka dni?

— Kilka dobrych dni. Powiedzmy szesnaście godzin — odpowiedział Alek.

— Alku, będziemy zapisywać każde zadanie na kartce. Zapisz to na kartce i wpisz liczbę szesnaście w jej rogu, abyśmy nie zapomnieli, jaka jest nasza ocena — powiedział Karol.

Alek zapisał kartę zadania 23.1.



Karta zadania 23.1. Kodowanie klas zarządzania stanami. 16 godzin

— Czy powinienem umieścić na karcie swoje inicjały, żebyś wiedział, że to moja karta?

— Nie. Z treści wynika, że prawdopodobnie będziesz wykonywał to zadanie. Ale lepiej będzie, jeśli w tym momencie nie będziemy przypisywać zadań do konkretnych osób.

— Muszę to przetestować — powiedział Przemek. — To nie powinno zająć dużo czasu, ale to jest pierwszy kod w tym projekcie i chcę mieć pewność, że wszystko zostało dobrze skonfigurowane. Oceniam, że testowanie zajmie dziesięć godzin.

Zapisał kartę zadania 23.2.



Karta zadania 23.2. Napisanie automatycznych testów dla klas zarządzania stanami. 10 godzin

— Karolu, powiedziałeś mi wcześniej, że powinienem podzielić swoją pracę na małe części — stwierdził Alek. — Nie mogę po prostu powiedzieć, że napisanie silnika ruchów dla pierścieni potrwa dwa tygodnie?

— Nie, tak nie można. Kiedy identyfikujemy zadania, chcemy, aby mogły być zrealizowane w czasie od jednej do szesnastu godzin. Byłoby idealnie, gdyby udało się zakończyć jedno zadanie każdego dnia, co oznacza, że średnio zadanie powinno być ocenione na pięć do sześciu godzin, bo codziennie zawsze jest kilka innych rzeczy do zrobienia.

— W takim razie — powiedział Alek — mam zamiar napisać najpierw taki silnik ruchów, który będzie wystarczająco inteligentny, aby zagrać sześć pól po kolei, by stworzyć pierścień. Nie będzie przeciwnika, który by go próbował zatrzymać. Zainicjuję silnik w losowym miejscu na planszy i wymyślę, jak wykonać pierścień w sześciu ruchach. Dalej napiszę silnik w taki sposób, aby pierścień został stworzony nawet wtedy, gdy przeciwnik próbuje to zablokować. Następnie przejdę do zadań defensywnych dotyczących pierścienia. Zaprogramuję silnik, który będzie się starał zablokować innego gracza próbującego stworzyć pierścień. To by było na tyle, jeśli chodzi o historię dla mnie.

— OK, zapisałem kartę zadania, które brzmi *Zaprogramowanie silnika ruchów do tworzenia pierścienia bez przeciwnika*. Takie zadanie chciałeś wykonać w pierwszej kolejności — powiedział Staś.

— Realizacja tego zadania prawdopodobnie zajmie mi większość dnia. Zapisz na karcie sześć godzin, proszę — powiedział Alek.

— Alku, bez urazy, ale zawsze podchodzisz optymistycznie na początku tworzenia nowego silnika — powiedziała Róża, grafik.

— Tak, masz rację. Lepiej będzie, jeśli pomnożę tę ocenę przez dwa — zgodził się Alek.

Staś skreślił na karcie szóstkę i napisał na niej dwanaście godzin.

Następnie Karol sprowadził dyskusję do omówienia innych zadań, które zidentyfikował Alek. — Czy są również inne zadania w ramach tej historyjki, które nie zostały jeszcze zdefiniowane? — spytał.

— Możemy dołączyć kilka zadań związanych z testowaniem! — powiedział Przemek. — Gdyby Alek mógł przekazać mi kod po wykonaniu każdego z jego zadań, to ja zaraz potem mógłbym zdefiniować zadania związane z pisanem automatycznych testów. Jeśli znajdę jakieś błędy, to Alek dowie się o nich jeszcze wtedy, kiedy będzie miał kod świeżo w pamięci. Zapisałem zadania na kartach w czasie, gdy Alek opisywał sposób ich kodowania, ale dotychczas jeszcze ich nie oceniłem. Czy możemy zrobić to teraz wspólnie?

Kiedy skończyli omawianie historyjki *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie*, zapisali karty zadań, które zestawiono w tabeli 23.7.

TABELA 23.7. *Zadania i oceny historyjki Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie*

Zadanie	Liczba godzin
Kodowanie klas zarządzania stanem.	16
Pisanie automatycznych testów dla klas zarządzania stanem.	10
Zaprogramowanie silnika ruchów do tworzenia pierścienia bez przeciwnika.	12
Pisanie automatycznych testów dla silnika tworzącego pierścienie bez przeciwnika.	12
Zaprogramowanie funkcjonalności tworzenia pierścienia przez silnik ruchu w sytuacji, gdy gracz-człowiek stara się go zablokować.	8
Określenie przypadków testowych dla silnika próbującego wykonać pierścienie z blokadą.	4
Automatyzacja przypadków testowych dla tworzenia pierścienia z blokadą.	4
Zaprogramowanie w silniku ruchu funkcjonalności blokowania prób stworzenia pierścienia podejmowanych przez gracza-człowieka.	12
Określenie przypadków testowych dla blokowania gracza-człowieka w dążeniu do stworzenia pierścienia.	4
Automatyzacja przypadków testowych dla blokowania gracza-człowieka w dążeniu do stworzenia pierścienia.	2

— To około osiemdziesięciu czterech godzin w tej iteracji. Praca została podzielona na Alka, Stasia i Przemka — powiedział Karol. — Na podstawie tego, co zobaczyłem, pracując tutaj przez kilka dni, sądzę, że trzeba planować sześć godzin wydajnej pracy dziennie na osobę. Pozostała część dnia upływa na odpowiadaniu na e-maile, rozmowach z innymi zespołami projektowymi, codziennych spotkaniach. Pół dnia co drugi tydzień schodzi na planowanie takie jak to, które właśnie robimy, i tak dalej.

— Sześć wydaje się dla mnie rozsądne.

— Dla mnie też. Być może ocena powinna być bliższa pięciu. Mogę jednak spróbować pracować przez sześć godzin nad tym projektem, aby przekonać się, jak należy do tego podejść, oraz nie robić kilku innych rzeczy, które pochłaniają mój czas każdego dnia — powiedział Staś.

— Teraz, po podzieleniu pierwszej historyjki na zadania — powiedział Karol — trzeba sobie zadać pytanie, czy możemy się zobowiązać do zakończenia tej historyjki.

— Historyjki czy zadań? — zapytał Franek. — Do czego mamy się zobowiązać?

— Dobre pytanie, Franku — powiedział Karol. — Zobowiązujemy się do realizacji historyjki. Definiujemy zadania po to, żeby zobaczyć, ile pracy jest do zrobienia, tak by było łatwiej podjąć zobowiązanie. Ze względu na to, że zobowiązujemy się do historyjki, a nie tylko do zadań, to jest ważne, abyśmy myśleli o tym, jak o zobowiązaniu całego zespołu. Nie chcę, żeby na końcu iteracji Staś powiedział, że skończył pracę nad klasami zarządzania stanami, ale Alek nie zrealizował swoich zadań. To nie przyniosłoby ci niczego dobrego. Możecie myśleć o zadaniach, ale zobowiązujemy się do historyjek.

— To interesujące — powiedział Franek. — Podoba mi się to rozgraniczenie.

— Mogę zobowiązać się do realizacji tej historyjki — powiedział Przemek. — Mam trzydzieści sześć godzin zadań związanych z testowaniem. Przy sześciu godzinach przez dziesięć dni mam do dyspozycji sześćdziesiąt godzin.

— A razem z Alkiem mamy czterdzieści osiem godzin zadań. Nie będzie problemu — powiedział Staś.

— Doskonale. Zaplanowaliśmy pierwszą historyjkę i zobowiązaliśmy się ją zrealizować — powiedział Franek. — Przejdźmy do następnej historyjki w hierarchii ważności.

— Franku, zanim to zrobimy, spróbujmy zanotować dostępność wszystkich członków zespołu w ciągu najbliższych dwóch tygodni — powiedział Karol. — Powiedzieliśmy, że zakładamy sześć godzin dziennie, ale czy ktoś nie planuje urlopu albo jakiejś innej absencji w biurze?

— Ja biorę dwa dni wolnego. Mam kilka prac na wystawie w Warszawie, dlatego chcę tam pojechać — powiedziała Róża.

— Ja prawdopodobnie wezmę jeden dzień wolnego. Piątek albo poniedziałek — powiedział Alek.

— Moja mama i tata będą u mnie w przyszłym tygodniu. Zbyt dużo czasu z nimi doprowadza mnie do szału, a zatem będę pracować więcej! — zażartowała Daniela.

W tym czasie Karol robił notatki na tablicy wiszącej w pomieszczeniu zespołu. Zaprezentowano je w tabeli 23.8. — Dzięki temu zestawieniu będziemy wiedzieć, do czego możemy się zobowiązać, kiedy będziemy pracować nad kilkoma kolejnymi historyjkami — wyjaśnił Karol.

TABELA 23.8. Notatki Karola na temat planowanych absencji podczas iteracji

Kto	Czas absencji
Franek	
Alek	1 dzień?
Róża	2 dni
Staś	
Przemek	
Daniela	

— Dziękuję, Karolu — powiedział Franek. — Jaka jest następna najważniejsza rzecz do zrobienia?

— Bardzo bym chciała, aby była możliwa gra człowieka przeciwko człowiekowi — powiedziała Daniela.

— To dobry krok — potwierdził Alek. — To pozwoli nam zaprogramować logikę, która rozpoznaje, kiedy ktoś wygrał. Kiedy zaprogramuję pierwszą wersję silnika, możemy z łatwością dodać funkcjonalności cofania i powtarzania ruchów.

— Spróbujemy podzielić historijkę *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze* na zadania — powiedział Franek.

Wszyscy się zgodzili. Po dyskusji zidentyfikowano zadania, które wyszczególniono w tabeli 23.9.

TABELA 23.9. Zadania i oceny historijki *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze*

Zadanie	Liczba godzin
Bardzo prosta plansza i grafika.	4
Narysowanie pustej planszy.	2
Kliknięcie sześciokąta dodaje nowy pion żadanego koloru.	4
Komputer wie, kiedy postawienie pionu powoduje zakończenie zwycięskiego wzorca.	6
Zaprojektowanie testów.	6
Zautomatyzowanie testów.	8

— Zatem czy jako zespół możemy się zobowiązać do realizacji tej historijki oprócz poprzedniej? — zapytał Karol.

— Poczekaj, czy ta historijka nie obejmuje innej historijki, którą napisaliśmy? Mamy historijkę *Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę*. Dodaliśmy ją w ramach historijki opisującej grę przeciwko człowiekowi — powiedział Przemek. — Czy powinniśmy ją usunąć?

— Jeszcze nie wiem. Dołączyliśmy ją przypadkowo. Zobaczmy, czy możemy się zobowiązać do wykonania prac, które właśnie zidentyfikowaliśmy. Jeśli nie, to możemy usunąć wymaganie, aby komputer rozpoznawał zwycięskie figury, i pozwolić, aby to gracze decydowali, kiedy gra jest skończona — powiedziała Daniela. — Jak uważacie? Czy możemy się zobowiązać także do tej historijki?

— Tak — odpowiedzieli prawie równocześnie Alek i Staś.

— Dla mnie wydaje się w porządku — powiedziała Róża.

— Prawie — dodał Przemek.

— Co masz na myśli, Przemku? — zapytał Karol testera.

— Licząc obie historijki, mam prace zaplanowane na pięćdziesiąt godzin. Wiem, że powiedzieliśmy, że mamy w iteracji sześćdziesiąt, ale to wydaje mi się dużo. Musimy zobaczyć, jak idzie.

— Powinieneś pamiętać, że zadania testowe nie są przypisane konkretnie do ciebie, Przemku — powiedział Karol. — Każdy może je wykonać. Jeśli będziesz trochę z tyłu, to możesz oczekiwać od pozostałych członków zespołu pomocy w realizacji zadań testowych.

— Oczywiście. Ja zazwyczaj wykonuję kilka testów — powiedziała Róża. — Chciałam zacząć od projektu planszy w tej iteracji, ale to może poczekać, jeśli będę potrzebna do robienia testów.

— Dziękuję, Rózo — powiedział Przemek. — Myślę, że nawet bez pomocy innych osób mogę się zobowiązać do realizacji tej historyjki. Ale zbliżam się do kresu swoich możliwości.

Następnie dyskusja przeszła do kolejnej historyjki: *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty*. Do czasu zakończenia tej dyskusji, omówienia zadań, które będą się składały na tę historyjkę, oraz ilości pracy zespół zidentyfikował zadania i oceny zestawione w tabeli 23.10.

TABELA 23.10. Zadania i oceny historyjki *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty*

Zadanie	Liczba godzin
Silnik potrafi znaleźć ścieżkę z jednego narożnika do drugiego (czyli stworzyć most).	4
Zdefiniowanie i zautomatyzowanie testów dla prostego projektu mostu.	6
Silnik potrafi utworzyć most wokół przeszkód.	12
Zdefiniowanie testów dla utworzenia mostu wokół przeszkód.	6
Automatyzacja testów dla tworzenia mostu wokół przeszkód.	4
Silnik wie, kiedy zrezygnować z tworzenia określonego mostu.	8
Zdefiniowanie testów dla rezygnacji z budowania mostu.	4
Automatyzacja testów dla rezygnacji z budowania mostu.	2
Silnik próbuje przeszkodzić innemu graczowi w zbudowaniu mostu.	4
Zdefiniowanie testów dla blokowania innemu graczowi zbudowania mostu.	4
Automatyzacja testów dla blokowania tworzenia mostu przez innego gracza.	4
Silnik potrafi podjąć właściwą decyzję przy stawianiu pionu w pozycji, która prowadzi do utworzenia mostu oraz ruchu prowadzącego do stworzenia pierścienia.	16
Zdefiniowanie testów dla podejmowania decyzji stworzenia mostu lub pierścienia.	6
Automatyzacja testów dla podejmowania decyzji związanej z wyborem między mostem a pierścieniem.	2

— Czy możemy podjąć zobowiązanie także dla tej historyjki? — zapytała Róża.

Franek zauważył, że Róża była jedyną członkinią zespołu, która zadawała pozostałym członkom zespołu pytanie o zobowiązanie do realizacji historyjki, i pochwalił ją za to. Wolał, aby to jeden z członków zespołu zadawał takie pytanie, niż on jako menedżer produktu miałby pytać: „czy możecie się zobowiązać?”.

— Zsumowałem wszystkie godziny w miarę postępów w dyskusji — powiedział Staś, wstając z krzesła. — Oto jak wygląda to podsumowanie. — Narysował tabelę 23.11.

TABELA 23.11. Podsumowanie godzin dla trzech historyjek przygotowane przez Stasia

Historyjka	Liczba godzin programowania	Liczba godzin na testowanie	Liczba godzin na grafikę
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie.	48	36	0
Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze.	12	14	4
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty.	44	38	0
Razem	104	88	4

— Nie wiem jak wy, ale ja mogę się zobowiązać do czterech godzin w ciągu najbliższych dwóch tygodni — zażartowała Róża.

— Przy dwóch programistach, z których każdy pracuje trzydzieści godzin w tygodniu oraz przy iteracji trwającej dwa tygodnie, myślę, że z programowaniem nie będzie problemów — powiedział Staś.

— Zgadza się — przytaknął Alek.

— Ale nie czułbym się komfortowo, gdyby było choć trochę więcej pracy — powiedział Staś.

— A poza tym to zbyt wiele testów jak dla samego Przemka.

— Zamierzam trochę pomóc przy testowaniu — powiedziała Róża.

— Ja też — podchwyciła Daniela. — Prawdopodobnie nie zdołam nic zrobić przy automatyzacji, ale mogę pomóc w specyfikacji dobrych przypadków testowych.

— Myślę, że z pewną pomocą liczba godzin testowania jest do przyjęcia — powiedział Przemek.

— Sądzę, że możemy się zobowiązać do realizacji historyjek w zakładanym czasie.

— Czy wszyscy się z tym zgadzają? — zapytał Franek.

Nikt nie zgłosił sprzeciwu.

— Chciałabym dodać do iteracji jeszcze kilka zadań. Tak na wszelki wypadek, aby mieć pewność, że o niczym nie zapomnieliśmy — powiedziała Daniela. — Naprawdę muszę przeprowadzić rozmowy jeszcze z kilkoma graczami. Nie rozmawialiśmy jeszcze o moich badaniach produktu, ale pojawiło się kilka spraw, których tropem chciałabym podążać. Byłoby świetnie, gdybyśmy przeznaczyli dodatkowe dwanaście godzin na badania produktu. Nadal jednak będę mogła poświęcić większość swojego czasu na testowanie.

— I chciałabym zrobić jakieś wstępne rysunki planszy i pionów tak, aby można było puścić je w obieg. Zawsze potrzeba trochę czasu, aby uzyskać o nich wystarczająco dużo opinii. Chcę narysować cztery plansze i zbiory pionów. Wykonanie każdego zestawu zajmie mi cztery godziny.

— To wydaje się rozsądne — powiedział Franek. — Dodajmy te godziny.

— Tak więc czy po dodaniu wszystkich pozycji, które wymieniliśmy, możemy swobodnie zobowiązać się do realizacji wskazanych historyjek? — zapytał Staś. — Razem mamy 224 godziny. Skoro jest nas pięcioro: Alek, Przemek, Róża, Daniela i ja, to wyjdzie około pięciu godzin na dobę. Ale tak by było, gdyby wszystko przebiegło bez problemów i gdyby każdy mógł każdego zastąpić. Tak niestety nie jest.

— A poza tym — powiedział Karol — lepiej dodać coś w środku iteracji, niż coś odjąć.

Wszyscy zgodzili się, że poziom wysiłku był odpowiedni — ani zbyt trudno, ani zbyt łatwo — w sam raz. Wszyscy, zgodnie jako zespół, zobowiązali się, że dostarczą wymienione trzy historyjki w dwa tygodnie.

— Fantastycznie — powiedział Franek. — To niesamowite, że udało się osiągnąć tak widoczny postęp w tak krótkim czasie. Zwykle zajmuje znacznie więcej czasu, zanim można coś pokazać.

— Zróbmy piętnaście minut przerwy — powiedział Karol. — Kiedy wrócimy, chciałbym przejść do drugiego celu tego spotkania: planowania wydania. Zamierzam zrobić jakieś pierwsze oszacowanie, jak długo — naszym zdaniem — potrwa realizacja tego projektu.

Planowanie wydania

Gdy wszyscy wrócili i usiedli wokół stołu, Franek rozpoczął spotkanie. — Danielo, opowiedz nam o badaniu produktu oraz jakie są Twoje wnioski.

— Otóż — zaczęła Daniela — wysłałam e-mailem ponad 500 zaproszeń do wypełnienia webowej ankiety. Poza tym rozmawiałam telefonicznie z trzydziestoma pięcioma potencjalnymi nabywcami gry podobnej do tej, nad którą pracujemy. W sumie otrzymaliśmy odpowiedzi od siedemdziesięciu pięciu potencjalnych nabywców. Celem ankiety było ustalenie, jakie funkcjonalności spośród tych, które zidentyfikowaliśmy, są obowiązkowe — po prostu muszą być w naszej grze. Chcieliśmy również się dowiedzieć, jakie funkcjonalności są ekscytujące. Mogłoby ich nie być w grze, ale dodanie do gry kilku wabików sprawia, że gra staje się naprawdę znacznie bardziej atrakcyjna, nie mówiąc już o tym, że możemy za nią zażądać większej ceny. Wreszcie jest kilka funkcjonalności, które można nazwać liniowymi. Im więcej liniowych funkcjonalności znajdzie się w produkcie, tym więcej osób będzie skłonnych za niego zapłacić. Przykładem liniowych funkcjonalności jest liczba koni mechanicznych silnika samochodu, liczba kanałów telewizji satelitarnej oraz liczba poziomów w grze. Nie będę dokładnie tłumaczyć, w jaki sposób przeprowadziłam badanie. Ale jeśli jesteście zainteresowani, mogę podać rozdział w książce (patrz rozdział 11., „Priorytety atrakcyjności”). Wyniki podsumowałam na jednej stronie — zakończyła Daniela, wręczając wszystkim kartki z tabelą 23.12.

— Danielo. Na twojej liście jest znacznie mniej elementów, niż zidentyfikowaliśmy historyjek — powiedział Przemek. — Dlaczego?

— Im dłuższa ankieta, tym mniej chętnych na jej wypełnienie — odpowiedziała Daniela. — Z tego powodu scalałam historyjki według mojej oceny. Na przykład, „estetyczny wygląd” w mojej tabeli obejmuje trzy historyjki dotyczące estetycznej grafiki, ekranu powitalnego oraz możliwości przełączenia wyglądu planszy i pionów pomiędzy metalowymi a drewnianymi. Z drugiej strony, jeśli połączę zbyt wiele historyjek w jednym pytaniu, to odpowiedzi, które otrzymamy, mogą nie być zbyt przydatne.

— Mogę powiązać trzy pierwsze kolumny z opisem trzech typów funkcjonalności, Danielo. Ale co znaczą pojęcia „obojętna”, „odwrotna” i „wątpliwa”? — zapytała Róża.

TABELA 23.12. Rozkład wyników ankiety wśród użytkowników

Historyjka lub temat	Wabik	Liniowa	Obowiązkowa	Obojętna	Odwrotna	Wątpliwa	Kategoria
Słaby przeciwnik	8	34	50	4	4	0	Obowiązkowa
Średni przeciwnik	12	34	51	1	1	1	Obowiązkowa
Silny przeciwnik	10	16	39	32	2	1	Obojętna Obowiązkowa
Gra przeciwko innemu człowiekowi	5	23	14	58	1	0	Obojętna
Estetyczny wygląd	20	63	24	2	0	1	Liniowa
Wskazówki, ostrzeżenia przed złymi ruchami	47	21	21	0	0	1	Wabik
Interaktywny samouczek	35	24	21	18	2	0	Wabik
Muzyka w tle	15	48	21	12	2	2	Liniowa
Pomoc online	10	42	36	5	3	4	Liniowa
Zwycięska figura miga	18	12	55	9	3	3	Obowiązkowa
Statystyki	12	28	26	34	0	0	Obojętna
Komentarze do zapisanych gier	14	15	32	36	1	2	Obowiązkowa Obojętna

— Dobre pytanie. Odpowiedź „obojętna” jest prosta — po prostu oznacza, że odpowiadający — tak czy inaczej — nie ma ustalonej preferencji — odpowiedziała Daniela. — Ze względu na sposób postawienia pytania możemy mieć problem z interpretacją losowej opinii. Mogłoby się zdarzyć na przykład, że ktoś powiedział, że nie podoba mu się muzyka w tle, ale również nie podoba mu się, gdy nie ma muzyki w tle. To się zdarza. Ludzie, gdy się spieszą, myślą się albo błędnie interpretują pytania. Łatwo zauważyć, że odsetek odpowiedzi „odwrotna” i „wątpliwa” jest dość niski.

— Danielo, czy ja to dobrze odczytuję? — zapytała Róża. — Z odpowiedzi wynika, że funkcjonalność słabych i przeciętnych przeciwników jest obowiązkowa. Ale powiedziałaś, że silny przeciwnik to funkcjonalność zarówno obojętna, jak i obowiązkowa. Jak tak może być? Dlaczego nie wybieramy odpowiedzi z najwyższym procentem?

— Tak. Funkcjonalności obsługi słabych i przeciętnych przeciwników są obowiązkowe. Przyczyna dwóch szczytów dla wartościowania funkcjonalności silnego przeciwnika polega na tym, że potencjalni nabywcy prawdopodobnie należą do dwóch grup — tych, którzy już grają w *Havannah* i są w tym dobrzy, oraz tych, o których nie można tego powiedzieć. Ci, którzy są dobrzy, uważają, że jeśli mają kupić grę, to musi ona obsługiwać rozgrywkę przeciwko silnemu przeciwnikowi. Gracze początkujący mówią, że wystarczy im przeciwnik o średniej sile.

— To, co mnie uderzyło, Danielo, to fakt, że na tej liście nie ma zbyt wielu funkcjonalności, które ankietowani oceniają jako „wabik” — powiedział Franek w taki sposób, że zabrzmiało to bardziej jak pytanie niż stwierdzenie.

— To prawda. Ale z samego faktu, że produkt nie ma wielu wabików, nie wynika, że nie będzie ekscytujący — odpowiedziała analityczka. — Pamiętajcie, że wabiki to nieoczekiwane funkcjonalności. Jeśli kupię sobie nowy model Porsche, to z pewnością nie znajdę w nim zbyt wielu funkcjonalności, które mogłabym określić jako nieoczekiwane. Ale sam samochód nadal jest bardzo oczekiwany. Zgadzam się jednak z opinią, którą wyraziliście. Jeszcze nie zidentyfikowaliśmy wszystkich funkcjonalności, które powinny się znaleźć w tym produkcie. Każdy produkt musi mieć co najmniej jeden lub dwa wabiki — coś, co sprawi, że użytkownicy powiedzą „Wow!”. Z ankiety wynika, że jeszcze nie znaleźliśmy takich funkcjonalności. Dlatego poprosiłam o czas w najbliższej iteracji na kontynuację tego badania. Chcę przeprowadzić kilka otwartych dyskusji z niektórymi uczestnikami ankiety. Mam nadzieję, że to pomoże nam znaleźć jeden lub dwa wabiki.

— Świetnie Danielo. Bardzo pomocne — powiedział Franek. — Czy ktoś ma jeszcze do niej jakieś pytania? — Franek zrobił pauzę, po czym kontynuował: — Jeśli nie, to porozmawiajmy o tym, czego potrzebujemy w wydaniu gry w *Havannah*.

Nikt nie miał żadnych dodatkowych pytań. Natychmiast przeszli do planowania wydania przez określenie priorytetów dla funkcjonalności.

— Oto historyjki, które uznałam za obowiązkowe i o które nawet nie pytałam w ankiecie — powiedziała Daniela, rozkładając kilka kart historyjek na stole w sali konferencyjnej. Karty historyjek wraz z ocenami wybrane przez Daniellę zestawiono w tabeli 23.13.

TABELA 23.13. Historyjki, które w opinii Daniela są w tak oczywisty sposób obowiązkowe, że nie uwzględniła ich w ankiecie

Tekst historyjki	Ocena
Jako gracz mogę zacząć nową grę.	1
Jako gracz mogę przywrócić zapisaną grę.	2
Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę.	2
Jak gracz chcę mieć możliwość ustawienia pionu na planszy za pomocą klawiatury lub myszy.	3
Jako gracz chciałbym widzieć wizualny wskaźnik określający, czyja jest kolej.	2
Jako gracz chciałbym mieć wizualny wskaźnik ostatniego ruchu (może on na przykład migać).	2
Jako gracz chciałbym mieć możliwość cofania i powtarzania ruchów.	2
Jako gracz chciałbym mieć możliwość zapisania gry.	3
Jako gracz chciałbym mieć możliwość wyjścia z gry.	1
Jako gracz chciałbym mieć możliwość zrestartowania gry, aby zrezygnować i zacząć od początku.	1
Jako gracz chciałbym, aby na komputerze PC z zegarem 2,0 GHz wykonanie ruchu przez komputer nie zajmowało więcej niż dwie sekundy.	8
Jako gracz chciałbym móc poruszać się między ruchami, aby prześledzić grę, w którą grałem.	5
Razem	32

— Żadna z pozycji, które znalazły się na powyższej liście, nie powinna zaskakiwać — kontynuowała Daniela. — Takie funkcjonalności są w każdej grze, którą tworzymy. Obsługa ich to w istocie „koszty wejścia na rynek”.

Daniela wzięła kolejny zbiór kart. — W tej części stołu mam zamiar rozłożyć historyjki, które nasi klienci uznali za obowiązkowe. — W tej grupie Daniela rozłożyła karty, które zestawiono w tabeli 23.14.

TABELA 23.14. *Obowiązkowe historyjki na podstawie wstępnych badań Daniela*

Tekst historyjki	Ocena
Jako gracz mogę wybrać poziom gry komputera.	1
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie.	8
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty.	5
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje rozwidlenia.	8
Jako gracz mogę grać przeciwko silnikowi średniej mocy.	8
Jak gracz mogę grać przeciwko mocnemu silnikowi.	20
Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze.	3
Jako gracz chciałbym, aby wszystkie pionory zwycięskiej figury migały lub świeciły, tak bym mógł widzieć zwycięską figurę.	3
Razem	56

— Danielo, dlaczego na tej liście znalazła się obsługa mocnego silnika? — zapytał Alek. — To mi się podoba. Ale zgodnie z tym, co pokazałaś nam kilka minut temu, ta funkcjonalność została uznana przez użytkowników zarówno za obowiązkową, jak i za obojętną.

— Dobra obserwacja, Alku — powiedziała Daniela. — Robię zalecenie, zgodnie z którymi tę funkcjonalność uznaję za obowiązkową, ponieważ uważam, że wielu użytkowników będzie przeceniać to, co definiujemy, jako silnik średniej mocy. Ci użytkownicy będą się nudzić grą, jeśli nie dołączymy do wydania mocnego silnika.

— Nie sądzicie?

Frankowi podobała się ta dyskusja. Zespół często pozostawiał wszelkie zadania związane z definiowaniem produktu wyłącznie jemu — menedżerowi produktu — oraz Danieli, analityczce. Podobało mu się, że teraz członkowie zespołu byli zaangażowani w dyskusję oraz uważnie śledzili omawiane tematy. To mogło tylko pomóc nowemu produktowi.

— Kolejna historyjka, którą zaliczyłam do obowiązkowych, a którą warto omówić to *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze*. Uznałam ją za obowiązkową, ponieważ już wcześniej stwierdziliśmy, że to dobry kamień milowy. Następny stos — powiedziała Daniela, pokazując zbiór kolejnych kart — to historyjki, których wartość jest liniowa. Czyli im więcej takich funkcjonalności, tym lepiej, ale żadna z nich nie jest obowiązkowa.

Daniela rozłożyła na stole historyjki zestawione w tabeli 23.15.

TABELA 23.15. *Historyjki, które według badań Danieli dodają wartość liniowo (im więcej, tym lepiej)*

Tekst historyjki	Ocena
Jako gracz chciałbym widzieć estetyczny ekran startowy podczas ładowania programu.	5
Jako gracz chciałbym widzieć estetyczną grafikę w tle, która integruje się z planszą gry.	5
Jako gracz chciałbym mieć możliwość wyboru między drewnianą planszą z drewnianymi pionami a metalową planszą z metalowymi pionami.	8
Jako gracz chciałbym, aby system odtwarzał muzykę w tle.	5
Jako gracz mogę wybrać podkład muzyczny odtwarzany przez system.	1
Jako nowicjusz w grze potrzebuję dostępu do systemu pomocy online programu.	8
Razem	32

— Grafika nie jest opcjonalna — powiedziała Róża. — Potrzebujemy estetycznego ekranu startowego oraz atrakcyjnych plansz i teł.

— Nie powiedziałam, że grafika jest opcjonalna, Rózo — stwierdziła Daniela. — Masz rację, potrzebujemy jej. Jest pewien poziom podstawowy, poniżej którego nie można zejść. Gra powinna wyglądać estetycznie i dlatego zadałam takie pytanie w ankiecie. Funkcjonalności związane z grafiką są liniowe, ponieważ im ich więcej, tym lepiej. Dwa warianty planszy to lepiej niż jeden, ale gdyby były trzy, to byłyby jeszcze lepiej.

— OK. To ma sens — powiedział Franek. — Nadal leży przed tobą kilka kart historyjek. Jakiego typu są to historyjki?

— Kolejnych kilka kart to wabiki. Potencjalni klienci nie oczekują żadnego z nich, ale powinniśmy kilka z nich dołączyć, ponieważ znacznie podnoszą atrakcyjność gry. I, tak jak powiedziałam wcześniej, kilka wabików sprawi, że będziemy mogli zażądać wyższej ceny.

Daniela ułożyła na stole w sali konferencyjnej karty zestawione w tabeli 23.16.

TABELA 23.16. *Lista wabików zgodnie z badaniami Danieli*

Tekst historyjki	Ocena
Jako gracz chciałbym czasem poprosić o podpowiedź.	1
Jako nowicjusz w grze chcę mieć możliwość przeglądania interaktywnego samouczka gry.	8
Jako nowicjusz w grze chciałbym, żeby system ostrzegał mnie, jeśli wykonam bardzo zły ruch, i otrzymać szansę na jego cofnięcie.	8
Jako nowicjusz w grze chciałbym, aby — gdy będzie moja kolej — system wyświetlił mi pola, gdzie powinienem umieścić pion, ponieważ jeśli tego nie zrobię, to zrobi to komputer i wygra	3
Razem	20

— Oprócz wabików — kontynuowała Daniela — pozostały dwie historyjki, które potencjalni gracze określili jako obojętne. Nie sądzę, że powinniśmy je dodawać. — Położyła na stole karty, które

zestawiono w tabeli 23.17. — Jest mało prawdopodobne, że sprzedamy więcej egzemplarzy gier lub uzyskamy wyższą cenę przez dodanie funkcjonalności, które są dla użytkowników obojętne.

TABELA 23.17. Funkcjonalności, które zgodnie z badaniami Danieli są obojętne użytkownikom

Tekst historyjki	Ocena
Jako gracz chciałbym, aby system śledził, ile gier wygrałem, a ile przegrałem (według poziomu gracza komputerowego?).	3
Jako gracz chciałbym móc wprowadzać komentarze do zapisanych gier.	3
Razem	6

— Wow! — powiedział Franek. — To znacznie więcej informacji, niż zazwyczaj mamy na tym etapie. Świetna robota, Danielo. Wszystkie te dane zdobyłaś od ankietowanych?

— Tak. Ankieta celowo jest łatwa i po prostu pyta, jak oceniliby sytuację, gdyby określona funkcjonalność była dostępna, a jak gdyby nie była dostępna. Dzięki temu, że połączyłam niektóre pojedyncze historyjki użytkownika w tematy, w ankiecie znalazły się zaledwie dwadzieścia cztery pytania. Włącznie ze zwykłymi uprzejmościami przeprowadzenie ankiety telefonicznej zajęło około piętnastu minut. Na naszej stronie internetowej śledziliśmy czas, jaki potrzebował każdy respondent na wypełnienie formularza. Średnio było to siedem minut. Jednej osobie zajęło czterdzieści, ale podejrzewam, że odeszła od komputera albo odebrała telefon.

— Jakie są zatem nasze priorytety, Franku? — zapytał Staś.

— Cóż, musimy zaimplementować wszystkie funkcjonalności, które Daniela wskazała jako obowiązkowe, oraz te, które ankietowani uznali za obowiązkowe.

— Według naszych ocen to osiemdziesiąt osiem punktów — powiedział Staś.

— Jeśli chodzi o funkcjonalności, które Daniela określiła jako liniowe, zgadzam się z Różą: potrzebujemy estetycznej grafiki — powiedział Franek. — Powinniśmy zrealizować wszystkie te historyjki. Nie wyobrażam sobie również, aby nie było podkładu muzycznego. Być może użytkownik nie będzie mógł zmienić utworu, ale to tylko jeden element, dlatego chcę powiedzieć, że chcemy muzyki w tle. Potrzebujemy też pomocy online. Nie każdy, kto kupuje tę grę, będzie wiedział, jak się w nią gra.

— Zatem chcecie zaimplementować wszystkie funkcjonalności liniowe. To kolejne trzydzieści dwa punkty — powiedział Staś.

— Zgadzam się. Czy chcę wszystkich tych funkcjonalności? Tak jak każdy menedżer produktu na świecie.

— Nie mam nic przeciwko temu, ale wiem, że chcemy, aby produkt był gotowy w cztery miesiące. W tym momencie mamy około 120 punktów.

— Czy to nie zajmie więcej niż cztery miesiące? — zapytał Franek nieco zaniepokojony.

— Jeszcze nie wiem. Przekonajmy się, czego chcesz, a wtedy będziemy mogli stwierdzić, jak długo to potrwa — powiedział Staś.

— Brzmi dobrze — odpowiedział Franek. — Spośród wabików podobają mi się podpowiedzi. Chętnie widziałbym też samouczek. Być może moglibyśmy zrezygnować z niego albo z pomocy online, ale chciałbym zaplanować obie te funkcjonalności. Podoba mi się pomysł, aby powiedzieć użytkownikowi, że zrobił zły ruch natychmiast po jego wykonaniu.

— Pamiętajcie, że to historyjka oceniona na osiem punktów — przypomniał Alek pozostałym członkom zespołu. — Programuję silnik ruchów, aby umiał znajdować dobre ruchy. Mogę go poprawić, aby wyszukiwał naprawdę złe ruchy, ale to nie jest trywialne.

— Tak właśnie myślę. Na razie nie uwzględniamy w planie tej funkcjonalności. Ale uwzględnijmy historyjkę dotyczącą ostrzegania gracza, że przegra, jeśli nie postawi piona w określonym sześciokącie w celu zablokowania komputera.

— To dwanaście punktów z listy wabików. Czy chcesz jakichś funkcjonalności z listy „obojętnych”? — zapytał Staś.

— Właściwie nie — odpowiedział Franek. — Jaką mamy teraz sumę?

— Mamy osiemdziesiąt osiem punktów z listy funkcjonalności obowiązkowych, trzydzieści dwa z listy liniowych oraz dwanaście spośród wabików. To w sumie 132 — powiedział Staś.

— I zaimplementujecie to wszystko w ciągu dwóch iteracji. Zgadza się? — zapytał Franek z uśmiechem.

— Gdybyśmy tylko mogli, to byśmy to zrobili.

— Mówiąc poważnie, skąd wiadomo, ile czasu to nam zajmie?

— Najlepsze, co można zrobić, to poczekać trzy iteracje i wtedy udzielić odpowiedzi — wyjaśnił Karol. — Po każdej iteracji możemy zsumować punkty dotyczące zaimplementowanych historyjek. Nazywa się to tempem. Tempo zmienia się od iteracji do iteracji, ale jego średnia wartość w czasie jest stosunkowo stabilna.

— Zgadza się — wtrącił się Alek — mogliśmy się mylić w naszych ocenach. Lub też możemy mieć trochę szczęścia albo pecha podczas implementacji jakiejś historyjki.

— Po trzech iteracjach powinniśmy mieć dobry pogląd na to, jak szybko zespół osiąga postępy. Możemy podzielić całkowitą ilość pozostałej pracy przez tempo, co pozwoli nam ustalić, ile czasu zajmie nam wykonanie tej pracy — powiedział Karol.

— I ta ocena będzie dość dokładna? — zapytał Franek.

— Powinna być. Po trzech iteracjach powinniśmy mieć dobry pogląd na to, ile punktów jesteśmy w stanie zaimplementować w ciągu dwutygodniowej iteracji — powiedział Karol.

— Powinniśmy pamiętać o tym, że mam zamiar porozmawiać z kilkoma dodatkowymi potencjalnymi nabywcami i zobaczyć, czy uda się znaleźć więcej wabików — powiedziała Daniela. — Jak już wspominaliśmy, nasze wabiki wydają się słabe. Możemy dodać kilka dodatkowych funkcjonalności.

— Czy jest jakiś sposób, aby stworzyć teraz oczekiwany harmonogram?

— Oczywiście, Franku — odpowiedział Karol. — W pierwszej iteracji planowaliście zaimplementowanie czterech historyjek. Możemy dodać punkty związane z każdą z nich i uznać tę wartość za nasze planowane tempo. Nie wiemy, czy takie będzie, ale takie jest nasze wstępne założenie.

— Już to zrobiłem — powiedział Przemek. — Suma wynosi osiemnaście. Mamy osiem punktów za historyjkę *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje pierścienie*. Pięć punktów za historyjkę *Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje mosty*. Mamy też dwa punkty za historyjkę *Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę* i trzy punkty za historyjkę *Jako gracz chciałbym mieć możliwość skorzystania z systemu, by móc grać przeciwko innej osobie na moim komputerze*.

— Ile iteracji potrwa projekt, jeśli nasze tempo wynosi osiemnaście?

— Nieco ponad siedem iteracji. Mamy 132 punkty. Siedem razy osiemnaście to 126. Czyli sześć punktów więcej niż siedem iteracji — powiedział Przemek.

— Czy możemy po prostu przyjąć siedem iteracji? — zapytał Franek.

— Moglibyśmy, ale nie powinniśmy — odpowiedział Karol. — Wolałbym, aby oceny były podawane w postaci zakresu. Jeśli rozpatrujemy zadania, których oceny mieszczą się w ośmiu iteracjach, to powinniśmy raczej powiedzieć, że realizacja projektu potrwa od sześciu do dziesięciu iteracji.

— Zdecydowanie nie podoba mi się, aby ten projekt potrwał dwadzieścia tygodni. Ale mówicie także, że może potrwać tylko dwanaście. Zgadza się? — zapytał Franek.

— Być może. Ale równie dobrze może potrwać dwadzieścia — powtórzył stanowczo Karol.

— Możesz przeciwdziałać zbyt długiemu czasowi trwania projektu przez usuwanie funkcjonalności o najniższym priorytecie, jeśli tempo pokaże, że zbliżamy się do dziesięciu iteracji. Pamiętajmy także, że Daniela nadal aktywnie poszukuje nowych funkcjonalności. Jeśli coś znajdzie, to będziemy musieli przedłużyć harmonogram albo usunąć taką samą liczbę punktów z funkcjonalności, jaką wybraliśmy dotychczas.

— OK. Realizację projektu w okresie od dwunastu do dwudziestu tygodni bez trudu uzasadnię innym menedżerom — powiedział Franek. — Niech to będzie nasz plan wyjściowy.

— Franku, muszę powtórzyć, że najlepiej będzie poczekać co najmniej dwa tygodnie, aby zespół wykonał jedną iterację. Następnie powinniśmy zweryfikować ten wstępny plan, korzystając z co najmniej jednej obserwacji rzeczywistego tempa — powiedział Karol.

— Zrozumiałem. Jednak wiele osób chce zyskać przynajmniej szacunkowe wyobrażenie, jak wygląda harmonogram tego projektu.

— Trzeba zadbać o to, by wiedziały one, że co dwa tygodnie jesteśmy w stanie dostarczyć dokładniejszych informacji. Zwłaszcza po pierwszych trzech iteracjach powinniśmy wiedzieć znacznie więcej — dodał Przemek.

— Czy zatem powinniśmy zaplanować, nad czym będziemy pracować w każdej z tych iteracji?

— zapytał Franek.

— Niekoniecznie — odpowiedział Karol. — Najważniejsze to wiedzieć, nad czym będziemy pracować w następnej iteracji. Zrobiliśmy to podczas planowania iteracji. Kolejną najważniejszą rzeczą jest poznanie tych funkcjonalności z dolnej części listy, które mogą, lecz nie muszą zostać zrealizowane podczas wydania.

— Rozumiem, dlaczego trzeba znać zadania o najwyższych priorytetach, ale dlaczego interesują nas te, które mają priorytet najniższy? — zapytał Franek.

— Nie jest to bardzo ważne dla tego pierwszego spotkania, ale czasami nabiera znaczenia, kiedy zbliżamy się do końca wydania. Na przykład nie chcielibyśmy, aby dział marketingu zaprojektował okładkę z napisem „Teraz z muzyką w tle”, jeśli ta funkcjonalność miałaby zostać usunięta w ostatniej chwili.

— OK. Ale dlaczego nie poświęcić dziś trochę czasu na zaplanowanie tego, co będziemy robić w każdej iteracji?

— To nie ma sensu — powiedział Karol. — Za dwa tygodnie będziemy wiedzieć o wiele więcej w porównaniu z tym, co wiemy dzisiaj. Po co planować, co będziemy robić w iteracji wcześniej, niż trzeba?

Nie musimy tego robić. W większych projektach, zwłaszcza takich z uczestnictwem kilku zespołów, które muszą koordynować pracę pomiędzy sobą, możemy planować na dwie lub trzy iteracje do przodu. Ale nie musimy tego robić w tym projekcie.

— Czy zatem możemy uznać plan wydania za zrealizowany? — zapytał Franek.

— Tak. Zebrałem karty historyjek, które wskazałeś jako niezbędne w tym wydaniu, i to jest nasz plan — powiedział Karol. — Zanim się znowu spotkamy za dwa tygodnie od dziś, powinniśmy porozmawiać z Danielą o historyjkach i wskazać te, które chcielibyście najbardziej widzieć w następnej iteracji. Na spotkaniu porozmawiamy o tym, dlaczego chcemy zrealizować te historyjki. Pozostali członkowie zespołu mogą was poprosić o przemyślenie tej decyzji i wskazanie jednej lub dwóch historyjek, nad którymi chcą pracować, ponieważ te historyjki są ryzykowne albo dlatego, że ich implementacja pozwala się wiele nauczyć. Jednak ostateczna decyzja dotycząca ustalania priorytetów należy do was.

— Nie zrozum mnie źle — powiedział Franek — ale jakoś tego dziś nie odczułem.

— Rzeczywiście tak było — powiedział Karol — ale wybór właściwych funkcjonalności do zaimplementowania w następnej kolejności to z pewnością wysiłek całego zespołu. Gdyby programiści wybrali zbiór technikaliów albo funkcjonalności, których nie byłbym w stanie sobie wyobrazić, to zatrzymałbym ich i polecił pracę nad innymi historyjkami. Zakładam, że jesteś zadowolony z historyjek, które zostały wybrane?

— Jestem zachwycony. Będzie wspaniale zobaczyć je wszystkie zaimplementowane za dwa tygodnie.

Dwa tygodnie później

— Chciałbym, żebyśmy wszystko skończyli. Ale pomimo to jestem pod wrażeniem tego, co mamy do tej pory — powiedział Staś koleżankom i kolegom z zespołu podczas oczekiwania na rozpoczęcie spotkania.

— Bardzo mi przykro, że musieliśmy zrezygnować z historyjki *Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę*. Ale bardzo potrzebowałem twojej pomocy w implementacji kodu silnika ruchów, Stasiu. Dziękuję — Alek podziękował drugiemu programiście.

— Nie ma problemu, Alku. To nasz wspólny wysiłek. Możliwość zademonstrowania silnika, który tworzy pierścienie i mosty, jest bardzo ważna — powiedział Staś.

— Czy zatem jesteśmy gotowi na demo? — zapytał Franek, wchodząc do pokoju punktualnie o dziewiątej. Położył na dużym stole pudełko precli.

— Absolutnie — powiedział Staś. — Czy wiadomo ci coś o tym, aby któryś z innych menedżerów albo innych uczestników projektu miał przyjść obserwować przegląd iteracji?

— Wiem, że Filip i Laura zapowiadali swoją obecność — powiedział Franek, mówiąc o dyrektorze generalnym i finansowym.

— Swoją obecność zapowiedziało również kilku inżynierów z innych zespołów. Są ciekawi naszego nowego procesu Agile — powiedział Alek.

W ciągu kolejnej pół godziny zespół zaprezentował, co zostało zrealizowane. Pokazał silnik gry, który tworzy pierścienie i mosty. Zademonstrował również grę przeciwko sobie dwóch osób, które przekazują między sobą klawiaturę w celu wykonywania ruchów. Udało się nawet połączyć ze sobą

dwie funkcjonalności, dzięki czemu człowiek mógł grać przeciwko komputerowi. Człowiek zwykle wygrywał, ale to nie było zaskakujące, biorąc pod uwagę, że zaimplementowano słaby silnik ruchów.

Grafika była bardzo prosta — proste ilustracje wykonane przez Różę. Ale Filip, dyrektor generalny, był zachwycony, widząc, że po zaledwie dwóch tygodniach udało się opracować pewną, niewielką część gry, która była w pełni funkcjonalna. Jego zadowolenie wzrosło jeszcze bardziej, gdy Przemek wręczył mu klawiaturę, by zagrał samodzielnie, i z trudem udało mu się wygrać nową grę.

— To fantastyczna praca wszystkich członków zespołu — ogłosił Filip, wstając ze swojego krzesła. — Czy co dwa tygodnie będziecie pokazywać mi więcej?

— Oczywiście. Co dwa tygodnie, tutaj o dziewiątej — powiedział Staś.

— Przyjdę także następnym razem. Umieszcze te spotkania w swoim kalendarzu. Te trzydzieści minut oglądania naprawdę działającego oprogramowania jest bardziej przydatne niż jakiegokolwiek raporty o stanie, które kiedykolwiek czytałem. A jak wygląda wasz harmonogram? Czy mogę o to spytać teraz?

— Możesz — powiedział Franek. — Na razie trzymamy się naszych szacunków. Zgodnie z nimi realizacja projektu potrwa od dwunastu do dwudziestu tygodni. Chcemy zaplanować następną iterację po tym spotkaniu. Będziemy wiedzieć więcej za dwa do czterech tygodni. Do tego czasu musimy być w stanie zobowiązać się do harmonogramu, o którym opowiemy wydawcom.

— Świetnie — powiedział Filip. Razem z Laurą wyszli z sali konferencyjnej. To samo zrobiło kilka innych osób, które przyszły obejrzeć postępy w projekcie.

Planowanie drugiej iteracji

— Jeśli dobrze pamiętam — powiedział Franek — szacowaliśmy tempo na osiemnaście punktów.

— Zgadza się — powiedziała Daniela.

— Ale nie udało się zakończyć historyjki dotyczącej rozpoznania stanu, gdy gra została wygrana. Zatem nasze tempo jest niższe niż osiemnaście. Czy przyznamy sobie jeden punkt za zaimplementowanie połowy tej historyjki? — zapytał Franek.

— Nie — powiedział Karol. — Ogólnie rzecz biorąc, w przypadku punktów za historyjki będziemy stosować podejście „wszystko albo nic”. Przyznawanie sobie części punktów jest zbyt trudne, ponieważ w rzeczywistości nie wiemy, ile pracy zostało. Zamiast zgadywać, nie przyznajemy sobie żadnych punktów do czasu, aż zdobędziemy wszystkie.

— Tak więc nasze tempo wyniosło szesnaście, ponieważ początkowo oceniliśmy niedokończoną historyjkę na dwa punkty — powiedziała Róża.

— Musimy uwzględnić w planie, że projekt potrwa dłużej, prawda? — zapytał Franek.

— Nadal mieścimy się w granicach dwunastu do dwudziestu tygodni, o których mówiliśmy menedżerom — powiedział Staś. — Zaczęliśmy od 132 punktów. Udało się zaimplementować 16, więc zostało jeszcze 116. Podzielenie 116 przez 16 daje wynik 7,25. Dla bezpieczeństwa oszacujmy, że potrzeba nam jeszcze ośmiu dwutygodniowych iteracji. Włącznie z dwoma tygodniami, które właśnie przepracowaliśmy, mamy w sumie osiemnaście tygodni.

— To o dwa tygodnie dłużej niż cztery miesiące, Franku. Czy to ważne? — zapytał Przemek.

— Nie sędzę. Wydanie nie zostało związane z żadną konkretną datą. Po prostu chcemy, aby zostało zrealizowane jak najszybciej — powiedział Franek. — Nie miałem okazji wam jeszcze o tym powiedzieć: nadal nie sfinalizowaliśmy kontraktu na grę *Deep Black & White* z japońskim wydawcą. Oni wciąż chcą wydać tę grę. Jesteśmy blisko podpisania ostatecznej umowy. — Pokazał palcami milimetrową odległość. — Nie chcą jednak wydać gry do czasu przeprowadzenia dużej kampanii marketingowej z reklamami we wszystkich głównych czasopismach. To prawdopodobnie potrwa cztery miesiące. I zgodnie z warunkami kontraktu nie zobaczymy żadnych tantiem jeszcze przez kwartał po wydaniu gry. Może się zdarzyć, że upłynie jeszcze osiem miesięcy, zanim gra *Deep Black & White* zacznie przynosić jakiegokolwiek realne pieniądze. Byłoby idealnie, gdyby gra *Havannah* okazała się naszym szybkim zwycięstwem. Być może nie w takiej skali jak *Deep Black & White*. Ale ponieważ ta gra nie wymaga wielkiej kampanii marketingowej, to dochody będą znacznie szybciej.

— Franku, przykro mi, ale musiałam zrobić więcej testów, niż się spodziewałam, dlatego w tej iteracji nie zrobiłam zbyt wielu postępów w badaniach produktu — powiedziała Daniela.

— OK. Nie ma problemu, ale tym razem potraktujmy je priorytetowo. Nawet gdyby oznaczało to konieczność rezygnacji z zadań związanych z testowaniem — powiedział Franek.

— Zatem Franku, jakie historyjki chciałbyś zrealizować w tej iteracji najbardziej?

Zespół zaplanował drugą iterację w taki sam sposób, w jaki planował pierwszą. Kiedy skończył, zobowiązał się do implementacji historyjek użytkownika zestawionych w tabeli 23.18.

TABELA 23.18. Historyjki użytkownika zaplanowane w drugiej iteracji

Tekst historyjki	Ocena
Jako gracz chciałbym, aby komputer umiał rozpoznać zwycięską figurę.	2
Jako gracz mogę grać przeciwko słabemu silnikowi, który rozpoznaje rozwidlenia.	8
Jako gracz chciałbym mieć możliwość wyboru między drewnianą planszą z drewnianymi pionami a metalową planszą z metalowymi pionami.	8
Razem	18

— Dlaczego planujemy w tej iteracji implementację osiemnastu punktów, skoro nasze tempo w ostatniej iteracji wynosiło szesnaście? — zapytał Franek.

— Użyliśmy tempa jako wskaźnika oceny postępów w realizacji planu wydania — powiedział Karol. — Nie planujemy automatycznie każdej nowej iteracji na dokładną liczbę punktów zrealizowanych w ostatniej iteracji ani na średnią wartość tempa z kilku ostatnich iteracji. Planujemy każdą iterację, biorąc pod uwagę, do jakiej ilości pracy możemy się zobowiązać. Dodajemy punkty do siebie dopiero wtedy, gdy uzgodnimy historyjki, do których możemy się zobowiązać. Gdy to zrobimy, tempo powinno być bliskie naszej historycznej średniej, ale to się będzie zmieniać.

— Dlaczego?

— Są dwie podstawowe przyczyny — zaczął Staś. — Po pierwsze, tym razem skuteczniej wykorzystamy Różę. Ona jest przede wszystkim grafiką, a nie testerem. Świetnie, że w poprzedniej iteracji tak bardzo pomogła nam z testowaniem, ale tym razem zajmie się tworzeniem grafik dla drewnianej i metalowej planszy i pionów. Napiszę kod, który pozwoli użytkownikom wybierać pomiędzy nimi, a Przemek prawdopodobnie zajmie się testami dla tej historyjki. Ale poza tym większą część

z ośmiu punktów tej historyjki dostarczy Róża. Po drugie, mamy trochę zrobione z historyjki dotyczącej rozpoznawania zwycięskiej figury. Zaczęliśmy ją w poprzedniej iteracji, ale nie doszliśmy zbyt daleko. Jestem optymistą, jeśli chodzi o historyjki, które wybraliśmy. Choć znowu mają wartość osiemnastu punktów, to po zsumowaniu mamy piętnaście godzin mniej niż w ostatniej iteracji.

— Ja czuję się przekonany — powiedział Franek. — Zróbmy to.

Kolejne dwa tygodnie później

Na następnym przeglądzie iteracji zespół ponownie zaprezentował dobre postępy. Zrealizował wszystkie historyjki, które zaplanował. Daniela znalazła czas, by zakończyć swoje badania produktu. Na spotkaniu ponownie pojawili się menedżerowie — Filip i Laura. Ponieważ rozeszła się wiadomość o tym, jak wiele zostało zrobione w ciągu pierwszych dwóch tygodni, na drugim spotkaniu przeglądu iteracji pojawiło się jeszcze więcej osób z innych zespołów.

— Gra nabiera kształtów. Nie mogę uwierzyć, że twój silnik ruchów już potrafi mnie pokonać — powiedział Filip. — Różo, twoje pierwsze dwie plansze wyglądają fantastycznie. Czy nadal mamy ten sam harmonogram? Wydanie nastąpi od ośmiu do szesnastu tygodni od dziś.

— Nasze tempo postępów wynosi prawie dokładnie tyle, ile oczekiwaliśmy — powiedział Franek. — Zatem harmonogram wydaje się niezagrożony. Daniela przeprowadziła podczas tej iteracji dodatkowe badania produktu i odkryła kilka interesujących własności, które być może warto uwzględnić. Przekażę informacje na ten temat, ale najpierw chciałbym usłyszeć opinię zespołu. Być może zaproponuję zmianę harmonogramu. Jesteśmy przekonani, że nowe funkcjonalności przyniosą nam więcej pieniędzy. Teraz muszę to rozważyć w zestawieniu z dodatkowymi kosztami prac projektowych i dodatkowym czasem.

— Podobają mi się te części, która mówi o większych przychodach — powiedziała Laura, główny dyrektor finansowy.

— Mnie też — zgodził się Filip. — Ale nie podobają mi się zbyt mocno możliwości opóźnienia terminu.

— Nie będzie opóźnienia, Filipie — powiedział Franek. — Jeśli chcesz dostarczyć produkt w pierwotnie planowanym okresie od dwunastu do dwudziestu tygodni, to nie ma z tym problemu. To, o czym będziemy dyskutować po tym spotkaniu, to ilość czasu potrzebna do stworzenia bardziej wartościowej gry. A wtedy zobaczymy, czy są jakieś funkcjonalności, które można usunąć, dzięki czemu można by uzyskać czas potrzebny na dostarczenie bardziej wartościowej gry w terminie bliższym dwudziestu tygodniom, które podaliśmy jako naszą górną granicę.

— Zawsze jestem zainteresowany tworzeniem bardziej wartościowych gier. Poinformuj mnie, jakie są wasze zalecenia — powiedział Filip, po czym wstał i wyszedł z sali konferencyjnej. Pozostali goście także wyszli. Na sali pozostali tylko członkowie zespołu projektowego.

Przegląd planu wydania

— Franek już wam powiedział, że odkryłam kilka nowych wabików — powiedziała Daniela.

— Kilkoro z was słyszało, jak rozmawiałam o nich przez telefon z potencjalnymi klientami. Dobrą wiadomością jest to, że nasza grupa docelowa naprawdę chce dwóch funkcjonalności, o których nie pomyśleliśmy. Przede wszystkim chce, aby gracz-komputer mógł mieć dwie różne osobowości, które

grają w różny sposób. Jedna osobowość ma być bardzo agresywna, a druga zachowawcza. Wielu klientów pyta o postać, która z nich szydzi. Mówi obraźliwe rzeczy po nieudanym ruchu, śmieje się z nich, kiedy przegrają, rzeczy tego rodzaju.

— To brzmi dość zabawnie — powiedział Alek. — To wpłynie na zmiany w silniku ruchów, ale prawdopodobnie nie będą one zbyt wielkie. Silnik już zapewnia możliwość konfigurowania w taki sposób, by była możliwa gra bardziej agresywna lub bardziej zachowawcza.

— To pierwsza nowa funkcjonalność — kontynuowała Daniela. — Drugą jest gra online. Potencjalni klienci chcą grać w tę grę online przeciwko innym osobom.

— To duża sprawa — powiedział Staś. — Musielibyśmy uzyskać miejsce w centrum danych, zakupić sprzęt i zatrudnić ludzi.

— Możemy zamówić wszystkie te usługi u partnerów — powiedział Franek. — Nie wykluczam możliwości zrealizowania tego wabika, ale masz rację, Stasiu, to duża sprawa. Danielo, która z tych funkcjonalności, twoim zdaniem, jest ważniejsza?

— Osobowości gry. Różne charaktery komputerowego przeciwnika to olbrzymi wabik. To funkcjonalność, która z pewnością spodoba się użytkownikom, a której się nie spodziewają — powiedziała Daniela.

— Danielo, właśnie powiedziałaś nam, że rozszerzyliśmy zakres. A już mieściliśmy się w górnej granicy naszej oceny: od dwunastu do dwudziestu tygodni — powiedział Przemek. — Dlaczego powiedziałaś, że masz dobre wieści? Czy masz nam jeszcze coś do zakomunikowania?

— Nie, to wszystko, Przemku — powiedziała Daniela. — Ale to jest dobra wiadomość. Właśnie znaleźliśmy dwie nowe funkcjonalności, które mogą nam pomóc sprzedać więcej egzemplarzy gry po wyższej cenie. Tak, jeśli zdecydujemy się na ich implementację, to harmonogram się zmieni. Ale dobrze jest mieć takie problemy. Możemy dostarczyć produkt, bazując na obecnym harmonogramie, i zarobić określoną sumę pieniędzy. Albo możemy zmienić harmonogram i zaplanować zarabianie większej sumy pieniędzy. Plany zawsze są prognozami reprezentującymi określony punkt w czasie. Właśnie znaleźliśmy lepszą przyszłość, do której warto zmierzać. Już napisałam historyjki użytkownika dla obu funkcjonalności. Powinniśmy je ocenić i zdecydować, czy chcemy zmienić plany, aby uwzględnić którąkolwiek lub obie.

— Zróbmy to — powiedział Franek. — Jestem gotów zmienić datę projektu, jeśli dzięki temu uda się zarobić więcej pieniędzy, a opóźnienie nie będzie zbyt duże. Wygląda na to, że funkcjonalności, o których mówimy, mogą być bardzo lukratywne.

— Ale, Danielo, dlaczego ankiety, które rozesłałaś, nie zidentyfikowały tych funkcjonalności? — powiedział Przemek.

— Ankiety są dobre do ustalania priorytetów tego, o czym już wiemy — powiedziała Daniela. — Z oczywistych względów nie są dobre do wykrywania funkcjonalności, o których nie wiemy. W niektórych odesłanych ankietach wspomniano o osobowościach i grze online, ale nie na tyle, abym pomyślała, że to jest trend. Ostatnią iterację poświęciłam na rozmowy z większą liczbą potencjalnych klientów, aby upewnić się, że to były dobre pomysły i dostosować modele finansowe w taki sposób, aby pokazały mi, o ile więcej dochodów mogą przynieść te nowe funkcjonalności.

— Podoba mi się to — powiedział Alek. — Jeśli gra zarobi więcej pieniędzy, to być może Franek, aby uczcić wydanie, kupi nam coś bardziej ekskluzywnego niż fatburgery.

— Mnie smakowały burgery, którymi uczciliśmy wydanie *Deep Black & White* — powiedział Przemek.

— A mnie nie. Po wydaniu tej gry będę namawiała Franka, żeby skoczył po sushi — powiedziała Róża.

Zespół oszacował nowe historyjki, które napisała Daniela. Podczas dokonywania ocen członkowie zespołu podzielili niektóre wyjściowe historyjki i połączyli inne. Kiedy skończyli, suma punktów dla gry online wyniosła trzydzieści. Utworzenie pierwszej osobowości oszacowano na piętnaście punktów. Każda następna osobowość oznaczałaby dodatkowych pięć punktów. Członkowie zespołu omówili problem i uzgodnili, że pięć osobowości będzie rozsądną liczbą. To oznaczało dodatkowych trzydzieści pięć punktów na implementację postaci o różnych osobowościach.

Ponieważ z prognoz finansowych Daniela wynikało, że osobowości prawdopodobnie wygenerują większe przychody niż gra online, wszyscy zgodzili się, że muszą skorygować plan tak, aby uwzględnić osobowości, ale nie uwzględniać wersji online.

— W pierwszych dwóch iteracjach udało się zrealizować najpierw szesnaście, a potem osiemnaście punktów. Właśnie dodaliśmy trzydzieści pięć punktów. Cofamy się! — zażartował Alek.

— Masz rację, Alku — powiedział Karol, sięgając po kalkulator. — Mamy 133 punkty. Przy naszym obecnym średnim tempie 17 punktów na jedną iterację, to oznacza 7,8 dodatkowych iteracji.

— Osiem dodatkowych iteracji plus dwie już wykonane oznacza, że skończymy projekt w dziesięciu iteracjach — powiedział Franek.

— Zbliżamy się bardzo blisko do ostatecznego terminu — powiedział Staś. — Tak, wygląda na to, że zrealizujemy projekt w dziesięciu iteracjach. Dobrze, że nasz pierwotny harmonogram określał zakres i nie powiedzieliśmy po prostu „osiem iteracji”. Jeśli jednak coś pójdzie nie tak — jeśli historyjka okaże się większa, niż sądziliśmy, jeśli spadnie tempo albo ktoś zachoruje — to realizacja projektu może potrwać dłużej niż osiem dodatkowych iteracji.

— Czy chcesz nam powiedzieć, że trzeba zmienić harmonogram? — zapytał Franek.

— Niekoniecznie — odparł Karol. — Jeśli jednak chcemy dotrzymać harmonogramu dziesięciu iteracji, to musimy przygotować się na ewentualność usunięcia kilku historyjek o niższych priorytetach w przypadku, gdy zacznie nam brakować czasu. Jeśli nie chcemy tego robić, to musimy przekazać prognozę, że realizacja projektu może zająć więcej niż dziesięć iteracji.

— Sądzę, że można usunąć kilka historyjek, jeśli to konieczne — powiedział Franek.

— Dobrze. Pamiętajcie, jak mówiłem wam, że to ważne, aby znać historyjki użytkownika z górnej części listy priorytetów oraz te, które są na dole tej listy? To jest ten moment, kiedy warto znać te, które są na dole. Będę czuć się bardziej komfortowo, utrzymując prognozy realizacji projektu w czasie nieprzekraczającym dziesięciu iteracji, jeśli uda wam się wskazać warte co najmniej siedemnaście punktów historyjki, bez których będziemy mogli się obyć, gdyby projekt miał trwać więcej niż dziesięć iteracji — powiedział Karol.

— Oto kilka historyjek, bez których mogę się obyć: *Jako nowicjusz w grze chciałbym, żeby system ostrzegał mnie, jeśli wykonam bardzo zły ruch, i otrzymać szansę na jego cofnięcie oraz Jako nowicjusz w grze chcę mieć możliwość przeglądania interaktywnego samouczka gry*. Każda z tych historyjek została oceniona na osiem punktów. Powiedziałem wcześniej, że mógłbym się obyć bez realizacji ocenionych na sześć punktów historyjek dla muzyki w tle.

— Zatem powinniśmy zaplanować je jako ostatnie. Jeśli w okolicach dziesiątej iteracji zacznie brakować czasu, będziemy mogli usunąć jedną lub kilka historyjek z dolnej części listy — powiedział Karol.

— A jeśli będziemy pracować zgodnie z harmonogramem i zdecydujemy się nie implementować tych historyjek, to będziemy mogli zrobić wydanie co najmniej jedną iterację wcześniej. Czy tak? Kiedy mogłoby być wydanie, gdybyśmy zrezygnowali z tych historyjek? — zapytał Franek.

— Mamy zaplanowane 133 punkty. Jesteście gotowi usunąć 22. To daje nam 111 — powiedział Karol, sięgając po kalkulator. — Przy tempie siedemnaście potrzebujemy dodatkowych siedmiu iteracji. Projekt może zostać zrealizowany w 18 zamiast w 20 tygodni.

— Zostawmy na razie te historyjki w planie. Usunę je, jeśli będzie to konieczne, ale one chyba są warte dwóch dodatkowych tygodni — powiedział Franek. — Chcę pokazać nowy plan Filipowi. Prosił, żebym poinformował go o tym, co postanowiliśmy. Czy ktoś chce iść do niego ze mną?

Daniela i Alek powiedzieli, że chcą. Fakt, że Daniela chciała iść z Frankiem, nie był żadnym zaskoczeniem. Jako analityczka Daniela często uczestniczyła w podejmowaniu decyzji dotyczących produktów i harmonogramów. Franek był jednak zaskoczony zainteresowaniem Alka. Zazwyczaj Alek — jedna z najbardziej zapalonych osób technicznych w firmie Bomb Shelter — nie chciał mieć nic wspólnego z dyskusjami na tematy biznesowe. Tymczasem mocno zaangażował się w te dyskusje, poczynawszy od rozpoczęcia projektu cztery tygodnie temu. Franek to doceniał. Uczestnictwo Alka mogło doprowadzić do podjęcia lepszych decyzji biznesowych.

— Idę tam teraz — powiedział Franek. — Chodźmy.

— Czy nie powinniśmy wcześniej przygotować czegoś dla Filipa? — zapytał Alek.

— Nie trzeba. Jest u niego tablica. Będziemy po prostu na niej rysować.

Zaprezentowanie Filipowi skorygowanego planu

— Witaj, Filipie. Czy nadal chcesz się dowiedzieć, co zdecydowaliśmy w związku z nowymi funkcjonalnościami w grze *Havannah*? — zapytał Franek.

— Oczywiście. Czy to są te funkcjonalności, o których Daniela wspominała dziś rano?

— Tak.

— Mam teraz około piętnastu minut. Czy tyle wystarczy?

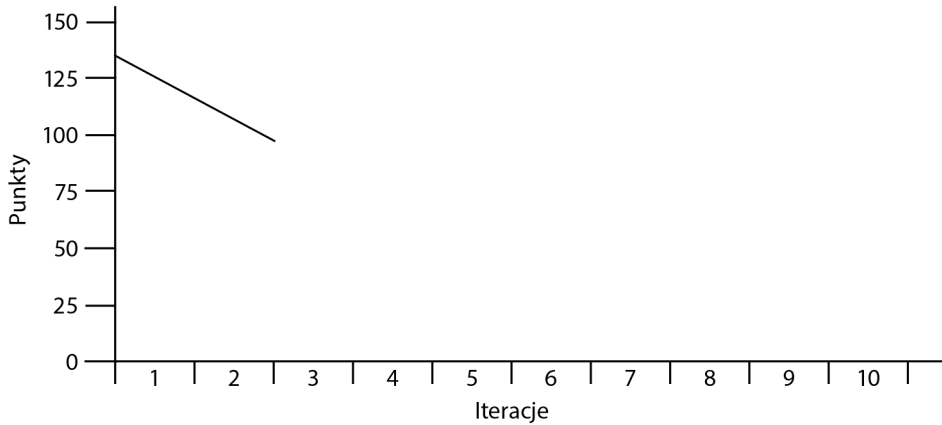
— Wystarczy z całą pewnością — odpowiedział Franek. Następnie poprosił Daniellę, aby wyjaśniła, jak to się stało, że jej badania doprowadziły do odkrycia nowych pomysłów na grę online i osobowości w grze.

— Filipie, czy pamiętasz, jak kilka tygodni temu opowiadałem ci o szacowaniu z wykorzystaniem punktów?

— Pewnie.

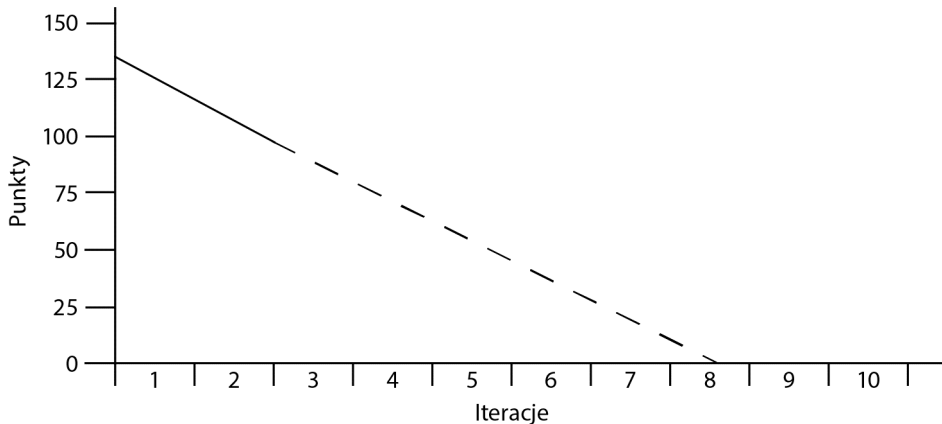
— Mówiłem wtedy, że te punkty przypisywane historyjkom są miarą wielkości. Zaczęliśmy od 132 punktów. W pierwszej iteracji zrealizowaliśmy szesnaście. W drugiej udało się zaimplementować osiemnaście. Alku, czy możesz to narysować? — zapytał Franek, wskazując na puste miejsce na tablicy Filipa. Alek podszedł do tablicy i narysował rysunek 23.5.

— To, co tutaj widzisz — kontynuował Franek — nazywamy wykresem wypalania wydania. Patrząc na ten wykres, można ocenić nasze postępy. Po dwóch iteracjach zrealizowaliśmy trzydzieści cztery punkty. Jeśli przedłużysz linię trendu na podstawie tych danych, to przekonasz się, że nasze tempo pozwala zakończyć projekt podczas ósmej iteracji. To w sumie szesnaście tygodni.



RYСУNEK 23.5. Wykres wypalania dla gry *Havannah* po dwóch pierwszych iteracjach

W czasie gdy Franek mówił, Alek narysował linie trendu pokazane na rysunku 23.6.



RYСУNEK 23.6. Linia trendu na podstawie wykresu wypalania dwóch pierwszych iteracji

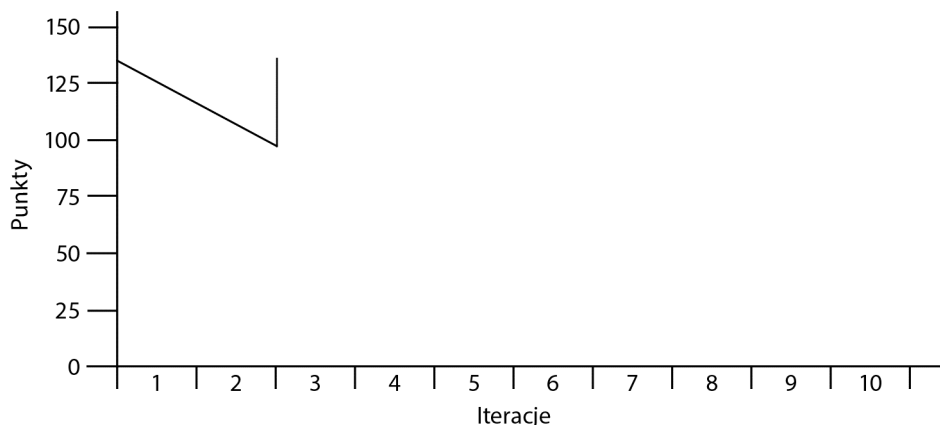
— Cały zespół *Havannah* uważa, że dodanie postaci zidentyfikowanych przez Danielę to dobry pomysł.

— Jak długo potrwa implementacja tych funkcjonalności? — zapytał Filip.

— Sądzimy, że potrzebujemy pięciu różnych postaci. To w sumie trzydzieści pięć punktów — powiedział Franek.

— Oznacza to, że nasz wykres wypalania w rzeczywistości wrócił do tego punktu — powiedział Alek. Narysował na tablicy pionową linię do punktu 133, tak jak pokazano na rysunku 23.7.

— Hmm. To nie wygląda zbyt dobrze. Po dwóch iteracjach sytuacja nie jest lepsza, niż była wtedy, gdy zaczynaliście.



RYСУNEK 23.7. Wykres wypalania gry *Havannah* po dołączeniu funkcjonalności osobowości

— Faktycznie jest znacznie lepiej. Przepracowaliśmy cztery tygodnie i osiągnęliśmy znaczny postęp — powiedział Alek, wskazując na postępy na wykresie. — Wyeliminowaliśmy niektóre najbardziej ryzykowne fragmenty projektu, ponieważ pokazałem, w jaki sposób będę dążył do zaprogramowania silnika ruchów. Dotąd zrealizowałem tylko słaby silnik, ale zaczynam mieć wizję, jak podejść do napisania silnego silnika. Poza tym dużo się nauczyliśmy. Dowiedzieliśmy się o nowej funkcjonalności, która — zgodnie z tym, co mówi Daniela — jest „ekscytująca” dla naszej docelowej grupy klientów. W innej sytuacji dowiedzielibyśmy się o tym znacznie później. Jeśli zastanowimy się nad tym, w jakim jesteśmy punkcie, to rzeczywiście teraz mamy tyle samo do zrobienia, ile według naszych ocen mieliśmy do zrobienia cztery tygodnie temu, ale teraz jestem bardziej pewny planu, niż byłem cztery tygodnie temu. I za każdym razem, gdy wykreślamy nowe punkty danych na tym wykresie wypalania, nabieram większej pewności pomimo tego, że nie wszystkie dane to dobre wieści. Jest coraz lepiej, jeśli mogę ufać postępom, które robimy. — Alek zatrzymał się. — Nasza sytuacja jest o wiele lepsza, niż była cztery tygodnie temu.

— Zatem jak długo potrwa projekt, jeśli dodamy postacie? — zapytał Filip. Franek był zaskoczony, ale mile, że pytanie było skierowane do Alka.

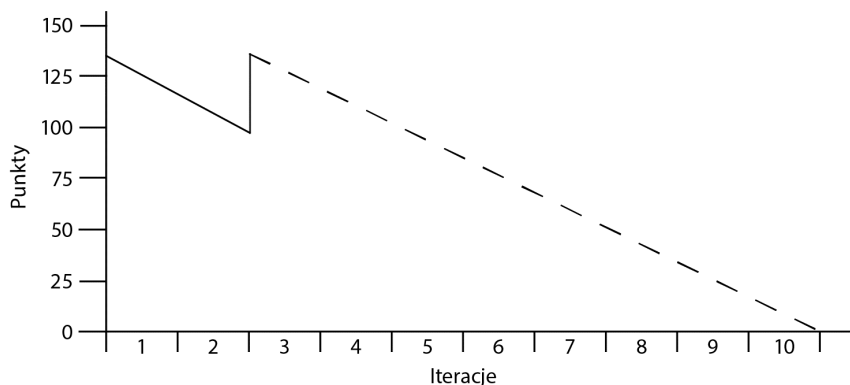
— Prawdopodobnie jeszcze osiem iteracji, czyli w sumie cały projekt potrwa dwadzieścia tygodni. W każdej iteracji implementujemy historyjki oszacowane łącznie na siedemnaście punktów. Tę wartość nazywamy tempem. Jeśli przeprowadzimy linię trendu, która pokazuje siedemnaście punktów postępu w każdej iteracji, to wychodzi, że projekt skończymy za osiem iteracji. — Alek narysował linię trendu pokazaną na rysunku 23.8. — To bardzo blisko górnego limitu terminu, dlatego wskazaliśmy kilka historyjek o niskim priorytecie, z których można zrezygnować, gdybyśmy zdecydowali się na wydanie po osiemnastu zamiast po dwudziestu tygodniach.

— I sprzedamy więcej egzemplarzy gier albo po wyższej cenie? — zapytał Filip.

— Tak, jestem tego pewien — powiedział Franek. — Chcę przeprowadzić jeszcze trochę badań i symulacji, aby ustalić, czy powinniśmy dążyć do sprzedaży większej liczby kopii za bieżącą cenę, czy też powinniśmy zwiększyć cenę o 5 dolarów.

— Ja czuję się przekonany. Dziękuję, że mi to pokazaliście — powiedział Filip.

Alek zaczął wycierać wykres, który narysował.



RYСУNEK 23.8. Zaktualizowany wykres wypalania pokazujący prognozowane zakończenie projektu *Havannah*

— Nie wycieraj. Zostaw ten rysunek — powiedział Filip. — Chciałbym go aktualizować co dwa tygodnie.

— Dziękuję za poświęcony czas, Filipie — powiedział Franek i razem z Danielą i Alkiem opuścili gabinet Filipa.

Osiemnaście tygodni później

— Świetna robota, zespole. Gra wygląda wspaniale — powiedział Franek na końcu ostatniego spotkania przeglądu iteracji. — Nie widzę, żeby czegoś brakowało. Silniki sprawują się bardzo dobrze. I Różo, postacie wyglądają doskonale. Sposób, w jaki każda osobowość jest przywiązana do silnika, sprawia wrażenie. Gracze zamiast poziomu trudności wybierają postać, przeciwko której grają. Przyznam, że początkowo byłem sceptyczny wobec animowanych postaci zamiast prawdziwych bohaterów, ale efekty są doskonałe. Podoba mi się, że pirat szydzi sobie z gracza oraz że z łatwym silnikiem ruchów jest powiązana przyjazna syrena. Rozmowa z nią z pewnością pomoże początkującym nauczyć się gry. Myślę, że jesteście gotowi do wydania.

— Przyznaję, że gra jest świetna, ale nadal nie mogę pogodzić się z uczuciem, że jesteśmy spóźnieni. Początkowo myśleliśmy, że skończymy projekt w nieco ponad czternaście tygodni. Ale powiedzieliśmy wszystkim, że będzie to trwało od dwunastu do dwudziestu tygodni. Teraz jeszcze przekroczyliśmy ten termin o dwa tygodnie — powiedział Przemek.

— Tak, i w końcu projekt trwał dwadzieścia dwa tygodnie — powiedział Staś. — Nigdy nie mówiliśmy, że projekt potrwa czternaście tygodni. Po pierwszym planowaniu iteracji powiedzieliśmy wszystkim, że projekt potrwa od dwunastu do dwudziestu tygodni, a bardziej dokładną ocenę podamy za kolejnych sześć tygodni.

— Przemku, pamiętaj, że gdybyśmy chcieli, to moglibyśmy zrealizować wydanie dwa tygodnie temu — powiedział Franek. — Na podstawie odpowiedzi uzyskanych od beta-testerów zdecydowaliśmy się poświęcić czas na wprowadzenie dodatkowych zmian w postaciach. Postacie bardzo im się spodobały, ale dali nam kilka świetnych propozycji, które postanowiliśmy uwzględnić. Na podstawie tych opinii szyderstwa pirata są wypowiedzane nieco częściej. Dodaliśmy też nieco więcej dialogów raperowi. To prawda, że początkowy harmonogram był błędny, ale to samo można powiedzieć o projekcie

produktu. Tamta gra nie zbliżyłaby nas nawet do zwrotu, jaki uzyskamy z tej, którą mamy teraz. Graliście w nową wersję. Wiecie, o ile jest teraz lepsza — zaledwie dwa tygodnie później w porównaniu ze wstępną oceną ostatecznego terminu. Gdybyśmy wydali wersję 1.0 bez postaci, to i tak zdecydowałbym się później szybko je wprowadzić. Ale nie możemy tak robić w naszej branży. Jestem bardzo zadowolony z produktu. Jest lepszy, niż się spodziewaliśmy, i zaledwie o dwa tygodnie spóźniony w porównaniu z oceną na początku projektu.

— I nie zapominajcie, że gra *Deep Black & White* była spóźniona o sześć miesięcy, a nie o dwa tygodnie.

— Franku, czy znów kupisz nam hamburgery, aby uczcić to wydanie?

— Nie, tym razem kupię wam cokolwiek zechcecie.

Bibliografia

- Abdel-Hamid Tarek, *Adapting, Correcting, and Perfecting Software Estimates: A Maintenance Metaphor*, „IEEE Computer” 1993, 26(3), s. 20 – 29.
- Ambler Scott, *Less Is More*, Software Development, listopad 2004.
- Anderson David, *Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results*, Prentice Hall, 2004.
- Andrea Jennitta, *An Agile Request For Proposal (RFP) Process*, Agile Development Conference 2003.
- Armour Phillip, *Ten Unmyths of Project Estimation*, „Communications of the ACM” 2002, 45, nr 11, s. 15 – 18.
- Baker Simon, e-mail do agileplanning@yahoogroups.com, 3 listopada 2004.
- Ballard Glenn i Howell Gregory A., *An Update on Last Planner*, materiały z 11. Dorocznej Konferencji Grupy International Group for Lean Construction, s. 1 – 13.
- Beck Kent, *Test Driven Development: By Example*, Addison-Wesley, 2002.
- Beck Kent i Fowler Martin, *Planning Extreme Programming*, Addison- Wesley, 2000.
- Beck Kent i in., *Manifesto for Agile Software Development*, www.agilemanifesto.org, 2001.
- Bentley Jon, *The Back of the Envelope*, „IEEE Software” 1999, 16(5), s. 121 – 125.
- Bills Mark, *ROI: Bad Practice, Poor Results*, „Cutter IT Journal” 2004, 17(8), s. 10 – 15.
- Bills Mark, *Useful Back-of-the-Envelope Calculations*, „Cutter IT E-Mail Advisor”, 27 października 2004.
- Boehm Barry, *Get Ready for Agile Methods, with Care*, „IEEE Computer” 2002, 35(1), s. 64 – 69.
- Boehm Barry, *Software Engineering Economics*, Prentice Hall, 1981.
- Boehm Barry i Fairley Richard, *Software Estimation Perspectives*, „IEEE Software” 2000, 17(6), s. 22 – 26.
- Bossi Piergiuliano, *Using Actual Time: Learning How to Estimate*, materiały z 4th International Conference In Extreme Programming and Agile Processes in Software Engineering, red. Michele Marchesi i Giancarlo Succi, 2003.
- Bossi Piergiuliano, Cirillo Francesco, *Repo Margining System: Applying XP in the Financial Industry*, materiały z 2nd International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP2001).

- Brenner Lyle, Koehler Derek i Tversky Amos, *On the Evaluation of One-sided Evidence*, „Journal of Behavioral Decision Making” 1996, 9, s. 59 – 70.
- Brooks Fred, *The Mythical Man Month: Essays on Software Engineering*, Addison-Wesley, 1975.
- Center for Quality of Management, Special Issue on Kano’s Methods for Understanding Customer-defined Quality, „Center for Quality of Management Journal” 2(4).
- Cirillo Francesco, *Tracking the Project with the Pomodoro*, nieopublikowany artykuł, 2005.
- Clark Kim i Wheelwright Steven, *Managing New Product and Process Development: Text and Cases*, The Free Press, 1993.
- Cockburn Alistair, *Agile Software Development Joins the „Would-Be” Crowd*, „Cutter IT Journal” 2002, 5(1).
- Cohn Mike, *User Stories Applied: For Agile Software Development*, Addison-Wesley, 2004.
- Constantine Larry i Lockwood Lucy, *Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design*, Addison-Wesley, 1999.
- DeGrace Peter i Stahl Leslie, *Wicked Problems, Righteous Solutions: A Catalogue of Modern Software Engineering Paradigms*, Prentice Hall, 1990.
- DeLuca Jeff, *FDD Implementations*, www.nebulon.com/articles/fdd/fddimplementations.html, 2002.
- DeMarco Tom i Lister Timothy, *Waltzing with Bears: Managing Risk on Software Projects*, Artech House, 2003.
- Duggan Jim, Byrne Jason i Lyons Gerald J., *A Task Allocation Optimizer for Software Construction*, „IEEE Software” 2004, 21(3), s. 76 – 82.
- Ganssle Jack, *The Middle Way. Embedded Systems Programming*, 14 października 2004.
- Genuchten Michiel van, *Why Is Software Late? An Empirical Study of Reasons for Delay in Software Development*, „IEEE Transactions on Software Engineering” 1991, 17(6), s. 582 – 590.
- Gilb Tom, *Principles of Software Engineering Management*, Addison-Wesley, 1988.
- Githens Greg, *Rolling Wave Project Planning*, materiały z 29th Annual Project Management Institute 1998 Seminars and Symposium.
- Goldratt Eliyahu, *Critical Chain*, North River Press, 1997.
- Goldratt Eliyahu, *Necessary But Not Sufficient*, North River Press, 2000.
- Goldratt Eliyahu, *The Goal: A Process of Ongoing Improvement*, North River Press, 1992.
- Goldratt Eliyahu, *What Is This Thing Called Theory of Constraints and How Should It Be Implemented?*, North River Press, 1990.
- Grenning James, *Planning Poker*, www.objectmentor.com/resources/articles/PlanningPoker.zip, 2002.
- Griffin Abbie i Hauser John, *The Voice of the Customer*, „Marketing Science” 1993, 12(1), s. 1 – 27.
- Hagafors Roger i Brehmer Berndt, *Does Having to Justify One’s Decisions Change the Nature of the Decision Process?*, „Organizational Behavior and Human Performance” 1983, 31, s. 223 – 232.
- Highsmith Jim, *Agile Project Management: Creating Innovative Products*, Addison-Wesley, 2004.
- Highsmith Jim, e-mail do agilemanagement@yahoogroups.com, 16 lutego 2004.
- Hobbs Charles, *Time Power*, Harper & Row Publishers, 1987.

- Hoest Martin i Wohlin Claes, *An Experimental Study of Individual Subjective Effort Estimations and Combinations of the Estimates*, materiały z 20th International Conference on Software Engineering, 1998, str. 332 – 339.
- Hunt Andrew i Thomas David, *The Pragmatic Programmer: From Journeyman to Master*, Addison-Wesley, 1999.
- Jeffries Ron, e-mail do extremeprogramming@yahoo.com, 1 lipca 2004.
- Jeffries Ron, Anderson Ann i Hendrickson Chet, *Extreme Programming Installed*, Addison-Wesley, 2001.
- Johnson Jim, referat otwierający na Third International Conference on Extreme Programming, 2002.
- Johnson Philip, Moore Carleton, Dane Joseph i Brewer Robert, *Empirically Guided Software Effort Estimation*, „IEEE Software” 2000,17(6), s. 51 – 56.
- Jørgensen Magne, *A Review of Studies on Expert Estimation of Software Development Effort*, „Journal of Systems and Software” 2004, 70(1 – 2), s. 37 – 60.
- Jørgensen Magne i Moløkken Kjetil, *Combination of Software Development Effort Prediction Intervals: Why, When and How?*, Fourteenth IEEE Conference on Software Engineering and Knowledge Engineering, 2002.
- Keen Jack i Digrius Bonnie, *Making Technology Investments Profitable: ROI Roadmap to Better Business Cases*, John Wiley & Sons, 2003.
- Kennedy Michael, *Product Development for the Lean Enterprise: Why Toyota's System Is Four Times More Productive and How You Can Implement It*, The Oaklea Press, 2003.
- Kernighan Brian i Plauger Phillip, *The Elements of Programming Style*, McGraw-Hill, 1974.
- Larson Carl i LaFasto Frank, *Teamwork: What Must Go Right/What Can Go Wrong*, SAGE Publications, 1989.
- Laufer Alexander, *Simultaneous Management: Managing Projects in a Dynamic Environment*, American Management Association, 1996.
- Leach Lawrence, *Critical Chain Project Management*, Artech House, 2000.
- Lederer Albert i Prasad Jayesh, *A Causal Model for Software Cost Estimating Error*, „IEEE Transactions on Software Engineering” 1998, 24(2), s. 137 – 148.
- Lederer Albert i Prasad Jayesh, *Nine Management Guidelines for Better Cost Estimating*, „Communications of the ACM” 1992, 35(2), s. 51 – 59.
- Macomber Hal, *Achieving Change in Construction Is a Matter of Mental Models*, „Reforming Project Management” 3(35), 29 sierpnia 2004.
- Malotaux Niels, *Evolutionary Project Management Methods*, www.malotaux.nl/nrm/pdf/MxEvo.pdf, 2004.
- Martin Robert, *PERT: Precursor to Agility*, Software Development, luty 2004.
- McClelland Bill, *Buffer Management*, The TOC Times, luty 2004.
- McConnell Steve, *Software Project Survival Guide*, Microsoft Press, 1998.
- Miranda Eduardo, *Improving Subjective Estimates Using Paired Comparisons*, „IEEE Software” 2001, 18(1), s. 87 – 91.
- Mugridge Rick i Cunningham Ward, *FIT for Developing Software: Framework for Integrated Tests*, Prentice Hall, 2005.

- Nejmeh Brian i Thomas Ian, *Business-Driven Product Planning Using Feature Vectors and Increments*, „IEEE Software” 2002, 34(6), s. 34 – 42.
- Newbold Robert, *Project Management in the Fast Lane: Applying the Theory of Constraints*, St. Lucie Press, 1998.
- Parkinson Cyril Northcote, *Parkinson’s Law: The Pursuit of Progress*, John Murray, 1958.
- Poppendieck Mary, *Lean Development and the Predictability Paradox*, Cutter Consortium, „Agile Software Development and Project Management Executive Report” 4(8), 2003.
- Poppendieck Mary i Tom, *Lean Software Development: An Agile Toolkit*, Addison-Wesley, 2003.
- Putnam Lawrence i Myers Ware, *How Solved Is the Cost Estimation Problem?*, „IEEE Software” 1997, 14(6), s. 105 – 107.
- Rakitin Steven, *Creating Accurate Estimates and Realistic Schedules*, „Software Quality Professional” 2002, 4(2), s. 30 – 36.
- Reinertsen Donald, *Managing the Design Factory: A Product Developer’s Toolkit*, Free Press, 1997.
- Rising Linda, *Agile Meetings*, „Software Testing and Quality Engineering” 2002, 4(3), s. 42 – 46.
- Saaty Thomas, *Multicriteria Decision Making: The Analytic Hierarchy Process*, RWS Publications, 1996.
- Sanders Glenn, *Self Presentation and Drive in Social Facilitation*, „Journal of Experimental Social Psychology” 1984, 20(4), s. 312 – 322.
- Sauerwein Elmar, Bailom Franz, Matzler Kurt i Hinterhuber Hans, *The Kano Model: How to Delight Your Customers*, materiały z konferencji International Working Seminar on Production Economics, 1996, s. 313 – 327.
- Schoonderwoert Nancy van, wywiad z Wardem Cunninghamem i Scottem Densmore’em, „Agile Times” 2004, 6, s. 61 – 69.
- Schwaber Ken i Beedle Mike, *Agile Software Development with Scrum*, Prentice Hall, 2002.
- Smith Preston i Reinertsen Donald, *Developing Products in Half the Time: New Rules, New Tools*, wydanie 2., Wiley, 1997.
- Standish Group International, Inc., *Extreme Chaos*, www.standishgroup.com/sample_research/PDFpages/extreme_chaos.pdf, 2001.
- Takeuchi Hirotaka i Nonaka Ikujiro, *The New Product Development Game*, „Harvard Business Review”, styczeń 1986.
- Tockey Steve, *Return on Software: Maximizing the Return on Your Software Investment*, Addison-Wesley, 2004.
- Vicinanza Steven, Mukhopadhyay Tridas i Prietula M. J., *Software Effort Estimation: An Exploratory Study of Expert Performance*, „Information Systems Research” 1991, 2(4), s. 243 – 262.
- Weinberg Gerald i Schulman Edward, *Goals and Performance in Computer Programming*, „Human Factors” 1974, 16(1), s. 70 – 77.
- Wiegiers Karl, *First Things First: Prioritizing Requirements*, Software Development, wrzesień 1999.

Skorowidz

A

aktualizacja planu wydania, 150
aktywności, 42, 43
analogia, 80
atrakcyjność, 127

B

błędy szacowania, 66
Bomb Shelter Studios, 253
bufory, 193
funkcjonalności, 194
harmonogramu, 195, 203
łączenie, 202
projektu, 199
ustalanie rozmiaru, 210
zasilające, 209

C

cel
iteracji, 161
planowania, 33
weryfikowania ocen, 90
czas idealny, 70

D

dane historyczne, 183
dezagregacja, 80
długość
iteracji, 148, 151, 175
całkowita wydania, 176
dni idealne, 71, 93–96

E

ekspert, 79
epiki, 78

F

funkcjonalności, 46, 245
ortogonalne, 138

H

harmonogram, 143
historyjki użytkownika, 63, 78
dzielenie, 135, 161
łączenie, 141
oszacowywanie, 147
podział według granic danych, 136
podział według granic
operacyjnych, 137
priorytety, 148, 152
punktacja, 63, 93, 171
wprowadzanie szczegółów, 206
wybór, 152, 161

I

informowanie o postępach, 37, 234
infrastruktura, 107
interfejs użytkownika, 108
iteracje, 52, 148, 151, 184, 246
koszty, 178
krótkie, 51
monitorowanie, 225
określanie celu, 161
planowanie, 155
podsumowanie, 235
rozwiniecie historyjek do zadań, 188
wybór długości, 175

K

kategorie odpowiedzi, 131
komunikacja, 213
koszt, 102
iteracji, 178
krótkie
iteracje, 51
sesje, 82

Ł

łączenie
buforów, 202
historyjek, 141
łączna ocena finansowa, 119

M

metody oceny, 75
miara rozmiaru, 71
miary finansowe, 120
model Kano, 127
ocena tematów, 129
monitorowanie planu
iteracji, 225
wydania, 215

N

niepewność, 35, 46, 176, 196, 247
nowa wiedza, 103

O

obliczanie
bufora, 200
nowych przychodów, 115
przychodów przyrostowych, 116
przychodów zatrzymanych, 116
sprawności operacyjnej, 117
ocena, 47, 72
czasu trwania, 61, 244
finansowa, 119
metody, 75
punktowa historyjek, 94, 95
rozmiarów, 61, 244
skala, 77
tematów, 129
w dniach idealnych, 96
weryfikacja, 85, 87
wyznaczanie, 79
zadań, 171
ogłaszanie planu, 232
okres zwrotu, 123
zdyskontowany, 124
określanie
priorytetów, 102
warunków satysfakcji, 150
opinia eksperta, 79
opóźnienia, 43

oszacowanie

- czasu iteracji, 187
- historyjek użytkownika, 147
- liczby godzin, 187
- szybkości, 148
- zadania, 165

P

plan

- ogłaszanie, 231
- poziomy, 55, 244
- z buforami, 193
- zmiana, 243

planowanie, 33, 38, 41, 247

- aktywności, 41
- funkcjonalności, 245
- iteracji, 155, 158
 - drugiej, 289
 - monitorowanie, 225
 - pierwszej, 273
 - sterowane tempem, 159
 - sterowane zobowiązaniami, 167

pod kątem wartości, 99

skuteczne, 241

wydania, 145, 158, 280

- aktualizacja, 150
- monitorowanie, 215

wypredzające, 207

z udziałem wielu zespołów, 205

zwinne podejście, 54

podejmowanie decyzji, 36, 179

podział

- historyjek, 135–140
- odpowiedzi na kategorie, 131

poker planistyczny, 81, 83

poziomy planowanie, 55

priorytety, 177

atrakcyjności, 127

biznesowe, 52

finansowe, 111

historyjek użytkownika, 148, 152

koszt, 102

nowa wiedza, 103

tematów, 101

wartość, 102

zagrożenia, 105

prognozowanie, 186

projekt

- Goodman, 181
- interfejsu użytkownika, 108
- Napa, 180

przegląd planu wydania, 291

przekazywanie informacji, 37

przychody

- nowe, 113
- przyrostowe, 113
- zatrzymane, 113

przydzielanie zadań, 157

punktacja historyjek, 63, 93, 171

R

redukowanie ryzyka, 35

rozmiar bufora

- projektu, 199
- zasilającego, 210

rozwiniecie historyjek do zadań, 188

ryzyko, 35

S

satisfakcja, 56, 127, 146, 150

sesje, 82

skala oceny, 77

skuteczność planowania, 241

sprawność operacyjna, 114

stopień niepewności, 176

stożek niepewności, 34

studium przypadku, 251, 253

sumowanie ocen, 168

szacowanie, 66, 69, 151, 247

kosztów, 118

rozmiaru, 63

szybkości, 151

tempa, 183

szybkość zespołu, 87

Ś

śledzenie, 213, 246

błędów, 227

poświęconego wysiłku, 228

wydania, 216

T

tablica zadań, 225

tematy, 78

tempo, 65, 66, 183, 217

indywidualne, 229

tworzenie harmonogramu, 143

U

- usuwanie problemów funkcjonalności przekrojowych, 138
- utrzymanie, 169

W

wagi względne, 132

wartość, 102

bieżąca netto, 121

pieniądza w czasie, 120

warunki satysfakcji, 146

WebPayroll, 115

weryfikacja ocen, 85–90

wewnętrzna stopa zwrotu, 122

wielozadaniowość, 44

witryna SwimStats, 85

wybór

długości iteracji, 151, 175

historyjek użytkowników, 152, 161

wydajność, 139

wykres

Gantt, 41

parkingowy, 223

wypalania iteracji, 228

wypalania wydania, 218, 220

wymagania wydajności, 139

wytwarzanie oprogramowania, 70

wyznaczanie

oceny, 79

zakresu, 188

wzbudzanie zaufania, 37

Z

zadania, 157, 245

zagrożenia, 105

zaufanie, 37

zdyskontowany okres zwrotu, 124

zespół Agile, 51–53

zmiany planów, 243

zmniejszanie niepewności, 35

zobowiązania, 167, 169

indywidualne, 169

zwinne podejście, 49, 54

Ż

źródła zwrotu, 112

PROGRAM PARTNERSKI

GRUPY WYDAWNICZEJ HELION

- 
1. ZAREJESTRUJ SIĘ
 2. PREZENTUJ KSIĄŻKI
 3. ZBIERAJ PROWIZJĘ

Zmień swoją stronę WWW
w działający bankomat!

Dowiedz się więcej i dołącz już dzisiaj!

<http://program-partnerski.helion.pl>

GRUPA WYDAWNICZA

 **Helion SA**